

Technical Bulletins

창원대학교 데이터베이스 연구실

■ 제 목 차 례 ■

▷ EXPORT	
EXPORT/IMPORT Q & A	1
▷ IMPORT	
IMPORT시 TABLE과 INDEX를 분리하여 저장하는 방법	4
▷ LOADER	
SAMPLE: SQL*LOADER SAMPLES PART I	5
SAMPLE: SQL*LOADER SAMPLE USING TRUNC FUNCTION	16
SAMPLE: SQL*LOADER SAMPLE PART II	17
▷ ODBC	
ODBC DRIVER를 이용하여 ORACLE DB에 접속하려는 경우	18
“ODBC DRIVER CALL FAILED”에러 메시지	20
VB, GLUE에서 STORED PROCEDURE 사용방법	21
ODBC & SQL*NET SUPPORT MATRIX	22
▷ ORACLE GRAPHICS	
UNIX 에서 사용되는 ORACLE 환경 변수	23
▷ RDBMS	
(V7.3) PARTITION VIEW 테스트 REPORT	31
(V7.3) PARTITION VIEWS IN RELEASE 7.3	45
(V7.3) STANDBY DATABASE IN ORACLE 7.3	47
ARCHIVE LOG MODE 운영방법	48
AUDITING 이란 무엇인가?	50
BITMAP INDEX 생성과 SPACE 비교	52
BITMAPPED INDEX	53
CONNET INTERNAL에 대해서 PASSWORD 설정하는 방법	58
CONTEXT SIZE & CURSORS(ORA-1040, 1050, 1051)	59
CONTROLFILE 재생성하는 방법	60
CORE DUMP 에 대한 설명	62
CORRUPTED BLOCK을 가진 OBJECT 찾는법	65
COST BASED OPTIMIZER 와 RULE BASED OPTIMIZER	66
DATA BLOCK CORRUPTION : ORA 1578 해결방법	69
DATABASE INTERNAL RECOVERY	72
DATABASE LINK 사용 방법	74
DATAFILE을 RENAME 하는 방법	75
DBA 정보와 CONVERT SCRIPT	77
DBMS PACKAGE : DBMS.ALERT을 이용하여 SESSION간 통신하는 방법	85
DBMS PACKAGE : DBMS.PIPE NOT DECLARED ERROR 발생시	90
DBMS PACKAGE : DBMS.PIPE사용(SESSION간의 MESSAGE SENDING)	91
DB_FILE_MULTIBLOCK_READ_COUNT와 퍼포먼스의 관계	94
DB_NAME과 SID 변경하는 방법	95
DB를 COLD BACKUP(FULL BACKUP) 받는 방법	97
DDB 기본기능 및 분산 트랜잭션 문제의 해결	98
DBMS PACKAGE : DEBUGGING PL/SQL PROGRAMS USING DBMS_OUTPUT	102
DISK STRIPING	108
DOWN TIME WITH OPS	109
DYNAMIC SQL(DBMS_SQL PACKAGE)	110
ENTERPRIZE BACKUP UTILITY	119
E7XP/IMP 를 이용하여 SCOTT의 데이터를 LARRY 옮기는 방법	124
EXPLAIN PLAN 사용법	125
EXPLAIN PLAN 을 자동으로 수행하는 방법	126
EXPORT 받은 화일의 CHARACTER SET 변경 프로그램	128
EXPORT받은 DUMP파일의 CHARACTER SET 확인 방법	130
EXPORT실행시 ORA-1403 조치방법	131
EXTENT된 영역의 FREESPACE	132
FOREIGN KEY CONSTRAINT 의 MASTER TABLE NAME의 확인	133
FREE LIST와 FREE SPACE관리	134

FULL TABLE SCAN을 보다 빠르게 하는 방법	136
HIERARCHICAL QUERIES	137
HP-UX: ASYNCH I/O USING FS_ASYNC KERNEL PARAMETER	138
INDEX 생성시 필요한 TEMP SEGMENT 크기 추정	139
INDEX_DESC HINT 사용법	140
INSTR 함수의 의미와 특정 CHARACTER를 지우는 방법	141
LAST ORA-1578 RECOVERY METHOD	142
LOCK 문제 확인과 해결 방법	144
LOCKING CONFLICTS를 DETECTING/RESOLVING	153
MESSAGE CONVERSION PROGRAM	164
METHOD 4를 이용한 SAM FILE생성 PRORAM	167
NETWARE LISTENER 활용(MAX USERS)	174
NETWARE SUPPORT MATRIX	175
NLS CHARACTERSET 변경방법 (DB REBUILD 없이)	176
OPEN VMS 시스템에서의 ORACLE AUTO STARTUP	178
OPS 에서 새로운 NODE 추가 방법	180
ORA-54 WHEN DROP A TABLE(SESSION KILL)	181
데이터베이스 STARTUP 시 ORA-312,313 에러	182
ORA-600[3339] DATA BLOCK CORRUPTION DETECTION	184
ORA-604 조치 방법	186
ORA-1118 조치 방법(MAXDATAFILES와 DB_FILES)	187
ORA-1547 조치 방법	190
ORA-1552 조치 방법	192
ORA-1555 조치 방법	193
ORA-1562 처리 방법	195
ORA-1628,1630,1631,1630 조치 방법	198
ORA-1652 조치 방법	198
ORA-4031 조치 방법	199
ORACLE DATABASE HOME DIRECTORY 변경	200
ORACLE DATABASE의 AUTOMATIC STARTUP/SHUTDOWN	202
Oracle Enterprise Manager Install Procedure	202
ORACLE MEDIA SERVER	208
ORACLE SERVER AND UNIX TRANSACTION PROCESSING(TP) MONITOR-1	211
ORACLE7 BACKUP 및 RECOVERY 방법	222
ORACLE7 INSTALL 후 데이터베이스 FILE의 구성	231
ORACLE7 에서 USER 등록	236
ORACLE의 MEMORY 사용량에 대한 테스트	237
OS 명령으로 DATA FILE 을 삭제한 경우 - ORA-1157, 1110	239
OUTER JOIN의 UIDE LINE	241
PARALLEL LOAD - SPACE USAGE	244
(V7.3) PARTITION VIEWS IN ORACLE7 RELEASE 7.3	245
PRIMARY KEY 재생성 방법	247
PRIMARY KEY를 REFERENCE 하는 FOREIGN KEY 찾는 SQL	248
PROC 의 실행화일 크기를 줄이는 방법	249
ORACLE SERVER AND UNIX TRANSACTION	250
RECOVERY 의 모든 것(DATABASE,TABLESPACE,DATAFILE)	261
REDO LOGFILE SIZE의 변경 또는 추가	264
ROLLBACK SEGMENT 관리	267
ROLLBACK SEGMENT 크기를 증가하여 재생성 하는 방법	271
ROLLBACK SEGMENT의 상태 해결	273
ROW CHAINING 과 ROW MIGRATION	275
ROWID에 대한 이해	276
IMPLEMENTING DATA REPLICATION	281
SHARED MEMORY 문제	287
SHARED POOL SIZE의 계산	291
SNAPSHOT 생성방법	293
SNAPSHOT 을 자동적으로 REFRESH 하는 방법	295
SNAPSHOT의 기본 개념과 예제	296
TABLE SIZE 와 INDEX SIZE 계산	309
TABLE 크기 추정 방법	310
TABLESPACE 의 내용을 다른 TABLESPACE로 옮기는 방법	314
TABLESPACE 크기를 확장하는 방법	316
TAPE 으로 ARCHIVE 를 직접 받는 방법	317

TKPROF 사용시 CORE DUMP 조치 사항	322
TPC Q&A	323
TRANSACTION ENTRY란 무엇인가?	325
TRANSACTION을 ROLLBACK SEGMENT에 할당하는 방법	326
TRIGGER 에서 ERROR 확인(SET SERVER OUTPUT ON 사용)	327
TWO-PHASE COMMIT	328
Using Oracle Database Manager 7.2 & 7.3	9
WINDOWS NT와 ORACLE7 TUNING GUIDE	333
새로운 DATABASE 생성	339
인포믹스 STATIC PARTITIONING KNOCKOFFS	341

▷ SQL FORMS

(V3.0) BASE TABLES FOR MENU 5.0 BASE TABLES	342
(V3.0) FIELD를 이동할 때 자동으로 한글/영문 MODE를 변경하려면?	344
(V3.0) FORMS3.0에서 USER EXIT이란?	345
(V3.0) FORMS3.0에서의 LOCKING	347
(V4.5.3.0) FRM-40735 ERROR에 관하여	349
(V4.5.5.X) SUPPORT FOR VBX AND OCX CONTROLS	356
(V4.5.7 LATER) SQL*NET V2 SUPPORT	358
(V4.5) CDE2 AND RDBMS	359
(V4.5) FORMS4.5 NEW FEATURE	361
(V4.5) CDE2 의 ON-LOCK 기능	366
(V4.5) DEVELOPER/2000 REGENERATE FORM	367
(V4.5) FORMS4.5간의 PARAMETER PASSING	368
(V4.5) FORMS4.5에서 REPORT2.5로의 DATA PARAMETER PASSING	371
(V4.5) DDE(DYNAMIC DATA EXCHANGE)	374
(V4.5) FORMS4.5와 TEXT FILE간의 INTERFACE	383
(V4.5) DEVELOPER2000으로 작성한 MODULE을 DATABASE에 저장하려면?	385
(V4.5) FORMS4.5 RUNTIME INSTALL 방법	386
(V4.5) FORMS4.5에서 FILE DIALOG 처리 방법	388
(V4.5) HOW TO IMPLEMENT MENU SECURITY IN FORMS4.5?	389
(V4.5) DEV/2000에서 DOCUMENTATION 생성방법	391
(V4.5) EXITING MULTIPLE FORM APPLICATIONS	393
(V3.0,4.5) PRE-QUERY TRIGGER 사용방법	394
(V4.5) ALERT을 이용하여 MESSAGE를 처리하는 방법 (ADVANCED)	396
(V4.5) ALERT을 이용하여 MESSAGE를 처리하는 방법 (BASIC)	401
(V3.0,4.5) USER가 아무 ACTION이 없이 일정 시간이 지나면 자동으로 LOGOUT시키는 방법	403
(V4.5) CUSTOMIZING LOGON BEHAVIOR	404
(V4.5) BASE TABLES FOR DEVELOPER 2000 TOOLS	411
(V4.5) APPLICATION을 DB에 저장하는 방법	420
(V4.5) F45에서 특정 FIELD에서 한/영키 자동 변환방법	421
(V4.5) FORMS4.5에서 GRAPHIC2.5로의 DATA PARAMETER PASSING	423
(V4.5) FORMS4.5에서 SERVER쪽 APPLICATION을 실행하는 방법	426
(V3.0, MENU V5.0) FORM과 MENU의 연결	429
(V3.0) BUFFER AND ARRAY SIZE WITHIN SQL*FORMS 3.0	430
(V3.0) FORMS사용시 ORA-942 ERROR 처리	431
FORM에서의 2차원 TABLE집계 처리	432
(V4.5) HOW TO IMPLEMENT MENU SECURITY IN FORMS4.5?	434
(V3.0) FORMS3.0 에서 LEFT,RIGHT KEY 를 사용하여 MENU DESIGN	436
(V4.5) MENU UPGRADE	438
(V4.5) MULTI FORM GENERATION (UPDATE)	443
(V4.5) NEW FEATURES IN 4.5.7 (DEV/2000 R1.3)	449
(V3.0) ORATERM 을 이용한 FORMS3.0 FUNCTION KEY 수정	454
(V3.0) OS의 환경변수를 받아들이는 방법	457
(V3.0) SQL*FORMS V3에서 SQL*REPORTWRITER로 PARAMETER전송하는 방법	458
(V3.0) FORMS3.0 및 STORED PROCEDURE 에서 ROLLBACK SEGMENT 사용	459
RUNNING SQL TRACE FROM ORACLE TOOLS	460
STORED PROCEDURE & DATABASE TRIGGER	461
(V4.5) USING DATABASE ROLES WITH ORACLE FORMS 4.5	464
(V3.0) VT100에서 KEY-F0 ~ KEY-F9사용법	467
(V4.5) WINDOWS PERFORMANCE TUNING FOR FORMS4	468
(V4.5) ORACLE FORMS 4.5 에서 WINDOWS APPLICATION을 호출하는 방법	470

▷ SQL NET

PRO*C 2.0의 새로운 특징	472
AUTOMATIC STARTUP AND SHUTDOWN OF SQL*NET LISTENERS	474
CLINET/SERVER 환경에서의 PERFORMANCE TUNING	482
SAMPLE: DB에 IMAGE FILE을 저장	489
DIGITAL UNIX 4.0/4.0A 사용시 PRO*C PRECOMPILE 주의 사항	491
DOS, WINDOWS에서의 PRO*C와 C COMPILER와의 관계	492
DYNAMIC SQL METHOD1(CREATE TABLE)	493
SAMPLE: DYNAMIC SQL METHOD2 SAMPLE(INSERT)	495
SAMPLE: DYNAMIC SQL METHOD2 SAMPLE(UPDATE)	497
SAMPLE: DYNAMIC SQL METHOD3 SAMPLE(SELECT)	499
FUTURE TCP 와 SQL*NET V2 사용시 PERFORMANCE	501
PRO*C 에서 HOLD_CURSOR, RELEASE_CURSOR 의 의미	502
SQL-2101(ORA-2101): INCONSISTENT CURSOR CACHE (UCE/CUC MISMATCH)	506
PRO*C ON MS-DOS AND MS-WINDOWS PLATFORMS	508
ORA-1002 FETCH OUT OF SEQUENCE IN PRO*C	510
ORA-1405 처리 방법	512
ORA-3113, 3114 의 발생원인 및 진단방법	513
ORA-6401 ERROR 발생시 처리 방법	514
ORACLE의 TWO-TASK DRIVERS	515
PC CLIENT TOOL 과 ORACLE DB 와의 INTERFACE 방법	517
PRECOMPILE에서 MODE OPTION으로 저장하는 방법	518
PROC 2.X 에서 INCLUDE FILE 사용방법	520
SAMPLE: PROC 에서 SESSION 을 KILL 하는 방법	521
PROC 에서 외부 환경변수를 받아 들이는 방법	522
PROC의 MAKE FILE을 이용하여 SHELL프로그램을 만드는 방법	524
SQL*NET SESSION의 DEAD CONNECTION 처리 방법	526
SQL*NET과 관련된 기본적인 점검사항	527
SQL*NET V1 FOR APPC/LU6.2 TEST	530
SQL*NET V1, V2의 STATUS를 확인하는 방법	532
SQL*NET V1, V2의 차이점	535
SQL*NET V1사용시 ORA-6107 조치 방법	537
SQL*NET V2 SETUP 방법	539
SQL*NET V2 사용시 ORA-12154 조치 방법	542
SQL*NET V2 에서 BUFFER SIZE 지정 방법	546
SQL*NET V2에서 DEDICATED로 접속하는 방법	547
WINDOWS PERFORMANCE TUNING	548
WINDOWS용 SQL*NET V1 사용할 때 ORA-6136 조치방법	550
WINDOWS용 SQL*NET V1, V2 사용시 ORA-9241, ORA-9301 조치방법	551
WINDOWS용 SQL*NET 사용시 ORA-6106	552
WINDOWS용 SQL*NET 사용시 ORA-6108	556
WINDOWS용 SQL*NET 사용시 ORA-6110	560
WINDOWS용 SQL*NET 사용시 ORA-6120	562
WINDOWS용 SQL*NET 사용시 ORA-6122	566
WINDOWS용 SQL*NET 사용시 TNS-12203	567
WINDOWS용 SQL*NET 사용시 TNS-12533	569
WINDOWS용 SQL*NET 사용시 TNS-12545	570
WINDOW용 SQL*NET 를 사용시 ORA-3121	571
SAMPLE: XA PROGRAM	574

▷ SQL PLUS

NUMBER DATA TYPE	582
DATE TYPE 데이터를 효과적으로 조회하는 방법	586
ORACLE7 OPTIMIZER 에 대하여	587
한글이 ??? 로 DISPLAY 되는 경우	589
SQLPLUS 기동시 WARNING MESSAGE 처리	591
SQL, SQL*PLUS 를 사용하여 SQL SCRIPT를 동적으로 생성, 수행	592
SQL TIPS(1): SORT	594
SQL TIPS(2): 특정조건의 순서대로 번호를 부여하는 방법	595
SQL TRACE FACILITY (TKPROF UTILITY 사용)	597
LONG COLUMN을 가진 TABLE의 COPY	600
SQL TIPS(3): DATE의 기간을 계산하는 방법	601
SQL TIPS(4): 중복된 RECORD 삭제 방법	602

외부에서 값을 받아서 SQLPLUS의 SCRIRT 실행하는 방법	603
SQL TIPS(5): DECODE이용사례 (1) : 월별로 집계하는 방법	604
SQL TIPS(6): 해당 COLUMN에 걸려 있는 CONSTRAINT확인	605
SQL TIPS(7): 한 CONSTRAINT에서 REFERENCE하는 COLUMN확인	606
SQL TIPS(8): PARENT-CHILD관계를 갖는 TABLE의 PRIMARY,FOREIGN KEY의 COLUMN명과 POSITION 확인	607
SQL TIPS(9): 해당 TABLE에 걸려있는 CONSTRAINT의 확인	609
SQL TIPS(10): CHECK CONSTRAINT의 조건확인	611
SQL TIPS(11): TABLE에 대하여 EXTENT 발생량 산정방법	612
SQL TIPS(12): PRINTING A HIERARCHICAL LIST OF REFERRING AND REFERENCED	613
SQL TIPS(13): DROPPING DATABASE OBJECTS FOR A USER	622
SQL TIPS(14): PROVIDING USERS WITH SESSION INFORMATION	624
SQL TIPS(15): FORECASTING ON NEXT EXTENTS	627
SQL TIPS(16): CREATING SCRIPTS TO RECREATE A TABLE STRUCTURE	629
SQL TIPS(17): AVAILABLE SPACE, FRAGMENTATION, AND EXTENTS	640
SQL TIPS(18): DBMS_OUTPUT을 이용하여 SQL*PLUS에서 REPORT 작성하기	643
SQL TIPS(19): MAKE A MAP OF YOUR ORACLE FILES	645
SQL TIPS(20): HIERARCHICAL DATA의 SORT와 ORDERING	648
SQL TIPS(21): DBA가 가지고 있어야 할 10개의 SQL	651
SQL TIPS(22): DETAILED LOOK AT DATABASE INDEX	656
SQL TIPS(23): EXPORTING TABLES BASED ON TABLESPACE NAME	660
SQL TIPS(24): CALCULATING HIT RATIO	662
SQL TIPS(25): ANALYZING TABLESPACE USAGE WITH AN OS-INDEPENDENT SQL SCRIPT	663
SQL TIPS(26): CREATING A DATABASE LAYOUT	666
SQL TIPS(27): DECODE 이용사례 (2)	668
SQL TIPS(28): DECODE 이용사례 (3)	669

▷ SQL REPORT

(V1.1) REPORTWRITER 에서 LOCAL PRINTER로 출력하는 방법	670
(V1.1) 출력물을 현 단말기에 연결된 PRINTER로 출력하는 방법	671
(V1.1) PAGE HEIGHT가 66보다 작은 SPECIAL FORM지에 출력하는 방법	672
(V2.5) ASCII DRIVER로 WIDE FORMAT 출력	673
(V1.1) SQL*REPORTWRITER 1.1에서 프로그램의 BACK-UP 및 RESTORE 방법	674
(V1.1) SQL*REPORTWRITER 1.1에서의 TERMINAL FUNCTION KEY 수정방법	675
(V1.1) SQL*REPORTWRITER 1.1에서 PRINTER DRIVER DEFINITION 수정방법	677
(V1.1) 고정된 위치에 합계를 PRINT 하는 방법	680
(V1.1) PRINTER로 찍을때 확대축소하려면?	681
(V2.5) ORACLE REPORT SERVER에 관하여	682
(V2.5) ORACLE REPORTS PERFORMANCE TIPS	684
(V1.1) REPORT SOURCE LISTING GENERATION	688
(V1.1) SQL*REPORTWRITER에서 SOURCE를 자동으로 BACKUP받는 방법	689
(V2.5) REPORT에서 자주 사용되는 SRW PACKAGED PROCEDURE	691
(V2.5X) 조건에 따라 HIGHLIGHTING, COLORING 처리하는 방법	695
(V2.5) ORACLE REPORTS에서의 FORMAT TRIGGER사용에 관하여	698
(V2.5) TABLE에 IMAGE FILE에 대한 PATH만 가지고 있는 경우의 처리	701
(V2.5) REPORT2.5에서 ORACLE GRAPHICS를 띄우려면?	702

▷ UNIX

SOLARIS 2.X SETUP PROBLEM	703
UNIX 에서 RAW DEVICE 사용하는 방법	705
ORACLE 과 UNIX SEMAPHORE 와의 관계	707
SUN SOLARIS C COMPILE 시 고려 사항	710
ORACLE DATABASE 의 AUTOMATIC STARTUP/SHUTDOWN	711
ORACLE 에서 CRON 사용	719
UNIX 에서 ORACLE PERMISSION 문제 - ORA-1034, ORA-7320,7429	723
UNIX 에서 SHARED MEMORY REQUIREMENT	726
UNIX 에서 SHARED MEMORY/SEMAPHORE 의 이해	730
RAID 와 ORACLE의 관계 소개	738
UNIX 에서 사용되는 ORACLE 환경 변수	740
CORE DUMP TRACING 방법	749
UNIX SVR4 에서 O/S의 REBUILD	751
SYSTEM RESOURCE FOR TUXEDO	754
SCRIPT FOR HOT BACKUP	757

EXPORT/IMPORT Q & A

Bulletin no : 10288

[질문1] RDBMS와 Export, Import의 연관 관계는 (catexp.sql 이란) ?

Export, Import이 이미 생성된 오브젝트의 정보를 데이터 디렉터리에서 읽어올 하는데 이러한 오브젝트의 정보가 데이터 디렉터리내의 여러 데이터에 나누어져 있다. 필요한 데이터 디렉터리 정보를 편리하게 이용하기 위하여 여러 가지의 블록 catexp.sql에 Script되어 있다. 이 스크립트 파일은 \$ORACLE_HOME/admin에 있으며 Install이 수행되도록 되어 있다.

[질문2] Export시 오브젝트의 백업 순서는 있는가 ?

Export하는 오브젝트의 순서는 존재 며 이는 Oracle의 Version 4.0에 의한 새로운 오브젝트가 만들어지거나 하면 Export되는 오브젝트의 순서는 전할 수 있다. 오브젝트의 Export순서는 다음과 같다.

- | | |
|--|--|
| 1. Tablespace | 14. Backupset logs |
| 2. Profiles | 15. Job Queue |
| 3. Users | 16. Backup Groups |
| 4. Roles | 17. Cluster Definitions |
| 5. System Privilege Grants | 18. Tables(Includes grants, column grants, comments, indexes, constraints, auditing) |
| 6. Role Grants | 19. Referential Integrity |
| 7. Default Roles | 20. PARTITION actions |
| 8. Tablespace Quotas | 21. Synonyms |
| 9. Resource Costs | 22. Views |
| 10. Rollback Segments | 23. Stored Procedures |
| 11. Database Links | 24. Triggers |
| 12. Sequences(Includes Grants) | 25. Default and System Auditing |
| 13. Backupsets (Includes grants, auditing) | |

[질문3] Export BUFFER와 RECORDLENGTH는 무엇인가?

-BUFFER

Export이 오브젝트 내에 있는 여러 개의 Row가 한꺼번에 Fetch된다. 디스크에서 Fetch된 정보는 파일에 Write하기 전에 메모리를 거치게 되며, 이때 할당되는 메모리의 양이 Buffer 파라미터의 값이다.

-RECORDLENGTH

메모리에 있는 Export할 자료를 파일에 Write하기 위해 한꺼번에 출력되는 양을 결정하는 파라미터이다.

(주의) 위의 BUFFER와 RECORDLENGTH는 O/S의 Block Size의 배수가 되도록하는 것이 효율적이다.

[질문4] 다량의 Row를 Export, Import시 어느 정도의 Row가 처리되었는지 알 수 있는가?

알 수 있다. U 7.1까지는 다량의 Row를 Export, Import시 처리된 정도를 알 수가 없어 현재 작업 중인지 시스템이 Hang인지 파악되지 않았으나 U 7.2부터는 `FEEDBACK`이라는 옵션을 이용하여 체크가 가능하다.

[질문5] Export시 한번에 몇 개의 Row가 Fetch되는가?

한번에 Fetch되는 Row의 수는 `Buffer Size`와 연관 관계가 있다. 하나의 Row가 Export시 차지하는 양은 각 Column Size의 합 $\times 4 \times$ (Column의 수)로 구할 수 있다. 한편 Fetch되는 Row의 수는 `Buffer Size / 한 Row의 Export시 Size`이다. 이를 이용하면 Export된 Output File의 Size는 대략 한 Row의 Export시 Size $\times$ Row 수 이다.

[질문6] Export, Import의 호환성은 어떻게 되는가?

Export, Import의 호환성은 Oracle의 버전과 직접적인 연관관계를 갖고 있다. 호환성은 4가지로 나누어 설명할 수 있으며 이를 아래의 가정을 이용해 설명하겠다. 가정 A라는 기계에 Oracle U 7.0, B라는 기계에 Oracle U 7.1이 설치되어 문명 중이라 가정하자. Oracle U7.0을 K라 하고 Oracle U7.1을 Y라고 하자.

- **Back Compatibility** : K의 exp를 이용해 K DB를 export하여 K의 imp를 이용해 K DB에 import하는 것을 말한다. 이는 당연히 지원한다.
- **Upward Compatibility** : K의 exp를 이용해 K DB를 export하여 Y DB에 Y의 imp를 이용해 import하는 것을 말한다. 이도 Oracle에서는 지원한다.
- **Downward Compatibility** : Y exp를 이용해 Y DB를 export하여 K DB에 K의 imp로 import하는 것을 말한다. 이는 지원될 수도 안될 수도 있다.
- **Cross Compatibility** : K exp를 이용해 Y DB를 export(BQLEdit 이용)하여 K 또는 Y DB에 import(imp는 적절한 것을 활용)하는 것을 말한다. 이는 지원이 안될 수도 있다.

[질문7] 어떤 경우에 Downward Compatibility가 실패하는가?

U7.2에 `back cluster expressions`라는 옵션이 있는데, 이를 이용해서 클러스터를 생성하여 사용 후 export한 것을 U7.0 또는 U7.1로 downward create cluster문에 옵션이 맞지 않아 실패하게 된다.

[질문8] EXP-37 에러(export views not compatible with database version)발생의 원인은 무엇인가 ?

이 에러는 Cross Compatibility에서 발생하는 문제로 이는 Export가 이용하는 `Ums/Catexp.sql`에 의해 생성된)가 Oracle Version과 일치하지 않아 발생한 문제로 이를 해결하기 위해 Exp에서 이용 가능한 Ums를 설치한다.

[질문9] Full Export는 DBA 권한을 갖고 있는 유저만 할 수 있는가 ?

Version 8 에서는 DBA권한을 갖고 있는 유저만 Full Export를 할 수 있으며, U7에서는 DBA가 아니더라도 `EXP_FULL_DATABASE` Role이 Grant되면 Full Export가 가능하다.

[질문10] 테이블 Import 시에 디폴트 테이블 스페이스가 아닌 곳으로 들어가는 경우는 왜 발생하는가?

예를 들어서 scott 용제의 디폴트 테이블 스페이스가 users 인데 임포트를 해보면 toolx 테이블 스페이스에 테이블이 만들어졌다고 하자. 그 이유는 다음과 같다. 즉, 임포트 하는 테이블이 원래 toolx 테이블 스페이스에 있었고 scott가 현재 toolx 테이블스페이스에 대한 Quota 를 가지고 있거나 아니면 Unlimited Tablespace 권한(Resource Role에 토)을 부여 받았기 때문이다.

Import시에 테이블을 디폴트 테이블 스페이스에 만들려면 디폴트 테이블스페이스 외의 테이블 스페이스에 대한 모든 Quota 를 0 로 만들고 Unlimited Tablespace 권한을 Revoke 시킨 다음에 임포트를 수행해야 한다. 그리고, 디폴트 테이블 스페이스에 대한 Quota만 Unlimited 로 한다.

예를 들면 다음과 같다.

```
$ sqlplus system/manager
```

```
BQL> alter user scott
      quota 0 on system
      quota 0 on toolx
      .....
      quota 0 on data
      quota unlimited on users;
```

```
BQL>revoke unlimited tablespace from scott;
```

이렇게 한 다음 Import 을 수행하면 된다. 물론 용제를 만들 때 quota를 주지 않은 테이블 스페이스는 상관 없으며 Unlimited Tablespace 권한(또는 Resource Role) 을 주지 않았다면 Revoke 명령도 사용할 필요가 없다.

[질문11] Import 시에 Core Dump/Segmentation Fault 가 발생하는 경우

오라클에는 Character Set이 있다. 국내에서는 UTF8B0II 또는 E018E05B0I를 주로 사용하는데 Export 할은 곳과 Import 하는 곳의 Character Set이 다르면 Import시에 Core Dump 가 발생하거나 원 인 불명의 에러가 발생하면서 임포트가 중단되는 경우가 발생한다.

이 경우에는 Export 할은 dump file 을 convert 프로그램들 이용하여 Import하는 곳의 Character Set 으로 변환 시킨 다음 임포트를 하는 방법이 있고, 아니면 어느 한 쪽 DB 의 Character Set 자체를 바꿔서 동일하게 맞추는 다음 Export/Import 하는 방법이 있다. 이중에서 Convert 프로그램들 이용하는 방법이 간단한데 이 프로그램은 Unix상에서 cc로 컴파일 하여서 사용하면 된다. 프로그램 소스는 선리만에서 go oracle 하여 정보 보 상고에서 구할 수 있다.

IMPORT시 TABLE과 INDEX를

분리하여 저장하는 방법

Bulletin no : 10338

Export/Import를 이용하여 Data를 옮기거나 Backup을 걸러낼 때 Table과 Index를 각각 다른 Tablespace에 분리하여 저장하기도 한다. 이러한 작업은 Oracle의 속도를 향상시키는 데 많은 도움을 준다.

Import할 때 Table과 Index를 생성시켰던 Script를 가지고 있 면 편리하게 이 작업을 할 수 있다. 먼저 Import할 때 Index에 option을 사용하여 Index를 제외한 나머지 Object를 Import하고, Index는 Script에서 Tablespace를 지정한 후 SQL File을 실행시켜서 생성하면 된다. 그러나 Index Script를 가지고 있지 않을 때는 한 가 복잡해진다.

그러한 경우에 Import Option중에서 IndexFile Option을 사용하여 Index script를 만들어 낼 수가 있다. 이 방법은 IndexFile만 만들기 때문에 속도가 빠르고, Index 뿐만이 아니라 Table Script도 만들어 지기 때문에 결과적으로 Table Script도 활용할 수 있다. 다음과 같이 만들 수 있다. 다음 작업은 Scott User의 모든 Data를 Test User로 옮기기 위해서 Export/Import를 이용하려고 한다. Import시에 Table과 Index는 Tablespace를 다르게하여 저장한다.

1. 다음과 같이 Scott User를 Export 한다.

```
$ exp scott/tiger file=scott.dmp
```

2. Export받은 File에서 Index를 제외한 나머지만 Import한다.

```
$ imp system/manager file=scott.dmp fromuser=scott touser=test indexes=consisty
```

위와 같이 실행하면 Test User에는 Index를 제외한 나머지 Object만 Import된다.

3. 그리고 다음과 같이 IndexFile Option을 이용하여 Index Script를 만든다.

```
$ imp system/manager file=scott.dmp indexfile=index.sql
```

위와 같이 명령을 실행하면 실제로 Data는 Import되지 않고, index.sql이라는 File만 만들어진다. 그리고 파일을 vi 등으로 열어서 확인해 보면 create table 문장과 create index문장이 있고 Create Table 문장은 REM으로 막혀 있어서 결과적으로 Create Index문만 실행할 수 있도록 되어 있다.

이 파일에서 Create Index문의 Tablespace만 바꾸어서 다음과 같이 SQLPlus에서 실행시킨다.

```
$ sqlplus test/test
```

```
SQL> @index
```

이렇게 하여 Table과 Index를 다른 Tablespace로 분리하여 저장할 수 있다.

이 IndexFile option은 Index만이 아니라 Table 생성문장이 필요할 때도 유용하게 사용하여 활용할 수 있다.

SAMPLE: SQL*LOADER SAMPLES PART I

Bulletin no : 101B4

1. SQL*LOADER SAMPLES

SQL*Loader는 외부 파일의 데이터를 DATABASE 데이터베이스의 테이블에 넣기 위한 용집리티입니다. SQL*Loader를 사용하려면 외부 데이터 파일과 컨트롤파일의 필요합니다. 컨트롤파일이라고 하는 것은 로드하는 데이터의 정보를 저장한 파일입니다. 간단한 샘플 컨트롤파일을 설명하겠습니다.

<code>load data</code>	제어파일의 선두에는 반드시 이 말이 필요합니다.
<code>infile sample.dat</code>	외부파일을 지정합니다.
<code>replace</code>	테이블에 데이터 넣는 방법 지정
<code>into table TABLE_NAME</code>	데이터를 로드하는 테이블을 지정
<code>field terminated by ','</code>	데이터 필드의 종결문자 지정
<code>(a integer external, b char)</code>	데이터의 열, 외부데이터 파일의 데이터 형을 지정

참고로 `replace` 외에 다음의 옵션이 가능합니다.

<code>replace</code>	데이터의 기존 행을 모두 삭제(delete)하고 민서프
<code>append</code>	새로운 행을 기존의 데이터에 추가
<code>insert</code>	미어 있는 테이블에 넣을 때
<code>truncate</code>	데이터의 기존 데이터를 모두 truncate 하고 민서프

SQL*Loader를 실행하면 아래의 파일이 작성됩니다.

- * 로드작업중 용각에 관한 통계 등을 포함한 로그파일(확장자는 log)
 - * 데이터 에러 때문에 로드가 만된 레코드를 저장한 파일(확장자는 bad)
 - * 사용자의 선택기준에 적합하지 않은 레코드를 저장한 파일(discard 파일)
- 이것은 discardfile 옵션으로 별도로 지정해야 생성됩니다.

실행방법은 다음과 같습니다.

```
$sqlload scott/tiger control=sample.ctl data=sample.dat
```

1.1 임의의 열에 전파열은 고정 문자열(값)을 입력한 경우

```
**** 테이블 구조 ****
create table emp_test
(a number,
b number,
c number,
d varchar(10))
```

```
**** 컨트롤 파일 ****
load data
infile emp.dat
replace
into table emp_test
fields terminated by ','
(a integer external, b integer external,
c CONSTANT '100', d char)
```

```

-- 데이터 로드 --
1,2,BATH
2,4,BATH2

```

```

-- 결과 --
SQL>select * from com_tact;

```

A	B	C
1	2	100 BATH
2	4	100 BATH2

[주의] 이 예에서 C열은 데이터 하입에 넣어서는 안됩니다. CONSTANT는 그것으로 완성된 열 지점의 하나가 됩니다. Integer external 데이터형은 수치데이터를 문자형식(ASCII CODE)로 나타낸 것입니다.

1.2 로드한 데이터 날짜를 데이터로 로드하고 싶은 경우

```

-- 데이터 구조 --
create table mydatewk
(a number,
 b date,
 c varchar(10))

-- 데이터 로드 --
load data
infile mydate2.dat
replace
into table mydatewk
fields terminated by ','
(a integer external,
 b mydate, c char(10))

-- 데이터 로드 --
111,BTBMING
222,BTBMING2

```

```

-- 결과 --
SQL>select * from mydatewk;

```

A	B	C
111	13-MAY-84	BTBMING
222	13-MAY-84	BTBMING2

[주의] 이 예에서는 B열은 데이터 하입에 넣어서 안됩니다. BYTE는 그것으로 완성된 열 지점의 하나가 됩니다. 새로운 시스템 날짜매김은 전편서널 패스에서는 실행시에 삽입된 각각의 레코드 매일마다 또, 다이렉트 패스의 경우는 로드된 각각의 레코드의 블록마다 사용됩니다.

1.3 임의의 수치열에 연속번호를 붙이고 싶은 경우

```

-- 데이터 구조 --
create table seqtk
(a varchar(10),
 b number, c varchar(10))

```


**** 콘트롤 화일 ****

```
load data
infile rec.dat
replace
into table recth
fields terminated by ','
(a char,
 b numeric,
 c char)
```

**** 외부 데이터화일 ****

```
A,a
B,b
C,c

**** 실행결과 ****
SQL>select * from recth;

A          B C
-----
A          1 a
B          2 b
C          3 c
```

(주의) 이 예에서는 B열은 데이터 화일에 넣어서는 안됩니다. BEGINNING은 그것으로 완전한 열지점의 하나가 됩니다. 연속번호는 1부터 차폐대로 1씩 더해져서 번호가 매겨집니다. 가산된 번호를 줄씩 건너 뛰거나 하는 것은 불가능 합니다.

1.4 임의의 수치열에 연속번호를 임의의 간격으로 붙이고 싶은 경우

**** 데이터 구조 ****

```
create table seqth
(a varchar(10),
 b number,
 c varchar(10))
```

**** 외부 데이터화일 ****

```
1,a
2,b
3,c
```

**** 콘트롤 화일 ****

```
load data
infile seq.dat
replace
into table seqth
fields terminated by ','
(a char,
 b sequence(100,5),
 c char)
```

**** 실행결과 ****

```
SQL>select * from seqth;

A          B C
-----
1          100 a
2          105 b
3          110 c
```

(주의) 이 예에서는 B열은 데이터 화일에 넣어서는 안됩니다. BEGINNING은 그것으로 완전한 열지점의 하나가 됩니다. 초기 값 100과 늘린 값 5는 다른 수치로 변경 가능합니다.

1.5 로드하는 논리 레코드를 구성 는 물리 레코드가 특수열로 구성된 경우

(물리 레코드의 1라이프레코로 판단되는 경우)

**** 데이터 구조 ****

```
create table contl_test
(a varchar(10),
 b varchar(10),
 c varchar(10))
```

**** 콘트롤 화일 ****

```
load data
infile cont1.dat
replace
continueif this
(1) = 'A'
into table cont1_test
fields terminated by ','
(a char,
 b char,
 c char)
```

**** 외부 데이터화일 ****

```
1,
2,
3
A,B
,C
A,B
C
A
,B
```

**** 결과결과 ****

BQL>select * from cont1_test;

A	B	C
1	2	3
A	B	C
A	Bcd	ef

(구의) 이 예의 경우 1라이프 패가 계속 향의 패를 위해서 사용되기 때문에, 1 라이프 패부터 시작해서는 안됩니다. 상기 예의 경우, 레코드의 선두 라이프가 'A'일 때 다음의 레코드가 연결됩니다.

1.B 외부데이터화일의 물리 레코드가 특수 레코드로 구성된 경우

(구성하는 물리 레코드 수가 모두 일정한 경우)

**** 데이터 구조 ****

```
create table con_test
(a varchar(10),
 b varchar(10),
 c varchar(10))
```

**** 콘트롤 화일 ****

```
load data
infile cont1.dat
replace
```

—일레모 모든 논리레코드가 그 레코드로 구성됩니다.

```
concatenate 2
into table con_test
fields terminated by ','
(a char,
 b char,
 c char)
```

**** 외부 데이터화일 ****

```
1,2,
3
A,B,
C
```

**** 결과결과 ****

BQL> select * from con_test;

A	B	C
1	2	3
A	B	C
A	B	C

[구의] 하나의 논리테이블이 모두 일정한 갯수의 물리 테이블로부터 성립되는 것 같은 단순한 경우에 한합니다.

1.7 데이터의 같은 문자를 데이터로 코드하고 싶은 경우

**** 데이터를 구조 ****

```
create table mnc
(a varchar(10),
 b varchar(10),
 c varchar(10))
```

**** 쿼리를 확인 ****

```
load data
infile mnc.dat
replace
into table mnc
fields terminated by ',' optionally enclosed by '"' and '"'
(a char,
 b char,
 c char)
```

**** 외부 데이터의 확인 ****

```
"abc,d",2,3
"a,,d",4,5
```

**** 검색결과 ****

```
BQL>select * from mnc;
A      B      C
-----
abc,d  2      3
a,,d   4      5
```

1.8 포지션 지정시 char형 데이터 전후의 블랭크도 코드하고 싶은 경우

**** 데이터를 구조 ****

```
create table pmth
(a varchar(10),
 b varchar(10),
 c varchar(10))
```

**** 쿼리를 확인 ****

```
load data
infile pmth.dat
replace
preserve blanks
into table pmth
(a position(01:05) char,
 b position(06:10) char,
 c position(11:20) char)
```

**** 외부 데이터의 확인 ****

```
12 4 B7B80 ab def hi
2   B7B80 ab def hi
```

**** 검색결과 ****

```
BQL>select * from pmth;
A      B      C
-----
12 4   B7B80   ab def hi
2      B7B80   ab def hi
```

**** 결과확인 ****

```
BQL>select length(a), length(c) from pmth;
LENGTH(A) LENGTH(C)
-----
5          10
5          10
```

1. B 어떤 데이터의 데이터 종류와 상관없이 데이터가 없는 경우 NULL 데이터를 넣도록 하고 싶다

**** 데이터를 구조 ****

```
create table test1
(a varchar(10),
 b varchar(10),
 c varchar(10))
```

**** 쿼리를 확인 ****

```
load data
infile test.dat
replace
into table test1
fields terminated by ','
trailing nullcols
(a char,
 b char,
 c char)
```

**** 외부 데이터 파일 ****

```
1,aa,
2,bb,ff
3,cc,
```

**** 검색결과 ****

SQL>select * from test1

A	B	C
1	aa	
2	bb	ff
3	cc	

(주의) trailing nullcols를 사용하지 않으면 1 레코드짜와 3 레코드짜가 데이터 애머가 됩니다. 데이터가 들어있기도 하고 없기도 한 열의 데이터는 데이터 파일의 최후로 가져옵니다.

1.10 CHAR형 필드가 BLANK로 채워져 있을 때 NULL을 삽입하고 싶은 경우

**** 데이터를 구조 ****

```
create table null1
(a varchar(10),
 b varchar(10),
 c varchar(10))
```

**** 쿼리를 확인 ****

```
load data
infile null.dat
replace
into table null1
fields terminated by ','
(a char,
 b char,
 c char(10) nullif c = blank)
```

**** 외부 데이터 파일 ****

```
aa,bb,
11,22,
BB,BB,AA
00,00,BB
```

**** 검색결과 ****

SQL>select * from null1;

A	B	C
aa	bb	
11	22	
BB	BB	AA
00	00	BB

(주의) 검색결과의 1 행째, 2 행째의 열 C는 블랭크가 아니라 NULL 입니다.

1.11 BYTE필드가 BLANK로 채워져 있을 때 NULL을 삽입하고 싶은 경우

**** 데이터를 구조 ****

```
create table nulltbl2
(a varchar(10),
 b varchar(10),
 c date)
```

**** 콘솔을 확인 ****

```
load data
infile null2.dat
replace
into table nulltbl2
fields terminated by ','
(a char,
 b char,
 c date "YY/MM/DD" nullif c = blank)
```

**** 외부 데이터 파일 ****

```
aa,bb,
11,22,
BB,BB,B2/11/11,
00,00,B4/12/12,
```

**** 검색결과 ****

```
BQL>select * from nulltbl2;
```

A	B	C
aa	bb	
11	22	
BB	BB	B2/11/11
00	00	B4/12/12

[주의] 검색결과 1 행째, 2 행째의 열 C는 블랭크가 아니라 NULL 입니다.

1.12 POSITION 지정시 BLANK를 그대로 로드하고 싶은 경우

**** 데이터를 구조 ****

```
create table nulltbl2
(a varchar(10),
 b varchar(10),
 c date)
```

**** 콘솔을 확인 ****

```
-- position 지정으로 블랭크를 그대로 입력 원하는 경우
-- preserve blanks를 지정한다.
```

```
load data
infile null3.dat
replace
preserve blanks
into table nulltbl2
(a position(1:2) char,
 b position(3:4) char nullif b = blank,
 c position(5:13) date "YY/MM/DD")
```

```
BQL>select length(a), length(b) from nulltbl2;
```

```
LENGTH(A) LENGTH(B)
-----
2          2
2
```

**** 외부 데이터 파일 ****

```
BBBBB2/11/11
B4/12/12
```

**** 검색결과 ****

```
BQL>select * from nulltbl2;
```

A	B	C
BB	BB	B2/11/11
		B4/12/12

[주의] 이 경우 2 레코드에는 A에 블랭크가 들어가고 B에 NULL이 들어갑니다.

1.13 데이터 파일의 수치 데이터 열에 BLANK가 들어가 있을 때 0을 입력하고 싶은 경우

**** 데이터를 구조 ****

```
create table def2
(a varchar(10),
 b varchar(10),
 c number)
```

**** 콘트롤 파일 ****

```
load data
infile def2.dat
replace
into table def2
fields terminated by ','
(a char,
 b char,
 c integer external defaultif c = blank)
```

**** 외부 데이터 파일 ****

```
11,11,123
22,22,
33,33,
44,44, 
```

**** 검색결과 ****

BQL>select * from def2;

A	B	C
11	11	123
22	22	0
33	33	0
44	44	0

1.14 데이터 NULL일 때 NULL이라고 하는 문자열을 넣고 싶은 경우

**** 데이터를 구조 ****

```
create table ifnullth
(a varchar(10),
 b varchar(10),
 c varchar(10))
```

**** 콘트롤 파일 ****

```
load data
infile ifnull.dat
replace
into table ifnullth
fields terminated by ','
(a char,
 b char "null(th,'NULL')",
 c char)
```

**** 외부 데이터 파일 ****

```
1,2,3,
A,,B
a,b,c
```

**** 검색결과 ****

BQL>select * from ifnullth;

A	B	C
1	2	3
A	NULL	B
a	b	c

(주의) NULL과 같은 BQL 함수는 DIRECT LOAD의 경우 BQL 인터페이스를 검증하지 않기 때문에 사용할 수 없습니다.

1.15 어떤 열들 모두 대문자(소문자)로 변환하여 로드하고 싶은 경우

**** 데이터를 구조 ****

```
create table upth
(a varchar(10),
 b varchar(10))
```

**** 외부 데이터 파일 ****

```
abcDEF, ABCDEF
0011,22222
```

**** 콘트롤 할일 ****

```
load data
into table upper.dat
replace
into table upth
fields terminated by ','
(a char "lower(a)",
b char "upper(b)")
```

**** 결과결과 ****

SQL>select * from upth;

A	B
akcdazg	AKCDKZF
cd11	FF6622

1. 1B ZONE업 데이터를 로드하고 실은 경우

ZONE의 데이터 형식은 1문자 1바이트로 나타냅니다.

숫자 +123이라면

bit	1234	5678	1234	5678	1234	5678
	1111	0001	1111	0010	1100	0011
10진		1		2	+	3

여기의 비트의 상위 비트(부호비트)로 정부가 지정됩니다.

1100(정) 1101(부)

가 됩니다.

원래는 EBCDIC코드에서

```
Hex F0 to FB → +0 to +B
Hex C0 to CB → +0 to +B(부호비트)
Hex 80 to BB → +0 to -B(부호비트)
```

이것을 아스키로 변환했기 때문에

부호비트	AKCDKZFHIJZELMDPQB
부호	+++++-----
수	01234567890123456789

가 됩니다.

**** 테이블 구조 ****

```
create table x
(a varchar(10),
i number,
j number)
```

**** 외부 데이터 할일 ****

A12345(12345)

**** 콘트롤 화일 ****

```
load data
infile filenum,del
replace
into table x
(a position (01) char,
 I position(02:07) xnumd(8),
 J position(08:13) xnumd(8))
```

**** 검색결과 ****

```
BQL>select * from x;

A          I          J
-----
A          123450    -123450
```

1.17 decimal형 데이터를 로드하고 실은 경우

decimal 데이터는 1바이트에 2개의 숫자가 지정됩니다.

숫자 +123 이라면

```
bit    1234 5878 1234 5878
        0001 0010 0011 1100
10진   1     2     3     +
```

같은 4바이트는 부호비트로 불리고

1100(정) 1101(부)

가 됩니다.

**** 데이터 구조 ****

```
create table dec
(a varchar(10),
 col1 number,
 col2 number)
```

**** 외부 데이터 형식 ****

데이터화일은 바이너리이기 때문에 numm 등으로 볼 수 없습니다.

```
4112 3458 7=12 3458 710a
```

**** 검색결과 ****

```
BQL>select * from dec;

A          COL1          COL2
-----
A          123,4587    -123,4587
```

(주의)

콘트롤화일에서 decimal형을 사용하는 것에 따라 (length, scale)를 지정합니다.

1.18 고정길이 레코드 데이터를 로드하고 싶은 경우

```

**** 테이블 구조 ****
create table fixth
(a varchar(10),
 b varchar(10))

**** 콘솔을 확인 ****
load data
infile fix.dat
"FIX B"
replace
into table fixth
(a char(3),
 b char(3))

```

```

**** 외부 데이터 확인 ****
123456abcd

**** 외부 데이터 확인 (18진) ****
3132 3334 3536 8182 8384 8586

**** 검색결과 ****
SQL>select * from fixth;

```

A	B
123	456
abc	def

(주의) 데이터 파일의 끝에는 DxDx(CARRIAGE RETURN)가 들어가면 안됩니다. DxDx도 1문자로 카운트 되기 때문입니다.

SAMPLE: SQL*LOADER SAMPLE

USING TRUNC FUNCTION

Bulletin no : 10187

TRUNC FUNCTION 을 사용한 LOADER 예

<< TABLE DESCRIPTION >>

Name	Null?	Type
EMPID	NOT NULL	NUMBER(4)
ENAME		VARCHAR2(10)
JOB		VARCHAR2(8)
MGR		NUMBER(4)
HIREDATE		DATE
BAL		NUMBER(7,1) <---
COMM		NUMBER(7,2)
DEPTNO		NUMBER(2)

<< CONTROL FILE >>

LOAD DATA

INFILE 't.dat'

INTO TABLE EMP2

```
( EMPID POSITION(01:04) INTEGER EXTERNAL,
  ENAME POSITION(08:15) CHAR,
  JOB POSITION(17:25) CHAR,
  MGR POSITION(27:30) INTEGER EXTERNAL,
  BAL POSITION(32:38) DECIMAL EXTERNAL "TRUNC(:BAL,1)", <---
  COMM POSITION(41:48) DECIMAL EXTERNAL,
  DEPTNO POSITION(50:51) INTEGER EXTERNAL)
```

<< DATA FILE - 't.dat' >>

```
*****
7894 MILLER      CLERK      7782      820.00          10
7588 JONES       MANAGER    7838      3123.75          20
7489 ALLEN       SALESMAN   7888      1800.00      300.00 30
7854 MARTIN      SALESMAN   7888      1312.50     1400.00 30
7858 CHAM        ANALYST    7588      3450.00          20
```

<< LOADER 실행 >>

```
$ sqlldr scott/tiger t.ctl
```

SAMPLE: SQL*LOADER SAMPLE PART II

Bulletin no : 10222

NUMBER 와 DATE TYPE 을 이*로 LOAD

1. DATA FILE

7588 31237 881010 20

2. CONTROL FILE

```
LOAD DATA
  INFILE 'u12.dat'
  INTO TABLE EMP
  APPEND
( EMPNO      POSITION(01:04) INTEGER EXTERNAL,
  SAL        POSITION(08:10) DECIMAL EXTERNAL ':SAL/100',
  HIREDATE    POSITION(12:17) DATE 'YYMMDD',
  DEPTNO      POSITION(18:20) INTEGER EXTERNAL)
```

3. LOADER 실행

```
$ sqlldr scott/tiger control=u12.ctl
```

4. TABLE 구조

Name	Null?	Type
EMPNO	NOT NULL	NUMBER(4)
SAL		NUMBER(7, 2)
HIREDATE		DATE
DEPTNO		NUMBER(2)

5. 실행결과

EMPNO	SAL	HIREDATE	DEPTNO
7588	312.37	10-OCT-88	20

ODBC DRIVER를 이용하여

ORACLE DB에 접속하려는 경우

Bulletin no : 10130

* Open Connectivity

Oracle? Workgroup Server는 사용자의 MS Windows응용프로그램과 데이터베이스 사이에 Open Connectivity를 제공한다. Open Connectivity는 사용자가 데이터베이스에 접근할 때 가장 최적의 방법으로써 접근할 수 있는 기술을 선택하기 위한 방법이다.

ODBC를 사용하기 위하여 다음과 같은 단계로 설정을 한다

[단계 1] Oracle? ODBC 프로그램 그룹을 연다.

[단계 2] Oracle? ODBC 윈도우가 나타난다.

(Oracle? ODBC 를 설치하지 않았다면 기본프로그램 그룹에 있는 제어판을 열면 ODBC Administrator 이라는 ICON이 있다.)

[단계 3] ODBC Administrator ICON을 Double-Click하면 다음과 같이 Data Source 윈도우가 나타난다. 만일 Data Source에 새로운 것을 추가 하려면 Add 버튼을 클릭한다.

[단계 4] 현재 사용자의 컴퓨터에 설치되어 있는 ODBC Driver가 나타나고 사용자는 이용하고 싶은 ODBC Driver를 선택한후 OK 버튼을 누른다.

[단계 5] Oracle? ODBC Setup 또는 ODBC Oracle Setup 다이얼로그 박스가 나타나며 그림 밑에 있는 [Section]은 ODBC Data Source 설치가 끝난 후 windows 디렉토리의 ODBC.INI에 저 되는 내용을 보여 주고 있다.

[ORACLE_ODBC]

Driver=C:\WINDOWS\SYSTEM\agora71.dll

Description=ORACLE ? ODBC DRIVER TEST

Server=T:brp2:BC

또는

[Oracle7]

Driver=C:\WINDOWS\SYSTEM\agora.dll

Description=Testing Y&C

Server=T:brp1:PM

UserId=acott

. Data Source Name : Data Source에서 참조할때 사용하기 위한 이름을 기술한다.

. Description : Data Source의 설명을 기술한다. (Option)

. BQLNET Connect String (Network Address)

- BQLNET Connect String을 기술한다.

BQLNET U1 - ADDRESS: Host name:Oracle_BID

BQLNET U2 - TNS: db_alias_name 또는 db_alias_name

[참조]

1. ABBREVIATION

- . A : Asynchronous, Dial-up Modem을 통하여 비동기 접속시
- . AT : Applink Driver, Applink사의 Applink 프로토콜
- . B : NETBIOS, Windows NT에서 로컬 접속과 WFW(Windows For Workgroup)에서 사용하는 대부분의 NetBIOS와 NetBEUI를 위해
- . D : DECnet, Digital Equipment사의 프로토콜
- . P : Named Pipes, WFW(Windows For Workgroup)
- . S : Single Task, MS-DOS상의 Device S DMB
- . T : TCP/IP, LAN 과 WAN에서 사용
- . TT : Unix machine상에서 local server를 위한 Two Task driver
- . K : BPK/IPK, Novell사의 LAN 프로토콜

2. DB_ALIAS_NAME

BQLENET V2를 이용할 때, `tcname.ocx` 파일내에 기술되는 기술자 서 DB_ALIAS_NAME은 사용자정의 필드로 DBACLE_BID과 동일하게 부여해서는 안된다.

. HOST_NAME : `hostc` 파일에 기술되어 있는 HOST_ALIAS /etc directory의 `hostc` 파일에 등록이 다음과 같이 되어있어야 한다.

HOST_ALIAS xxx.xxx.xxx.xxx IP Address

. PORT_NUMBER : `ocxdrv`, `listname`는 다음의port를 통해 서비스 받는다.

NET U1일 경우에는 1525를 사용

NET U2일 경우에는 1521를 사용.

. /etc directory의 `servicex` 파일에 등록이 다음과 같이 되어 있어야 한다.

`ocxdrv` 1525/TCP

`listname` 1521/TCP

[Example] 다음은 BQLENET V2의 `tcname.ocx`에서 접속하려는 서버에 대한 내용이다.

```
DB_ALIAS_NAME =
(DESCRIPTION =
  (ADDRESS_LIST =
    (ADDRESS =
      (PROTOCOL = TCP)
      (HOST = HOST_ALIAS 또는 HOST_IP_ADDRESS)
      (PORT = PORT_NUMBER)
    )
  )
)
(CONNECT_DATA =
  (SID = DBACLE_BID)
)
)
```

“ODBC DRIVER CALL FAILED”에러 메시지

Bulletin no : 10131

Visual Basic, Excel, Access,등 ODBC Driver를 이용하여 ODBC 데이터베이스에 접속시 ODBC connect string(Data Source) 윈도우의 SETUP버튼 또는 ADP...버튼을 통하여 새로운 Data Source를 추가시 설정하는 BQL:SET connect string)이 제대로 부여되었다는 가정하에서 다음과 같이 에러가 난다면 다음과 같은 5가지 접근을 통하여 고칠수 있다.

Error Message : ODBC Driver call failed.

1. Client에서 ODBC ODBC Driver를 1.10.0000 ~ 1.11.0002를 사용한다고 가정하면 c:\windows\system 디렉토리에 sqora7.dll과 sqora71.dll 파일이 있는지 확인한다.

[구경] ODBC version 1.11.0002이상을 사용하면 sqora71.dll과 sqora71.dll 파일을 확인한다.

2. ODBC.INI 파일의 [data source name] 드라이버 section에 다음과 같이 지정이 되어 있는지 확인한다.

Driver=C:\WINDOWS\SYSTEM\sqora7.dll

[구경] ODBC version 1.11.0002이상인 경우

Driver=C:\WINDOWS\SYSTEM\sqora71.dll

3. ODBC HOME 디렉토리가 path에 설정이 되어 있는지 확인한다.

4. ODAWIN.BUL 파일이 존재하여 있지 않았나 확인합니다. (C:\WINDOWS\SYSTEM은 반드시 확인한다.)

5. Application을 실행하지 않은 상태에서 새로운 항목으로 U7.0 BDF를 재설치 한다.

[구경] ODBC version 1.11.0002이상을 사용하면 U7.1 BDF를 재설치한다.

<<< 참고 >>>

ODBC API Conformance에는 CODE, LEVEL 1, LEVEL 2의 3가지가 LEVEL 정의를 있으며 이를 이용하여 Application을 개발할 경우 현재까지는 ODBC Driver에서 제공하는 Function은 CODE, LEVEL 1 Function Call 만을 지원합니다.

<< ODBC API CODE FUNCTION CALLS >>

BQLAllocConnect, BQLAllocEnv, BQLAllocStmt, BQLBindCol, BQLCancel, BQLColAttribute, BQLConnect, BQLDescribeCol, BQLDisconnect, BQLError, BQLExecDirect, BQLExecute, BQLFetch, BQLFreeConnect, BQLFreeEnv, BQLFreeStmt, BQLGetCursorName, BQLNumResultCols, BQLPrepare, BQLRowCount, BQLSetCursorName, BQLSetParam, BQLTransact

<< ODBC API LEVEL 1 FUNCTION CALLS>>

BQLColumn, BQLDriverConnect, BQLGetConnectOption, BQLGetData, BQLGetFunctions, BQLGetInfo, BQLGetStmtOption, BQLGetTypeInfo, BQLParamData, BQLPutData, BQLSetConnectOption, BQLSetStmtOption, BQLSpecialColumn, BQLStatistics, BQLTable

<< ODBC API LEVEL 2 FUNCTION CALLS>>

BQLBrowseConnect, BQLColumnPrivileges, BQLDataSource, BQLDescribeParam, BQLExtendedFetch, BQLForeignKey, BQLHandleData, BQLNativeSql, BQLNumParam, BQLParamOptions, BQLPrimaryKeys, BQLProcedureColumn, BQLProcedure, BQLSetFax, BQLSetScrollOptions, BQLTablePrivileges

VB, GLUE 에서 STORED PROCEDURE 사용 방법

Bulletin no : 10235

Visual Basic에서 DBACLE에 지정되어 있는 Stored Procedure를 수행하는 방법은 다음과 같습니다.

1) Button(txt1)에 입력된 내용

```
Private Sub Command3_Click()  
    Dim Response As String, cmd As String, IBC As Integer  
  
    cmd = "begin adhc_otp.adhc_otp('RESPONSE CENTER'); end;"  
    IBC = ExecuteSQL(txt1, cmd)  
End Sub
```

2) Stored Procedure의 이름 및 내용

```
create or replace package adhc_otp is  
    g_deptno number;  
procedure adhc_otp(v_dept in dept.deptno%type);  
end adhc_otp;  
/  
create or replace package body adhc_otp is  
procedure adhc_otp(v_dept in dept.deptno%type) is  
    begin  
        select max(deptno) + 1 into g_deptno from dept;  
        insert into dept(deptno, dname, loc) values  
            (g_deptno, v_dept, 'DBACLE KOREA');  
    end adhc_otp;  
end adhc_otp;
```

3) 수행 결과

BQL> select * from dept;

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEWYORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON
41	RESPONSE CENTER	DBACLE KOREA
41	RESPONSE CENTER	DBACLE KOREA
43	RESPONSE CENTER	DBACLE KOREA
44	RESPONSE CENTER	DBACLE KOREA

ODBC & SQL*NET SUPPORT MATRIX

Bulletin no : 10317

모든 버전의 ODBC DRIVERS와 SQL*NET은 특정 버전의 Required Support Files(RSF)를 요구하는데 구체적인 사항은 다음과 같다.

ODBC driver version	16/32 bit	Supported Platform	Supported SQL*NET ver.	RSF version	IBM B version
1.11.00.05	16 bit	Windows 3.1	2.0.K for Windows	7.0.K	7.0 or later
1.11.01.05	16 bit	Windows 3.1	2.1.K for Windows	7.1.K	7.1 or later
1.14.01.00	16 bit	Windows 3.1	2.2.K for Windows	7.2.K	7.2 or later
1.15.3.0.1A	16 bit	Windows 3.1	2.3.K for Windows	7.3.K	7.3 or later
1.13.05.00	32 bit	Windows NT and Windows95	2.2.K (32 bit)***	7.2.K	7.2 or later
1.13.05.04	32 bit	Windows NT and Windows95	2.2.K (32 bit)***	7.2.K	7.2 or later
1.15.3.0.1A	32 bit	Windows NT and Windows95	2.3.K (32 bit)***	7.3.K	7.3 or later

UNIX 에서 사용되는 ORACLE 환경 변수

Bulletin no : 10181

The document describes each Oracle environment variable with its name, its use, general example values, and specific examples in the following manner:

ENVIRONMENT VARIABLE NAME

Use: What is this environment variable?

General: What are some example values?

Example: Specific example

In addition, UNIX environment variables that affect Oracle are briefly described.

General Notes

=====

1. \$D_H == \$ORACLE_HOME
2. \$D_B == \$ORACLE_BID
3. An environment variable followed by a "*" applies to Oracle7.
4. An environment variable followed by a "B" is used during database creation for Oracle Version 8 only.
5. For more information, see your IUG, ICG, or the "ORACLE7 Server for UNIX Administrator's Reference Guide"
6. Note that not all UNIX platforms use all of these environment variables.

Oracle Environment Variables

=====

APIPATH*

Use: version 7.0.12 only: directory containing Tool Kit II *.exe files

General:

Example: \$D_H/oracle:\$D_H/tk2/admin

BOOK_LOCALREFERENCE*

Use: Oracle*Book new ver

General: \$ORACLE_HOME/book/admin

Example: \$ORACLE_HOME/book/admin

BOOK_RESOURCE*

Use: directory for Oracle*Book resource files

General: \$D_H/book/admin/resource/US, \$D_H/book/admin/resource/JA

Example: \$ORACLE_HOME/book/admin/resource/US

BOOK_GLOBALPREFERENCES*

Use: DesktopBook new var
General: \${BOOK_HOME}/book/adm
Example: \${BOOK_HOME}/book/adm

BOOK_HELP*

Use: directory for DesktopBook help files
General: \${BOOK_HOME}/book/adm/help/US
Example: \${BOOK_HOME}/book/adm/help/US

CASE_HP_CMD

Use: Command to plot a file from CASE*Designer
General: Actual plot command or name of a script
Example: lpr -Pplot

CASE_PS_CMD

Use: Command to print postscript file from CASE*Designer
General: Actual print command or name of a script
Example: lpr -P

CASE_RESOURCE

Use: Tool Kit I terminal type for CASE
General: file/device (file, r from \${D_H}/dict50/adm/etc) and (device from Desktop)
Example: case_hpxterm

CASE_BITMAP

Use: to print CASE*Designer screen prints to file or PostScript printers
General: printer name
Example: lu

CASE_WINDOW

Use: Tool Kit I GUI window type for CASE
General: xterm, hpterm, sixterm, dxterm, etc.
Example: xterm

CASE_HOME

Use: home directory of CASE Generator
General:
Example: \${D_H}/cgm20

DEBUG_BITMAP*

Use: Debug for YEE tools
General: 0, 1, ?
Example: 1

DNS_FILE

Use: Initial database file name
General: \${D_H}/dkc/dkc\${D_B}.dkf will load code dk file (you will not
be able to move \${D_H}); setting to dkc\${D_B}.dkf will allow you to move \${D_H}
Example: dkcexecio.dkf

DBS_SIZE

Use: Initial database file size
 General: numberK, numberM
 Example: 5000K or 5M minimum, up to partition size

FORMS_DEVICE

Use: device(s) for YE2 tools (CBE tools)
 General: vt220, vt100, lp, null (see \$ORACLE_HOME/tk2/admin/terminal)
 Example: lp

FORMSPATH

Use: directory containing Form 3.0 (YE 1) resource files
 General:
 Example: \$ORACLE_HOME/forms30/admin/resource

LOG

Use: file to record the install procedure
 General: \$O_H/install/install.log
 Example: \$O_H/install/install.log

LOG_FILE1

Use: first redo log file name
 General: \$O_H/dbs/log1\$O_B.dbf will hard code log file (you will not be able to move \$O_H); setting to log1\$O_B.dbf will allow you to move \$O_H
 Example: log1orac1a.dbf

LOG_FILE2

Use: first redo log file name
 General: \$O_H/dbs/log2\$O_B.dbf will hard code log file (you will not be able to move \$O_H); setting to log2\$O_B.dbf will allow you to move \$O_H
 Example: log2orac1a.dbf

LOG_SIZE

Use: Initial log file size
 General: numberK, numberM
 Example: no minimum, 500K default, no maximum (tune to your database needs)

MENU5PATH

Use: directory containing Menu 5.0 (YE 1) resource files
 General:
 Example: \$ORACLE_HOME/menu5/admin/resource

NN_MESOURCE

Use: ?
 General: \$ORACLE_HOME/nn/admin/resource/US
 Example: \$ORACLE_HOME/nn/admin/resource/US

NLS_LANG

Use: National Language Support language, territory, and character set
General: language_territory.characterset
Example: american_american.us7ascii

NO_MAKE

Use: set to true if your system doesn't have C dev tools (cc, make, ld, etc.)
General:
Example: false

ORACLE_BASE

Use: use directory structure for Oracle 7 to follow Oracle Flexible Architecture
General: see OGG "Preparing to Install Oracle Products"
Example: can be any directory

ORACLE_HELP

Use: directory containing help files?
General: \$ORACLE_HOME/help/adain/resource
Example: \$ORACLE_HOME/help/adain/resource

ORACLE_HOME

Use: dir containing top level Oracle directories
General: any directory with enough space
Example: /usr/oracle

ORACLE_ICONS

Use: dir containing icons for GUI tools
General: \$ORACLE_HOME/guicommon/tk2/adain/icons
Example: \$ORACLE_HOME/guicommon/tk2/adain/icons

ORACLE_LPAGES

Use: which lp arguments to use for Easy*SQL, SQL*Calc, SQL*Form,
SQL*Menu, SQL*Report, or SQL*ReportWriter
General:
Example: -c -x

ORACLE_LPPROC

Use: which lp command to use for Easy*SQL, SQL*Calc, SQL*Form,
SQL*Menu, SQL*Report, or SQL*ReportWriter
General: lp, lpr, print
Example: lp

ORACLE_LPSTAT

Use: which lp status command to use
General: lpstat, lpr
Example: lpstat

ORACLE_OWNER

Use: UNIX Operating System usercode who owns the Oracle files
 General: can be any user, default is oracle
 Example: oracle

ORACLE_PAGE0

Use: which UNIX pager to use (more, less, pg) in Oracle Products like SQL*Menu
 General:
 Example: more

ORACLE_PATH

Use: directory for reading and writing to and from SQL*Form, SQL*Menu 5.0, SQL*Plus
 General: can be any directory (if not set, read and write from current directory)
 Example: \$HOME/oracle

ORACLE_REMOTE

Use: For client only install
 General: Y if client only, F if there will be a local database
 Example: Y

ORACLE_SID

Use: Oracle System Identifier
 General: must begin with a letter, followed by a number or character; limited by some OS's to 4 chars
 Example: v712

ORACLE_TERM*

Use: Tool Kit II now var pointing to the tk2c\${ORACLE_TERM}.exe file under \$ORACLE_HOME/rdhnc/adaln/terminal (tk2c stands for character toolkit2)
 General: any file like tk2c\${ORACLE_TERM}.exe
 Example: vt100

ORACLE_TERMINAL

Use: directory where the Tool Kit II .exe files reside
 General: \$D_H/tk2/adaln/terminal
 Example: \$D_H/tk2/adaln/terminal

ORACLE_TRACE

Use: allow verification/trace to be turned on and off during install (includes everything done by sh)
 General: Y does not -x; anything else is off
 Example: Y

ORACLE_VERIFY

Use: If true, run the .verify scripts during install
 General: Y; anything else is off
 Example: Y

ORACLE_HOME

Use: points to directory containing oracle doc starting v7.0.18
 General: to any directory containing oracle doc (file,old)
 Example: \$ORACLE_HOME/doc

ORACLE_ARE

Use: If set to anything, prompts for \$D_B or \$D_H when (c)oracle is invoked
 General: NO or anything
 Example: NO

ORACLE_PATH

Use: set to directory containing the v1 oracle resource file
 General: \$ORACLE_HOME//admin/resource
 Example: \$ORACLE_HOME/force30/admin/resource

ORACLE_EDITOR

Use: which editor to use for OracleMail
 General: vi, emacs, ed
 Example: vi

ORACLE_NBS

Use: number of components that make up a set
 General: 1-UNIX maximum
 Example: 80

ORACLE_PIPE

Use: sets the default pipe driver to v1 or v2
 General: v1, v2
 Example: v2

ORACLE_PATH

Use: directory containing Oracle (V1) resource files
 General:
 Example: \$ORACLE_HOME/oracle/admin/resource

PATCH_HOME

Use: directory containing patches and patch.list
 General: any directory, \$D_H/patch, \$D_H/install/patch
 Example: \$ORACLE_HOME/install/patch

RY_STATUS

Use: directory for BQLTextRetrieval V2.0 to find forms
 General: can be any directory (if not set, read and write from current directory)
 Example: \$HOME/oracle

SQL_HOME

Use: directory containing top level CASE Dictionary directories
 General: can be any directory containing top level CASE directories
 Example: \$D_H/dict50/admin

BDD_PRINT

Use: for CBB: set to UNIX print command to use
 General: lp, lpr
 Example: lp

BDD_UPRINT

Use: for CBB: set to UNIX print command to use for wide
 General: lp, lpr
 Example: lp

BQLPATH

Use: now superseded by \$ORACLE_PATH
 General: can be any directory (if not set, read and write from current directory)
 Example: \$HOME/oracle

BQLTB_STATUS

Use: directory for BQL*Text*Status V1.1 to find forms
 General: can be any directory (if not set, read and write from current directory)
 Example: \$HOME/oracle

BBI_TMP

Use: directory to use for BQL*Report*Writer temporary files
 General: any directory with >10M of disk space
 Example: /usr/tmp

TEZMENU*

Use: version 7.0.12 only: Tool Kit II terminal resource file
 General: vt100 portion from tk2_vt100.rmc
 Example: vt100

TNS_ADMIN*

Use: directory containing BQL*Net v2 files
 General: any dir (see manual for order followed--looks at /etc, \$HOME, and \$TNS_ADMIN)
 Example: \$D_H/network/admin

TUD_TABE

Use: hoststring to use for BQL*Net v1
 General: p:, f:, xihost:\$D_B, dihost:\$D_B, xtar: host:\$D_B,
 tihost:\$D_B, ttihost:\$D_B, xihost:\$D_B
 Example: tihost:oracle

UNIX Environment Variables Affecting Oracle

=====

ADA_PATH set to directory containing Ada compiler

ADYCOMP_DIR set to directory containing Ada compiler

DISPLAY
 Use: Tell K-based tools which Display to use
 General: 'hostname':0.0
 Example: fubar:0.0

HOME \$HOME

LANG en_US, de_DE

LANGUAGE american_english

LOGPTS ld options for compiling

LODPATH directory containing shared object libraries

LD_LIBRARY_PATH directory containing shared object libraries

LOG when set to a filename, log information from install section

LOGNAME 'logname' or 'whoami'

NALLOCTYPE 3.1, 3.2 for telling AIX which malloc type to use

NLSPATH DB directory containing language message files

NOMAMESERVER
 Use: For tcp/ip networks without a named server
 General: 1 for no named server, 0 for named server
 Example: 1

PATH \$PATH

PRINTER

SHELL /bin/csh or /bin/sh

TERM any valid term: xterm, left, hp, mac2, xms, xterm, vtxxx

THPPID some UNIX boxes allow /tmp to be renamed to \$THPPID (any directory)

TZ sets local time zone

REQUIREMENT set to file Deskit, or other KII resource files

KAPPLIBDIR set to dir containing resource specifications file
 Deskit, or other KII resource files

(V7.3) PARTITION VIEW 테스트 REPORT

Bullittin no : 10388

1. 개요

대용량 Database를 구축하고 관리하는 데 있어서 필연적으로 나타날 수 있는 문제는 시간적 증가 또 지역적으로 확대 됨에 따라 어느 TABLE의 SIZE가 증가함에 따라 이에 따른 관리 방법의 효율화 일 것이다.
이는 주기적인 재편성(Rew-Organization)뿐만 아니라 Backup, Index의 리대화 등에 따른 Query의 효율저하 등 여러가지 문제를 야기 시킬 수 있다.
이에 대한 해결책인 Partition View(Ver. 7.3 이상)를 실무적용사항을 통해서 구축하는 방법, Partition View의 효율적 사용 방법, 주의 할 점등을 말아보고자 한다.

2. 구축 과정

1) Partition View를 위한 Parameter

initDBDBFILE.ora File에 다음의 Parameter Setting

```
partition_view_enabled=TRUE
```

2) Partition View 대상 Table 및 Partition Column을 선정

이는 물리적 설계 시 결정

Partition Column의 주요 후보는 일시, 지역 등 명확히 구분되는 Column이어야 함

3) Table Create->Partition Column에 반드시 해당 범위의 Check Constraint를 반드시주어야 함. (주의 사항)

Partition Column의 DATATYPE를 CHAR로 하지 말 것 (NLC) -> VARCHAR2 사용 (NLC로 등록되어 있음)
Partition View를 구성하는 모든 Table Schema는 완전히 동일 해야 함

(Column의 갯수, Datatype, Null허용 여부 등)

실제로 테스트에서 TYPE의 불일치로 문제가 발생했었음.

1998.01월 Table(TOUCHSTONE_199801)의 한 Column의 Type이 다른 Table은 모두 VARCHAR(8) 인데 이 Table만 VARCHAR2(10)로 되어있어서 Partition View를 JOIN하면 VIEW 내의 모든 TABLE이 FULL TABLE SCAN을 할 DATATYPE를 동일하게 해준 후 정상적으로 동작

4) Index Create --> 모든 Table이 반드시 동일한 Index를 가져야 함.

Partition Column에 Index가 반드시 필요한 것은 아님.

5) 각 Table에 ANALYZE TABLE를 수행 (가능한 한 COMPUTE STATISTICS로)

8) Partition 내의 모든 Table을 UNION ALL 한 Partition View를 생성

3. 구축 사례

TABLESPACE_TBBS07 - TABLESPACE_TBBS08 (15개 Table)

각 Table의 예상 Row 수는 최소 1,200 만 이상으로 15개월을 보관함을 원칙
으로 하여 총 Row는 2억건 정도이다.

CREATE TABLE TABLESPACE_TBBS07

```
( USB_NO          varchar2(8)      not null,
  CABD_NO          varchar2(18)     not null,
  REQ_BEG_CLBB      char(1)         not null,
  LD_AD_CLBB        char(1)         not null,
  MED_MGMT_NO       varchar2(8)     not null,
  AUTH_CLBB         char(2)         not null,
  AUTH_DATE         varchar2(8)     not null, --> Partition Column
  AUTH_TIME         char(8)         not null,
  AUTH_AMT          number(13,2)    not null,
  EN_MODE           char(2)         not null,
  AUTH_REQ_MTHD      char(2)         not null,
  AUTH_REQ_CLBB      char(1)         not null,
  SALE_BEF_PL       char(2)         ,
  REQ_MED_MGMT_NO    varchar2(8)     ,
  SALE_PL_NO        varchar2(12)     ,
  REQ_VALD_LIN       char(8)         ,
```

```
-----
-----
-----
-----
-----
-----
```

```
USB_AMT            number(11)       ,
RCP_AMT            number(11)       ,
MED_RCP_FEE        number(11)       ,
RCP_DATE           char(8)          ,
POB_CODE           char(2)          ,
CONSTRAINT C_AUTH_DATE_TBBS07
CHECK (AUTH_DATE BETWEEN '18850701' AND '18850731'))
--> Partition Column Constraint

TABLESPACE TB_TUNING01
PCTFREE 10
STORAGE(INITIAL 100M NEXT 100M PCTINCREASE 0
        MAXEXTENTS UNLIMITED FREELIST 10);
```

* Index 정의

Index 1 : CAMP_NO

```
CREATE INDEX TCAMPUSE_18507_IDX1
ON TCAMPUSE_18507(CAMP_NO)
TABLESPACE TS_TUNING_101
STORAGE(INITIAL 50M NEXT 50M PCTINCREASE 0
        MAXEXTENTS UNLIMITED)
PARALLEL (DEGREE 4);
```

Index 2 : USE_NO

```
CREATE INDEX TCAMPUSE_18507_IDX2
ON TCAMPUSE_18507(USE_NO)
TABLESPACE TS_TUNING_101
STORAGE(INITIAL 50M NEXT 50M PCTINCREASE 0
        MAXEXTENTS UNLIMITED)
PARALLEL (DEGREE 4);
```

Index 3 : AUTH_DATE + HER_MCHT_NO

```
CREATE INDEX TCAMPUSE_18507_IDX3
ON TCAMPUSE_18507(AUTH_DATE, HER_MCHT_NO)
TABLESPACE TS_TUNING_101
STORAGE(INITIAL 50M NEXT 50M PCTINCREASE 0
        MAXEXTENTS UNLIMITED)
PARALLEL (DEGREE 4);
```

Index 4 : HER_MCHT_NO + AUTH_DATE

```
CREATE INDEX TCAMPUSE_18507_IDX4
ON TCAMPUSE_18507(HER_MCHT_NO, AUTH_DATE)
TABLESPACE TS_TUNING_101
STORAGE(INITIAL 50M NEXT 50M PCTINCREASE 0
        MAXEXTENTS UNLIMITED)
PARALLEL (DEGREE 4);
```

* Table Analyzing

```
analyze table TCAMPUSE_18507 compute statistics;
```

* Partition View Creation

```
CREATE OR REPLACE VIEW PV_TCAMPUSE
AS
SELECT * FROM TCAMPUSE_18507
```

```

UNION ALL
SELECT * FROM TCAMPUSE_188508
UNION ALL
SELECT * FROM TCAMPUSE_188509
UNION ALL
SELECT * FROM TCAMPUSE_188510
UNION ALL
SELECT * FROM TCAMPUSE_188511
UNION ALL
SELECT * FROM TCAMPUSE_188512
UNION ALL
SELECT * FROM TCAMPUSE_188601
UNION ALL
SELECT * FROM TCAMPUSE_188602
UNION ALL
SELECT * FROM TCAMPUSE_188603
UNION ALL
SELECT * FROM TCAMPUSE_188604
UNION ALL
SELECT * FROM TCAMPUSE_188605
UNION ALL
SELECT * FROM TCAMPUSE_188606
UNION ALL
SELECT * FROM TCAMPUSE_188607
UNION ALL
SELECT * FROM TCAMPUSE_188608
UNION ALL
SELECT * FROM TCAMPUSE_188609

```

4. Test Procedure

1) Partition View에 동작 여부 확인

(1) Partition Column에 '-'로 비교 되었을 때

```

SELECT CAMP_NO, USE_NO, AUTH_DATE, AUTH_AY
FROM PU_TCAMPUSE
WHERE USE_NO = '00002037'
AND AUTH_DATE = '18860723';

```

SQL> Execution Plan

```

-----
0  SELECT STATEMENT  Cost: 10000
1    VIEW OF 'PU_TCAMPUSE'
1      UNION-ALL (PARTITION)
0        FILTER
0          TABLE ACCESS OF 'TCAMPUSE_188507'
0            INDEX (RANGE SCAN) OF 'TCAMPUSE_188507_IDX2'

```

```

0      FILTER
0      TABLE ACCESS DF 'TBCAMUSE_18B508'
0      INDEX (NAME BCAM) DF 'TBCAMUSE_18B508_IDX2'
0      FILTER
0      TABLE ACCESS DF 'TBCAMUSE_18B508'
0      INDEX (NAME BCAM) DF 'TBCAMUSE_18B508_IDX2'
0      FILTER --> Access 불 필요 Skip Operation
0      TABLE ACCESS DF 'TBCAMUSE_18B510'
0      INDEX (NAME BCAM) DF 'TBCAMUSE_18B510_IDX2'
0      FILTER
0      TABLE ACCESS DF 'TBCAMUSE_18B511'
0      INDEX (NAME BCAM) DF 'TBCAMUSE_18B511_IDX2'
0      FILTER
0      TABLE ACCESS DF 'TBCAMUSE_18B512'
0      INDEX (NAME BCAM) DF 'TBCAMUSE_18B512_IDX2'
0      FILTER
0      TABLE ACCESS DF 'TBCAMUSE_18B801'
0      INDEX (NAME BCAM) DF 'TBCAMUSE_18B801_IDX2'
0      FILTER
0      TABLE ACCESS DF 'TBCAMUSE_18B802'
0      INDEX (NAME BCAM) DF 'TBCAMUSE_18B802_IDX2'
0      FILTER
0      TABLE ACCESS DF 'TBCAMUSE_18B803'
0      INDEX (NAME BCAM) DF 'TBCAMUSE_18B803_IDX2'
0      FILTER
0      TABLE ACCESS DF 'TBCAMUSE_18B804'
0      INDEX (NAME BCAM) DF 'TBCAMUSE_18B804_IDX2'
0      FILTER
0      TABLE ACCESS DF 'TBCAMUSE_18B805'
0      INDEX (NAME BCAM) DF 'TBCAMUSE_18B805_IDX2'
0      FILTER
0      TABLE ACCESS DF 'TBCAMUSE_18B806'
0      INDEX (NAME BCAM) DF 'TBCAMUSE_18B806_IDX2'
1      TABLE ACCESS DF 'TBCAMUSE_18B807'
2      INDEX (NAME BCAM) DF 'TBCAMUSE_18B807_IDX2'
0      FILTER
0      TABLE ACCESS DF 'TBCAMUSE_18B808'
0      INDEX (NAME BCAM) DF 'TBCAMUSE_18B808_IDX2'
0      FILTER
0      TABLE ACCESS DF 'TBCAMUSE_18B808'
0      INDEX (NAME BCAM) DF 'TBCAMUSE_18B808_IDX2'

```

(설명) Partition View의 가장 밑단적인 사용 Case로 Partition Column이
 *4로 되고 지었을 때는 Access 불 필요 Table은 Skip하는 Operation
 (FILTER)이 동작 함.

(2) Partition Column이 " BETWEEN " AND " *로 되고 질 때

```

SELECT CAMP_NO, USE_NO, AUTH_DATE, AUTH_AMT
FROM   PU_TPCAMPUSE
WHERE  USE_NO = '00002037'
AND    AUTH_DATE BETWEEN '19850701' AND '19850731'

```

----- Execution Plan -----

```

0  SELECT STATEMENT    GOAL: CHOOSE
1    VIEW OF 'PU_TPCAMPUSE'
1    UNION-ALL (PARTITION)
0      FILTER
0        TABLE ACCESS OF 'TPCAMPUSE_198507'
0          INDEX (RANGE SCAN) OF 'TPCAMPUSE_198507_I0K2'
0      FILTER
0        TABLE ACCESS OF 'TPCAMPUSE_198508'
0          INDEX (RANGE SCAN) OF 'TPCAMPUSE_198508_I0K2'
0      FILTER
0        TABLE ACCESS OF 'TPCAMPUSE_198509'
0          INDEX (RANGE SCAN) OF 'TPCAMPUSE_198509_I0K2'
0      FILTER
0        TABLE ACCESS OF 'TPCAMPUSE_198510'
0          INDEX (RANGE SCAN) OF 'TPCAMPUSE_198510_I0K2'
0      FILTER
0        TABLE ACCESS OF 'TPCAMPUSE_198511'
0          INDEX (RANGE SCAN) OF 'TPCAMPUSE_198511_I0K2'
0      FILTER
0        TABLE ACCESS OF 'TPCAMPUSE_198512'
0          INDEX (RANGE SCAN) OF 'TPCAMPUSE_198512_I0K2'
0      FILTER
0        TABLE ACCESS OF 'TPCAMPUSE_198501'
0          INDEX (RANGE SCAN) OF 'TPCAMPUSE_198501_I0K2'
0      FILTER
0        TABLE ACCESS OF 'TPCAMPUSE_198502'
0          INDEX (RANGE SCAN) OF 'TPCAMPUSE_198502_I0K2'
0      FILTER
0        TABLE ACCESS OF 'TPCAMPUSE_198503'
0          INDEX (RANGE SCAN) OF 'TPCAMPUSE_198503_I0K2'
0      FILTER
0        TABLE ACCESS OF 'TPCAMPUSE_198504'
0          INDEX (RANGE SCAN) OF 'TPCAMPUSE_198504_I0K2'
0      FILTER
0        TABLE ACCESS OF 'TPCAMPUSE_198505'
0          INDEX (RANGE SCAN) OF 'TPCAMPUSE_198505_I0K2'
0      FILTER
0        TABLE ACCESS OF 'TPCAMPUSE_198506'
0          INDEX (RANGE SCAN) OF 'TPCAMPUSE_198506_I0K2'
1    TABLE ACCESS OF 'TPCAMPUSE_198507'
2      INDEX (RANGE SCAN) OF 'TPCAMPUSE_198507_I0K2'

```

```

0      FILTER
0      TABLE ACCESS DF 'TDCAMPUSE_188808'
0      INDEX (RANGE SCAN) DF 'TDCAMPUSE_188808_IDX2'
0      FILTER
0      TABLE ACCESS DF 'TDCAMPUSE_188808'
0      INDEX (RANGE SCAN) DF 'TDCAMPUSE_188808_IDX2'

```

(설명) Partition Column이 BETWEEN ~ AND ~ 로 비교 되었을 때는
Access 불 필요 Table은 Skip하는 Operation(FILTER)이 동작 함

(3) Partition Column이 ~ LIKE ~로 비교 될 때

```

SELECT CAMP_NO, USE_NO, AUTH_DATE, AUTH_AMT
FROM   PU_TDCAMPUSE
WHERE  USE_NO = '00002037'
AND    AUTH_DATE LIKE '188807%'

```

Block Execution Plan

```

-----
0 SELECT STATEMENT  GOAL: CHOOSE
1  VIEW OF 'PU_TDCAMPUSE'
1    UNION-ALL (PARTITION)
0      TABLE ACCESS DF 'TDCAMPUSE_188807'
1      INDEX (RANGE SCAN) DF 'TDCAMPUSE_188810_IDX2'
0      TABLE ACCESS DF 'TDCAMPUSE_188808'
1      INDEX (RANGE SCAN) DF 'TDCAMPUSE_188810_IDX2'
0      TABLE ACCESS DF 'TDCAMPUSE_188808'
1      INDEX (RANGE SCAN) DF 'TDCAMPUSE_188810_IDX2'
0      TABLE ACCESS DF 'TDCAMPUSE_188810'
1      INDEX (RANGE SCAN) DF 'TDCAMPUSE_188810_IDX2'
0      TABLE ACCESS DF 'TDCAMPUSE_188811'
1      INDEX (RANGE SCAN) DF 'TDCAMPUSE_188811_IDX2'
0      TABLE ACCESS DF 'TDCAMPUSE_188812'
1      INDEX (RANGE SCAN) DF 'TDCAMPUSE_188812_IDX2'
0      TABLE ACCESS DF 'TDCAMPUSE_188801'
1      INDEX (RANGE SCAN) DF 'TDCAMPUSE_188801_IDX2'
1      TABLE ACCESS DF 'TDCAMPUSE_188802'
2      INDEX (RANGE SCAN) DF 'TDCAMPUSE_188802_IDX2'
0      TABLE ACCESS DF 'TDCAMPUSE_188803'
1      INDEX (RANGE SCAN) DF 'TDCAMPUSE_188803_IDX2'
0      TABLE ACCESS DF 'TDCAMPUSE_188804'
1      INDEX (RANGE SCAN) DF 'TDCAMPUSE_188804_IDX2'
0      TABLE ACCESS DF 'TDCAMPUSE_188805'
1      INDEX (RANGE SCAN) DF 'TDCAMPUSE_188805_IDX2'
0      TABLE ACCESS DF 'TDCAMPUSE_188806'
1      INDEX (RANGE SCAN) DF 'TDCAMPUSE_188806_IDX2'
1      TABLE ACCESS DF 'TDCAMPUSE_188807'
2      INDEX (RANGE SCAN) DF 'TDCAMPUSE_188807_IDX2'

```

```

0  TABLE ACCESS OF 'TTCAMUSE_18808'
1  INDEX (NAME SCAM) OF 'TTCAMUSE_18808_IDX2'
0  TABLE ACCESS OF 'TTCAMUSE_18808'
1  INDEX (NAME SCAM) OF 'TTCAMUSE_18808_IDX2'

```

(설명) Partition Column에 NULL이 없고 비었을 때는
FILTER가 동작하지 않음. (주의 요망)

(4) Partition Column에 "DB"로 비고 질 때

```

SELECT CAMP_NO, UBE_NO, AUTH_DATE, AUTH_AMT
FROM   PU_TTCAMUSE
WHERE  UBE_NO = '00002037'
AND    ( AUTH_DATE BETWEEN '1880801' AND '1880830'
        OR AUTH_DATE BETWEEN '1880701' AND '1880730' )

```

Block Execution Plan

```

0  SELECT STATEMENT  GOAL: CHOOSE
1  VIEW OF 'PU_TTCAMUSE'
1  UNION-ALL (PARTITION)
0  TABLE ACCESS OF 'TTCAMUSE_18807'
1  INDEX (NAME SCAM) OF 'TTCAMUSE_18810_IDX2'
0  TABLE ACCESS OF 'TTCAMUSE_18808'
1  INDEX (NAME SCAM) OF 'TTCAMUSE_18810_IDX2'
0  TABLE ACCESS OF 'TTCAMUSE_18808'
1  INDEX (NAME SCAM) OF 'TTCAMUSE_18810_IDX2'
0  TABLE ACCESS OF 'TTCAMUSE_18810'
1  INDEX (NAME SCAM) OF 'TTCAMUSE_18810_IDX2'
0  TABLE ACCESS OF 'TTCAMUSE_18811'
1  INDEX (NAME SCAM) OF 'TTCAMUSE_18811_IDX2'
0  TABLE ACCESS OF 'TTCAMUSE_18812'
1  INDEX (NAME SCAM) OF 'TTCAMUSE_18812_IDX2'
0  TABLE ACCESS OF 'TTCAMUSE_18801'
1  INDEX (NAME SCAM) OF 'TTCAMUSE_18801_IDX2'
1  TABLE ACCESS OF 'TTCAMUSE_18802'
2  INDEX (NAME SCAM) OF 'TTCAMUSE_18802_IDX2'
0  TABLE ACCESS OF 'TTCAMUSE_18803'
1  INDEX (NAME SCAM) OF 'TTCAMUSE_18803_IDX2'
0  TABLE ACCESS OF 'TTCAMUSE_18804'
1  INDEX (NAME SCAM) OF 'TTCAMUSE_18804_IDX2'
0  TABLE ACCESS OF 'TTCAMUSE_18805'
1  INDEX (NAME SCAM) OF 'TTCAMUSE_18805_IDX2'
0  TABLE ACCESS OF 'TTCAMUSE_18806'
1  INDEX (NAME SCAM) OF 'TTCAMUSE_18808_IDX2'
1  TABLE ACCESS OF 'TTCAMUSE_18807'
2  INDEX (NAME SCAM) OF 'TTCAMUSE_18807_IDX2'
0  TABLE ACCESS OF 'TTCAMUSE_18808'

```



```

1      INDEX (NAME SCAM) OF 'TTCAMUSE_188808_IDX2'
0      TABLE ACCESS OF 'TTCAMUSE_188808'
1      INDEX (NAME SCAM) OF 'TTCAMUSE_188808_IDX2'

```

(설명) Partition Column이 *로 되어 있고 피엡을 때는
 FILTER가 동작하지 않음. (NULL로 등록되어 있음)

(5) Partition Column이 * BETWEEN *으로 비교되고 다른 비교
 Column이 없을 때

```

SELECT CAMP_NO, USE_NO, AUTH_DATE, AUTH_AMT
FROM   PU_TTCAMUSE
WHERE  AUTH_DATE BETWEEN '18880801' AND '18880830'

```

EXECUTION PLAN

```

-----
SELECT STATEMENT
  VIEW PU_TTCAMUSE
    UNION-ALL (PARTITION)
      FILTER
        TABLE ACCESS (BY ROWID) OF TTCAMUSE_188507
          INDEX (NAME SCAM) TTCAMUSE_188510_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TTCAMUSE_188508
          INDEX (NAME SCAM) TTCAMUSE_188510_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TTCAMUSE_188508
          INDEX (NAME SCAM) TTCAMUSE_188510_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TTCAMUSE_188510
          INDEX (NAME SCAM) TTCAMUSE_188510_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TTCAMUSE_188511
          INDEX (NAME SCAM) TTCAMUSE_188511_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TTCAMUSE_188512
          INDEX (NAME SCAM) TTCAMUSE_188512_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TTCAMUSE_188801
          INDEX (NAME SCAM) TTCAMUSE_188801_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TTCAMUSE_188802
          INDEX (NAME SCAM) TTCAMUSE_188802_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TTCAMUSE_188803
          INDEX (NAME SCAM) TTCAMUSE_188803_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TTCAMUSE_188804

```

```

INDEX (NAME BCAM) TBCAMUSE_199804_IDX
FILTER
TABLE ACCESS (BY EQUIP) OF TBCAMUSE_199805
INDEX (NAME BCAM) TBCAMUSE_199805_IDX
TABLE ACCESS FULL TBCAMUSE_199806
FILTER
TABLE ACCESS (BY EQUIP) OF TBCAMUSE_199807
INDEX (NAME BCAM) TBCAMUSE_199807_IDX
FILTER
TABLE ACCESS (BY EQUIP) OF TBCAMUSE_199808
INDEX (NAME BCAM) TBCAMUSE_199808_IDX
FILTER
TABLE ACCESS (BY EQUIP) OF TBCAMUSE_199809
INDEX (NAME BCAM) TBCAMUSE_199809_IDX

```

(B) Partition Column에 "DB"로 비교되고 다른 비교 Column이 없을 때.

```

SELECT CAMD_NO, USE_NO, AUTH_DATE, AUTH_AMT
FROM PU_TBCAMUSE
WHERE AUTH_DATE BETWEEN '19980501' AND '19980531'
OR AUTH_DATE BETWEEN '19980601' AND '19980630'
OR AUTH_DATE BETWEEN '19980701' AND '19980731'

```

EXECUTION PLAN

SELECT STATEMENT

VIEW PU_TBCAMUSE

UNION-ALL (PARTITION)

```

TABLE ACCESS FULL TBCAMUSE_199807
TABLE ACCESS FULL TBCAMUSE_199808
TABLE ACCESS FULL TBCAMUSE_199808
TABLE ACCESS FULL TBCAMUSE_199810
TABLE ACCESS FULL TBCAMUSE_199811
TABLE ACCESS FULL TBCAMUSE_199812
TABLE ACCESS FULL TBCAMUSE_199801
TABLE ACCESS FULL TBCAMUSE_199802
TABLE ACCESS FULL TBCAMUSE_199803
TABLE ACCESS FULL TBCAMUSE_199804
TABLE ACCESS FULL TBCAMUSE_199805
TABLE ACCESS FULL TBCAMUSE_199806
TABLE ACCESS FULL TBCAMUSE_199807
TABLE ACCESS FULL TBCAMUSE_199808
TABLE ACCESS FULL TBCAMUSE_199808

```

※ 시약의 경우

(?) 위 (B)의 Query를 "BETWEEN"으로 변환 경우

```

SELECT /*+ FULL(PV_TCCARRUSE) */
      CAMP_NO, USE_NO, AUTH_DATE, AUTH_AMT
FROM   PV_TCCARRUSE
WHERE  AUTH_DATE BETWEEN '19880501' AND '19880731'

EXECUTION PLAN
-----
SELECT STATEMENT
  VIEW PV_TCCARRUSE
    UNION-ALL (PARTITION)
      FILTER
        TABLE ACCESS (BY ROWID) OF TCCARRUSE_198507
          INDEX (RANGE SCAN) TCCARRUSE_198510_IDX
      FILTER
        TABLE ACCESS (BY ROWID) OF TCCARRUSE_198508
          INDEX (RANGE SCAN) TCCARRUSE_198510_IDX
      FILTER
        TABLE ACCESS (BY ROWID) OF TCCARRUSE_198509
          INDEX (RANGE SCAN) TCCARRUSE_198510_IDX
      FILTER
        TABLE ACCESS (BY ROWID) OF TCCARRUSE_198510
          INDEX (RANGE SCAN) TCCARRUSE_198510_IDX
      FILTER
        TABLE ACCESS (BY ROWID) OF TCCARRUSE_198511
          INDEX (RANGE SCAN) TCCARRUSE_198511_IDX
      FILTER
        TABLE ACCESS (BY ROWID) OF TCCARRUSE_198512
          INDEX (RANGE SCAN) TCCARRUSE_198512_IDX
      FILTER
        TABLE ACCESS (BY ROWID) OF TCCARRUSE_198801
          INDEX (RANGE SCAN) TCCARRUSE_198801_IDX
      FILTER
        TABLE ACCESS (BY ROWID) OF TCCARRUSE_198802
          INDEX (RANGE SCAN) TCCARRUSE_198802_IDX
      FILTER
        TABLE ACCESS (BY ROWID) OF TCCARRUSE_198803
          INDEX (RANGE SCAN) TCCARRUSE_198803_IDX
      FILTER
        TABLE ACCESS (BY ROWID) OF TCCARRUSE_198804
          INDEX (RANGE SCAN) TCCARRUSE_198804_IDX
      TABLE ACCESS FULL TCCARRUSE_198805
      TABLE ACCESS FULL TCCARRUSE_198806
      TABLE ACCESS FULL TCCARRUSE_198807
      FILTER
        TABLE ACCESS (BY ROWID) OF TCCARRUSE_198808
          INDEX (RANGE SCAN) TCCARRUSE_198808_IDX
      FILTER
        TABLE ACCESS (BY ROWID) OF TCCARRUSE_198809

```

2) Insert, Delete, Update의 사용

현재까지는 Partition View에 직접적으로 Insert/Delete/Update를 할 수 없다.

(이는 B.O Version에 가능)

따라서 현재는 2 가지 방법으로 가능.

(1) DYNAMIC SQL을 이용하는 방법

(2) LOGIC적으로 IF-ELSE 문법을 사용하여 가능한 일이 날 수 있는 모든 경우를 기술하는 방법

(비고) (1)의 방법은 Application 작성이 간단하기는 하나 LOGIC만에서 수행되거나
민첩히 수행되는 경우에는 Parsing Overhead가 심각해질 정도로 증가 됨.

(2)의 방법은 (1)의 단점을 해소 할 수 있으나 Partition Table이 추가, 변경
점에 따라 주기적으로 APPLICATION이 같이 변경해 주어야 한다.

3) Partition View의 효율적인 사용 방법

(1) Parsing Time의 감소

Partition View는 많은 수의 Table을 UNION ALL View로 만든 것이므로
Query 수행 시 Parsing Time의 Overhead로 인해 성능 저하를 가져 올 수 있다.

이에 대해서 물리적인 방법(HOLD_CURSOR, CURSOR_SPACE_FOR_TIME-table 등)으로
Re-Parsing Time을 줄일 수 있으나 Partition View를 효율적으로 생성하므로써
Overhead를 줄일 수 있다.

(가) 수직적 분할

(사례)에서는 약 100개 정도의 Column에 대해서 "Select * "의 형태로
되어 있어 Data Dictionary Call이 그만큼 많아 진다. 그러나 Table의
속성을 보면 필수 NOT NULL Column(10개)을 제외한 80개의 Column이
선택적으로는 '신용판매', '현금서비스', '해피이용'의 3가지로 CODING
되는 것으로 PARTITION VIEW자료를 3개 정도로 분할(수직적인 분할)하고
사용하여 Overhead를 줄인다.

(나) 수평적 분할

가장 ACTIVE한 데이터를 정하여 수평적으로 분할하여 Partition View를
생성한다. (사례)에서는 3개월, 6개월, 8개월 단위의 Partition View를
만들어 사용하고 있다.

(2) Partition Column이 비교절에 나오지 않을때

이 때는 어쩔 수 없이 Partition Column이 사용 될 수 없으므로 일관적인
UNION-ALL View형태로 Access 될 수 밖에 없다.

그러나 SELECT 결과가 1건일 경우에는 ROWID 제한을 시켜 질 수 있는 한

TABLE ACCESS를 제한시킨다.

(UNION ALL의 순서를 역순으로 하여 VIEW를 생성)

```
CREATE OR REPLACE VIEW PU_TRACKTABLE
```

```
AS
```

```
SELECT * FROM TRACKTABLE_188808
```

```
UNION ALL
```

```
SELECT * FROM TRACKTABLE_188808
```

```
UNION ALL
```

```
SELECT * FROM TRACKTABLE_188807
```

```
UNION ALL
```

```
SELECT * FROM TRACKTABLE_188808
```

```
UNION ALL
```

```
SELECT * FROM TRACKTABLE_188805
```

```
UNION ALL
```

```
SELECT * FROM TRACKTABLE_188804
```

```
UNION ALL
```

```
SELECT * FROM TRACKTABLE_188803
```

```
UNION ALL
```

```
SELECT * FROM TRACKTABLE_188802
```

```
UNION ALL
```

```
SELECT * FROM TRACKTABLE_188801
```

```
UNION ALL
```

```
SELECT * FROM TRACKTABLE_188512
```

```
UNION ALL
```

```
SELECT * FROM TRACKTABLE_188511
```

```
UNION ALL
```

```
SELECT * FROM TRACKTABLE_188510
```

```
UNION ALL
```

```
SELECT * FROM TRACKTABLE_188508
```

```
UNION ALL
```

```
SELECT * FROM TRACKTABLE_188508
```

```
UNION ALL
```

```
SELECT * FROM TRACKTABLE_188507
```

```
SELECT * FROM PU_TRACKTABLE
```

```
WHERE USE_ID = '123456' --+
```

```
AND CARD_ID = '45681278182212' --> (UNIQUE)
```

```
AND ROWNUM = 1;
```

(삭제)에서 시간적으로 최근의 TABLE을 먼저 UNION ALL하여 사용하였다.

이는 원하는 정보가 삭제에 있을 확률이 높기때문에 먼저 삭제되면 더이상 TABLE의 ACCESS를 하지 않기 때문이다.

(3) HINT의 사용

Partition View에 Hint 사용은 보통의 Query에 대해서와 같이 사용한다.

(1) PARALLEL HINT

```
SELECT /*+ PARALLEL(PU_TABLES, 2) */
      SUM(AUTH_AMT)
FROM   PU_TABLES
WHERE  AUTH_DATE BETWEEN '19850501' AND '19850731'

TABLES_198505, TABLES_198506, TABLES_198507 Table를
Parallel 처리한다.
```

(2) INDEX_KEC HINT

이 INDEX_KEC HINT에 들어가야 하므로 원칙적으로는 불가능하지만
상품 질 Table에 명확하게 정의된다면 상품 할 수도 있다.

```
SELECT /*+ INDEX_KEC(a TABLE_1985_IDX1) */
      *
FROM   PU_TABLES a
WHERE  CARD_NO = '4589243141947231'
AND    AUTH_DATE BETWEEN '19850501' AND '19850531'
AND    BOUNUM = 1
```

(V7.3) PARTITION VIEWS IN RELEASE 7.3

Bulletin no : 10380

1. 소개

Partition view 는 Oracle Database 7.3 에서 대용량 테이블(예를 들면, data warehousing에서 주요 fact table)을 지원하기 위해서 추가된 새로운 방법이다. Partition View는 대용량 테이블에 대한 관리성 및 가용성을 향상시키고, 이들 테이블에 대한 질의 속도를 향상시킬 수 있다.

Partition view 의 예를 보면,

```
create view BALANCE as
    select * from jan_sales
    union all
    select * from feb_sales
    ...
    union all
    select * from dec_sales;
```

1월에서 12월까지의 sales table 들을 합한 뷰가 *BALANCE* 이다. 이들 월별 테이블들은 column name, data type, index 가 모두 같아야 한다. 또한 이들 월별 테이블들은 partition column 에 check constraint 가 반드시 존재해야 하는데, 위의 예를 통해서 보면 jan_sales 의 data column 의 값들은 1월 1일에서 31일 사이의 date 값이 가능하도록 check constraint 가 있어야 한다.

Base table, indexes, constraints 뿐 만 아니라 UNION-ALL view 의 정의는 DBA 에 의해 생성된다.

2. 관리성 및 가용성에 대한 장점

Partition view 는 대용량 테이블의 관리를 매우 쉽게 해 준다. 예를 들면, 위에서 정의한 *BALANCE* 라는 대용량 테이블을 살펴보자. 매월마다 DBA 는 새로운 달의 sales data 를 *BALANCE* 에 추가해야 한다. 따라서, DBA 는 *BALANCE* 대한 모든 index 들을 drop 하고, 새로운 테이블을 추가한 후에 index 를 재 생성해야 할 것이다. 더구나, *BALANCE* 가 큰 테이블이므로 이 작업 시간도 상당히 오래 걸릴 것이다. 또한, 이 작업 동안 *BALANCE* 데이터는 사용할 수 없게 된다.

그러나, Partition view 를 사용하면, 새로운 달의 데이터를 새로운 Partition을 만들어 추가하고, 이 새로운 Partition 에 index 를 생성하는 동안 현재 사용 중인 Partition 을 계속 사용할 수 있다. 따라서, 새로운 Partition 과 index 가 완성되었을 때, DBA 는 이 새로운 Partition 을 포함하는 UNION-all view 를 재 생성 하면 된다. 이 UNION-ALL view 를 재생성하는 짧은 시간 동안만 *BALANCE* 가 사용 불가능 하게 된다.

3. Partition view 의 성능 향상

Database 7.3 부터 제공하는 UNION-ALL view 를 사용하면, 단일 테이블 접근 보다 적어도 같거나 대부분의 경우 더 좋은 성능 향상을 얻을 수 있다. 단, 모든 Partition 들은 반드시 같은 column 에 check constraint 가 있어야 하고, column 정의와 index 가 같아야 성능 향상의 효과를 얻을 수 있다. Partition view 예는 다음 두 가지의 성능이 향상되었다.

① Partition 제거

⑧ UNION-ALL view 의 Parallel 수행

먼저, Partition 제거한 다음과 같다. 위에서 생성된 *BALB* partition view 에 대한 임의의 질의들은 일부의 Partition 들로부터 데이터를 가져온다. 예를 들어,

```
select sum(revenue) from BALB
where sales_date between *15-JAN-BB* and *15-FEB-BB*;
```

과 같은 질의는 Partition view 로 처리 될 경우, JAN partition view와 FEB partition view 만으로 산출된다. 즉, 나머지 10개의 Partition 은 접근하지 않는다. 이것을 Partition 제거라고 한다. Partition 제거는 Partition column 에 대한 질의에서만 사용될 수 있다. 위의 예에서는 sales_date column 이 partition column 이다. 이러한 제약 사항은 있지만, 속도 향상은 주목할 만 한다. 위의 예에서 보듯이 12개중 10개의 Partition 을 제외하여 8배의 성능 향상을 얻었다.

Partition view 를 사용 했을 때, 또 다른 장점은 UNION-ALL view 를 parallel 수행한다는 것이다. Parallel Query Option 이 설치된 경우, 모든 UNION-ALL view 에 대한 질의는 parallel 로 수행된다. 이것은 다른 정장 최상급의 parallel query 구조(이를 제공들은 데이터 partition 의 물리적 구조가 parallelism 을 결정한다)과는 다르게 parallelism degree 라는 독립적으로 수행된다는 점은 매우 특이하다. 오라클은 UNION-ALL view 의 데이터를 모든 parallel query processes 에게 동적으로 균등하게 배분하고, 위에서 언급한 par

* Partition view 의 제약 사항

Partition view 는 SQL 연산을 제공하지 않는다. 따라서, READ-ONLY application (data warehousing 등등) 에 적합하다.

(V7.3) STANDBY DATABASE IN ORACLE 7.3

Bulletin no : 10322

Standby database 에 대하여

1. 오라클 7.3 버전에 추가된 new feature 로써, archive mode 상태에서 online tablespace backup 이 필요없는 새로운 backup/recovery 방법이다. 즉, archive log mode 에서도 각 tablespace 의 주기적인 online backup 이 필요하였고, 이 작업에 따라 회복시간이 매우 컸었다. standby database 를 잘 유지하여 줄 수 있는 환경이라면, online tablespace backup 대신 유용하게 사용될수 있다.

2. 회복 시간은 작업량과 standby database 에 archive log file 가 어느 정도 잔여 되었느냐에 따라 다르다.

3. DFB 과의 비교

Instance 들 두개 이상 사용하는 Oracle Parallel Server와 비교하는 것 보다 위의 1번 에서 소개한 환경과 비교하는 것이 더 적절하다.

4. 생성 방법

4.1 create the standby database

: standby db 의 생성

1) back up the datafiles

2) startup mount;

alter database create standby controlfile as 'filname';

3) alter system archive log current;

4) transfer *.log, *.dbs to the safe place for standby database

4.2 maintain the standby database

: log file 들 standby db 에 적용시켜 최신의 데이터로 유지함.

1) startup mount;

2) alter database mount standby database;

3) recover standby database ;

4.3 activate the standby database

: primary database 가 정상적으로 startup 할 수 없을 때

1) alter database activate standby database;

!!! 일단 standby database 를 기동시켰으면, 이전의 primary database 상태로 되돌릴수 없으니 신중하게 사용하여야 함.

ARCHIVE LOG MODE 운영 방법

Bulletin no : 103B4

Oracle에서 Oracle Backup을 받거나 완전한 Recovery 작업을 수행하기 위해서는 DB를 Archive log mode로 운영하여야 한다.

Archive log mode로 운영하기 위해서는 다음과 같이 설정하여야 한다.

1. InitDB.ora 파일에서 다음의 parameter를 setting한다.

(1) LOG_ARCHIVE_START = TRUE

-> ARCH process 가 기동

-> log switch 발생시 automatic archive를 수행한다. 만약 이 parameter가 false이면 manual archive를 실시하여야 한다.

(2) LOG_ARCHIVE_DEST = /home/oracle7/dbs/archive_files/arc

-> archive장소의 디렉토리라 확장자를 포함하지 않는 파일명을 지정

-> 여기에서 archive_files라는 directory이며 마지막에 있는 arc는 archive log의 file initial명이다.

(3) LOG_ARCHIVE_FORMAT = %s.log

-> archive file의 확장자와 log sequence번호의 형식을 지정

-> 이는 (2)에서 정의된 archive log의 initial file명과 함께 나열한다.

[예] arc123.log, arc124.log (123과 124는 log sequence number 이다.)

InitDB.ora 파일에 위의 정보를 setting한후 save한다.

2. 다음과 같이 작업하여 archive log mode로 전환한다.

```
$ sqlplus lmsadm
```

```
SQL> connect internal
```

```
SQL> startup mount
```

-> (1)

```
SQL> alter database archivelog
```

-> (2)

```
SQL> archive log list
```

-> (3)

```
Database log mode
```

```
ARCHIVELOG
```

-> (4)

```
Automatic archival
```

```
ENABLED
```

-> (5)

```
Archive destination
```

```
?/dbs_arc/affiles_log/affiles
```

-> (6)

```
Oldest online log sequence
```

```
123
```

-> (7)

```
Current log sequence
```

```
125
```

-> (8)

```
SQL>alter database open;
```

-> (9)

(1) DB를 startup mount까지만 한다.

(2) 이 Command를 이용하여 archivelog mode로 DB를 변경한다.

(3) Archivelog mode로 변경되었는지를 확인한다.

(4) DB가 archivelog mode임을 나타낸다.

만약 NOARCHIVELOG로 되어있으면 연결되지 않은 것을 의미한다.

(5) initdb.ora 파일에서 LOG_ARCHIVE_LOG parameter를 TRUE로 정의하였음을 나타내며 false인 경우에는 DISABLE로 나타낸다.

(6) initdb.ora 파일의 LOG_ARCHIVE_DEST parameter에서 정의한 archive 할 장소이다.

(7) 3개의 redo log중 가장 오래된 redo log의 sequence가 123임을 의미한다.

(8) 현재 사용중인 redo log의 sequence가 125임을 의미한다. 만약 이전부터 archivelog mode로 운영 중이었다면 archivelog file은 log sequence 124까지 archivelog되어 있다는 것을 의미한다.

(9) Archive mode로 운영 가능하도록 연결 후 DB를 사용 가능하도록 open한다.

AUDITING 이란 무엇인가?

Bullietin no : 10188

1) 개 요

AUDITING 기능은 사용자의 행동을 감시하거나 데이터베이스에 관한 통계자료를 얻는 목적으로 사용된다. AUDITING 기능을 사용함으로써 누가 어떤 데이터를 언제 사용하고, 언제 어떤 작업을 하는지를 기록할 수 있다. 하지만 데포드에 대한 AUDITING 은 불가능하며 AUDITING 을 걸게 되면 시스템에 부하를 줄 수도 있기 때문에 자주 사용되는 기능은 아니다. AUDITING 에서 제외되지 않는 기능은 사용자가 트리거를 만들어서 구현을 해야 한다.

2) 종 류

AUDITING에는 3가지 종류가 있다.

STATEMENT AUDITING : 명령의 종류에 따라서 AUDITING 설정

EX)AUDIT TABLE BY SCOTT BY ACCESS WHENEVER SUCCESSFUL;

※ott 유저가 내리는 명령중 데이터에 관련된 명령(commit, rollback 등)이 성공한 경우 기록된다.

PRIVILEGE AUDITING : 사용되는 PRIVILEGE 에 따른 AUDITING 설정

EX)AUDIT CREATE TABLE BY SCOTT BY SESSION;

※ott 유저가 CREATE TABLE 권한이 필요한 명령(CREATE TABLE)을 내린 경우 접속된 세션에 대해서 한전만 기록된다.

OBJECT AUDITING : 적용되는 OBJECT 에 따른 AUDITING 설정

EX)AUDIT ALL ON SCOTT.EMP;

※ott.emp 데이터에 대한 모든 명령(SELECT, DELETE, DROP 등)에 관한 사항이 기록된다.

설정된 AUDIT 기능은 NOAUDIT 명령으로 열릴 수 있다.

EX)NOAUDIT ALL ON SCOTT.EMP;

3)설 정

AUDIT 기능은 init.ora 파일에 AUDIT_TRAIL 파라미터를 설정하고 DB를 shutdown/startup 하면 작동한다.

AUDIT_TRAIL=TRUE, AUDIT_TRAIL=DB 로 설정이 되어 있으면 결과는BYB.AUD\$ 데이터에 쌓이게 되고 AUDIT_TRAIL=DB 로 설정하면 결과가 파일로쌓이게 된다.

BYB.AUD\$ 데이터에서 얻을 수 있는 정보는 매우 많은데 대표적인 것으로 다음을 들 수 있다.

USER NAME : 오라클 유저 이름

SESSION ID : 오라클 세션 ID

TERMINAL ID : 유닉스 시스템 상의 터미널 ID
OBJECT NAME : 테이블, 뷰 등의 이름
ACTION ID : 명령 코드
DATE : 명령이 실행된 날짜, 시간
COMPLETION CODE : 0 이면 성공, 실패하면 에러코드

ACTION ID 는 사용된 명령을 나타내는데 이것은 숫자로 기록이 되어 있으므로 AUDIT_ACTIONS 디서너미를 통하여 실제의 명령으로 변환해 볼 수 있다.

4) 특 징

- * 내려진 명령이 ROLLBACK 되더라도 AUDIT 에 의해 기록된 정보는 그대로 남아 있다.
- * BYB 또는 INTERNAL 로 접속하여 행하는 명령은 AUDIT 에 의해서 기록되지 않는다.
- * 오라클은 AUDIT 기능의 세팅 여부라는 관계없이 STARTUP, SHUTDOWN, CONNECT INTERNAL 과 같은 정보는 DB 상의 확립로 기록을 남겨준다.
- * REMOTE DB 에 대해서 행해지는 작업은 AUDIT 대상이 되지 않는다. 이것은 REMOTE DB 에서 처리해 주어야 한다.
- * 만약 두개의 AUDIT 가 동시에 적용되는 경우라면 하나의 레코드만 생성된다. 예를 들어 TABLE 과 CREATE TABLE 이 적용되는 경우이다.
- * OBJECT AUDITING 은 모든 용저에 대해서 적용된다. 특정한 용저를 지정하지 못한다.
- * BY SESSION 은 접속된 세션에 대해서 하나의 레코드만 생성하고 BY ACCESS 는 해당 명령이 내려질 때 마다 정보를 기록한다.
- * DBL 들 AUDIT 하는 SYSTEM AUDIT, PRIVILEGE AUDIT 의 경우는 BY ACCESS 가 디폴트 이고 그외의 경우는 BY SESSION 이 디폴트이다.

BITMAP INDEX 생성과 SPACE 비교

Bulletin no : 1031B

. Bitmap index 의 생성 방법은 다음과 같다.

```
SQL> create bitmap index id_dept on dept (dname);
```

. Explain plan 의 결과

```
select /*+ index( dept ID_DEPT ) */ *
from dept where dname = 'sales'
```

SQL> Explain Plan

```
0 SELECT STATEMENT  cost: 1000000
1 TABLE ACCESS (BY ROWID) OF 'DEPT'
1 BITMAP CONVERSION (TO ROWIDS)
2 BITMAP INDEX (SINGLE VALUE) OF 'ID_DEPT'
```

. Index space 이용도 비교

. bitmap index 도 B-tree index 과 같이 where 절에 주로 나타나는 column 들 지정하여 생성한다.
 . single bit 에대한 lock 강판이 없어 row level lock 대신 block level lock 이 적용되므로 OLTP 전용 업무에서는 불리하다.
 . bitmap index 는 많은 DML load 가 있는 table 은 불리하다

Indexed column	Cardinality of column indexed	Index Size (in MB)	
		conventional B-tree index	bitmap index
EMPID	1000000	14.33	23.80
EMPJOB	500000	15.28	12.36
EMPDEPT	250000	15.27	8.48
EMPNAME	100000	15.21	5.25
EMPSSN	40000	15.08	3.75
EMPDO	10000	14.34	2.88
EMPIN	1000	14.28	2.25
EMPID	100	13.40	1.38
EMPID	25	13.40	1.31
EMPID	10	13.40	1.11
EMPID	5	13.40	0.78
EMPID	4	13.40	0.87
EMPID	2	13.40	0.37

BITMAPPED INDEX

Bulletin no : 10208

최근에 각광받고 있는 DatabaseIndexing환경 과 End-User-Computing환경등에서 중요한 RDBMS의 첨단 Indexing Access기법으로 경쟁사에서는 Bitmap Index를 발표하고 있습니다. 이 기법은 Oracle Server 97.3에서도 Bitmapmed Index라는 이름으로 발표될 예정이므로 이에 대한 정확한 이해를 위해 다음사항을 기술해 보았습니다.

1. Bitmapmed Index란 ?

RDBMS의 Table로부터 특정 자료의 검색을 위해 기존의 일반 RDBMS에서는검색효율의 향상을 위해 B-Tree Index를 구현하여 사용하여 왔습니다. 그외에도 B-tree Cluster Index, Hash Cluster Index등을 이용하여 대부분의 검색효율을 보장하여 왔지만 B-Tree Access를 위한 특성으로 인해 몇가지 어떠한 문제를 만들고 있었습니다.

1) B-Tree Index Access 방식을 위해선 십지 조건비교되는 Column값에 대한Table의 원시값을 Index에도 보관하고 있어야 함으로 인해 Index를 위한 십지 data의 중복 저장으로 저장공간의 낭비를 감수하여야 하며, 특히 현실 상황에서 자주 사용게 되는 여러 컬럼의 Concatenated Index (결합 인덱스) 에서 자료량이 많은 Table에 대해서는 크나큰 부담이었습니다.

2) Index설정 Column값의 분포도(Unique성)가 넓은 Column에 대한 (예:성별)조건검색시에는 Index access를 했을때 보다 오히려 Full Table Scan방식이 오히려 빠른 성능을 보장하기 때문에 RDBMS의 Cost Based Optimizer는 Bitmap access비용이 많이드는 Index access를 포기하고 빠른속도의 순차 Table Scan을 하여 높은 Hit Ratio를 보장하여 하는 경우를 우리는 흔히 보게됩니다.

3) End-User-Computing 환경에서 흔히 예상되는 복잡한 질의조건으로 인해 Optimizer가 최적 실행계획에서 Index를 포기하게되는 경우를 초래합니다. (예: 복잡한 DB 연관자등)

이러한 상황을 해결하기 위한 새로운 Index Access 방법 로 Bitmapmed Index가 새롭게 등장하게 되었습니다.

다음 상황을 통하여 구체적 설명을 제시합니다.

예) 한국모닥솔(주) 에서 관리하고 있는 고객정보 Table이 다음과 같이 정의 되어있다고 가정합니다.

고객번호	혼인여부	성별	종로등급
1001	결혼	여	하
1002	미혼	남	상
1003	재혼	남	중

상기 Column들의 속성을 보면 고객번호는 동일한 값들이며 B-Tree Index로 막대한 조처성능 보장. 혼인여부(3종류),성별(2종류),종로등급(3종류) 은 30 - 50%의 넓은 분포도를 갖는 값들로서 B-Tree Index사용시 효율성이 거의 없음.

즉, B-Tree Index에서는 자료의 값에 따라 Sort하여 이분식(Binary) 가지치기(Leaf & Node) 구조를 유지하여, 조건에 들어오는 값에따라 검색자료 범위(Access Range)를 조개어 나가는데 만해, 저장의

어 있는 자료가 동등(같은값)이 많다면 해야할일(Access Range)이 줄지 않고 비용이 많이드는 Random Access(Index에서 Block을 가지고 실행 Table Data를 찾을때)만 많아지고, B+Tree해야할 자료가지(Leaf) 수는 쉽게 줄지않게되어 B-Tree Index방식으로는 빠른 성능을 보장하기가 어렵습니다.

그렇다면 다음과 같이 분포도가 좋지않은 중요등급 Column을 몇 개의 Bit를만 가지고 정보를 저장키로 하여봅시다.

중요등급 = '상'	1	0	0	0	0	0
중요등급 = '중'	0	1	0	0	1	1
중요등급 = '하'	0	0	1	1	0	0

*존민여부(3종류), 성별(2종류)도 같이 적용하였다고 가정.

이런 상황에서 아래의 SQL을 실행한다고 가정하여 봅시다.

```
Select count(*) from 고객정보
where 존민여부 = '미혼' and 중요도 in ('상','중');
('미혼자 인 고객중에 중요도가 '상' 이거나 '중'인 고객은 몇 ?')
```

이 SQL을 실행하기 위한 Optimizer가 이러한 Operation을 다음과 같이 결정한다면 훨씬 빠른속도의 성능을 낼 수 있을 것 입니다.

존민여부 = '미혼' AND (중요도 = '상' OR 중요 = '중')
 즉, '011001' AND ('100000' OR '010011')
 다시 물어보면 '011001' AND '110011' 이고 결과는 '010001'이다.
 Optimizer는 두개 컬럼의 논미연산결과가 '010001'인 Row들을 찾아 전수를 세면 될 것 입니다.

상기와 같이 논미연산과 Bit처리 방식은 Computer에게 훨씬 쉽게 처리할 수 있는 길을 만들어주게 되면서, 몇종류 만되는 값들과의 점도의 조그만 연결표(Bitmap)와 몇Bit만의 저장공간만을 가지고 Index 구성을 가능케 함으로써 공간의 절약효과도 상당할 것이다. 특히, Data warehousing, Decision Support System 에서와 같은 광대한 정보량에서 통용할 수 있습니다.

2. Bitmapped Index의 장단점 ?

Index를 사용하는데 있어서 우리는 몇가지의 고려사항을 검토해야 합니다. 조회성능측면, 저장공간측면, 유지관리측면 정도를 고려할 수 있습니다. 앞으로, 이러한 측면에서의 Bitmapped Index특성을 설명하기로하겠습니다. 우선, Bitmapped Index의 장,단점을 간추려보면 다음과 같습니다. 장점으로는 아주적은 Index저장공간을 사용하여 좀 많은 분포도의 값을 갖는 다량의 자료를 빠른속도로 Access 할 수 있다는 것과 특장다양한 조건에 대해 Index Access Path를 적용할 수 있다는 점이 있는 반면, 조건 유형이 Pattern Match형태가 자주 사용될시 효율성이 극소화되며, B-Tree Index와 같이 모든 Query에 대해 Index Path로 사용될 수 없다는 단점이 있습니다. 한예로 Insert, Update, Delete와 같은 Query에서는 주의미 합니다.
 Bitmapped Index는 다음과 같은 측면에서 각각 통용한 이점이 있습니다.

조회성 측면

- 각각의 독립적인 Column들에 대하여 여러유형 (AND,OR,...)의 조건절에 대해 Index가 사용 되어지게 하는 공식이 불필요하다. (and how query)

- 분포도가 나쁜 값에 대한 Index Access가 빠름.
- Index Column으로 추가 필요시 독립적으로 추가 및 적용 가능.
- 복잡하게 길어지는 조건절에서도 모두 용이하게 작동된다. (복잡한 질의 및 ad hoc query에서 유용)
- 특히 다량의 자료에 대한 계 질의(aggregate query)에서 탁월. (예:COUNT operator)
- 분포도가 좋은 (Unique성:값의종류가 많다) 값에대한 Index는 불리.

저장공간 측면

- Index에 가지고 있어야 할 자료값에대한 공간 절약.
하지만, Bitmapped Index로 적용하게될 Column의 특성상 실제 값의 Bize도 크지않다 예를들어 '남' 과 '여'값의 경우 2Byte 이하일 것이다. 즉, Bitmapped Index적용 Column의 후보는 대개 5가지 정도 이내의 값을 갖는 경우가 되므로 실제 값의 Bize도 작게됨.고로, 이러한 측면에서의 저장공간 절약 측면 보다는 실제 현업상 요구되는 Index구성에서 예를 들면 더 나은 이해가 될 것 입니다.
예) 기존에 고객정보 검색속도를 위해 Index를 다음과 같이 구성 하였다고 가정 하겠습니다.

B-Tem Index1 = (혼민여부,성별,종교도)

이경우는 Where절에서 혼민여부가 조건에 오지않는 경우나 조건절에 성별만 혹은 종교도만을 가지고 검색할 경우에는 Index1을 사용할 수 없기에 Index2(성별,종교도,혼민여부), Index3(종교도,성별,혼민여부)등의 추가 Index가 있어야 만족한 성능을 낼 수 있었다면 거의 10 - 100배의 공간절약을 하며 Bitmapped Index1(혼민여부), Index2(성별), Index3 (종교도)등의 독립적인 3개의 적은공간으로 만족한 성능을 보장할 수 있다. 이러한 측면에서의 이점이 클 것 입니다.

동지완리 측면

- Bitmapped Index는 Partition Support System과 같은 조치전용 업무나 OLTP업무 비중이 작은 업무에 서 적합.
- 아직 Single Block에 대한 Lock방안이 없고, Bitmapped Index에서 Row-Level Locking대신에 Block-Level Locking이 적용되기 때문에 OLTP전용 업무에서는 Lock Contention 및 Deadlock 가능성 많음.
- Bitmapped Index에서 Update등의 Transaction은 Block level Lock를 사용하여야 하므로 Oracle의 기존Locking 인 Row level Lock 사용할 수 없게 되며 결국 Oracle block (예:2KB)내의 한Row에 대한 전경이 필요시(Update등)에는 해당 Block전체가 Locking되므로 같은 전경이 예상되는 OLTP업무에서는 큰부담이 되므로 Batch성 Bulk Operation 이 가미되어 운용할 수 있는 방안이 필요.
예) Data warehousing에서는 주업무가 조회이며, 자료전경 추가시는 대부분 Batch성으로 처리하는 사례에서 Batch작업시 Bitmapped Index들을 Disable시키고 처리후 Enable하 것이 바람직.

3.Oracle's Bitmapped Index의 특성 ?

Oracle에서의 Bitmapped Index는 다음의 이점을 제공합니다.

접속권 기술

- 이미 Oracle7의 Text Server에서 Bitmapped Technology를 적용하여 사용하여 왔으며 이러한 기술을 Oracle7 Release7.3에서 Production 으로 제공합니다.

통합기능으로 제공

- 별도의 기능옵션 추가없이 Oracle Server에 통합되어 제공됩니다.
- 기존 사용 모든 환경과 수정없이 통합 적용이 가능합니다.
(예: DB Trigger, Distributed, Parallel, Integrity constraints...)
- 각종 SQL, Text 및 Utility, Application등에서 수정이 필요없습니다.

점점 Index 생성 지원

- 광대한 자료에 대한 Index생성시 점점수행으로 Bitmapped Index생성

압축기능

- 저장공간을 현저히 줄일 수 있게 구현된 압축기능을 제공합니다.

삽입/수정/삭제 지원

- Oracle의 Bitmapped Index는 Insert, Update, Delete 를 지원하지 않습니다.
(은행 계정입주 와 같은 주미한 OLTP transaction이 아닌 하루입주 중에 약 10% 이내의 자료전경
입주는 그대로 사용 가능)
- 사용자료 하여금 가끔 용탈되는 전경에 대해서 Bitmapped Index를 재생성이 불필요하게 투명하게
사용가능.

범접 Index 검색 지원

- 대량의 Table에 대한 검색시 범접로 Bitmapped Index 검색 가능.

여러컬럼에 대한 Bitmapped Index 지원

- 분포도가 나쁜 몇개의 컬럼이 항상 같이 조건에 오는 경우 B-Tree Index에서의 Concatenated Index
같이 Multicolumn Bitmapped Index 사용가능.

동연성 보장

- 하나의 Table에 B-tree Index와 Bitmapped Index를 동시에 혼용가능.
(Clustered Index도 혼용 생성 가능)

Oracle에서의 테스트 예제

Oracle에서의 Bitmapped Index의 특성을 확인할 수 있는 예시 입니다.

고객정보자료는 약100만건의 자료로 구성되어 있으며 현재 고객번호는 유일한 값으로 구성되어
Primary Key로 선언되어 있습니다. 그러나, 성명 과 종모도, 혼인여부 컬럼의 값은 2가지 내지는 3가
지 종류이어서 B-Tree Index로는 검색수행 속도를 보장할 수 없는 상황임.

1) Select * from 고객정보 where 성명 = '남':

은 BQL을 Optimizer는 최적의 Execution Plan으로 Parallel Full
Table Scan을 선택하게 됩니다. (전체의 50%이므로)

2) Select * from 고객정보 where 고객번호 = 1001:

은 BQL을 Optimizer는 최적의 Execution Plan으로 Unique B-tree
Index를 Access 하여 Rowid를 가지고 Table 을 1번 Scan.

3) Select * from 고객정보

where 성명 = '남' and 종모도 in ('상', '중');

은 BQL을 Optimizer는 최적의 Execution Plan으로 Bitmapped Index
를 선택하게 됩니다.

4) Select * from 고객정보

where 성명 = '남' and 고객번호 = 1001;

은 BQL을 Optimizer는 최적의 Execution Plan으로 B-tree Index를
Range Scan하는 것 을 선택하게 됩니다.

(성명 Bitmapped Index 보다는 Unique한 고객번호 Index가 효율적)

상기과같이 Oracle 에서의 Bitmapped Index는 Cost Based Optimizer에 의해 투명하고 동연성있게 수

행속도를 보장하게 되며, 필요시 Oracle의 특징인 Optimizer 힌트 선택기능으로 사용의도에 맞추어 사용이 가능합니다.

결 론

Data Warehousing 또는 Decision Support System에서의 중요한 요소인 Oracle의 Bitmapmed Index는 Oracle7.3 Server에 탑재되지 않았으며 기존 DBMS에서의 조회 수행속도의 몇가지 결함들을 해결해 주었고, 또한 광대한 자료량에 대한 Index의 저장공간 절약 및 End-user-computing에 꼭 필요한 Indexing 구현기법입니다.

단, Bitmapmed Index만이 지상 최대의 해결이 아니란 것도 우리는 말아야 하며 사전에 OLTP에서의 부담감에 대한 분석이 필요 하며, 과연 나쁜분포도에서 저장공간에 대한 낭비에는 많은 관심이 없고 단지 좋은 조회속도를 보장키 위한다면 이미 기존 Oracle7에서부터 사용해온 Clustered Index를 검토해보는 것이 바람직하다고 사료됩니다.

CONNET INTERNAL 에 대해서

PASSWORD 설정하는 방법

Bulletin no : 10240

connect internal 시에 password 를 입력하도록 하려면 다음과 같이 하면 됩니다. BB Extender와 같은 Client Tool을 이용하여 서버에 접속하는 경우에도 마찬가지로 적용됩니다.

1)orapwd를 이용해서 password file을 만들어 줍니다.

```
orapwd file=orapwDBA7 password=manager entries=1
```

이 파일은 \$ORACLE_HOME/dbs 디렉토리에 만들어 주며 ORACLE SID가 DBA7인경우 파일 이름은 orapwDBA7 으로 해 줍니다

2)initDBA7.ora 파일에서 다음을 추가합니다.

```
remote_login_passwordfile=exclusive  
compatible=7.1 또는 compatible=7.2
```

이렇게만 해 주면 connect internal 시의 password가 manager로 설정됩니다.

이것은 Bus Bosonic 2.4, Oracle 7.2.2 에서 테스트 해 본 것인데 다른 기종에서는 방법이 다를 수도 있습니다. 자세한 방법은

Oracle7 Server Documentation Addendum(Bosonic 7.1) Chap.5

를 참조하면 됩니다.

BB Extender에서 접속할 때 DBA-BB10 에러가 발생하는 경우도 똑같이 이 방법으로 password file을 만들어 주면 됩니다. 단, 이 경우에는 initDBA7.ora 파일에 remote_login_passwordfile 을 설정할 필요까지는 없고 compatible 만 설정하고 password file을 만들어 주면 됩니다.

CONTEXT SIZE & CURSORS

(ORA-1046,1050,1051)

Bulletin no : 10209

1. Context size 에 관련한 error message

```
.ora-1046 can't acquire space to extend context area.  
.ora-1050 can't acquire space to open context area.  
.ora-1051 maximum context area extent exceeded.
```

2. Context size란 무엇인가?

(1) 쉽게 말하면 Cursor의 initial size이다 . 즉, Cursor 에 allocate 되는 user memory 이다.
(2) 이는 init.ora 의 CONTEXT_SIZE 에 의해 결정된다.
(3) Cursor 에 할당되는 additional space 는 CONTEXT_INCB 에 의하며 50 extent 를 갖는다.
(4) Recommended context size increment 는 4096 bytes(4K) 이다.
(5) SQL statement가 수행시마다 cursor 가 open 되며, 같은 cursor가 reuse 되도록 design 되어
SQL*PLUS session은 2-3 개 이상 open 되어지지 않는다 그러나 SQL*PLUS 는 여러 다른 task 를 수행
하므로 많은 cursor를 open한다 . (100 or more)

(B)Cursor 가 hold 하는 item
* the SQL statement
* the parsed SQL statement
* one row of the result

3. ORA-1051 은 무엇이 문제인가?

(1) cursor 의 size 를 줄인다
(2) CONTEXT_SIZE,CONTEXT_INCB 를 늘린다.

4. OPEN_CURSORS 수를 줄이는 전략

(1) Commit을 자주한다.
(2) Synonym이나 view 를 사용하지 않음으로써 implicit cursor 수를 줄인다.
(3) SQL*PLUS 에서 select 문대신 COPY 로 바꿔 사용한다.
(4) SQL*PLUS 에서 large form을 여러개의 작은 form 로 나눈다
(5) ASAP, EXEC SQL CLOSE CL: 을 수행한다.
(6) HOLD_CURSOR=NO & RELEASE_CURSOR=YES 를 사용한다.

CONTROLFILE 재생성하는 방법

Bulldozer no : 10088

1. CONTROL FILE 은 다음 경우에 다시생성한다.

- CONTROL FILE 이 손상되었으나 이용가능한 BACKUP FILE 이 없을 경우
- 데이터베이스 이름을 변경 할 경우
- DATA FILE , LOG FILE 의 NAME 갯수를 확장할때

CONTROL FILE 을 다시 생성하는경우 ONLINE REDO LOGFILE 및 DATA FILE 이 손상될 경우가 있으므로 CONTROL FILE 을 생성하기전에 모든 데이터베이스의 FULLBACKUP 을 실행한다.

CONTROL FILE 을 재 생성 하기 위해서는 DATABASE 에 관계되는 모든 FILE NAME 을 알아야한다. 다음 SQL 문을 SQL 구조가 변경될때 즉 DATA FILE을 추가하거나 DROP 할때 실행하여 DATA FILE 과 LOG FILE 이름을 항상 알아둔다.

```
BQLEDBA > SELECT * FROM V$DBFILE;
```

```
FILE# NAME
```

```
-----
1 /u01/oracle/dbs/redo01a7.dbf
2 /u01/oracle/dbs/redo02a7.dbf
3 /u01/oracle/dbs/temp01a7.dbf
4 /u01/oracle/dbs/u01ora7.dbf
```

```
BQLEDBA > SELECT * FROM V$LOGFILE
```

```
GROUP# STATUS MEMBERS
```

```
-----
1 /u01/oracle/dbs/log101a7.dbf
2 /u01/oracle/dbs/log201a7.dbf
```

2. CONTROL FILE 생성 절차

1) DATABASE 를 SHUTDOWN 한다.

```
BQLEDBA > SHUTDOWN;
```

2) 모든 DATABASE FILE 을 BACKUP 받는다. (DATA FILE, ONLINE REDO LOG FILE)

3) INSTANCE 를 START 한다.

```
BQLEDBA > STARTUP NOMOUNT
```

4) CONTROL FILE 을 생성한다.

```
SQL> CREATE CONTROLFILE (RESET )
[SET] DATABASE DATA
LOGFILE '/usr/oracle/disk/log100007.dbf',
        '/usr/oracle/disk/log20007.dbf',
        '/usr/oracle/disk/log30007.dbf'
NORESETLOGS ( RESETLOGS )
DATAFILE
        '/usr/oracle/disk/data0007.dbf',
        '/usr/oracle/disk/data0007.dbf',
        '/usr/oracle/disk/temp0007.dbf',
        '/usr/oracle/disk/undo0007.dbf',
        '/usr/oracle/disk/data0007.dbf'
MAXLOGFILES 10
MAXDATAFILES 50;
```

- [RESET] : 기존에 CONTROL FILE 이 존재하는 경우에 사용. 만약하면
?/disk 디렉토리에 CONTROL FILE 이 있어야 한다.
- [SET] : 데이터베이스 이름들 변경할때 용

5) ?/disk/init0007 혹은 ?/disk/config.ora (DATABASE 의 경우) FILE 내의 CONTROL_FILES PARAMETER 에 새로 생성된 CONTROL FILE 을 추가한다.

```
CONTROL_FILES = ('/usr/oracle/disk/control0007.ctl',
                  '/usr/oracle/disk/control20007.ctl')
```

B) 데이터베이스를 MOUNT, RECOVER, OPEN 한다.

```
SQL> SHUTDOWN
SQL> STARTUP MOUNT
```

[NORESETLOGS] OPTION 을 사용한경우

```
SQL> RECOVER DATABASE
Media recovery complete.
SQL> ALTER DATABASE OPEN;
```

[RESETLOGS] OPTION 을 사용한경우

```
SQL> RECOVER DATABASE
USING BACKUP CONTROLFILE
UNTIL CANCEL;
```

Incomplete cancel-based recovery 을 실행한다.

```
SQL> ALTER DATABASE OPEN RESETLOGS;
Statement processed
```

CORE DUMP 에 대한 설명

Bulletin no : 10311

1. 서론

여기서는 오라클 product 가 발생하는 core dump 로 부터 stack trace 를 모르는방법을 설명 하고자 한다.

1) 'core dump' 는 무엇인가?

core dump 는 process 들의 image copy 이다. 이것은 file 의 형태로 구성되어 현재 수행중인 directory 에 발생한다

2) 발생하는 원인

원인은 여러가지가 있을 수 있으나 대체적으로 임의의 process 가 DB가 원하지 않는 일을 수행하려 할때 발생한다.

공통적으로

- . program 이 자신의 메모리의 memory 를 access 하고자 시도할 때 .
- . program 이 고갈되었거나 사용이 불가능한 resource 얻고자 할 때.
- . illegal instructions 에 의해 주연가가수행되려 할 때 .

Unix system 에서는 process의 불합리한 행위를 감하고자 signal 을 보내는데 이것은 user가 이러한 signal 을 manual 하게 보냄으로써 발생시킬수도 있다.

2. 처리 방안 ?

1) 우선 각 product 의 full version 을 check 하여야 한다.

ORACLE, PL/SQL , 각 연관된 product 의 version 을 check 한다. - Eg: Forms and Menu 가 연결되었다면 BQLMenu version 도 check 한다.

2) 수행하고자 한 정확한 command 을 check 한다.

Eg: 만약 BQLMenu 문제이면 'runform30' 으로 note 한다.

3) 다음을 순서대로 수행한다.

(1) 해당 command 을 수행했던 directory 도 ORACLE_HOME/bin 또는 \$ORACLE_HOME/bin/core_HMMN 에서 'core' file을 check 한다.

(2) ORACLE id 로 login 을 해서 \$ORACLE_HOME/bin directory 로 간다. 그리고 다음 command를 수행한다.

```
$ file program
```

(여기에서 program 은 core dump 를 발생시킨 command이다)

만약 이 command의 결과가 'dynamic' 또는 'dynamically linked' 이면 해당 executable을 relink 후core를 다시 생성한다 .

오래된 dynamic link 를 support 하지 않기 때문이다.

(3) \$ chmod to program (read permission 부여)

(4) Logout 후 server 가 말했던 user로 login 한 후 다음 command를 실행한다.

이 command는 UNIX version 에 dependent 하다

Command	OS	Exit command	Stack Trace command
dhx		quit	where
xdk	(HP ONLY)	quit	t
xdk		q	t
xdk		\$x (or Ctrl-D)	\$c

무선 해당 directory 로 이동 후

```
DBK: $ script /tmp/bugtrack
$ dhx $DBHOME/bin/program core
      (dhx) where
      ...
      (dhx) quit
```

\$ exit

```
ADB: $ script /tmp/bugtrack
$ xdk $DBHOME/bin/program core
      (xdk) t
      ...
      (xdk) quit
```

\$ exit

```
BBB: $ script /tmp/bugtrack
$ xdk $DBHOME/bin/program core
      (xdk) t
      ...
      (xdk) q
```

\$ exit

```
ADB: $ script /tmp/bugtrack
$ xdk $DBHOME/bin/program core
$c    << NB: xdk has no prompt so just enter $c
```

...

\$x

\$ exit

...

\$x

\$ exit

(5) 만약 debug command 가 실패하면 위 list 의 다른 debugger 를 사용해 다시 실행함.

만약 모든 debugger 가 실패하면 file type 의 permission 에 문제가 있는경우 이다.

system에 따라 만들어지는 core file의 maximum size를 control 하는 kernel parameter가 있는데 값이 너무 작으면 core file이 쓸모가 없고 다시 재현하는 것이 불가능할 수 있다.

이는 다음 command로 check해볼수 있다.

```
llimit          in the C shell
DB ulimit -a    in the Bourne / Korn shells.
```

5) `mystack`이라는 trace file이 생성되면 적당한 exit command를 실행한다

Stacked trace file를 print해서 분석에 reporting한다.

CORRUPTED BLOCK을 가진 OBJECT 찾는 법

Bulletin no : 1005B

DNA-157B : FILE ID # B, BLOCK # B2 라는 BLOCK 가 나올 경우 해당하는 OBJECT를 찾아야 한다.
다음과 같은 SQL문으로 해당 OBJECT를 찾아서 RECOVERY 를 실행한다.

```
$ sql> sql> connect internal
sql> select * from dba_extents
      where file_id = B and
      B2 between block_id and block_id + blocks -1;
```

COST BASED OPTIMIZER 와 RULE BASED OPTIMIZER

Bulletin no : 10238

Prior to 07.3 the CBO still behaves somewhat erratically by making poor execution plan choices and actually slowing down the query compared to the RBO implementation. The use of SQL HINTS is strongly encouraged to help the CBO make "more informed" decisions about the choice of execution plans. The general reaction of the audience was that CBO "has a long way to go" before becoming a mature, reliable product. I would recommend that you suggest to your customer that they approach the CBO carefully by setting up various tests to compare RBO and CBO performance. You should encourage the customer to begin experimenting with CBO and tell them that by the time they upgrade to 07.3 the product will have matured into a dependable tool that will produce quality execution plans. In my opinion, some of the negative feedback from customers using CBO occurred because they did not take the time to learn how it works and how best to use it.

1.) The RBO will be supported in Oracle8, but will not be enhanced and will eventually be deprecated.

2.) PL/SQL V1 (which is the PL/SQL engine linked with most tools) is basically an Oracle 8 product and does not contain all the logic to support CBO. Only PL/SQL V2 understands the CBO concept of HINTs.

3.) The CBO does not understand the characteristics associated with an application, nor can it fully understand the impact of complex relationships between joined tables (the cost of the analysis would far outweigh the benefit). Therefore, hints may be required to ensure a specific behavior of the CBO.

4.) The CBO is still relatively immature and (under some conditions) improperly estimates the cost of some query components.

5.) The CBO assumes evenly distributed values and therefore does not always estimate selectivity correctly. In the case of bind variables, selectivity is assumed to be some fixed value (maybe 25%). Oracle 7.3 will address some of this by utilizing column histograms.

6.) Use of the LIKE predicate sometimes results in unexpected execution plans. In particular, the CBO does not propagate LIKE-predicate transitivity.

7.) Application queries should be tuned and monitored for performance degradation, regardless of the optimizer type being used.

8.) All things being equal, the CBO will select a driving table working from left-to-right in the FROM list. This is just the opposite of the RBO behavior.

8.) By default, the CBO optimizes for throughput (overall resource consumption), NOT response time. Response time should not be confused with throughput.

10.) Generally speaking, interactive/Form-based applications with casual queries should be tuned for response time (OPTIMIZED GOAL=FIRST_ROWS).

11.) Observation of CBO behavior suggests that SORT-MERGE Joins are often favored over NESTED-LOOP Joins. This has a negative impact on the response time for the first row(s).

12.) Accurate statistics for all tables & indexes are a MUST for proper behavior of the CBO. (The CBO will make hard-coded assumptions for statistics that are not provided).

13.) If OPTIMIZED_MODE is set to CHOOSE, the CBO will be used if ANY of the tables in the query has statistics. The CBO will optimize for throughput, NOT response time.

14.) If OPTIMIZED_MODE is set to RULE, the CBO will be used regardless of the presence of statistics.

15.) If OPTIMIZED_MODE is set to FIRST_ROWS or ALL_ROWS (or if the session OPTIMIZED_GOAL is set to FIRST_ROWS or ALL_ROWS), then the CBO will be used regardless of the presence of statistics. FIRST_ROWS is optimized for response time, and ALL_ROWS has a goal of optimizing throughput.

16.) Whenever possible statistics should be updated by using the ANALYZE...COMPUTE command (the ESTIMATE option had some problems in earlier releases of Oracle7).

17.) Performing an ANALYZE operation on a table automatically cascades to all related indexes.

18.) If tables are too large to allow a full ANALYZE... COMPUTE, statistics should be updated with ANALYZE... ESTIMATE using at least 20-25% of the table. A full ANALYZE... COMPUTE should be performed on the INDEXES following the ANALYZE... ESTIMATE operation on the table.

19.) The ANALYZE command should be performed regularly on dynamic tables. The frequency of ANALYZE operations should be commensurate with the dynamics of the table.

20.) When formatting trace files using TEXPRT, it is important to understand that optimally selected EXECUTION PLANS are derived using the current optimizer environment (the plans are NOT stored in the trace file). Settings such as OPTIMIZED_MODE should be consistent with the settings at the time the trace file was created.

21.) For complicated multi-table joins, the number of execution plan permutations can be very large. Therefore, though Oracle documentation might suggest otherwise, the CBO uses heuristics to estimate certain costs and to prune execution plan options. (The order of the tables in the FROM clause can therefore have an impact on the final execution plan selected).

22.) PL/SQL blocks executed from SQLPLUS and PL/SQL stored procedures use an optimizer goal of CHOOSE when the session has been altered to a goal of FIRST_ROWS. I assume this is a bug. This prevents optimization for time unless you provide a specific HINT.

23.) There are some known bugs associated with the optimizer goal of FIRST_ROWS and queries that have ORDED BY — in some cases the optimizer may not use an available index in lieu of a SORT operation. (see bugs 211500, 247531)

24.) Since Oracle Forms uses the PL/SQL v1 engine, HINTs are stripped from the SQL statement. There is a block property to supply a HINT. Additionally, the session may be altered to optimize for response time (FIRST_ROWS goal for all queries) by invoking the FORMS_DLL built-in: FORMS_DLL(alter session set optimizer_goal=FIRST_ROWS).

25.) Optimizer Goal can be set at an instance level in both Oracle 7.0.1B and 7.1.4. Though the feature is not documented in 7.0.1B, the parameter OPTIMIZER_MODE=FIRST_ROWS can be used (but not officially supported).

26.) There is a bug associated with the ANALYZE ... ESTIMATE option that sometimes results in division by zero errors. It appears that this problem is most likely encountered when estimating statistics on smaller tables.

27.) There is a bug in pre-7.1 releases of Oracle7 that can cause the CBO to return no rows, but the RBO will correctly return a row. The bug number is 187532 and is a duplicate of 184830. Basically, queries with outer joins involving more than 3 tables may return incorrect number of rows when using the cost based optimizer. Use the ORDED hint as a possible work-around.

28.) Most of these issues relate to Oracle 7.0.1B and 7.1.4. My understanding is that some of the CBO problems and BUGs are being fixed in Oracle 7.2.

29.) Any suspected problems should be documented in an Oracle TAB. USB should open a BUG for any newly discovered problems. Problems that don't get BUG'ed don't get fixed.

One other thing. If hints are used and are specified incorrectly (ie, improper syntax) or the hint makes no sense (improper constants) then the hint is simply ignored without any error message or other feedback to the user.

DATA BLOCK CORRUPTION :

ORA-1578 해결 방법

Bulletin no : 10082

모든 오라클 데이터 블록은 sequence 번호(seq)와 incarnation 번호(inc)를 갖고 있다. ORA-1578 에러는 seq=0 이고 inc<>0(새로운 블록이 아님)일 때 발생한다. ORA-1578 에러는 ORA-800[3338] 에러와 함께 발생하곤 한다.

ORA-1578 에러가 발생하면 corruption 발생한 파일번호와 블록번호를 알려준다. 여기서의 파일번호를 f, 블록번호를 b 라고 부르기로 한다.

해결방법

1) 우선 해야 할 일은 어떠한 오류객체가 corrupt 되었는가를 알아내는 것이다.
다음의 스크립트를 이용하면 알 수 있다.

```
SQL>select segment_name, segment_type
      from dba_segments
      where file_id = f and
             b between block_id and block_id + blocks - 1;
```

2) 만약 해당 세그먼트가 minmax이면 drop 시키고 다시 생성하면 된다.

3) 만약 해당 세그먼트가 테이블이면 corrupt 블록의 데이터는 손상된 것이다.

4) 만약 해당 데이터가 들어있는 엑스포트 파일이 있다면 손상된 데이터를 drop 시키고 임포트 하는 것이 제일 간단한 방법이다. 하지만 만약 엑스포트 같은 파일이 없거나 작업해줄 파일도 없다면 해당 데이터에 minmax가 생성되어 있는 경우에 한해서 다음의 방법을 사용해서 복구를 하도록 한다.

5) 만약 해당 데이터에 대한 minmax가 생성되어 있다면 이를 이용해서 corrupt 된 블록을 지워갈 수 있다. 이 방법은 다음과 같다.

- msys, mmax, dmpno 를 컬럼으로 가지는 msp 데이터가 corrupt되었다고 가정하자. 그리고 msys 컬럼에 minmax가 생성되어 있다고 하자. 슬러시(/)까지 많은 모든 데이터들은 유니크한 rould를 가진다. rould를 varchar2/hexadecimal 형식으로 표현하려면 rouldtochar 함수를 이용한다.

```
SQL>select rouldtochar(rould) from msp;
```

rould는 총 18자로 블록어드레스(8자), 점(1자), 오프 어드레스(4자), 점(1자), 파일 어드레스(4자)로 구성되어 있다.

```
SQL>select msysn, rould
      from msp
      where msysn > 0
```

위의 스크립트를 실행시키면 다음과 같은 결과를 얻게 된다.

EMPNO	ROWID
----	-----
100	00000003.0000.0008
101	00000003.0001.0008
102	00000003.0002.0008
103	00000003.0003.0008
.	
500	00000004.0000.000A
501	00000004.0001.000A
.	
755	0000001A.0005.000A
758	0000001A.000C.000A

만약 인덱스가 character 컬럼에 대한 것이었다면 위의 rowid 문자를 다음과 같이 바꿀 수 있다.

```
SQL>select empno, rowid
      from emp
      where empno > 11;
```

예를 들어 다음과 같은 에러메시지가 떨어졌다고 하자.

```
01578, 00000, "ORA-01578: data block corrupted (file # 10, block # 4)"
```

그러면 다음의 스크립트를 사용하여 손상된 블록에 있는 empno에 대한 empno를 구할 수 있다.

```
SQL>select empno from emp
      where empno > 0
      and rowidtochar(rowid) like '00000004.%.000A';
```

EMPNO	ROWID
----	-----
500	00000004.0000.000A
501	00000004.0001.000A

이제 emp 테이블과 같은 구조를 갖는 새로운 데이터를 만든다.

```
SQL>create table temp
      as select * from emp
      where 1 = 2;
```

그리고는 손상된 부분을 비워서 새로운 데이터에 손상된 데이터의 데이터를 추가한다.

```
SQL>insert into temp select * from emp where empno < 500;
```

```
SQL>insert into temp select * from emp where empno > 501;
```

손상된 데이터를 drop 시키고 temp 데이터의 이름을 emp로 변경한다.

그리고 백업된 자료나 문서자료를 통하여 손상된 부분에 대한 정보를 추가한다.

B)손상된 블록에 여러개의 로우가 존재하고 있다면 다음의 쿼리를 이용한다.

```
BQL>create table mpage as
  select mpage from msp
  where mpage>0
  and validatechar(rowid) not like '00000004,2,0000';
```

이 스크립트를 이용하면 손상된 블록에 포함되지 않은 mpage 들을 알수 있다.
다음의 스크립트를 계속 실행시켜 복구를 한다.

```
BQL>create table temp as select * from msp where 1 = 2;
BQL>insert into temp
  select msp.mpage, msp.mmax, msp.dpage
  from msp, mpage
  where msp.mpage > 0
  and msp.mpage = mpage.mpage;
```

7)만약 데이터 디서너티의 레이블이나 인덱스에서 손상된 블록이 발견했다면 지워야 한다.

DATABASE INTERNAL RECOVERY

Bulletin no : 10149

< Internal parameters >

```
_allow_resetlogs_corruption = true
. avoid rolling forward
_offline_rollback_segments=r01,..
. take the rollback segments offline during transactions recovery.
. temporarily prevents RMAN from cleaning up the uncommitted transaction in the rollback
  segments involved.
_corrupted_rollback_segments=r01,...
. uncommitted transactions are marked as committed.
. used to corrupt the rollback segments and the rollback segment can be dropped
_db_block_cache_protect
. force every process to lock a block in memory before writing to it.
. if a process writes it in a location memory that it hasn't locked,
  then access violation occurs, giving diagnostic trace information.
  UMS, MVS, OS/2, MVS, OS/390, not implemented on most UNIX
_db_block_checksum
. used to debug from disk to cache, oracle will compute and write a
  checksum to the block. next time the same block is read, oracle
  compute the checksum again and compare it with the value that's
  in the block. if that differs, the block is corrupted on disk.
```

< Automatic recovery failure >

```
a. ora-800[3338]
t1 - . use _offline_rollback_segments=r01,r02,.. in init.ora
. try to open the db
t2 - . if rollback segment corrupted, use
      _corrupted_rollback_segments= r01,r02.. in init.ora
. rebuild rollback segments
t3 - . use _corrupted_rollback_segments=r01,r02
. use _allow_resetlogs_corruption=true
. make a controlfile as resetlogs and open the db using resetlogs.
. rebuild rollback segments

b. ora-800[3338]
t1 - . put _db_block_cache_protect=true on init.ora and
      try to startup 2-3 times
t2 - . set events in init.ora
      event='10210 trace name context forever, level 10'
      event='10211 trace name context forever, level 10'
```

```

c. ora-800[2882]
  t1 - . retry shutdown abort and startup 3 times
  t2 - . event="immediate trace name adjust_xco level 1" in init.ora
  t3 - . _corrupted_rollback_segments = r01,r02
      . use _allow_resetlogs_corruption=true

d. ora-800[18224], ora-472
  t1 - . event="10052 trace name context forever"
      . startup db
      * event 10052 : don't close up ahj!$

e. ora-800[3020] when start recovery
  before recovery, don't startup db

f. ora-800[4308]
  t1 - . event="10081 trace name context forever"
      * event 10081 : disable BMDM from closing temp segment

```

< Database mount and hanging >

```

a. if archivelog mode, check whether archivelog is normal or not
  sql> archive log list
  ...
  automatic archive    enable
  current log seq      1000
  ..
  next archive log seq 999 --> abnormal
  . check the disk space of archive log directory
  . automatic archive = enable

b. database not open and waiting
  . check alert.log file - if it is changing, then rolling back
  . bg processes are idle and no changing in alert.log - data block
    corrupted

  after database open
  Beginning crash recovery of 1 threads
  ... waiting
  t1 - use _corrupted_rollback_segments=r01,r02 and try to startup
  t2 - make controlfile as resetlogs
      _allow_resetlogs_corruption=true
      _corrupted_rollback_segments = r01,r02

```

< Current online log deleted >

```

. make controlfile as resetlogs
. use _allow_resetlogs_corruption and _corrupted_rollback_segments=r01,
. rebuild rollback segments

```

DATABASE LINK 사용 방법

Bulletin no : 10081

무선 고려되어야 사항은 DATABASE INSTANCE가 두 개 이상이고 각기 다른 PLATFORM에서 실행된다는 가정 하에서 각각의 HOST NAME과 DATABASE는 다르고 NLS_CHARACTERSET은 동일하게 되어 있어야 합니다.

만약 같은 MACHINE에서 INSTANCE의 DATABASE가 같다면 TNS SNAMES가 발생 할 것이다.

또한 미래를 위해 다른 MACHINE이라 할지라도 DATABASE는 공식에 의해 다르게 가져가는 것이 좋습니다.

그리고 NLS_CHARACTERSET이 동일하게 되어 있지 않으면 DATA 입 출력시 한글 데이터가 ?????로 나타날 것입니다.

그럼 환경 점검이 끝났으니 DATABASE에서 DATABASE USER(DATABASE의 경우도 비슷)에 있는 TABLE의 DATA를 DB LINK를 이용하여 SELECT하거나 VIEW를 작성하여 보겠습니다.

	U7	U8
HOST NAME	U7DB	U8DB
DATABASE_SID	U7SID	U8SID

이와 할때

1> U7 에서 U8로 DBLINK생성(이하 U7 시스템에서)

- Public Database Link 생성

sqlplus system/manager로 DATABASE에 LOGIN

```
BQL> create public database link U7TOU8 using 't:U8DB:U8SID';
```

- Private Database Link 생성

scott/tiger 로 Login

```
BQL> create public database link U7TOU8 connect to scott  
identified by tiger using 't:U8DB:U8SID';
```

또 하면 된다.

2> U8 에 있는 TABLE의 select 및 view 작성

```
BQL> select * from emp@U7TOU8;
```

```
BQL> create view emp_view as select * from emp@U7TOU8 a  
where a.deptno = 10;
```

3> U7 에서 SYNONYM을 생성하여 사용하는 경우

```
BQL> create synonym emp for emp@U7TOU8;
```

```
BQL> select * from emp;
```

또 한다면 간단히 분산 DB의 환경에서 사용 할 수 있습니다..

(단 BDBUSER TO BDBUSER로 NETWORK 환경이 구축되어 있어야 합니다.)

DATAFILE을 RENAME 하는 방법

Bulletin no : 10071

▶ NON-SYSTEM TABLESPACE 의 DATAFILE이름을 변경하는 경우

SPACE 때문에 데이터 파일을 다른 디렉토리에 옮겨야 하는 경우가 있다.

데이터 파일을 다른 디렉토리에 옮기는 것은 데이터 파일의 이름이 변경되는 것이므로 다음 SQL 문을 사용하면 된다.

```
ALTER TABLESPACE ts1111111111 RENAME DATAFILE 'f111111111' TO 'f11111111';
```

1. 데이터베이스 BACKUP을 시행하고 데이터베이스의 모든 작업을 중단하고 다음 작업을 실시한다.

2. 해당 TABLESPACE를 다음 명령어로 DB 가 OFFLINE 상태에서 OFFLINE 한다.

```
ALTER TABLESPACE ts1111111111 OFFLINE;
```

3. DB COPY 명령어를 사용하여 새로운 DIRECTORY에 FILE을 COPY 한다.

4. ALTER TABLESPACE 명령어를 사용하여 DATABASE FILE을 RENAME 한다.

5. TABLESPACE 를 ONLINE으로 전환한다.

```
ALTER TABLESPACE ts1111111111 ONLINE;
```

6. DB 상에서 OLD FILE을 삭제한다.

위의 작업을 하기 위해서는 OLD와 NEW 데이터 파일이 존재해야한다.

또한 데이터 파일의 이름이 변경 되었으므로

```
ALTER DATABASE BACKUP CONTROLFILE TO 'f111111111'
```

▶ SYSTEM TABLESPACE DATAFILE 이름을 변경하는 경우

SYSTEM TABLESPACE에 있는 DATAFILE NAME을 변경하는 경우 SYSTEM TABLESPACE는 OFFLINE 할 수 없으므로 DATABASE를 OPEN 하지 않고 MOUNT된 상태에서 ALTER DATABASE RENAME FILE 명령어를 사용한다.

1. EXPORT로 FULL BACKUP을 받는다.

```
$ exp system/manager file=system.dmp full=y
```

2. 데이터베이스를 정상적으로 DOWN 한다.

```
$ sqlplus /nolog
```

```
SQL> connect internal
```

```
SQL> shutdown
```

3. DATAFILE를 편집하고자 하는 파일 이름으로 COPY 한다.

```
$ cp /oracle/dbs/cyct_01d.dbf /oracle/dbs/cyct_000a.dbf
```

4. DATAFILE의 이름을 RENAMES 한다.

```
$ sqlplus lscdswy
SQLDBA > connect internal
SQLDBA > startup mount
SQLDBA > alter database rename file '/oracle/dbs/cyct_01d.dbf' to
        '/oracle/dbs/cyct_000a.dbf' ;
SQLDBA > alter database open;
```

5. 데이터베이스가 정상적으로 OPEN 되는지 SHUTDOWN 후 다시 STARTUP 한다.

6. 데이터베이스가 정상적으로 STARTUP되면 DLB 파일을 삭제한다.

```
SQLDBA > rm /oracle/dbs/cyct_01d.dbf
```

DBA 정보와 CONVERT SCRIPT

Bulletin no : 100BS

BBA는 데이터블럭에 대한 *zibb*와 *blockb*의 조합으로 이루어져 있다. 길이는 32bit 이며 상위비트들은 *zibb*를 나타내고 하위비트들은 *blockb*를 나타낸다.

UB에서는 대부분의 통신 플랫폼에서 *zibb*로 8bit를 사용했다. *zibb*를 이용하여 가능한 최대 데이터화일의 수를 계산하는 공식은 다음과 같다.

$$\text{최대 데이터화일 수} = 2^m - 2$$

여기서 *m*은 *zibb*를 표시하는 비트수를 나타낸다. 만약 *m*=8이라면

$$\text{최대 데이터화일 수} = 82 \text{ 개}$$

가 된다.

UF에서는 플랫폼마다 *zibb*, *blockb* 를 구성하는 비트수가 달라진다. 일반적으로 *zibb*에는 10비트가 사용되는데 그러면

$$\text{최대 데이터화일 수} = 2^{10} - 2 = 1022 \text{ 개}$$

가 된다.

그 동안 UF에서 BBA의 *zibb* 를 어떻게 구성할까 하는가에 대해 많은 논란이 있어왔다. 어떠한 구조를 가지는 모든 플랫폼에서 최소한 1022개의 데이터 화일을 지원할 수 있고 UB의 BBA가 UF에서도 그대로 사용될 수 있도록 하여야만 했다.

이는 UB에서 UF으로의 migration을 용이하게 해주기 때문이다. 우선 1022개의 화일을 지원하려면 10bit가 필요하므로 BBA의 *zibb*는 10bit로 결정되었다. 하지만 이렇게 함으로써 *zibb*가 8bit로 구성된 UB와의 호환성이 문제가 되었다. 아래에 한가지 예를 들었다.

(*mx*)*blockb*=1, *zibb*=1 일때

2진수로

UB: 0000 0100 0000 0000 0000 0000 0000 0001

UF: 0000 0000 0100 0000 0000 0000 0000 0001

16진수로

UB: 04000001

UF: 00400001

UB와의 호환성 문제를 해결하기 위하여 간단한 방법이 이용된다. 즉 앞에서 결정한 UF의 *zibb* 10bit(아직 실제로 UF에서 사용하는 값이 아님)에서 HBb와 LBb를 바꾼 값이 실제로 UF의 *zibb*로 사용하는 값이 되는 것이다.

HBb와 LBb의 bit 수는 다음과 같이 결정된다.

$$\begin{aligned}\text{HB\#의 bit 수} &= 10 - (\text{UB\#에서의 film\# bit 수}) \\ &= 4(\text{일반적으로})\end{aligned}$$

$$\begin{aligned}\text{LB\#의 bit 수} &= 10 - (\text{HB\#의 bit 수}) \\ &= 8(\text{일반적으로})\end{aligned}$$

하나 예를 들어보기도 하자

```
(ex)film# = 4, block#=3 일때
film#를 10bit 이진수로 표시하면 00 0000 0100
HB# = 0000
LB# = 0001 00

따라서
film# = 0001 0000 00
그러므로
BNA = 0001 0000 0000 0000 0000 0000 0000 0000
```

이 값은 UB와 호환점을 알 수 있다.

다시한번 UB에서 film#과 block#을 가지고 BNA를 계산하는 방법을 알아보기도 하자.

- 1) film #를 10bit 이진수로 변환한다.
- 2) UB에서 UB에서 film#로 사용되는 bit 수를 낸다. 이 값을 K 라고 부르기로 한다.
- 3) 과정 1)에서의 10bit에서 K 만큼의 HB#를 구하여 그 나머지와 순서를 바꾼다.
- 4) 과정 3)의 결과를 32bit 이진수의 HB#로 설정한다. 나머지 22bit는 0으로 설정한다.
- 5) block#를 32bit 이진수로 변환한다.
- 6) 과정 4)와 과정 5)의 결과를 64 번칸에 의하여 bit단위로 결합한다.
- 7) 이 값을 18진수로 변환한다.

하지만 UB BNA의 film#가 들쭉날쭉 다 어떻게 달라지는가는 아직 설명하지 않았다. UB의 film#가 8bit로 구성되어 있을 수도 있기 때문이며 과정 2)에서 K 라는 값을 설정하였음을 기억하기 바란다.

UB에서 film#를 구성하는 bit 수를 아는 방법은 다음과 같다.

```
BQL>select dump(characterid('00000000,0000,0001')) from dual;
```

이것은 varchar2의 내부저장형태를 보여줌으로써 film#의 구성을 알 수 있다.

ex)ALL#에서

```
BQL>select dump(characterid('00000000,0000,0001')) from dual;
```

```

BUMP(CCHARVARCHAR2('00000000,0000,0001'))
-----
Type=BB Len=8: 1,0,0,0,0,0,0
1 row selected

```

앞의 4개의 숫자가 BNA(film#, block#)를 가리킨다. 뒤의 2개의 숫자는 row# 이다.

이 값을 2진수로 쓰면

```
00000001 00000000 00000000 00000000
```

AIK클래스의 UB에서 `filenb` 가 위와 같이 표현되었으므로 `filenb`로 `Block`가 사용됨을 알 수 있다.
UB에서 사용되는 `Block`의 `filenb`는 보통 5~12bit로 구성된다. UB에서 `filenb`를 구성하는 bit수를 아는 방법은 두가지가 있다.

하나는 해당 클래스의 가능한 최대 데이터화일 수를 알아서 다음의 공식으로 변환하면 된다. `n`은 `filenb`의 bit 수 이다.

$$2^n - 2 = \text{최대 데이터화일 수}$$

다른 하나는 `filenb=####` 인 경우에 대해 `select` 문을 사용하여 알아 보는 방법이다.

```
BQL>select dump(charactorid('00000000,0000,####')) from dual;
```

```

DUMP(CCHARACTORID('00000000,0000,####'))
-----
Type=BB Len=8: 255,182,0,0,0,0
1 row selected
```

2진수로는

```
11111111 11000000 00000000 00000000
```

여기서 AIK에서는 10bit가 `filenb`에 사용됨을 알 수 있다. -END-

다음은 `filenb` 및 `blocknb`와 `BB`간의 상호 변환을 수행하는 스크립트이다.
여기에 나오는 함수들을 이용하여 변환을 하기 바란다.

```
BBN ****
BBN CATUNP,BQL - (run as BYTBN or other BBN user)
BBN
BBN This utility script creates stored objects for dumping blocks:
BBN (1) dha(filn,blk,port) -- returns decimal dha or level.
BBN (2) dhax(filn,blk,port) -- returns hex dha.
BBN (3) dharev(decidha,port) -- returns filn, block.
BBN (4) dharev(hexidha,port) -- returns filn, block.
BBN (5) block(filn,blk,port) -- dumps the specified block
BBN
BBN Needs ALTER SESSION, CREATE PROCEDURE/FUNCTION/TABLE to be
BBN explicitly granted. Privs to execute DBMS_SQL routines also.
BBN
BBN ****

BBN connect system/manger;
```

```

drop table portinfo;
drop function dha;
drop function dharvi;
drop function dhaxi;
drop function dharvax;
drop function dactohaxi;
drop function hactohaxi;

drop public synonym dha;
drop public synonym dharvi;
drop public synonym dhaxi;
drop public synonym dharvax;

create table portinfo
( portid          number primary key,
  cfdhaacthite    number,
  cfdhaacthite    number,
  port varchar2(25));
insert into portinfo values (1, 8, 10, 'DEC VMS');
insert into portinfo values (2, 8, 10, 'HP/UX');
insert into portinfo values (28, 5, 8, 'BCD Unix');
insert into portinfo values (30, 12, 12, 'IBM MVS');
insert into portinfo values (84, 5, 8, 'BCD HPK');
insert into portinfo values (87, 8, 10, 'DEC Alpha');
insert into portinfo values (58, 8, 10, 'DEC MIPS Ultrix');
insert into portinfo values (118, 8, 10, 'Pyramid DC/DBx');
insert into portinfo values (182, 8, 10, 'DC/UX Avilion');
insert into portinfo values (188, 8, 10, 'BCI Unix');
insert into portinfo values (188, 8, 10, 'Beguani DYNIX/ptx');
insert into portinfo values (202, 8, 10, 'ATT 7000');
insert into portinfo values (283, 8, 8, 'DB/2');
insert into portinfo values (318, 8, 10, 'IBM RS/6000');
insert into portinfo values (358, 8, 10, 'ATT/NOA 3000');
insert into portinfo values (451, 8, 10, 'Sun 4 Unix');
insert into portinfo values (453, 8, 10, 'Sun Solaris');
insert into portinfo values (810, 8, 10, 'IBM SP2');
insert into portinfo values (811, 8, 10, 'Pyramid MESH');
commit;

create or replace FUNCTION hactohax(x IN VARCHAR) RETURN NUMBER IS
  x NUMBER := 0;
  y NUMBER;
  z VARCHAR2(1);
  BEGIN
    IF length(x) > 8 THEN -- only allow 32-bit DBAs
      RAISE invalid_number;
    END IF;

```

```

FOR i IN 1..length(x) LOOP
    x := upper(substr(x,i,1));
IF x = 'A' THEN          -- decode() leads to bug 240820
    y := 10;
ELSEIF x = 'B' THEN
    y := 11;
ELSEIF x = 'C' THEN
    y := 12;
ELSEIF x = 'D' THEN
    y := 13;
ELSEIF x = 'E' THEN
    y := 14;
ELSEIF x = 'F' THEN
    y := 15;
ELSE
    y := to_number(x,8);
END IF;
    x := x * 18;
    x := x + y;
END LOOP;
RETURN x;
END;
/

create or replace FUNCTION decodehex(x IN NUMBER) RETURN VARCHAR IS
    x VARCHAR2(8) := '';
    y VARCHAR2(1);
    z NUMBER;
    w NUMBER;
BEGIN
    IF x > power(2,32) OR x < 0 THEN
        RAISE invalid_number;
    END IF;
    w := x;
    WHILE w > 0 LOOP
        x := w mod 18;
        IF x = 10 THEN
            y := 'A';
        ELSEIF x = 11 THEN
            y := 'B';
        ELSEIF x = 12 THEN
            y := 'C';
        ELSEIF x = 13 THEN
            y := 'D';
        ELSEIF x = 14 THEN
            y := 'E';
        ELSEIF x = 15 THEN
            y := 'F';

```

```

ELSE
    y := to_char(x);
END IF;
    u := trunc(u / 18);
    x := concat(y,x);  -- build x string backwards
END LOOP;
RETURN x;
END;
/

create or replace function
    dhk(filn_id number, block_id number, port_id number)
return number is
    u number;
    w number;
    diff number;
    ohltz number;
    whltz number;

begin
    begin
        select cfdhazfhltx, cfdhazfhltx into ohltz, whltz
        from portinfo
        where portid = port_id;
    EXCEPTION
        WHEN no_data_found THEN
            RAISE invalid_number;
    end;
    end;
    IF filn_id > 1022 THEN
        RAISE invalid_number;
    END IF;
    diff:= whltz-ohltz;
    u := floor(filn_id/power(2, ohltz));  /* value of higher DIFF bits */
    w := mod (filn_id, power(2, ohltz));  /* value of lower OBLTS bits */
    return (u*power(2, diff) + w)*power(2, 32-ohltz) + block_id;
                                     /* swap higher/lower, add block_id */
end;
/

create or replace function
    dhazw (dazila number, port_id number)
return varchar is
    ohltz number;
    whltz number;
    fl number;
    lk number;
    diff number;
    wazlk number;

```

```

begin
  begin
    select cfdhazfhitx, cfdhazfhitx into ohitx, ohitx
      from portinfo
     where portid = port_id;

    EXCEPTION

      WHEN no_data_found THEN
        RAISE invalid_number;

    end;

    IF (dazdha > power(2,32)) OR (dazdha < 0) THEN
      RAISE invalid_number;

    END IF;

    diff:= ohitx-ohitx;
    maxkk:= power(2,32-ohitx);          /* prepare block mask */
    kk:= mod(dazdha, maxkk);           /* extract block_id value */
    fl:= floor(dazdha/maxkk);          /* extract file_id value */
    fl:= mod(fl, power(2,diff))*power(2,ohitx) +
         floor(fl/power(2,diff));      /* map "diff" bits */
    return (to_char(fl) || ' ' || to_char(kk)); /* "FILE , BLOCK" */

  end;
/

create or replace function
  dha (file_id number, block_id number, port_id number)
return varchar is

begin
  return dactohaz(dha(file_id,block_id,port_id));
end;
/

create or replace function
  dhazex (hazdha in varchar2, port_id in number)
return varchar is

begin
  return dhazex(hextodba(hazdha), port_id);
end;
/

create or replace procedure
  block (file_id in number, block_id in number, port_id number) as
cur integer;
dummy integer;
for number;

begin
  cur:= dha_sql.open_cursor;

```

```

inv:= dha(fill_id, block_id, port_id);
IF (inv > power(2,32)) OR (inv < 0) THEN
  RAISE invalid_number;
END IF;
dha_sql.parse(cur, 'alter session set events
  ''immediate trace name blockdump level '||inv|| ' ''', dha_sql.v7);
dump:= dha_sql.execute(cur);
dha_sql.close_cursor(cur);
end;
/

grant execute on dha to public;
grant execute on dharov to public;
grant execute on dhax to public;
grant execute on dharovx to public;
grant execute on block to public;

create public synonym block for block;
create public synonym dha for dha;
create public synonym dharov for dharov;
create public synonym dhax for dhax;
create public synonym dharovx for dharovx;

```

DBMS PACKAGE : DBMS_ALERT을 이용하여 SESSION간 통신하는 방법

Bulletin no : 103B4

DBMS_ALERT이란?

DBMS_ALERT은 DATABASE에 일어나는 변화를 자동으로 알려줄 수 있는 FUNCTION이다. APPLICATION은 관심있는 TABLE에 대해 일어나는 변화에 대한 확인을 언제든지 할 수 있게 된다. 이렇게 되기 위해서 해당 TABLE에 대해 변화가 일어났을 때 ALERT을 줄 수 있는 몇개의 PROCEDURE가 DATABASE에 등록되어 있어야 한다.

DBMS_ALERT은 어떤 TABLE에 대해 일어나는 INSERT, UPDATE, DELETE등의 변화를 MONITORING하는 PROCEDURE들에 이용될 수 있겠다.

ALERT을 사용하려면?

다른 DBMS PACKAGE와 마찬가지로 DBMS_ALERT PACKAGE는 SYS가 OWNER이며 CATALOG.BDL을 실행하여 생성된다. DBMS_ALERT PACKAGE를 사용하려면 이 PACKAGE에 대한 EXECUTE PRIVILEGE가 있어야 한다.

SQL> GRANT EXECUTE ON DBMS_ALERT TO USER_NAME;

한편 EXECUTE PRIVILEGE를 받으면 다른 BULKFILE, PROC, PROCEDURE, PL/SQL같은 다른 ORACLE TOOL에서 DBMS_ALERT을 사용할 수 있다.

DBMS_ALERT은 어떻게 실행되는가?

여기에서는 몇 개의 SESSION간에 ALERT이 어떻게 사용되는지 설명하기 위해 DBMS_ALERT PACKAGE중 4개의 PROCEDURE를 사용하겠다.

REGISTER PROCEDURE:

- 이 PROCEDURE는 ALERT을 등록한다.
- 하나의 SESSION에서 여러개의 ALERT을 등록할 수 있다.
- DBMS_ALERT.REGISTER(ALERT_NAME IN VARCHAR2);**

SIGNAL PROCEDURE:

- 이 PROCEDURE는 ALERT에 SIGNAL을 보내고, MESSAGE를 보낸다.
- SIGNAL을 TRANSACTION에서 COMMIT을 만나야지만 명함 있게 된다.
- 만약 TRANSACTION에서 ROLL BACK을 하면 SIGNAL을 아무런 명함을 받지 않는다.
- DBMS_ALERT.SIGNAL(ALERT_NAME IN VARCHAR2,
MESSAGE IN VARCHAR2);**

WAITFOR PROCEDURE:

- 이 PROCEDURE는 SESSION이 특정한 ALERT이 발생되기를 기다리게 한다.

SIGNAL이 실행되면, MESSAGE는 WAITING하고 있는 모든 BEBIDN에 전달되게 된다.
 TIME-DUT BECDNDB에 지정된 시간 내에 SIGNAL이 오지 않으면 WAITING은 STATUS 1을
 RETURN하게 된다. 그 전에 ALERT이 일어나면 STATUS 0을 RETURN한다.
 TIME-DUT시간은 USER가 선택적으로 지정할 수 있다.

```

NAME_ALERT, WAITING(Alert_NAME IN VARCHAR2,
                      MESSAGE DUT VARCHAR2,
                      STATUS DUT INTEGER,
                      TIME-DUT IN NUMBER);
  
```

만약 한 BEBIDN에서 여러개의 ALERT를 사용할 경 예는 WAITANY PROCEDURE를
 사용하여야 한다.

BEBIDN PROCEDURE:

이 PROCEDURE는 등록LIST에서 해당 ALERT명을 삭제하는 기능을 한다.
 NAME_ALERT, REMOVE(ALERT_NAME IN VARCHAR2);

더 자세한 사항은 DBACLE7 BEBIDN APPLICATION DEVELOPER'S GUIDE 참조 하함

SAMPLE

NAME_ALERT와 TIME-DUT를 이용하여 BEBIDN간에 TABLE에 관련된 정보를 전달하는 과정을 본다.
 세번째 BEBIDN에서 EMP TABLE에 UPDATE, DELETE, INSERT문장을 실행하면 SIGNAL을 보내는 DB
 TIME-DUT가 FINISH하게 된다.
 그러면 기다리고 있는 2개의 PROC, PL/SQL BEBIDN에서 SIGNAL을 받아 MESSAGE를 내게 되는 것이다.

첫번째 BEBIDN은 PROC PROCNAME이다.
 이 BEBIDN에서는 I_DUT_IT이라는 이름의 ALERT를 지정하여 SIGNAL을 기다린다.

```

#include
#include
#define NAME_LEN 20
#define PUB_LEN 40
VARCHAR username[NAME_LEN];
VARCHAR password[PUB_LEN];
  
```

EXEC SQL BEGIN DECLARE SECTION:

```

int status;
int timeout=20;
varchar name[40];
char message[100];
short name_len;
short message_len;
short status_len;
  
```

EXEC SQL END DECLARE SECTION:

```

#include
/* Declare error handling function. */
void sql_error();
  
```



```

main()
{
    strcpy((char *) username, "BOB", USERNAME_LEN);
    username_len = strlen((char *) username);
    strcpy((char *) password, "TIGER", PASSWORD_LEN);
    password_len = strlen((char *) password);

    EXEC SQL WHENEVER SQLERROR GO sql_error("DBACLE error\n");

    EXEC SQL CONNECT username IDENTIFIED BY :password;
    printf("\nConnected to DBACLE as user: %s\n", username);

    strcpy((char *)msg, "I GOT IT");
    msg_len = strlen((char *) msg);

    memset(msgbuf, '\0', 100);
    EXEC SQL EXECUTE
        BEGIN
            DBMS_ALERT.REGISTER(:msg);
        END;
    END-EXEC;
    printf("\n WE ARE WAITING FOR WAITONE SIGNAL\n", msg);
    EXEC SQL EXECUTE
        BEGIN
            DBMS_ALERT.WAITANY(:msg;msg_buf, :msgbuf;msgbuf_len,
                               :status;status_buf, :timeout);
        END;
    END-EXEC;
    if (:status)
    {
        printf("\n WE HAVE RECEIVED THE SIGNAL\n");
        printf("\n THE MESSAGE THAT HAS BEEN RECEIVED IS\n", msgbuf);
        printf("\n THE STATUS\n\t\t THE TIME\n");
        printf("\n\t\t %d\t\t %d\n", status, timeout);
        printf("\n\t\t day\n");
    }
    else
        printf("NO MESSAGE HAS BEEN RECEIVED, EXIT SESSION\n");
    EXEC SQL EXECUTE
        BEGIN
            DBMS_ALERT.UNREGISTER(:msg);
        END;
    END-EXEC;
    EXEC SQL COMMIT WORK RELEASE;
    exit(0);
}
-----

```

두번째 BESSID은 PL/SQL이다.

이 BESSID에서도 I_CDT_IT이라는 이름의 ALERT를 지정하여 SIGNAL을 기다린다.

```
-----
DECLARE
    status NUMBER;
    timeout NUMBER;
    message CHAR(30);
    name VARCHAR2(20);
BEGIN
    name:='I_CDT_IT';
    timeout:=20;
    DBMS_ALERT.REGISTER(name);
    DBMS_ALERT.WAITANY(name, message, status, timeout);
    IF status = 0 THEN
        DBMS_OUTPUT.PUT_LINE(' RECEIVED SIGNAL ' || name);
        DBMS_OUTPUT.PUT_LINE('THE MESSAGE THAT HAS BEEN RECEIVED IS ' || message);
    ELSE
        DBMS_OUTPUT.PUT_LINE('NO MESSAGE HAS BEEN RECEIVED. EXIT BESSID');
    END IF;
    DBMS_ALERT.SEND(name);
END;
/
-----
```

세번째 BESSID을 SQL*PLUS에서 SQL을 실행하는 BESSID이다.

이 BESSID은 먼저 다음과 같이 단순 게 I_CDT_IT이라는 ALERT를 다음과 같이 등록한 후 INSERT, UPDATE, DELETE등의 SQL을 실행한다.

```
SQL> EXECUTE DBMS_ALERT.REGISTER('I_CDT_IT');
```

위의 3개의 BESSID을 생성하기 전에 사용자는 먼저 다음과 같은 TRIGGERS를 등록해 놓아야 한다.

```
-----
CREATE OR REPLACE TRIGGER emp_trig
AFTER INSERT OR UPDATE OR DELETE ON EMP
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        DBMS_ALERT.SIGNAL('I_CDT_IT', 'INSERT INTO EMP ' || :new.name);
    END IF;
    IF UPDATING THEN
        DBMS_ALERT.SIGNAL('I_CDT_IT', 'UPDATE NAME TO ' || :new.name);
    END IF;
    IF DELETING THEN
        DBMS_ALERT.SIGNAL('I_CDT_IT', 'DELETE FROM EMP ' || :old.name);
    END IF;
END;
/
-----
```

위의 PROC, PL/SQL SESSION을 실행한 후 3번째의 SQL*PLUS SESSION을 실행한다.
SQL*PLUS SESSION에서 다음과 같은 SQL문장을 실행한 후 COMMIT한다.

```
SQL> UPDATE EMP SET ENAME = 'HARI' WHERE EMPID = 7844;
```

COMMIT되고나면 TRIGGER가 FIRE되어 SIGNAL이 실행되고, PROC, PL/SQL SESSION에서 ALERT를 받아 MESSAGE를 내게 된다.

< 위의 PROC들만을 ORDB 7.1.8 , PROC 2.0.8에서 작성된 것이다. >

DBMS PACKAGE : DBMS_PIPE NOT DECLARED ERROR 발생 시

Bulletin no : 10342

발생원인 1)

DBMS_PIPE는 oracle install시에 \$ORACLE_HOME/rdbms/admin/catproc.sql을 실행함으로써 생성되도록 되어 있는 stored package이다. 다 stored package(DBMS_OUTPUT,...)들은 생성직후 user를 owner로 한 stored package에 대해 public synonym을 생성한 후 바로 이 synonym에 대해 public에게 execute privilege가 grant되지만 DBMS_PIPE는 생성만 되도록 되어 있기 때문에 사용하고자 하는 user에게 별도의 grant작업이 필요하다.

-> 확인사항 및 해결 방법

1) sqlplus에서

```
select * from dba_objects where object_name = 'DBMS_PIPE';
```

를 수행하여 DBMS_PIPE에 관한 자료가 user가 owner인 package spec, package body, synonym만 나타나야 한다.

2) sqlplus에서

```
grant execute on DBMS_PIPE to user_name;
```

를 실행하여 해당 user에게 grant한다.

발생원인 2)

DBMS_PIPE는 user가 owner이어야 한다.

user의 심수로 connect나 다른 별도의 user로 connect하여 catproc.sql을 실행한 경우에 발생한다.

-> 확인사항 및 해결방법

1) sqlplus에서

```
select * from dba_objects where object_name = 'DBMS_PIPE';
```

를 수행하여 DBMS_PIPE에 관한 자료가 user나 public이 아닌 다른 user가 owner로 되어 있는 DBMS_PIPE object를 display된다.

2) sqlplus에서

```
grant execute on DBMS_PIPE to user_name;
```

를 실행하여 해당 user에게 grant한다.

DBMS PACKAGE : DBMS_PIPE 사용 (SESSION 간의 MESSAGE SENDING)

Bulletin no : 10300

DBMS_PIPE Package는 동일한 Instance내의 둘 혹은 그 이상의 Session간에 통신을 가능하게 한다. DBMS_PIPE는 UNIX에서 사용하는 PIPE의 개념과 유사하나, DBMS_PIPE는 OS PIPE의 MECHANISM을 사용하지 않는다.

DBMS_PIPE를 통해 전해지는 정보는 BCP에 저장되며 LRU Algorithm을 사용한다. (DB CREATE 시에 COMPACT BCP이 실행되면서 자동 생성된다.)

* 이 Package를 사용하려면 PL/SQL Version 2.0이상이어야만 지원이 가능하며, 따라서 Form3.0이나 Form4.5등에서는 사용할 수 없으며 경우에 따라 Stored Procedure를 작성하여 Call함으로써 사용할 수 있다.

또한 BYB에서 해당 User에게도 Execute할 수 있는 권한을 Grant 시켜주어야만 리모트 실행할 수 있다. (예: BQL> grant execute on dbms_pipe to scott;)

* 한 User에 의하여 정보가 입력되면, Buffer는 비 되고, 같은 PIPE라 하더라도 다른 User가 읽을 수 있게 된다.

* 예제

```
create or replace procedure SendMessage(info VARCHAR2)
is
    n NUMBER;
begin
    DBMS_PIPE.PACK_MESSAGE(info);
    n := DBMS_PIPE.RECEIVE_MESSAGE('pipe_name');
end;
/
(여기서 pipe_name은 임의로 줄 수 있다.)
```

```
create or replace function ReceiveMessage
return VARCHAR2 is
    n NUMBER;
    c VARCHAR2(200);
begin
    n := DBMS_PIPE.RECEIVE_MESSAGE('pipe_name', 5);
    /* 5 는 timeout기간(초단위)이며 default maximum은
       8640000(1000 일) 이다. */

    DBMS_PIPE.UNPACK_MESSAGE(c);
    return(c);
end;
/
```

* SQLPLUS에서 DATA PASSING

```

SQL> execute RecvMessage('Hello');

PL/SQL procedure successfully completed.

SQL> variable x varchar2(200);
SQL> execute x := RecvMessage;

PL/SQL procedure successfully completed.

SQL> print x;

X
-----
Hello

```

* FORMS에서 DATA PASSING

[A FORM]

1. DEPT Table와의 Default Block으로 Form을 작성한다.
2. Button을 만들고 그 Button에 WHEN-BUTTON-PRESSED Trigger를 아래와 같이 쓴다.

```

DECLARE
    x VARCHAR2(200);
BEGIN
    x := :DEPT_NAME;
    WHEN_PIE_PUSH('pie_name');
    RecvMessage(x);
END;

```

[B FORM]

1. PRE-QUERY TRIGGER

```

BEGIN
    :DEPT_NAME := RecvMessage;
END;

```

2. WHEN-BUTTON-PRESSED TRIGGER

```

execute_query;

```

* DBMS_PIPE Package의 Function과 Procedure

보안적인 면에 따라, PUBLIC PIPE와 PRIVATE PIPE중 선택하여 사용한다.

FUNCTION	DESCRIPTION
CREATE_PIPE	임시적 PIPE를 생성한다. (PRIVATE PIPE를 사용하는데 필요)
SEND_MESSAGE	NAMED PIPE에 MESSAGE를 보낸다. (NAMED PIPE가 없는 경우 묵시적인 PUBLIC PIPE를 생성한다)
RECEIVE_MESSAGE	NAMED PIPE에서 LOCAL BUFFER로 MESSAGE를 COPY한다
NEXT_ITEM_TYPE	BUFFER내의 다음 ITEM의 DATA TYPE을 RETURN한다
REMOVE_PIPE	NAMED PIPE를 만든다
UNIQUE_SESSION_NAME	UNIQUE SESSION명을 RETURN한다

PROCEDURE	DESCRIPTION
PACK_MESSAGE	LOCAL BUFFER 안에 MESSAGE를 만든다
UNPACK_MESSAGE	BUFFER내의 다음 ITEM을 ACCESS한다
PURGE	NAMED PIPE내의 내용을 없앤다
RESET_BUFFER	LOCAL BUFFER의 내용을 없앤다

* 더 자세한 사항은 Oracle® Application Developer's Guide 를 참조.

DB__FILE__MULTIBLOCK__READ__COUNT와 퍼포먼스의 관계

Bulletin no : 10101

* `db_file_multiblock_read_count` 파라미터는 한련의 I/O 작업으로 읽어들이는 최대 블록 수를 가
리키며 Multi Block I/O 에 관련된 파라미터이다. 디스크 값은 D/B에 따라서 다른데 보통
`db_block_buffers`과 `Procsessors`에 의해서 결정되며 4~32 정도가 많이 사용된다.

이 값을 키우고 `BDWT/HWBCS` 를 이용하는 것이 `Nested Loop`를 이용하는 경우 보다 퍼포먼스가 증가
하는가 하는 문제는 `Query`의 종류와 읽어들이는 데이터의 망(전체에 대한 `Percentage`)에 따라서 달
라지게 된다.

즉, 이 값이 크면 한련의 I/O로 여러개의 블록을 읽 으므로 `Full Table Scan`시에는 퍼포먼스가 증
가한다. 하지만 특정 블록만이 필요한 경우에 불필요한 블록까지 함께 읽어들이므로 디스크를 읽는
시간이 증가할 뿐 아니라 크기가 한정된 `Buffer Pool`을 많이 차지해 버림으로써 자주 쓰이는 데이터
를 읽어내어 오히려 `Cache`의 효율을 떨어뜨리는 결과를 가져올 수도 있으므로 작업특성에 따라서 적
절한 값을 세팅하여야 한다.

오라클의 `Optimizer`는 이 값이 크면 작업의 효율성을 위해서 `Full Table Scan` 을 선택할 가능성이
커지게 된다.

DB__NAME과 SID 변경하는 방법

Bulletin no : 10182

Modifying a database to run under a new ORACLE_SID

1. Shutdown instance
2. Backup all control, redo and data files.
3. Go thru the .profile, .cshrc, .login, orstah, tnsnames.ora (for net v2) , and redefine the environment variable ORACLE_SID to a new value.
In search thru disks and do a grep ORACLE_SID *
4. cd \$ORACLE_HOME/dbs and remove the following files:
 - a init.ora (or use pfile to point to the init file.)
 - a control file(s) This is optional if you don't remove any of the controlfiles, and the control_files parameter is used. control_files would be set in the initSID.ora file or in a file it references with the pfile parameter. Make sure control_files doesn't point to any old file names, if you removed them.
 - a crd1.sql & crd2.sql This is optional. These are only used at database creation.
5. cd \$ORACLE_HOME/rdbms/admin and remove the files:
 - a startup.sql This is optional.(On some platforms, this file may be in \$ORACLE_HOME/rdbms/install.)
Make sure the contents of this file do not reference old initSID.ora files that have been removed. This file simplifies the process to "startup exclusive" your database.
6. To remove the database files and redo log files, you would follow the instructions in the bulletin: 88883,723.
7. Change the ORACLE_SID environment variable to the new value.
8. start up database and verify it works. Once you have done this, shutdown the database and take a final backup of all control, redo and data files.
9. When the instance is started, the control file gets updated with the current ORACLE_SID.

Changing the dbname for a database

1. sqlplus
2. connect internal
3. alter database backup controlfile to trace;
This will write in a trace file, the CREATE CONTROLFILE command that would recreate the controlfile as it currently exists.
4. Exit and go to the directory where your trace files are located.

They are usually in the \$ORACLE_HOME/rdbms/log directory.
 If user_dump_dest is set in the initSID.ora, then go to the directory
 listed in the user_dump_dest variable.
 The trace file will have the form "ora_NNNN.trc" with NNNN being a
 number.

5. Get the CREATE CONTROLFILE command from the trace file and put it in
 a new file called something like ccf.sql.
6. Edit the ccf.sql file and modify the CREATE CONTROLFILE command.
 Just change the word "RESET" to "SET", and "NORESETLOGS" to
 "RESETLOGS", and modify the dbname.

Old line:

```
CREATE CONTROLFILE RESET DATABASE "olddbname" NORESETLOGS ...
```

New line:

```
CREATE CONTROLFILE SET DATABASE "newdbname" RESETLOGS ...
```

Then save the ccf.sql file.

7. Rename the old control files for backup purposes and so they are not
 in the way of creating the new ones.
8. Edit initSID.ora so that db_name="newdbname".
9. sqlplus
10. connect internal
11. startup nomount
12. Recf
13. alter database open;
14. Make sure the database is working. Shutdown and backup the database.

DB를 COLD BACKUP(FULL BACKUP) 받는 방법

Bulletin no : 10187

콜드 백업은 데이터베이스가 정상적으로 SHUTDOWN 된 상태에서 데이터파일, 로그 파일, 콘트롤 파일을 모두 백업 받는 것을 말한다. SHUTDOWN 하지 않고 OPEN 된 상태에서 백업을 받으면 백업받은 내용을 나중에 사용할 수가 없으므로 유의해야 한다.

콜드 백업을 위해서 데이터베이스를 SHUTDOWN 할 때에는 NORMAL, IMMEDIATE 옵션을 사용해야 하며 ABORT 를 사용해서는 안된다. IMMEDIATE 를 사용한 경우에는 SHUTDOWN 후에 다시 STARTUP 하고 NORMAL 로 SHUTDOWN 하도록 한다.

콘트롤 파일과 데이터 파일 및 로그 파일의 위치를 확인하여 이들을 tar, cpio, dd 등의 명령을 이용하여 백업 받도록 한다.

콘트롤 파일 확인 방법

```
BQLBDBA>show parameter control_files
```

데이터 파일 확인 방법

```
BQLBDBA>select name from v$datafile
```

로그 파일 확인 방법

```
BQLBDBA>select member from v$logfiles;
```

필요에 따라서는 파라미터 파일까지 받아주는 것이 좋다.

파라미터 파일은 \$ORACLE_HOME/dbs/init.ora 이다. 여기서 는 오라클 인스턴스 이름으로 플리스 환경 변수 ORACLE_SID 로 지점이 되어 있다.

DDB 기본기능 및 분산 트랜잭션 문제의 해결

Bulletin no : 10285

여기서는 DBMS(분산 데이터베이스)의 기능과 역할 및 사용 시에 발생하는 문제점들에 대해서 알아보기로 한다. 분산 DB 환경과 관련된 init.ora파일의 파라미터는 다음과 같은 것들이 있다.

- DB_NAME
- DB_DOMAIN
- GLOBAL_NAME
- DISTRIBUTED_TRANSACTION_HOLD_TIME
- DISTRIBUTED_TRANSACTION
- DISTRIBUTED_LOCK_TIMEOUT
- COMMIT_POINT_THRESHOLD
- OPEN_LINKS

분산 DB와 관련된 VIEW와 TABLE은 다음과 같다.

- GLOBAL_NAME
- DBA_DB_LINKS, ALL_DB_LINKS, USER_DB_LINKS
- DBA_2PC_PENDING
- DBA_2PC_NEIGHBORS

* RECO:Recovery Process

분산 트랜잭션의 Failure를 해결하는 Process로 RECO는 In-doubt 분산 트랜잭션을 가지고 있는 다른 DB에 자동적으로 Connect 하여 모든 In-doubt 트랜잭션을 해결한다. 만일 RECO가 Remote Server에 Connection을 시 했을때 Remote Server가 사용 가능하지 않거나 Network가 복구 되어 있지 않을 경우 RECO는 일정한 시간 후에 다시 Connect를 시도한다.

* DATABASE LINK

1. Database Link에는 Private Database Link와 Public Database Link의 두 종류가 있는데 일반적으로 Public Database Link로 정의하여 사용한다.

2. Public Database Link

Local 데이터베이스의 모든 사용자가 Remote 데이터베이스의 데이터를 사용할 수 있다. 이 경우 Remote Database에 대해서 필요한 권한을 가지고 있어야한다.

3. Public Database Link Creation

```
SQL> CREATE PUBLIC DATABASE LINK scott.acme.com USING 'dbsqling';
```

4. Dropping Database Link

```
SQL> DROP PUBLIC DATABASE LINK scott.acme.com;
```

분산처리 문제와 해결 방법

1. Two-Phase Commit을 인터럽트하는 Failure

[현상]

DBA-02050: Transaction ID rolled back, some remote dbe may be in-doubt

DBA-02051: Transaction ID committed, some remote dbe may be in-doubt

DBA-02054: Transaction ID in-doubt

만일 프로그램이 사용 중 위의 에러 가운데 하나가 발생한다면 그 트랜잭션에 대한 정보 자동으로 저장되며 이 정보는 후에 분산 트랜잭션에 대한 Manual Recovery 시에 사용될 수 있다.

[조치 방법]

DBA는 특별한 Action을 취할 필요가 없다. DBD Process가 Session Trans의 모든 노드에서 같은 결과가 발생하도록 (즉 모두 Commit 즉 모두 Rollback) in-doubt 트랜잭션을 자동적으로 Recovery 한다. 그러나 장기적인 Power Failure인 경우 Lock된 리소스를 Release 하기 위해 강제적으로 Commit나 Rollback을 시켜준다.

2. 데이터의 Access를 금지시키는 Failure

[현상1.]

Transaction Timeout

DBA-02048: time-out : distributed transaction waiting for lock

Local의 DBL 문장이 Remote의 다른 Transaction에 의해 Lock 되어 있는 데이터를 수정 및 삭제 하려고 할 때 Time-out이 발생하고 그 명령은 Rollback된다.

(Scenario)

[In REMOTE]

BQL>update dept set deptno=10;

[In LOCAL]

BQL>update dept set deptno=10;

BQL>update dept@DBA7 set deptno=10;

DBA-02048 : time-out

[조치 방법]

Local DB에는 Update가 발생하므로 항상 Rollback을 시켜야 한다. RemoteDB는 변경되지 않으므로 Timeout의 결과로 다른 조치는 필요 없다. 위 상황에서 Time-Out Interval은 DISTRIBUTED_LOCK_TIMEOUT 파라미터로 Time Out시간을 조정할 수 있다.

[현상2.]

Lock from in-doubt Transaction

DBA-01581 : Lock held by in-doubt distributed transaction ID

Local DB에 Lock을 발생시키는 DBL 문장을 in-doubt 분산 트랜잭션에 의해 Lock된 Resource에 계속해서 실행할 경우 발생하는 에러

(Scenario)

(1) update dept00007 set dname='AA';

(2) Remote DB down

(3) Commit

ORA-02054 : transaction 2,1,207 is in-doubt transaction

(4) select * from dept;

ORA-01581 : Lock held by in-doubt distributed transaction id 2,1,207

Local에서 dka_2pc_pending으로부터 in-doubt 트랜잭션 정보를 확인할 수 있다.

[조치 방법]

이 경우 BDL 문장은 즉시 Rollback 되지만 Lock이 계속해서 걸려있으면 BDL에게 알리도록 한다. 이런 현상은 드물게 발생하며 Network나 System Failure가 빠르게 복구되면 문제는 자동적으로 해결된다.

3. 두개 NODE 사이에 Connection이 만들어진 이후에 Network Failure가 발생한 경우

(Scenario)

(1) Form 화면에서 Data Update

(2) Commit - DB Trigger에 의해서 Server Machine에 Server process가 생기며 Network과 Remote DB가 정상적인 경우 Commit 된다.

(3) Network Fail(Line 들 Disconnect)

(4) Form 화면에서 Update

(5) Commit

(6) Form Screen이 Holding 상태로 됨

[조치 방법]

Network이 복구되면 자동적으로 Commit 되지만 Network Failure가 장기화될 경우 강제로 Abort시키는 조치가 필요하다. 강제로 Abort 이후 자동적으로 Rollback 되므로 더 이상 조치가 필요하지 않다.

4. Network이 Down된 상태에서 Connection을 시도하려는 경우

(Scenario)

(1) Network Down

(2) Form 화면에서 Data 수정

(3) Commit

[조치 방법]

모든 Transaction이 Rollback 되므로 더 이상 조치가 필요하지 않으나 다만 Network Down된 상태에서 Connection을 시도하려고 하는 시간을 줄여서 곧바로 Error가 발생되도록 해야 한다.

5. Server Machine의 oracle가 Down된 상태에서 Connection을 하려고 하는 경우

: ORA-08108 error 발생

B. In-doubt 트랜잭션을 Manually Overriding

ORA-01581 Message가 발생하여 User가 조치를 요구하는 경우

In-doubt 트랜잭션이 Rollback Request의 Extension을 막는 경우

Network 및 System Failure가 장기화 될 경우

t. In-doubt 트랜잭션에 대한 정보를 얻는 방법

- a. User Feedback를 기록
- b. Local dha_2pc_pending View를 조회
- c. Local dha_2pc_neighbore View를 조회
- d. 정상적으로 통신이 복구된 후에 Mixed Outcome Flag를 Check

* Record User Feedback

DBA-01581: Lock held by in-doubt distributed transaction 1,21,17
1,21,17 : local transaction id

```
- Query dha_2pc_pending  
  < global_db_name format >  
global_database_name, hhhhhhh, local_transaction_id  
of ) global_database_name, global_coordinator_db_name  
      hhhhhhh-internal db id
```

- Query dha_2pc_neighbore

In-doubt 트랜잭션이 관 계어 있는 Connection에 대한 정보를 조회한다.

각 Connection에 대한 정보는 Connection@ Inbound, Outbound 이냐에 따라 다르다.

만일 Connection@ Inbound이면 그 Mode는 다른 Mode 의 Server이다. 만일 Connection@ Outbound이면 그 Mode는 다른 Server의 Client이다. Interface Column 은 Local Mode 혹은 Subordinate Mode가 Commit Point Bits 인지를 알려준다. 만일 Local Transaction Id와 Global Transaction Id가 일치하면 그 Mode 는 Global Coordinator이다.

- Check for Mixed Outcome

트랜잭션이 Manual로 Commit 혹은 Rollback된 후에 해당 Row가 Pending 트랜잭션 데이터에 남아있다. 그 트랜잭션의 상태는 "forced commit" or "forced abort" 로 변경되어지고 Instance 사이에 Connection이 복구되면 RECD는 트랜잭션의 Global Outcome를 Check하여 정상적인 경우 Row를 삭제하지만 물론 강제로 트랜잭션을 Force 시키면 삭제되지 않고 Mixed Column@ "Yes" 로 변경된다.

7. Manual Committing In-Doubt Transaction

```
COMMIT FORCE 'BABEL,ACME.COM,55d1c583,183,28';  
COMMIT FORCE 'global_id', 828381883  
                                (system commit number)
```

8. Manually Rolling Back In-Doubt Transaction

```
ROLLBACK FORCE '2,8,4';
```

9. Disabling and Enabling RECD

```
ALTER SYSTEM DISABLE DISTRIBUTED RECOVERY;  
ALTER SYSTEM ENABLE DISTRIBUTED RECOVERY;
```

10. Commit Point Bits 설정

- . 전체 Commit하는 Mode
- . In-Doubt Data를 가질 수 없다.
- . Prepared 상태로 될 수 없다.
- . 분산 데이터베이스에 참여하는 시스템 중 가장 안정된 시스템에 가장 높은 값을 준다.

DBMS PACKAGE : DEBUGGING PL/SQL PROGRAMS USING DBMS_OUTPUT

Bulletin no : 10281

Introduction

To aid in the development of PL/SQL programs, stored procedures, and triggers, Oracle provides a public package named **DBMS_OUTPUT** for displaying debugging information. The procedure in this package provides much the same functionality as the **printf()** function in C.

To create the **DBMS_OUTPUT** package, the script **DBMSOUTP.BQL** must be run by the user **SYS**. This script is run automatically by the **CATPACG.BQL** script.

Both of these scripts can be found in **\$ORACLE_HOME/rdbms/admin** on Unix systems, or in the **ORA_DBMS** directory on VMS. These scripts create the package - to run it you will also need execute privilege on it.

DBMS_OUTPUT and how it is used

The package is provided by Oracle to help programmers in debugging PL/SQL programs, procedures, and triggers. It is not designed to be used for report displaying, or formatting output to the standard output. The package is designed to display debugging info in stream-like processing. That is, the variables, string literals, and other info are written to a buffer. The buffer is divided into lines, each of which can have up to 255 characters. There is no other structure on the buffer. **DBMS_OUTPUT** is most helpful when used with **BQL*Plus** and **BQL*DBA**. In order to use the package with these two tools, you must set the 'serveroutput' before any info can be displayed to the screen. Use the following command in **BQL*Plus** or **BQL*DBA**:

```
BQL> set serveroutput on
```

Note that 'serveroutput' is one word. If serveroutput is not turned on, nothing will be displayed even if you call the **DBMS_OUTPUT.ENABLE** packaged procedure.

The way DBMS_OUTPUT works is as follows

In a PL/SQL program, calls to **DBMS_OUTPUT.PUT_LINE** will add lines to the buffer. Nothing is actually displayed until the program executes completely and control is transferred back to **BQL*Plus** or **BQL*DBA**. At this point, if serveroutput is on, **BQL*Plus** or **BQL*DBA** will make a call to **DBMS_OUTPUT.GET_LINES** and print the results to the screen.

If your PL/SQL program runs for a long time, you actually have to wait until the program executes and control goes back to **BQL*Plus** in order to see any information displayed by **DBMS_OUTPUT.PUT_LINE**. Consider the following example - I had to wait about 13 minutes before the output was displayed:


```

-- PL/SQL script out1.sql displays time in hh-mm-ss format at the beginning
-- of the script, loops to eat cpu time, and displays time at the end of the
-- script.
-- Calls to the DBMS_OUTPUT package are capitalized for emphasis - as with
-- all database objects they are really case-insensitive.

```

```

DECLARE
  p1      varchar2(80);
  p2      number;
BEGIN
  SELECT to_char(sysdate, 'hh-mm-ss')
    into p1
    from dual;

```

```

  DBMS_OUTPUT.PUT_LINE(p1);

```

```

  p2 := 0;
  LOOP
    IF p2 > 100000 THEN
      exit;
    END IF;

```

```

    p2 := p2 + 1;
  END LOOP;

```

```

  SELECT to_char(sysdate, 'hh-mm-ss')
    into p1
    from dual;

```

```

  DBMS_OUTPUT.PUT_LINE(p1);

```

```

END;
/

```

```

SQL> Out1
12-43-28
12-58-18

```

PL/SQL procedure successfully completed.

Notice how long the program took to execute. Both prints were done at the same time - they are not printed as the block is executing.

Calls to DBMS_OUTPUT.PUT_LINE are displayed even if a rollback was issued at the end of the PL/SQL script, the following example demonstrates this:

```

SQL> BEGIN
  2   dbms_output.put_line('test rollback');
  3   ROLLBACK;
  4 END;
  5 /
test rollback

```

PL/SQL procedure successfully completed.

Even if the script has an unhandled exception, all the calls to DBMS_OUTPUT.PUT_LINE before the unhandled exception are displayed. Example:

```
BQL> DECLARE
  2   x number;
  3 BEGIN
  4   dbms_output.put_line('test error');
  5   SELECT 'Junk' into x from dual;
  6 END;
  7 /
```

test error

declare

*

ERROR at line 1:

ORA-08502: PL/SQL: numeric or value error

ORA-08512: at line 5

Notice how the line was displayed regardless of the unhandled exception. It should be noted that in versions of SQL*Plus prior to 3.1.2.2, SQL*Plus did not call dbms_output.put_line to display the information in the buffer if an unhandled exception occurred.

Procedures in DBMS_OUTPUT package

procedure enable(bufferize IN default 2000);

This procedure enables calls to put, put_line, new_line, get_line, and get_lines. If the dbms_output package has not been enabled, calls to these procedures will be ignored. If you use the dbms_output.disable procedure in your PL/SQL block, you will have to call dbms_output.enable if you later want to re-enable calls to procedures put, put_line, new_line, get_line, and get_lines.

This procedure is used to change the default size of the buffer. The default and minimum size is 2000. Maximum is 1,000,000. When you set serveroutput on,

dbms_output.enable is implicitly called, with the default size of 2000.

Example:

```
BQL> set serveroutput on
```

```
BQL> BEGIN
  2   dbms_output.put('Example, show that enable is not needed ');
  3   dbms_output.put_line('when "set serveroutput on" is used. ');
  4 END;
  5 /
```

Example, show that enable is not needed when "set serveroutput on" is used.

PL/SQL procedure successfully completed.

procedure disable;

This procedure will disable all calls to put, put_line, new_line, get_line, and get_lines. It will also clear the buffer of all information. If you call the procedure after calling

dbms_output.put_line, nothing will be displayed. The SQL*Plus and SQL*Net command "set serveroutput off" does the same thing.

Example:

```
BQL> set serveroutput on
BQL> BEGIN
  2  dbms_output.put_line('Example, show that enable is not needed');
  3  dbms_output.disable;
  4  END;
  5  /
```

PL/SQL procedure successfully completed.

Notice that the buffer was cleared before the block ended, so nothing was displayed.

```
procedure put(a varchar2);
procedure put(a number);
procedure put(a date);
Input parameters:
  - a
  - line to buffer
```

This procedure puts the specified piece of information in the buffer. The procedure dbms_output.new_line or dbms_output.put_line must be called after one or more calls to dbms_output.put to end the line. A line that has not been terminated with a null character will not be displayed by a subsequent call to dbms_output.get_line.

```
procedure put_line(a varchar2);
procedure put_line(a number);
procedure put_line(a date);
Input parameters:
  - a
  - line to buffer
```

This procedure puts the specified piece of information in the buffer followed by an end-of-line marker.

```
procedure new_line;
```

This procedure places an end-of-line marker in the buffer.

```
procedure get_line(line OUT varchar2, status OUT Integer);
procedure get_line(line OUT chararr, nullterm IN OUT Integer);
Output parameters:
  - line
  - Holds the line to be displayed
  - status
  - This will be 0 upon successful completion of the call. A 1 means there are no more lines.
```

```

-- line
-- type chararr is a PL/SQL table defined in the DBMS_OUTPUT package.
-- type chararr is table of varchar2(255) indexed by binary_integer;
-- When using this type in your PL/SQL script, you should declare a
-- variable as
--   declare
--     var1 dbms_output.chararr;
--   If you redefine the type again, an Oracle error will be issued.
Input parameters:
-- null_line
--   The maximum number of lines to be returned.

```

These two procedures are used to get lines back from the buffer into varchar2 variables declared in the PL/SQL program. If you call put_line after calling get_line or get_lines, the buffer will be cleared and the only thing displayed is the information provided with call to put_line afterward.

The difference between get_line and get_lines is :
 get_lines can return more than one line from buffer.

Example:

```

SQL> set serveroutput on
SQL> declare
  2   p1 varchar2(80);
  3   p2 number;
  4 begin
  5   dbms_output.put_line('First line before dbms_out.get_line');
  6   dbms_output.put_line('Second line before dbms_out.get_line');
  7   dbms_output.get_line(p1,p2);
  8   dbms_output.put_line('Line after dbms_out.get_line');
  9 end;
10 /
Line after dbms_out.get_line

```

PL/SQL procedure successfully completed.

Notice how the buffer was cleared by the call to get_line, so the only line displayed is the argument to put_line after the get_line call.

Notes

1) As the package is implemented currently, there is no way to flush the buffer, print the contents, and then add new lines to it. This is because the buffer is not actually displayed to the screen until the block ends, and SQL*Plus or SQL*DBA calls get_lines and prints the buffer.

2) If you run into the error:

ORA-20000: ORA-10027: buffer overflow, limit of 2000 bytes just call enable with a larger buffer size - It can go up to 1,000,000. It is possible also to modify buffer size in SQL*PLUS and SQL*ORA by using the following command:

SET SERVEROUTPUT ON SIZE n

where n is an integer between 2,000 and 1,000,000.

3) There is little reason to use the procedure in DBMS_OUTPUT with the precompiler. You can make calls to put_line, but the buffer will not be printed to the screen automatically. You could call get_line to get data from the buffer back into host variables and display them using regular host language output statements, but in this case, it would make more sense to assign to the host variables directly and bypass the buffer entirely.

4) DBMS_OUTPUT is not designed to be a report writer. It is meant to be used for debugging. It is not really powerful enough for much more than this.

5) For more information, look at Appendix A of the Oracle7 Server Application Developer's Guide. The DBMSOUTP.BQL file also has descriptions of the package and the procedures.

DISK STRIPING

Bulletin no : 10153

1. 서로 다른 SIZE의 DATAFILE 또는 TABLESPACE 를 만든다.

```
$ sqlplus system/manager
SQL> CREATE TABLESPACE disk_etc
      DATAFILE '/disk1/fil1.dbf' SIZE 100M,
      '/disk2/fil2.dbf' SIZE 100M,
      '/disk3/fil3.dbf' SIZE 100M;
```

2. 'CREATE TABLE' 명령을 사용하여 disk_etc TABLESPACE 에 TABLE을 생성한다.

```
$ sqlplus scott/tiger
SQL> CREATE TABLE scottptab (
      c01 NUMBER(2),
      c02 VARCHAR2(10))
      TABLESPACE disk_etc
      STORAGE ( INITIAL 5M NEXT 5M
      MINEXTENTS 3 PCTINCREASE 0);
```

DOWN TIME WITH OPS

Bulletin no : 10043

I can only answer in a generic format. First, to understand how long fall over will take, you must first understand the different operations that takes place on fall over. My short answer would be, no, a 5 second average down time is not feasible. My long answer is, it depends:

- 1) How long does it take for the other node(s) to realize that a node in the cluster is down?
- 2) How long it takes to "reenter" the locks?
- 3) How long does it take for Oracle to perform instance recovery?

All of these play a part on how long fall over will take. My testing on fall over has been focused on Pyrexid's, but I believe it should be the same.

B1 is a tunable parameter - how often do you want each node in the cluster to send a heart beat across to the other nodes? On our machines, it's set to 20 seconds before cluster rebuild and reentering of locks occur if a node goes down.

B2 Only takes minutes at most. I've been testing reentering of locks any where between 1000 locks -> 200,000 locks and worst case scenario(reentering all 200,000 locks) has only taken less than 4 minutes. Setting gc_ah_locks = 1000 only took seconds to reenter the locks.

B3 is probably the most time consuming. It's really dependent upon how much redo has been generated and I don't have any timing statistics on this.

A few things that we need to realize is that not all the locks may need to be reentered - only the locks which were found "dubious" so the timing statistics I gave on that are worst case scenario's for reentering locks.

Also, there's a possibility of absolutely no wait time if the data block you're trying to access is cached by a node which is up and running.

DYNAMIC SQL(DBMS_SQL PACKAGE)

Bulletin no : 10172

DYNAMIC SQL 이란?

PL/SQL은 `binding`이 `compile`시에 이루어므로 `database object`의 `name`이 `compile`시에 고정되어야 하는 등의 제한이 있다. PL/SQL 2.1(DBMS 7.1)이후 `version`에서는 DBMS_SQL package로 Dynamic SQL statement의 사용을 가능하게 한다. 이는 `database object`의 `name`을 `runtime`에 줄 수 있을 뿐더러 SQL문장을 기술 할 수도 있는 장점이 있다.

Function open_cursor

: SQL문의 실행에 필요한 새로운 cursor를 열고 cursor ID number를 return 한다.

Function is_open

: 주어진 cursor가 현재 `open`되어 있으면 `TRUE`를, 아니면 `FALSE`를 return한다.

Procedure parse

: statement를 check하고 cursor와 결합시킨다.

Procedure bind_variable

: program내에서 data를 저장한 placeholder의 값을 제공하는 역할을 한다.

Procedure define_column

: cursor로부터 select된 column의 값을 받는 변수를 지정한다.

Function execute

: SQL문을 실행하고 처리된 Row의 수를 return한다.

(insert, update, delete인 경우에만 해당)

Function fetch_row

: cursor로부터 row를 fetch하고 실제로 fetch된 row의 수를 return 다. 이 row들은 `buffer`에 들어가며, `column_value`를 호출하여 읽어들이야 한다.

Function execute_and_fetch

: `execute`와 `fetch row`를 동시에 수행하고 실제로 fetch된 row의 수를 return한다.

Procedure variable_value

: 주어진 변수의 값을 return한다.

Procedure column_value

: `fetch_row`에 의해 fetch된 data의 값을 return한다.

Procedure close_cursor

: cursor를 닫는다.

Using the DBMS_SQL Package to Execute DDL statements:

< Example 1: >

procedure는 table을 create하는 것으로 table name과 column, type을 parameter로 받는다.

```
CREATE OR REPLACE PROCEDURE ddlproc (tablename varchar2,
    col1 varchar2) AS
    cursor1 INTEGER;
BEGIN
    cursor1 := dbms_sql.open_cursor;
    dbms_sql.parse(cursor1, 'CREATE TABLE ' || tablename ||
        ' ( ' || col1 || ' )', dbms_sql.v7);
    dbms_sql.close_cursor(cursor1);
END;
```

```
BQL> execute ddlproc ('MYTABLE', 'COL1 NUMBER, COL2
    VARCHAR2(10)');
```

PL/SQL procedure successfully completed.

BQL> desc mytable;

Name	Null?	Type
COL1		NUMBER
COL2		VARCHAR2(10)

DDL statement는 parse command에 의해 수행된다.

그러므로 DDL statement에서는 bind variable을 사용할 수가 없다.

다음은 DDL statement에 bind variable을 사용한 잘못된 예이다.

```
CREATE OR REPLACE PROCEDURE ddlproc (tablename
    VARCHAR2, colname VARCHAR2, coltype VARCHAR2)
AS
    cursor1 INTEGER;
    lgcurs INTEGER;
BEGIN
    cursor1 := dbms_sql.open_cursor;
    dbms_sql.parse(cursor1, 'CREATE TABLE :x1 (:y1 :x1)', dbms_sql.v7);
    dbms_sql.bind_variable(cursor1, ':x1', tablename);
    dbms_sql.bind_variable(cursor1, ':y1', colname);
    dbms_sql.bind_variable(cursor1, ':x1', coltype);
    lgcurs := dbms_sql.execute(cursor1);
    dbms_sql.close_cursor(cursor1);
END;
```

Procedure를 create할때에는 error를 만나지 않았지만, rustline에는 "ORA-00903: invalid table name" 이라는 error가 난다.

```
BQL> execute ddlproc ('MYTABLE', 'COL1', 'NUMBER');
begin ddlproc ('MYTABLE', 'COL1', 'NUMBER'); end;

*
ERROR at line 1:
ORA-00903: invalid table name
ORA-08512: at 'SYS.DBMS_SQL', line 238
ORA-08512: at 'SYS.DBMS_SQL', line 25
ORA-08512: at 'SYS.DBMS_SQL', line 8
ORA-08512: at line 1
```

< Example 2: >

table을 drop하는 procedure로 table name을 parameter로 받는다.

```
create or replace procedure droptable (table_name varchar2) as
    cursor1 integer;
begin
    cursor1 := dbms_sql.open_cursor;
    dbms_sql.parse(cursor1, 'DROP TABLE ' || table_name,
                    dbms_sql.v7);
    dbms_sql.close_cursor(cursor1);
end;
/
```

```
BQL> begin
2   droptable('MYTABLE');
3 end;
4 /
```

PL/SQL procedure successfully completed.

< Example 3: >

DDL statement를 수행하는 procedure로 DDL statement 자체를 parameter로 받는다.

```
create procedure sayddl (s1 varchar2) as
    cursor1 integer;
begin
    cursor1 := dbms_sql.open_cursor;
    dbms_sql.parse(cursor1, s1, dbms_sql.v7);
    dbms_sql.close_cursor(cursor1);
end;
/
```

```
BQL> execute myddl('CREATE TABLE MYTABLE (COL1 NUMBER)');
```

PL/SQL procedure successfully completed.

```
BQL> desc mytable;
```

Name	Null?	Type
-----	-----	----
COL1		NUMBER

```
BQL> execute myddl('drop table mytable');
```

PL/SQL procedure successfully completed.

Using the DBMS_SQL Package to Execute Dynamic SQL Statements:

DBMS_SQL package dynamic SQL statement을 수행하는데 이용되어 집 수 있는데 이는 runtime전에 statement의 일부분 혹은 전체를 알 수 없는 경우에 쓰여진다.

< Example 4: >

이 예제는 runtime에 주어진 number보다 더 높은 employee number를 가진 모든 employee 의 이름과 employee number를 return한다.

```
CREATE OR REPLACE PROCEDURE row_greater_than (low_value number)
AS
  cursor1 integer;
  row_processed integer;
  myname number;
  myname varchar2(20);
```

```
BEGIN
```

```
  cursor1 := dbms_sql.open_cursor;
  dbms_sql.parse (cursor1, 'select name, empno from emp
    where empno > :x', dbms_sql.v7);
  dbms_sql.bind_variable(cursor1, 'x', low_value);
  dbms_sql.define_column (cursor1, 1, myname);
  dbms_sql.define_column (cursor1, 2, myname, 20);
  row_processed := dbms_sql.execute (cursor1);
  loop
    if dbms_sql.fetch_row (cursor1) > 0 then
      dbms_sql.column_value (cursor1, 1, myname);
      dbms_sql.column_value (cursor1, 2, myname);
      dbms_output.put_line(to_char(myname) || ' ' || myname);
    else
      exit;
    end if;
  end loop;
  dbms_sql.close_cursor (cursor1);
```

```
EXCEPTION
```

```

WHEN OTHERS THEN
    dbms_output.put_line(sqlerrm);
    if dbms_sql.is_open (cursor1) then
        dbms_sql.close_cursor (cursor1);
    end if;
END;
/

```

DBMS_OUTPUT package를 사용하기 전에 먼저 SET SERVEROUTPUT ON command를 사용하여야 한다.

```

SQL> set serveroutput on
SQL> execute raise_greater_than(7500);
7521  WARD
7588  JONES
7854  MARTIN
7899  BLAKE
7792  CLARK
7799  SCOTT
7839  KING
7844  TURNER
7876  ADAMS
7900  JAMES
7902  FORD
8100  MILLER

```

PL/SQL procedure successfully completed.

```

SQL> execute raise_greater_than(8000);
8100  MILLER

```

PL/SQL procedure successfully completed.

< Example 5: >

이 예제는 where clause의 parameter를 column_name과 operator(<, <=, =, >=, >), min_value로 같아서 employees 테이블의 number를 return한다.

```

CREATE OR REPLACE PROCEDURE get_rows (column_name
varchar2, comparison_type varchar2, min_value number)
AS
    cursor1 integer;
    rows_processed integer;
    mynum number;
    mynum varchar2(20);
BEGIN
    cursor1 := dbms_sql.open_cursor;

```

```

dhsc_sql.parse (cursor1, 'select empno, name from emp
                        where ' || column_name ||
                        ' ' || comparison_type || ' ' || ix',
                        dhsc_sql.v?);
dhsc_sql.bind_variable(cursor1, 'x', new_value);
dhsc_sql.define_column (cursor1, 1, symname);
dhsc_sql.define_column (cursor1, 2, symname, 20);
row_processed := dhsc_sql.execute (cursor1);
loop
  if dhsc_sql.fetch_row (cursor1) > 0 then
    dhsc_sql.column_value (cursor1, 1, symname);
    dhsc_sql.column_value (cursor1, 2, symname);
    dhsc_output.put_line(to_char(symname) || ' ' || symname);
  else
    exit;
  end if;
end loop;
dhsc_sql.close_cursor (cursor1);
EXCEPTION
  WHEN OTHERS THEN
    dhsc_output.put_line(sqlerrm);
    if dhsc_sql.is_open (cursor1) then
      dhsc_sql.close_cursor (cursor1);
    end if;
END;
/

```

```

SQL> begin
2   get_row('EMPNO', '<', 2000);
3 end;
4 /
1111

```

PL/SQL procedure successfully completed.

```

SQL> execute get_row('BOL', '>', 3000);
7588 JONES
7788 SCOTT
7838 KING
7802 FORD

```

PL/SQL procedure successfully completed.

```

SQL> begin
2   get_row('DEPTNO', '>=', 20);
3 end;
4 /
7368 SMITH

```

```

7488 ALLEN
7521 WARD
7588 JONES
7854 MARTIN
7888 BLAKE
7988 SCOTT
7944 TURNER
7978 ADAMS
7800 JAMES
7802 FORD

```

PL/SQL procedure successfully completed.

< Example B: >

이 예제는 where clause 전체를 null값에 parameter로 받는다.

```

CREATE OR REPLACE PROCEDURE get_rows (where_clause varchar2) AS
    cursor1 integer;
    rows_processed integer;
    myname number;
    myname varchar2(20);
BEGIN
    cursor1 := dbms_sql.open_cursor;
    dbms_sql.parse (cursor1, 'select myname, name from emp where '
                        || where_clause, dbms_sql.v7);
    dbms_sql.define_column (cursor1, 1, myname);
    dbms_sql.define_column (cursor1, 2, myname, 20);
    rows_processed := dbms_sql.execute (cursor1);
    loop
        if dbms_sql.fetch_rows (cursor1) > 0 then
            dbms_sql.column_value (cursor1, 1, myname);
            dbms_sql.column_value (cursor1, 2, myname);
            dbms_output.put_line(to_char(myname) || ' ' || myname);
        else
            exit;
        end if;
    end loop;
    dbms_sql.close_cursor (cursor1);
EXCEPTION
    WHEN OTHERS THEN
        dbms_output.put_line(sqlerrm);
        if dbms_sql.is_open (cursor1) then
            dbms_sql.close_cursor (cursor1);
        end if;
END;
/

```

```
BQL> execute get_row('NAME = 'KING' ');
7838 KING
```

PL/SQL procedure successfully completed.

```
BQL> execute get_row('BAL > 1000 AND DEPTNO = 10');
7782 CLARK
7838 KING
8100 MILLER
```

PL/SQL procedure successfully completed.

< Example 7: >

이 예제는 anonymous SQL statement를 실행한다.

```
create procedure saysql (s1 varchar2) as
  cursor1 integer;
  return_value integer;
begin
  cursor1 := dbms_sql.open_cursor;
  dbms_sql.parse(cursor1, s1, dbms_sql.v7);
  return_value := dbms_sql.execute(cursor1);
  dbms_sql.close_cursor(cursor1);
end;
```

```
BQL> execute saysql('CREATE TABLE MYTABLE (COL1 number, col2 varchar2(3))');
```

PL/SQL procedure successfully completed.

BQL

```
BQL> desc mytable;
```

Name	Null?	Type
COL1		NUMBER
COL2		VARCHAR2(3)

```
BQL> execute saysql('INSERT INTO MYTABLE VALUES(1, 'ABC')');
```

PL/SQL procedure successfully completed.

BQL> begin

```
2   saysql(
3     'declare
4       var1 varchar2(3);
5     begin
```

```

8      select col2
9      into var1
10     from mytable
11    where col1 = 1;
12     dbms_output.put_line('var1 = ' || var1);
13 end;
14 /
var1 = ABC

```

PL/SQL procedure successfully completed.

~~~~~

더욱 자세한 사항은 Oracle® Server Documentation Addendum를 참조.

~~~~~

Oracle 3.0은 pl/sql 1.1만을 지원하므로 dynamic SQL을 쓸 수 없으며 stored procedure를 만들어 호출해서 사용해야 한다. 이때 Oracle가 부른 stored procedure나 function은 그 내에서 commit을 사용할 수 없게 되어 있기 때문에 COMMIT이나 ROLLBACK을 만나면 ORA-00034: Commit and Rollback from PL/SQL disabled for this session 이란 error가 난다.

그러므로 SQL statement도 사용될 할 수 없다.

ENTERPRISE BACKUP UTILITY

Bulletin no : 10385

1. The Problem

과거 메인 프레임 환경에서 사용하던 고객들이 저 비용, C/S환경으로 Migration시에는 주로 Open system과 Oracle DBMS를 채택하고 있다. 이런 환경으로 해당 제품 시장은 고 성장을 구가하고 있기도 하다.

이러한 고 성장을 가능케 하는 요소는 물론 H/W, S/W에서의 혁신적인 진보이나, 메인 프레임급에 걸 맞는 대용량 DB를 관리하고 Backup하는 데 필수적인 System management tool은 과거와 크게 달라진 바가 없다.

tar, cpio, dd같은 UNIX backup tool들은 실제 backup media의 관리 측면에서 걸 기여 를 하지 못하고 있다. 대용량 DB를 위한 Backup script는 source만 해도 수백line에 달하며 test와 troubleshooting에서도 많은 문제점을 갖고 있다. 뿐만 아니라 대용량 Database(이하 ULDB)의 Restore는 Backup보다도 훨씬 더 복잡한 문제를 갖고 있다.

최근의 Backup file이 어느 곳에 저장 되었는지를 알기 위해 Backup tape를 분류하고, damaged file들을 restore 해야 하는 등의 일련의 작업이 필요하다.

2. The Solution

최근 몇몇 media management vendor들은 media management기능, scheduling기능, 강력한 security기능, UNIX system에서 제공하는 backup tool이상의 신뢰성을 갖춘 강력한backup product를 공급하기 시작했다. 이러한 product들은 system level에서의 관리를 용이하게 해주는 장점은 있으나 이미 위에서 지적된 ULDB의 backup시 제기되는 문제점들을 완전히 해결해 주지는 못한다.

Oracle7 Enterprise Backup Utility(이하 EBU)는 이런 문제점을 해결하는 solution이다. EBU는 backup/restore의 media management product와 강력한 인터페이스를 제공하며 다른 어느것과 비교 할 수 없는 장점을 갖고 있다.

3. Introducing Oracle7 Enterprise Backup Utility

EBU는 open system상에서 메인 프레임급 성능을 제공하여 한 기업 전란에 걸쳐 전략적으로 중요한 역할을 담당하게 된다. 이 utility는 DB의 신속한backup과 restore를 위해disk와 backup된 media device에서H/W parallelism을 이용한다.open system의 full capacity를 이용하기 위해 device가 add 될 때 performance가 증가된다.parallel H/W, backup configuration check ,error detection and cleaning ,restore시의 database의block verification과 같은 특징들이EBU가 high reliability를 갖게 하는 특징들이다.

Oracle7 데이터베이스는 data files, control files, redo log files로 구성되는데 EBU는data file, control files를 backup하고 archived redo log files를 맡는다. (version2.0.8) Media failure의 경우 DB를 full restore/partial restore가 가능하며 data file도 선택적으로 최근의 last backup까지, 어느 밀접 시점 까지 의 선택이 가능하다. 또한 recovery도 밀접 시점으로 완전recovery가 가능하다.

4. Architecture

EBU는 크게 두 부분으로 구성되는데 backup하여 하는 DB에 관한 모든 current정보와history를 관리 하는 catalog부분, 그리고 실행 파일 부분으로 구성된다. 실행 파일들과 catalog는 반드시 같은

backt내에 존재할 필요는 없다. 실행 파일들은 다음과 같이 구성된다.

*obackup : 모든 다른 프로세스들을 monitor하며 backup하며하는 BB, catalog과
의 communication을 담당한다.또한 Instance Manager를 running시
킨다.

*kbrn : obackup, kbrk, kbrp사이 조정 역할을 수행한다.parallel I/O stream당
하나의 kbrn process가 발생한다.

*kbrk : Disk file의 read/write를 조절한다. File당 하나의 kbrk를 발생시키므로
만일 여러 개의 file들을 하나의 single data stream으로 결합시키는 경우 하
나의 I/O stream내에 여러 개의 kbrk process를 create한다.

*kbrp : Tape file들의 read/write를 조절한다. I/O stream당 하나의 kbrp create.

*kbrd : Instance Manager로써 backup catalog, obackup를 monitor하는
daemon process이다. backup을 위한 cleanup과 미정상 종료된 작업을 복구
한다.

Backup동안 database file들은 disk process인 kbrk에 의해 읽혀져 backup buffer area로 넘겨지고
그 후 third vendor의 media management s/w를 경유하여 backup device에 kbrp process에 의해 write
된다. Backt내에는 현재의 과정이 진행된다. 위의backup buffer는 hard disk와 tape device간에 발
생하는 I/O상의 speed mismatch를 조절하는 역할을 한다.

5. EBU의 장점(Benefits)

Third vendor의 media management product와 함께 사용되는 EBU는 기존의 UNIX 에서 제공하는
backup tool보다 다음과 같은 강력한 장점을 지닌다.

(1) 강력한 Reliability.

*. Consistent backup procedure

기존의 hot backup script의 경우 만일 datafile이 추가될 경우 script에 새로 지정
해 주어야 하며 ULBB의 경우 script만 해도 각file에 대하여 관리상 전거로운 계
사실 이였다. 그러나 EBU의 경우는 어느 datafile이 어느directory에 위치하는지를
작업자가 알 필요가 없고 script는 짧고 단순하여 누구나 쉽게 알아볼 수 있다.

또한 BB 구성이 자동으로 확인되며 backup media를 자동으로 인식 및 관리한다.

```
예).full online backup script      .partial online backup script
backup online database             backup online
db_name = "PRODDB"                db_name = "PRODDB"
oracle_sid = "PROD"               oracle_sid = "PROD"
                                   tablespaces = "TABLES"

.full restore script
restore database
db_name = "PRODDB"
oracle_sid = "PROD"
```

이와 같이 간단한 script에 의해 BB는 ULBB를 쉽게 backup과 restore를 할 수
있으며 작업에 필요한 세부정보는 backup catalog내에 있게 된다.

*. End-to-End checksum

Backup시 EMI는 각 backup data의 byte를 계산하고, restore시도 계산하여 정상 수행 여부를 판단 시 checksum 값을 상호 체크 한다.

*. Instance Manager

Instance Manager는 backup/restore시 진행 상황을 monitor하고 작업 중에 에러가 발생시 복구 및 각종resource를 reallocate시키는 기능을 담당하게 한다.

(2) High Performance

다음과 같은 EMI의 특징과 고속media device의 지원 아래 USB의 backup을 적은시간 내에 완전하게 수행 할 수 있게 되었다.

*. Parallel Hardware Capability

Backup/restore시 multiple device를 사용하여 동시에 여러 tablespaces에 대한 작업을 수행 할 수 있다. USB와 같이 다수의 tablespaces, datafiles이 여러 disk에 분산되어 있는 경우 이와 같이 물리적으로 다수의 device를 parallel하게 사용함으로써 performance를 극대화 시킬 수 있다. Backup시 동시 사용되는 device의 수와 performance는 정비례 하게 나뉘난다.

*. Multiplexing

EMI는 backup device가 최고의 속도로 backup을 수행 할 수 있게 script에 지정 할 수가 있다. 이것은 backup media access속도보다 disk access 속도가 훨씬 작은 경우에 여러 disk에 분산된 datafile들을 한device에 multiplexing되게 지정함으로써 가능하다.

*. Null Block Compression

EMI는 backup시 null data block를 skip시키고 restore시 재구성함으로써 공간을 절약하며 performance를 증가시킨다.

*. Buffered I/O

EMI는 disk I/O와 device I/O사이의 access speed의 mismatch를 이기 위하여 여러 모든 I/O stream을 buffering한다. 망목의 I/O는 shared memory buffer를 거치게 함으로써 backup/restore수행 중 항상 일정 속도를 유지하게 한다. shared memory는 user가 parameter로 구성할 수 있으며 기본적으로는 parallel data stream의 개수와 buffer size에 dependent하다.

(3) Availability

한 기업에서 전략적으로 중요한 DB들은 대부분 24시간*7일 full로 가동되며, 사용 중에 backup을 수행할 수 략에 없다. Offline full backup은 당연히 지원되며, EMI는 DB를 online상태로 full backup이 가능 며, 또한partial backup도 가능하다. 기존의 online backup이 backup으로 인해 performance를 저하시키는 단점이 있었으나 EMI는DB 운영에 최소의 영향만을 미친다.

Recovery시에도 full로 restore 시킬 수 있으며 나머지는 사용 중인 상태에서 필요 부분만 partial restore가 가능하다. 일단 EMI가 data를 restore시킨 후에는 기존의 Oracle의 표준 recovery process가 적용되어 진다.

(4) Usability

*. Auto-configuration

Online backup을 수행 시 EMI는 current DB configuration과 backup catalog

내의 가장 최근의 configuration정보를 비교하여 backup catalog정보가 더 오래 전의 것이면 EBU는 DB의 backup작전으로 update시킨다. Auto-configuration은 default이며 명시적으로 지정하여 disable시킬 수 있다.

Offline backup시는 backup script내에 backup command기술 전에 register라는 명령어를 기술 해야만 configuration정보를 update시킬 수 있다.

4. Light-Out backup

Third party의 media management s/w를 이용하여 EBU를 operator의 개입 없이 자동으로 수행이 가능하다. UNIX의 cron에서도 자동으로 수행되게 할 수 있다.

4. Raw device support

EBU는 raw device로 잡힌 DB도 operator의 특별한 조치 없이 일관 file system의 backup과 똑같이 수행할 수 있게 한다.

4. Aggregated restore

기업의 mission-critical system에서의 backup 전략들은 흔히 partial backup에 의존하게 된다. 이와 같은 환경에서는 backup data내의 모든file에 대한 current version을 검색하는 일이 중요할때 EBU는 이것을 자동으로 수행한다. partial backup data는 restore시 media failure이전에 취해진 가장 최근의 full backup data와 aggregated 되어져 이용된다.

4. Point-in-Time restore

EBU는 backup catalog내에 backup하더라도 DB에 대한 모든 configuration history를 갖고 있어서, 만일 하나의 tablespaces가 drop된 후 다시 필요하게 되면 비록 그 후 일정 시간이 경과되고 많은 전설이 일어났더라도 drop작전으로 일부분만 restore하여 사용이 가능하게 할 수 있다. 이 같은 tablespaces는 다른 machine에 restore 될 수도 있어 현재의 DB작동에 아주 영향을 미치지 않고 수행할 수가 있다.

4. Backup catalog

EBU에 의해 제공되는 backup catalog는 모든 backup대상 DB의 configuration과 history의 저장소로서 제공된다. backup catalog에 저장된 정보는 backup된 file, tablespaces 그리고 multiplexing정보, 마지막backup이나 restore의 시간기록, backup type(full or subset), backup들이 저장 되어 있는 file set들을 포함하고 있다. 하나의 backup catalog는 기업 내 모든 분산된 DB들을 관리할 수 있다.

4. Dry Run

UIEBU를 backup시 켜는 문제 중 하나는 과연 script가 정상적으로 수행이 완료되는 지를 검증하는 것이다. 특별한 장치가 없는 한 긴 시간 동안 수행시켜 끝난 후 확인 할 수 밖에 없는데 EBU는 I/O없이 간단히 test로 수행을 시킬 수가 있다. 이로 인해 긴 처성 있는 backup전략을 DB가 수립 가능하게 한다.

4. Enterprise-wide backup

이것은 backup이 단순히 single database에 국한되지 않고 한 기업 전체에 걸쳐서 global하게 수행 가능함을 말한다. 예로 전국적으로 다수의 site에 database가 구축 되어진 상황에서 각database들이 two phase commit, snapshot 같은 분산처리 작업을 통해 다른 database들을 synchronous하게 또는 asynchronous하게 update할 경우 기업 전체를 하나의 logical database로 보고 이런 logical database를 중앙에서 EBU를 통해 backup시킬 수 있다. EBU는 중앙의 catalog내에 모든database

들에 대한 backup정보를 간직하고 있다.

이와 같은 용량의 한 지점을 통하여 기업 전체 모든 database들에 대한 backup수행 기능은 EMI의 가장 큰 장점 중 하나이며 이런 경우 H/W상에 대량의 data가 전송되어 network의 bandwidth가 backup의 중요한 요소로 부각된다.

6. Flexibility

*. EMI API

EMI는 앞서서도 소개되었듯이 단독으로 사용되는 것이 아니고 third-party에서 제공하는 media management product와 같이 사용된다. 이때 두 product사이의 interface는 backup/recovery Application Programming Interface(API Glue)에 의해서 이루어 진다. 이것은 third party에 의해 제공되며 EMI Installer에 의해 자동적으로 access된다.

*. Media management products

Epoch의 EpochBackup, IBM Adstar의 Distributed Storage Manager, Legato ByteStream/NetWorker, HP의 DataBackup, Spectralogic의 Alexandria, StorageTek의 BKL, etc.

*. Media device

4mm and 8mm DATB, 3490x, 3480x, UD80, Writable Optical Devices and automated systems such as stacks and cllow.

7. Pricing

EMI는 주상으로 공급되며 사용을 위해서는 third vendor의 media management product를 구입하여 설치하여야 한다.

E7XP/IMP 를 이용하여

SCOTT의 데이터를 LARRY로 옮기는 방법

Bulletin no : 10188

scott의 데이터를 larry로 옮기려면 export/import를 이용해야 한다.
larry가 만들어져 있지 않다면 다음과 같이 만든다.

```
$sqlplus system/manager
```

```
SQL>create user larry
      identified by lion
      default tablespace users
      temporary tablespace temp
      quota unlimited on users;
```

default tablespace, temporary tablespace는 시스템에 따라 알맞게 설정한다.

다음에 권한을 부여한다.

```
sql>grant connect,resource to larry
sql>revoke unlimited tablespace from larry
```

출문 users 데이터스페이스는 이미 존재한다고 가정한다.

scott로 익스포트를 하고 larry로 임포트 한다.

```
$exp scott/lion owner=scott files=scott.Boy
$imp larry/lion fromuser=scott touser=larry files=scott.Boy
```

만약 scott가 dba 권한을 갖고 있었다면 다음과 같이 임포트를 해야 한다.

```
$imp system/manager fromuser=scott touser=larry files=scott.Boy
```

만약 임포트 도중 데이터 영역이 부족해서 예외가 발생한다면 익스포트를 할 때 compress 옵션을 사용하며, 임포트시 쿨락 세그먼트 문제가 발생한다면 임포트시 commit 옵션을 사용하면 쉽게 해결이 가능하다.

EXPLAIN PLAN 사용법

Bulletin no : 10312

Explain plan 이란 DB가 이 요구한 데이터를 추출하기 위해 어떠한 작업이 차례로 수행하는 작업 방법을 말한다.

1. Plan_table 생성

explain plan 을 sql 에 포함해서 수행하면 물리마이저가 실행 계획과지만 수립하여 plan_table 에 저장해 준다. 이 table 을 생성하기 위한 script는 ?/rdhmc/admin/utlxplan.sql 이다.

2. index 생성

데이터를 생성후 수행 속도 향상과 동일한 statement_id 가 생성되는 것을 방지 하기 위해 index 을 생성한다.

```
create unique index plan_index on plan_table(statement_id,id);
```

3. SQL 문 사용

```
EXPLAIN PLAN SET STATEMENT_ID='s1' FOR  
SELECT .....
```

4. Plan_table 을 select 하는 sql 문

```
Select lpad(operation, length(operation)+ 2*(level-1)) ||  
       decode(id,0,'cost estimate: ' ||decode(position,'0',  
       'N/A',position),null) || ' ' ||options ||  
       decode(object_name,null,null,':') ||lpad(object_name,  
       length(object_name)+1,' ') || object_name ||  
       decode (object_type,'UNIQUE' , '(U)' , 'NON_UNIQUE',  
       '(N)') || decode(object_instance,null,null,'(' ||  
       object_instance || ')')  
FROM PLAN_TABLE  
START WITH ID= 0 and STATEMENT_ID = 's1id'  
CONNECT BY prior ID=PLAN_ID and STATEMENT_ID='s1id'
```

Explain plan 문의 'statement_id =' 에서 부여한 제목을 'id'에 지정하고 다음처럼 실행한다.

```
sql> desc id = s1  
sql> explain
```

EXPLAIN PLAN 을 자동으로 수행하는 방법

Bulletin no : 10353

다음은 `sqlplus` 에서 자동으로 `explain plan` 을 creation 하는 방법이다. 이 방법은 `explain` 을 실행하는 구문을 기억할 필요없이 현재 수행하는 `sql` 문장을 쉽게 체크해 볼 수 있습니다.

다음은 script 의 내용입니다.

```
save into expl          -- Save current script
truncate table plan_table; -- Clean Plan Table
get tota script         -- Restore script

1                          -- setup explain statement
ch //explain plan set statement_id='MyTest' for /
/

                                -- Col
select lpad(' ',2*(level-1))||operation|| '||
       option|| '||object_name||
       '||object_type|| '||object_instance|| '||
       decode(id,0,'Cost = '||position) "Query Plan"
from   plan_table
       start with id = 0 -- start with statement_id = 'MyTest'
       connect by prior id = parent_id;

get tota script          -- Restore script
```

EXAMPLE:

위의 script 를 'plan.sql' 라는 file 로 저장한후 `sqlplus` 를 login 한다.

BQL) select * from dept;

DEPTNO	NAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

BQL) Desc

Write file tota

Table truncated.

1* select * from dept

1* explain plan set statement_id='MyTest' for select * from dept

Explained.

Query Plan

```
-----  
SELECT STATEMENT      Cost = 1  
  TABLE ACCESS FULL DEPT 1
```

Script 는 plan.sql 로 저장 되어 있으므로 user 는 buffer 를 query 해서 어느 정도의 수정을 가할 수 있다.

```
SQL> 1  
1* select * from dept  
SQL> 1  
2 where deptno =20;
```

DEPTNO	NAME	LOC
20	RESEARCH	DALLAS

```
SQL> Dplan  
Write file to
```

Table truncated.

```
1* select * from dept  
1* explain plan set statement_id='MyTest' for select * from dept
```

Explained.

Query Plan

```
-----  
SELECT STATEMENT      Cost = 1  
  TABLE ACCESS BY ROWID DEPT 1  
    INDEX UNIQUE SCAN DEPT_PRIMARY_KEY UNIQUE  
-----
```

EXPORT 받은 화일의

CHARACTER SET 변경 프로그램

Bulletin no : 1015B

```

/*****
 *
 *      convdat : Data Conversion Program
 *
 *      . converts exported files in different db-character
 *      to specific db-character files.
 *
 *****/

#include
#include
#include
#include

int ue?_to_kcc, kcc_to_ue?; /* conversion flag */

main(argc,argv)
int argc;
char *argv[];
{
    ue?_to_kcc = kcc_to_ue? = 0;
    if(argc != 3) {
        usage(argv[0]);
        exit(1);
    }

    if(strcmp(argv[2], "EBCS801"))
        ue?_to_kcc = 1;
    else if(strcmp(argv[2], "U87ASCII"))
        kcc_to_ue? = 1;
    else
        usage(argv[0]);
    data_conversion(argv[1]);
    printf("conversion completed successfully !!!\n");
}

usage(prog_name)
char *prog_name;
{
    printf("usage: %s exported-file-name character\n", prog_name);
    printf("      - character : EBCS801 U87ASCII\n");
}

```

```

printf("      , EBC5B01 - convert uc7uc011 format to kcc5B01 format\n");
printf("      , UB7AB011 - convert kcc5B01 format to uc7uc011 format\n");
}

data_conversion(ucf11a)
char *ucf11a;
{
    int fd;
    char conv_buffer[2];
    if((fd = open(ucf11a, 2)) < 0) {
        perror("open");
        exit(1);
    }
    if(!seek(fd, 1, 0) < 0) {
        perror("lseek");
        exit(1);
    }
    if(uc7_to_kcc){
        conv_buffer[0] = 3;
        conv_buffer[1] = 'H';
    }
    else if(kcc_to_uc7){
        conv_buffer[0] = '\0';
        conv_buffer[1] = 1;
    }
    if(write(fd, conv_buffer, 2) != 2) {
        perror("write");
        exit(1);
    }
    close(fd);
}

```

EXPORT 받은 DUMP파일의 CHARACTERSET 확인 방법

Bulletin no : 10329

다른 server에서 export받은 dump 파일을 import시킬경우 자주 겪게 되는 문제가 character set에 관련된 것이다. 양쪽의 character set이 같고 profile의 환경 변수(MLB_LANG)도 맞다고 생각되는 데도 import시 segment fault가 발생되거나, log-18, 38, 39, 39, 42, 43, 45, 48등의 error가 발생되면 dump받은 파일을 의심해볼 필요가 있다.

이때 dump 파일을 8진수로 변환해 보면 현재 export받은 파일의 character set을 쉽게 알 수 있다.

모든 export 파일은 특정한 character set을 사용해서 쓰여진다.

version5, 6에서는 export시 단지 두개의 character set-ASCII와 EBCDIC-만이 지원 되었으나, version7에서는 지원하는 모든 character set이 export시에 사용될 수 있다. export시 사용된 character set은 dump 파일내에 code로써 기술하는데 파일의 첫 byte가 이것을 나타낸다.

1st byte 의미

1	version5, 6의 ASCII character set
2	version5, 6의 EBCDIC character set
3	version7의 character set.

“3”일 경우 두, 세번째 byte를 보고서 character set을 판단한다. 이때 2nd byte 보통 000이며 아래 예는 우리가 흔히 접하는 character set에 대한 character set ID이다.

ID	value
----	-------

1	US7ASCII
2	US8EBC
31	US818080SP1
37	US180805801
:	:
:	:

export 파일을 체크하기 위해서는 다음의 UNIX command를 사용한다.

```
$ od -t export.dmp | more
```

```
0000000 003 000 001 105 130 120 117 122 124 072 128 080 087 058 080 082
0000020 058 080 082 012 104 112 115 113 111 115 012 122 124 101 102 114
:
```

위의 경우 1st byte가 003이므로 version7이며 3rd byte가 001이므로 ID가 1인인 US7ASCII가 character set이 된다.

EXPORT 실행시 ORA-1403 조치 방법

Bulletin no : 10104

EXPORT 실행시 ORA-1403이 발생되는 경우가 있는 데, 이 예외는 데이터를 혹은 인덱스에 문제가 있는 경우 발생할 수 있다. 여기서는 RDM1B를 사용하여 데이터를 복구시키는 방법을 소개한다.

1. 기존 데이터와 같은 구조를 갖는 데이터를 만든다.

```
SQL> CREATE TABLE TEMP
      AS SELECT *
      FROM EMP
      WHERE 1=2;
```

2. 기존 데이터에서 RECORD를 FETCH하여 새로운 데이터에 입력.
이 때, INDEK가 설정되어 있는 COLUMN을 WHERE 조건에 부여함.

```
declare
row_id char(18);
cnt number;
cursor c1 is select rowid
      from emp
      where empno>0;

begin
  cnt := 0;
  open c1;
  loop
    fetch c1 into row_id;
    insert into temp select * from emp where rowid=row_id;
    cnt:=cnt+1;
    if cnt = 100 then      /* commit per 100 records */
      commit;
      cnt:=0;
    end if;
  exit when c1%notfound;
  end loop;
  commit;
end;
```

3. commit.sql 파일이랑

```
sql> @commit
```

EXTENT 된 영역의 FREESPACE

Bulletin no : 10241

1. TABLE 에 할당된 SEGMENT 의 크기

```
BQL> select bytes SEGMENT_SIZE  
      from user_segments where segment_name='EMP';
```

2. TABLE 이 사용하고 있는 영역(ORACLE BLOCK 단위)

```
BQL> select count(distinct(substr(rowid,1,8)||substr(rowid,15,4)))  
      * 2048 TABLE_SIZE from emp;
```

3. TABLE SEGMENT 의 FREESPACE

FREE_SPACE = SEGMENT_SIZE - TABLE_SIZE

FOREIGN KEY CONSTRAINT의 MASTER TABLE NAME의 확인

Bulletin no : 10081

FOREIGN KEY CONSTRAINT 이름으로 REFERENCE 하는 TABLE(MASTER TABLE)을
찾는 SQL SCRIPT

```
col Primary_key_table format a20
col Constraint_name format a20
select a.object_name Primary_Key_table,
       c.name          Constraint_name
from dba_objects a,
     constraint b,
     constraint c
where c.name = 'EMP_FOREIGN_KEY' -- CONSTRAINT NAME
      and b.constraint = c.constraint
      and b.parent = a.object_id
/
```

FREE LIST와 FREE SPACE 관리

Bullittin no : 10103

* Free List 는 해당 세그먼트가 다시 사용할 수 있는 block에 관한 List 이다.

Free Block이 추가될 때는 Free List의 Head에 덧붙여진다. 예를 들면 Block B5가 Free List FL에 있고 다른 BB block이 FL에 덧붙여질 때

FL : BB → B5

즉 BB 가 free block으로 먼저 쓰여진다.

앞으로 사용할 막대 : U1(UB.0 ~ UB.0.34)

U2(UB.0.35 ~ U7)

* Free List에는 다음과 같은 것들이 있다.

. Txfl(Transaction Free List) : 모든 트랜잭션에 할당되는 Free List이다.

한 세그먼트에 최소 1B Txfl 이 있고 이는 블록이 작아질 때 까지 필요한 만큼 증가한다. 모든 프로세스는 해당 인스턴스에 주어진 Txfl에 접근 할 수 있다.

. Pefl(Process Free List) : 하나의 프로세스에 할당되는 Free List 이다.

하나의 프로세스는 다른 프로세스에 할당 되어진 Pefl 을 사용하지 않는다.

. Bgfl(Begumt Free List) : Pefl과 같다.

. Hxfl(Master Free List) : 모든 프로세스가 사용 할 수 있도록 미리 할당 되어진 Pefl이다.

* 세그먼트는 연속된 블록의 모음인 익스텐트로 구성 되어 있다. 세그먼트가 생성될때 첫번째 익스텐트의 첫번째 블록을 Begumt Header라고 한다.

Begumt Header는 Extent Control, Extent Map Table, Begumt Header, Begumt Free Space, Transaction Free Space로 이루어져 있다. Extent Control과 Extent Map Table은 세그먼트에 할당된 익스텐트에 관한 정보를 준다. Begumt Header Control은 Transaction Free List 정보와 세그먼트가 생성 될 때 명시된 Instance List(nfl)과 Process Free List(nfl)의 갯수를 가지고 있다.

* Instance Free List(nfl) 갯수는 디폴트가 1이며 1보다 클 경우 Pefl의 갯수는 Pefl/Bgfl = nfl * nfl 이다. nfl은 Init.ora 의 free_list_init, nfl은 free_list_proc 에서 지정된다.

* UB.0.35에서는 기존의 Block Type 5에 Block Type 12가 추가됨으로써 Begumt Header는 Type 5 또는 12가 가능하다. 이의 구별은 명시된 Free Group의 갯수에 의해 결정된다. 즉, Free Group의 갯수가 1보다 크면 Begumt Header의 block Type은 12로 설정된다.

* UB.0.35 이상에서는 Free List의 관련 정보를 Storage 항목에서 표시할 수 있다.

....storage (freelists = freelist groups n)

강점은 데이터 디서너리에 Free List의 정보가 저장 되어진다는 것이다. 즉 U1에서 서랍 Free List를 바꾸기 위해 데이터베이스를 Restartup 할 필요가 없다.

* U1에서는 Freelist를 위해 Begumt Header인 한 블록이 할당된다. 그러나 U2에서는 각 Free List Group에 한개의 Free List Block이 존재한다. 예를 들어, N=2 이면 Begumt Header와 다음 2 block이 할당 되어지는데 각 블록은 Free List Group을 위한 것이다. (여기서 N은 Free Group이다).

< Mapping Function 과 Free Space 검색 알고리즘>

(Mapping Function)

* Mapping Function은 단순히 Thread Number $\times$ Freelist Group과 Process Number $\times$ Process Freelist이다. 예를 들어 N=3,M=5는 3개의 Free List Group과 각각은 5개의 Process Free List를 가짐을 의미한다.

1(1 2 3 4 5) 2(1 2 3 4 5) 3(1 2 3 4 5)

. 임의의 프로세스가 인스턴스 2 에 접속되어 있고 Thread Number는 10 이고 Process Number는 28 이라고 가정하면 어떤 Freelist 가 할당되어질까?

이런 경우에는

$$10 \div 3 = 1$$

$$28 \div 5 = 1$$

즉, 첫번째 그룹의 첫번째 Pool 을 선택하게 된다.

* 만약 인스턴스를 특정한 Freelist Block에 할당하고자 할 때는 Init.ora Parameter 인 Instance_number를 이용한다.

(검색 알고리즘)

* 9.0.4 이하에서는 다음 순서를 따른다.

1. 자신의 Transaction Free List에서 Free List를 찾는다. 찾으면 B7, 아니면 B2로 간다.
2. Process Freelist 의 Free Space 을 찾는다. 찾으면 B7, 아니면 B3 으로간다.
3. Commit된 Txlist를 찾는다. 만약 Commit되지 않은 Txlist Space를 발견하면 B4로 가고 아니면 각 Commit된 Txlist Space 을 한 Pool에 할당한다. 만약 Pool > Txlist 이면 어떤 Pool은 Space를 넘지 못하고 Pool < Txlist 이면 어떤 Pool은 보다 많은 Space를 넘으며 B2 로 간다.
4. High Water Mark를 Bump Up할 수 없으면 B5로 가고 가능하면 원래대로 Space를 Pool에 특정한 후 B2로 간다.
5. 데이터 디서너미에서 fat8을 검색하고 익스텐트를 세그먼트에 할당하고 B4로 간다. 만약 fat8 에 Space 가 없으면 B8로 간다.
6. 에터할당. 데이터스페이스에 더이상 Space 가 없다.
7. Space 을 사용한다.

* 9.0.35 이상에서는 다음 순서에 의한다.

1. 자신의 Transaction Free List에서 Free List를 찾는다. 찾으면 B8, 아니면 B2 로 간다.
2. Process Freelist 의 Free Space 을 찾는다. 찾으면 B8, 아니면 B3 으로 간다.
3. Master Freelist를 검색한다. 만약 Space를 찾으면 Pool에 Space의 조각을 특색하고 B2, 아니면 4 로 간다.
4. Commit된 Txlist를 찾는다. 만약 Commit되지 않은 Txlist Space를 발견하면 B5, 아니면 각 Commit된 Txlist Space 을 Master Freelist 에 특색하고 B3으로간다.
5. High Water Mark 를 Bump Up할 수 없으면 B8으로 가고 가능하면 Bump Up하고 Space를 Pool 에 특색한 후 B2로 간다.
6. 데이터 디서너미에서 fat8을 검색하고 익스텐트를 세그먼트에 할당하고 B4로 간다. 만약 fat8 에 Space 가 없으면 B7로 간다.
7. 에터 할당 데이터스페이스에 더 상 Space 가 없다.
8. Space 을 사용한다.

FULL TABLE SCAN을 보다 빠르게 하는 방법

Bulletin no : 10102

* `db_file_multiblock_read_count` 는 Full Table Scan 시에만 영향을 미치는 파라미터이다. 오라클의 최대 I/O 크기는 84KB 이므로

`db_block_size * db_file_multiblock_read_count <= 84KB`
이어야 한다.

Index를 이용한 Query 인 경우에는 Full Table Scan 과는 상황이 달라진다. 이 때에는 `db_block_size`의 설정이 중요하다. 오라클은 원하는 블록 크기를 선택할 수 있으며 대부분의 유닉스 환경에서 최대 8KB (시퀀스는 16KB)까지 가능하다. 같은 크기의 B-tree Index 에서 블록크기가 8KB 라면 2KB 인 경우보다 거의 4배 많은 데이터를 읽어들이게 된다.

한편 Index Block 의 `Pctfree`도 중요한 의미를 갖는데 이 값이 너무 크고 인덱스의 Entry Block 가 늘어나지 않으며 기존의 인덱스 값에 새로운 레코드가 더 이상 추가되지 않는다면 공간의 낭비가 심하게 되므로 적절한 값으로 설정해야 한다.

`db_file_multiblock_write` 파라미터는 인덱스의 재구성시 퍼포먼스에 별로 영향을 끼치지 않는다.

HIERARCHICAL QUERIES

Bulletin no : 10325

START WITH root row of the hierarchy
CONNECT BY parent row 와 child row 의 관계를 맺어준다.
 (parent row의 children 들 찾기위해 PRIOR operator를 사용)

예제 1) 다음은 전 사원의 구성도 hierarchical order 에 따라 select 하는 방법이다. 각 사원번호는 manager 번호와 연결되어 있다.

```
select name, mname, mgr, job
from emp
start with job= 'president'
connect by prior mname= mgr ;
```

예제 2) 테이블이 아래처럼 구성되어 있고 데이터가 들어있다고 가정할때 Tree가 Dynamic하게 변화하여도 값을 가져오기 위한 SQL문은 다음과 같다.

Tree Table :

```
create table Tree (name varchar2(2), parent varchar2(2));
insert into Tree(name , parent)
values('a', NULL);
insert into Tree(name , parent)
values('b', 'a');
insert into Tree(name , parent)
values('c', 'a');
.....

select name
from tree x
where not exists (select name
                  from tree y
                  where x.name = y.parent)
start with name = 'b'
connect by prior name = parent
```

만약 Parent만큼 구하고자 한다면 위의 SQL을not exists 를 exists로 바꾸어주면 된다.

HP-UX: ASYNCH I/O USING FS_ASYNC KERNEL PARAMETER

Bulletin no : 10037

This type of I/O caching is also found in numerous third-party disk cache products, which allow disabling "write-through". Since Oracle cannot tell if an I/O actually made it TO DISK, this is not supported on any OS, including OpenVMS. This is the same reason why using a RAM disk is not supported....

Use of FS_ASYNC is very dangerous and should not be used on any database that is not a toy. This kind of async, means that the Oracle writes to disk are not guaranteed to have gotten to disk. This can corrupt your database if it crashes before the data is written to disk, when Oracle thinks it already has.

This is not a new HP-UX feature. I believe it may have existed all the way back to 4.2BSD to support extremely fast but unreliable writes on old 1 Mip Vaxes.

The Async I/O which requires raw devices makes use of a special HP device driver which allows writes and reads to be "batched" so we can do a lot of writes with one system call. The device driver kicks off multiple I/O operations at a lower level where disk scheduling can be optimized for maximum parallelization, and minimum OS overhead. This async device driver only works with raw devices, i.e. raw disks or raw logical volumes. This is an HP restriction. We wish we could use it on file systems as well. Oh, yes, and this type of async is safe. Oracle is reliably notified if the data made it all the way to disk or not.

INDEX 생성시 필요한 TEMP SEGMENT 크기 추정

Bulletin no : 10215

새로운 인덱스를 저장하기 위해 필요한 TEMPORARY SEGMENT SPACE의 양은 다음 공식을 사용하여 추정가능 하다.

$1.3 * key_storage$

where : $key_storage = (number_of_rows) * (10 + sum_of_column_sizes + number_of_columns)$

- COLUMNS 는 INDEX 에 있는 COLUMN 들이다.
- NUMBER_OF_COLUMNS 는 COLUMN 당 1 LENGTH Byte.
- 10 은 ROWID 와 추가적인 OVERHEAD

* 1.3 은 ROWING 을 위해 필요한 추가 메모리의 계량치. 이 값은 메모의 순서로 된 데이터에 대해서 가장 적당한 값이고 만일 데이터를 완전히 정렬대로 ROWING 하면 주위의 메모리가 필요하다.

만일 데이터가 완벽하게 ROWING 되어 있으면 오직 INDEX 를 저장하기 위한 SPACE 만 필요하고 이때 CONSTANT 값은 1 이 된다.

INDEX__DESC HINT 사 용 법

Bulletin no : 10309

- INDEX_DESC hint 사용법

```
select /*+ index_desc(table_alias_name index_name) */
      key
from   table_name table_alias_name
where  key1 = :k1
and    rownum = 1

(index_name : key1key)
```

* SINGLE TABLE QUERY에서는 HINT절에 TABLE명을 주는것이 원칙이다.

* HINT절의 table_name은 alias를 사용하였을 경우에는 반드시 table alias name으로 지정하여야 한다. (이는 Version 7.2 / 7.3부터 적용 된다.)

* 그 이유는 다음과 같이 BQLE이 사용 될 경우도 있기 때문이다.

(SELF-JOIN)

```
select /*+ index_desc(a a_pk) */
      *
from   t1 a, t1 b
where  a.key1 = 'a'
and    a.key1 = b.prev_col(*)
and    a.key = b.prev_col(*)
and    rownum = 1
```

위와 같이 동일한 TABLE이 FROM절 뒤에 연속으로 나오는 경우(RECURSIVE 관계)가 있을 수가 있는데 이때 HINT절의 table명을 동일하게 줄수가 없으므로 반드시 table alias name을 주어야 원하는 답이 나온다.

INSTR 함수의 의미와

특정 CHARACTER 를 지우는 방법

Bulletin no : 10324

1) INSTR function 의 의미

입력의 character 의 position 을 찾아내는 function 이다.

```
INSTR(char1, char2[, n[, m]])
```

char1 의 n 번째 position 부터 m 번째 char2 의 position 을 찾는 function 이다. 만약 m이 음수이면 char1 의 뒤 더 찾으라는 의미이다.

eg) select INSTR('corporate floor', 'or', 3, 2) "instring" from dual;

```
instring
-----
      14
```

2) 특정 character 를 지우는 방법(position 을 모를때)

같은 위치에 있지 않은 특정 character 를 지우고 select 하기를 원할때는 다음과 같은 sql문을 이용한다.

```
SELECT DECODE(SUBSTR(col, INSTR(col, ',', 1), ','),
SUBSTR(col, 1, INSTR(col, ',')-1)||SUBSTR(col, INSTR(col, ',')+1),
col) FROM test;
```

이 문장은 col 에서 comma 의 위치를 알아내어 그 comma를 제외한 data 값만을 select 하는 문장이다.

3) 특정 character 의 position 을 알때

특정 character 인 0 의 position 이 col 의 1번째 위치임을 알고 이 0을 제거하고 select 하고자 할때는 다음과 같다.

```
SELECT DECODE(SUBSTR(col, 1, 1), '0', SUBSTR(col, 2, col) FROM
table;
```

LAST ORA-1578 RECOVERY METHOD

Bulletin no : 10058

Subject: 최후의 ORA-1578의 처리방법

일반적으로 ORA-1578 : ORACLE data block corrupted (file # num, block #num)은 해당 object를 drop하고 recreate하여 처리 할수 있습니다. 하지만, backup이 만되어 있거나 되어 있더라도 시점에 문제가 생기면,당황할 수 밖에 없습니다. 따라서,손상된 db block만을 제외한 다른 block들을 recover하는 방법을 알려 드립니다.

1. 손상된 table과 똑같은 name의 new table생성

2. 아래의 pl/sql 실행

```
/*
** This program uses a cursor to select the ROWID from INDEX
** for the corrupted table. And insert the all of the normal rows
** into the new table.
**
**
*/

DECLARE
CURSOR c1 IS
SELECT ROWID FROM corrupted_table
WHERE indexed_column > smallest_value_of_the_index
AND SUBSTR(ROWID,1,8) != '0000050A';

/* Here, the value of '0000050A' is displayed in decimal
when ORA-1578 error happens. */

rowid_value CHAR(18);
count_value NUMBER;

BEGIN
OPEN c1;

FOR i IN 1..total_number_of_the_rows_from_corrupted_table LOOP
FETCH c1 INTO rowid_value;
EXIT WHEN c1%NOTFOUND;
INSERT INTO new_table
SELECT * FROM corrupted_table WHERE ROWID = rowid_value;
count_value := count_value + 1;
IF count_value = 10000 THEN /* Let's think commit per 10000 rows */
COMMIT;
count_value := 0;
END-IF;
END LOOP;
```



```

END LOOP;

CLOSE c1;
END;
/

```

3. new table의 data건수를 확인후에 기존 table을 다른 이름으로 rename한 후
new table을 original table name으로 rename

```

SQL> rename corrupted_table to table_xxxx;
SQL> rename new_table to corrupted_table;

```

4. 필요한 index생성이나 constraint make

5. 손상된 row에 대한 내용은, 최소한 primary key나 indexed column에 대한 내용들

```

SQL> spool chk.log
SQL> select rowid, indexed_column1, indexed_column2, indexed_column3...
       from corrupted_table
       where indexed_column1 > smallest_value_of_the_index;
SQL> spool off

```

를 통하여 chk.log에 저장후,

```
SQL> select count(*) from corrupted_table;
```

확인

table export등을 통하여 확인된 rowid의 block number를 비교하여 보면 됩니다.
주의사항은 여기에서 ora-1578과 관련된 block number는 decimal이니, hex로 변환해야 합니다.

LOCK 문제 확인과 해결 방법

Bulletin no : 10221

- 응용프로그램들의 기능이 복잡해져가면서 Lock 문제가 빈번하게 발생함에 따라 이를 해결하는 것이 매우 중요한 일이 되었다. 도대체 어떠한 프로그램이 Lock을 발생시켜 데이터베이스를 Hangup 상태에 이르게 하는가를 알아내는 것은 쉬운 일이 아니지만 지금부터 이 문제를 해결하는 방법에 대하여 알아보도록 한다.

BQLMON MONITOR BEGIN

```
& TERN=vt100:export TERN
```

```
& myidba
```

```
[ connect internal ] -> [ key=0(lock key) ]
```

[MONITOR BEGIN]

Session ID	Serial Number	Process ID	Status	Username	Lock Waited	Current Statement
8	35	28	ACTIVE	LT02	C2928384	UPDATE
8	70	18	INACTIVE	SYSTEM		SELECT
12	15	25	INACTIVE	LT0		INSERT
14	17	27	ACTIVE	LT03	C2928438	DELETE
15	30	28	ACTIVE	SVS		UNION

- Session id : SID - Session Identifier
- Serial No : Session Serial Number, Session 의 Object 별로 지정된다.
- Process id : OS/DBCS에서 PID(Database Process Identifier)
- Status : Session의 상태
- Username : Oracle Username
- Lock waited: 대기중인 Lock Address

여기서 가장 중요한 컬럼은 Lock Waited이다. 여기에 무엇인가 기록이 되어 있다면 이 세션은 특정 리소스를 얻기 위하여 대기 중인 상태임을 알 수 있다. 위의 예에서 LT02,LT03이 그 경우에 해당 된다.

[MONITOR LOCK]

Username	Session ID	Serial Number	Lock Type	Resource ID 1	Resource ID 2	Mode Held	Mode Requested
LT02	8	35	TM	2284	0	OK	NONE
LT02	8	35	TM	2285	0	OK	NONE
LT02	8	35	TM	282187	87	NONE	K
LT02	8	35	TM	327882	80	K	NONE

LYD2	12	15	TK	2294	0	BK	NONE
LYD	12	15	TK	2295	0	BK	NONE
LYD	12	15	TK	282187	87	K	NONE
LYD3	14	17	TK	2294	0	BK	NONE
LYD3	14	17	TK	2295	0	BK	NONE
LYD3	14	17	TK	282187	87	NONE	K
LYD3	14	17	TK	188838	87	K	NONE

- Username : Oracle Username.
- Session Id : SID - Session Identifier.
- Serial No : 앞의 설명 참조

-----+-----+-----

Type of Lock	Resource ID1	Resource ID 2
--------------	--------------	---------------

-----+-----+-----

TK(Transaction): 락 세그먼트 번호와 슬롯(Slot Number)을 10진수로 변환한
ID 번호를 10진수로 변환한 값(락(슬롯이 재사용된 횟수))

-----+-----+-----

TM(Table Lock): 수정된 테이블의 Object ID 항상 0

-----+-----+-----

BM(Block Mail): File #, Block #를 10진수 ID 블록내의 Row를 10 수로 변환
ID 로 변환한 값 (UB) ID 한 값

-----+-----+-----

UL(User Defined): Oracle? Concepts Manual Ch10, Oracle? Admin Guide
Locks ID Appendix B B1 의 리스트 참조

-----+-----+-----

Mode Held ID 걸고 있는 Lock Mode

-----+-----+-----

Mode Requested ID 요구한 Lock Mode

-----+-----+-----

* 다음 사용자들은 Lock를 기다리고 있는 상태이다.

LYD2	8	35	TK	282187	87	NONE	K
LYD3	4	17	TK	282187	87	NONE	K

* 이들이 기다리고 있는 Lock는 다음과 같다.

LYD	12	15	TK	2294	0	BK	NONE
LYD	12	15	TK	2295	0	BK	NONE
LYD	12	15	TK	282187	87	K	NONE

TK Lock의 경우, 동일한 리소스 대한 Lock를 요구하는 경우 각각의 ID1, ID2는 동일한 값을 갖는다. 하나의 데이터 블록 내에는 그 블록을 사용하고 있는 트랜잭션에 대한 정보가 들어있는데 만약 이 트랜잭션이 Commit, Rollback 되지 않으면 다른 트랜잭션들은 그 트랜잭션이 잡고 있는 리소스의 Lock에 해제 되기를 기다리게 된다.

하나의 트랜잭션이 여러개의 데이터에 대해서 작업을 하고 있는 경우에는 Lock과 연관된 리소스가 어떤 것인지를 쉽게 파악하기가 힘들다. 이 때에는 Monitor Session과 Monitor Table을 모두 보아야지만 확실하게 알 수 있다.

[MONITOR SESSION]

Session ID	Serial Number	Process ID	Status	Username	Lock Waited	Current Statement
5	31	18	INACTIVE	LTD		INSERT
8	43	25	ACTIVE	LTD2	C30320F4	UPDATE
8	1	28	ACTIVE	LTD3	C30320CB	DELETE

[MONITOR TABLE]

Session ID	Schema Name	Table Name
5	LTD	DEPT
8	LTD	EMP
8	LTD	EMP

* 위의 경우를 보면 Session 8과 Session 8가 LTD.EMP를 사용하려고 기다리고 있음을 알 수 있다.

■ 해결방법

대부분의 Lock 문제는 응용프로그램의 잘못된 설계 때문에 발생하게 된다. 이를 해결하기 위해서 다음과 같은 방법으로 Lock을 Release 시켜야 한다.

1. Lock Holder에게 Commit, Rollback을 요청한다.
2. Lock Holder의 세션을 Kill 시킨다.

```
SQL>ALTER SYSTEM KILL SESSION 'sid,serial#';
```

3. 유닉스 Shadow Process(Server Process)를 Kill 시키는 것은 바람직하지 않다. 특히 NTB 환경하에서는 각별한 주의가 필요하다.
4. 2PC Pending 트랜잭션인 경우는 Rollback Force, Commit Force 로 해결한다.

■ LOCK 문제를 일으키는 SQL 명령 찾기

* 다음 Query는 Lock과 관련된 트랜잭션을 출력해준다.

```
column username format a10
column sid format BB
column lock_type format a15
column NAME_HOLD format a11
column NAME_REQUESTED format a10
column LOCK_ID1 format a8
column LOCK_ID2 format a8
select s.sid,
```

```

decode(s.type,
'MB', 'Media Recovery',
'BT', 'Backup Thread',
'UN', 'User Name',
'TN', 'Transaction',
'TH', 'DML',
'UL', 'PL/SQL User Lock',
'BN', 'Distributed Nction',
'CF', 'Control File',
'IS', 'Instance State',
'FS', 'File Set',
'IR', 'Instance Recovery',
'BT', 'Disk Space Transaction',
'IR', 'Instance Recovery',
'BT', 'Disk Space Transaction',
'TB', 'Temp Segment',
'IV', 'Library Cache Invalidation',
'LS', 'Log Start or Switch',
'BU', 'Base Unit',
'BQ', 'Sequence Number',
'TE', 'Extended Table',
'TT', 'Temp Table',
s.type) lock_type,
decode(s.mode,
0, 'None',           /* Max Lock equivalent */
1, 'Null',           /* N */
2, 'Row-B (BB)',     /* L */
3, 'Row-K (BK)',     /* B */
3, 'Row-K (BK)',     /* B */
4, 'Block',          /* B */
5, 'B/Row-K (BBK)', /* C */
8, 'Exclusive',      /* K */
to_char(s.mode)) mode_held,
decode(s.request,
0, 'None',           /* Max Lock equivalent */
1, 'Null',           /* N */
2, 'Row-B (BB)',     /* L */
3, 'Row-K (BK)',     /* B */
4, 'Block',          /* B */
5, 'B/Row-K (BBK)', /* C */
8, 'Exclusive',      /* K */
to_char(s.request)) mode_requested,
to_char(s.id1) lock_id1, to_char(s.id2) lock_id2
from v$lock s
where (id1, id2) in
  (select k.id1, k.id2 from v$lock k where k.id1=s.id1 and
    k.id2=s.id2 and k.request>0)
/

```

(위의 Query를 실행시키면 다음과 같은 내용이 출력된다.

BID	LOCK_TYPE	MODE_HOLD	MODE_REQUEST	LOCK_ID1	LOCK_ID2
5	Transaction	Exclusive	None	282172	80
8	Transaction	None	Exclusive	282172	80
8	Transaction	None	Exclusive	282172	80

BID 8과 8는 BID 5가 걸고 있는 Lock에 풀리기를 기다리고 있음을 알 수 있다.

* 다음 Query는 Lock과 관련된 데이터를 출력해 준다.

```
column username format a10
column lockwait format a10
column sql_text format a80
column object_owner format a14
column object format a15
select b.username username, c.sid sid, c.owner object_owner,
       c.object object, b.lockwait, a.sql_text BQL
from v$sqltext a, v$sqlmon b, v$sqlmon c
where a.address=b.sql_address and
       a.hash_value=b.sql_hash_value and
       b.sid = c.sid and c.owner != 'SYS'
/
```

(위의 Query를 실행하면 다음과 같은 결과가 출력된다.

USERNAME	BID	OBJECT_OWNER	OBJECT	LOCKWAIT
BQL				
LY02	8	LY0	EMP	030320F4
update lto.emp set salary =25 where salary=7788				
LY03	8	LY0	EMP	030320C8
delete from lto.emp where salary=7788				
LY0	5	LY0	DEPT	
insert into lto.dept values (80,'PRODCAHRES','LOB AMCEL08')				

여기서는 USERNAME에 나와있는 유저가 OBJECT에 나와있는 데이터를 수정하려고 함을 나타낸다. LY02, LY03은 LY0가 Commit, Rollback 하기를 기다리고 있음을 알 수 있다. 하지만 여기에는 가장 최근의 SQL 명령 하나만 나와있기 때문에 여기 나온 명령이 반드시 Lock을 걸고 있는 명령이라고 단정지을 수는 없다.

■ 관련된 프로세스

* 다음 Query를 실행하면 프로세스와 관련된 정보를 얻을 수 있다.

```
column 'DBLOCK USERS' format a11
column SERIAL# format BBBBBB
column 'DB USERS' format a8
select substr(s.usename,1,11) "DBLOCK USERS", p.pid "PROCESS ID",
       s.sid "SESSION ID", s.serial# "SERIAL#", osuser "DB USERS",
       p.spid "PDB SPID", s.process "BDBB SPID", s.lockwait "LOCK WAIT"
from v$process p, v$session s, v$process s
where s.sid=s.sid and
       p.addr=s.paddr and
       s.usename != 'SYS'
/
```

* 위의 Query를 실행하면 다음과 같은 결과가 출력된다.

DBLOCK USERS	PROCESS ID	SESSION ID	SERIAL#	DB USERS	PROCESS SPID	BDBB SPID	LOCKWAIT
----	-----	-----	-----	-----	----	----	-----
LYD	18	5	31	ucupgost	17312	17308	
LYD2	25	8	43	ucupgost	17313	17310	C3B320F4
LYD3	28	8	1	ucupgost	17314	17311	C3B320B8

이 때는 다음과 같이 조치한다.

1. LYD에게 Commit, Rollback 할 것을 요구한다.
2. SQL>ALTER SYSTEM KILL SESSION '5,31';
3. \$kill -B 17308 (유닉스상의 Shadow Process)
stop/ld= (PDB SPID=BDBB SPID on vnc running single task)

여기서 BYB 용어는 제외시켰는데 필요하다면 Query의 Where 조건에서 s.usename != 'BYB' 부분을 삭제하면 된다.

■ CATBLOCK.SQL & UTLLOCKT.SQL

\$ORACLE_HOME/rdbms/admin 디렉토리에 있는 스크립트 가운데 catblock.sql과 utllockt.sql을 사용하여 Lock 상황을 쉽게 파악할 수 있다. 이들은 다음과 같이 실행한다.

```
%cd $ORACLE_HOME/rdbms/admin
%sqlsh sqlplus
SQL>connect internal
SQL>@catblock
SQL>@utllockt
```

결과는 다음 Query 문으로 알아 본다.

```
column waiting_session format a8
select lpad(' ',34(level-1)) || waiting_session,
       lock_type,
       mode_requested,
       mode_held,
       lock_id1,
       lock_id1,
       lock_id2
from lock_holders
connect by prior waiting_session = holding_session
start with holding_session is null;
```

위의 Query에 의한 출력은 다음과 같다.

WAITING_	LOCK_TYPE	MODE_REQUESTED	MODE_HELD	LOCK_ID1	LOCK_ID2
S	None				
B	Transaction	Exclusive	Exclusive	282172	B0
B	Transaction	Exclusive	Exclusive	282172	B0

여기서 Session B, Session B가 Session S를 기다리고 있음을 알 수 있다.

■ Lock & Hanging 문제를 추정하는 방법

프로그램 상에서 어느 부분이 Lock, Hanging 문제를 일으키는지 알아내기가 여러치 많을때 다음과 같은 방법들 사용해 보기 바란다.

1. Init, oracli sql_trace를로 세팅하면 연관된 SQL 명령이 출력될 것이다.
2. DB 상에서도 Process 상태를 점검하여 본다.
3. DB 상의 Debugging 기능들 사용하거나 만약 가능하다면 orachk를 사용한다. Platform 에 따라서 얻을 수도 있다.
4. 여러가지 Monitoring 방법과 Locking/Blocking 스크립트를 이용한다.

■ PROCESS STACKS 확인

때로는 Hanging Process나 디소스를 점유하고 있는 Process의 Process Stack을 점검해 보는 것이 문제 해결에 도움이 되는 경우가 있다.

1. DB Debugging 프로그램들 이용하여 Hangup 되기 전의 마지막 call을 확인 한다.

```
oradebug -p
```


2. oradbx(오라클 debugging 프로그램)은 Support, Development 시에만 사용된다.

```
select substr(s.usernam,1,11) "ORACLE USER",
       p.pid "PROCESS ID", s.sid "SESSION ID", s.serial#,
       osuser "DB USER", p.spid "PROC SPID"
from v$session s, v$process p
where s.sid=s.sid and
       p.addr=s.paddr and
       s.username != 'SYS'
/
```

위의 Query를 실행하면 다음과 같은 결과가 출력된다.

ORACLE USER	PROCESS ID	SESSION ID	SERIAL#	DB USER	PROC SPID	SESSION SPID	LOCUT
----	-----	-----	-----	-----	-----	-----	-----
LTD	18	5	31	unsupport	17312	17308	
LTD2	25	8	43	unsupport	17313	17310	C3032074
LTD3	28	8	1	unsupport	17314	17311	C3032088

만약 oradbx가 없다면 다음과 같이 해서 만들어 준다.

```
%cd $ORACLE_HOME/oradbx/11g
%make -f oradbx.mk oradbx
```

LTD Process가 무엇을 하고 있는지 알고 싶으면 Process Stack을 보면 알수 있다.

```
%x -nf 1 grep 17312
unsupport 17312 17308 0 Sep 15 ? 0:00 oracliu713(DESCRIPTION=(LOCAL=YES

type
debug 17312 (o) 중지된 oracliu shadow process)
dump stack
dump procstat
```

위에서 생성된 트레이스 파일(user_dump_dest 에 생성됨)을 이용하면 Hanging 문제를 해결하는데 큰 도움이 된다.

■ 자주 발생하는 LOCK 문제들

1. Parent-Child 관계로 묶인 데이터에서 Child 데이터에 Index가 없는 상황에서 Child 데이터를 수정하게 되면 Parent 데이터에 TABLE LEVEL SHARE LOCK이 걸리게 되어서 Parent 데이터에 대한 모든 Update가 금지된다.
2. 블록의 PCWBLOCK가 매우 작다면 한 블록에 여러개의 레코드가 들어 있기 때문에 한 블록에 과도한 프랜잭션이 들어와서 블록의 Transaction Layer가 Fullness 되기를 기다리게 되는 경우도 있다.

Ex)

```
create table ttest (a number) initrans 1 maxtrans 1;
```

```
SYSTEM: Insert into ttest values (5);           /* completed */
SCOTT:   Insert into SYSTEM.ttest values (10);  /* Scott waits */
```

SID	USER	LOCK_TYPE	MODE_HELD	MODE_REQUEST	LOCK_ID1	LOCK_ID2
7	SYSTEM	Transaction	Exclusive	None	186838	54
10	SCOTT	Transaction	None	Share	186838	54

LOCKING CONFLICTS ≡ DETECTING/RESOLVING

Bulletin no : 10148

Problem Description:

With the growing complexity of applications, it has become a challenge to resolve and properly diagnose locking (latching) issues. Although it may seem like quite a challenge to find out what is causing your application or your database to hang, we are providing you with a methodology of things to check when faced with a difficult "latching" scenario.

Problem Explanation:

Bulletin 108450.509 by LYD explains how to use a combination of utilities: SQL, MONITORING FACILITY and LOCKING scripts provided by Oracle.
The screens used for monitor are session, lock, and table. The locking scripts used are catlock.sql and utllockt.sql

USE OF MONITOR SCREENS:

MONITOR SESSION:

Session Serial Process					Lock	
ID	Number	ID	Status	Username	Waited	Current Statement

8	35	28	ACTIVE	LYD2	C2928384	UPDATE
8	70	18	INACTIVE	SYSTEM		SELECT
12	15	25	INACTIVE	LYD		INSERT
14	17	27	ACTIVE	LYD3	C2928438	DELETE
15	30	28	ACTIVE	BYB		UNKNOWN

Name	Description
Session Id	SID - Session Identifier
Serial No	Session serial number. Used to uniquely identify a session's objects. Guarantees that session-level commands are applied to the correct session objects in the event that the session ends and another session begins with the same session ID
Process Id	PID is v\$process, Oracle Process Identifier.
Status	Status of the session.
Username	User name.
Lock waited	Address of lock waiting for ; null if none.

The most important column would be LOCK WAITED. If it is NOT NULL, then

this user is waiting for a resource. In this example, LYD2 and LYD3 are waiting on locks.

MONITOR LOCK:

	Session	Serial	Lock	Resource	Resource	Mode	Mode
	ID	Number	Type	ID 1	ID 2	Held	Requested

LYD2	B	35	TM	2284	0	BK	NONE
LYD2	B	35	TM	2285	0	BK	NONE
LYD2	B	35	TK	282187	B7	NONE	K
LYD2	B	35	TK	327882	B0	K	NONE
LYD	12	15	TM	2284	0	BK	NONE
LYD	12	15	TM	2285	0	BK	NONE
LYD	12	15	TK	282187	B7	K	NONE
LYD3	14	17	TM	2284	0	BK	NONE
LYD3	14	17	TM	2285	0	BK	NONE
LYD3	14	17	TK	282187	B7	NONE	K
LYD3	14	17	TK	188838	B7	K	NONE

Username User name.

Session ID SID - Session Identifier.

Serial No Same as above.

Type of Lock LOCK ID1

LOCK ID2

TK(Transaction) Decimal representation of rollback segment number and slot number

Decimal representation of "wrap" number (number times the rollback slot has been reused)

TM(Table Locks) Object ID of table being modified.

Always 0.

BU(Row Unit) Decimal representation of file no. and block no. (Version 8)

Decimal representation of row within block

UL(User Defined Locks) Complete list is found in chpt 10 of the Oracle 7 Concepts Manual or in Appendix B-B1 in the Oracle 7 Admin Guide. Please note that the BU sequence was taken out after version 8.0.34 and may be added back in a future release.

Mode Held Lock mode held.

Mode Requested Lock mode requested.

The following users are waiting:

LYD2	B	35	TK	282187	B7	NONE	K
LYD3	14	17	TK	282187	B7	NONE	K

They are waiting on:

LTD	12	15	TM	2284	0	BK	NONE
LTD	12	15	TM	2285	0	BK	NONE
LTD	12	15	TM	282187	87	K	NONE

For the TM lock, lock 1d1 and lock 1d2 are the cases when they are contending for the same resource. Within the block, we have a record of the each transactions pertaining to the data in that particular block. If the transaction is not committed or rolled back, then other subsequent transactions may have to wait for that resource.

Often, the user(s) may be modifying many tables within the same transaction. At times, this will make it difficult to find out which resource the WAITED is contending from the HOLDER. This is easily resolved by looking at a combination of two monitors. MONITOR SESSION will tell you which user is waiting on a lock and MONITOR TABLE will tell you the table that the user is currently trying to modify.

MONITOR SESSION:

Session Serial Process

Lock

ID	Number	ID	Status	Username	Waited	Current Statement

S	31	1B	INACTIVE	LTD		INSERT
B	43	2S	ACTIVE	LTD2	C30320F4	UPDATE
B	1	2B	ACTIVE	LTD3	C30320CB	DELETE

MONITOR TABLE:

Session

ID	Schema Name	Table Name

S LTD		DEPT
B LTD		EMP
B LTD		EMP

Session B and B are waiting for locks. They are both attempting to modify its emp TABLE.

RESOLUTION:

Most locking issues are application specific. To resolve locking contention, one needs to free the resource by:

1. Asking the HOLDER to commit or rollback,
2. Killing the session which holds the lock,
ALTER SESSION KILL SESSION sid, serial#; or
use KILL USER SESSION menu found in the sqldrda form.
3. Killing the unix shadow process which is not recommended.
When killing the unix shadow process, please be careful of shared servers in a multi-threaded environment.
4. ROLLBACK FORCE or COMMIT FORCE if 2pc pending transaction.

SQL STATEMENTS TO DECIPHER LOCKING ISSUES:

TRANSACTIONS INVOLVED:

This query defines the transactions involved in a locking situation:

```
column username format a10
column sid format BBB
column lock_type format a15
column NAME_HOLD format a11
column NAME_REQUESTED format a10
column LOCK_ID1 format a8
column LOCK_ID2 format a8
```

```
select s.sid,
       decode(s.type,
              'MR', 'Media Recovery',
              'ST', 'Standby Thread',
              'UM', 'User Name',
              'TX', 'Transaction',
              'TM', 'DML',
              'UL', 'PL/SQL User Lock',
              'MK', 'Distributed Mutation',
              'CF', 'Control File',
              'IS', 'Instance State',
              'FS', 'File Set',
              'IR', 'Instance Recovery',
              'BT', 'Disk Space Transaction',
              'IB', 'Instance Recovery',
              'BT', 'Disk Space Transaction',
              'TS', 'Temp Segment',
```

```

'IV', 'Library Cache Invalidation',
'LB', 'Log Start or Switch',
'ML', 'Mode Malt',
'BQ', 'Sequence Number',
'TE', 'Extended Table',
'TT', 'Temp Table',
s.type) lock_type,
decode(s.mode,
0, 'None',           /* None Lock equivalent */
1, 'Null',           /* N */
2, 'Row-B (BB)',     /* L */
3, 'Row-K (BK)',     /* B */
4, 'Share',          /* S */
5, 'B/Row-K (BBK)',  /* C */
8, 'Exclusive',      /* X */
to_char(s.mode)) mode_held,
decode(s.request,
0, 'None',           /* None Lock equivalent */
1, 'Null',           /* N */
2, 'Row-B (BB)',     /* L */
3, 'Row-K (BK)',     /* B */
4, 'Share',          /* S */
5, 'B/Row-K (BBK)',  /* C */
8, 'Exclusive',      /* X */
to_char(s.request)) mode_requested,
to_char(s.id1) lock_id1, to_char(s.id2) lock_id2
from v$lock s
where (id1,id2) in
(select k.id1, k.id2 from v$lock k where k.id1=s.id1 and
k.id2=s.id2 and k.request>0)
/

```

Sample output:

BID	LOCK_TYPE	MODE_HELD	MODE_REQUEST	LOCK_ID1	LOCK_ID2
5	Transaction	Exclusive	None	282172	80
8	Transaction	None	Exclusive	282172	80
8	Transaction	None	Exclusive	282172	80

This is very similar to the monitor lock output. Again, SID 8 and 8 are waiting on SID 5.

OBJECTS INVOLVED:

This query will defined the objects that are involved in the locking conflict:

```

column username format a10
column lockwait format a10
column sql_text format a80
column object_owner format a14
column object format a15

select b.username username, c.xid xid, c.owner object_owner,
       c.object object, b.lockwait, a.sql_text BQL
from v$sqltext a, v$session b, v$process c
where a.address=b.sql_address and
       a.hash_value=b.sql_hash_value and
       b.xid = c.xid and c.owner != 'SYS'
/

```

Sample output:

USERNAME	BID	OBJECT_OWNER	OBJECT	LOCKWAIT
BQL				
LYD2	B LYD	EMP		C3B320F4
update lta.emp set salary =25 where empno=7788				
LYD3	B LYD	EMP		C3B320CB
delete from lta.emp where empno=7788				
LYD	S LYD	DEPT		
Insert into lta.dept values (80,'PROGRAMMER','LOS ANGELES')				

One can see the DMLs resulting from LYD2 and LYD3 who are the WAITERS. They are waiting for LYD to commit or rollback; unfortunately, the DML statement listed under LYD is only the most recent DML. This may not indicate the statement that is holding the resource. In this example, it is NOT the DML. The only DML was an update statement. OBJECT_OWNER, OBJECT is the object USERNAME is attempting to modify.

PROCESSES INVOLVED:

To ascertain process information, one can execute this query:

```

column "ORACLE USER" format a11
column SERIAL# format 999999
column "DB USER" format a8

select substr(x.username,1,11) "ORACLE USER", p.pid "PROCESS ID",
       x.xid "SESSION ID", x.serial#, owner "DB USER", p.spid "PROC SPID",

```



```

    x.process 'BESS SPID', x.lockwait 'LOCK WAIT'
from v$process p, v$session s, v$process x
where s.sid=x.sid and
      p.addr=s.paddr and
      x.username != 'SYS'
/

```

Sample output:

```

ORACLE USER PROCESS ID SESSION ID SERIAL# OS USER PROC SPID BESS SPID LOCKW
-----
LTD          18          5          31 unsupported 17312      17308
LTD2         25          8          43 unsupported 17313      17310 C3B320F4
LTD3         28          8          1 unsupported 17314      17311 C3B320C8

```

What can you do??

- 1) Can ask LTD to commit or rollback or
- 2) alter system kill session '5,31'; or
- 3) kill -8 17308 (shadow process on unix)
 stop/id= (PROC SPID-BESS SPID on vms running single task)

Notice that the user SYS has been excluded. If your application was created under SYS, then you may want to include it in the query.

USING CATALOCK.BQL and UTILLOCK.BQL:

There are very helpful scripts that oracle provides with the other BQMS Installation scripts (CATALOC.BQL, CATPROC.BQL, etc.) For example, ?/rdhuc/adm directory on unix and ora_rdhuc directory on vms.

First run catlock.sql as sys and then run utillock.sql as sys.

Print out the result in a tree structured fashion:

```
column waiting_session format a8
```

```

select lpad(' ',3*(level-1)) || waiting_session waiting_session,
       lock_type,
       mode_requested,
       mode_hold,
       lock_id1,
       lock_id2
from lock_holders
connect by prior waiting_session = holding_session
start with holding_session is null;

```

Sample Output:

WAITING_	LOCK_TYPE	MODE_REQUEST	MODE_HELD	LOCK_ID1	LOCK_ID2
5	None				
B	Transaction	Exclusive	Exclusive	282172	80
B	Transaction	Exclusive	Exclusive	282172	80

Processes B and B are waiting on session 5.

DRASTIC METHODS IN DECIPHERING A LOCKING OR "HANGING" SITUATION:

At times, it may be difficult to find out the root cause of the "hanging" problem within your application. One may need to resort to drastic measures which include both tracing on the database and operating system layer. Here are some recommendations:

- 1) Turn on sql_trace. This will reveal the sql statements involved.
- 2) Set tracing on and monitor processes on the operating system side as well. This will reveal if the process is getting any cpu and the status of the process.
- 3) Utilize the operating system debugging facilities or utilize "oradbx" if available. (oradbx is only for support and development use and may not be available on your platform.)
- 4) Utilize monitor screens and the locking (blocking) scripts.
- 5) One can build your own self monitoring script.

For example:

Create a file called monitor.sql:

```
select b.username username, c.sid sid, c.owner object_owner,
       c.object object, b.lockwait, a.sql_text BQL
from v$sqltext a, v$sqlmon b, v$sqlmon c
where a.address=b.sql_address and
       a.hash_value=b.sql_hash_value and
       b.sid = c.sid and
       c.owner != 'SYS'
/
D77rdhsw/admln/utlllockt
```

lpx -st

lclmp

Create a file name runtest with executable permissions:

```
sqlplus user/password << test
spool output.txt
set time on
set echo on
Run test
```

ASCERTAINING PROCESS STACKS:

In some rare cases, one may encounter a "hanging" problem that can not be fixed. One may find that a process stack of the hanging process or the process that holds the resource may be helpful in resolving the problem.

- 1) Operating system debugging tools can be used to find out the last call before it hangs. In, trace -p
- 2) ORACLE's debugging utility, oradbx, is for SUPPORT and DEVELOPMENT's use ONLY.

```
select substr(s.username,1,11) "ORACLE USER",
       p.pid "PROCESS ID", s.sid "SESSION ID", s.serial#,
       owner "DB USER", p.spid "PROC SPID"
from v$session s, v$process p
where s.sid=p.sid and
       p.address=paddr and
       s.username != 'SYS'
/
```

Sample Output:

ORACLE USER	PROC ID	SESSION ID	SERIAL#	DB USER	PROC SPID	SESS SPID	LOCK	WAIT
LYD	18	5	31	ucsupport	17312	17308		
LYD2	25	8	43	ucsupport	17313	17310	C90320F4	
LYD3	28	8	1	ucsupport	17314	17311	C90320C8	

```
cd $ORACLE_HOME/rdbms/lib
```

```
make -f oracle.mk oradbx
```

To find out what LYD process is actually doing, one can dump the process stack.

```
ps -ef | grep 17312
```

```
ucsupport 17312 17308 0 Sep 15 ? 0:00 oracle@713 (DESCRIPTION=(LOCAL=YES
```

```
type
```

```
debug 17312 (which is the oracle shadow process for this user)
```

```
dump stack
```

```
dump procstat
```

These trace files may be instrumental in resolving a hanging problem. Please remember that Oracle's debugging utility, oradbx, is for Support and Development's use ONLY. The trace files should go to your USER_DUMP_DEST and should be sent to Worldwide Support for debugging.

UNUSUAL LOCKING PROBLEMS:

- 1) When your application has referential integrity and attempts to modify the child table, Oracle will get a TABLE LEVEL SHARE LOCK on the parent table when there is NO index on the foreign key.
- 2) When a table's PCTFREE is set too low, the block is full with data, and there are many concurrent DML's occurring on rows within the block, one may see a Share Lock being requested when doing a DML. To my knowledge, this is only time we grab the SHARED lock. Instead of waiting for a lock, this process is waiting for some extra space or a release of an INITRANS within the transaction layer of the block.

Here's a simple example:

```
create table test (a number) initialize 1 extent 1;
```

```
SYSTEM: insert into test values (5); /* completed */
```

```
SCOTT: insert into SYSTEM.test values (10); /* Scott waits */
```

In normal situations, SCOTT should not have to wait but SCOTT has to wait until SYSTEM releases the one and only INITRANS in the block. If the MAXTRANS was set to 1, then Oracle will try to allocate another INITRANS in the block if space permits. I believe each INITRANS is about 24 bytes.

BID	OWNER	LOCK_TYPE	MODE_HELD	MODE_DEQUE	LOCK_ID1	LOCK_ID2
7	System	Transaction	Exclusive	None	188838	54
10	Scott	Transaction	None	Share	188838	54

LOCKING CHART:

The following table describes which table accesses are gotten for which table operations and what equivalent DML modes are actually gotten in various conditions.

Operation	Table Access	DML Table Lock Mode				Row-Locking Serializability
		Yes No	Yes No	No No	No Yes	
Select	Read	MULL	B	MULL	B	
Select For Update	Read-Read	BB	B	BB	B	
Insert	Read-Write	BK	BK	BBK	BBK	
Update	Read-Read-Write	BK	BBK	BBK	BBK	
Delete	Read-Read-Write	BK	BBK	BBK	BBK	
Lock For Update	Read-Read	BB	B	BB	B	

Lock Share		B	B	B	B
Lock Exclusive		K	K	K	K
Lock Row Share		BB	BB	BB	BB
Lock Row Exclusive		BK	BK	BK	BK
Lock Share Row Exclusive		BBK	BBK	BBK	BBK
Alter	Update	K	K	K	K
Drop	Update	K	K	K	K
Create Index		B	B	B	B
Drop Index	Update	K	K	K	K

Lock Mode Compatibility:

	NULL	BB	BK	B	BBK	K
NULL	YES	YES	YES	YES	YES	YES
BB	YES	YES	YES	YES	YES	NO
BK	YES	YES	YES	NO	NO	NO
B	YES	YES	NO	YES	NO	NO
BBK	YES	YES	NO	NO	NO	NO
K	YES	NO	NO	NO	NO	NO

MESSAGE CONVERSION PROGRAM

Bulletin no : 10029

많은 `kitm`에서 명문 `BB`에서 한글 메시지를 사용하고 싶다고 문의가들어옵니다. 현재는, 지금 가지고 있는 `*ko.msk`를 가지고 명문 `BB`에서 사용하면 ???로 display가됩니다. 이것을 해결하기 위해 2가지 방법이 있습니다.

1. `langm`를 다시 만드는 것입니다.

```
Ex> langm lachk.msg format30 lsc korana_koraa,uz?ascii
```

또 하면 `lachk.msk`로 만드는 동시에 이 파일에 명문이라는 정보가 들어가기 때문에 명문 `BB`에서 아무런 문제없이 사용하실 수 있습니다.

2. `langm`는 각 `system`마다 가지고 있어야 합니다. `Bun`에서 만든 `*ko.msk`를 `hp`나 다른 `system`에서 사용할 수 없습니다. 현재 각 `system`에 해당하는 `langm`를 보관하고 있는것은 아닙니다.

그래서 현재 각 `tool`에 있는 `message`에 `korana` 정보를 `uz?ascii`로 바꾸는 `program`를 만들었습니다.

사용하는 방법

```
$ cc compile.c -o compile
```

```
$ sh -x k (compile 파일이름 ED1B8B05B01 --> 파일에 명문 BB를 ED BB로 바꾸는것) 또는 sh -x u  
(compile 파일이름 UB79BC11 --> 파일에 ED BB를 명문 BB로 바꾸는것) 을 하시면 됩니다 .
```

- `compile.c` 프로그램.

```
#include
```

```
#include
```

```
#include
```

```
int uz_to_ka, ka_to_uz; /*message conversion flag*/
```

```
main(argc,argv)
```

```
int argc;
```

```
char *argv[];
```

```
{
```

```
    uz_to_ka = ka_to_uz = 0;
```

```
    if ( argc != 3) {
```

```
        usage(argv[0]);
```

```
        exit(1);
```

```
    }
```

```
    if (strcmp(argv[2], "ED1B8B05B01"))
```

```
        uz_to_ka = 1;
```

```
    else if (strcmp(argv[2], "UB79BC11"))
```

```
        ka_to_uz = 1;
```

```
    else
```

```
        usage(argv[0]);
```

```
    data_conversion(argv[1]);
```

```

printf("conversion completed successfully !!!\n");
}

usage(prog_name)
char *prog_name;
{
    printf("Usage: %s exported_file_name character %s", prog_name);
    printf("        character : E01B8B05B011U87ABC11 %s");
    printf("        , E01B8B05B01 - convert uc7ascii format to  

        kb1B8B05B01 format\n");
    printf("        , U87ABC11 -convert kb1B8B05B01 format to uc7ascii  

        format\n");
}

data_conversion(ucf11a)
char *ucf11a;
{
    int    fd;
    char   conv_buffer[11];
    if((fd = open(ucf11a, 2)) < 0) {
        perror("open");
        exit(1);
    }
    if(lseek(fd, 0, 0) < 0) {
        perror("lseek");
        exit(1);
    }
    if(uc_to_kb){
        conv_buffer[0] = 11;
        conv_buffer[1] = 15;
        conv_buffer[2] = 28;
        conv_buffer[3] = '1';
        conv_buffer[4] = 11;
        conv_buffer[5] = 18;
        conv_buffer[6] = 3;
        conv_buffer[7] = ' ';
        conv_buffer[8] = '1';
        conv_buffer[9] = 27;
        conv_buffer[10] = 28;
    }
    else if(kb_to_uc){
        conv_buffer[0] = 21;
        conv_buffer[1] = 18;
        conv_buffer[2] = "'";
        conv_buffer[3] = 1;
        conv_buffer[4] = 18;
        conv_buffer[5] = 3;
        conv_buffer[6] = '\t';
    }
}

```

```

        conv_buffer[7] = '\t';
        conv_buffer[8] = '\0';
        conv_buffer[9] = '\0';
        conv_buffer[10] = '\0';
    }

    if (write(fd, conv_buffer, 11) != 11) {
        perror("write");
        exit(1);
    }

    close(fd);
}

```


METHOD 4를 이용한 SAM FILE 생성 PROGRAM

Bulletin no : 10302

아래의 Program은 Procc의 Dynamic Method 4를 응용하였으며, BQLoader에서 사용할 수 있는 Data File 및 Control File을 자동으로 생성해 준다.

(Source File은 선택한 "go oracle" 로 가서 구할 수 있다)

(1) 사용 방법 :

```
$ db2fil id/procc Table_name
```

(2) 출력 File :

```
Table_name.dat, Table_name.ctl
```

(3) BQLoader로 Loading방법:

```
$ sqlload id/pwd control=Table_name.ctl
```

아래의 Program을 \$ORACLE_HOME/proc/dm2 디렉토리로 db2fil.pc라는 이름으로 생성한 후 make fil을 수행한다.

```
$ make -f proc.mk BFILE=db2fil OBJ=db2fil.o
```

(현재 Max Item의 갯수는 40으로 설정되어있으며, 필요에따라 수정하여 사용 한다. 또한 자세한 내용은 Procc 매뉴얼을 참조.)

```
#include
```

```
#include
```

```
#include
```

```
EXEC SQL BEGIN DECLARE SECTION;
```

```
UNIONNAME unionname [40];
```

```
UNIONNAME statement[40];
```

```
EXEC SQL END DECLARE SECTION;
```

```
EXEC SQL INCLUDE sqlca.h;
```

```
EXEC SQL INCLUDE sqlcs.h;
```

```
#define MAX_ITEM 40
```

```
#define MAX_UNIONNAME_LEN 30
```

```
#define MAX_STATEMENT_LEN 30
```

```
BQLDA *bind_dp;
```

```
BQLDA *select_dp;
```

```
extern BQLDA *sqladd();
```

```
extern void sqlproc();
```

```

int precision, scale;
short null_ok;
extern void sqlnull();
void print_row();

FILE *fp, *fp_ctl;
main(argc, argv)
int argc;
char **argv;
{
    int i, j, num_out = 0;
    char filename[12], ctrl_file[12];

    if(argc != 3) {
        printf("\n\n    Input data type error !!!\n");
        printf("\n    Usage: db2filn scott/tiger map %s\n\n");
        exit(0);
    }

    EXEC SQL WHENEVER SQLERROR GOTO sqlerror;

    strcpy (username,argv, argv[1]);
    username_len = strlen(username,argv);
    EXEC SQL CONNECT :username;

    if (alloc_descriptor(MAK_ITEMS, MAK_UNAME_LEN, MAK_IAME_LEN)
        != 0)
        exit(1);

    sqlprintf(statement,argv, "select * from %s", argv[2]);
    statement_len = strlen(statement,argv);

    /* Prepare the statement and Declare a cursor */
    EXEC SQL PREPARE S FROM :statement;
    EXEC SQL DECLARE C CURSOR FOR S;

    bind_dp->N = MAK_ITEMS;
    EXEC SQL DESCRIBE BIND VARIABLES FOR S INTO bind_dp;

    /* If "P" is negative, there were more bind variables
       than we originally allocated using "sqlfld" */

    if (bind_dp->P < 0) {
        printf("\nToo many bind variables for descriptor.");
        goto sqlerror;
    }

    /* Set the max number of array elements to the number found */

```

```

kind_dp->N = kind_dp->F;

EXEC BQL OPEN C USING DESCRIPTOR kind_dp;
EXEC BQL DESCRIBE SELECT LIST FOR B INTD select_dp;

/* If "F" is negative, there were more select list variables
   than was originally allocated using "sqlaid" */

if(select_dp->F < 0) {
    printf("\nYou may select list variables for descriptor:
           %d", -(select_dp->F));

    goto sqlerror;
}

sprintf(filenames, "%s.dat", argv[2]);
sprintf(ctl_filename, "%s.ctl", argv[2]);
fp      = fopen(filenames, "a");
fp_ctl  = fopen(ctl_filename, "a");

printf("\n Table %s data unloading ..... \n", argv[2]);

/* Set the max number of elements in descriptor to that found */

select_dp->N = select_dp->F;

/* Now allocate storage for each of the SELECT list variables */

/* Control File을 생성하는 부분(Header) */
printf("Control file creating ..... \n");

sprintf(fp_ctl, "load data\n");
sprintf(fp_ctl, "infile '%s'\n", filenames);
sprintf(fp_ctl, "insert\n");
sprintf(fp_ctl, "into table %s\n", argv[2]);
sprintf(fp_ctl, "fields enclosed by R'22' \n\n");

for(i=0; iF; i++) {

    sqlaid (&(select_dp->V[i]), &(select_dp->V[i]), &null_ok);

    switch (select_dp->V[i])
    {
        case 1 : /* CHAR datatype */
            break;

        case 2 : /* NUMBER datatype */
            sqlprec(&(select_dp->N[i]), &precision, &scale);
            /* Max size of NUMBER */

```

```

        If(precision = 0) precision = 40;
/* Also allow for decimal point and possible sign */
        select_dp->L[1] = precision;
/* allow for negative scale */
        If(scale < 0) select_dp->L[1] += -scale;
        break;

case 8 : /* LONG datatype */
        select_dp->L[1] = 240;
        break;

case 12 : /* DATE datatype */
        select_dp->L[1] = 8;
        break;

case 23 : /* RAW datatype */
        break;

case 24 : /* LONG RAW datatype */
        select_dp->L[1] = 240;
        break;
}

If(select_dp->T[1] != 2)
        select_dp->U[1] = (char *) realloc(select_dp->U[1],
                select_dp->L[1] + 1);
else
        select_dp->U[1] = (char *) realloc(select_dp->U[1],
                select_dp->L[1]);

/* Convert ALL datatypes except for LONG RAW's to character */
If(select_dp->T[1] != 24) select_dp->T[1] = 1;

If(select_dp->T[1] == 2)
        If(scale > 0) select_dp->T[1] = 4;
        else
                select_dp->T[1] = 3;

/* Control File을 생성하는 부분 */

If(select_dp->T[1] == 2) {
        If(select_dp->F != 1+1) {
                fprintf(fp_ctl, "%s.%s"   CHAN(xd) NULLIF "%s.%s"=BLANKS,
                        "\n",select_dp->C[1],select_dp->S[1],
                        (1st)select_dp->L[1],select_dp->C[1],
                        select_dp->S[1]);
        }
        else {
                fprintf(fp_ctl, "%s.%s"   CHAN(xd) NULLIF

```

```

        "\x,%c\x"=BLANKS \n", select_dp->C[1],
        select_dp->B[1], (int)select_dp->L[1],
        select_dp->C[1],select_dp->B[1]);
    }
}
else {
    if(select_dp->F != 1+1) {
        fprintf(fp_ctl, "\x%-,%c\x" CHAR(2d) NULL IF
            "\x%-,%c\x"=BLANKS \n", select_dp->C[1],
            select_dp->B[1], (int)select_dp->L[1],
            select_dp->C[1],select_dp->B[1]);
    }
    else {
        fprintf(fp_ctl, "\x%-,%c\x" CHAR(2d) NULL IF
            "\x%-,%c\x"=BLANKS \n", select_dp->C[1],select_dp->B[1],
            (int)select_dp->L[1], select_dp->C[1],select_dp->B[1]);
    }
}

}

} /* end of for */

fprintf(fp_ctl, "\n");
fclose(fp_ctl);

EXEC SQL WHENEVER NOT FOUND GOTO end_select_loop;

for (;;)
{
    EXEC SQL FETCH C USING DESCRIPTOR select_dp;

    for (i = 0; i < select_dp->F; i++)
    {
        if(select_dp->I[i] < 0)
            if (select_dp->Y[i] == 4)
                fprintf(fp, "\x%-,%c\x", (int)select_dp->L[i]+3, ' ');
            else
                fprintf(fp, "\x%-,%c\x", (int)select_dp->L[i], ' ');
            else
                if(select_dp->Y[i] == 3) /* int datatype */
                    fprintf(fp, "\x%0d\x", (int)select_dp->L[i],
                        *(int *)select_dp->U[i]);
                else if(select_dp->Y[i] == 4) /* float datatype */
                    fprintf(fp, "\x%0.2f\x", (int)select_dp->L[i],
                        *(float *)select_dp->U[i]);
                else /* character string */
                    fprintf(fp, "\x%-,%c\x", (int)select_dp->L[i],
                        (int)select_dp->L[i], select_dp->U[i]);
    }
}

```

```

    }
    fprintf (fp, "%n");
}

end_select_loop:
    printf ("%n\n Unloaded to %s (rowid %d) %n",
            filename, xglcn, xglrowid[2]);

    /* Free up space used by descriptors (gracefull exit) */
    /* 1. Free the "malloc"ed space for the select list variables */
    for (i=0; iF; i++) {
        free (select_dp->Q[i]);
        free (select_dp->I[i]);
    }

    /* 2. Free up space used by the descriptors themselves */
    xglclu (hld_dp);
    xglclu (select_dp);

    fclose(fp);

EXEC SQL CLOSE C;
EXEC SQL COMMIT WORK RELEASE;

return;

/* Define Error Handler */
xglerror:
    printf ("%s %d\n", xglcn, xglrowid, xglerror);
EXEC SQL UNDOOVER SQLERRM CONTINUE;
EXEC SQL ROLLBACK WORK RELEASE;

return;
}

alloc_descriptors(xlcn, max_vname_len, max_fname_len)
int xlcn;
int max_vname_len;
int max_fname_len;
{
    int i;

    if ((hld_dp =
        xglld(xlcn, max_vname_len, max_fname_len)) == (SQLDA *) 0)
    {
        fprintf(stderr,
            "Cannot allocate memory for hld descriptor.");
        return -1; /* Have to exit in this case. */
    }

    if ((select_dp =

```

```

    sqlfld (size, max_value_len, max_index_len) = (BQLDA *) 0)
{
    fprintf(stderr,
        "Cannot allocate memory for select descriptor.\n");
    return -1;
}

select_dp->N = MAX_ITEMS;

/* Allocate the pointers to the indicator variables, and the
   actual data. */
for (i = 0; i < MAX_ITEMS; i++) {
    bind_dp->I[i] = (short *) malloc(sizeof (short));
    select_dp->I[i] = (short *) malloc(sizeof (short));
    bind_dp->U[i] = (char *) malloc(1);
    select_dp->U[i] = (char *) malloc(1);
}

return 0;
}

```

NETWARE LISTENER 활용 (MAX USERS)

Bulletin no : 10034

현재 netware listener(xpxsrv)의 최대 연결가능 사용자수가 기본으로 30명이 설정되어 있습니다.
이를 변경하려면 다음 parameter를 추가하세요.

어디에 : netware server의 SYS:SYSTEM\config.scr

주먹들 : BPK_MAX_CLIENTS = 사용자수 (DEFAULT:30)

NETWARE SUPPORT MATRIX

Bulletin no : 1031B

NetWare	I	I	I 3.11	I	I	I 4.1	I 4.1		
Version	I 3.11	I 3.12	IBPTIII	I 4.01	I 4.02	I 4.1	IBHP	IBPTIII	
B.0.3B/B.0.3P	I Yes	I Yes	I Yes	I No	I No	I No	I No	I No	
7.0.15.4.x	I Yes	I Yes	I Yes	I Yes	I Yes	I No	I No	I No	
7.0.15.5.0 ^	I No	I Yes	I No	I No	I No	I No	I No	I No	
7.0.15.5.x	I Yes	I Yes	I Yes	I Yes	I Yes	I No	I No	I No	
7.0.1B.8.x	I Yes	I Yes	I Yes	I Yes	I Yes	I No	I No	I No	
7.1.3.3.x	I Yes	I Yes	I Yes	I Yes	I Yes	I Yes	I No	I No	
7.1.4.3.x	I Yes	I Yes	I Yes	I Yes	I Yes	I Yes	I No	I No	
7.2.2.4.x	I Yes	I Yes	I No	I No	I No	I Yes	I Yes	I Yes	
7.3.2.1.x *	I No	I Yes	I No	I No	I No	I Yes	I Yes	I Yes	

DRACLES의 RELEASE NOTE에 보면 NETWARE에 대한 PATCH를 요구하는 경우가 있으므로 해당 PATCH를 MODEL로부터 찾아서 INSTALL을 한 다음에 DRACLES을 설치해야 한다.

7.0.15.5.0 버전은 DracInWare를 위한 특별버전으로 기존에 이 버전을 사용하던 고객은 모두 UPDATE를 하였으므로 이 버전은 현재 사용되지 않는다.

NLS CHARACTERSET 변경 방법 (DB REBUILD 없이)

Bulletin no : 10018

데이터베이스의 CHARACTER SET은 데이터 디렉터리 레이블인 `sys.props`에 들어 있다

```
BQL>desc sys.props$
```

Name	Null?	Type
NAME	NOT NULL	VARCHAR2(30)
VALUE\$		VARCHAR2(2000)
COMMENT\$		VARCHAR2(2000)

```
BQL>column c1 format a30
```

```
BQL>select name c1, value$ c1 from sys.props$;
```

c1	c1
DICT_NAME	2
NLS_LANGUAGE	AMERICAN
NLS_TERRITORY	AMERICA
NLS_CURRENCY	\$
NLS_ISO_CURRENCY	AMERICA
NLS_NUMERIC_CHARACTER	.
NLS_DATE_FORMAT	DD-MON-YY
NLS_TIME_ZONE	AMERICAN
NLS_CHARACTERSET	US7ASCII
NLS_BINARY	BINARY
GLOBAL_DB_NAME	MLBUP,MLBUP

여기서 `NLS_CHARACTERSET`에 현재 DB의 `CHARACTER SET`이 들어 있는데 이 값을 변경하여 DB의 `CHARACTER SET`을 변경할 수 있다. 여기서는 `US7ASCII`에서 `EU16CS801`로 옮기는 경우를 알아보자.

우선 바꾸고자 하는 `CHARACTER SET`이 지원되는 지를 다음 명령으로 확인한다.

```
select convert('a','EU16CS801','US7ASCII') from dual;
```

만약 이 Select 문에서 `ORA-01432` 에러가 발생하면 지정한 `CHARACTER SET`이 지원하지 않는 경우이며 에러가 발생하지 않으면 `CHARACTER SET`을 변경할 수 있다.

작업을 하기전에는 만약을 위해서 DB 전체를 백업 받아주도록 한다.
CHARACTER SET 을 잘못 설정하면 DB 를 오픈 할수가 없기 때문이다.

1. 다음의 Update문을 실행하여 CHARACTER SET을 변경한다.

```
UPDATE sys.props$  
SET value$ = 'E01B8E8C5B01'  
WHERE name = 'NLS_CHARACTERSET';
```

Update 시에 NLS_CHARACTERSET을 지원하지 않는 값으로 잘못 설정하거나
실수로 큰따옴표 문자 같은 것이 들어가게 되면 DB가 Shutdown 된 다음에는
Startup 이 안 되므로 Update 후에 다음 경험으로 확인을 한 다음에 Commit을
하도록 한다.

```
select name, value$  
from sys.props$  
where value$ = 'E01B8E8C5B01';
```

Select 제대로 출력되면 Commit 하고 Shutdown 했다가 Startup 하게 되면
새로운 CHARACTER SET 값을 갖게 된다. SELECT가 안 되면 ROLLBACK하고 UPDATE
부터 다시 하도록 한다.

2. 환경 변수 NLS_LANG 을 변경한다.

.profile (or .cshrc)에서

```
NLS_LANG=American_American, E01B8E8C5B01: export NLS_LANG
```

or

```
setenv NLS_LANG American_American, E01B8E8C5B01
```

OPEN VMS 시스템에서의 ORACLE AUTO STARTUP

Bulletin no : 10132

* OpenVMS 의 SYSTEM Account는 DBA 관련 작업을 할 수 없게 되어 있기 때문에 오라클의 Startup도 할 수 없다. 이 문제를 해결하기 위해서는 두가지 방법을 생각해 볼 수 있다.

[방법1] 시스템의 기동 시에 DBACLE7 에 의해서 작업이 수행되도록 설정한다.

1. 다음과 같은 내용을 갖는 프로시저(BTASTUP_DBACLE7.COM)를 만든다.

```
$ddcm:[oracle_root_directory,UTIL]DBAUSER.COM
$OROA_BNAME:INBDBACLE.COM
$ddcm:
[oracle_root_directory,DB_dname]BTASTUP_EXCLUDE_DNAME.COM
```

여기서

```
ddcm:                : 오라클이 민스플런 디스크
oracle_root_directory : 오라클의 최상위 디렉토리
dname                : 오라클 데이터베이스 이름
```

만약 경험 서버를 사용중이라면 프로시저 이름을 BTASTUP_EXCLUDE_DNAME.COM에서 BTASTUP_EXCLUDE_DNAME.COM 으로 바꾼다. 그리고 여러 개의 데이터베이스를 Startup 시키려고 한다면 세션과 닮은 같은 항목을 각각의 데이터베이스에 대해서 모두 기술해 주도록 한다.

2. 위에서 만든 프로시저를 DBACLE7 에 의해서 수행되도록 한다.

```
$BIBINIT/DBACLE7 BTASTUP_DBACLE7.COM
```

여기서 DBACLE7은 OpenVMS 의 DBACLE7 의 Account 이다. 필요한 내용이 있으면 BIBINIT 명령에 추가해도 된다. 예를 들어 DBACLE7을 위의 프로시저로 Startup 시킬 때 예머가 발생하는 경우를 고려하여 메시 로그 파일을 저장하려고 한다면 /LOG/NOBIBINIT 스위치를 사용하면 된다.

[방법2] SET UIC 명령으로 DBACLE7 을 Startup 시키기

1. 다음의 명령들을 시스템의 startup procedure 에 추가한다.

```
OpenVMS V5.x: BYB$BTASTUP:BYBTASTUP_V5.COM
OpenVMS V6.x: BYB$BTASTUP:BYBTASTUP_V6.COM

$ SET UIC(DBACLE7)
$ SET RIGHTS_LIST/ENABLE DBA_BNA
$ ddcm:[oracle_root_directory,UTIL]DBAUSER.COM
$ OROA_BNAME:INBDBACLE.COM
```

```

$ ddccs:[osrcis_root_directory, DB_dhname]STARTUP_EXCLUSIVE_dhname.COM
$ SET UIC(BYTEIN)

```

여기서

```

ddccs           : 오라클이 인스톨된 디스크
osrcis_root_directory : 오라클의 최상위 디렉토리
dhname          : 오라클 데이터베이스 이름
DBACLEAR       : DpmUMB DBACLEAR account
SYSTEM         : DpmUMB SYSTEM account

```

만약 컴퓨 서버를 사용중이라면 프로시저 이름을 STARTUP_EXCLUSIVE_dhname.COM에서 STARTUP_PARALLEL_dhname.COM 으로 바꾼다. 그리고 여러 개의 데이터베이스를 Startup 시키려고 한다면 여섯번째 다민과 같은 항목을 각각의 데이터베이스에 대해서 모두 기술해 주도록 한다.

819 절의 DBA_DBID 와 같은 DBACLE 권한 Identifier를 사용한다면 이와 같은 것들도 Enable 시켜야 한다. DBA_DBID 권한 Identifier 를 사용하지 않는다면 이것은 Enable 시키지 말도록 하고 만약 DBA_DBID 와 DBA_DBID 둘 모두 사용한다면 두 지 모두 Enable 시킨다.

2. DpmUMB SYSTEM Account 가 DBA_DBID and/or DBA_DBID 권한 Identifier를 부여하였음을 확인하고 그렇지 않다면 다음과 같이 한다.

```

$ SET DEFAULT SYS@SYSTEM
$ RUN AUTHORIZE
  UAF>GRANT/IDENTIFIED DBA_DBID SYSTEM
    and/or
  UAF>GRANT/IDENTIFIED DBA_DBID SYSTEM
  UAF>EXIT

```

OPS 에서 새로운 NODE 추가 방법

Bulletin no : 10182

1. hardware와 o/s setting(disk)이 완벽한 상태

2. oracle home directory의 software를 모두 copy .

3. 새로운 node의 init.ora file 수정

- o instance_number = 3 (ex: 기존에 두개의 node에서 추가시)
- o thread = 3
- o rollback_segment = () --> 원하는 만큼 설정.

4. primary node에서 작업

- o alter database add logfile thread 3
group 7 '/dev/vst/c0t1d0c1' size 8M,
group 8 '/dev/vst/c0t1d0c2' size 8M,
group 9 '/dev/vst/c0t1d0c3' size 8M;
alter database make public thread 3;

- o 3번에서 지정한 rollback segment를 설정하고 online 시킨다.

5. 추가된 node에서 database를 기동한다. (parallel node)

ORA-54 WHEN DROP A TABLE(SESSION KILL)

Bulletin no : 10080

TABLE에 LOCK이 걸려 DBL 및 DBL 명령 사용시 ORA-54 ERROR 가 나오는 경우가 있다. 이때 DB를 RESTART하면 되지만 더 효율적인 해결 방법은 다음과 같이할 수 있다. (serial number 가 negative 인 경우 그 값에 8538 을 더해야 함)

1. 예를 들어 LOCK 이 걸려 있는 이물이 RHP 라고 가정하면 다음 SELECT문으로 RHP 데이터에 LOCK 을 걸고있는 SESSION을 찾는다.

```
> sqlplus system/manager
```

```
BQL> select  a.sid,a.serial#
        from  v$session a, v$lock b, dba_objects c
        where  a.sid=b.sid and
               b.id1=c.object_id and
               b.type='TM' and
               c.object_name upper('RHP');
```

BID	SERIAL#
5	1
8	1

2. 다음 명령으로 SESSION들을 KILL 한다.

```
BQL> alter system kill session '5,1';
```

```
BQL> alter system kill session '8,1';
```

데이터베이스 STARTUP 시 ORA-312,313 에러 (ONLINE LOG CRASH)

Bulletin no : 10214

[ONLINE LOG 가 손상되었을때 DB에 OPERATION 이 멈췄던 경우는 다음과 같은 절차로 DB 를 OPEN 할수있다 - 확률 70%]

1. CONTROLFILE 생성

- 손상된 online log 는 포함시키지 않는다.
- resetlogs option 으로 생성한다.
- reset option 은 생략하고 기존 controlfile 은 다른이름으로 move 시킴.

```
sqlplus> startup mount
sqlplus> alter database backup controlfile to trace;
```

위와 같이 명령을 입력하면 ORACLE_HOME/oracle/log 디렉토리에 트레이스 파일이 생긴다. 그 트레이스 파일에서 create controlfile 명령부분을 남기고 삭제 한다.

콘트롤파일 생성 문장 예 - : GROUP 1 이 ONLINE LOG 라고 가정

```
CREATE CONTROLFILE DATABASE 'ORCL' RESETLOGS NOARCHIVELOG
  MAXLOGFILES 32
  MAXLOGMEMBERS 2
  MAXDATAFILES 30
  MAXINSTANCES 8
  MAXLOGHISTORY 800
LOGFILE
  GROUP 2 '/oracle/oracle/rlc/log2ORCL.dbf' SIZE 5M,
  GROUP 3 '/oracle/oracle/rlc/log3ORCL.dbf' SIZE 5M
DATAFILE
  '/oracle/oracle/rlc/dataORCL.dbf',
  '/oracle/oracle/rlc/rldORCL.dbf',
  '/oracle/oracle/rlc/torORCL.dbf',
  '/oracle/oracle/rlc/urORCL.dbf',
  '/oracle/oracle/rlc/tempORCL.dbf',
  '/oracle/oracle/rlc/redo.dbf'
;
```


2.절차

```
$ sqlplus / as sysdba
SQL> connect internal
SQL> shutdown abort
SQL> startup mount
      statement processed
SQL> desc
SQL> recover database using backup controlfile until cancel;
      ...
      ...
      CANCEL (Return)
      Recovery canceled
SQL> alter database open resetlogs;
```

: 만일 정상적으로 open 되면 log file 추가

```
SQL> alter database add logfile 'F:\db\log100A722.dbf' size 1M;
```

: 정상적으로 open 만되면 RC에 다시 연결

ORA-600[3339]

DATA BLOCK CORRUPTION DETECTION

Bulletin no : 10081

개 요 : ORA-00800 [3339] [arg1] [arg2] [] [] [] []

ORA-1578 : Data block corrupted in file # block #

ORA-600 에러는 ORACLE이 직접 퍼퍼로 데이터를 읽어들이기 때 읽은 블록의 DBA(Data Block Address)가 잘못되었음(INVALID)을 의미한다. arg1은 실제 읽어들이는 블록의 DBA를 나타내며 arg2는 ORACLE이 읽고자 하였던 블록의 DBA를 나타낸다. arg1과 arg2이 다르면 위에서와 같은 에러가

발생하며 주로 DB나 HUI의 문제가 그 원인이 되는 경우가 많다. 데이터의 손상 원인은 여러가지가 있으며 대개 데이터가 손상될 당 예는 그 사실이 드러나지 않다가 손상된 부분의 데이터가 사용될 때 미묘한 손상을 사실이 알려지게 된다.

ORACLE은 모든 정보를 블록 형태로 관리한다. ORACLE 데이터 블록은 하나 이상의 DB 블록으로 이루어져 있다. 그 크기는 init.ora 파일의 DB_BLOCK_SIZE에 나타나 있다.

ORACLE 데이터 블록은 블록의 정보를 담은 fixed header를 갖고 있으며 이 정보를 이용하여 각각의 블록과 데이터베이스 전체의 INTENTIVITY를 유지한다. DBA는 fixed header의 한 부분으로 32bit 길이의 정수이며 데이터베이스의 파일 번호와 파일에서 블록 위치를 나타낸다.

DBA에 문제가 있으면 ORACLE은 DBI(3339)와 함께 위에서 말한 2개의 인수, 즉 arg1과 arg2를 표시하게 된다. 이 두 값이 다르면 DBI(3339)가 발생하게 되는 것이다.

ORACLE은 파일에 블록을 읽고 쓸때 시스템 정션 줄을 이용한다. 여기 는 lseek(), read(), readv(), write(), writev() 등이 포함된다. 블록이 DB에 의해서 읽히지면 BAO로 들어가며 끝이어서 DBA에 대한 검사가 이루어진다.

원인과 증상:

경우1)

ORA-00800 [3339] [0] [large number] [] [] [] []

ORA-1578: Data block corrupted file # block #

이 경우에는 arg1은 0이고 arg2는 ORACLE이 갖고 있는 DBA이다. 이 에러의 원인은 ORACLE 블록의 일부가 모두 0이기 때문인 경우가 많다.

ORACLE 블록내의 첫번째 DB 블록은 디스크상의 문제로 해서 DB 또는 디스크 복구 프로그램이 해당 블록을 복구하려는 과정에서 모두 0으로 되어버리곤 한다.

유닉스 플랫폼의 경우에는 여러개의 BLOB을 동시에 작동시키면 이와같은 문제가 발생하는 것으로 알려져 있으나 DB.9.33.2 이후 버전에서부터는 수정되었다.

ORA-1578 메시지는 반드시 DBI(3339)를 통괄시키지는 않는다.

원인)

arg1이 0이되는 또다른 원인은 ORACLE의 버그 때문인데 플랫폼그런트가 드롭되기 전 그 플랫폼 그런트를 포함한 데이터스페이스가 먼저 드롭되면 발생한다. 이 문제는 DB.9.32 이후의 버전에서 수정되었다.

경우2)

DBA-00600 (3338) [large number] [large number] [] [] [] []

DBA-1578: Data block corrupted file # block #

원인1)

이런에는 디스크상의 물리적 블록의 BBA가 틀린 경우이다. 이것은 블록이 메모리 상에서 손상되었으나 그대로 디스크에 기록된 경우에 발생한다. 이 문제는 매우 드물게 나타나며 이 때의 BBA는 놓아두려미없는 값이다.

만약 메모리에 문제가 있다고 생각된다면 다음과 같은 event parameter를 init.ora에 추가함으로써 블록 검사를 할 수 있다.

```
event = "10210 trace name context forever, level 10"
```

```
event = "10211 trace name context forever, level 10"
```

DBWR가 디스크에 데이터를 쓰기 전에 캐쉬에서 손상된 블록을 발견하면 DBW1(3338) 메시지를 출력하고 인스턴스를 정지시킬 것이다. 따라서 문제의 블록은 디스크에 저장되지 않는다. 이 때 BBA를 포함한 많은 변수들이 DBW1(3338) 내부 에터 서티기에 전달된다. 이와 같은 경우에는 인스턴스를 다시 기동시키고 이전된 변수와 trace 파일을 가지고 Oracle World Wide Customer Support에 연락하여야 한다.

원인2)

블록이 데이터파일상에 기록될 위치를 잘못 찾는 경우이다. 이것은 "write blocks out of sequence" 라고 불린다. 때 DBW1(3338)에서 표시되는 BBA는 모두 중요한 값이다. 이것은 DBACLE이 lseek()를 호출했을 때 DB가 블록을 잘못된 곳에 기록해서 생기는 경우가 많다.

어떤 HLU/DB 공급업체들은 4.2G 를 넘는 큰 파일을 다룰 수 있는 기능을 제공한다. 이것은 32bit unsigned number로 다룰 수 있는 범위를 벗어나기 때문에 DB는 offset 값을 DBACLE이 사용할 수 있도록 적절히 변환시켜야 한다. DBACLE은 DB의 지원 여부와 관계없이 2G 이상의 파일을 지원하지 않으며 위와같은 큰 파일을 다룰 수 있는 환경에서는 lseek() 시스템 콜이 정확한 위치로의 변환을 시켜주지 못함때마다 보다 작은 크기의 파일에서도 문제가 발생하는 경우도 있다.

원인3)

세번째 원인은 I/O기능이 전혀 작동하지 않는 경우이다. 이 경우에 표시되는 BBA 값은 모두 중요한 값들이지만 arg1은 이전에 BBA로 입력된 블록의 BBA 값이 된다. DBACLE은 lseek(), read() 시스템 콜이 진행되는 에터 코드를 검사하며 read()가 읽어들이는 바이트 수가 BLOCK SIZE의 정수배인지를 검사한다. 이 검사를 통과하면 DBACLE은 성공적으로 BBA가 수행되었다고 가정한다. 만약 BBA가 정확하지 않다고 체크되면 DATAFILE에 대한 읽기 요구는 실패하기 때문에 실제의 블록 읽기는 일어나지 않는다. 이 경우에 이전된 BBA는 실제로는 다른 파일의 다른 블록을 지시하게 될 수도 있다.

원인4)

또 다른 원인은 동일한 디바이스에서 다른 블록을 읽어온 경우이다. 이것은 작업이 매우 빠른 디스크에서 발생하는 한다. 어떤 경우에는 수백개 이상 떨어진 곳의 블록을 읽어오는 경우도 있다. 이때에도 BBA 값은 모두 중요한 값이다.

세번째와 네번째 경우에는 다시 한번 동작을 반복함으로써 문제는 해결될 수 있으며 이것은 일반적으로 DBACLE의 문제가 아니라 DB나 HLU의 문제인 경우가 많다.

DBW1(3338)에서 표시되는 변수들의 값은 플랫폼에 따라 다르다.

ORA-604 조치 방법

Bulletin no : 10082

이 예머는 내부적으로 BQL명령이 실행될 때 발생한다. 예를 들어 현재 할당된 익스텐트가 가득 차서 다음 익스텐트를 할당 받으려고 할 때 오라클이 다음 익스텐트의 크기와 위치를 설정하기 위하여 BQL명령을 내리게 되는 것과 같은 경우이다.

이 문제가 발생하 우선 `limit.log` 파일을 검사하여 ORA-800 과 같은 예머가 발생했는가를 확인한다. ORA-800 예머가 발생했다면 오라클측에 지원을 요청하도록 하고 그렇지 않다면 다른 원인을 검사해 봐야 한다.

가장 먼저 고려할 사항은 `?/dba/limit.ora` 파일에 지정된 `opm_curmax` 의 크기를 알아보는 것이다. 이 값이 설정이 만치되어 있으면 default가 50이므로

```
opm_curmax=255
```

과 같이 설정하도록 한다. 이 값을 단지 커서의 최대 값을 지정하는 것이므로 커서를 적게 쓰는 프로그램에 아무런 영향을 끼치지 않는다. `opm_curmax`를 편집하고 DB를 Shutdown 하고 Startup 시키면 된다.

만약 이 방법으로 해결이 안되면 다음의 방법을 따른다.

정확한 예머의 원인을 찾기 위해서 `limit.ora` 파일에 다음과 같은 라인을 추가한다.

```
events = "ORA trace name errorstack"
```

이렇게 `limit.ora`를 편집하고 DB를 Shutdown 하고 Startup 하면 ORA-604 예머가 발생하는 경우에 자세한 정보를 `Trace` 파일에 기록해 주므로 이 파일을 검사하여 예머의 원인을 찾을 수 있다.

예머의 다른 원인으로는 `limit.ora` 파일의 파라미터 가운데 `BC_FREE_EXTENTS` 나 `SQL_CACHE_REQUIRE` 의 값이 너무 작게 설정된 경우를 생각해 볼 수 있다.

이와같은 경우는 이들 값을 크게 설정해 주도록 한다.

데이터 스페이스가 가득 차거나 Extent 갯수의 최대 허용값을 초과해서 예머가 발생하는 경우 ORA-604 예머가 함께 발생할 수가 있는데 이와같은 경우에는 이들 문제를 먼저 해결하면 ORA-604 예머는 함께 해결 된다.

ORA-1118

조치 방법(MAXDATAFILES 와 DB_FILES)

Bulletin no : 10088

데이터 스페이스를 만들거나 데이터 파일을 추가하다 보면

ORA-01118: cannot add any more database files: limit of MAX exceeded

과 같은 에러가 발생하는 경우가 있다. 오라클의 데이터 파일의 최대 갯수는 MAXDATAFILES와 DB_FILES 에 의해서 제한을 받는데 여기서는 이 에러를 해결하는 방법을 알아보기로 한다.

데이터베이스를 처음 만들 때 사용되는 CREATE DATABASE 명령에서 MAXDATAFILES 라는 파라미터를 찾아볼 수 있다. 여기서 지정된 값(명시를 안하면 디폴트로 설정)은 콘트롤 파일에 기록된다. 이 값은 데이터베이스에 대해서 설정된 최대 데이터파일 갯수이다. 이 값을 변경하려면 콘트롤 파일을 다시 만들어야 한다.

한편, init.ora(UNIK에서는 init.ora, UDB에서는 _init.ora)에는 DB_FILES 라는 파라미터가 있는데 이 값은 해당 인스턴스에 대해서 지정된 최대 데이터파일 갯수이다. DB_FILES는 단순히 에디터 상에서 init.ora 파일을 수정한 다음에 DB를 Restartup 하면 새로운 값이 적용된다.

1. 왜 MAXDATAFILES 과 같은 한계값을 설정하는가?

→ UDB는 오라클 파일의 갯수를 지정하기 위하여 특정한 갯수 limit를 사용하며 이 값이 Platform에 따라 다르다. MAXDATAFILES는 이 값의 범항 하에 있게 된다.

일반적인 최대 값은 다음과 같다.

	UB	U7
UNIK	82	1022
UDB	254	1022
OSB	254	N/A

[참조]일부 유닉스 Platform상의 U7.0.1B 이전 버전의 경우 1022보다 작은 경우도 있다.

2. 왜 MAXDATAFILES 값을 가능한 최대로 설정해 두지 않는가?

→ MAXDATAFILES 를 크게 지정하면 그만큼 콘트롤 파일의 크기도 늘어나기 때문이다.

3. 왜 DB_FILES를 MAXDATAFILES 의 크기만큼 크게 설정해 두지 않는가?

→ DB_FILES를 늘리면 각 User Process 에 할당되는 PGA(Program Global Area) 의 크기가 커지기 때문이다.

4. 사용하는 시스템 MAXDATAFILES를 알 수 있는 방법은?

→ 시스템에 따라서 다르므로 Installation Guide 를 참조해야만 한다.

5. 콘트롤 파일의 위치를 아는 방법은?

→ BQLDBA>show parameter control_files;

또는

BQLDBA>select * from v\$controlfile; (7.0.13 이상)

위의 Query 를 이용하거나 \$ORACLE_HOME/dbs/config.ora 파일을 보면 알 수 있다.

[DBA-11B 해결방법]

→ DBA-11B 에서는 데이터 파일의 갯수가 MAXDATAFILES 값에 도달한 경우 발생한다. DB_FILES 값에 도달한 경우라면 DBA-SB 에러가 발생한다. DBA-SB 에러는 init.ora 의 DB_FILES 를 늘려주고 DB 를 Restarting 하면 해결 되지만 DBA-11B 에러는 이와같이 간단하게 해결되지는 않는다. 다음과 같은 방법이 있다.

1. 여러개의 데이터파일로 구성된 데이터 스페이스가 있으면 이를 Export 라고 데이터스페이스를 Backup한 다음 하 리 큰 데이터파일을 갖도록 데이터 스페이스를 만들고 Import를 한다.

2. 9B.0.33 이전 버전을 사용중이라면 MAXDATAFILES를 늘리기 위해서는 DB를 새로 만들어야 하며 그 이후 버전을 사용중이라면 콘트롤 파일을 새로 만들어서 MAXDATAFILES를 늘릴 수 있다.

1. DB가 mount 또는 open 된 상태에서

BQLDBA>alter database backup controlfile to trace;

과 같은 명령을 내리면 예 지정된 디렉토리 -

\$ORACLE_HOME/admin/log - 에 트레이스 파일이 하나 생긴다. 이 파일 찾으면 해당 디렉토리에서 가장 최근에 생긴 트레이스 파일을 찾으면 된다.

이 파일을 다른 이름으로 복사한 다음 에디터로 열어서 CREATE CONTROLFILE 명령 부분 외의 불필요한 부분과 'REUSE' OPTION 을 지우고 MAXDATAFILES를 늘려 준다. 이 파일의 이름을 mount1.sql 로 하기로 한다.

2. \$ORACLE_HOME/dbs 에 있는 Control file 을 다른 이름으로 MOVE 한다.

3. DB를 NOMOUNT 또는 IMMEDIATE로 Shutdown 하고, 콘트롤 파일 생성시 DB 명령을 줄 수 있기 때문에 만약을 위해서 DB 전체를 백업 받도록 한다.

4. 현재 사용중인 콘트롤 파일을 다른 이름으로 옮기고 다음을 실행한다.

BQLDBA>startup nomount

BQLDBA>mount

BQLDBA>alter database open;

5. 이제 필요한 모든 작업은 끝났지만 여기서 다시 한번 Full Backup 을 받는 것이 좋다.

[참고] 다음은 콘트롤 파일을 생성하는 명령의 예이다.

BQLDBA> STARTUP NOMOUNT

BQLDBA> CREATE CONTROLFILE REUSE

DATABASE DATABASE

```

LOGFILE '/u00000/oracle/disk/log10BACLE.dbf',
        '/u00000/oracle/disk/log20BACLE.dbf',
        '/u00000/oracle/disk/log30BACLE.dbf'
RESETLOGS
DATAFILE '/u00000/oracle/disk/sysdBACLE.dbf',
        '/u00000/oracle/disk/rbdbACLE.dbf',
        '/u00000/oracle/disk/tempdBACLE.dbf',
        '/u00000/oracle/disk/tao1dBACLE.dbf',
        '/u00000/oracle/disk/uxr0dBACLE.dbf'
NAMEDATAFILES 121;
SQL> ALTER DATABASE OPEN RESETLOGS;

```

ORA-1547 조치 방법

Bulletin no : 10083

ORA-1547 에러 발생의 원인은 TABLESPACE가 에러에 걸린 DBFILE block 수 만큼의 요청된 EXTENT를 할당할 충분한 FREE SPACE를 갖고있지 못할 경우에 발생된다. 현재의 EXTENT가 실제 데이터 파일에 존재하는 FREE SPACE 중에서 할당 가능한 연속적인 DBFILE block를 요구하게 된다.

< ORA-1547 에러가 발생하는 경우 >

1. 데이터 INSERT나 UPDATE시 DATA BLOCK이 차지하게될 연속적인 DBFILE 블록을 할당할지 못할 경우에 발생한다.

2. 인덱스를 생성할 경우에 발생한다.

- DATABASE BLOCK이 사용할 SBB 또는 USER TABLESPACE의 영역이 부족하여 발생할 수 있다.
- 인덱스 생성시 SORT 영역으로 사용되는 TEMPORARY TABLESPACE의 SPACE의 부족으로 발생할 수 있다.

3. SQLFORM30, SQLFORMUTILITY등의 프로그램들 데이터베이스에 [BASE] 관련 데이터들을 포함하고 있는 BYTEXT 또는 TOOLS TABLESPACE등의 영역이 부족한 경우에 발생된다.

< ORA-1547 에러 확인 >

이러한 경우 EXTENT에 관련된 DATA DICTIONARY VIEW인 USER_TABLES, USER_EXTENTS, USER_BLOCKS와 DBA_FREE_SPACE등을 조회해서 관련 내용을 확인한다.

예를 어, 데이터 INSERT시 "ORA-1547 : Failed to allocate extent of size 'num' in tablespace 'TOOLS'" 에러가 발생할 경우를 고려해 보자.

1. [USER_TABLES]에서 INSERT에 관련된 데이터의 NEXT_EXTENT 크기를 확인 한다.

```
SQL> SELECT * FROM USER_TABLES
WHERE TABLE_NAME = 'EMP';
```

INITIAL_EXTENT	NEXT_EXTENT	PCT_INCREASE	MIN_EXTENTS	MAX_EXTENTS
10240	180484	50	1	121

(A)

(A) : 다음에 할당되는 EXTENT의 크기를 나타내며 BYTES 단위이다.

2. [DBA_FREE_SPACE]에서 현재 TABLESPACE에 존재하는 FREE SPACE 중 가장 큰 연속된 영역을 확인한다. [DBA_FREE_SPACE]는 SQLFORM에서 확인한다.

```
SQLFORM> SELECT MAX(BYTES) MAX_CONTIGUOUS_SPACE
FROM DBA_FREE_SPACE
WHERE TABLESPACE_NAME = 'TOOLS';
```


MAX_CONTIGUOUS_BYTES

18730432

(B)

(B) : 현재 TABLESPACE에 남아있는 FREE SPACE 중 가장 큰 연속된 영역으로 BYTES 단위로 나타낸다.

3. 위에서 살펴본바와 같이 2)-(B)의 MAX(BYTES) 크기가 1)-(A)의 NEXT_EXTENT 크기보다 커야하므로 최소 1)-(A)의 NEXT_EXTENT 크기 이상의 데이터 파일을 "TOOLB" TABLESPACE에 "ALTER TABLESPACE tablespacename ADD DATAFILE..." 명령을 이용하여 추가한다.

< ORA-1547의 처리 방법 >

1. 해당 TABLESPACE내에서 연속된 영역의 DOUBLE block 할당할 수 있도록 데이터 파일을 추가한다.
3) 에서 "TOOLB" TABLESPACE 에 영역을 추가하는 방법 다음과 같다.

```
$ sqlplus system/manager  
SQL> ALTER TABLESPACE toolb ADD DATAFILE  
      '/u01/./toolb00007.dbf' SIZE 50M;
```

<참고>DATAFILE 의 이름 중 '/u01/./toolb00007.dbf' 는 이미 존재하는 DATAFILE 이름과 다르게 함.

2. TABLE의 STORAGE PARAMETER에서 INITIAL EXTENT, NEXT EXTENT의 크기를 조정하여 TABLE을 재구축할 수 있다. 즉, TABLE의 STORAGE PARAMETER 중에서 NEXT를 현재 TABLESPACE에 남아 있는 FREE SPACE 중 가장 큰 연속된 영역(MAX_FREE_SPACE의 MAX(BYTES)) 다 각계 편정할 수 있다.

예를 들면, "ALTER TABLE..." 명령을 이용하여 다음과 같이 재 조정한다.

```
SQL> ALTER TABLE EMP STORAGE ( NEXT 100K );
```

3. 다음의 방법으로는 완전 TABLESPACE를 재구성하는 것이다. 여기서는 TABLESPACE를 재 구축한 후 TABLE을 시 생성함으로써 SIZE FRAGMENTATION을 없앨 수 있다.

ORA-1552 조치 방법

Bulletin no : 10088

ORA-1552 (CANONLY USE SYSTEM BOLLBACK BECHMENT FOR NON-BYTE TABLESPACE 'X') 에서는 SYSTEM TABLESPACE 이외의 TABLESPACE를 포함한 DEEDATION을 위하여 SYSTEM TABLESPACE의 BOLLBACK BECHMENT를 사용할 경우에 발생하는 현상이다. SYSTEM TABLESPACE가 추가적 로 하나의 BOLLBACK BECHMENT를 포함하기 전에는 새로운 TABLESPACE에 임의의 데이터베이스 오브젝트를 생성할 수 없다.

ORA-1552 발생 원인을 해결 하기위해 SYSTEM TABLESPACE에 하나 이상의 BOLLBACK BECHMENT를 추가한 다음, 데이터베이스 오브젝트를 생성한다. CREATE BOLLBACK BECHMENT 문에서 PRIVATE/PUBLIC로 BOLLBACK BECHMENT를 생성한 후, DBACLE 데이터를 SHUTDOWN, STARTUP 한다. PRIVATE으로 생성된 BOLLBACK BECHMENT는 INITPID.ORA 파일의 "BOLLBACK_BECHMENTS" PARAMETER에 등록한다.

< DBACLE US>

\$ sqlplus

SQL> CONNECT INTERNAL;

SQL> CREATE BOLLBACK BECHMENT NO;

\$ORACLE_HOME/dbs/initPID.ora 파일의 "BOLLBACK_BECHMENTS" PARAMETER에 NO를 등록한다.

SQL> SHUTDOWN

SQL> STARTUP

< DBACLE?>

\$ sqlplus /nolog

SQL> CONNECT INTERNAL

SQL> CREATE BOLLBACK BECHMENT NO;

SQL> ALTER BOLLBACK BECHMENT NO ONLINE;

ORA-1555 조치 방법

Bulletin no : 10105

ORA-1555 Snapshot Too Old 에러가 발생하는 원인은 여러가지가 있을 수 있다.

이를 이해하기 위해서 오라클의 내부 메커니즘에 대해서 알아볼 필요가 있다. 오라클은 항상 Statement-level Read Consistency를 유지한다. 즉, 하나의 Query 에서 읽어들이는 값은 그 Query 가 발생한 시점의 값, 즉 Snapshot 을 반영한다. 따라서 Query 가 진행되는 동안 데이터가 변하더라도 Query의 결과값에는 아무런 영향을 끼치지 않는다.

오라클은 특정한 시점을 SCN(System Change Number)로 관리한다. 달리 말하면 SCN은 특정한 시점에서 데이터베이스의 상태라고도 할 수 있다. Read Consistency를 유지하기 위하여 Query가 실행되는 순간 해당 시점의 SCN이 세팅된다. 어떠한 Query는 최초 시점 이후의 SCN 값을 갖는 데이터를 읽어들이는 것은 안된다.

Read Consistent Snapshot을 구현하기 위한 방법으로 물락 세그먼트가 사용된다. 데이터가 변경되면 변경되기 이전의 값을 물락 세그먼트에 기록되고 데이터 물락 헤더에는 변경되기 이전 값이 기록된 물락 세그먼트 물락의 위치가 기록된다. 모든 데이터 물락에는 가장 최근의 커밋 시점의 SCN이 기록되어 있다. Query가 진행되는 데이터 물락에서는 Query의 SCN 보다 이전의 SCN값을 갖는 물락만이 임혀지고 그 이후의 SCN 값을 갖는 물락과 커밋되지 않은 값을 갖는 물락에 대한 Query는 물락 세그먼트에서 임혀지게 된다. 이 때 만약 물락 세그먼트에 원하는 물락이 없어서 읽어들이 못하게 되면 ORA-1555 에러가 발생하게 된다.

물락 세그먼트는 트랜잭션이 끝나기 전까지 변경된 데이터의 원래 상태를 기록하고 있다. 그런데 트랜잭션이 종료되면 물락 세그먼트에 의해 점유된 영역은 Free 상태로 되면서 다른 트랜잭션에 의해 덮어쓰워질 도 있게 된다. 따라서 Query가 필요로하는 값을 가진 물락 세그먼트 물락이 더 이상 존재하지 않는 상황에서 그 물락을 요구하게 되면 ORA-1555 에러가 발생하게 되는 것이다.

ORA-1555 에러가 발생하는 경우

1. 데이터의 변경이 심한 데이터베이스에서 물락 세그먼트의 갯수와 크기가 작은 경우에 발생한다. 많은 트랜잭션이 데이터를 자주 변경하고 커밋하게 되면 커밋된 트랜잭션이 이용하던 물락 세그먼트 공간을 다른 트랜잭션이 이용하게 될 가능성이 많아진다.

따라서 긴 Query의 경우 원하는 값을 물락 세그먼트에서 얻고자 할 때 이미 다른 트랜잭션이 그 값이 저장된 공간을 이용해 버리는 결과가 발생할 수 있다. 이와같은 경우에는 크기가 큰 물락 세그먼트를 이용하면 어느정도 예방이 가능하다.

2. 물락 세그먼트가 손상되어 읽을 수 없게 된 경우

3. Fetch Across Commit

한 데이터에 대하여 Query가 커서를 열고 루프 내에서 데이터를 Fetch하고 변경하고 커밋하는 과정에서 발생한다. 이 경우에는 ORA-1555 에러가 자주 발생할 소지가 있는데 예를 들어 커서가 SCN=10에 Open되었다고 하자. 따라서 이 커서에 관련된 Fetch는 SCN<=10 인 물락만을 읽어들이어야 한다. 이 프로그램은 데이터를 Fetch 한 후에 변경하고 다시 커밋하는 과정을 계속 반복하는데 SCN=20에서 커밋했다고 하자. 만약 이후의 Query가 이전의 커밋된 물락의 데이터를 요구할 경우 그 값이 이미

전접되었으므로 물락 세그먼트를 검색 하지만 많은 전접이 있어왔기 때문에 BCM-10인 물락을 찾지 못하고 DMA-1555 에터를 용탈할 수 있다. 이를 방지하기 위해서는 커서가 Opmm된 상태에서는 커밋을 자주하지 않고 물락 세그먼트 크기를 키워 나가도록 한다.

4. Fetch Across Commit and Delayed Block Clean Out

Delayed Block Clean Out 도 이 에터를 용탈할 수 있다. 데이터 물락이 전접되고 커밋되면 오타들은 물락 세그먼트 헤더에 그 프랜잭션이 커밋되었다고 기록하지만 데이터 물락을 바로 전접하지는 않는다 (Fast Commit). 그리고 다음 프랜잭션이 전접된 물락을 요구할 때야 비로소 전접 시키는데 이를 Delayed Block Clean Out 이라고 한다.

예를 들어 3 과 같은 경우에서 두개의 데이터에 대한 Query를 알아보자.

즉, 하나의 데이터로부터 데이터를 Fetch 하고 다른 데이터의 데이터를 전접한다고 하자. 데이터가 한쪽 데이터에서 커밋되고 있지만 데이터를 가져오는 데이터에 대해서는 Clean Out이 이루어지지 않았기 때문에 DMA-1555 에터가 발생할 수 있다. 이 문제는 커서를 사용하기 전에 Full Table Scan을 해주면 예방이 가능하다. 커서가 모은된 상에서의 커밋은 ANSI 표준에는 들어있지 않지만 오타들에서는 지원이 된다. 하지만 DMA-1555 에터를 일으킬 수도 있다는 점에 유의하여야 한다.

ORA-1562 처리 방법

Bulletin no : 10088

INFOB2등의 대량 데이터를 입출력으로 처리시 발생하는 ORA-1562(Failed to extend rollback segment(id = %s)) 에러에 대하여 살펴보자

ORA-1562 에러 발생의 원인으로선 현재 사용중인 ACTIVE 상태의 ROLLBACK SEGMENT가 다음 EXTENT를 할당하고자 할 경우, 해당 ROLLBACK SEGMENT에 대하여 할당 가능한 최대 EXTENT 수를 초과할때 발생된다. DATA DICTIONARY VIEW중 DBA_ROLLBACK_SEGS를 살펴보면 MAX_EXTENT에 지정된 수 만큼의 EXTENT가 할당될 수 있음을 알수 있다.

```
SQL> SELECT * FROM DBA_ROLLBACK_SEGS;
```

SEGMENT_NAME	SEGMENT_ID	INITIAL_EXT	NEXT_EXT	MIN_EXTENT	MAX_EXTENT
SYSTEM	0	51200	51200	2	121
ROLL	1	131072	131072	2	121
ROLL	2	131072	131072	2	121
ROLL	3	131072	131072	2	121
ROLL	4	131072	131072	2	121

예를 들어, 현재 사용중인 ROLLBACK SEGMENT인 ROLL(SEGMENT_ID=1)이 NEXT_EXTENT(131 KB) 크기의 EXTENT를 MAX_EXTENT인 121개까지 확장시킨 다음, 122번째를 할당시킴 경우에 ORA-1562 에러가 발생된다. 이것은 상대적으로 적은 NEXT_EXTENT의 크기(131 KB)로 인하여 EXTENT가 빈번하게 발생되어 MAX_EXTENT 수에 도달하는 경우이다.

대부분의 시스템에서는 한 ROLLBACK SEGMENT에서 확장 가능한 최대 EXTENT 수를 121개로 제한한다. 이러한 경우 ALTER ROLLBACK SEGMENT 문장에 의해서 EXTENT의 최대 수를 증가 시키더라도 시스템 제한으로 인해 아무런 영향을 미치지 못하므로 ROLLBACK SEGMENT의 재설정시 요구된다.

해당 ROLLBACK SEGMENT를 재설정하기 위해서는,

첫번째 문제의 ROLLBACK SEGMENT를 OFFLINE 시킨후 ROLLBACK SEGMENT를 DROP 한다.

```
SQL> ALTER ROLLBACK SEGMENT ROLL OFFLINE;
```

```
SQL> DROP ROLLBACK SEGMENT ROLL;
```

두번째 ROLLBACK SEGMENT 생성시 TABLESPACE 접두어 이용하여 재설정한다.

ROLLBACK SEGMENT를 재설정시 INITIAL_EXTENT의 크기와 NEXT_EXTENT의 크기를 증가시킴으로써 EXTENT 발생 횟수를 줄일 수 있다.

```
SQL> CREATE ROLLBACK SEGMENT ROLL TABLESPACE( INITIAL IN NEXT IN )
```

```
TABLESPACE ROLL;
```

```
SQL> ALTER ROLLBACK SEGMENT ROLL ONLINE;
```

만약, ORA-1562에서 명시된 ROLLBACK SEGMENT의 SEGMENT_ID가 "SYSTEM" TABLESPACE에 포함된 "SYSTEM" ROLLBACK SEGMENT인 경우에는 DROP ROLLBACK SEGMENT문을 수행한 후, 재설정할 수 없음을 명시하자. 이러한 경우에는 현재의 프랜잭션이 "SYSTEM" ROLLBACK SEGMENT가 아닌 다른 TABLESPACE에 포함된 ROLLBACK SEGMENT를 이용할 수 있도록 지정후, 프랜잭션을 수행할 수 있다.

ORA-1628,1630,1631,1630 조치 방법

Bulletin no : 10084

다음 DBA 에러들은 오라클의 오브젝트들이 MAXEXTENT에 도달 했을때 발생하는 것들이다.

```
01B2B, 00000, "max # extents (%s) reached for rollback segment %s"
01B30, 00000, "max # extents (%s) reached in temp segment in tablespac
          %s"
01B31, 00000, "max # extents (%s) reached in table %s.%s"
01B32, 00000, "max # extents (%s) reached in index %s.%s"
```

또한 DBA-1628 다음에는 DBA-01582 도 함께 발생한다.

이 에러들은 다음 모든 LEVEL 에서 발생 될수 있다.

- . 애플리케이션 LEVEL (OL, AOL, Financial, Etc)
- . TOOLS LEVEL (Reports, Forms, Etc)
- . 커널 LEVEL (Insert, Update, Delete)

이 에러의 이유는 오브젝트의 익스텐트가 MAX # 에 도달 했기 때문에 발생되며 오브젝트의 MAXEXTENT는 BYSPACE 의 MAXEXTENTB 파라미터에 의해 결정된다.

다음 예를 보기도 하자.

```
BQL> INSERT INTO TAB1 SELECT * FROM TAB1;
```

```
DBA-01B31: max # extents (2) reached in table JAME.TAB1
```

```
BQL> SELECT INITIAL_EXTENT, NEXT_EXTENT, MAX_EXTENTS
        FROM USER_TABLES
        WHERE TABLE_NAME = 'TAB1';
```

INITIAL_EXTENT	NEXT_EXTENT	MAX_EXTENT
8144	10240	2

위 예에서 오브젝트의 MAXEXTENTB 는 2 인데 이 값은 HAMBDOBB인 MAX 값이 아니다. HAMBDOBB 인 MAXEXTENTB의 최대 값은 데이터베이스가 생성될 당시 지정된 DB_BLOCK_SIZE 의 값에 따라 다르다.

DB_BLOCK_SIZE	최대 MAXEXTENTB 값
512	25
1K	57
2K	121
4K	249
8K	505

만일 최대 MAXEXTENTB 값 보다 더 큰 MAXEXTENTB를 Storage 에서 지정하면 다음 에러가 발생한다.

```
DBA-0222B, 00000, "Invalid MAXEXTENTB value (max allowed: %s)"
```

다음은 DBA-0183x 에대한 해결 방법이다. 만일 문제들의 최대 MAXEXTENTS 에 도달이 만치었으면

```
ALTER TABLE .. STORAGE (MAXEXTENTS =)
```

를 사용하여 최대 MAXEXTENTS 값보다 작은 수로 MAXEXTENTS를 늘려준다.

만일 최대 MAXEXTENTS 값에 도달 했으면 해결 할수있는 방법은 MAX 제한에 도달되지 않도록 EXTENT SIZE 를 더 크게하여 오브젝트를 다시 생성하는 것이다.

만일 이 예머가 ROLLBACK SCHEMT 에서 발생되면 DROP하고 다시 생성 한다.

만일 예머가 TEMPORARY TABLESPACE에서 발생되면 TEMPORARY TABLESPACE의 STORAGE를 변경한다. TEMP SCHEMT는 그것이 생성된 TABLESPACE의 Default Storage Parameter를 사용하기 때문에 다음과 같은 방법으로 해결 할수 있다.

```
ALTER TABLESPACE 'temp' DEFAULT STORAGE (INITIAL = NEXT =);
```

. = 은 기존 지정된 값보다 큰값을 지정 한다.

```
ALTER TABLESPACE 'temp' DEFAULT STORAGE (PCTINCREASE =);
```

. = 은 기존 지정된 값보다 큰값을 지정한다.

만일 예머가 TABLE에서 발생되면 import/export utility를 사용하여 그 TABLE을 다시 생성한다.

[단계]

1. COMPRESS-OPTION을 사용하여 백스프롬을 실행함.

예) \$ exp scott/tiger file=exp.dmp table=scmp compress=y

2. 기존 TABLE을 RENAME 함.

예) SQL> RENAME EMP TO EMP_OLD;

3. 사용자 의 SCHEMA 에 임포트를 실행하여 TABLE을 생성함.

예) \$ imp scott/tiger file=exp.dmp commit=y

4. ALTER TABLE 명령을 사용하여 임포트된 TABLE의 STORAGE를 변경함.

예) SQL> ALTER TABLE emp STORAGE(NEXT IN);

5. 기존 테이블을 DROP 함.

예) SQL> DROP TABLE EMP_OLD;

위 작업의 결과로 한개의 큰 EXTENT를 가진 TABLE이 생성 되고 모든 CONSTRAINT, INDEX, COMBTAINT 들이 그대로 보존 된다.

ORA-1652 조치 방법

Bulletin no : 10100

데이터나 인덱스 등을 만들 때 자신의 TEMP TABLESPACE 가 아닌 곳에서 DBA-1652(temp tablespace가 부족함) 에러가 발생하는 경우가 있다. 이와같은 문제는 U7.1 에서만 발생하는 데 이것은 사용자를 상당히 혼란스럽게 만든다.

U7.1 에서는 데이터, 인덱스 등을 컴파일 생성할 수 있다. 이를 위하여 실제로 데이터 등이 생성될 공간에 Temporary Segment 을 만들게 되는데 이 과정에서 Temporary Segment 을 만들 공간이 부족하게 되면 실제의 데이터가 생성되는 데이터 스페이스에 대하여 DBA-1652 에러가 발생하게 되는 것이다.

DBA-1652 에러를 해결하는 방법은 에러메시지에서 보여주는 대로 해당 데이터스페이스에 Temporary Segment 가 생성될 만한 연속된 공간을 마련하여 주는 것이다. 즉,

1. 데이터 파일을 추가하여 데이터스페이스의 크기를 확장한다.

```
SQL> ALTER TABLESPACE data ADD DATAFILE  
      '/u01/.. /oracle/data2.dbf' SIZE 100M;
```

2. 데이터의 storage parameter 을 조정하여 현재 남아있는 메모리에 들어갈 수 있도록 한다.

```
SQL> ALTER TABLE emp STORAGE(NEXT 1M);
```

3. 데이터스페이스가 fragmentation 이 심한 상태이면 exp/imp 을 이용하여 데이터스페이스를 재구성 한다.

[예제] 다음의 데이터 생성 문장을 보자

```
SQL> CREATE TABLE FEATURE  
      (feature_code varchar2(4) primary key,  
       feature_desc varchar2(3) );  
DBA-01652, 00000, "unable to extend temp segment by 8144 in tablespace  
      "UEBBEL"
```

데이터 스페이스 UEBBEL 에 남아있는 가장 큰 연속된 공간을 확인해 보면

```
SQL> SELECT MAX(block#), MAX(bytes)  
      FROM DBA_FREE_SPACE  
      WHERE TABLESPACE_NAME = 'UEBBEL';  
  
      block#          bytes  
      -----  
      B143           12,580,884
```

위의 결과를 보면 현재 UEBBEL 에 남아있는 가장 큰 연속된 공간은 B143 블록인데 모따들은 B144 블록이 사용하였다 이를 할당받지 못하여 에러가 발생하게 된 것이다.

ORA-4031 조치 방법

Bulletin no : 10085

우리는 다음과 같은 작업 수행시 Oracle 이 Shared pool 에서 연속적인 메모리 부분을 찾지못해 ORA-4031 Error 을 발생시키는 것을 볼수 있다.

- PL/SQL Routine , Procedure 수행시 , Compile 시
- Form Generate 또는 Running 시 , Object 생성하기 위해 Installer 사용시

ORA-1547과 유사한 Error들은 서로 다른 Application, 서로 다른 환경에서 발생하지만 이 문제들은 여전히 같이 취급되고 같은 방법으로 해결 되고 있다. (eg. ORA-4031)

[Problem 설명]

Error 발생의 주된 원인은 Shared Pool 사용가능한 Memory 가 시간이 흐름에 따라 작은 조각으로 분할 되어 있다는 것이다. 그래서 큰 부분의 Memory 를 할당하려 한다면 Shared Memory 가 부족하다는 ORA-4031 Error 가 발생한다. 즉 전체적으로는 많은양의 사용가능 한 Space 가 있다하더라도 충분한 양의 연속적인 Space 가 없으면 이 Error가 발생한다.

[Problem 해결 방안]

이 Error 해결방법을 살펴 보면 다음과 같다.

1. Object를 Shared Pool에 맞추어 Fragmentation을 줄인다
(Shrink_Shared_Pool Procedure 이용)
2. Shared Pool 을 효율적으로 사용하도록 Application Program을 조절한다
3. Shared Pool 의 Size 를 적절히 조절한다. 이 Size는 Default 값이 3.5M이며 이 Size를 수정시 는 DB를 Shutdown 후 다시 Start 시켜야 하므로 항상 가능한 해결 방법이 될수 없다.

```
< ORACLE_HOME/rdwr/initBIB.ora >
```

```
shared_pool_size = 8000000      <- 단위 : Byte  
-----
```

위와 같이 편집하고 DB 를 RESTARTUP 한다.

SHRINK_SHARED_POOL BYDBWR PROCEDURE

이 utlshr package는 dbxpool.sql을 포함하며 7.0.13 이상 version에서 사용 가능하다. 이는 다음과 같이 3가지 부분으로 나누어 진다.

```
Procedure shrink(shrink number);
```

-> Shared_Pool 안에서 점해진 Size 보다 큰 Object 를 보여준다

```
Procedure keep(name varchar2, flag char Default 'P')
```

-> Object (Only Package)를 Shared Pool 에 유지한다. 또한 일단 Keep한 Object는 LRU Algorithm 에 영향을 받지 않으며 Alter System Flush Shared_Pool Command 에 의해 Package 의 Compiled Version 이 Shared Pool 에서 Clear 되지 않는다.

```
Procedure unkeep(name varchar2);      -> keep() 의 반대기능이다
```

이 Procedure들과 사용법에 대해 보다 더 자세한 정보를 위해서는

\$ORACLE_HOME/rdwr/adpls/dbxpool.sql script 를 참조.

ORACLE DATABASE HOME DIRECTORY 변경

Bulletin no : 10080

ORACLE HOME DIRECTORY를 옮기는 과정은 다음의 세 단계로 나누어 진다.

1. 오라클 소프트웨어의 이동
2. 데이터 파일, 로그 파일, 콘트롤 파일 등의 데이터베이스 부분의 이동
3. 각종 환경 설정의 편집

만약 오라클을 다른 MACHINE으로 옮기는 경우라면 두 DB 가 Binary Compatible 해야 한다. 일반적으로 DB 가 같더라도 버전이 다르면 Binary Compatible 하지 않은 경우가 있으니 유의해야 한다.

모든 작업은 오라클을 셧다운 한 상태에서 진행한다.

1. 오라클 소프트웨어의 이동

DB 상의 명령어(`tar`, `cp`, `mv` 등)를 사용하여 오라클 소프트웨어를 새로운 디렉토리로 옮긴다. ORACLE HOME DIRECTORY 이하 부분을 모두 옮기면 된다. 작업이 끝난 후에 파일의 크기나 `checksum`(sum 명령)을 이용하여 제대로 옮겨졌는지를 확인하도록 한다.

2. 데이터베이스의 이동

데이터베이스를 구성하는 파일들을 다음 명령을 이용하여 확인한 다음 말의 1.과정에서 복사되지 않은 파일이 있으면 새로운 디렉토리로 복사한다.

```
SQL>select * from dba_data_files;
SQL>select * from v$logfiles;
SQL>show parameter control_files
```

그 다음에 다음과 같은 과정을 거쳐 데이터 파일과 로그 파일의 이동 상태를 콘트롤 파일에 반영한다. 이 명령은 위에서 출력된 데이터 파일과 로그 파일 모두에 대해서 수행되어야 하며 한편의 명령에 여러개의 파일을 지정할 수 있다. 여기서는 2개의 파일에 대한 예를 들었다.

```
SQL> startup mount
SQL> alter database rename file
        '/fil1', '/fil2' to
        '/fil1', '/fil2';
```

만약의 경우를 위하여 콘트롤 파일을 생성해 주는 스크립트를 다음 명령으로 만들 수도 있다.

```
SQL>alter database backup controlfile to trace
```

이 결과는 디렉토리에서 가장 나중에 생긴 트레이스 파일에 약간의 편집을 통하여 사용할 수 있는 형태로 저장된다.

3. 환경 설정의 변경

`/etc/oratab` 또는 `/var/opt/oracle/oratab` 파일을 새로운 값으로 수정한다.
그리고 `$ORACLE_HOME` 를 설정할 때에는 소스스킵 파일을 포함해서도 만든다.
`init.ora`의 내용과 여기에 `init`로 기술된 `config.ora` 파일의
내용을 수정한다. 기본적인 것은 다음과 같은데 이외의 내용도 있을 수도 있다.

```
init.ora
control_files
background_dump_dest
log_archive_dest
db_name(전정권 경우)
```

만약 다른 MACHINE으로 옮기는 경우라면 `SHARED MEMORY` 와 `REMAPTABLE`의
설정도 적절히 세팅되어야 하며 `Dump Dest`도 고려하여야 한다.

다음과 같은 환경 수를 새로운 값으로 변경한다.

```
$ORACLE_HOME
$ORACLE_SID(전정권 경우)
$ORACLE_PATH
$ORACLE_TPATH
$TIME_ADJUST
```

그리고 `$ORACLE_HOME/rdbms/lib/npcllib1ct`에 접대패스로 지정된 것이 있으면
이 값도 새로운 값으로 변경한다.

`makefile`의 내용중 접대패스로 지정된 것이 있으면 이 값도 변경한다.

새로운 시스템으로 옮긴 경우에는 `NO-BUILD flag`이 설정되어 있지 않아야
한다. 대부분의 시스템은 `/etc/oratab` 파일에 이 `flag`를 갖고 있다.

`ORACLE INSTALLER`에서 사용되는 `User Default File`인 `usedfl.log` 파일은
데이터베이스 미마스트로 삭제한다. 이 파일은 일반적으로 `$ORACLE_HOME/orainst`
디렉토리에 있다.

시스템을 변경하는 경우라면 `/etc/passwd` 파일에 `ORACLE Installer` 항목을
추가하고 `ORACLE OWNER ID`를 만들고 `ORA` 그룹에 등록한다.

기본에 만든 스크립트나 `ora_ora` 가운 디렉토리를 참조하는 부분의 변경이
필요하면 수정한다.

ORACLE DATABASE의 AUTOMATIC STARTUP/SHUTDOWN(한글판)

Bulletin no : 10188

시스템이 `startup` 하면서 오라클도 함께 `startup` 하거나 시스템을 `shutdown` 할 때 오라클도 같이 `shutdown` 하는 것이 가능하다. 그 방식은 DB마다 차이가 있는데 여기서는 UNIX 에 기반한 여러 플랫폼에 대해서 알아보기로 한다.

본 내용에서 가장 많이 사용되는 파일은 다음과 같다.

```
$ORACLE_HOME/bin/dbsstart  
$ORACLE_HOME/bin/dbsout  
/etc/oratab 또는 /var/opt/oracle/oratab
```

`dbsstart` 과 `dbsout` 은 오라클을 `startup/shutdown` 시키는 스크립트인데 이것은 `automatic startup/shutdown` 시에만 사용하도록 하고 정상시의 경우에는 사용하지 않도록 한다.

`oratab` 파일은 일련 텍스트 파일로서 시스템에 설치된 오라클 인스턴스에 대한 정보를 가지고 있는데 보통 3개의 필드로 이루어져 있으며 첫번째 필드는 `ORACLE_SID`, 두번째 필드는 `ORACLE_HOME`, 세번째 필드는 `Y` 또는 `N` 으로 구성되어 있다. 해당 인스턴스를 `autostartup` 시키려면 꼭 `Y`로 설정되어 있어야 한다.

SYSTEM U

`System U` 에 기반한 시스템은 `/etc/passwd` 디렉토리에 초기화 스크립트를 가지고 있다. 여기서 는 해당 스크립트의 `run-level` 을 의미한다. 일반적으로 `run-level 0` 은 `power shutdown mode` 이고 `run-level 2` 는 `multi-user mode` 이다. 각각의 디렉토리에 들어있는 초기화 스크립트는 다음과 같은 형태의 이름을 갖고 있다.

[**E1B**]two-digit number]descriptive filename]

예를 들면 `B75crac`, `E30tcr` 등이 있을 수 있다.

`B`로 시작되는 스크립트는 `startup`시에 실행되는 것이고 `E`로 시작하는 것은 `shutdown` 시에 실행된다.

two-digit number가 의미하는 것은 스크립트가 실행되는 순서인데 숫자가 작은 것부터 먼저 실행된다. 따라서 오라클 스크립트는 보통 다음과 같은 이름을 갖게 된다

`B00orac`, `E01orac`

이와같이 설정하면 시스템이 `startup` 될 때 모든 초기화가 끝난 후 마지막으로 오라클이 `startup` 되며 시스템을 `shutdown` 할 때에는 오라클을 가장 먼저 `shutdown` 하게 된다.

BBU UNIX

BBU 에 기반한 시스템은 시스템이 startup 될 때 초기화 파일로 /etc/rc(예를 들면, /etc/rc, /etc/rc.local) 을 사용한다.

시스템을 shutdown 할 때에는 보통은 /etc/shutdown 만 실행하게 되므로 특별히 실행되는 스크립트가 없지만 /etc/rc.shutdown 이 실행되는 플랫폼도 있다.

이제 각 플랫폼별로 자세히 알아보기로 한다.

주의 : 는 oracle owner id 로 대해 <ORACLE_HOME> 는 oracle home directory 경으로 대해

(1)SunOS 4.1.3

SunOS 4.1.3 에서는 /etc/rc.local 을 startup 스크립트로 사용한다. 즉 시스템이 startup 되면서 /etc/rc.local 이 실행된다. 따라서 이 파일의 끝에 다음과 같은 라인을 추가하면 된다.

```
su -c <ORACLE_HOME>/bin/dkstart
```

예를 들어 oracle owner oracle? 이고 ORACLE_HOME 이 /home/oracle 이라면 다음과 같이 해주면 된다.

```
su oracle? -c /home/oracle/bin/dkstart
```

여기서 /home/oracle 대신에 ORACLE_HOME 과 같은 환경변수를 이용하여 세팅하면 만된다.

SunOS 에서는 시스템이 shutdown 될 때 실행되는 스크립트는 따로 없다.

따라서 dkshut 을 실행시키는 방법은 다음과 같은 스크립트를 만들어서 시스템을 shutdown 시킬 때 /etc/shutdown 을 실행시키지 말고 대신 이것을 실행시켜야 한다.

```
#!/bin/csh
```

```
su -c <ORACLE_HOME>/bin/dkshut
```

```
/etc/shutdown &*
```

만약 오라클을 shutdown 하지 않고 시스템을 shutdown 하면 다음에 오라클을 startup 하려고 할 때 startup이 되지 않는다. 이 경우에는 ORACLE_HOME/bin/sgadm.dkt 파일을 지우고 startup 시키면 된다.

(2)OSF/1(DIGITAL UNIX)

```
/etc/init.d          : 초기화 과정을 컨트롤  
/sbin/rc3            : run-level 3 에서 사용되는 script  
/sbin/rc0            : run-level 0 에서 사용되는 script  
/sbin/rc3.d/BBOracle : /sbin/init.d/oracle 로 link  
/sbin/rc0.d/EOracle  : /sbin/init.d/oracle 로 link
```

/etc/init.d 파일에는 초기화 과정에 대한 정보가 들어 있다. 예를 들면 다음과 같다.

```
rc3:3:init:/sbin/rc3 < /dev/console > /dev/console 2>&1
```

이렇게 설정하면 run-level 3 일 경우 /sbin/rc3 스크립트가 실행된다. /sbin/rc3 은 /sbin/rc3.d 디렉토리에 들어있는 스크립트를 실행한다. 만약 스크립트 이름이 E 로 시작하면 이것은 stop 파라미터를 갖고 실행되어서 이 스크립트에 지정된 프로세스를 정지시킨다. 한편, 스크립트가 S 로 시작하면 /sbin/rc3 은 이 스크립트에 startup 파라미터를 부여하고 여기서 지정된 프로세스를 실행시킨다. 따라서 /sbin/rc3.d/S00ssapc를 사용하여 dhstart를 실행한다.

shutdown 과정도 거의 비슷하다. /etc/init.d 에 다음과 같은 라인을 추가 한다.

```
rc0:0:init:/sbin/rc0 off < /dev/console > /dev/console 2>&1
```

이것은 /sbin/rc0 을 실행 키는데 /sbin/rc0 은 /sbin/rc0.d 디렉토리에 들어있는 스크립트를 실행시킨다. 스크립트 이름이 S 로 시작하면 run time parameter 로 start 를 사용하게 되고 E 로 시작하면 stop 을 사용한다. 따라서 /sbin/rc0.d/E01ssapc 스크립트는 stop 파라미터를 갖고 실행되며 이 스크립트는 \${BASIC_HOME}/bin/dhshut 스크립트를 실행시켜서 오라클을 shutdown 시킨다.

/sbin/rc3.d/S00ssapc 과 /sbin/rc0.d/E01ssapc 는 모두 /sbin/init.d/ssapc 에 대한 symbolic link 이다. /sbin/init.d/ssapc 에 대한 symbolic link 는 이를 위해 더 있을 수 있다.

(3)Balaric 2,3 & BSD UNIX

```
/etc/init.d      : 초기화 과정을 콘트롤
/etc/rc2        : run-level 2 에서 사용되는 스크립트
/etc/rc0        : run-level 0 에서 사용되는 스크립트
/etc/rc2.d/S00dhstart : /etc/init.d/dhstart 에 대한 link
/etc/rc0.d/E01dhshut  : /etc/init.d/dhshut 에 대한 link
/var/opt/oracle/oratab
```

Balaric 의 경우는 DBF/1 과 매우 유사하다. 차이점이라고 한다면 /sbin 디렉토리 대신에 /etc 디렉토리에 필요한 스크립트가 들어있다는 것이다.

rc0.d, rc2.d 같은 디렉토리는 실제로 /etc 디렉토리에 위치하고 있으며 rc0, rc2 과 같은 스크립트는 /sbin 에 존재하는 같은 이름의 스크립트에 대한 symbolic link 이다.

/etc/init.d/dhstart 과 /etc/init.d/dhshut 스크립트의 내용은 각각 다음과 같이 한줄로 구성되어 있다.

```
cu -c ${BASIC_HOME}/bin/dhstart
```

```
cu -c ${BASIC_HOME}/bin/dhshut
```

Balaric 에서는 start, stop 같은 파라미터는 /etc/init.d/dhstart, /etc/init.d/dhshut 에서는 무시된다.

/etc/init.d/dhstart, /etc/init.d/dhshut 파일은 owner 가 oracle owner id

이어야 한다. /etc/init.d/NETWORK 을 읽어보면 좀 더 자세한 내용이 설명되어 있으니 참조하기 바란다.

(4) AIX

/etc/init.d : 시스템 초기화 파일
/etc/rc : 시스템 초기화 스크립트
/etc/rc.d : /etc/init.d 을 수정하는 용도
/etc/shutdown : 시스템 shutdown 스크립트

AIX 에서는 모달을 autostartup 란것은 여러가지가 가능하지만 shutdown 은 한가지만 가능하다. 우선 autostartup 은 다음과 같은 란것 가운데 하나를 사용하면 된다.

a) /etc/rc.d 를 사용하여 /etc/init.d 에 entry 를 추가한다. 즉,

```
/etc/rc.d "i2init:/bin/cu -c  
                <@BACLE_HOMED>/bin/dkstart > /dev/console 2>&1"
```

이렇게 하면 /etc/init.d 파일에 dkstart 기능이 추가된다. 이렇게 하면 시스템이 뜨면서 run-level 2 밑 경주 모달들이 자동으로 startup 된다.

b) 다음과 같은 란것도 가능하다.

```
/etc/rc.d "rcconsole:i2init:/etc/rc.console > /dev/console 2>&1"
```

그리고 root 소용의 /etc/rc.console 스크립트를 다음과 같이 작성한다.

```
cu </bin/dkstart  
EOF
```

c) /etc/rc 스크립트의 마지막에 다음을 추가한다.

```
/bin/cu - c <@BACLE_HOMED>/bin/dkstart
```

AIX 에서 시스템 shutdown 은 /etc/shutdown 에 의한다. 시스템을 내릴 때 모달들을 shutdown 하려면 별도의 시스템 shutdown 스크립트(root 소용도)를 다음과 같이 만들어서 /etc/shutdown 을 사용하지 않고 이것을 사용해야 한다.

```
#!/bin/sh  
/bin/cu -c <@BACLE_HOMED>/bin/dkstart  
/etc/shutdown &*
```

(5) HP/AIX

/etc/rc : 시스템 startup 파일
/etc/shutdown : 시스템 shutdown 실행파일

/etc/shutdown.d : 시스템 shutdown 디렉토리

HP/UX에서는 시스템 startup 시에 /etc/rc 가 실행된다. 따라서 다음 타민을 /etc/rc 스크립트의 마지막에 추가하면 autostartup 이 가능해진다.

```
su -c <${HOME}/bin/dhstart
```

시스템의 shutdown 은 /etc/shutdown 파일을 실행하여 수행된다. 이 프로그램은 /etc/shutdown.d 디렉토리에 들어있는 스크립트를 실행시키는데 다음과 같은 세가지 방법 가운데 하나로 autoshutdown 을 설정할 수 있다.

- a) \${HOME}/bin/dhshut 파일을 /etc/shutdown.d 디렉토리 아래에 copy
- b) /etc/shutdown.d/dhshut 을 \${HOME}/bin/dhshut 으로 symbolic link 지정
- c) 다음과 같은 스크립트 작성

```
#!/bin/sh
```

```
su -c <${HOME}/bin/dhshut
```

단, 이 스크립트는 root 의 소유이고 root 에 의해서만 실행가능해야 한다.

만약 UB 와 UF 를 동시에 사용하는 경우라면 UF 에서 제공하는 dhshut를 사용하도록 한다.

Oracle Enterprise Manager Install Procedure

Bulletin no : 10403

System requirement

HM (32M RAM, 30 M DISK)

BU (MS Windows NT ver 3.51 or Windows 95)

1. 먼저, Oracle server 7.3 sqlplus 에서DBN 용어를 생성한다. (grant resource, connect, dba to user) 이때, 반드시 default tablespace 를 지정하고, 가급적 다른 tablespace 와 분리하는 것이 좋다. (이 용어가 각자의 repository 용어이며 DBN 작업에 대한 정보가 저장된다)

2. Oracle installer(Default 또는 Setup) 을 수행 시킨 후에 Oracle Enterprise Manager와 Performance pack 을 설치한다.

3. Oracle Enterprise Manager 폴더에서 Network Topology Generator 를 선택 하거나 /Oracle85/bin/voot.exe 를 수행시킨다.

(이때, /Oracle85/network/admin/topology.ora 가 없기 때문에 ALERT 메시지가 뜨는데, DE를 클릭한다.) topology.ora 파일을 만들 때는 listener, database, agent 순으로 만든다.

4. Oracle for windows 95 폴더에서 BDLxp1uc 32 를 수행하여 Oracle Server 7.3 으로 tnsnames를 이용하여 접속하여 Network 접속 여부를 확인한다. (잘 만된다면, tnsnames.ora 를 다시 설정) Oracle Enterprise Manager 폴더에서 BDL worksheet를 통하여 /Oracle85/ysman/admin/empcore.sql 과 perfcore.sql을 차례로 수행 시킨다.

ORACLE MEDIA SERVER

Bulletin no : 10189

ORACLE MEDIA SERVER OVERVIEW

Oracle Media Server는 정보가 어떻게 저장되고, 검색되며, 목적지까지 도착하는 지를 처리하여주는 멀티미디어용 "자료 저장고(Data Warehouse)" 소프트웨어입니다. 초점컴퓨터(Maximally Parallel Computer)를 지원하는 강력한 데이터베이스 및 메시징 기술을 근간으로, 멀티미디어 서버는 초당 수천개의 프랜잭션을 처리할 뿐 아니라 대화형 서비스(Interactive Service)에 필요한 여러가지 데이터-오디오, 비디오 및 텍스트-를 만 하여 줍니다.

Oracle Media Net은 여러가지 복잡한 전송기술(전환선, 위성시스템 및 케이블 연결 등)을 감주어 투명하게 해주고, 정보를 가점으로 전달하기 위해 "TV"를 통신합니다.

Oracle Media Objects는 요구즉응 비디오(Ultimate-on-demand), 요구 즉 뉴스(Now-on-demand), 그리고 홈 쇼핑(Home Shopping)과 같은 차세대의 애플리케이션을 창출해 낼 수 있는 최초의 애플리케이션 제작 도구(Authoring Tool)입니다.

최근에 정보 고속도로와 관련하여 발표된 내용은 소비자가 어떻게 500여개의 TV 채널을 받고 조종할 수 있을 것인가에 그 초점이 맞추어지고 있지만 Oracle의 새로운 소프트웨어는 사용자들이 각자의 구미에 맞도록 원하는 채널만을 파악할 수 있도록 하여줍니다.

예를 들면, 소비자들은 미디어 서버의 서비스를 이용하여 저녁 7시에 최근하고도 즐겨보는 방송의 8시 뉴스를 시청할 수 있고, 또는 템플릿(Template)을 만들어 자신이 원하는 모드의 뉴스만큼-스포츠뉴스 같은 -스캔(scan)하여 볼 수도 있는 것입니다.

쇼핑도 현재의 TV쇼핑 프로그램이 상품선전 화면을 끝까지 지켜봐야 보아야하는 것과는 달리 소비자가 원하는 상품을 원하는 때 구매할 수 있습니다. 예를 들어 소비자들은 종종으로 부터 십크롤라투스(Shrinkwrap)를 구입할 수 있고, 여러 상품으로부터 데니스 신발 가격을 비교해 볼 수도 있으며 크리스마스 선물도 집에서 구입할 수 있게 됩니다. 또한 엔터테인먼트-영화, TV, 음악 기타 등등-도 요구즉응 서비스로 가능해 집니다.

다수의 전신전화 회사와 컴퓨터 제조회사들이 Oracle과 협력관계를 모색하고 있으며 이미 대화형 서비스 분야의 선두를 달리는 망대 전화회사-Bell Atlantic과 US West가 그들의 대화형 서비스에 Oracle Media Server를 사용하겠다고 발표한 하 있습니다. 대화형TV 제공의 수석자인 Philips, Compression Labs, Kalida, 3DO, Echelon, General Magic, Apple Computer등도 Oracle의 미디어 서버를 지원하겠다고 발표하였으며, nCUBE와 Silicon Graphics 도 그들의 강력한 컴퓨터로 Oracle Media Server를 지원하겠다고 약속하였습니다.

OMS Release

지난 수년동안 Oracle은 DBMS기술을 Database 서비스의 세계에 의욕적으로 적용해 왔습니다. 이러한 노력의 일환으로 Oracle의 소프트웨어는 비디오, 오디오, 그리고 문자 데이터들을 수용할 수 있게 되었습니다. 또한 Oracle은 초점컴퓨터(Maximally Parallel Computer)라고하는 새로운 형태의 컴퓨터에서 작동하는 최초의 상품 소프트웨어를 공급하는 회사가 되었습니다.

초점컴퓨터는 값이 비싸지 않은 마이크로프로세서 수천개를 배열하고 각각의 프로세서에 독자적인 메모리를 부여하여 외부와의 통신로를 제공하여 줍니다. 따라서 전체 시스템은 기존의 컴퓨터에서 첨가된 문제로 여겨졌던 밀집된 데이터 경로를 지할 수 있고 수천명의 사용자들이 동시에 요청하는 요

구독형 즉, 요구즉응 영화(Now-on-demand)와 같은 서비스를 제공할 수 있게되는 것입니다. mCMB 2와 같은 전형적인 초점집시스템에서 작동하는 미디어 서버의 경우, 25,000개의 개별적인 비디오 채널-또는 스트림(stream)-을 채널당 500달러의 가격으로 제공하며 이때 사용자들은 기존의 UCB과 똑같이 pause, rewind, fast forward 같은 기능을 이용 할 수 있게 됩니다.

Dracm 미디어 서버는 4가지의 주요 구성품으로 나누어 집니다.

- * 시스템 관리 도구(System management tools)는 시스템관리자로 하여금 서버를 관리하도록 하여주는데 이에는 멀티미디어 자료의 코딩과 목록만들기 (cataloging), 사용자 프로파일(profile) 갱신, 시스템 파라미터(parameter)의 세팅 등이 포함됩니다.
 - * 서비스(Service)는 사용자의 요청에 의해 작동이 되어 요청된 정보를 라이브러리(library)에서 찾을 수 있도록 메시지를 전송하는 것과 관련된 일련의 기능을 총칭하는 것입니다.
 - * 액세스 라이브러리(Access library)는 서버의 데이터베이스내의 데이터를 액세스 하기 위한 데이터 조작루틴(manipulation routines)을 총칭합니다.
- 이로 인해 데이터의 위치지정, 갱신 및 검색등이 가능해 지며, 정지,빨리감기,되감기 등 기존의 UCB에서와 같은 조정/조작도 가능해 집니다.
- * 데이터베이스(Database)는 대화형의 서비스를 지원하기 위해 여러종류의 데이터를 저장합니다. 미디어 서버에는 8개의 데이터베이스가 있습니다.

- 실시간 스트림 데이터베이스(Real-time stream database)는 실시간으로 처리될 오디오 및 비디오 정보를 저장합니다. 사용자가 요구 즉응 오디오나 비디오 서비스를 요청하면 이 데이터베이스가 1초내에 요청된 자료를 찾아 소미자에게 보내게 됩니다.
- 관계형데이터베이스 서버(Relational database server)는 Dracm 7 데이터베이스이며 청구서를 위한 정보, 영화 디스크, 사용자 프로파일 등 포의 형식으로 분류된 정보를 관리합니다.
- 화일서버(File server)는 DB의 화일에 대한 액세스를 제공합니다.
- BLOB(Binary Large Object)데이터베이스는 이미지 및 실행화일을 저장합니다.
- 오브젝트 데이터베이스(Object database)는 오브젝트-오리엔티드 프로그래밍을 위한 오브젝트를 관리합니다.
- 텍스트 데이터베이스(Text database)는 텍스트와 도큐먼트 신속하게 추적, 검색하기 위한 것으로 인덱싱과 요약기능 등이 포함됩니다.

Dracm의 미디어 서버는 1994년 3월에 시중에 공급될 예정입니다. 오브젝트 서버와 텍스트 서버는 이때 제공될 것입니다. 다른 모든 Dracm 제품과 마찬가지로 미디어 서버도 역시 플랫폼에 제한 받지 않도록 설계되어 있습니다. mCMB에 포함하는 것을 시작으로 내년까지는 다른 초점집시스템과 다중서버 시스템에도 포함될 것입니다.

OMO Release

Dracm Media Object Software는 맞춤형식의 홈 서비스(Customized home services)를 제공하기 위해 가칭 "스마트(smart) TV" 시스템에서 DDP(Object-oriented programming)를 통하여 요구즉응 비디오나 홈 쇼핑 등을 가능하게 해주는 소프트웨어입니다. 미디어 오브젝트는 대화형 애플리케이션의 프로토타이핑, 테스트등을 신속하게 처리해 주는 몇 가지 유사한 언어인 Dracm Media Talk 와 그래픽 필드 플,그리고 재사용이 가능한 오브젝트들로 이루어 집니다. 미디어 오브젝트의 기본 구조는 여러개의 "스택(stack)"이며 이들 각각의 스택은 여러 개의 "카드(card)"로 이루어 집니다. 각각의 카드는 스크린 상에서 사용자가 직접 원격제어를 할 수 있도록 텍스트와 그래픽 등으로 이루어진 하나의 스크린 정보를 의미합니다. 사용자들은 카드 즉, 스크린을 마음대로 바꿀 수 있게 됩니다. 개발자들은 미디어 오브젝트의 애플리케이션용 오브젝트 세트를 이용하여 사용하기 쉬운 코드의 그래픽 사용자

인터페이스를 만들어 낼 수 있습니다. 오브젝트란 독특한 모습과 작동원리가 부여된 하나의 구조물로써 스크린 상에 나타나는 것을 말합니다. 오브젝트의 작동에 관한 용례는 미디어 포크(Media Fork)로 하게 됩니다. 미디어 오브젝트의 개발 환경은 윈도우 매킨토시에서 모두 가능하며, 오브젝트를 생성하고 편집하는 그래픽 인터페이스와 미디어 포크 스크립트(script)는 물론 데스크톱 디버깅을 위한 런타임(runtime)환경도 모두 지원됩니다.

OMN Release

Omni Media Net은 가정으로 대화형의 서비스를 제공하는데 사용되는 여러가지 전송기법을 사용자로 부터 감추어주는 즉, 투명하게 해주는 고도의 프로토콜입니다. 미디어 네트워크를 통하여 요구응답 서비스에 대한 요청이 미디어 서버와 연결되는 것입니다.

Omni 미디어 네트워크는 비대칭형(asymmetric)과 대칭형(symmetric)을 추가지의 네트워크 형태를 지원합니다. 비대칭형 네트워크는 서로 다른 랜드 속을 갖는 주개의 방향이 다른 채널로 이루어지며, 이중 하나는 아주 빠른 속도로 가정으로 향하고, 또 하나는 그 보다는 느린 속도로 가정으로부터 빠져나오게 됩니다. 가정의 전화선이 좋은 예입니다. 1994년 3월중에 계획되어 있는 Bell Atlantic과 Omni의 최초의 대화형서비스에는 기존의 "Tulstul Pair" 네트워크에 ATM 이라고 하는 프로토콜을 적용시켜 요구응답 비디오 서비스 즉, 비디오 온 디맨드 서비스를 공급하게 됩니다.

대칭형 네트워크는 소위 말하는 "브로드밴드(broadband)"서비스를 말하며, 가정으로 들어오고 나가는 양쪽 채널 모두 고속의 랜드 속을 제공합니다. 케이블과 파이버(fiber)로 구성이 되며 ATM(Asynchronous Transmission Mode) 방식이 주로 쓰이는 프로토콜입니다.

미디어 트는 또한 가정으로부터의 사용자 요청을 서버로 보내주는 소프트웨어를 제공합니다. 미디어 네트워크는 서비스 단위의 메시징을 기본으로 하기 때문에 특정한 비디오 또는 음소징 프랜차이즈들은 한 번의 문의(조회)로 처리 됩니다.

또한 비디오 컨퍼런스(conference)나 작두전, 홈 뱅킹(home banking)등과 같이 지정된 상대와 메시지를 주고 받는 대화형식의 메시징도 지원합니다. Omni 미디어네트는 미디어 서버와 어떠한 셋-탑박스(set-top box:TV위에 장착되는 부수장비)나 "스마트 TV"라도 연결시켜 줌으로 사용자는 마음에 드는 시스템을 선택하면 됩니다.

ORACLE SERVER AND UNIX TRANSACTION PROCESSING(TP) MONITOR-1

Bulletin no : 10021

This file contains commonly asked questions about Oracle Server and UNIX transaction Processing Monitors (TPMs). The topics covered in this article are

- o What is a Transaction Processing Monitor (TPM)?
- o What is the K/Open Distributed Transaction Processing Model?
- o How does the Oracle Server work with TPMs?
- o How should I position TPMs with my customer?
- o What Oracle products must a customer purchase?
- o Where can my customer purchase a TPM?
- o Availability and packaging

What is a Transaction Processing Monitor?

Under UNIX, a Transaction Processing Monitor (TPM) is a tool that coordinates the flow of transaction requests between front-end client processes that issue requests and back-end servers that process them. A TPM is used as the "glue" to coordinate transactions that require the services of several different types of back-end processes, such as application servers and resource managers, possibly distributed over a network.

In a typical TPM environment, front-end client processes perform screen handling and ask for services from back-end server processes via calls to the TPM. The TPM then routes the requests to the appropriate back-end server process or server processes, wherever they are located on the network. Through configuration information, the TPM knows what services are available and where they are located. Generally, the back-end server processes are specialized so that each one handles one type of requested service. The TPM provides location transparency as well and can send messages through the network utilizing lower-level transport services such as TCP/IP or DBF BCE.

The back-end servers process the requests as necessary and return the results back to the TP monitor. The TP monitor then routes these results back to the original front-end client process.

A TPM is instrumental in the implementation of truly distributed processing. Front-end clients and back-end processes have no knowledge of each other. They operate as separate entities, and it is this concept that provides flexibility in application development. Front-end and back-end processes are developed in the UNIX client-server style, with each side optimized for its particular task. Server functionality can be deployed in stages, which makes it easy to add functionality as needed later in the product cycle. It also makes it easy to distribute both the front-end and back-end processes throughout the network on the most appropriate hardware for the job. In addition, multiple back-end server processes of the same type might be activated to handle increasing numbers of users.

What is the X/Open Distributed Transaction Processing Model?

The K/Opus Transaction Processing working group has been working for several years to establish a standard architecture to implement distributed transaction processing on open systems. In late 1991, K/Opus published the Initial Distributed Transaction Processing (DTP) model specification and defined the first of several interfaces that exist between the components of the model. Subsequently, other publications and a revised model specification have been published.

An important function of the TPM in the K/Opus DTP model is the synchronization of any commits and rollbacks that are required to complete a distributed transaction request. The Transaction Manager (TM) portion of the TPM is the entity responsible for ordering when distributed commits and rollbacks will take place. Thus, if a distributed application program is written to take advantage of the TM portion of the TPM, then it, and not the DBMS, becomes responsible for enabling the two-phase commit process. Article 2 has more detail on this model.

How does the Oracle Server work with TPMs?

When a TPM is used without invoking an K/Opus TM component to manage the transactions, Oracle Server needs no special functionality. The transaction will be managed by Oracle itself. However, when the TPM K/Opus TM component is used to manage the transaction, the Oracle Server, that is the Oracle DBMS, acts as a Resource Manager—a type of back-end process. In the case of TPM-managed transactions, the TM needs a way to tell the RM about the stages of the transaction. This is done by a standard, K/Opus defined interface called KA. Article 2 of this document gives more information about both the K/Opus model and Oracle's use of KA.

Because the KA interface provides a standard interface between the TM and the resource manager, it follows that the TM can communicate with any KA-compliant resource manager (e.g., DBMS), and, conversely, that a resource manager can communicate with any KA-compliant TM. Thus, the Oracle Server, beginning with Oracle7, works with any KA-compliant TM.

How should I position TPMs with my customer?

There's been a great deal of confusion about the need for TPM technology. Some software suppliers, most notably IBM, will assert that a TPM like CICS is a necessary requirement for high volume OLTP. Other vendors will assert that there is seldom a need for such technology. And yet others promote TPMs as providers of higher transaction throughput.

From Oracle's standpoint, customers might choose TPM technology under any of the following conditions:

1. For heterogeneous database access, especially for 2PC capability

This means that a TPM can be used to coordinate 2PC between Oracle DBMS and any other KA-compliant database, such as Informix. This does NOT provide SQL heterogeneity — SQL calls to Oracle DBMS may be different than SQL calls to Informix. The TPM handles the routing, communication, and two-phase commit portion of the transaction, but does not translate one type of SQL call into another.

2. For transaction monitoring and workload control

The leading TPMs supply tools to actively manage the flow of transactions between clients and servers and to load balance the work load across all available processors on a network, not just on a single multi-processor system. Some TPMs also have the ability to dynamically bring up additional back-end services during peak work hours.

3. For more flexible application development and installation

One of the key features of the DTP model is application modularity. Modularity, that is, the decomposition of a large program into small, easily defined, coded and maintained "mini-programs" makes it easy to add new functionality as needed. Modularity also makes it much easier to distribute the front-end and back-end processes and the resource managers across hardware throughout a network.

4. For isolating the client from details of the data model

By using the service oriented programming model, the client program is unaware of the data model. The service can be recoded to use a different one with no change to the client. To get this advantage, the application developer must explicitly code the server and client to fit the service model.

5. For connection of thousands of users

TP Monitors, because of their three-tier architecture, can be used to connect users to an intermediate machine or machines, removing the overhead of handling terminal connections from the machine running the database. See Article 4 for more information.

There are also several cases where TPM technology is not the right answer.

These include:

1. If the customer is simply looking for a performance improvement The customer may have heard a theory that "higher performance is possible for large scale applications only if they use a TP monitor". First, no performance gain can be achieved for existing applications: In fact, they won't even run under a TP Monitor without recoding. Second, performance improvements have only been documented for large numbers of users, and "large" means many hundreds or thousands. Without a TP Monitor, Oracle Server can handle several hundred users with its normal two-task architecture and several times that using the Multi Threaded Server. For more on performance, see Article 4.

2. If the customer has made large investment in his existing Oracle applications

TP monitor applications must be designed from the ground up to take advantage of TP monitor technology. Current Oracle customers will find it difficult to "retrofit" a TP monitor to their existing applications. The Multi Threaded Server, on the other hand, allows the use of existing Oracle applications without change.

3. If the customer is committed to the Oracle tool set

Currently, some of Oracle's front-end tools (Oracle Forms, etc.) is designed to work with TP monitors. It is possible to invoke a TP Monitor by using user exits. However, the fact that the TP Monitor model hides the data model from the client means that only the screen display parts of Forms can be used, not the automatic mapping from screen blocks to tables.

4. If the customer does not have a staff of experienced software engineers

This is still very young technology for UNIX. There is not a lot of knowledge in the industry on how to build TP monitor applications or what techniques are most useful and which are not. Furthermore, integrating products from different vendors, even with the support of standard interfaces, is more complex than deploying an integrated all-Oracle solution. Because TP monitor technology is fairly complex, we recommend that you let the TP monitor supplier promote the virtues of their technology and differentiate themselves from their competitors.

What Oracle products must a customer purchase?

If your customer is only interested in building Oracle-managed TP Monitor transactions, the only Oracle products required are the Oracle Server and the appropriate Oracle precompiler for whatever language the application is being written in--most likely C or Cobol. If TPM-managed transactions are required, the Oracle7 Server with the distributed option is also required. BQL-Wait is optional because the TPM takes care of the network services. Article 2 describes when you would choose to have the TP Monitor manage the transactions.

Where can my customer purchase a TPM?

There are many vendors offering the UNIX TPM products. (Oracle does not sell these TPMs.) Information on the most well known products is provided below:

The following support KAs:

Product & Vendor	FCB	Known DB/Platform Ports
-----	--	-----
"TUXEDO System/T" UNIX System Laboratories 180 River Road Burlington, NJ 07801	1988	UNIX BDB4 & BDB3: Amdahl, AT&T, Bull, Compaq, Dell, Fujitsu, ICL, Motorola, Olivetti, Pyramid, Sequent, Sun, Toshiba, Unisys, NCR, Stratus Other: IBM AIX, HP/UX, DEC Ultrix
"TOP END" NCR Corporation 1334 B. Patterson Blvd. Dayton, OH 45478	1982	UNIX BDB4: NCR
"ENCINA" Transarc Corporation 707 Grant Street Pittsburgh, PA 15219	1982	IBM AIX, HP, Sun (SunDB and Solaris) Other: DB/2, DB3, HP-UX, SYBASE (Depends on BCE)
"CICS/3000" IBM Corporation	1983	AIX: IBM (Depends on BCE)

"CICS 8000"	1984	HP-UX
HP		

The following do not currently support KA:

Product & Vendor	FCB	Known OS/Platform Ports
-----	---	-----
"VIB/TP"	unknown	unknown
VISystems, Inc.		
11810 Greenville Avenue		
Dallas, TX 75243		
"Unixix"	1980	UNIX: AIX, AT&T, MCB, Pyramid,
Unixix		Sequent, Sun, Unisys
"MicroFocus	1983	BCD Unix, AIX
Transaction System"		
Micro Focus		
28 West Street		
Northbury BC13 1TY		
UK		

There are also several third parties who are reselling the products listed above.

In addition, Groups Bull, Digital, Blommer-Mindorf, and several other hardware vendors are planning to redesign their proprietary TPMs to be KA-compliant and suitable for use on UNIX systems.

Availability and Packaging

On what platform is the KA Library available?

Oracle provides the KA interface with Oracle[®] Server on all platforms that support an KA-compliant TPM. Support for KA is included as part of the Oracle[®] Server distributed option and has no extra charge in and of itself.

Which version of KA does Oracle Server support?

Oracle[®] Server supports the Common Application Environment (CAE) version of KA, based on the specification published by K/Dyn in late 1981. It will require that the TM also be at that level. This means Tuxedo /T version 4.2, for example.

Oracle Server supports all required KA functions. There are some optional features Oracle Server does not support, such as asynchronous operation. None of these options affect application programming.

This file contains commonly asked questions about Oracle Server and UNIX Transaction Processing Monitors (TPMs). The topics covered in this article are

- o Oracle Server Working with UNIX TPMs
- o TPM Application Architecture

The questions answered in part 2 provide additional detail to the information provided in part 1.

Oracle Server Working with UNIX TP Monitors

Do I need KA to use Oracle Server with TPMs? If I don't use it, what are the consequences?

There are a number of real applications running today with Oracle Server and TPMs but not using KA. To use a TPM with Oracle without using KA, the user would write an "application server" program which could handle one or more "services". For example, a server program might handle a service called "debit_credit". The key requirement is that the entire transaction, including the "commit work", must be executed within a single service. This is the restriction which KA will remove, as we'll see later. Each server process can serially handle requests on behalf of different clients.

Because a server process can handle many client processes, this can reduce the total number of active processes on the server system, thereby reducing resource requirements and possibly increasing overall throughput.

When Oracle is used with a TPM in this mode, we call it an Oracle-managed transaction since the transaction commit or rollback is done with a SQL statement.

What is KA? How does KA help Oracle? work with UNIX TPMs?

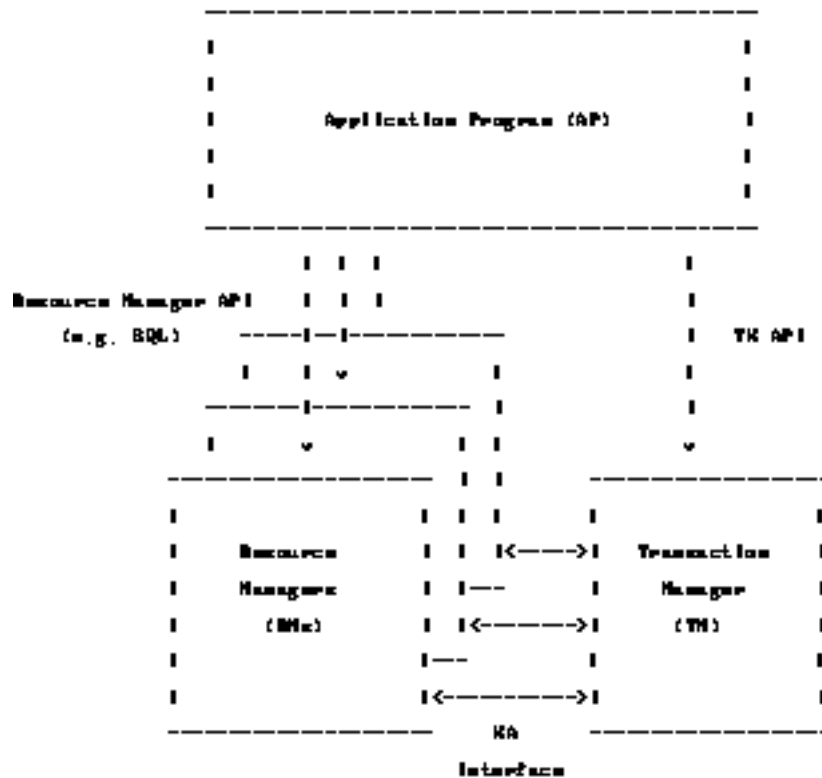
KA is an industry standard interface between a Transaction Manager and a Resource Manager. A Resource Manager (RM) is an agent which controls a shared, recoverable resource; such a resource can be returned to a consistent state after a failure. For example, Oracle? Server

is an RM and uses its redo log and undo segments to be able to do this. A Transaction Manager (TM) manages a transaction including the commitment protocol and, when necessary, the recovery after a failure. Normally, Oracle Server acts as its own TM and manages its own commitment and recovery. However, using a standards-based TM allows Oracle? to cooperate with other heterogeneous RMs in a single transaction.

The commonly used TPMs include a TM component for this purpose. In order to use the TM capability of the TPM rather than Oracle?'s own transaction management, the application uses a transaction demarcation API (called TX) provided by the TPM rather than the SQL transaction control statements (e.g. "commit work"). For each TX call, the TM then instructs all RMs, by the appropriate KA commands, to follow the two-phase commit protocol. We call this a TPM-managed transaction.

The following picture shows these interfaces within a monolithic application program model. This is the model most commonly described in the DTP literature. We'll see later what the picture looks like when we add Oracle? and when we switch to a modularized client-server

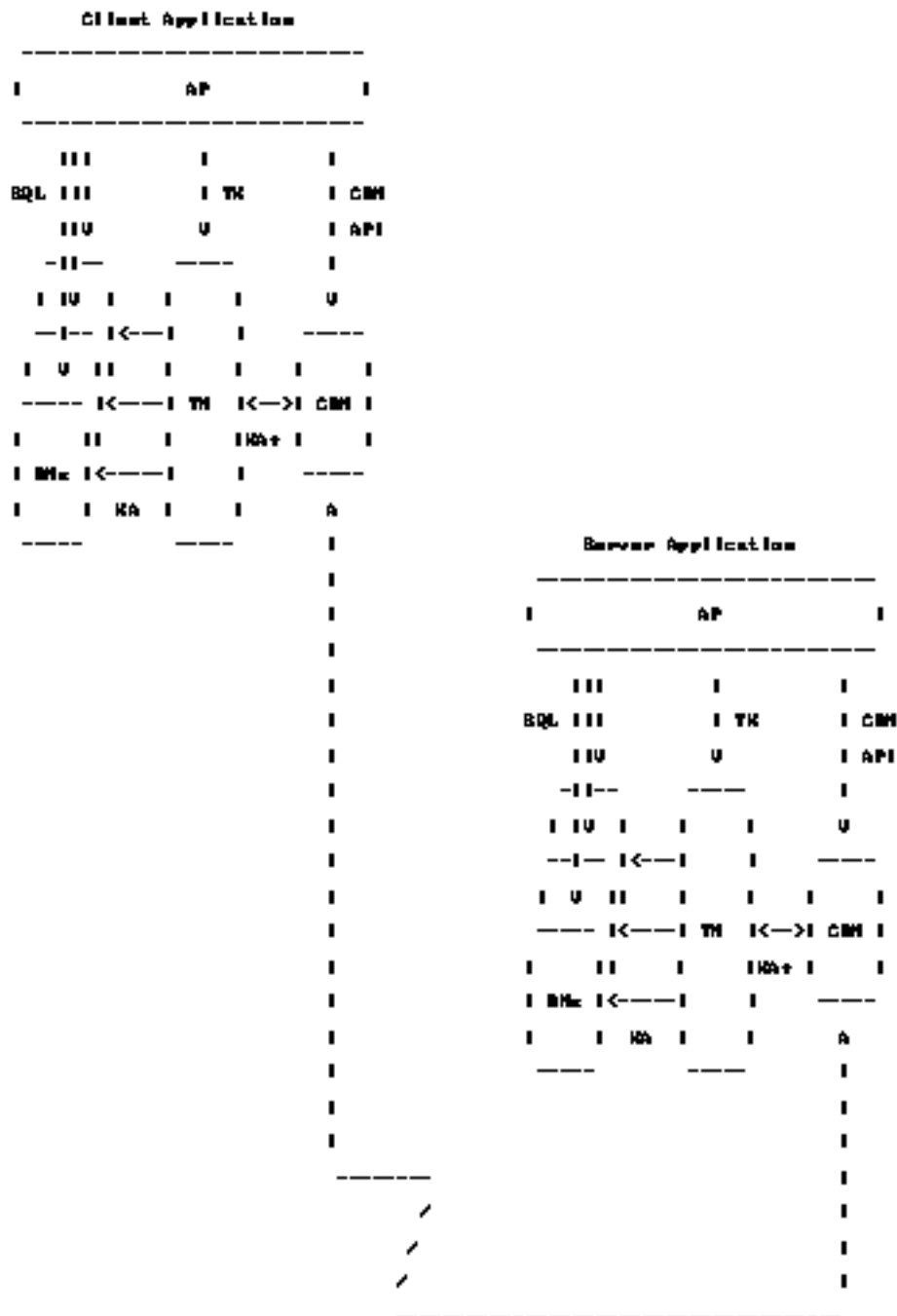
application program model.



The KA Interface is an interface between two system components, not an application program interface: the application program does not write KA calls nor need to know the details of this interface. The TM cannot do transaction coordination without the assistance of the RM: the KA Interface is used to get that assistance.

How does the RTP Model support client-server?

The above picture was actually simplified to make it easier to explain the role of KA. In a true distributed transaction architecture, there are multiple applications, each with an Application Program, a Resource Manager, and a Transaction Manager. The applications communicate by using a Communication Resource Manager. The CRM is generally provided as a component of the TPM. It includes the transaction information when it sends messages between applications, so that both applications can act of behalf of the same transaction. The following picture illustrates this:



Most TP Monitor products include both a TM and a CM, and also provide additional functions such as task scheduling and workload monitoring.

What is KAT? What does Oracle need to do to comply with it?

KAT is an interface that lets the K/Open model actually be distributed because it allows a communication resource manager to tell a TM on the server that a message from a client just came in for a particular transaction. Oracle is not currently planning to provide an K/Open communication resource manager, so we don't have any plans right now to do KAT. Version 2

of the RTP model paper from K/Open describes it. The status of the current K&T specification is "complete".

When would I choose an Oracle-managed transaction vs a TPM-managed transaction?

Oracle Server is very efficient at managing its own transactions. If the TPM manages the transaction, in general some additional overhead will be incurred.

The two main reasons a customer might prefer to use a TPM-managed transaction are as follows:

(1) He may need to update BNC from different vendors. Experience so far has been that the most common case is wanting to update both Oracle and a TP Monitor managed resource such as a transactional queueing service in the same transaction (see Article 3).

(2) He may want to use the model of having several different services in a transaction, even to the same database. For example, the "debit_credit" service could be split into a "debit" service and a "credit" service. This is a very attractive model, but this type of modularity does exact a performance penalty (see Article 4).

Can I get a version of K&T to run on Oracle Server version 8?

No, the K&T functionality uses two underlying mechanisms in the Oracle Server which are not available in version 8: two-phase commit and session switching. The upi calls for these functions do not exist in version 8.

When would I use K&T vs Oracle? to coordinate all-Oracle distributed transactions?

Generally speaking, Oracle Server should be used to coordinate all-Oracle distributed transactions. The main reason for using K&T to coordinate transactions would be that you want to use the TP Monitor service-oriented architecture. That is, you would like to construct an application built of services and service requests in order to benefit from the modularity and workload control such an environment provides.

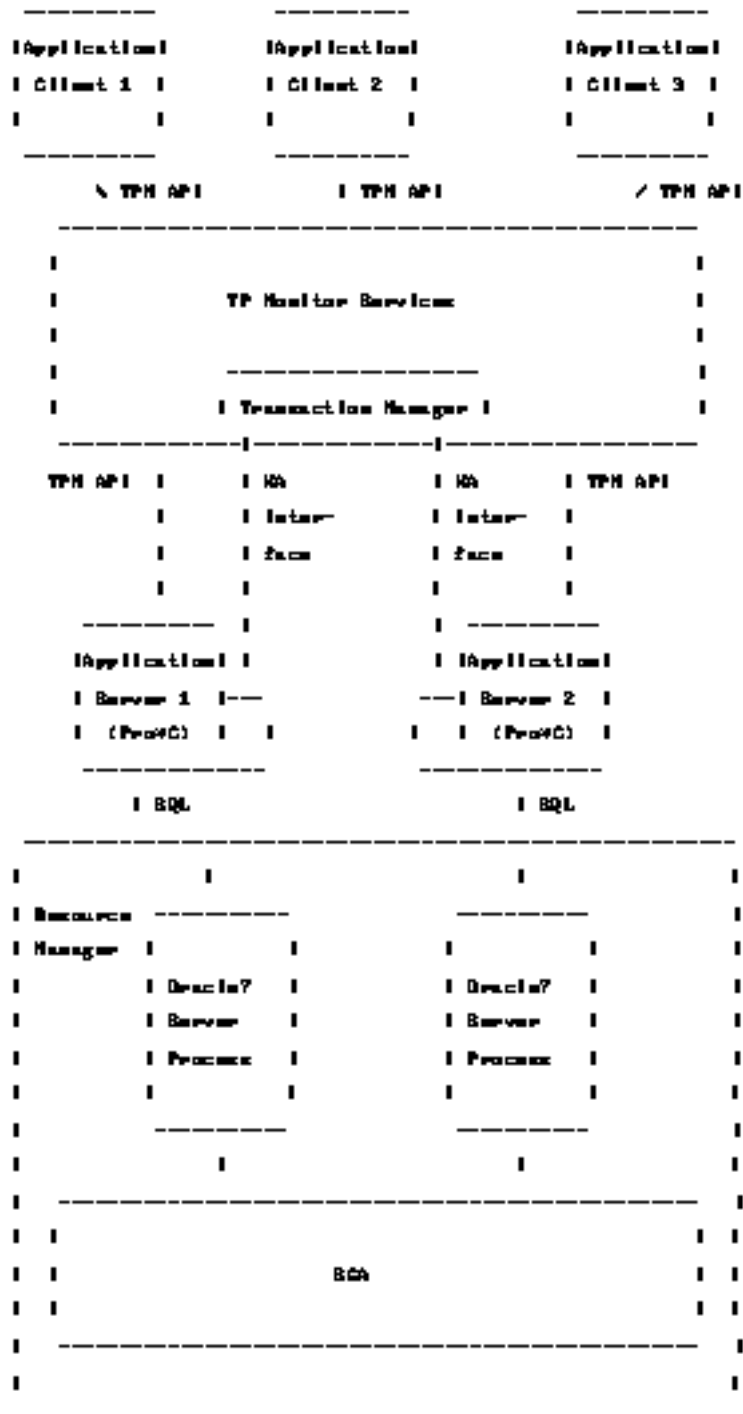
TP Monitor Application Architecture

What might a TP Monitor application look like?

Most TPM applications will consist of two main parts: a front-end client program which request services and back-end server program which provide services. In this case, the TPM supplies an additional capability which is transactional communication. The client describes the boundaries of the transaction, through the use of the TK API, and the TPM relays that transaction information to each requested service.

The overall application structure generally looks like the following in the client-server model. The "TP Monitor Services" box is not necessarily a process. It could be one or more processes, or just libraries coordinating through shared memory. Each client process and server process could be on a different machine. Normally, the application server processes

would be connected to their Oracle Server processes using the IPC driver; the TPM would be used to deliver messages between application client processes on one machine and application server processes on another. However, the application server processes could also be connected with the standard Oracle SQL*Net to shadow processes on different machines. This might be useful if one of the databases was on a machine which did not support TPMs.



Application client programs might be written in C and be linked with TPM libraries.

Alternatively, they could use a screen painter product. Application server programs could be written in Pro*C or Pro*COBOL and be linked with TPM libraries, the normal Oracle user-side libraries and libclnt.s. The Oracle Server process is the regular Oracle executable.

More complicated application architectures can also be constructed. Most of the TPMs allow a server to become a client of another service, so you can involve additional servers.

Could I use Oracle's Multi Threaded Server as the BQL*Net connection in the previous picture?

Yes, but that will not be needed in many cases. For example, both application server processes in the previous picture could talk to a single Oracle Server process through the Multi Threaded Server in the previous picture. However, since the TPM architecture typically reduces the number of server processes, the reduction in processes using Multi Threaded Server may be less significant than in an architecture without TPMs. If the application will use database links, however, then MTS will be required.

How do I write an Oracle TP Monitor application?

The actual API used to talk to the TPM varies between vendors, so you need to get the documentation from the vendor. However, all have a way to indicate where a transaction begins and ends and a way to send a request and receive a response from a client to a server. Some use an RPC model, some use a pseudo-RPC model, and some use a send/receive model. The TK API described earlier is a subset of the TPM API as defined by each of the TPM providers.

The client program and server program might look something like the following examples. Use it (such as Tuxedo's `tpscall`)

ORACLE7 BACKUP 및 RECOVERY 방법

Bulletin no : 10080

DATABASE 의 BACKUP 은 DB 관리자 및 DB 를 사용하는 모든 사용자에게 가장중요한 사항이다. ORACLE 은 DB BACKUP 을위해 몇 가지방법을 제시 하고 있으나 선택은 데이터의중요도, DB 이용도 등에 따라 가장적절한 방법을 택하고 가능하면 여러가지 방법으로 BACKUP 을 할 것이 안전하다.

< BACKUP 및 RECOVERY 방법 >

- . EXPORT & IMPORT
- . INCREMENTAL EXPORT & IMPORT
- . IMAGE BACKUP (COLD, HOT BACKUP)
- . ARCHIVING

1. EXPORT & IMPORT

EXPORT 은 DATA 를 ORACLE DATABASE 으로부터 ORACLE Binary 형태의 DB FILE로 만들고 IMPORT 을 이용하여 DATABASE 로 LOADING 한다.

< EXPORT 의 형태 >

- Entire Database (FULL) - 모든 DATABASE 를 EXPORT 한다.
- User(s) - 특정 USER 의 OBJECT 를 EXPORT 한다.
- Table(s) - 특정 TABLE 을 EXPORT 한다.

< 예 1 > 전체 데이터베이스 EXPORT (Interactive Method)

```
$ exp system/manager
Connected to: ORACLE7 Server Release 7.0.1B.4.0 - Production
With the procedural and distributed options
PL/SQL Release 2.0.1B.1.0 - Production
Enter array fetch buffer size : 4096 > (RETURN)
Export file : expdat.dmp >
(1) E(entire database), (2) U(user), (3) T(table) : (2) U > u
Export grants (Y/N) : Y > y
Export table data (Y/N) : Y > y
Compress exports (Y/N) : Y > y
About to export the entire database...
. exporting tablespace definitions
. exporting profiles
. exporting user definitions
. exporting roles
. exporting rollback segment definitions
. exporting database links
. exporting sequence numbers
. exporting sequence numbers
. exporting cluster definitions
```



```

. exporting stored procedures
. about to export SYSTEM's tables ...
. about to export SCOTT's tables ...
. exporting synonyms
. exporting views
. exporting referential integrity constraints
. exporting triggers
Export terminated successfully without warnings.

```

< 예 2 > 전체 데이터베이스 EXPORT (Command Line Method)

```
$ exp user=system/manager fully compress=y grant=y row=y
```

< 예 3 > 전체 데이터베이스 EXPORT (Dynamic Method)

EXPORT PARAMETERS 를 다음과 같은 FILE(tucc.par) 형태로 만든다.

```

system/manager
fully
compress=y
grant=y
row=y

```

```
$ exp parfile=tucc.par
```

< 예 4 > USER only EXPORT

```

$ exp system/manager
Connected to: ORACLE7 Server Release 7.0.1B.4.0 - Production
With the procedural and distributed options
PL/SQL Release 2.0.1B.1.0 - Production
Enter array fetch buffer size : 4096 > (RETURN)
Export file : expdat.dmp >
(1) Entire database, (2) User(s), (3) Tables : (2) U > u
Export grants (Y/N) : Y > y
Export table data (Y/N) : Y > y
Compress extents (Y/N) : Y > y

```

```

About to export specified users
User to be exported: (RETURN to quit) > scott
. exporting synonyms
. exporting synonym log
. exporting database links
. exporting sequence numbers
. exporting sequence numbers
. exporting cluster definitions
. exporting stored procedures
. about to export SCOTT's tables ...

```

```
. exporting synonyms
. exporting views
. exporting referential integrity constraints
. exporting triggers
Export terminated successfully without warnings.
```

< 예 5 > USER only EXPORT (Command Line Method)

```
$ exp user=system/manager owner=scott compress=grants=rcmp=y
```

< 예 6 > USER ONLY EXPORT (Dynamic Method)

EXPORT PARAMETERS 를 다음과 같은 FILE(tucc.par) 형태로 만든다.

```
system/manager
owner=scott
compress=y
grants=y
rcmp=y
```

```
$ exp parfile=tucc.par
```

< 예 7 > TABLES EXPORT

```
$ exp scott/tiger tables=emp,dept compress=grants=rcmp=y
```

< DATA IMPORTING >

DATA IMPORT 는 EXPORT 와 비슷하고 항상 EXPORT 된 FILE 요만 IMPORTING이 된다.

< 예 8 > 전체 데이터베이스 EXPORT FILE로부터 SINGLE TABLE 들 IMPORT (INTERACTIVE MODE)

```
$ imp system/manager
Connected to: ORACLE7 Server Release 7.0.1B.4.0 - Production
With the procedural and distributed options
PL/SQL Release 2.0.1B.1.0 - Production

Import file: expdat.dmp > (RETURN)
Enter import buffer size (minimum is 4096) 30720 > (RETURN)
Export file created by EXPORT:V07.00.1B
List contents of import file only (yes/no) : no >
( "yes" 이면 data는 Import 되지않고 Display 만점 )
Ignore create errors due to object existence (yes/no) : yes >
("yes" : 만일 table 이 존재하면 record 가 추가된다.
"no" : 이미 존재한 Object 에대해서 Import 를 하지않는다.)
Import grants (yes/no) : yes >
```

```

Import table data (yes/no) : yes >
Import entire export file (yes/no) : yes > no
User name : BCDTT
Enter table name. Null list name all tables for user
Enter table name or . if done : TEST
Importing BCDTT's objects into BCDTT
Importing table TEST      800 rows imported
Import terminated successfully.

```

주의 : IMPORT 작업은 보통 생성되는 INDEX 수에 따라 1.5 - 4 배의 시간이 걸리는데 EXPORT 는 "CREATE INDEX" 문란을 FILE 에 쓰고 IMPORT 시에는 INDEX 를 생성하기 때문이다.

(COMMAND LINE MODE)

```
$ imp system/manager full=generic rowkey tablename=commit
```

주의 : IMPORT 시는 데이터를 DB 에 WRITE 하기 때문에 ROLLBACK BECHMENT가 사용된다. commit은 row 만큼 데이터를 LOAD 하고나서 COMMIT 을 하기 때문에 ROLLBACK BECHMENT 가 커지는것을 피할수있다. commit은 이런 TABLE 단위로 IMPORT 하고나서 COMMIT 이 되기 때문에 ROLLBACK BECHMENT 확장 BECHMENT 가 발생할수있다.

2. INCREMENTAL EXPORT & IMPORT

< INCREMENTAL EXPORT >

INCREMENTAL EXPORT 는 BYB 혹은 SYSTEM USER 로서 EXPORT 한다. 만일 FILE 이름들 "expdat.dmp" 에서 다른 이름으로 변경하고자 할때는 COMMAND LINE 에서 FILENAME 만 지정하면된다.

INCREMENTAL EXPORT 의 3 가지 형태

- . COMPLETE - <예 1> 에서과같이 전체 데이터베이스를 EXPORT 한다.
- . CUMULATIVE - 마지막 "COMPLETE" 혹은 "CUMULATIVE" EXPORT 이후에 변경된 정보를 EXPORT 한다.
- . INCREMENTAL - 마지막 "COMPLETE", "CUMULATIVE" 혹은 "INCREMENTAL" EXPORT 이후에 변경된 정보를 EXPORT 한다.

BACKUP 방법은 매일 1 밀에 "COMPLETE", 주말에 "CUMULATIVE", 주중에는 "INCREMENTAL" 로 하면 가장 적절하다.

< 예 B > INCREMENTAL EXPORT

```

$ exp system/manager full=yes complete file=com0304.dmp
$ exp system/manager full=yes incremental file=inc0305.dmp

```

주의: INCREMENTAL EXPORT 데이터량은 "NEW 혹은 UPDATE" 된 TABLE 의 갯수에 따라 크게 다를수있다. 즉 하나의 큰 TABLE 이 매일 UPDATE 되면 항상 EXPORT 되고 UPDATE 가 안되면 FULL EXPORT 의 1% 정도 될 수있다. 따라서 이방법은 모든 TABLE 이 때 UPDATE 가 발생하면 비효율적인 방법이 된다.

< INCREMENTAL IMPORTING >

< 예 10 >

EXPORT 된 FILE 이 다음과 같다고 하면

```
F : full C : cumulative I : incremental
F1 I1 I2 C1 I3 I4 C2 I5 I6
```

- a. 데이터베이스를 다시 생성한다. (CREATE DATABASE 문장을 사용)
- b. IMPORT 하기 위해 충분한 ROLLBACK SEGMENT 를 추가한다.
- c. 가장 최근 KEY 정보를 IMPORT 한다.

```
$ imp system/manager ictype=SYSTEM full=y file=I6
```

- d. 가장 최근 COMPLETE 도 EXPORT 한 FILE 을 IMPORT 한다.

```
$ imp system/manager ictype=SYSTEM full=y file=F1
```

- e. 모든 CUMULATIVE IMPORT 를 순서적으로 실행한다.

```
$ imp system/manager ictype=system full=y file=C1
```

```
$ imp system/manager ictype=system full=y file=C2
```

- f. INCREMENTAL IMPORT 를 순서적으로 실행한다.

```
$ imp system/manager ictype=system full=y file=I5
```

```
$ imp system/manager ictype=system full=y file=I6
```

3. IMAGE BACKUP

IMAGE BACKUP 은 모든 데이터베이스 FILE 들을 다른 장소로 COPY 하는 것이다. 이 방법은 가장 빠르고 안전한 방법이나 BACKUP 시점까지 데이터를 저장 할 수 밖에 없고, BACKUP 을 하는동안 데이터베이스를 DOWN 해야하는 문제가 있다. 그러나 IMAGE BACKUP 관련된 문제는 대부분 ARCHIVING 과 같이 해결하면 해결 될 수가 있다. 또한 IMAGE BACKUP 은 PERFORMANCE 를 고려하여 BFILE 도 하는 것이 좋다. DATABASE 를 BACKUP 하고나서 TAPE 로 COPY 하면 DB 가 DOWN 되는 시간을 줄일수 있기 때문이다.

- a. 데이터베이스를 정상적으로(NORMAL) DOWN 한다.
- b. 모든 DATA FILES, CONTROL FILES, ONLINE REDO LOG FILE 들을 DB 의 BACKUP UTILITY(UNIX 경우 cp, tar, cpio) 를 사용하여 COPY 한다.

< 복구방법 >

데이터복구는 BACKUP 했던 시점까지 복구되고 이후전경된 데이터는 데이터베이스가 NOARCHIVELOG MODE 이면 복구 불가능 하다.

- a. BFILE 에있는 모든 데이터베이스 FILE (CONTROL, ONLINE REDO LOG, DATA) 를 삭제한다.
- b. COLD BACKUP 을 RECOVER 한다.
- c. 데이터베이스를 BACKUP 한다.

4. ARCHIVING

ARCHIVING 은 주어진 기간동안에 데이터베이스에 발생한 전경사항을 모두 저장하는 방법이다. 따라서 IMAGE BACKUP 을 하고난 이후에 모든 ARCHIVE FILE을 가지고있다면 어떠한 데이터베이스 장애에도 최근에 전경된 내용까지 복구가 가능하다.

- < 예 11 > 필요일 시 : 전체 데이터베이스 INARCH BACKUP
 종료일 : ARCHIVE FILE 에 모든 변경된 내용이 저장
 종료일 : 다음 INARCH BACKUP 전에 DB 상태

< 복구 방법 >

필요일 시 INARCH BACKUP 을 RESTORE 한다.
 모든 ARCHIVE FILE 들 DB 에 적용한다.
 ONLINE REDO LOG (아직 ARCHIVE 되지 않은 내용)을 적용한다.

< 데이터베이스를 ARCHIVELOG MODE 로 전환하는 방법 >

- a. ARCHIVELOG MODE 로 전환하기 위해서 ONLINE REDO LOG FILE 이 3개이상
 되어야한다. (DEFAULT 는 기본적으로 3 개)
 ONLINE REDO LOG FILE 을 추가 하는 방법은 다음과 같다. (DEFAULT 경우)

```
$ cd $ORACLE_HOME
SQL> CONNECT INTERNAL
SQL> STARTUP MOUNT
SQL> ALTER DATABASE ADD LOGFILE 'F:\db\log\redo1.dbf' SIZE 5M;
SQL> ALTER DATABASE OPEN;
```

- b. 자동으로 ARCHIVING 하기 위해서 F:\db\init1.dbf,arc FILE 에 다음내용을
 추가한다.

```
LOG_ARCHIVE_START=TRUE
LOG_ARCHIVE_DEST=/arch/log (/arch DIRECTORY 가 생성 되어야한다)
LOG_ARCHIVE_FORMAT=arc,arc
```

- c. 데이터베이스를 MOUNT 한다

```
$ cd $ORACLE_HOME
SQL> CONNECT INTERNAL
SQL> STARTUP MOUNT
SQL> ALTER DATABASE ARCHIVELOG
SQL> ALTER DATABASE OPEN
```

(ARCHIVING 상태를 DISPLAY 하기 위하여)

```
SQL> ARCHIVE LOG LIST

Database log mode                ARCHIVELOG
Automatic archival                ENABLED
Archive destination               /arch/log
Dissect online log seq.          155
Next log sequence to archive     156
Current log sequence             156
```

데이터베이스를 위와같이 ARCHIVELOG MODE 로 사용하면 ONLINE REDO LOG FILE 이 LOG SWITCH 가 발
 생 할때 ARCHIVER(arc) PROCESS 는 CURRENT ONLINE REDO LOG FILE 을 /arch DIRECTORY 에 COPY 한

다. ARCHIVE FILE NAME는 log15B.arc, log157.arc 등과 같이 LOG SEQUENCE NO 가 FILE NAME에 붙여진다. ARCHIVE PROCSB는 ORACLE BACKGROUNd PROCSB 와 비동기적으로 작동 하기때문에 COPY 하는 동안에 DISK I/O 가 증가되는 현상은 발생하나 데이터베이스의 동작이 중단되는 현상은 발생하지 않는다.

< 복구 >

ARCHIVE FILE 을 사용하여 데이터베이스를 복구하는 경우는

- . MEDIA FAILURE
- . 데이터 FILE 이 DROP 되기전에 REMOVE 했을경우
- . ORACLE BLOCK 이 CORRUPT 된경우
- . ONLINE LOG FILE 이 CORRUPT 된경우 등이다.

복구방법으로는 COMPLETE MEDIA RECOVERY 와 INCOMPLETE MEDIA RECOVERY 등이있다.

방법 1 non-SYSTEM TABLESPACE 데이터 FILE 이 CORRUPT 된경우

CORRUPT 된 DATA FILE 을 복구하거나, TABLESPACE 를 복구한다. TABLESPACE를 복구하는경우 TABLESPACE 에있는 모든 DATA FILE 을 복구한다.

- a. CORRUPT 된 DATAFILE 을 삭제하고 BACKUP 된 FILE 을 RESTORE 한다.

```
$ tar xvf /dev/rmt0 ucw1.dbf
```

- b. CORRUPT 된 DATAFILE 을 복구 한다.(TABLESPACE 복구는 생략)

데이터베이스를 STARTUP 하면 MOUNT까지는 실행되지만 OPEN 할때 DATA FILE 의 LOG SEQUENCE 가 일치하지않기 때문에 OPEN 되지 않는다.

```
BQLBHA > STARTUP MOUNT
```

```
BQLBHA > RECOVER DATAFILE '/uuc/oracle7/dbs/ucw1.dbf';
```

```
ORA-00279: Change 5035 generated at 05/05/84 18:10:00 needed for thread 1
```

```
ORA-00289: Suggestion : /arch/log15B.arc
```

```
ORA-00280: Change 5035 for thread 1 is in sequence # 15B
```

```
Specify log : (suggested | filename | AUTO | FROM logsource |
```

```
CANCEL |
```

(RETURN 을 하면 제시된 ARCHIVE LOG FILE 이 APPLY 한다.)

```
Applying suggested logfile...
```

(계속해서 제시된 ARCHIVE LOG FILE 이 APPLY 하면 최종적으로 "Media recovery complete" MESSAGE 가 나타나면서 복구가 완료된다.)

```
BQLBHA > ALTER DATABASE OPEN;
```

방법 2. SYSTEM TABLESPACE 데이터 FILE이 CORRUPT 된경우

- ㉔. COPYFILE 된 DATAFILE 들 삭제하고 BACKUP 된 FILE 들 RESTORE 한다.
\$ tar xvf /dev/rmt0 exp100A7.dbf

- ㉕. 데이터베이스 전체를 복구 한다.

SQL> STARTUP MOUNT

SQL> RECOVER DATABASE;

```
ORA-00279: Change 5035 generated at 05/05/94 18:10:00 needed for thread 1
ORA-00289: Suggestion : /arch/log15B.arc
ORA-00280: Change 5035 for thread 1 is in sequence # 15B
Specify log : (-suggested | filename | AUTO | FROM logsource |
CANCEL |
```

(RETURN 을 하면 제시된 ARCHIVE LOG FILE 이 APPLY 한다.)

Applying suggested logfile...

(계속해서 제시된 ARCHIVE LOG FILE 이 APPLY 하면 최종적으로
"Media recovery complete" MESSAGE 가 나타나면서 복구가 완료된다.)

SQL> ALTER DATABASE OPEN;

방법 3. CONTROL FILE 과 ONLINE REDOLOG FILE 복구방법

- ㉔. CONTROL FILE 은 서로다른 DISK 에 2 개이상 COPY 하여 분열하면
하나의 FILE 이 CORRUPT 된경우 다른 CONTROL FILE 들 COPY 하여 사용한다.
- ㉕. ONLINE REDO LOG FILE 은 ORACLE7 에서는 HIDDENED ONLINE REDO LOG
방법을 사용하여 분열하면 하나의 GROUP 에있는 모든 REDO LOG FILE
이 CORRUPT 된경우를 제외하고 데이터베이스 분열에 장애를 받지 않는다.

5. HOT BACKUP

데이터베이스를 DOWN 하지않고 INACE BACKUP 하는방법이다. 이방법은 COLD
BACKUP 보다 더 어렵지만 데이터베이스를 OPEN 중에 할수 있고 또한 TABLESPACE
별로 BACKUP 을 할수있는 장점이있다.
HOT BACKUP 은 항상 데이터베이스를 ARCHIVELOG MODE상태에서 실시한다.

< 방법 > TABLESPACE 단위로 BACKUP 을 실시한다.

SQL> CONNECT INTERNAL

SQL> ALTER TABLESPACE SYSTEM BEGIN BACKUP;

(SYSTEM TABLESPACE 의 모든 DATAFILE 에대해서 DB BACKUP 한다)

\$ tar cvf /dev/rmt0 /usr/oracle7/dbs/exp100A7.dbf

\$ tar cvf /dev/rmt0 /usr/oracle7/dbs/exp1200A7.dbf

```
BQLBNA > ALTEN TABLESPACE BYTES END BACKUP;
```

```
BQLBNA > ALTEN TABLESPACE USES BEGIN BACKUP;
```

```
$ tar cvf /dev/rmt0 /usr/obj/obj_107/dbs/ucor100007.dbf
```

```
$ tar cvf /dev/rmt0 /usr/obj/obj_107/dbs/ucor200007.dbf
```

```
BQLBNA > ALTEN TABLESPACE USES END BACKUP;
```

(다 TABLESPACE 에 대해서도 같은 방법으로 한다)

주의: BEGIN 과 END 사이에는 해당 TABLESPACE 의 DATAFILE HEADERS가 편집되지 않는다. 따라서 DB BACKUP 이 종료됨과 동시에 "ALTEN TABLESPACE .. END BACKUP" COMMAND 을 실행하여 DATAFILE의 HEADERS 가 편집되도록 한다.

특구방법은 ARCHIVING 경우의 REDUNDY 방법과 동일하다.

ORACLE7 INSTALL 후

데이터베이스 FILE 의 구성

Bulletin no : 10070

ORACLE7 INSTALL 이 끝나면 ORACLE7 이 작동되기위해서 필요한 기본적인 DATABASE FILE 즉 CONTROL FILE, LOG FILE, DATA FILE 들이 생성 된다.

이러한 FILE 들은 UNIX 밑에서 ORACLE_HOME/dbs 밑에 생기며 INSTALL 과정에서 변경도 가능하다. 이 FILE 들은 항상 정상적인 상태로 존재해야하고 TABLESPACE 및 DATA FILE 들 SHUT, OFFLINE 하기전에 DB 상에서 RECOVER 해서는 안된다. 각 FILE 들의 NAME 과 사용 내역을 알기위해 ORACLE HOME DIRECTORY 과 ORACLE SID 를 확인해야 하는데 ORACLE 환경 변수는 \$ mv 를 해보면 알수있다.

```
ORACLE_SID = ORA7
ORACLE_HOME = /uuser/oracle7
```

1. PARAMETERS FILE

내용 : 각종 PARAMETERS

FILE NAME : initORA7.ora

2. CONTROL FILE

내용 : DATABASE NAME
DB CREATION TIMESTAMP
DATABASE 및 ONLINE LOG FILE의 이름, 위치
현재 LOG SEQUENCE NUMBER
CHECK POINT 정보

FILE NAME : ctf1ORA7.ctf, ctf2ORA7.ctf, ctf3ORA7.ctf

지정된 FILE : configORA7.ora

```
control_files = (/uuser/oracle7/dbs/ctf1ORA7.ctf,  
/uuser/oracle7/dbs/ctf2ORA7.ctf,  
/uuser/oracle7/dbs/ctf3ORA7.ctf)
```

4. ONLINE LOG FILE

내용 : TRANSACTION ID
COLUMN ADDRESS
NEW DATA

FILE NAME : log1ORA7.dbf, log2ORA7.dbf, log3ORA7.dbf

5. DATA FILE

DATAFILE NAME	TABLESPACE	비고
sys001a7.dbf	SYSTEM	DATA DICTIONARY TABLE, CATALOG TABLE 와 VIEW DB TRIGGERS, PACKAGE, PROCEDURE, FUNCTION
redo001a7.dbf	REDO	ROLLBACK SEGMENT (=01,=02,=03,=04)
temp001a7.dbf	TEMP	ORACLE TOOLS(FOYMS, REPORTWRITER 등)의 SOURCE 들 DB 에 저장하기 위해 필요한 TABLE, VIEW 및 INDEX
temp001a7.dbf	TEMP	각종 BODY 들 위한 TEMPORARY 영역
user001a7.dbf	USERS	USERS 데이터

TABLESPACE 구성시 유의사항

- SYSTEM TABLESPACE 에 생성되는 OBJECTS(TABLE, VIEW, etc) 의 수를 최소화시킨다. 만일 새로운 TABLE 들 다른 TABLESPACE 에 생성될 하더라도 SYSTEM TABLESPACE 에 SPACE 가 없으면 그 TABLE 에대한 새로운 정보를 입력하지 못하기 때문에 TABLE 들 생성하지 못한다.
또한 NON-BYTES TABLESPACE 에 다른 OBJECT 들 생성하면 DB 를 관리하기가 용이해진다. SYSTEM TABLESPACE 보다 NON-BYTES TABLESPACE 를 RECOVER 하기가 더쉽고 MEDIA FAILURE, TABLESPACE RESIZING 혹은 DATABASE FILE LAYOUT 의 TUNING 등이 보다 쉬어진다.
- I/O 를 분산하기 위해 TABLE 과 연관된 INDEX 를 분리한다.
여러개의 ORACLE 에 I/O 가 분산되도록 DISE STRIPING 을 사용한다.
- 많은 TABLE 및 INDEX 에 DATA 가 입력되고 REDO 이 되면 TABLESPACE 에 FRAGMENT 가 상당히 발생하여 SPACE를 효율적으로 사용하지 못한다.
따라서 충분한 INITIAL 값으로 TABLE 들 생성하므로써 FRAGMENT 를 감소시킬수 있다.

DB CREATION 및 TABLESPACE 생성 문장(?/dbs/crd001a7.sql, crd020a7.sql)

```
----- crd001a7.sql -----
SQL> Start the instance (ORACLE_SID here must be set to ).
SQL>
connect internal
startup nomount pfile=/usr3/oracle7/dbs/init001a7_0.ora

SQL> Create the database.
SQL> SYSTEM tablespace configuration guidelines:
SQL>   General-Purpose ORACLE REDO      5MB
SQL>   Additional dictionary for applications 10-50MB
SQL> Redo Log File configuration guidelines:
SQL>   Use 3+ redo log files to relieve "cannot allocate new log..." errors.
SQL>   Use "100MB per redo log file per connection to reduce checkpoints.
```

```

RDM >
create database "ORA7"
    maxinstances 8
    maxlogfile 32
    character set "UB7A8C11"
    datafile
        '/usr3/oracle7/dbs/ctr1100A7.dbf' size 25M
    logfile
        '/usr3/oracle7/dbs/ctr1100A7.dbf' size 500k,
        '/usr3/oracle7/dbs/ctr1100A7.dbf' size 500k,
        '/usr3/oracle7/dbs/ctr1100A7.dbf' size 500k;

disconnect

----- c:\ORA7.sql -----
connect internal

RDM > Install data dictionary views:
D:\usr3\oracle7\rdms\admin\Catalog.sql

RDM > Create additional rollback segment in SYSTEM before creating tablespaces.
RDM >
connect internal

create rollback segment r0 tablespace system
storage (initial 18k next 18k minextents 2 maxextents 20);

RDM > Use ALTER ROLLBACK SEGMENT ONLINE to put r0 online without shutting
RDM > down and restarting the database.
RDM >
alter rollback segment r0 online;

RDM > Create a tablespace for rollback segments.
RDM > Rollback segment configuration guidelines:
RDM > 1 rollback segment for every 4 concurrent sessions.
RDM > No more than 50 rollback segments.
RDM > All rollback segments the same size.
RDM > Between 2 and 4 homogeneously-sized extents per rollback segment.
RDM > Attempt to keep rollback segments to 4 extents.
RDM >
create tablespace rbs datafile
    '/usr3/oracle7/dbs/ctr1100A7.dbf' size 4M
default storage (
    initial 128k
    next 128k
    pctincrease 0
    minextents 2

```

```
);
```

```
REM * Create a tablespace for temporary segments.
```

```
REM * Temporary tablespace configuration guidelines:
```

```
REM * Initial and next extent sizes = k * DBT_AREASIZE, k in {1,2,3,...}.
```

```
REM *
```

```
create tablespace temp datafile
```

```
  '/u03/oracle7/dbs/ctrl00A7.dbf' size 550k
```

```
default storage (
```

```
  initial      258k
```

```
  next         258k
```

```
    pctincrease 0
```

```
    optimal     1M
```

```
);
```

```
REM * Create a tablespace for database tools.
```

```
REM *
```

```
create tablespace tools datafile
```

```
  '/u03/oracle7/dbs/ctrl00A7.dbf'      size 15M;
```

```
REM * Create a tablespace for miscellaneous database user activity.
```

```
REM *
```

```
create tablespace users datafile
```

```
  '/u03/oracle7/dbs/ctrl00A7.dbf'      size 1M;
```

```
REM * Create rollback segments.
```

```
REM *
```

```
create rollback segment r01 tablespace rbs;
```

```
create rollback segment r02 tablespace rbs;
```

```
create rollback segment r03 tablespace rbs;
```

```
create rollback segment r04 tablespace rbs;
```

```
REM * Use ALTER ROLLBACK SEGMENT ONLINE to put rollback segments online
```

```
REM * without shutting down and restarting the database. Only put one
```

```
REM * of the rollback segments online at this time so that it will always
```

```
REM * be the one used. When the user shuts down the database and starts
```

```
REM * it up with initSID.ora, all four will be brought online.
```

```
REM *
```

```
alter rollback segment r01 online;
```

```
REM * alter rollback segment r02 online;
```

```
REM * alter rollback segment r03 online;
```

```
REM * alter rollback segment r04 online;
```

```
REM * Since we've created and brought online 2 more rollback segments,
```

```
REM * we no longer need the second rollback segment in the SYSTEM tablespace.
```

```
alter rollback segment r0 offline;
```

```
drop rollback segment r0;
```

```
REM * Alter SYS and SYSTEM users.
```

```
REM *
```

```
alter user sys temporary tablespace temp;
```

```
alter user system default tablespace tools temporary tablespace temp;
```

```
REM * For each DBA user, run DBA synonyme SQL script. Don't forget that EACH
```

```
REM * DBA USER created in the future needs dba_syn.sql run from its account.
```

```
REM *
```

```
connect system/manager
```

```
@/usr3/oracle7/rdbsa/admin/catdba_syn.sql
```

ORACLE7 에서 USER 등록

Bufile는 : 10088

단계 1 USER 가 사용하고자하는 TABLESPACE 생성 한다. 만일 존재하는 TABLESPACE 를 사용하더라도 단계는 생략한다.

```
$ sqlldr load=Y
SQL> CONNECT INTERNAL
> CREATE TABLESPACE data_tx
DATAFILE '/usr/oracle/dbs/data_tx.dbf' SIZE 50M;
```

단계 2 USER 를 생성하면서 DEFAULT TABLESPACE, TEMPORARY TABLESPACE 등을 지정하고 각 TABLESPACE에 USER 가 사용 할수 있는 QUOTA 를 할당 한다.

제한없이 사용하고자 할때는 UNLIMITED 로 한다.

```
SQL> CREATE USER user1 IDENTIFIED BY user1_pass
DEFAULT TABLESPACE data_tx
TEMPORARY TABLESPACE temp
QUOTA UNLIMITED ON data_tx
QUOTA UNLIMITED ON temp;
```

단계 3 USER에게 ROLE 을 GRANT 한다. (DB에서는 USER 가 TABLE 생성 및 ACCESS 를 하려면 RESOURCE, CONNECT 권한을 부여하였으나 U7에서는 CONNECT ROLE 만 부여하여도 된다.)

```
SQL> GRANT CONNECT TO user1;
```

단계 4 DB 에 CONNECT 하여 사용한다.

```
sqlplus user1/user1_pass
SQL> CREATE TABLE test(col1 VARCHAR2(2)
col2 NUMBER);
```

위와 같이 TABLESPACE 를 생략하고 TABLE 을 생성하는 경우 USER 의 DEFAULT TABLESPACE 인 data_tx 에 TABLE 이 생성된다.

ORACLE의 MEMORY 사용량에 대한 테스트

Bulletin no : 10175

ORACLE 7.1 의 MEMORY 요구사항

오라클의 메모리 사용량은 시스템에 따라서 다르지만 큰 차이는 없으므로 여기서는 BENCHMARK 기종에서 테스트 결과를 알아보도록 한다. 사용된 시스템은 각각 12MB(MAINMACHINE), 8MB(BENCHMARK FOR BENCHMARK) 의 MAIN MEMORY 를 갖는 BENCHMARK 기종이다.

88MB VERBIDN 은 7.1.2 이고 BENCHMARK TCP/IP 2.1 을 사용하였다.

TEST 절차 및 측정결과

1)Initiator 기종 : 약 800 KB 소요

2)ORACLE startup : 약 8 MB 소요

kernel back ground process 가 BDWM, PMDM, DBWR, LGWR, RECO, ARCH 등 총 8 개가 fork 될때의 경우이고, 만약 dispatcher 나 shared server를 하나 더 띄우면 하나당 약 1MB 가 더 소요 된다.

3)sqlplus scott/tiger 경우 : 약 2MB 소요

이 경우는 sqlplus 명령어 자체와 Dedicated ORACLE Server 1개가 동시에 기동되어 소요되는 양을 합친값

4)sqlplus scott/tiger@MAINMACHINE:DBA? 경우 : 약 1.5 MB 소요

이 경우는 sqlplus 명령어 자체만이 소요하는 메모리 양을 측정하기 위한 것임. 위의 3) 경우와 조합하면 sqlplus 는 1.5 MB , Server는 0.5 MB 소요한다는 결론이 나오는데 Server가 500 KB 만 소요한다는 것은 Pipe를 사용하여 연결되기 때문이다.

5)BENCHMARK 에서 sqlplus scott/tiger@BENCHMARK 경우 : 0 MB

이 경우는 BENCHMARK에서 MAINMACHINE으로 BENCHMARK U2 NTB를 이용하여 연결 경우인데 Main test machine 인 MAINMACHINE에서 아주 프로세서도 추가로 기동되지 않고 다만 기존의 이미 기동된 dispatcher 를 통해 연결된다.

결론적으로 NTB로 연결할 때에는 이미 기동된 dispatcher와 shared server가 이를 흡수하므로 하나의 dispatcher와 server 가 각각 20 명의 Client를 감당한다고 하면

dispatcher : 1MB/20 = 50KB,

shared server : 1MB/20 = 50KB

이므로 사용자 1명당 100KB 정도로 계산하면 된다. 사용자 수가 많아지면 BGA Blk 조정도 필요하므로 이점을 감안하여 전체 메모리 소요량을 산정해 야 한다.

6)BENCHMARK에서 sqlplus scott/tiger@MAINMACHINEd 경우: 약 1 MB 소요

이 경우는 BENCHMARK U2를 이용하여 dedicated server를 사용하여 MAINMACHINE에 연결된 경우 이다.

Option 으로 "(BIB-BE BICATE)" 을 (BIB-) 다듬에 추가하면 BQWent UI 처럼 사용할 수 있다.

전체 메모리 소요량 산정 예 : 동시 사용자 100 명을 가정

1)Multi-thread Shared Server 사용하여 연결 : Best case

- BGA : 30 MB
- Kernel Process : 약 7MB (Listener 포함)
- dispatcher 및 shared server : 10 MB (100KB * 100명)
- 총 합계 : 약 47 MB

2)Dedicated Server 사용하여 연결 : Worst case

- BGA : 30 MB
- Kernel Process : 약 7 MB (Listener 포함)
- Dedicated Server : 100MB (1MB * 100명)
- 총 합계 : 약 137 MB

위의 두 경우는 Client/Server 경우 Server의 메모리 소요량이고 Host base 인 경우에는 해당 Tool 의 메모리 소요량도 같이 계산해야 한다.

OS 명령으로 DATA FILE 을 삭제한 경우

- ORA-1157,1110

Bulletin no : 10088

DATABASE RECOVERY에 앞서 ORACLE INSTANCE(즉, ORACLE RDBMS)의 STARTUP 단계를 우선 살펴보기로 하자.

첫번째 단계로 INSTANCE를 START시키며, 여기서는 initORACLE_BID.ora 파일의 파라미터를 참조하여 SGA(SYSTEM GLOBAL AREA)를 할당하고 백그라운드 프로세스를 START 시킨다.

두번째 단계로 DATABASE의 MOUNT이며 파라미터 파일에 명시된 CONTROL FILE을 모은다. CONTROL FILE로부터 DATABASE NAME과 REDO LOG FILE의 이름을 읽는다.

세번째 단계로 CONTROL FILE의 정보를 이용하여 모든 데이터 파일을 모은다.

```
SQLORA> CONNECT INTERNAL;
Connected.
SQLORA> STARTUP;
ORACLE instance started.
Database mounted.
Database opened.

Total System Global Area      1813188 bytes
      Fixed Size                27784 bytes
      Variable Size            1787128 bytes
Database Buffers               85536 bytes
Redo Buffers                   32788 bytes
```

DBMS의 STARTUP시 문제의 데이터 파일이 CONTROL FILE 정보에서는 존재하지만 실제로 D/B상에서는 존재하지 않으므로 DATABASE OPEN 단계에서 삭제된 데이터 파일을 OPEN 할 수 없다. 따라서 다음과 같은 데이터 파일 오류에 관련된 메시가 발생된다 :

```
SQLORA> STARTUP;
ORACLE instance started
Database mounted
ORA-01157 : cannot identify data file 11 - file not found
ORA-01110 : data file 11 : '/u0001/oracle7/dbs/u0002.dbf'
Attempting to dismount database .... Database dismounted
Attempting to shutdown instance .... ORACLE instance shut down
```

DATABASE OPEN 단계에서 CONTROL FILE에서는 ORA-1157 메시에서 나타난 11번 데이터 파일이 존재하는 것으로 인식하지만 실제로 D/B상의 데이터 파일

(ORA-1110 메시에 명시된 '/u0001/oracle7/dbs/u0002.dbf' 파일)이 삭제된 상태이다.

이러한 경우에는 DATABASE STARTUP시 STARTUP MOUNT 단계까지 실행한 후, 문제의 데이터 파일을 OFFLINE 시킨 다. 데이터베이스를 오픈한다. 단 데이터베이스 오픈이 정상적으로 수행되면 문제가 발생한 데이터 파일을 포함하고 있는 TABLESPACE를 DROP하지 않을 경우에는 DATABASE STARTUP시 항상 데이터 파일의 오픈 단계에서 예외가 발생된다. 따라서, 문제의 데이터 파일의 OFFLINE과 TABLESPACE의 DROP전에 반드시 해당 TABLESPACE를 사용하고 있는 USER의 데이터 파일을 수행해야 한다.

데이터 파일의 OFFLINE과 관련된 명령은 다음과 같다.

SQL*Plus를 COMMAND LINE MODE로 기동시킨다.

```

$ sqlplus /nolog
SQL*Plus> CONNECT INTERNAL;
SQL*Plus> STARTUP MOUNT;
        ORACLE instance started.
        Database mounted.
SQL*Plus> ALTER DATABASE DATAFILE '/u0001/oracle7/dbs/u0002.dbf'
        OFFLINE DROP;
        Statement processed.
SQL*Plus> ALTER DATABASE OPEN;
        Statement processed.
SQL*Plus> DROP TABLESPACE tablespace_name INCLUDING CONTENTS;
        Statement

```

정상적으로 DATABASE가 오픈된 후 CONTROL FILE로 부러의 데이터베이스 정보를 갖는 DATA DICTIONARY TABLE인 USDATAFILE(BYB USER에서 액세스 가능)의 내용과 데이터베이스 파일에 관한 정보를 가지고 있는 DATA DICTIONARY VIEW인 DBA_DATA_FILES(BYB SYSTEM USER)을 조회하면 아래와 같은 내용을 확인할 수 있다 :

(1) SQL> SELECT * FROM USDATAFILE :

FILES	STATUS	NAME
8	ONLINE	/u0001/oracle7/dbs/toolc.dbf
10	ONLINE	/u0001/oracle7/dbs/u0001.dbf
11	RECOVER	/u0001/oracle7/dbs/u0002.dbf

(2) SQL> SELECT * FROM DBA_DATA_FILES :

FILE_NAME	FILE_ID	TABLESPACE_NAME	STATUS
/u0001/oracle7/dbs/toolc.dbf	8	TOOLB	AVAILABLE
/u0001/oracle7/dbs/u0001.dbf	10	TEST	AVAILABLE
/u0001/oracle7/dbs/u0002.dbf	11	TEST	AVAILABLE

OUTER JOIN의 UIDE LINE

Bulletin no : 10388

1. 개념

1.1 Outer Join의 정의

다음의 용어에 대해 우선 살펴보자 :

'outer join column' - a column reference followed by the
symbol (+), e.g. EMPNO(+) and
DEPT, DEPTNO(+) are outer join column

'simple predicate' - a logical expression containing no
AND's, OR's, or NOT's (usually a
simple relation such as A = B)

'outer join predicate' - a simple predicate containing one
or more outer join column

2. OUTER JOIN 사용법 - RULES

An outer join predicate may only contain outer join column from one table (in other words, all outer join column in a simple outer join predicate must belong to the same table). This means, for example, that the following statement is illegal :-

EMP.EMPNO(+) = DEPT.DEPTNO(+) - outer join column from
two tables

Also, if a column in a predicate is an outer join column, then all column from the same table must be outer join column in that predicate. This means, for example, that the following statement is illegal :-

EMP.BAL + EMP.COMM(+) = SALGRADE.HIGH - mixed column from
one table

In a predicate, the table referenced with a (+) is directly 'outer joined' to all other tables in the predicate. It is indirectly 'outer joined' to any tables to which those other tables are themselves 'outer joined'. A predicate may not be directly or indirectly 'outer joined' to itself. This means, for example, that the following combination of predicates is illegal :-

```

EMP, EMPNO(+) = PERS, EMPNO
AND PERS, DEPTNO(+) = DEPT, DEPTNO
AND DEPT, JOB(+) = EMP, JOB      - circular outer
                                  join relationship

```

3. OUTER JOIN EXECUTION

For a given table, T, there may be both outer join and non-outer join predicates. Execution occurs (conceptually) as follows :-

1. The result of joining all tables mentioned in table T's outer join predicate is formed (by recursive application of this algorithm).

2. For each row of the result, a set of composite rows is formed, each consisting of the original row in the result joined to a row in table T for which the composite row satisfies all of table T's outer join predicates.

3. If a set of composite rows is the null set, a composite row is created consisting of the original row in the result joined to a row similar to those in table T, but with all values set to null.

4. Rows that do not pass the non-outer join predicate are removed.

This may be summarized as follows. Outer join predicates (those with (+) after a column of table T), are evaluated BEFORE table T is augmented with a null row. The null row is added only if there are NO rows in table T that satisfy the outer join predicates. Non-outer join predicates are evaluated AFTER table T is augmented with a null row (if needed)

4 OUTER JOIN – RECOMMENDATIONS

Certain types of outer joins in complicated logical expressions may not be well formed. In general, outer join columns in predicates that are branches of an OR should be avoided. Inconsistencies between the branches of the OR can result in an ambiguous query, and this may not be detected. It is best to confine outer join columns to the top level of the 'where' clause, or to nested AND's only.

5 OUTER JOIN – ILLUSTRATIVE EXAMPLES

5.1 Simple Outer Join

```

SELECT ENAME, LOC
FROM   DEPT, EMP
WHERE  DEPT, DEPTNO = EMP, DEPTNO(+)

```

The predicate is evaluated BEFORE null augmentation. If there is a DEPT row for which there are no EMP rows, then a null EMP row is concatenated to the DEPT row.

5.2 Outer Join With Single Post-Join Predicate

```
SELECT ENAME, LOC
FROM   DEPT, EMP
WHERE  DEPT.DEPTNO = EMP.DEPTNO(+)
AND    EMP.DEPTNO IS NULL
```

The second single predicate is evaluated AFTER null augmentation, since there is no (+), removing rows which were not the result of null augmentation and hence leaving only DEPT rows for which there are no corresponding EMP row.

5.3 Outer Join With Additional Pre-Join Predicate

```
SELECT ENAME, LOC
FROM   DEPT, EMP
WHERE  DEPT.DEPTNO = EMP.DEPTNO(+)
AND    'CLERK'      = EMP.JOB(+)
AND    EMP.DEPTNO IS NULL
```

The predicate on EMP.JOB is evaluated at the same time as the one on EMP.DEPTNO - before null augmentation. As a result, a null row is suggested to any DEPT row for which there are no corresponding clerks in the EMP table. Therefore, this query displays departments containing no clerks.

Note that if the (+) were omitted from the EMP.JOB predicate, no rows would be returned. In this case, both the EMP.JOB and EMP.DEPTNO IS NULL predicates are evaluated AFTER the outer join, and there can be no rows for which both are true.

PARALLEL LOAD - SPACE USAGE

Bulletin no : 10052

I have put together a short summary of how Parallel Load handles storage allocation in 7.1 which I am going to send to a customer.

This could also quickly turn into a bulletin or PR entry (if one doesn't already exist).

I'd appreciate if people could review and give corrections/enhancements.

Here is how storage allocation works with parallel loads.

For each parallel loader session, Oracle

- 1) creates a temporary segment with the INITIAL and NEXT from the table that is being loaded, and in the datafile designated by the FILE option.
- 2) When a load is complete, the final extent used by that parallel load is trimmed back to the last block which it used.
- 3) Lastly, the extents belonging to the temporary segments are then converted to extents belonging to the table being loaded.

This creates a couple of problems:

- * If your INITIAL and NEXT are different for the table, then you will end up with multiple extents sized the same as INITIAL, which doesn't follow general table growth patterns.
- * Extent sizes are not predictable for the last extent of each load. This can lead to tablespace fragmentation.

What to do about this?

- * Size INITIAL and NEXT to be the same.
This will eliminate the first problem listed.

- * Oracle 7.2 will have a feature to specify the storage parameters used by the parallel load (bug 187318):

```
OPTIONS (STORAGE=(MINEXTENTS =  
                MAXEXTENTS =  
                INITIAL      = [KIM]  
                NEXT        = [KIM]  
                PCTINCREASE =))
```

- * Enhancement Request 238552 has been filed to allow the user to select no trimming of the extents.

(V7.3) PARTITION VIEWS IN ORACLE7 RELEASE 7.3

Bulletin no : 10234

Introduction

Partition views are a new feature in Release 7.3 design to provide better support for very large tables that are commonly found in data warehousing environments.

The partition view feature offers two primary benefits:

- Improved manageability and availability of large tables
- Improved performance for table scans on certain queries of these large tables

What is a partition view?

An example partition view is:

```
create view sales as
  select * from jan_sales
union all
  select * from feb_sales
...
union all
  select * from dec_sales
```

Each of the base tables (the monthly sales tables) must be identical in terms of column names, column datatypes, and indexes. These tables must also have a CHECK constraint on its partitioning column (thus, the jan_sales table must have a check constraint on the date column which constrains the date to fall between Jan 1 and Jan 31). All of these base tables, indexes, constraints as well as the UNION-ALL view definition are to be created by the DBA.

Manageability and availability benefits

A partition view greatly simplifies the administration of very large tables.

Consider the example of a data warehousing containing a large 'sales' table. Once per month, the DBA loads all of new sales data into this table. Thus, the DBA would need to drop all of the indexes on the sales, load the new data, and rebuild the indexes. Since the sales table is very large, this could be a lengthy operation. Moreover, the sales table is (for most practical DBS applications) is not available while these load and index operations are occurring.

Using the partition view feature, the DBA could load the new month's data into a separate partition and build indexes on this new partition without impacting the original partition view. Thus, after the new partition is entirely built and indexed, the DBA could recreate the UNION-ALL view to include the new partition. The sales partition view is unavailable for a very short length of time ... only while the UNION-ALL view is being built. Moreover, because the indexes are much smaller, the length of time to load and index a new month's worth of data is dramatically decreased.

Performance benefits of partition views

Any UNION-ALL view (even in earlier releases of Oracle®) can reap the aforementioned manageability benefits; however, unless the UNION-ALL view offers reasonable query performance, the manageability benefits are useless.

The enhancement in Release 7.3 is to insure that the query performance on UNION-ALL views will at least equivalent to (and, in many cases, much better than) single-table access. Note that these performance enhancements are only effective when all of the partitions have the appropriate CHECK constraints and when all of the partitions have identical column definitions and indexes.

There are two basic performance enhancements for partition views:

- partition elimination

- parallel execution of UNION-ALL views

Certain queries may not require data from all of the partitions of a view partition. For example, consider the following query:

```
select sum(revenue) from sales
where sales_date between '15-JAN-88' and '15-FEB-88';
```

With 7.3's new support for partition views, Oracle will evaluate the above query using *only* the January partition and the February partition; the remaining ten partitions will not be accessed. This feature is commonly called 'partition elimination'. Partition elimination is only effective when querying based on the partitioning column; in this example, the partitioning column is the sales_date column. But the performance savings can be significant. In the previous example, the partition view feature results in ten of the twelve partitions being eliminated from query processing. This could represent six-fold performance gain.

An additional enhancement in 7.3 is the parallel execution of UNION-ALL views. All queries on UNION-ALL views can be executed in parallel (when using the Parallel Query Option). It is *very*

important to note that the partitioning scheme is absolutely independent of the degree of parallelism (this starkly contrasts with many of our competitor's parallel query architecture, in which the physical data partitioning determines the degree of parallelism). Oracle will dynamically distribute the data of a UNION ALL view across all parallel query processes, and partition elimination will not impact the degree of parallelism.

Limitations of Partition Views

Partition views do not support DML operations. For this reason, partition views are most appropriate for read-only applications (such as data warehouses).

Conclusion

Partition views can be very effective for handling very large tables in data warehousing environments. The manageability of these large tables is vastly improved, with significant performance improvements for many queries.

PRIMARY KEY 재 생성 방법

Bulletin no : 10232

1. PRIMARY KEY DROP

```
BQL> alter table EMP drop primary key;
```

2. PRIMARY KEY 생성

```
BQL> alter table EMP add constraint EMP_PK primary key(EMPNO)  
using index storage(initial is next is pctincrease 0)  
tablespace USERS;
```

PRIMARY KEY를 REFERENCE 하는 FOREIGN KEY 찾는 SQL

Bulletin no : 10314

```
$ sqlplus sys/manager@BYB user & login 함)
```

```
sql> select c.name constraint_name
```

```
      from dba_objects a,
```

```
      constraint b,
```

```
      constraint c
```

```
where a.object_name = 'TABLE_NAME'
```

```
      and a.object_id = b.object#
```

```
      and b.constraint = c.constraint
```

```
/
```

PROC 의 실행파일 크기를 줄이는 방법

Bulletin no : 10242

- Oracle Server 7.2 이상에서는 Shared Library 를 제공하므로 이것을 사용하여 Compile 하면 실행 파일의 크기를 줄일수 있다.

[실행방법]

1. Proc 2.4 의 경우

```
$ cd $ORACLE_HOME/proc/dmso
$ make -f proc.mk compiled DBA_CLIENT_LIB=shared
또는
$ make -f proc.mk RDB=xyc RDTB="xyc.o" DBA_CLIENT_LIB=shared
```

2. Proc 1.5 의 경우

```
$ cd $ORACLE_HOME/proc1B/dmso
$ make -f proc1B.mk compiled DBA_CLIENT_LIB=shared
```

ORACLE SERVER AND UNIX TRANSACTION PROCESSING(TP) MONITOR-2

Bulletin no : 10022

This file contains commonly asked questions about Oracle[®] Server and UNIX Transaction Processing Monitors (TPMs). The topics covered in this article are

- o Oracle Parallel Server and TP Monitors
- o Oracle and DCE-based TP Monitors
- o Other commonly asked questions

The questions answered in part 3 provide additional detail to the information provided in part 1.

Oracle Parallel Server and TP Monitors

How does Oracle Parallel Server (OPS) work with TP Monitors?

If you are using Oracle-managed transactions, there are no special considerations. But if you are using TPM-managed transactions, and thus need to use the KA interface, then Oracle requires release 7.1.3 or later and a special version of the Distributed Lock Manager, called

the enclave-based lock manager. This version of the DLM is not yet available for all platforms. To understand this restriction, let's take a look at one of the technical details of KA.

The KA specification requires that the Resource Manager be able to move a transaction from one process to another, and even to be able to commit in a separate process. In Oracle, transactions are attached to enclaves, so that users that we also have to be able to move enclaves. Therefore, the enclave/transaction can't have any state which is tied to a particular process. The first generation distributed lock managers were all built to use the process id as the lock owner, which doesn't work for locks which need to move with the transaction.

Oracle and DCE-based TP Monitors

How does Oracle interface to the Encina TP monitor? To CICS/B000? I've heard that they require DBF DCE facilities in order to run?

Oracle interfaces to Encina and CICS/B000 just as it does to any other TP Monitor. The TP Monitor issues KA commands to control transactions, and Oracle executes the commands. Encina

and CICS/BOO do use BCE features for their own operation. However, this use is transparent to the Oracle Server.

What BCE facilities can Oracle products take advantage of when working with a BCE-based TP Monitor?

The two most commonly mentioned BCE features which might be useful to Oracle users are multi-threading and security. We look at these in the subsequent questions in this section.

Oracle documentation suggests that a Resource Manager such as Oracle can be either single-threaded or multi-threaded? Which way is Oracle RA implemented?

The Oracle RA implementation is single-threaded, as is any Oracle client.

Within a single process, at most one thread can access Oracle at a time.

Does that mean that only a single Oracle application can access an instance of Oracle transactionally at any given moment?

No. Oracle RA is only single-threaded within a single application server process. Multiple applications can access Oracle simultaneously using RA by using different application processes. Oracle allows

(1) serial reuse of a single server by different clients. There are two options for this. The server can use long term reservation but be defined to be in shared or concurrent access mode, which allows the server to be used by another client as soon as an MPC completes. Alternatively, the server can use default reservation and exclusive mode, which allows the server to be used by another client as soon as the current transaction ends.

(2) concurrent execution by multiple servers, even if they are accessing the same Oracle database. These may be executing the same or different procedures.

These two features should let you get as much concurrency as you need.

Why isn't the Oracle RA library multi-threaded?

The RA specification specifically states that its use of the phrase "thread of control" means a process. If an RM were to multi-thread its RA, it would be in violation of the specification. This restriction was put place in because at the time the specification was written, there were numerous thread packages; if the TM used one, the application another, and perhaps the RM yet a third, there's no way it could work. As threads standards settle down, the later versions of RA will probably relax this restriction.

Will Oracle change if the RA specification changes?

Very likely. The exact time frame will of course depend on the priority of all work items at that time.

Does Oracle use DCE security via the TP Monitors?

The integrity of the connection between a DCE TP Monitor client and DCE TP Monitor server is protected by the DCE security functionality. Theoretically, the TP Monitor could make the DCE-protected client security information available to Oracle. Unfortunately, there's no standard way for a TP Monitor to pass security information information to a Resource Manager such as Oracle. Oracle is leading an effort to extend the K/Dyns model to allow use of the security information provided by the Monitor. In the meantime, the basic DCE security features such as encryption are useful within TP Monitors.

Effective use of DCE security would normally also mean that the security of the TP Monitor client be passed through the TP Monitor, through the Oracle client (application server), to the Oracle Server, and possibly on to other Oracle Servers through database links. The ability to transfer security information to other processes, called delegation, is missing in DCE version 1.0. DCE version 1.1, expected to emerge in late 1994, has some delegation features. Oracle is examining these features to see how they might be used.

Are there any special considerations for CICS/B000?

There are two:

(1) It is inefficient to run without KA. CICS/B000 is designed to use KA. It uses KA so that the CICS server can log on to Oracle when it starts, after which it makes that Oracle connection available to any transaction it executes. If you don't use KA, the CICS server does not itself log on to Oracle so each transaction has to log on and log off - a very expensive mode of operation. Also, it is very un-cics-like in that the application does the {login,off} and also commits - in a mainstream CICS database program CICS would implicitly do these operations. Oracle does not recommend this mode because of the performance penalty.

(2) CICS servers are generic and dynamically load application modules.

In order for these modules to access the Oracle connection made by CICS, the applications must be built with a shared object version of the Oracle libraries. This is an installation option on platforms which support CICS/B000 and other products using its architecture such as CICS B000.

Other commonly asked questions

What other Resource Managers can be included in an Oracle KA transaction?

Several other relational database vendors have an KA implementation available or in progress. There is an KA C-IBAM product from Comshare Telecomputing. There are also Resource Managers contained within some of the TP Monitors which can be coordinated in the same transaction. For example, CICS/B000 has UBAH files and other data stores, Encina has its RQS queuing system, and Tuxedo has its /Q queuing system.

What is Recoverable Queuing Service (RQS) and how does it interoperate with Oracle7 and Execlm? What about /Q?

Recoverable Queuing Service is a feature provided by Execlm which allows transactional, distributed queuing (enqueue/dequeue). Tuxedo has a similar product called /Q. Because these products are themselves coordinated by the TM component of the TP Monitor, their queue operations are atomically coordinated with with operations on KA Resource Managers such as Oracle7 Server. That is, they can atomically put something on one of their queues and commit as Oracle transaction, then at some later time dequeue as entry atomically with doing some other Oracle transaction. The queue system guarantees that the message will not be lost or transmitted twice.

Can I mix TP Monitor applications with standard Oracle7 Server applications?

Yes, you can have existing Oracle applications connected to the database with alongside TPM applications against the same database. The TPM does not manage the whole database, just those transactions which are started by the TPM. The Oracle Server will properly handle concurrency control between the transactions managed by itself and those managed by the TPM.

Is Oracle planning to change its tools to be more suitable for TP Monitors?

With Oracle Procedure Builder 1.5, to be available with CME2, Oracle will provide a foreign function interface that allows you to dynamically set up PL/SQL calls that access C functions. In other words, you can access C routines in Windows DLLs from within your PL/SQL procedures. This will allow PL/SQL under Windows easy access to TP Monitor APIs.

Does Oracle7 Server itself use KA-compliant TPMs as the interface to foreign DBs?

No, for this purpose Oracle Server uses the BQL/Connect products or the new Transparent and Procedural Gateway products.

Does Oracle7 Server use KA to coordinate Oracle7-only distributed transactions?

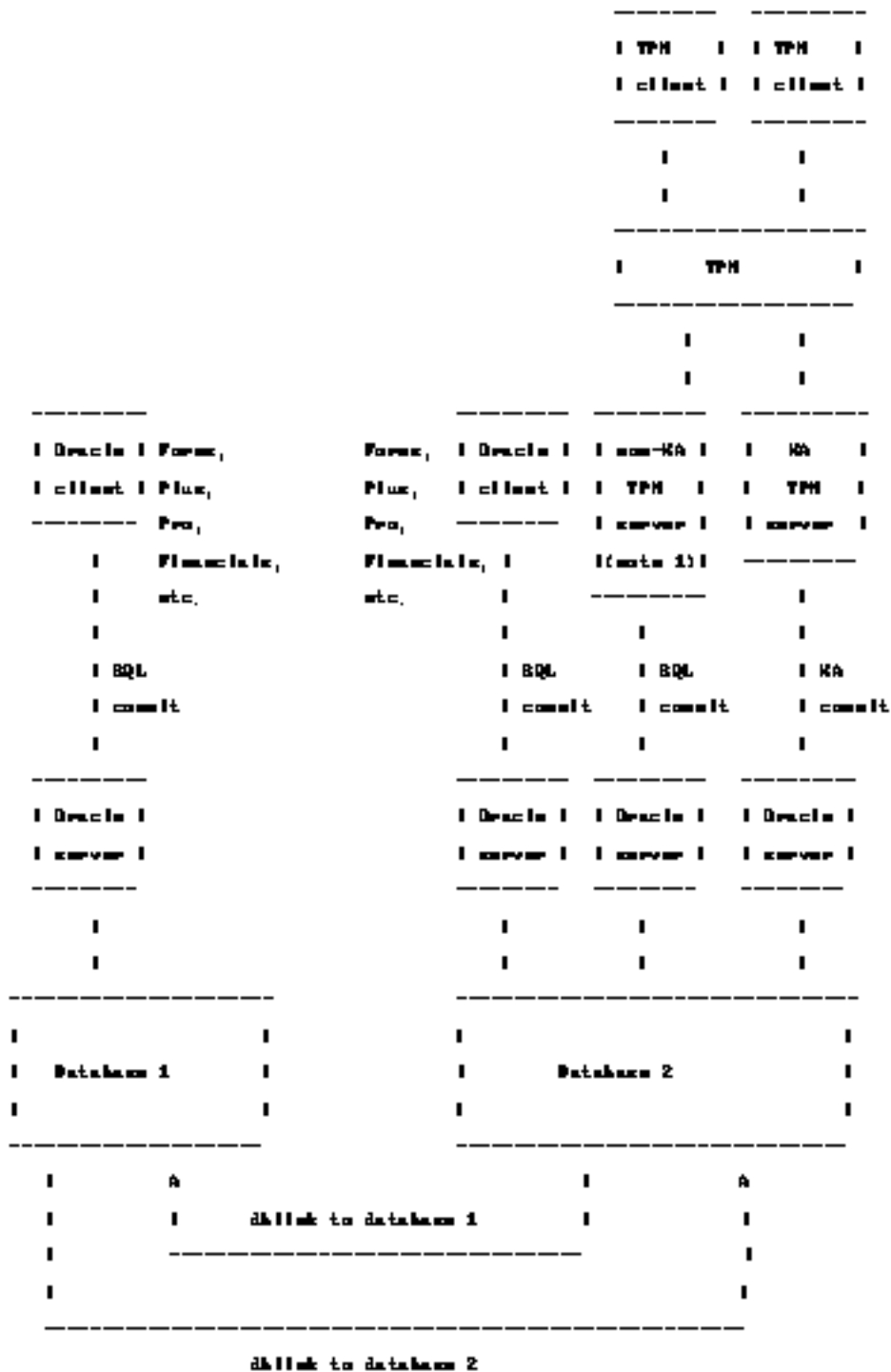
No, it uses an internal mechanism.

Can database links be used with KA?

If an Oracle7 database is running under KA, it can access other Oracle7 databases through database links, with some restrictions. First, the access to the other database must use BQLNet V2 and be running NTB. Second, it must currently be to another Oracle7 database. Assuming those restrictions, the Oracle 7 database can do distributed updates to another Oracle 7 database by using a database link, whether it is started by an Oracle application or a TP Monitor application. The TPM will see Oracle as only a single RM, but Oracle7 will propagate all the transaction commands to the other database, including the two-phase commit. If the transaction is started by a TP Monitor application and is using KA, it can also update

non-Oracle resources managed by the TPM. If it is started from an Oracle application, it can only include resources managed by Oracle.

Here's a sample configuration:



Note 1: Oracle will work having both KA and non-KA servers but some TPMs may have restrictions on this.

Are multiple direct connections possible from a Pro4 program?

Using KA, you can not only specify multiple direct connections to Oracle7 database, you can also update them both in the SAME transaction. The way to do this is to use a precompiler feature called a named database. When you use a named database, you qualify the SQL statement with the database name. For example, you write EXEC SQL AT dbname UPDATE emp

We have a complementary feature in the xa open string to let the user associate the name with a particular DB instance, called the DB clause. You will also want to use the SqlNet clause in the open string so you can give the two different BIDs. This clause does not require the use of the SQLNet product, it is just a naming convention. For more information, see Oracle7 Server for UNIX Administrator's Reference Guide.

Some TP Monitors may not support having multiple Resource Managers in the same server; check with the TPM vendor.

Is there any collateral available for KA or TP Monitors?

Oracle At Work	52884.0882
Oracle7 Server for UNIX Administrator's Reference Guide	BA10324-1
Guide to Oracle's Products and Services	BA10580
Oracle7 Server and CICS/B000	BA14200

Where can I get more information on the RTP model?

K/Open's address is:

K/Open company Ltd (Publications)
P O Box 108
Parsippany
High Wycombe
 Bucks HP10 8NP

Tel: +44 (0)484 813844
Fax: +44 (0)484 814888

Request

6307 Distributed Transaction Processing: Reference Model Version 2
K/Open Guide 6307 ISBN 1-858120-18-8 28cm, 44p, pbk, 220g, 11/93

This file contains commonly asked questions about Oracle Server and UNIX Transaction Processing Monitors (TPMs). The topics covered in this article are

- o Performance with Oracle Server and TP monitors
- o Performance using Oracle's KA Library

The questions answered in part 4 provide additional detail to the information provided in part 1.

Performance with Oracle Server and TP Monitors

I have heard that Transaction Processing Monitors (TPMs) will increase Oracle Server performance. Is this true?

Several hardware and TPM vendors have made the claim that TPMs will increase RDBMS performance. This claim is based on TPC-A benchmarks. The key point to understand about TPC-A is that it requires, for every transaction-per-second, ten times that many users to be connected. For example, to get 800 TPS, you need 8000 users. The next question will answer in more detail how the three-tier architecture addresses this requirement, but first let's look more generally at what TP Monitors can and can't do to improve performance.

TP Monitors can provide better performance:

- (1) When there are more than several hundred users connected.

This is because of the TP Monitor's role in the three-tier architecture, described in the next question. In this architecture, terminal handling is offloaded to one or more separate machines, freeing up those cycles to do database work. Note that this does *NOT* mean that Oracle itself runs faster, just that we've given it more CPU cycles to use.

- (2) When, because of the high potential concurrency of requests, significant resource contention exists. Use of a TP Monitor can limit the degree of concurrency and thus reduce contention.

TP Monitors can not provide better performance:

- (1) For existing applications. The applications must be designed to fit the TP Monitor architecture.

- (2) For applications which are highly interactive in their use of the database. These applications put many messages through the transport system, and the TP Monitor is not as efficient as BQL/Met for point-to-point communication.

- (3) For CPU intensive single-query decision support. When executing a single large command, Oracle query facilities work efficiently, especially with the use of Oracle Parallel Query, available in 7.1.

How does the three-tier solution help TPC-A, or other situations with thousands of on-line users?

The TPC-A test calls for a large number of users to produce a given result. In the high-end results we produced in June, 1992, for example, 8150 terminals were simulated to produce 818 TPC-A transactions. Thus, terminal concentration accounts for a large portion of the total processing time used.

Clients = 8150	Terminal concentrators = 17
	TP Monitor instances = 17
Application server processes	Oracle Server processes
= 17*8	= 17*8

The TPM is the software component of the terminal concentrator. In this role it offloads terminal handling from the machine running Oracle Server. Since more than one terminal concentrator can be configured, whereas the database in this case had to run on a single machine, concentrator machines can be added until the performance of the back-end machine was optimized. This three-tier solution resulted in the outstanding transaction throughput associated with Oracle7 Server. Even with Oracle Parallel Server, it may pay to offload the terminal handling so that the cluster can be exclusively used for database operations.

Can you summarize the performance discussion for me?

Depending on the number of users required, different architectures may be used in a client/server environment to maximize performance:

1) For a small number of users, the traditional Oracle two-task architecture can be used. In this case, there is a one-to-one correspondence between client processes and server processes. It's simple, straightforward, and efficient.

2) For a large number of users, Multi Threaded Server might be a better approach. Although some tuning may be required, Multi Threaded Server can handle a relatively large number of users for each machine size compared to the traditional Oracle approach. Using this approach, customers will be able to handle many hundreds of users on any platform. Furthermore, current Oracle applications can move to this environment without change.

3) For a very large number of users, where transactions are simple and terminal input concentration is the overriding performance focus, a 3-tier architecture incorporating a TPM may be useful. In this case, terminal concentration is handled by the TPM in the middle tier. As you might expect, it is a more complex environment requiring more system management. For existing Oracle customers, significant Oracle application modifications will be required.

Oracle provides all of these choices.

Performance using Oracle's XA Library

=====

Are there any performance implications to using the XA library (in other words, to using TPM-managed transactions)?

(1) The XA library imposes some performance penalty. You should use TPM-managed transactions only if you actually need them. Even if you are getting the on-plate commit optimization, the code path is longer because we need to swap back and forth between external formats and

internal ones. Also, prior to 7.1, OB requires you to release all cursors at the end of a transaction, which results in extra parsing. Even with shared cursors, there is time spent looking up the one you need and revalidating it. This has been improved for 7.1.

(2) If you need to use two-phase commit, this will incur additional cost since extra I/Os are required. If you do need 2PC, you need to account for that when sizing the application.

(3) Although some TPMS allow parallel execution of services (such as Tuxedo's "tpscall"), this will not normally enhance performance unless different resource managers are being used. In fact, Oracle Server must coordinate accesses to the same transaction by the same Oracle instance, and the block/rowid code will in fact degrade performance in that case compared to running the services sequentially.

RECOVERY 의 모든 것 (DATABASE ,TABLESPACE, DATAFILE)

Bulletin no : 10928

(3) DATABASE RECOVERY

=====

- . Complete, incomplete recovery 가 가능
- . Offline recovery 만 가능하다 (즉 database가 recovery 시 open 되어 있지 말아야 함.)
- . SYSTEM tablespace 의 recovery 도 가능하다.
- . recovery 시 database access가 불가능하고 복구시간은 적용되어야 하는 redo 의 양에 dependent 하다.

Complete recovery

- 1 % cp backup_file oracle_file_directory
- 2 SQL> startup restrict mount
- 3 SQL> alter database rename file 'old' to 'new'
- 4 Recovery를 위한 모든 datafile 이 online 인지 check 한다.
SQL> select file#,status,name from V\$DATAFILE;
SQL> alter database datafile 'filename' online;
- 5 RECOVER DATABASE ;
- 6 alter database open;

Incomplete recovery

- (incomplete recovery 시 datafile 이 offline 되어 있으면 복구가 불가능하다)
- 5 RECOVER DATABASE UNTIL TIME '1998-10-10:10:37:00';
 - 6 ALTER DATABASE OPEN RESETLOGS;

(2) TABLESPACE RECOVERY

```
RECOVER TABLESPACE tablespacename [,tablespace_name...]
```

. 이 방법은 `tablespace` list 에 등록된 모든 `data file` 의 `media recovery` 를 위한 `command` 이다.

`Oracle` 은 `tablespace` 가 갖는 `datafile` 이 어떤것인가를 아는 것은 반드시 `database` 가 `open` 되어있을 경우이므로 이 `tablespace recovery command` 은 `database` 가 `open` 되었을때 비로소 가능하다.

. `command` 을 실행하기 위해 반드시 `tablespace` 는 `offline` 되어 있어야 한다.

. `offline` 되어지지 않는 (예를 들면 `BYTEXT`) `tablespace` 는 `recovery` 가 불가능하다.

. `incomplete recovery` 는 불가능하다 .

특구 순서

1 `SQL>>> alter tablespace tx_name offline;`

2 `recovery` 가 필요한 `tablespace` 의 `datafile` 들 `restore` 한다.

3 `lost` 되었거나 손상된 `datafile` 이 `invalid` 일경우는 `control file` 에 `datafile` 의 위치가 변경되어야 한다.

```
SQL>>> alter database rename file 'old' to 'new';
```

4 `SQL>>> RECOVER TABLESPACE tx_name;`

5 `alter tablespace tx_name online;`

(3) DATAFILE RECOVERY

```
RECOVER DATAFILE 'filename' ['filename']
```

. `media recovery lock` 만 `datafile` 에 얻을 수 있다면 `online,offline recovery` 가 모두 가능하다.

. `Online recovery` 시 (`database` 가 `instance` 의 `instance` 에 의해 `open` 되어 있다면) `datafile recovery` 는 `offline` 되어있는 `datafile` 만 `recovery` 가 가능하다.

. `incomplete recovery` 는 불가능하다.

복구순서

+++Online Recovery

- 1 SQL> startup mount
- 2 SQL> alter database datafile 'filenames' offline;
- 3 SQL> alter database open
- 4 SQL> alter database rename 'old' to 'new'
- 5 SQL> RECOVER DATAFILE 'datafile'
- 6 SQL> alter database datafile 'filenames' online

+++Offline Recovery

- 1 SQL> alter database rename 'old_filenames' to 'new'
- 2 SQL> startup restrict mount
- 3 SQL>) alter database datafile 'filenames' online
SQL>) alter database datafile 'filenames' offline
양쪽 모두 가능하다.
- 4 SQL> RECOVER DATAFILE 'datafile'
- 5 SQL> alter database datafile 'filenames' online;
- 6 SQL> alter database open

REDO LOGFILE SIZE의 변경 또는 추가

Bulletin no : 10282

* Oracle에서 사용중인 REDO Logfile은 U7에서는 Default로 Size가 512K인 3개의 REDO Logfile이 생긴다.

DB를 이용하는 Application의 목적에 따라 REDO Logfile의 수를 늘리거나 REDO Logfile의 Size를 크게 할 필요가 있다. REDO Logfile의 Size를 변경 하거나 수를 늘리는 것은 아래와 같은 방법을 이용한다.

- REDO Logfile의 Size 변경

(1) 먼저 ORACLE중인 DB는 Shutdown후에 Startup Mount로 기동 시킨다.

```
SQL> startup mount
ORACLE instance started.
Database mounted.
```

(2) V\$log과 V\$logfile을 이용해 그 것이 Current Log인지 또한 각 Logfile의 Datafile명인지를 확인한다.

```
SQL> select * from v$log;
```

GROUP#	THREAD#	SEQUENCE#	BYTES	MEMBERS	ARC	STATUS	FIRST_CHANGE#	FIRST_TIME
1	1	55	512000	1	NO	CURRENT	85081	2/08/95 18:32:57
2	1	53	512000	1	NO	INACTIVE	84111	1/21/95 11:08:14
3	1	54	512000	1	NO	INACTIVE	84801	1/21/95 11:41:44

3 rows selected.

```
SQL> select * from v$logfile;
```

GROUP#	STATUS	MEMBER
1		/usr4/ora713/dbs/log100A713.dbf
2		/usr4/ora713/dbs/log200A713.dbf
3	BYFILE	/usr4/ora713/dbs/log300A713.dbf

3 rows selected.

(3) V\$logfile에서 확인된 inactive Logfile을 먼저 아래의 Command를 이용해 Logfile을 Drop한다. 정상적으로 Drop되면 Hot Command를 이용해 Datafile을 Recover한다.

```
SQL> alter database drop logfile
```

```
        '/usr4/ora713/dhr/log200A713.dbf';  
Statement processed.
```

```
SQL> lrm /usr4/ora713/dhr/log200A713.dbf
```

(4) Decoin U7에서는 Default로 3개의 REDO Logfile이 생기는데 최소한 2개 이상의 REDO Logfile이 필요하다. 즉 위의 (3)과 같이 2번째 Logfile을 Drop 하였으므로 Logfile은 2개만 남아 있는데 1개를 또 Drop하고자 하니 아래와 같이 Ora-1587 & Ora-1532 Error가 발생한다. 이때는 다음과 같이 Logfile의 Size를 크게하여 이미 Drop된 2번 Logfile을 Add한 후 Drop하여야 한다.

```
SQL> alter database drop logfile  
        '/usr4/ora713/dhr/log300A713.dbf';  
ORA-01587:dropping log 3 would leave less than 2 log files in  
        thread 1  
ORA-00312:online log 3 thread 1: '/usr4/ora713/dhr/log300A713.dbf'
```

(5) Drop한 Logfile을 추가한 후에 Drop 한다. Logfile Add이 여기에서는 3으로 정의하여 크기를 조정하였다.

```
SQL> alter database add logfile  
        '/usr4/ora713/dhr/log200A713.dbf' size 3M;  
Statement processed.
```

```
SQL> alter database drop logfile  
        '/usr4/ora713/dhr/log300A713.dbf';  
Statement processed.
```

```
SQL> lrm /usr4/ora713/dhr/log300A713.dbf
```

(6) (5)에서 Drop된 3번 Logfile을 Add한다.

```
SQL> alter database add logfile  
        '/usr4/ora713/dhr/log300A713.dbf' size 3M;  
Statement processed.
```

(7) U8Log에서 Current Logfile은 바로 Drop할 수 없다. 이를 Drop하기 위해서는 현재의 Current Logfile을 Inactive로 하여야 한다. Inactive로 하려면 'Alter System Switch Logfile' Command를 이용하여야 하며 이는 DB가 Down되어 있어야 한다.

```
SQL> alter system switch logfile;  
ORA-01108: database not open  
  
SQL> alter database open;  
Statement processed.  
  
SQL> alter system switch logfile;  
Statement processed.
```

(B) 위의 (7)번 작업으로 Current Logfile를 inactive한 상태로 하였으며 이를 Drop하기 위하여DB를 다시 Alter Database Close를 이용해 DB를 Close 한다. Close후에는 (5)번, (8)번 작업과 같이 Logfile를 Drop하고 Datafile의 Remove후 Logfile의 Size를 변경하여 Add한다.

```
SQL> alter database close;
```

```
Statement processed.
```

```
SQL> alter database drop logfile
```

```
          '/usr4/ora713/dhr/log1000713.dbf';
```

```
Statement processed.
```

```
SQL> lsc /usr4/ora713/dhr/log1000713.dbf
```

```
SQL> alter database add logfile
```

```
          '/usr4/ora713/dhr/log1000713.dbf' size 3M;
```

```
Statement processed.
```

```
SQL> alter database add logfile
```

```
          '/usr4/ora713/dhr/log1000713.dbf' size 3M;
```

```
Statement processed.
```

필요한 Logfile의 Size를 모두 변경하고 나면 DB를 Normal Shutdown후 Startup하여 사용하면 된다.

(B) (1)에서 (B) 까지는 기존의 Logfile를 Drop후 새로이 만드는 것에 관한 설명이며 이외 기존에 3개 Logfile를 4개로 변경 할 경우에는 아래와 같이 Logfile를 필요한 갯수만큼 Add하면된다.

```
SQL> alter database add logfile
```

```
          '/usr4/ora713/dhr/log1000713.dbf' size 3M;
```

```
Statement processed.
```

ROLLBACK SEGMENT 관리

Bulletin no : 10088

롤백 세그먼트는 다른 세그먼트처럼 익스텐트로 구성되어 있다. 롤백 세그먼트는 이들 익스텐트를 Circular 하게 이동한다. 하나의 트랜잭션은 롤백 세그먼트의 Current Pointer 위치에 데포드를 기록하고 Current Pointer를 데포드 크기만큼 증가시킨다. 롤백 세그먼트에 데포드가 기록되는 현재 위치를 Head 라고 부르기로 한다. 그리고 롤백 세그먼트 상에서 가장 오래된 트랜잭션 데포드의 시작 위치를 Tail 이라고 부르기로 한다.

트랜잭션들이 필요로 할 때 마다 롤백 세그먼트를 사용할 수 있도록 적절한 갯수의 롤백 세그먼트를 유지해야 하며 각각의 롤백 세그먼트는 트랜잭션 부하를 감당하도록 충분한 크기를 가져야한다.

<롤백 세그먼트의 갯수와 크기>

롤백 세그먼트가 사용되는 방법은 다음과 같다.

1. 하나의 트랜잭션은 하나의 롤백 세그먼트 만을 사용한다.
2. 롤백 세그먼트의 Head는 Tail 에 의해서 사용되는 익스텐트를 식별하지 는다.
3. 팀 형태로 구성된 롤백 세그먼트 만에 더 이상 할당할 익스텐트가 없으면 새로운 익스텐트가 할당되어 팀 만에 포함된다.

따라서 롤백 세그먼트의 크기를 적절히 유지하는 것은 매우 중요하다.

롤백 세그먼트의 적절한 크기는 데이터베이스에서 수행되는 트랜잭션의 활동 상태에 따라서 달라진다. 필요한 롤백 세그먼트의 갯수는 현재의 롤백 세그먼트 사용 상태를 모니터 해서 결정한다.

롤백 세그먼트의 적절한 크기와 갯수를 결정하는 것은 다음 테스트과정들거쳐서 결정한다.

1. 롤백 세그먼트를 저장할 데이를 스페이스 생성
2. 테스트할 갯수의 롤백 세그먼트를 1)에서 만든 데일을 스페이스에 생성
3. 롤백 세그먼트의 익스텐트 크기는 모두 동일하게 설정. 스템트 크기는 세그먼트가 최대로 확장되었을 때 10~30 개 정도가 되도록 한다.
4. 롤백 세그먼트의 Maximum 은 2 이상.
5. 테스트할 롤백 세그먼트의 Status 를 In Use 상태로 변경하여 활성화 시킨다
6. 애플리케이션을 실행한다
7. 롤백 세그먼트 Contention을 측정한다.
8. 롤백 세그먼트의 최대 확장상태를 측정한다.

테스트 과정에서 가장 큰 상태가 될 때의 롤백 세그먼트 크기를 Minimum Coverage Size 라고 하자. 만약 Contention 이 발생하면 롤백 세그먼트의 갯수를 늘려가면서 테스트를 반복 한다.

<믹스펀트의 갯수와 크기>

물락 세그먼트의 믹스펀트 크기는 모두 같고 물락 세그먼트가 포함된 데이터 스페이스의 크기는 믹스펀트 크기의 정수배인 것이 바람직하다. 물락 세그먼트의 일반적인 크기는 20개 정도의 믹스펀트가 적합하다.

<오라클? 에서의 기능>

오라클?에서는 물락 세그먼트의 `rollback` 는 항상 0이다. `optimal` 값을 지정하면 오라클은 물락 세그먼트를 항상 이 크기 이하로 유지하려고 하게 된다. 이와같은 `Rollbacking`은 물락 세그먼트의 `Head` 가 하나의 믹스펀트에서 다음 믹스펀트로 어갈 때 일어난다. 오라클은 가장 오래된 비활성 상태의 믹스펀트부터 제거하여 `Rollback` 한다.

물락 세그먼트 모니터에서 물락 세그먼트에 할당되었던 최대 믹스펀트와 `Optimal`, `Rollback` 횟수 등이 나타나는데 여기서 적절한 `Optimal` 값을 설정 할 수 있다.

오라클?에서는 용적으로 물락 세그먼트의 `Online/Offline`이 가능하다. 물락 세그먼트가 처음 생성되었을 때는 `Offline` 상태이므로 이것을 `Online` 상태로 만들려면 `Alter Rollback Segment` 명령을 `Online` 옵션으로 사용하거나 `init.ora` 파일에 해당 물락 세그먼트를 등록하고 데이터베이스를 `Restartup` 하면 된다. 반대로 `Alter Rollback Segment` 명령을 이용하여 `Offline` 시킬 수도 있다. `Offline` 된 물락 세그먼트를 `Online` 시키려면 `Alter Rollback Segment` 를 이용하거나 `init.ora` 의 `rollback_segments` 에 지정을 하고 데이터베이스를 `Restartup` 해야 한다.

<물락 데이터 스페이스의 디자인>

프랜잭션에 의해서 발생되는 물락 정보의 크기는 다음의 스크립트를 이용하여 추정할 수 있다.

```
set feedback off
set timeout off
column name format A40
define undo_overhead=54
DROP TABLE undo$log1;
DROP TABLE undo$und1;
CREATE TABLE undo$log1 ( writec number );
CREATE TABLE undo$und1 ( writec number );
INSERT INTO undo$log1
      SELECT sum(writec) FROM v$logstat;
set timeout on
set feedback on
UPDATE test1 SET col1=99 WHERE col3 = 'Y';

set timeout off
set feedback off
INSERT INTO undo$und1
      SELECT sum(writec) FROM v$logstat;
set timeout on
```

```

set feedback on
SELECT ((u.writes - b.writes) - undo_overhead)
" number of bytes generated"
FROM undo$angle b, undo$end u;
set timeout off
set feedback off
DROP TABLE undo$angle;
DROP TABLE undo$end;

```

위의 스크립트가 출력하는 결과는 트랜잭션 동안 발생하는 UNDO 이다.
따라서 데이터베이스에 다른 트랜잭션이 없다면 이 값은 정확한 값을 보여준다.

아래의 스크립트는 활성화된 트랜잭션을 실행하는 유저와 각 트랜잭션이 사용중인 블록 세그먼트를 보여준다.

```

SELECT p.name "ROLLBACK SEGMENT NAME",
       l.pid "ORACLE PID",
       u.pid "BYTETH PID",
       NVL ( p.usrname , 'NO TRANSACTION'),
       p.termint
FROM v$lock l, v$process p, v$rollname r
WHERE l.pid = p.pid(+)
AND TRUNC (l.lid(+)/85536) = r.usrn
AND l.type(+) = 'TX'
AND l.mode(+) = B
ORDER BY p.name

```

여러가지 경우에 대해서 알아보자.

1. 보통의 트랜잭션인 경우

블록 데이터 스페이스의 크기는 앞에서 접점한 블록 세그먼트들이 Minimum Coverage Size인 경우와 거의 같은 크기로 접점한다. 블록 세그먼트를 생성 할 때 필요한 크기를 모두 할당 받도록 한다. 따라서 블록 세그먼트를 만들고 나면 데이터 스페이스에 더 이상의 빈공간은 남지 않게 된다.

2. 매우 크고 자주 발생하는 트랜잭션인 경우

이 경우에는 다른 블록 세그먼트들보다 큰 블록 세그먼트를 하나 만들어서 큰 트랜잭션이 사용하도록 한다. 예를 들어 가장 큰 트랜잭션의 크기가 45M 이고 minimum coverage size 가 10M 라면 큰 블록 세그먼트의 크기는 55M 가 된다. 다른 블록 세그먼트들은 Minimum Coverage Size 로 생성된다. 큰 블록 세그먼트를 사용하고자 할 때에는 Bat Transaction Use Rollback Segment 을 사용하여 지점한다. 만약 큰 트랜잭션이 여러개가 동시에 발생하면 큰 블록 세그먼트도 여러개가 필요하다.

3. 매우 크지만 자주 발생하지는 않는 프랜잭션인 경우

이 경우에는 큰 프랜잭션을 위한 큰 블록 세그먼트를 하나 만들지 필요한 경우에만 만들어서 사용하고 사용이 끝나면 삭제하는 방법을 사용한다. 아니면 아예 정도의 블록 데이터를 스페이스와 큰 블록 세그먼트를 만들어서 작업중에 사용하고 작업이 끝나면 삭제하여도 된다.

실제로는 위와 3가지 경우가 복합되어 발생하므로 상황에 따른 적절한 대응이 필요하다.

<오라클?>

오라클?에서는 여타가 기능이 추가되어서 블록 세그먼트의 설계가 달라진다. Optimal 값을 설정하면 좋은 점도 있지만 그 값이 너무 작다면 익스텐트의 할당과 Reallocation 이 빈번히 발생하여 퍼포먼스의 저하를 가져올 수도 있다. 그리고 현재 블록 세그먼트가 Optimal 크기로 돌아 지을 수 없다는 점도 문제이다.

1. 보통의 프랜잭션인 경우

UB 의 경우와 같다. 단지 데이터를 스페이스에 여유공간을 쓸 경우 Optimal 을 설정하여 초기에 설정한 값을 유지하도록 한다. Optimal 값은 Minimum Coverage Size 보다 작게 수면 퍼포먼스가 저하되므로 유지하도록 한다.

2. 매우 크고 자주 발생하는 프랜잭션인 경우

여기서 자주 발생한다는 것은 모든 블록 세그먼트가 optimal 크기로 돌아가기 전에 또다시 큰 프랜잭션이 발생한다는 의미이다. (큰 프랜잭션이란 이것이 발생시키는 블록 정보를 담은 블록 세그먼트를 만들 수 없는 것을 말한다. 이 경우에는 Optimal 은 해결방법이 되지 못한다. 해결방법으로는 우선 블록 세그먼트의 갯수를 줄여서 각각의 블록 세그먼트가 충분히 커질 수 있도록 하는 것이다. 하지만 이렇게 하면 Contention 이 일어 서 퍼포먼스가 저하될 수 있다. 다른 방법은 큰 블록 세그먼트를 만들어서 Bat Transaction Use Rollback Segment 경험으로 해당 프랜잭션이 이것을 지정하여 사용하도록 하는 방법이다. 하지만 이 방법은 애플리케이션에서 블록 세그먼트를 지정하는 경험을 사용할 수 있어야만 사용 가능하다.

3. 매우 크지만 자주 발생하지 않는 프랜잭션인 경우

블록 데이터를 스페이스를 블록 세그먼트들의 Minimum Coverage Size를 큰 프랜잭션의 정보를 담을 만한 크기로 만든다. 그리고 모든 블록 세그먼트의 Optimal 을 Minimum Coverage Size 로 설정한다. 그렇게 하면 큰 프랜잭션이 어느 블록 세그먼트에 할당되어도 완전한 처리가 가능해진다.

다음의 큰 프랜잭션이 들어올 때 쯤이면 optimal 설정에 의해서 필요한 공간을 항상 보유하고 있게 된다. 하지만 optimal 설정에 따른 약간 퍼포먼스 저하가 발생할 수도 있다.

ROLLBACK SEGMENT 크기를 증가하여 재생성 하는 방법

Bulletin no : 10072

ROLLBACK SEGMENT 는 LONG TRANSACTION 을 수행하면 프게되는 경우가 있다. 확장된 ROLLBACK SEGMENT 는 줄어들지 않게되므로(OPTIMAL을 지정하지 않는 경우) 주기적으로 재생성 하여 TABLESPACE 의 FREESPACE 영역을 확보한다.

1. ROLLBACK 기동한다.

```
$ sqlha loadwp
sqlha > connect internal
```

2. ROLLBACK SEGMENT 을 OFFLINE 시킨다. ROLLBACK SEGMENT 을 OFFLINE 시키기 위해서는 그 ROLLBACK SEGMENT 가 INACTIVE상태여야 하므로 모든 TRANSACTION 을 종료 하고 ROLLBACK SEGMENT #01~#04를 OFFLINE 시켜야 한다.

```
sqlha > alter rollback segment #01 offline;
sqlha > alter rollback segment #02 offline;
** #03 ~ #04 도 같은 방법으로 OFFLINE 함.
```

3. ROLLBACK SEGMENT 을 DROP 한다.

```
SQLHA > drop rollback segment #01;
SQLHA > drop rollback segment #02;
** #03 ~ #04 도 같은 방법으로 DROP 함.
```

4. ROLLBACK SEGMENT 을 생성한다. SYSTEM ROLLBACK 이외에 모든 ROLLBACK SEGMENT 을 DROP 한경우는 SYSTEM TABLESPACE 에 ROLLBACK SEGMENT 을 한개 이상 생성후에 NON-SYSTEM TABLESPACE 에 ROLLBACK SEGMENT 을 생성해야한다.

```
SQLHA > create rollback segment #0;
SQLHA > alter rollback segment #0 online;

SQLHA > create rollback segment #01 tablespace rbs
storage(initial 512k next 512k minextents 2);
SQLHA > create rollback segment #02 tablespace rbs
storage(initial 512k next 512k minextents 2);
** #03 ~ #04 도 같은 방법으로 생성 함.
```

5. 데이터베이스를 재 오픈 한다.

```
SQLDBA> shutdown
SQLDBA> startup
```

DB TABLESPACE를 재구성 후 ROLLBACK SEGMENT를 생성하는 방법

6. DB TABLESPACE를 DROP 하기 위해서는 1-2 과 같이 ROLLBACK SEGMENT를 전부 OFFLINE을 한다.

ROLLBACK SEGMENT를 OFFLINE할수 있는 다른 방법은

ORACLE_HOME/dbs/init.ora 파일의 rollback_segments=r01..

부분을 R으로 COMMENT 처리하고 DB를 다시 startup 시키면 된다.

7. DB TABLESPACE를 DROP 한다.

```
SQLDBA> drop tablespace DBS including contents;
SQLDBA> ! rm /oracle/dbs/rbs.dbf
```

8. DB TABLESPACE를 다시 생성한다.

```
SQLDBA> create tablespace DBS datafile '/.. /oracle/dbs/rbs001.dbf'
size 50M default storage(initial 512k next 512k);
```

9. ROLLBACK SEGMENT r0, r01-r04를 생성 한다.

```
SQLDBA> create rollback segment r0;
SQLDBA> alter rollback segment r0 online;
```

```
SQLDBA> create rollback segment r01 tablespace DBS;
-- r02 - r04 도 같은 방법으로 생성한다.
```

10. 1 에서 COMMENT 처리했던 부분들 다음과 같이 줄고 DB를 다시 startup 한다.

```
rollback_segments=r01,r02,r03,r04
```

```
SQLDBA> shutdown
SQLDBA> startup
```

ORA-1532 조치 방법

DB TABLESPACE 용량이 부족한경우 8 번 과 같이 TABLESPACE를 더큰 크기로 다시 생성 할수 있지만 DATAFILE을 추가하여 용량을 확장하는 방법이 있다.

11. DB TABLESPACE 용량 확장

```
SQLDBA> alter tablespace DBS add datafile '/oracle/dbs/rbs2000.dbf'
size 50M;
```

ROLLBACK SEGMENT의 상태 해결

Bulletin no : 10094

* 롤백세그먼트의 정보를 보여주는 `dba_rollback_seg`를 보면 `Status` 컬럼이 있다. 일반적으로 이 컬럼은 `Online/Offline` 값을 갖지만 롤백 세그먼트가 손상된 경우에는 `Media Recovery` 값을 갖게 된다.

* `Media Recovery` 상태의 롤백 세그먼트는 `Online/Offline` 으로 변경하거나 `Init.ora` 파일의 `Rollback_segment` 다민에서 제거하는 것만으로는 문제가 해결되지 않는다. `Media Recovery` 상태가 생기는 이유는 롤백 과정에서 어떤 이유에서든 문제가 생겨서 실패하기 때문이다. 롤백은 사용자가 `Rollback` 명령을 내린 경우와 `Shutdown Abort` 후 `Startup` 시에 자동적으로 수행되는 `Instance Recovery` 시에 일어난다.

* 롤백이 일어나면 수정된 데이터 블록에 기록된 결과는 취소되어야 하지만 해당 데이터 블록에 접근이 불가능하여 롤백 작업을 하지 못하면 롤백 세그먼트의 상태가 `Media Recovery` 상태로 되는 것이다.

* 데이터 블록에 접근이 불가능한 경우로는 다음과 같은 예가 있다.

1. 데이터 스페이스나 데이터 파일이 `Offline` 이거나 없는 경우
2. 해당 데이터 블록이 속한 `Object`가 손상된 경우
3. 롤백 세그먼트의 데이터 블록이 손상된 경우

* 이 문제를 해결하는 방법은 다음과 같다.

1. 모든 데이터 스페이스와 데이터 파일이 `online` 상태인지를 확인한다.

이것은 `DBA_DATA_FILES` 와 `DBA_TABLESPACES` 의 `STATUS` 컬럼을 조회하면 알 수 있다.

2. 다음 문장을 `init.ora` 파일에 추가한다.

```
event = "10015 trace name context forever, level 10"
```

위의 `Event`는 롤백하는 프랜잭션과 `Object`에 관한 정보를 담은 `Trace File`를 생성시켜 준다.

3. 손상된 롤백 세그먼트를 `init.ora` 의 `Rollback_segment` 다민에서 삭제한다.

4. 데이터스페이스를 `Shutdown` (가능한 `Normal`, `Immediate` 를 사용하고, 만질 경우 `Abort` 를 사용하고 다시 `Startup` 한다.

5. `USER_DUMP_DEST`에 지정된 디렉토리에 `Trace` 파일이 생성되었는지 확인한다.

B. `Trace` 파일을 보면 `Error Recovery tx(B,B) Object B` 와 같은 메시지가 있을 것이다. 여기서 `tx(B,B)`은 프랜잭션의 정보를 가지고 `Object B` 는 `DBA_OBJECTS` 의 `OBJECT_ID` 와 같다.

7. 다음의 `Query`를 이용하여 어떠한 `Object`에 대해서 `Recovery` 가 수행되는가를 확인한다.

```
select owner, object_name, object_type, status
from dba_objects
where object_id=;
```

B. 출력된 Object를 Drop 한다.

B. init.ora 에서 삭제했던 플럭 세그먼트를 다시 Rollback_segment 라면에 넣고 Event를 삭제한 다음 데이터베이스를 Shutdown 하고 Startup 한다.

ROW CHAINING 과 ROW MIGRATION

Bulletin no : 10381

Row chaining 은 단일 레이블 상의 특정 Row의 길이가 증가해서 더 이상 동일한 레이블 블록에 들어갈수 없을때 발생한다. 이때 DBMS는 또 다른 레이블 블록을 찾는다. 이 레이블 블록은 원래 블록 연결되어 있다. 이 경우 레이블 블록이 하나의 I/O 작업과 동일한 일을 수행하기 위해 두 개의 I/O를 사용해야 한다는 점이다. 이 상황은 여러분의 데이터베이스 성능을 빠르게 악화시킬 것이다.

Data Block상의 하나의 Row는 길이가 증가하면서 갱신되며, Block의 Free-space가 0%일 때, Row는 Migration을 일으킨다. 물론, 전체 Row가 들어갈 만한 크기의 새로운 Block에 Row에 대한 Data가 Migration된다.

이 경우 DBMS는 Row에 대한 정보를 가져오기 위해 하나 이상의 Data Block를 반드시 읽어야 하므로 I/O Performance는 감소한다.

<해결안>

1) ANALYZE Command를 통해 Chaining과 Migrating의 횟수를 조사한다.

```
analyze table emp list chained rows;
```

1. 데이터 열을 CHAINED_ROWS 테이블의 ROWID를 사용하여 원래 데이터와 같은 행 구조를 가진 중간 테이블(intermediate table)로 이동시킨다.
2. 옮겨진 데이터 열을 CHAINED_ROWS 테이블의 ROWID를 사용하여 삭제 한다.
3. 중간 테이블로부터 열들을 다시 원래 데이터로 삽입한다.
4. 중간 테이블을 버린다.
5. CHAINED_ROWS 테이블의 레코드를 삭제한다.

이 과정이 수행되고 나면 analyze 명령은 다시 수행되어야 한다. row가 다시 CHAINED_ROWS 테이블에 쌓이면 어떤 블록에도 전체row 가 들어갈 충분한 공간이 없기 때문이다. 이것은 한 레이블 블록의 한 row 가 너무 길어서 이거나 데이터의 PCTFREE 가 적절하지 못하기 때문이다. 전자의 경우는 chaining 현상이 일어날수 밖에 없고 후자의 경우 다음과 같이 PCTFREE 를 수정한다.

1. 데이터에 대한 더 나은 퍼센트 프리 요소(percent free factor)를 결정한다.
2. 전체 데이터를 그 모든 의존관계(예를 들면, 인덱스, 그랜트(grant), 약조건들 등)과 함께 export한다.
3. 원래 데이터를 버린다.
4. 새로운 사양으로 다시 만든다.
5. 데이터를 import한다.

ROWID에 대한 이해

Bulletin no : 10308

Fact and Dangerous

The Misunderstood Oracle ROWID By Bert Bosman, Ph.D.

Oracle ROWID는 중요하고 powerful한 SQL programming tool이다.

그러나 언제, 어떻게 안전하게 사용되어야 하는지에 대해 다소 잘못된 이해된 점들이 있다.

정상적으로 올바른 환경에서의 ROWID의 사용은 SQL이나 PL/SQL CODE의 실행을 보다 빠르게 하여 준다.

그러나 주의할 것은 ROWID는 당신의 PROCS를 독립하게 동작하게하거나 아니면, 전혀 엉뚱한 동작을 하게 만들수 있다는 것이다.

그러므로 ROWID의 사용은 독립한 주의를 기울여야 한다.

Just What Is It Exactly ?

Oracle ROWID는 무엇인가 ? Oracle table의 모든 Row는 ROWID라고 하는 물리적인 Address로 assign 된다.

Non-clustered table 에 대해서는 이러한 address는 unique하다.

Clustered table에 대해서는 서로다른 table에서 추출된 row들은 동일한 data block내에 존재하며, 정확히 같은 ROWID를 가지게 될것이다.

누군가가 ROWID를 사용하여 한 table만의 서로 다른 column에 대하여 검색 하고, 제한하는 기능들을 사용한다하더라도 이것은 pseudo-column이다.Oracle의 pseudo column은 table의 실제적인 column은 아니므로 table의 describe를 통해서도 보이지 않지만 매우 유용하게 사용된다.

Pseudo column의 다른 예는 CUMVAL, NEXTVAL (sequence number generator)과 LEVEL (for hierarchical queries), ROWNUM (a row's relative number) 같은 것들이 있다.

내부적으로 ROWID는 binary 값을 가진다. 그러나 외부적으로 Display 될때에는 VARCHAR2 type의 column에 저장된 3 part의 hexadecimal string으로 표현된다.

첫번째 부분(position 1 through 8) 은 block ID를 나타낸다.

두번째 부분(position 10 through 13) 은 block 내부에서 row의 sequence number를 나타낸다.

세번째 부분(position 15 through 18) 은 data file ID를 나타낸다.

BINARY function을 사용하여 ROWID를 구분하여 SELECT하는 구문을 보면,

SELECT ROWID

```
      SUBSTR(ROWID, 15, 4) AS "FILE",
      SUBSTR(ROWID, 1, 8) AS "BLOCK",
      SUBSTR(ROWID, 10, 4) AS "ROW"
FROM dual;
```

위의 SELECT 구문은 다음과 같은 ROWID에 대하여 아래의 결과를 출력할 것이다.

```
ROWIDFILEBLOCKROW
-----
00000339.0000.00010001000003390000

ROWID          00000339.0000.0001
FILE           0001
BLOCK          00000339
ROW            0000
```

위의 예에서 보듯이 ROW의 값은 BLOCK 내에서의 첫번째 ROW이다. (BLOCK 내에서의 첫번째 ROW의 OFFSET은 ZERO이기 때문.) 더 나아가서 이 ROW는 한 DATA FILE내의 828 BLOCK 내에 위치하고 있다. 이것이 이 ROW의 PHYSICAL ADDRESS 이다.

It's Fact!

ROWID를 사용하여 table의 row를 access하는 것은 빠르다. 사실상 rule-based optimizer에서는 ROWID를 이용하여 access하는 것이 가장 상위로 rank 된 access 방법이다.

Oracle Application Developer Guide에 따르면 access path 선택의 rule은 Figure 1에 기술된 순서와 같다.

NAME	ACCESS PATH
1	Single row by ROWID
2	Single row by cluster join
3	Single row by hash cluster key with unique or primary key
4	Single row by unique or primary key
5	Cluster join
6	Hash cluster key
7	Indexed cluster key
8	Composite key
9	Single-column indexes
10	Bounded range search on indexed column
11	Unbounded range search on indexed column
12	Sort-merge join
13	MAX or MIN of indexed column

```

14          ORDER BY an indexed column
15          Full table scan

```

Figure 1 : The rules for access path selection.

보신 바와 같이 가장 첫번째의 (빠른) access 방법은 ROWID를 사용하는 것이다. 이러한 access 방법은 아래의 사항이 만족되어질때에만 사용가능하다.

1. WHERE 절에서 ROWID로 row를 선택할수 있도록 기술되거나
2. CLUSTERED BY CLUSTER 접근 기술되어야 한다.

위의 사항에 해당한다면 Oracle Server는 해당하는 physical address를 가지고 data row를 access 할 것이다. 결과는 가장 빠르게 생성될 것이다. 그러나 빠른 access 가 가능한 만큼 허점도 가지고 있는데, ROWID가 변화할 수 있다는 것이 그것이다.

Oh... Oh... Changes

row의 address는 서로 다른 환경하에서는 변화할수 있다. 예를들면, Import 직후의 export는 일관적으로 서로 다른 ROWID를 생성한다. 그러 이 예는 전혀 해가 없는 예이다. 왜냐하면 ROWID가 user의 transaction의 영역 밖에서 일어났기 때문이며, 이러한 변화는 전혀 고려할 만한 사항이 아니다.

반면에 어떤 user가 DDL 작업을 하는 도중에 ROWID가 변화 하였다고 생각해보자.

만약 user의 program이 전체적인 범위에서 ROWID의 consistency가 유지되어야 한다고 가정해보면, 이것은 정말 큰 문제가 될 것이다.

ROWID는 단지 transaction의 length 동안만 consistency의 유지가 보장된다.

한 transaction은 commit 되는 순간부터 commit이나 rollback을 만나거나, 또 다른 transaction이 시작되는 시간까지 유지된다. 이러한 이유 때문에 user program 내부에 하나 이상의 transaction이 존재한다면, ROWID가 변화할수도 있다.

An Example

이러한 문제의 예로는 Figure 2. 에서 보이는 transfer_all procedure를 들 수 있다.

```

PROCEDURE transfer_all
(
  commit_after_in IN INTEGER := 100,
  query_after_in IN INTEGER := 1000)
IS
  more_data BOOLEAN := TRUE;
  commit_count INTEGER := 0;

  CURSOR trans_cur
  IS
    SELECT ROWID FROM transfer_table
    WHERE transfer_indicator = 'U'

```



```

        AND BOUNUM <= query_after_in;

BEGIN
    WHILE more_data
    LOOP
        more_data := FALSE;
        FOR trans_rec IN trans_cur
        LOOP
            more_data := TRUE;

            INSERT INTO master_table
            SELECT *
            FROM    transfer_table
            WHERE ROWID = trans_rec.ROWID;

            UPDATE transfer_table
            SET      transfer_indicator = 'Y'
            WHERE   ROWID= trans_rec.ROWID;

            commit_count := commit_count + 1;
            IF commit_count = commit_after_in
            THEN
                COMMIT;
                commit_count := 0;
            END IF;
        END LOOP;
    END LOOP;
END;

```

위의 Procedure는 몇가지 제한조건을 가지고 ROWID를 select 하여 TRANS_CUR 라는 CURSOR를 생성한다. 이 Cursor로 Return 되는 Row는 For Loop 안에서 사용된다. 이 Loop 내에서는 current cursor record의 ROWID를 reference 하는 INSERT 문장과 UPDATE SQL command 가 사용되고 있다. 또한 이 cursor For loop 안에서는 argument로 넘겨받은 interval 연수에 따라 Commit 문이 실행된다. Commit 문이 실행되면, cursor에 의해 사용되던 ROWID는 전파할수 있게 된다.

이러한 상황이 single-user의 환경에서 발생하는 것이라면 아무런 문제가 없으나, Multi-user의 concurrent transaction 환경이라면 많은 문제가 발생할것이다.

- . 다른 user 가 당신이 사용하고 있는 ROWID를 invalidate 하게 하는 UPDATE나 INSERT, DELETE 작업을 수행할수도 있다.
- . 또 한가지는 INSERT나 INSERT를 줄이는 UPDATE 작업 등에서 내부적으로 automatic 하게 ROWID 를 변경할수도 있다.

이러한 문제는 잘 드러나지 않을 뿐만 아니라 Error를 detect 하기도 힘들뿐만 아니라, 거의 Debug가 불가능한다.

Conclusion

`Rowid`는 엄격한 제한조건을 가지고 주의깊게 사용되어야 한다. 저자의 생각으로는 작업되는 `row`에 `locking`을 하지 않고는 `ROWID`를 사용하는 `sql statement`를 사용하여서는 안된다.

`Oracle 7 Server Concept Manual`에서 언급된 사항은 아래와 같다.

“ `SQL` 문장에서 사용되는 `ROWID`는 그것이 변화되지 않을 것이라는 사실이 검증되고, 보장되어야 한다. 이렇게 변화하지 않기 위해서는 사용되는 `Row`는 `locked`되어야 한다.”

`Oracle Performance Tuning` (Corrigan and Curry, O'Reilly & Associates, 1993)에서는 아래와 같이 언급하고 있다.

“ 당신이 `Record`를 `Query` 하거나, `update`하기 위하여 `query`를 수행할 때, 선택된 `record`가 다른 `process`로 부러 보존되고, `record`의 `rowid`는 당신의 환경 밖에서 변화되어야 한다.”

`Transfer_all` procedure의 경우 `error`를 `fix` 하기 위하여 2가지가 변화 (또는 추가)되어야 한다.

첫번째로는 `cursor`의 선언 부분에

```
“ FOR UPDATE OF ROWID “ 가 추가 되어야 하며,
```

두번째로는 `FOR loop`의 랑으로 `COMMIT statement`를 옮겨주어야 한다.

이러한 변화가 `program`을 다소 `non-flexible`하게 만드는 것처럼 보일지도 모르지만, 확실하게 `program`을 `reliable` 하게 만들어 준다.

IMPLEMENTING DATA REPLICATION

Bulletin no : 10375

Here's how to move beyond the basics to implement an update-anywhere symmetric-replication system, including a sample macro you can download.

Read this article as it originally appeared in the July/August 1998 issue of Oracle Magazine.

Listing 1: A sample Microsoft Word macro you can use to build an Oracle symmetric-replication environment (reprinted with permission from Automated Learning).

```
Dim Shared surrogateid$
  REM USED ID FOR SURROGATE BYE USED
Dim Shared surrogatepassword$
  REM PASSWORD FOR SURROGATE BYE USED
Dim Shared repadid$
  REM USED ID FOR REPADMIN USER
Dim Shared repadpassword$
  REM PASSWORD FOR REPADMIN USER
Dim Shared schemas
  REM NUMBER OF REPLICATED SCHEMAS
Dim Shared schemasid$(2)
  REM ARRAY OF REPLICATED SCHEMA IDS
Dim Shared schemaspassword$(2)
  REM ARRAY OF REPLICATED SCHEMA PASSWORDS
Dim Shared masterdefname$
  REM MASTER DEF SITE NAME
Dim Shared masterdefpassword$
  REM MASTER DEF SITE BYE PASSWORD
Dim Shared mastername$
  REM MASTER SITE NAME
Dim Shared masterpassword$
  REM MASTER SITE BYE PASSWORD
Dim Shared snapshot
  REM NUMBER OF SNAPSHOT BYTES
Dim Shared snapshotname$(8)
  REM ARRAY OF SNAPSHOT SITE NAMES
Dim Shared snapshotpassword$(8)
  REM ARRAY OF SNAPSHOT SITE BYE PASSWORDS
Dim Shared snapshotmaster$(8)
  REM ARRAY OF SNAPSHOT MASTER SITES
```

BuA MAIN

BEN BUILDS A SQL COMMAND SCRIPT YOU CAN USE TO GENERATE AN
BEN ORACLE SYMMETRIC REPLICATION ENVIRONMENT. THE SCRIPT
BEN IS OUTPUT TO THE CURRENT OPEN MICROSOFT WORD DOCUMENT.

BEN GET MASTER DEFINITION DATABASE NAME

```
masterdefname$ = InputBox$("Enter the master definition site name " + "  
"(e.g ., SALES,ACME.COM):")
```

BEN GET BYB PASSWORD FOR MASTER DEFINITION SITE

```
masterdefpassword$ = InputBox$("Enter BYB password at master definition site:")
```

BEN GET INFORMATION ABOUT MASTER SITE

```
button1 = MsgBox("Create one other master site?", 282)  
If button1 = - 1 Then  
    mastername$ = InputBox$("Enter the master site name (e.g ., SALES,ACME.COM):")  
    masterpassword$ = InputBox$("Enter corresponding BYB password for master site:")  
End If
```

BEN GET INFORMATION ABOUT SNAPSHOT SITES

```
button2 = MsgBox("Create snapshot sites?", 282)  
If button2 = - 1 Then  
    snapshot = Val(InputBox$("Enter the number of snapshot sites to create " + "  
"(max = 10):")  
    For count = 0 To snapshot - 1  
        snapshotname$(count) = InputBox$("Enter the snapshot site name (e.g., SALES,ACME.COM):")  
        snapshotpassword$(count) = InputBox$("Enter corresponding BYB password for snapshot site:")  
        snapshotmaster$(count) = InputBox$("Enter corresponding master site of snapshot site " + "  
"(e.g ., SALES,ACME .COM):", masterdefname$)  
    Next  
End If
```

BEN GET SURROGATE BYB USER ID

```
surrogateid$ = InputBox$("Enter a name for the surrogate BYB account at " + "  
"all master sites:", "password")
```

```

REM GET SURROGATE BYB PASSWORD
surrogatepassword$ = InputBox$("
Enter a password for the surrogate BYB account:")

REM GET REPLICATION ADMINISTRATOR NAME
repadminid$ = InputBox$("
Enter a name for the replication administrator " + \
"account:", "repadmin")

REM GET REPLICATION ADMINISTRATOR PASSWORD
repadminpassword$ = InputBox$("
Enter a password for the replication " + \
"administrator account:")

REM PROMPT FOR NUMBER OF REPLICATED SCHEMAS
schemas$ = Val(InputBox$("
How many replicated schemas do you want to create " + \
"(maximum = 3)?:")

REM GET REPLICATED SCHEMA INFORMATION
For count = 0 To schemas - 1
    schemaid$(count) = InputBox$("Enter replicated " + \
    "schema name:")
    schemapassword$(count) = InputBox$("
Enter corresponding replicated schema password:")
Next

REM BUILD A CORRESPONDING SQL COMMAND SCRIPT IN CURRENT
REM OPEN DOCUMENT

REM CREATE ACCOUNTS, SCHEMAS, AND LINES AT MASTER
REM DEFINITION SITE
Insert "REM MASTER DEFINITION SITE BUILD"
InsertPara
CreateAccounts(masterdefname$, masterdefsympassword$)
CreateSchemas(masterdefname$, masterdefsympassword$)
If button1 = - 1 Then
    CreateLinks(masterdefname$, mastername$, 1)
End If

REM CREATE ACCOUNTS, SCHEMAS, AND LINES AT MASTER SITE
If button1 = - 1 Then
    Insert "REM MASTER SITE BUILD"
    InsertPara
    CreateAccounts(mastername$, mastersympassword$)
    CreateSchemas(mastername$, mastersympassword$)

```

```

CreateIndex(mastername$, masterdbname$, 1)
End If

```

BEN CREATE ACCOUNTS, SCHEMAS, AND LINES AT SNAPSHOT SITE

```

If button2 = - 1 Then
    Insert "BEN SNAPSHOT SITE BUILD(B)"
    InsertPara
    For count = 0 To snapshot - 1
        CreateSchema(snapshotname$(count), \
            snapshotpassword$(count))
        CreateIndex(snapshotname$(count), snapshotdbname$(count), 0)
    Next
End If
End Sub

```

```

Sub CreateAccount(site$, snapshot$)

```

BEN SUBROUTINE TO CREATE ACCOUNTS AT GIVEN SITE

```

Insert "BEN Create accounts at master site " + site$
InsertPara
Insert "BEN Connect as BYB"
InsertPara
Insert "CONNECT user/" + snapshot$ + "D" + site$ + ";"
InsertPara
InsertPara
Insert "BEN Create surrogate BYB account"
InsertPara
Insert "CREATE USER " + surrogateid$ + \
    " IDENTIFIED BY " + surrogatepassword$ + ";"
InsertPara
Insert "EXECUTE dbac_repcat_auth.grant_surrogate_repcat" + \
    "(" + surrogateid$ + ")";"
InsertPara
InsertPara
Insert "BEN Create replication administration account"
InsertPara
Insert "CREATE USER " + repadminid$ + " IDENTIFIED BY " + \
    repadminpassword$ + ";"
InsertPara
Insert "EXECUTE dbac_repcat_admin.grant_admin_repadmin" + \
    "(" + repadminid$ + ")";"
InsertPara
InsertPara
End Sub

```

```

Sub CreateSchema(site$, snapshot$)

```

BEN SUBROUTINE TO CREATE REPLICATED SCHEMAS AT GIVEN SITE

```

Insert "DBN Create replicated schemas at " + site$
InsertPara
Insert "DBN Connect as BYB"
InsertPara
Insert "CONNECT sys/" + syspassword$ + "D" + site$ + ";"
InsertPara
InsertPara
For count = 0 To schemas - 1
    Insert "CREATE USER " + schemaId$(count) + \
        " IDENTIFIED BY " + schemapassword$(count) + ";"
    InsertPara
    Insert "EXECUTE dbms_repcon_admin.grant_admin_privileges" + \
        "(userid =>' " + schemaId$(count) + "')";"
    InsertPara
    Insert "GRANT EXECUTE ON dbms_defer to " + \
        schemaId$(count) + ";"
    InsertPara
Next
InsertPara
End Sub

```

Sub CreateLinks(site1\$, site2\$, master)
 DBN SUBROUTINE TO CREATE NECESSARY DATABASE LINKS FROM
 DBN SITE1 TO SITE2

```

If master = 1 Then
    Insert "DBN Create master-to-master database links at " + \
        site1$
    InsertPara
    Insert "DBN Create link for surrogate BYB account"
    InsertPara
    Insert "CONNECT " + surrogateId$ + "/" + \
        surrogatepassword$ + "D" + site1$ + ";"
    InsertPara
    Insert "CREATE DATABASE LINK " + site2$
    InsertPara
    Insert "CONNECT TO " + surrogateId$ + " IDENTIFIED BY " + \
        surrogatepassword$ + ";"
    InsertPara
    InsertPara
    Insert "DBN Create link for replication " + \
        "administrator account"
    InsertPara
    Insert "CONNECT " + repadminId$ + "/" + repadminpassword$ \
        + "D" + site1$ + ";"
    InsertPara

```

```

Insert "CREATE DATABASE LINK " + s11a2$
InsertPara
Insert "CONNECT TO " + repda1a1d$ + " IDENTIFIED BY " + \
    repda1apassword$ + ";"
InsertPara
InsertPara
End If
For count = 0 To schemaa1a - 1
    Insert "REM Create Link for " + schema1a$(count)
    InsertPara
    Insert "CONNECT " + schema1a$(count) + "/" + \
        schema1apassword$(count) + "@" + s11a1$ + ";"
    InsertPara
    Insert "CREATE DATABASE LINK " + s11a2$
    InsertPara
    Insert "CONNECT TO " + schema1a$(count) + \
        " IDENTIFIED BY " + schema1apassword$(count) + ";"
    InsertPara
    InsertPara
Next
End Sub

```

Steve Bohrouck is president of Animated Learning (<http://www.animatedlearning.com>), a software company that develops multimedia coursesware to teach people about client/server technology. Read this article as it originally appeared in the July/August 1998 issue of Oracle Magazine.

SHARED MEMORY 문제

Bulletin no : 10305

1. 왜 Problem 이 생기나?

Oracle 은 Process와 SGA(Bytes Global Area) 간의 Communication을 위해 Shared Memory 와 Semaphore 를 사용한다.

Oracle Instance 가 올 때 SGA를 Create 하기 위해 Main Memory 의 일부분을 할당하는데 이때 Shared Memory Semaphore 가 적절하지 않으면 이에 관련한 Error가 발생한다.

2. 해결 만

SGA는 Shared Memory 안에 생기므로 Shared Memory 는 각 Process에게 사용 가능해야 한다.

Shared memory 와 Semaphore parameter 는

- SHMMAX = 1개의 shared memory segment 의 maximum size
- SHMNM = 1개의 shared memory segment 의 minimum size
- SHMNI = shared memory identifier 의 숫자
- SHMSEG = 1개의 process에 attach되는 shared memory segment의 maximum개수
- SEMMIB = system의 semaphore 개수
- SEMMI = 시스템에서 identifier 를 killing하는 semaphore 수

* 추천하는 Semaphore와 Shared Memory Parameter

Operating System	Shared Memory Parameters	Semaphore

Bus DB		
	SHMSEG = 32768	SEMMIB = 200
	SHMNI = 50	SEMMI=50
Boinx		
	SHMMAX = 8388608	SEMMIB = 200
	SHMSEG = 20	SEMMIBL = 50
	SHMNI = 100	SEMMI = 70
HP/UX		
	SHMMAX = 0x4000000 (64MB)	SEMMIB = 128
	SHMSEG = 12	SEMMI = 10
Digital Unix (DEC Alpha OSF/1)		
	SHMMAX = 4194304	SEMMIB = 80
	SHMSEG = 32	SEMMIBL = 25
Unix		
	Use System Default	SEMMIB
		SEMMIBL = 5

AT&T Unix	BHMMAX = RAM-Dependent	BEMHMS = 200
8 or 16MB RAM	BHMMAX = 5 MB	For All RAM
32 MB RAM	BHMMAX = 8 MB	Value
64 MB RAM	BHMMAX = 16 MB	
128 MB RAM	BHMMAX = 32 MB	
256 MB RAM	BHMMAX = 64 MB	
512 MB RAM	BHMMAX = 128 MB	
1024 MB RAM	BHMMAX = 256 MB	
2048 MB RAM	BHMMAX = 512 MB	
	BHMBEC = 8 for all RAM	Value
	BHMHIN = 1 for all RAM	Value
Dynix/PTX	BHMMAX = 1101004B	BEMHMS = 200
	BHMBEC = 20	BEMHBL = 65
Other Parameter	NOFILES = 128	
OS/386	BHMMAX = 41B4304	BEMHMS = 200
	BHMBEC = 15	

Shared Memory 와 Semaphore Parameter 는 OS 의 Kernel Configuration File 에 반드시 지정되어야 하며, File 의 위치는 OS마다 차이가 있다.

현재의 Shared Memory 와 Semaphore Configuration 을 알기 위해서는 다음의 Command를 이용한다.

```
% sysctl kern
```

* HP-UX (relevant sections only) 예제의 예:

Semaphore 관련 Parameters

- maximum value for semaphore(count)= 18384
- Semaphore max(count)= 4096
- number of semaphore identifiers(countl) = 4096
- total number of semaphore in the system(count) = 8192
- number of semaphore undo structures(count) = 1536
- semaphore undo entries per process(count) = 512
- semaphore maximum value(count) = 32767

Shared Memory 관련 Parameters

- maximum shared memory segment size in bytes(chmax) = 538870812
- minimum shared memory segment size in bytes(chmin) = 1
- maximum shared memory segments in system (chmax1) = 512
- maximum shared memory segments per process(chmaxg) = 512

NOTE: BMMHAK는 현 system에 B개의 instance가 수행될 수 있는 충분한 값이다.

* Shared memory 또는 semaphore parameters 를 변경하기 위해서는 ...

1. Oracle instance를 Shut Down 한다.
2. DB의 Kernel Configuration File이 있는 곳으로 간다.
3. System Utility 또는 Editor를 이용해서 필요한 값을 바꾼다.
System Utility는 다음과 같다.

```

-----
| DB          | Utility      |
-----
| HP/UX       | BAH          |
| BSD         | BYBAMMBH    |
| AIX         | BHIT         |
| Solaris     | ADMINTOOL   |
-----

```

4. Kernel 을 Reconfigure 한다.
5. System을 Reboot 한다.
6. Oracle instance를 Startup시킨다.

[예제] Solaris 2.3/2.4 parameters and commands:

1. BQLBHA 예시 :

```

BQLBHA> shutdown
BQLBHA> exit

```

2. Superuser(root)로 login 하고 :

```

# cd /etc

```

3. /etc/system file 에 다음을 추가 한다:

```

set shmax:ichinfo_chmax=8388608
set shmax:ichinfo_chmin=1
set shmax:ichinfo_chmax1=100
set shmax:ichinfo_chmaxg=20
set semsys:icinfo_nsems=200
set semsys:icinfo_nsems1=70

```

4. Kernel을 reconfigure 한다.

```
# touch /reconfigure
```

5. Machine 을 reboot 한다.

```
#init B
```

B. BQLBHA 예시 :

```
BQLBHA> startup
```

```
BQLBHA> exit
```

Oracle의 Init.ora Parameter 에는 BGA 의 영향을 주는 Parameter가 있다. DB의 Shared Memory와 Sgastructure Parameter에 연결된 이 Parameter의 Setting은 System과 Oracle의 Performance에 영향을 미친다.

SHARED POOL SIZE의 계산

Bulletin no : 10989

모든 instance를 background process + BGA라고 말할때 처음 instance가 기동 되면 BGA가 할당된다. 이때 이 BGA를 구성하는 메모리 영역은 크게 3부분으로 구성된다고 할수있다. - Shared pool, Redo log buffer, BB buffer cache.

여기서는 BGA를 구성하는 shared pool의 size를 시스템에 맞게 산출하는 방법에 대해서 알아본다.

Shared pool영역 구성은 MTB방식으로 접속된 session의 PGA, Shared SQL area 그리고 dynamic하게 할당되는 data structure로 구성된다.

1. Shared Pool

Shared pool은 7/dbs/initdbora(parameter file)의 SHARED_POOL_SIZE라는parameter로 그 크기를 지정하는데 default값은 3.5M을 갖게 된다. 일반적으로 shared pool이 얼마나 사용되는가 하는 문제는 application dependent하므로 각 application을 조사하는 게 필요하다. 시스템에서 필요로 하는 크기를 검사하기 위해 아주 큰 크기로 parameter file에 지정하여 dynamic하게 할당되는 BGA가 충분히 큰 값을 갖게 한 후, 검사가 끝난 다음 아래에서 계산된 size로 변경 해 수도록 한다.

2. 계산 공식

- Session당 최대메모리사용량(Max Session Memory)
- * 동시 접속하는 User의
- + Shared SQL 영역으로 사용되는 메모리량
- + Shared PL/SQL을 위해 사용하는 메모리 영역
- + 최소 30%의 여유 공간
- Minimum Shared Pool

3. 계산 예제

(1) 적당한 user session에 대한 session id를 찾 다.

```
BQ1084> select sid from v$process p, v$session s
2   where p.addr=s.paddr and s.username='SCOTT';

      SID
-----
      28
1 row selected.
```

(2) 이 session id에 대한 maximum session memory를 찾는다.

```

SQLDBA> select value from v$eventstat e, v$eventname n
        2   where e.statistic# = n.statistic#
        3         and n.name = 'execution uga memory max' and eid=28;

VALUE
-----
273877
1 row selected.

```

(3) Total shared SQL area를 구한다.

```

SQLDBA> select sum(shared_size) from v$sqlarea;

SUM(SHARED_SIZE)
-----
8838825
1 row selected.

```

(4) PLSQL shared memory area를 구한다.

```

SQLDBA> select sum(shared_size) from v$db_object_cache;

SUM(SHARED_SIZE)
-----
4823537
1 row selected.

```

(5) Shared pool size를 계산한다.

```

274K shared memory * 400 users
+   SM Shared SQL Area
+   SM PLSQL Shared Memory
+   SM Free Space (30%)

=   184M Shared Pool

```

이 예제에서는 Shared pool의 size는 184M가 적당하다고 할수있다.

4. Shared Memory 부족 (ORA 4031)에 대한 대처

ORA-4031이 발생하는 원인은 2가지 이다. 첫째 절대적으로 shared pool size가 작아서 나는 경우
과, 둘째로 memory는 있으나 적제하려 하는 PL/SQL package가 너무커서 연속된 shared pool 영역을 점
용 하지 못하는 경우가 있다. 만일 첫런의 경우라면 적당한 계산 과정으로 계산하여 parameter file
에서 SHARED_POOL_SIZE를 늘려주고, 두 번째 경우라면 다음과 같은 방법으로 에러를 피해 갈 수 있
다. - "Exec, dbms_shared_pool, keep" procedure 사용.

[참고] 위 package를 사용하려면 ?/dbms/admin/dbmspool.sql, poolpool.plb를 수행시켜 package를
create시킨 후 사용한다. (자세한 사항은 bulletin 10085, Oracle7 Server Tuning 4장12를 참조한다.)

SNAPSHOT 생성 방법

Bulletin no : 10201

OPERATION GUIDE FOR CREATE SNAPSHOT

BQL00A> shutdown

. InitSID.ora file 에 다음의 3 parameter 를 setting 합니다.

SNAPSHOT_REFRESH_PERIODS= 1

SNAPSHOT_REFRESH_INTERVAL=80

SNAPSHOT_REFRESH_KEEP_CONNECTION=FALSE

BQL00A>startup

1.run script file(default :run script file)

"/rdhsc/rdsls/ctscmp.sql

2.run script file(for using stored pkg)

"/rdhsc/rdsls/dhscmp.sql

global_name=false

3.BQL> connect scott/tiger

BQL> create database link pu_1k

connect to scott identified by tiger

using 'tspuser:DBATJ';

4.BQL00A> connect sys/manager;

BQL00A> grant create snapshot to scott;

BQL00A> connect scott/tiger;

5.BQL> connect scott/tiger

BQL> create snapshot my_pu

refresh 10 second 80

refresh fast

start with sysdate

next sysdate +1

as select * from myDpu_1k

```
BQL> desc emp$emp_pu
```

```
1 BQL>connect scott/tiger
BQL> create snapshot log
BQL> desc slog$emp
```

Manual operation test

```
BQL> connect scott/tiger
BQL> update emp set ...
BQL> commit;
```

```
2 BQLDBA> connect internal;
```

```
BQLDBA> execute dbms_snapshot.refresh('scott.emp_pu','f');
```

```
BQLDBA> select * from scott.emp$emp_pu;
```


SNAPSHOT 을 자동적으로 REFRESH 하는 방법

Bulletin no : 10075

SNAPSHOT 을 REFRESH 하기위해서 SNAPSHOT에 관련된 PARAMETERS (7/26/14itBIB.doc) 들을 다음과 같이 SETTING 한다.

```
SNAPSHOT_REFRESH_PROCESSES = 1 (1 이상)  
SNAPSHOT_REFRESH_INTERVAL = 80  
SNAPSHOT_REFRESH_KEEP_CONNECTIONS = FALSE
```

각 PARAMETERS 의 기 준 살펴보면 다음과 같다.

. SNAPSHOT_REFRESH_PROCESSES (Default : 0)

각 INSTANCE 당 SNAPSHOT_REFRESH_PROCESSES 의 수를 의미한다. 만일 SNAPSHOT 을 자동으로 REFRESH 하기위해서는 1 이상의 수를 SETTING 하고 동시에 REFRESH 하는 SNAPSHOT 수가 대량이 아니 1 이 충분하다. SNAPSHOT_PROCESSES 는 최대 10 까지(SMP0 - SMP9) 가능할때 이 PROCESSES 들은 주기적으로 WAKE-UP 하여 자동적으로 REFRESH 되도록 SCHEDULE 되어있는 SNAPSHOT 을 REFRESH 하고 PROCESSES 가 여러개일때는 SHARE 를 한다.

. SNAPSHOT_REFRESH_INTERVAL (Default : 80 초)

SNAPSHOT_REFRESH_PROCESSES 가 WAKE-UP 하는 INTERVAL 이다.

. SNAPSHOT_REFRESH_KEEP_CONNECTIONS (Default : FALSE)

SNAPSHOT_PROCESSES 가 모든 SNAPSHOT 을 REFRESH 후에 BENDTE 데이터 레이스 CONNECTION 들 보존 할 것인지를 결정한다. 만일 FALSE 이면 모든 SNAPSHOT 이 COMPLETE 된후에 SNAPSHOT 을 REFRESH 하기 위해 만들었던 BENDTE DATABASE CONNECTION 들 CLOSE 한다.

SNAPSHOT 을 MANUAL 로 REFRESH 하기위해서는 DBMS_SNAPSHOT.REFRESH_ALL를 실행한다. SMP# 이 실행중일때는 DBMS_SNAPSHOT.REFRESH_ALL 은 필요없다.

SNAPSHOT의 기본 개념과 예제

Bulletin no : 10958

먼저 Decima7 에서의 snapshot 에 대해 논하고 implementation 하는 장점과 refresh 하는 장점에 대해 알아보자 .

1) 서론

Decima7 에서는 분산 network 환경에서 서로 다른 노드의 갯수에 제한이 없이 master table 의 replication 을 위한 table snapshot 을 제공한다.

Snapshot 은 update 가 불가능하다. 이것은 read 만 가능한 table 과 같다고 할수 있겠다. Update 는 master table 에서만 가능하다.

각 snapshot (replicate)의 편집된 사항은 master table 에 비동기적으로 반영 되어진다. refresh 는 SQL 명령어를 이용해 수동적으로 수행되어질 수도 있고 , refresh 시간에 따라 자동적으로 수행되어질 수도 있다.

2) Simple 과 Complex

Simple snapshot 은 각 row 가 한개의 remote table 의 1개의 row 에 기초한 것이다. 즉 Group by, Order by, subqueries, joins, set operations 이 없는 경우이다. 반대로 snapshot 에 정의된 query 가 이러한 구문을 포함하면 complex snapshot 이라 한다.

3) snapshot 의 장점

. remote table 에 대한 snapshot 을 local 에 가짐으로써 network 를 통한 query 를 하지 않아도 되어 시간을 절약할수 있다.

. 여러가지 원인에 의해 master table 을 access 하지 못하더라도 master site 사용자들은 여전히 snapshot을 사용할 수 있다.

4) 사용 방법

CREATE SNAPSHOT , snapshot_name

PCTYPE	integer
PCTYPEID	integer
INITNAME	integer
MAINTNAME	integer
TABLERSPACE	tablespace
STORAGE	storage_clause
CLUSTER	cluster (column_list)
REFRESH	

START WITH date

NEXT date

5) Snapshot 에 관련된 오브젝트들

Master table
Snapshot base table
Snapshot log

6) Snapshot 에 관련된 Data dictionary Views:

DBA_SNAPSHOTS
DBA_SNAPSHOTS_LOCS
USER_SNAPSHOTS
ALL_SNAPSHOTS
VIEWS

7) Snapshot 에 관련된 Data dictionary tables:

SNAPS
MLOGS

8) 연관된 Packages

DBMS_SNAPSHOT

이 것은 다음의 두 스크립터를 순서적으로 수행하면 생성된다.

- (1) dbmsnap.sql
- (2) snaproc.sql (which runs dbmsnap.plb, besides other scripts)

— 이 Package 에서 수행되는 것들
—
— purge_log - 필요없는 row 의 log 를 purge
— refresh - 주어진 snapshot 의 refresh
— refresh_all - 모든 snapshots 를 refresh
— drop_snapshot - snapshot 를 drop
— set_up - snapshot refresh 를 위해 master를 setup
— wrap_up - master side 에서 refresh를 record
— get_log_age - log 에서 가장 오래된data 를 check
— testing - snapshots를 test (currently null)
— l_a_a_refresh - flag used to let trigger identify refreshes

이 package 는 snapshot 를 생성하기 전에 반드시 존재 하여야 한다.

9) Init.ora 에 적용되는 파라미터들

snapshot 의 refresh process 를 위한 파라미터는 다음과 같다.

- o SNAPSHOT_REFRESH_PERIODS (number)
- o SNAPSHOT_REFRESH_INTERVAL (seconds)
- o SNAPSHOT_REFRESH_REFRESH_CONNECTIONS (true/false)

10) Snapshot Mechanism

Snapshot이 생성되면, Oracle은 이 snapshot에 관련한 여러 다른 오브젝트들도 함께 생성한다. 이들 오브젝트는 변화되어지지 않는다.

snapshot class에서는 snapshot 에서 정의된 query 에 의해 가져온 row 들 저장할 base table 을 생성한다. 또한 이 table 들 기반으로 view 들 생성한다. 이 view 는 read-only 이고 snapshot 들 이용하
는 사용자들 이용가능하다 .
또 다른 view 또는 snapshot 들 refresh 하기 위한 view 가 있다.

Snapshot refreshing:

주기적으로 snapshot 은 refresh 하여 최근의 변화된 사항들을 master table 에 반영한다. Snapshot process는 snapshot 에서 정의된 query를 이용하여 변화된 사항들을 가져와 이전의 snapshot data
를 전환한다.

각 snapshot 은 각기 프랜잭션에서 refresh 된다.

Snapshot log:

snapshot log 는 마스터 테이블 과 관련이 있고 마스터 테이블 위에 존재한다. 여기에는 마스터 테이블에
서 update/delete/insert 되는 row 가 저장되며 snapshot refresh 시 이용된다. 임의의 마스터 테이블
위에 기초한 simple snapshot 이 refresh 되면 snapshot log의 해당 row 가 반영된다. 이것을 "fast
refresh" 라고 일컫는다. 한 snapshot log 는 같은 master table 에 설정된 여러개의 snapshot log 에
사용되어 진다. log 의 특정 row 는 모든 snapshot 에 의해 필요 없게 되면 삭제가 되어 그 크기가
작게 유지된다.

그러나 필요로 하는 snapshot 이 하나라도 있으면 지워지지 않는다.

Snapshot 이 simple snapshot 이 아니고 snapshot log 가 없으면 fast refresh 는 불가능하며 이 경
우 "complete refresh" 에 의해 refresh 된다.

사용 예 :

```

*****
[1] Simple snapshot 과 Snapshot log (on Master):
*****

```

MARTIN BITE (MARTIN)

Create a snapshot log on the master table:

```
sqlplus scott/tiger
```

```

create snapshot log on my
tablespace users
storage (initial 10K pctincrease 50)
pctfree 5;

```

NOTE: Init.ora 파라미터는 필요하지 않다.

SNAPSHOT SITE (LOCAL)

1. 다음처럼 Init.ora 파라미터를 설정한다.

```

snapshot_refresh_interval = 20

snapshot_refresh_processes = 2

snapshot_refresh_keep_connections = true

open_cursors = 250

```

2. sqlplus

```

connect internal

shutdown

startup (Init.ora 파라미터 적용)

```

3. sqlplus system/manager

```

create public database link chicago
connect to scott identified by tiger
using 'chicago';

```

NOTE: BQLENet V2 의 'chicago' 는 tnsnames.ora에 정의되어 있어야 한다.

4. sqlplus scott/tiger

```

create snapshot my_snap

pctfree 5

pctused 80

```

```
refresh fast

start with update

next update + (1/288)      /* 5 분마다 refresh */

as select * from emp@chicago;
```

MASTER SITE (REMOTE)

1. sqlplus scott/tiger

```
insert into emp (empno, ename, job, hiredate, deptno)
```

```
values (1234, 'SCOTT', 'DBA', sysdate, 10);
```

```
commit;
```

SNAPSHOT SITE (LOCAL)

1. sqlplus scott/tiger (or simply switch to client window)

```
select count(*) from emp_emp;
```

snapshot 이 자동으로 refresh 될때까지 만족한다. (약 5 분)

이 랑집은 fast refresh 가 작동됨을 보여준다.

snapshot 은 dbms_snapshot.refresh procedure 를 이용함으로써 수동으로 작동 시킬수 있다.

MASTER SITE (REMOTE)

1. sqlplus scott/tiger

```
insert into emp (empno, ename, job, hiredate, deptno)
```

```
values (12345, 'SCOTT', 'DBA', sysdate, 10);
```

```
commit;
```

SNAPSHOT SITE (LOCAL)

1. sqlplus system/manager

```
select count(*) from scott.emp_emp;
```

```
execute dbms_snapshot.refresh('SCOTT.EMP_EMP','?');
```

PL/SQL procedure successfully completed.

```
select count(*) from scott.emp_emp;
```

NOTE: Snapshot 이 수동으로 refresh 되었음을 확인할수 있다.

snapshot 은 dbms_snapshot.refresh_all 을 수행하여 refresh 할 수 있다.

만일 주어진 snapshot 이 1:00 에 refresh 되도록 되어 있다면, 1:00시 전에 refresh_all 을 수행하
도 refresh 는 불가능하다. 이 예제는 아래에 제시되어 있다.

MASTER SITE (MEMOTE)

1. sqlplus scott/tiger

```
insert into emp (empno, name, job, hiredate, deptno)
```

```
values (3456, 'SCOTT', 'DBA', sysdate, 10);
```

```
commit;
```

BMSPHOT SITE (LOCAL)

1. sqlplus system/manager

```
select count(*) from scott.emp_emp;
```

```
execute dbms_snapshot.refresh_all;
```

PL/SQL procedure successfully completed.

```
select count(*) from scott.emp_emp;
```

NOTE: refresh_all 프로시저를 이용하여 수동으로 Snapshot 을 refresh 할수 있다.

[2] snapshot log 가 없는 sqlplus Snapshot

(1) FAST REFRESH OPTION 을 이용한 BMSPHOT .

이 때는 마스터 데이터베이스 snapshot log 를 가질 경우만 fast refresh 가 가능 함을 보여준다. 만일 "refresh snapshot"으로 정의된 snapshot 이 snapshot log 가 없다면 마스터 데이터베이스 수정될 때 자동 으로 refresh 되지 않는다.

 SNAPSHOT BITE (LOCAL)

Create Blueprint

```
BQL>connect system/manager
Connected.
BQL>grant resource to scott;
```

Great success!

```
BQL) connect aJ/aJ
Connected.
```

Create Database Link
=====

```
BQL) create database link a1x1ink
2 connect to system identified by manager
3 using 't10x1x:U71B';
```

Database link created.

Check the Database Link created

BDL: [dmsc deptDnlxl link](#)

NAME	UNIT	TYPE	DEPTH
NUMBER(2)			
NAME		UNICHAN2(14)	
LDC		UNICHAN2(13)	

Create the Budget (to be refreshed every 5 minutes)

BDL) create complete dependency

```
2 platform 5
3 returned 80
4 refresh fact
5 start with updates
```



```
B exec update * (1/288)
? as select * from system,dept0a,link;
```

Budget created.

Check the Budget created

BQL) select * from dept0a;

DEPTNO	NAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON
50	a,b	

MASTER SITE (REMOTE)

BQL) connect system/manager

Connected.

Check the Master table

BQL) select * from dept;

DEPTNO	NAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON
50	a,b	

BQL) delete from dept where deptno = 50;

1 row deleted.

BQL) commit;

Commit complete.

```

-----
SNAPSHOT SITE (LOCAL)
-----

```

BQL) select * from deptemp;

```

      DEPTNO  DNAME          LOC
-----
      10 ACCOUNTING      NEW YORK
      20 RESEARCH         DALLAS
      30 SALES              CHICAGO
      40 OPERATIONS         BOSTON
      50 n, h

```

BQL) /

```

      DEPTNO  DNAME          LOC
-----
      10 ACCOUNTING      NEW YORK
      20 RESEARCH         DALLAS
      30 SALES              CHICAGO
      40 OPERATIONS         BOSTON
      50 n, h

```

After 10 minutes:

BQL) /

```

      DEPTNO  DNAME          LOC
-----
      10 ACCOUNTING      NEW YORK
      20 RESEARCH         DALLAS
      30 SALES              CHICAGO
      40 OPERATIONS         BOSTON
      50 n, h

```

RESULT: Snapshot 이 refreshed fact 로 만들어져 있고 , snapshot log 가 있기 때문에 이 snapshot 은 refresh 되지 않는다.

dbms_snapshot.refresh 패키지 프로시저를 수행하면 refresh 가 수행된다.

(2) REFRESH COMPLETE OPTION 을 이용한 SNAPSHOT.

```

-----
SNAPSHOT SITE (LOCAL)
-----

```

Budget 샘플

BQL) connect system/manager

Connected.

BQL) grant resource to sa;

Grant succeeded.

Database Link 샘플

BQL) create database link a1xlink

2 connect to system identified by manager

3 using 'titanix:U71B';

Database link created.

Database Link 샘플을 점검

BQL) desc dept@a1xlink

Name	Null?	Type
DEPTNO		NUMBER(2)
DNAME		VARCHAR2(14)
LOC		VARCHAR2(13)

Create the Budget

BQL) connect sa/sa

Connected.

BQL) create budget deptcomp2

2 refresh 5

3 returned 80

4 refresh complete

5 start with update

6 next update + 1/(288*20)

/* REFRESH EVERY 15 SECONDS */

7 as select * from system,dept@a1xlink;

Budget created.

Budget 샘플 검

BQL) select * from deptemp2;

DEPTNO	NAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON
50	a,b	

MASTER SITE (REMOTE)

BQL) connect system/manager
Connected.

마스터 데이터를 점검

BQL) select * from dept;

DEPTNO	NAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON
50	a,b	

BQL) delete from dept where deptno = 50;

1 row deleted.

BQL) commit;

Commit complete.

SNAPSHOT SITE (LOCAL)

BQL> select * from deptemp2;

DEPTNO	NAME	LOC
--------	------	-----

```

10 ACCOUNTING    NEW YORK
20 RESEARCH      DALLAS
30 SALES         CHICAGO
40 OPERATIONS    BOSTON
50 n,l

```

After 15 seconds:

BQL> /

```

DEPTNO  NAME      LOC
-----  -
10 ACCOUNTING    NEW YORK
20 RESEARCH      DALLAS
30 SALES         CHICAGO
40 OPERATIONS    BOSTON

```

REPLY: o| Snapshot o| refreshed complete 도 생성 되었기 때문에 자동으로 refresh 된다.

```

-----
MASTER SITE (REMOTE)
-----

```

Check the Master table

BQL> select * from dept;

```

DEPTNO  NAME      LOC
-----  -
10 ACCOUNTING    NEW YORK
20 RESEARCH      DALLAS
30 SALES         CHICAGO
40 OPERATIONS    BOSTON

```

BQL> insert into dept values(50, 'FINANCE', 'BFD');

1 row inserted

BQL> commit;

Commit complete.

BQL> select * from dept;

```

DEPTNO  NAME      LOC
-----  -

```

10 ACCOUNTING	NEW YORK
20 RESEARCH	DALLAS
30 SALES	CHICAGO
40 OPERATIONS	BOSTON
50 FINANCE	BFD

 SNAPSHOT SITE (LOCAL)

BQL) select * from deptemp2;

DEPTNO	NAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

After 15 seconds:

BQL) select * from deptemp2;

DEPTNO	NAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON
50	FINANCE	BFD

RESULT: Snapshot of refreshed complete로 감지되었기 때문에 자동으로 refresh된

TABLE SIZE 와 INDEX SIZE 계산

Bulletin no : 1015B

1. TABLE SIZE 계산 공식 (DATA BLOCK SIZE : 2K 로 가정)

```
& sqlplus scott/tiger
SQL> SELECT  DBMS_UTILITY(4, ceil(ROW_COUNT /
              ((round(((185B - (initrans * 23)) *
              ((100 - PCT_FREE) / 100)) / ABS_ROW_SIZE))) * 2)
              TableSize_Kbytes
FROM dual;
```

- *. 한개의 BLOCK에 Available 한 Bytes - 185B
- *. 각 initrans 는 23 Bytes
- *. PCT_FREE : Table 의 pctfree 값(default 10)
- *. ABS_ROW_SIZE : 각 row 의 평균 SIZE 추정치
- *. ROW_COUNT : table 의 row 의 갯수

2. INDEX SIZE 계산공식

```
SQL> SELECT  DBMS_UTILITY(4, ((1.01) * ((ROW_COUNT /
              ((floor(((204B - 11B - (initrans * 23)) *
              (1 - (percent_free/100))) /
              ((10 + uniqueness) + number_col_index +
              (total_col_length)))))) * 2))
              IndexSize_Kbytes
FROM dual;
```

- *. 한개의 BLOCK에 Available 한 Bytes (1895 = 204B - 11B)
- *. 각 initrans 는 23 Bytes
- *. ROW_COUNT : table 의 row 의 갯수
- *. PCT_FREE : index 의 pctfree 값(default 10)
- *. number_col_index : index 에서 column 의 수
- *. total_col_length : index 의 길이 추정치
- *. uniqueness : 만일 unique index 면 1, non-unique index 면 0.

TABLE 크기 추정 방법

Bulletin no : 10085

TABLE(NON-CLUSTER TABLE)의 초기 사용량을 추정하는 방법을 살펴보자.

여기서는 BLOCKS BY ROW를 기준으로 하여 설명 하기 하며, TABLE의 초기 사용량을 계산하는 단계를 다음과 같이 나눌수 있다.

- STEP 1 - 총 블록 헤드 크기(BLOCK HEADER SIZE)를 계산
- STEP 2 - 데이터 블록 당 사용 가능한 데이터 영역을 계산
- STEP 3 - 평균 ROW의 전체 컬럼의 길이(COMBINED COLUMN LENGTH)를 계산
- STEP 4 - 총 평균 ROW 크기를 계산
- STEP 5 - 데이터 블록내의 평균 ROW 수를 계산
- STEP 6 - 데이터에서 요구되는 블록과 하이프 수를 계산

1. [STEP 1] : 총 블록 헤드 크기(TOTAL BLOCK HEADER SIZE)를 계산

1) total block header

$$\text{total block header} = \text{block header, part A} + \text{block header, part B}$$

$$\text{block header, part A} = (\text{fixed header} + \text{variable transaction header})$$

$$\text{block header, part B} = (\text{table directory} + \text{row directory})$$

** fixed header	: 57 bytes (고정된 블록 헤드)
variable transaction header	: 23 * I
	(I는 해당 TABLE의 INITRANS의 값)
table directory	: 4
row directory	: 2 * N
	(N은 블록의 ROW 수, STEP 5에서 계산)

2) 예를 들어, INITRANS = 1일때 경우의 total block header

$$\begin{aligned} \text{total block header} &= (57 + (23*1)) + (4 + (2*N)) \\ &= 80 + (4 + (2*N)) \text{ bytes} \end{aligned}$$

2. [STEP 2] : 데이터 블록 당 사용 가능한 데이터 영역

(DATA SPACE PER DATA BLOCK)를 계산

1) available data space

$$\begin{aligned} \text{available data space} &= (\text{block size} - \text{total block header}) - \\ &\quad ((\text{block size} - \text{block header, part A}) * \\ &\quad (\text{PCTFREE}/100)) \end{aligned}$$

※ block size : 데이터베이스의 블록 크기
(BQL*DBA의 BLOCK PARAMETER에서
db_block_size를 확인)

2) 예를 들어, db_block_size=2K, PCTFREE=10 일 경우

$$\begin{aligned} \text{available data space} &= (2048 - (80 + (4 + 28))) - \\ &\quad ((2048 - 80) * (10 / 100)) \\ &= (1968 - 28) - (1968 * 0.1) \\ &= (1968 - 28 - 196) \\ &= (1738 - 28) \text{ bytes} \end{aligned}$$

3. [STEP 3] : 평균 ROW의 전체 컬럼의 길이
(COMBINED COLUMN LENGTH)를 계산

ROW 길이를 계산하기 위하여 TABLE 정의에서 컬럼 수, 각 컬럼의 데이터 타입, 가변 길이 컬럼의 평균 크기들을 참조한다.

1) B (data space/average row) : table T 가 다음의 키마로 정의되어
있을 경우의 ROW의 평균 길이

- create table T (A char(10), B date, C number(10,2))

$$B (\text{data space/average row}) = (A + B + C)$$

※ A(컬럼 A의 길이) : 10 bytes → CHAR 타입인 경우 fixed length

B(B의 길이) : 7 bytes → DATE 타입인 경우

C(C의 길이) : 5 bytes = (P / 2) + 1

→ NUMBER 타입인 경우에는

PRECISION을 고려해서 계산됨

(단, PRECISION에서 NUMBER의 길이를 나타내므로 SCALE은 무관함)

4. [STEP 4] : 총 평균 ROW 크기를 계산(TOTAL AVERAGE ROW SIZE)

여기서는 데이터의 ROW에 의해 요구되는 최소 사용 영역을 계산하는 공식을 제공한다.

1) bytes/row

$$\text{bytes/row} = \text{row header} + P + U + B$$

** row header : ROW 당 3 bytes (NON-CLUSTERED TABLE)
 F : 250 bytes 이하를 사용하는 컬럼의 TOTAL LENGTH BYTES
 (각 컬럼 당 1 bytes)
 U : 250 bytes 이상을 사용하는 컬럼의 TOTAL LENGTH BYTES
 (각 컬럼 당 3 bytes)
 B : [STEP 3]의 B (data space/average row)

2) 예를 들어, [STEP 3]의 데이터를 T의 TOTAL AVERAGE ROW SIZE
 - 250 bytes 이하의 컬럼 수 = 3
 250 bytes 이상의 컬럼 수 = 0
 B (data space/average row) = 22 인 경우
 - avg. row size, table T = (3 + (1 * 3) + (3 * 0) + 22)
 = 28 bytes

5. [STEP 5] : 데이터 블록내의 평균 ROW 수(AVERAGE ROWS PER BLOCK)를 계산

1) B (avg. # of rows/block)

$$B \text{ (avg. \# of rows/block)} = \text{available space} / \text{average row size}$$

** available space : [STEP 2]의 available data space
 average row size : [STEP 4]의 bytes/row

2) 예를 들어, [STEP 2]의 available space가 (1788 - 28) bytes이고,
 [STEP 4]의 데이터를 T의 bytes/row가 28bytes일 경우

- $B = (1788 - 28) \text{ bytes} / 28 \text{ bytes}$
 $28B = 1788 - 28$
 $30B = 1788$
 $B = 58 \text{ rows/block}$

B. [STEP 6] : 데이터에서 요구되는 블록과 바이트 수를 계산
 (NUMBER OF BLOCKS AND BYTES)

1) B blocks for table와 B bytes for table

$$B \text{ blocks for table} = B \text{ rows} / B$$

** B rows : TABLE의 ROW 수
 B : [STEP 5]의 B rows/block

$$B \text{ bytes for table} = B \text{ blocks for table} * 2048 \text{ bytes/block}$$

** 2048 bytes/block : 데이터베이스의 블록 크기 (db_block_size)

2) 예를 들어, 테이블 T가 10000개의 row를 가진 경우의 테이블 당 블록 수

- # blocks for table T = 10000 rows / 50 rows/block
= 173 blocks

- # bytes for table T = 173 blocks * 2048 bytes/block
= 354304 bytes
= 348 Kbytes

TABLESPACE 의 내용을 다른 TABLESPACE 로 옮기는 방법

Bulletin no : 10087

현재 USER가 사용중인 TABLESPACE의 데이터를 다른 TABLESPACE의 영역으로 옮기는 방법을 살펴보기로 하자. 우선 EXPORT UTILITY를 이용하여 다른 TABLESPACE로 이전할 데이터에 대한 작업을 수행한다.

(1) EXPORT UTILITY를 이용하여 USER별 데이터 작업

- USER NAME의 EXPORT

```
$ exp user_name1/user_password

Export: Release 7.0.15.4.0 - Production

Export file : expdat.dat > EXP100.dmp
(2) U(user), or (3) T(table) : (2) U > U
Export grants (yes/no) : yes > YES
Export table data (yes/no) : yes > YES
Compress statistics (yes/no) : yes > NO

About to export USER_NAME1's objects ....
. exporting compact
. exporting compact logs
.
```

다음으로 이전하고자 하는 TABLESPACE를 생성하고

(2) USER 용 TABLESPACE 를 생성

```
- SQL> CONNECT INTERNAL;
SQL> CREATE TABLESPACE tablespace_name
DATAFILE '/user1/oracle_data/aaa.dbf' SIZE 100M;
```

3) 해당 USER를 아래와 같은 방법으로 다시 생성하여야함

```
- SQL> DROP USER user_name1 CASCADE;

- SQL> CREATE USER user_name2 IDENTIFIED BY user_password
DEFAULT TABLESPACE tablespace_name
```

```

        QUOTA 100M ON tablespace_name;
SQL>> GRANT CONNECT TO user_name2;

```

4) 생성된 user_name 과 tablespace_name 확인

```

- $ sqlplus user_name2/user_password
SQL> SELECT * FROM DBA_TB_QUOTAS;

```

5) IMPORT UTILITY를 이용하여 전정예정된 TABLESPACE에 데이터 RESTORE

```

- $ imp system/manager file=exp100.dmp fromuser=user_name1
  touser=user_name2 commit=y

```

6) IMPORT 후 각 TABLE에 해당 TABLESPACE로 전정되었는지 확인

```

- $ sqlplus user_name2/user_password
SQL> SELECT * FROM DBA_TABLES;

```

TABLESPACE 크기를 확장하는 방법

Bulletin no : 10185

오라클 7.1 이하에서는 테이블 스페이스를 확장하려면 해당 테이블 스페이스에 데이터 파일을 추가하는 방법을 사용한다. 이 때 추가하는 데이터 파일의 이름은 기존 파일과 동일한 이름이 아니기만 하면 되지만 편의상 기존의 파일에 밀런런호를 붙여서 사용하는 것이 일반적이다.

예를 들어 테이블 스페이스 TOLB를 확장한다고 가정하면

```
$sqlplus system/manager
```

```
SQL>select file_name, bytes
       from dba_data_files
       where tablespace_name = 'TOLB';
```

이와같이 하면 현재 TOLB 테이블 스페이스를 구성하고 있는 파일이름과 크기 (bytes)가 출력된다. 여기서 출력된 file_name 이 /orac10/dbs/tolb001a.dbf 라고 한다면 다음과 같이 하여 테이블 스페이스를 확장한다.

```
SQL>alter tablespace tolb
       add datafile '/orac10/dbs/tolb200a.dbf' size 50M;
```

여기서는 파일의 크기를 50M 로 주었는데 이것은 디스크의 FREE SPACE 과 맞으므로 기존의 데이터 파일의 크기 및 맞으로 들어갈 데이터의 크기 등을 고려하여 적절한 값으로 설정하도록 한다.

오라클 7.2 에서는 위의 방법 외에도 기존의 데이터 파일의 크기를 변경시켜서 확장시킬 수 있다.

예를 들어 TOLB 테이블 스페이스가 현재 50M 크기의 /orac10/dbs/tolb001a.dbf 파일로 구성되어 있다면 다음과 같이 해서 이 파일의 크기를 100M 로 늘릴 수 있다.

```
SQL>alter database datafile
       '/orac10/dbs/tolb001a.dbf' resize 100M;
```

RESIZE 옵션은 47.2 에서 추가된 것으로 기존의 데이터를 확장 또는 축소할 수 있다. 축소하는 경우는 데이터가 들어 있는 경우 하한선 이하로 내려가지는 않는다.

한편, 다음과 같은 명령을 이용하여 데이터가 계속 들어가서 테이블 스페이스를 꽉 채우게 되면 자동적으로 테이블 스페이스를 확장할 수도 있다.

```
SQL>alter database datafile
       '/orac10/dbs/tolb001a.dbf'
       autoextend on
       next 10M
       maxsize 200M;
```

이렇게 하면 데이터가 늘어면서 자동적으로 10M 씩 데이터파일의 크기가 늘어나게 된다. 여기서 는 최대 200M 까지 늘어 날 수 있도록 설정하였다.

TAPE 으로 ARCHIVE 를 직접 받는 방법

Bulletin no : 10155

ARCHMON: REDO LOG ARCHIVING TO TAPE

OVERVIEW

This paper describes how to archive redo logs to tape and how to monitor it through ARCHMON.

UNDERSTANDING LOG ARCHIVING

Any changes made to the database are recorded both in database and redo log buffers. Both buffers exist in the SGA. The redo log buffer is written to an online redo log file whenever a transaction is committed or the redo buffers are full. Every database needs at least 2 online redo log files. The point where the system stops writing to one file and begins writing to another is called a log switch.

A log switch triggers a checkpoint and checkpoints are done at every log_checkpoint_interval. A checkpoint is when all the modified database buffers are written to the datafiles on disk.

When the database is running in ARCHIVELOG mode, the archive daemon ARCH archives filled online redo log files off line or to the archive destination. These archived files can then be used to recover the database from a consistent backup in case of media failure. In versions 8.0.27 and later, ARCH can archive redo log files directly to tape for some Unix platforms.

REQUIREMENTS FOR ARCHIVING TO TAPE

1. One terminal session must be dedicated to the Archive Monitor Program, ARCHMON, which communicates with ARCH. It prompts when a new tape must be mounted or when a full tape must be dismounted, confirming when it is successful, and displays error messages. An operator must check it from time to time for prompts, error messages, etc.

2. Tape device: any type of supported tape device.

3. Network hardware and software: BBP-based systems require BQI-Wnet TCP/IP to be installed on top of TCP/IP networking hardware and software to permit communications between ARCH and ARCHMON.

FACTORS TO CONSIDER FOR ARCHIVING TO TAPE

Database operations will halt if the system cannot write to the current online redo log file. This happens when it has filled the active redo log file but for some reason cannot write to the next one. This may be caused by file corruptions or instance failure (e.g. LGWR died).

If the database is running in archiving mode, this can happen when the current redo log file cannot be written to because ARCH is still archiving the log file that LGWR is trying to log switch to. Factors that can affect this include the size and number of log files and writing rate. Writing a large file to tape may have a significant impact on database performance. More log files of smaller size are recommended for archiving to tape.

Tape archiving includes ARCHMON which communicates with ARCH. When the end of tape is reached, ARCH stops and signals ARCHMON to prompt the user for a new tape. When ARCHMON knows the tape is mounted, it tells ARCH to resume the archiving operation. It needs regular operator monitoring.

SETTING UP REDO LOG ARCHIVING

The following parameters that must be specified for automatic archiving of redo log files are:

log_archive_dest: It indicates the destination for the archiving of the redo log file.

syntax: log_archive_dest=device[:value#(K|B)]

where,

device: specifies the location (device file or regular file) where the redo file is to be archived.

value: This is applicable only for tape archiving. It indicates the size of the tape volume. The tape must be able to hold at least the size of one log file, plus an overhead of 5K bytes.

M, K, B: indicates whether value refers to megabytes(M), kilobytes(K) or bytes(B). The default is bytes.

To archive the online redo log files to tape device /dev/rmt0 to tape which can hold 100M, specify:

log_archive_dest = /dev/rmt0:100M

log_archive_start: It indicates whether archiving is to be automatic or manual when the instance is started. It can have value 'false' or 'true'. The default is 'false'. This parameter is only meaningful if the database is running in archiving mode.

Perform following steps to set up automatic archiving to tapes:

1. Set init.ora parameters log_archive_start and log_archive_dest.

2. Starting ARCHMON

ARCHMON for System V based systems: On System V-based operating systems, ARCHMON can only monitor tape archival of database instances local to its machine.

To start ARCHMON, type:

```
$ archmon host:telnetlog
```

For example, to start ARCHMON for database identified by ORACLE_SID value "TEST" enter:

```
$ archmon SP:TEST
```

ARCHMON for BSD-based systems: On BSD-based systems, the ARCHMON requires BQLNet TCP/IP to work. It communicates with ARCH through archive. To start archive enter:

```
$ tcpctl start
```

To start archmon enter:

```
$ archmon host:telnetlog
```

where host:telnetlog is a valid BQLNet TCP/IP host:telnetlog. For example, to start ARCHMON on host named "landmark" with ORACLE_SID of "TEST" enter:

```
$ archmon SP:landmark:TEST
```

3. Put the database in archiving mode and set automatic archiving. Enter the following BQLNet statements:

```
BQLBBA> startup mount
BQLBBA> connect internal
BQLBBA> alter database archiving;
BQLBBA> archive log start 'destination';
BQLBBA> alter database open;
```

The destination specifies the destination for archive files. It is same as specification for log_archive_dest init.ora parameter.

You get the message: 'connected.' from the ARCHMON session, which is confirmation that it has connected to the archiving process.

When the tape volume is full, ARCH suspends operation, tells ARCHMON to prompt for a new tape, waits for user's response, and then resumes log archiving.

STOPPING ARCHMON AND ARCHIVING

ARCHMON will stop when the database is shutdown or log archiving has been stopped. When the database is shutdown with the normal or immediate option, ARCHMON signs off with following message, after archiving all remaining redo log files:

```
Message from ora_arch_TEST91andshark at 2:35...
Normal, successful termination.
```

If the database is shutdown with the abort option, the archiving of some logfiles will be incomplete. This will not ensure complete recovery up to the last log file written to.

ARCHMON can be interrupted, killed and reloaded. However, if ARCHMON is not reloaded, the database will hang when all redo logs are full until it is restarted.

Archiving can be turned off when the database is up or down. When the database is up, enter following SQL*DBA statements to turn off archiving mode:

```
SQLDBA> connect internal
SQLDBA> alter database close;
SQLDBA> alter database noarchivelog;
SQLDBA> disconnect;
SQLDBA> shutdown
SQLDBA> startup
```

If the database is down, enter the following SQL*DBA statements to turn off archiving mode:

```
SQLDBA> startup mount
SQLDBA> connect internal
SQLDBA> alter database noarchivelog;
SQLDBA> alter database open;
```

To check the status of database, whether it is in archiving mode or not, enter following SQL*DBA statement:

```
SQLDBA> archive log list;
Database log mode                NOARCHIVELOG
Automatic archival                DISABLED
Archive destination               /u01ppart/B033p/dbs/arch.dbf
Dissect online log sequence       47
Current log sequence              48
```

The online redo log is archived as tape as filename 'arch_*.dkt'.

For example, online redo log with sequence# 28 will be stored as tape as file 'arch_28.dkt'.

If offline redo log files are required to recover the database, you may retrieve archived log files using the 'cpio' command. For example, to check the contents of the tape for drive /dev/rmt0, enter:

```
$ cpio -lttc < /dev/rmt0
```

It will display the contents of the tape:

```
arch_28.dkt
arch_28.dkt
arch_30.dkt
..... etc....
```

Use the cpio command to read necessary files from the tape. For example, to get arch_28.dkt enter:

```
$ cpio -lclv arch_28.dkt < /dev/rmt0
```

TKPROF 사용시 CORE DUMP 조치 사항.

Bulletin no : 10310

TKPROF 사용시 Core Dump 조치 사항

Oracle 97.3.2.1 에서 tkprof를 이용하여 Explain을 미 보려면 Core Dump가 떨어 집니다. 이것은 Oracle Bug 378851 때문이며 97.3.3에서 Fix 될 예정 이며, 이 현상을 피해 가는 방법 2가지 방법은 아래와 같습니다.

1. \$ORACLE_HOME/rdbms/admin/utltpkrf.sql 을 execute 수행

2. utltpkrf.sql 에서 생성된 tkprof 라는 role을 tkprof 사용 할 user에게 권한 부여

```
SQL> grant tkprof to scott;
```

3.1 BQL*PLUS 에서 생성된 trace files를 아래와 같이 재 작업 후 tkprof 수행

```
$ grep -v "APPNAME" file.trc > newfile.trc
$ tkprof newfile.trc a explain=scott/tiger
```

3.2 또는, BQL*PLUS 대신 Server Manager에서 수행하면 grep을 사용 만해도 정상 작동됨

```
SQL*PLUS> connect scott/tiger
SQL*PLUS> alter session set sql_trace=true;
SQL*PLUS> select * from tab;
```

* 간단히 PLAM을 미 보기 위해서는 BQL*PLUS 에서 set autotrace on 을 이용해도 됩니다.(97.3에서)

TPC Q&A

Bulletin no : 10180

- TPC란 무엇입니까 ?

TPC는 Transaction Processing Council의 약자로서 여러 하드웨어 및 소프트웨어 회사 및 소수의 사용자 기관들을 멤버로 하는 비영리 단체입니다. TPC의 구성 목적은 "프랜차이즈 프로세싱 벤치마크를 정의 하고 그 결과 판별가능한 성능을 널리 알리는 데" 두고 있습니다.

따라서 멤버들은 TPC의 벤치마크 테스트를 통해 자신의 제품이 타사의 제품(예, H/MB/S)과 얼마나 잘 결합될 수 있는지에 대한 증빙자료를 마련하게 됩니다.

- TPC-C란 무엇입니까 ?

TPC-C는 8개의 테이블로 구성된 데이터베이스에 주문입력 시스템과 주문처리 시스템을 모의, 입기전송 및 갱신쿼리의 혼합 업무부담을 처리해내는 OLTP 벤치마크입니다.

- TPC에서 정의하는 다른 종류의 벤치마킹에는 어떤 것이 있습니까 ?

우선 TPC-C보다 단순한 형태의 OLTP를 벤치마킹하는 A와 B가 있었습니다. A, B는 1985년 8월부로 폐지되었으며, 간단히 설명하면 A는 일본의 Banking System Simulation으로서 3 Update와 1 Insert를 포함하는 한가지 프랜차이즈 라임만을 벤치마킹하는 것이었고, B는 이와 유사하나 "Think Time" 요구사항 및 커밋, 네트워크 애플리케이션이 빠진 것이었습니다.

TPC-B라고 하는 새로운 BSB 벤치마크 테스트가 85년 4월부로 공식화되어 (TPC-C와 유사하며 대용량 DB에 17개의 Query로 테스트) 최적화 및 집의 프로세싱의 효율성을 측정하기로 되었으나 아직 그 누구도 실시 하지 않았으며 앞으로 TPC-B라고 하는 서버 벤치마크, TPC-C/B라고 하는 클/서 벤치마크 및 TPC-E라고 하는 엔터프라이즈 벤치마크 등이 급년 하반기 중 공식화 될 전망입니다.

- TPC-C는 실제상황을 그대로 대변합니까 ?

TPC-C는 단순미고를 위해서는 통용할 수 있으나 실제상황을 그대로 반영한다고 하긴 어렵습니다. 대부분의 벤치마크가 하나의 프랜차이즈로 수행된다는 점을 감안하면 실제상황과는 많이 다를 수 있고 따라서 이와 관련한 데이터의 충돌 등이 테스트되지 않는다고 볼 수 있으며, 실제로 중요한 Data Integrity나 Backup/Recovery는 아예 테스트되지 않습니다.

- 이제까지 오라클이 TPC-C에 참여하지 않은 이유는 무엇입니까 ?

가 큰 이유는 TPC-C의 규정에 맞추지 못하였기 때문입니다. 이 부분은 TPC-C의 규정 중 "Transaction Isolation"에 국한되는 것으로 이제까지의 오라클의 Concurrency Control 기법과는 다른 Sync를 요구하였던것입니다. 따라서 Oracle이 TPC-C를 수행할 수 없었다기 보다는 결과가 만족스럽지 못할 것으로 생각되어 7.3까지 기다린 것입니다.

- TPC-C 테스트를 위해 7.3에 추가된 것은 무엇입니까 ?

한마디로 "Serializable Transaction"이라는 새로운 Concurrency Control 기능을 추가한 것입니다. (자세한 내용은 원문 PB 참조)

- Oracle7 Release 7.3은 아직 베타인데 결과치를 발표할 수 있습니까 ?

그렇습니다. TPC의 "Fair Use Rule"에 따르면, 테스트 된 제품의 "배포시기"가 명시된 뒤엔 Press Release하는 데 문제가 되지않습니다. 그러나 테스트에 관한 모든 것을 밝혀주는 Full Disclosure Report는 88년 2월경에 배포가 가능하니 이점 양지하시기 바랍니다.

- 만약 가망 고객이 다른 플랫폼에 Oracle7의 TPC-C 결과치를 원하면 어떡하나요 ?

가장 예상되는 문제 중의 하나이지만 현재 HP/Oracle 발표이후 다른 하드웨어 업체들도 매우 적극적으로 오라클과 협력하여 그들 플랫폼에서의 결과치를 원하고 있습니다. 이는 그간 TPC-C에 오라클이 불참 함으로써 하드웨어 플랫폼의 비교 그 자체가 주춤 했기 때문입니다. 대부분의 TPC 테스트가 사실 그간 DBMS 보다는 H/W 선전에 많은 영향력을 보였던 것을 고려해도 맞으므로 다른 플랫폼에 Oracle을 TPC-C 런치마킹하는 것은 시간 문제라고 생각합니다. 오라클 서버 기술부 및 각 제품부는 각 하드웨어 벤더와 긴밀히 협조 조만간 새로운 결과들을 속속 발표할 것입니다. 따라서 고객들에게 이 점을 주지하셔야 할 것 입니다.

- Informix가 HP9000/T500 8CPU 에 테스트를 한 것과 이번의 오라클 결과치와는 CPU 갯수의 차이 때문에 어렵지 않은가요 ? (오라클 12CPU를 이용)

일췌 보면 오라클이 결과를 생각해서 민토릭스보다 더 많은 CPU를 이용한 것처럼 보입니다. 결과는 민토릭스가 3118 tpmc, 오라클이 5388 tpmc로서 CPU 갯수 차이에 의한 당연한 차이로 인식되기 쉽습니다. 그러나 정말 중요한 사실은 Informix가 8개 CPU를 넘는 하드웨어로는 런치마킹을 하지 않는다는 것 입니다. 따라서 Scalability면에서 오라클을 따라오지 못한다는 사실이 담겨있는 것 입니다. 이 점을 강조 하시기 바랍니다.

TRANSACTION ENTRY란 무엇인가?

Bulletin no : 10283

* Transaction Entry는 Table Creation시 정의되는 Inittrans와 Maxtrans에 연관되어 있다.

- Transaction Entry는 사용자 Process가 Row를 변경하는 모든 Data Block에 있는 Transaction의 제어를 위해 사용된다. 이것은 하나의 Transaction이 각각의 변경된 Data Block을 위해서 Transaction Table내에서 하나씩의 Slot만을 가짐을 의미한다.

예를 들어, Process A가 5개의 Data Block에 걸쳐 있는 100개의 Row에 대해 갱신 연산을 하고자 한다고 하자. Oracle은 Transaction 내에서 변경되어지는 각 Block에 대해 하나씩의 슬롯만을 취하면 된다. 즉 여기에서는 5개이다.

- Process A가 해야 할 첫번째 일은 Transaction Entry를 할당하는 것이다. 그리고 이 Transaction Entry에 Lock을 건 후 Transaction에 대한 동적(Dynamic)인 정보를 기록한다. 만약 Transaction이 갱신하고자 하는 Row가 Lock이 걸려 있음을 Process가 발견하면 Transaction 공간 내의 Lock을 풀것이다. 일단, Transaction이 Commit을 하거나 Rollback을 한 후에 Entry에는 새로운 Transaction이나, 밀려 혹은 삭제가 자동롭게 이루어질 수 있다.

- 낮은 INITTRANS Parameter의 제정은 큰 Transactional Table에서 Process가 Transaction Entry를 검색하고 할당하기 위해 공간을 찾는 중 찾지 못했을 때 발생하는 경쟁(Contention)이다. Process는 다른 Entry가 Free 될 때까지 기다려야 한다.

- Default INITTRANS는 Table이 10이고 인덱스는 2이다. 모든 Transaction Entry는 23 Bytes 정도를 사용한다. 이것은 DB에 따라 다르며, DBA는 USType_Size Units를 참조하여 확인할 수 있다.

TRANSACTION을 ROLLBACK SEGMENT에 할당하는 방법

Bulletin no : 10389

Oracle은 다음과 같은 규칙에 의해 각 Transaction이 사용하는 Rollback segment를 결정한다.

1. Active Transaction의 수가 가장 작은 Rollback Segment에 할당한다.
2. 만약 1의 조건에 만족하는 Rollback segment가 하나 이상이면, 가장 마지막에 할당된 rbs 다음의 rbs를 할당한다. 이것은 undo 기능을 더 오래 지속시키도록 해 준다.

그러나 Application이 큰 rollback segment가 필요하다면 위와 같이 자동적인 rollback segment의 할당이 아닌 Manual하게 특정 rollback segment를 할당 가능하다. 다음의 방법을 사용한다.

- a. 큰 rollback segment 를 만든다. 큰 rollback segment 를 만들기 위해서는 rbs tablespaces 에 영역이 충분히 커야하므로 필요할경우 tablespace 확장하는 작업을 한다.

```
$ sqlplus system/manager
sql> alter tablespace rbs add datafile '7/rbs1818.dbf' size 100m;

sql> create rollback segment big_rbs storage(initial 10m next 10m)
tablespace rbs;
sql> alter rollback segment big_rbs online;
```

- b. Transaction 들 assign 한다.

```
SQL> set transaction use rollback segment big_rbs;
```

....

```
exec sql commit work;
exec sql set transaction use rollback segment big_rbs;
```

이와 같이 지정하면 이후에 commit 또는 rollback이 일어날 때 까지 한 transaction에만 해당된다.

TRIGGER 에서 ERROR 확인 (SET SERVEROUTPUT ON 사용)

Bulletin no : 10382

1. Trigger 생성 (trg.sql)

```
create trigger dept_trg
after insert on dept
for each row
begin
    insert into emp(empno, ename, deptno)
        values(7000, 'TEST', :new.deptno);
exception
    when others then
        DBMS_OUTPUT.PUT_LINE('sqlcode : '||to_char(sqlcode));
end;
```

2. SQLPLUS 에서 SET SERVEROUTPUT ON setting

```
$ sqlplus scott/tiger
SQL> set serveroutput on
```

3. DEPT Table 에 Data 입력

```
SQL> insert into dept values(10, 'Sales', 'Seoul');
```

-- Error 가 발생하는 경우 sqlcode 가 Display 됨

TWO-PHASE COMMIT

Bulletin no : 10294

1. Two-Phase Commit이란?

- (1) Two-Phase Commit은 Application Programmer가 별도의 프로그램없이 Global Database의 Integrity를 보장하는 기법이다.
- (2) Two-Phase Commit 은 분산 환경 Database(DBS)를 사용하는 경우에 사용된다. 이는 Oracle 97.00 이후 Version에서 제공된다.

2. Two-Phase Commit의 구성

- Two-Phase Commit은 Prepare Phase과 Commit Phase로 나누어 진다.

(1) Prepare Phase

Global Coordinator(분산 Transaction을 일으킨 Node)가 분산 Transaction에 참여한 Node들에 대해 Commit이나 Rollback을 수행해도 되는가를 확인하는단계이다.

(2) Commit Phase

분산 Transaction에 참여한 Node가 Global Coordinator에게 분산 Commit해도 좋다는 Response(즉 Prepare 되었음)를 받고 Transaction을 Commit한다. 어떠한 Node라도 Prepare가 안되었다는 Response가 있으면 Transaction을 Rollback한다.

3. Two-Phase Commit의 수행

- (1) Prepare Phase에 대한 Response는 Prepared, Read-Only, Abort가 있다.
 - A. Prepared는 Data가 정상적으로 수정되었으며 Prepare되었음을 의미한다.
 - B. Read-Only 는 해당Node에 Data의 수정이 없었다는 의미이다.
 - C. Abort는 Prepare되지 않았음을 의미한다.
- (2) 모든 Node에서 Prepared가 되면 Commit Phase가 수행된다.
그러나 한 Node라도 Abort되면 Commit Phase에서 전 Node에 Rollback을 수행한다.

Using Oracle Database Manager 7.2 & 7.3

Document ID: 11128318.81

Title: Using Oracle Database Manager 7.2 & 7.3

Product: Database Manager

Product Version: 7.2/7.3

Platform: Windows NT, Windows BS

Information Type: Advisory

Impact: Medium

Abstract: This bulletin explains how to perform a few common Database Manager tasks.

Keywords: database:manager:DBACLE.INI,VB10.INI:ALIAS:configuration

Using Oracle Database Manager 7.2 & 7.3

Oracle Database Manager is a remote database administration tool that allows a database administrator to perform three key functions:

- 1.) starting up a database
- 2.) shutting down a database
- 3.) checking the status (running or not) of a database.

This bulletin is intended to clarify the steps to perform a few common Database Manager tasks that are not easily understood from the provided documentation.

The following topics are covered in this bulletin:

- Creating a new database alias
- How Database Manager uses the ORACLE.INI file
- Creating a new database configuration
- Finding a configuration file
- Shutting down a database

1. Creating a new database alias

1. Click on the Aliasname button.

A dialog box titled Database Connection appears.

2. Fill in the Alias and Database fields.

The Database field must contain a valid Oracle database connect string. A connect string could be a local one such as "2:" or "2:SID", a BQLENet V1 connect string such as "tk802000:SID", or a BQLENet V2 alias.

3. Click on the Add button to save the Alias/Database pair.

2. How Database Manager uses the ORACLE.INI file

The above process is straightforward, however the use of the ORACLE.INI file by the 32-bit Database Manager (the Windows NT and Windows 95 versions) needs clarification. When you click on the Add button in the Database Connection dialog box the following line is created in the [Oracle] section of the ORACLE.INI file:

```
BQLENET DBNAME MyAlias=My_DB_Name
```

where MyAlias and My_DB_Name are the values you typed in the Alias and Database fields. This is an abnormal behavior since all 32-bit Oracle tools must use the Windows NT or the Windows 95 registry to store initialization information. Most of the 32-bit tools are affected by this abnormality, but the 16-bit tools may be affected since these tools normally use the ORACLE.INI file for storing initialization information.

If you experience connection problems with your 16-bit tools on a 32-bit platform after a database alias is created via Database Manager, try removing the above line. NOTE: The ORACLE.INI file is normally located in the Windows base directory, (typically C:\WINDOWS.)

3. Creating a new database configuration

Database Manager includes many preset configurations, which are available from the Configuration drop-down list. Clicking on the Configure button brings up the Configure Initialization Parameters dialog box. This is where you can edit a few parameters of the selected configuration. Unfortunately, Database Manager does not allow you to create a new configuration and edit all of its parameters in one step. You must save an existing configuration with a different name and edit the .DBA file of the new configuration.

Although this may be awkward, creating a new configuration for your database instance is a very important task because the preset configurations are designed to use the default instance, whose SID is ORCL. A Windows NT configuration, for instance, will try to use the CTX1DBCL.DBA control file, but if an instance tries to use this control file while the default ORCL instance is already running on the same computer, you will get an ORA-800 (2141) error.

1. Choose one of the preset configurations from the Database Manager main window and click on the Configure button to open the Configure Initialization Parameters dialog box.

2. In the dialog box, click on the Save As button.

3. Another dialog box titled "Configuration: Save As" appears.

4. Enter the name of your new configuration and the name of the configuration file which will store your new initialization parameters information. By default, this file will be saved in the

C:\ORANT\WB1\ORATED directory,

where C:\ORANT is the local Oracle home directory

(C:\ORAWINBS for Windows BS

(C:\ORAWIN for Windows 3.1x, or 16-bit Database Manager
running on Windows NT or Windows BS)

You can click on the Browse button to choose another location for the file.

5. After a new configuration is created, exit Database Manager and restart it. You should also do this after making any change to a configuration file. This step is necessary because Database Manager will use the cached version rather than reading the configuration file each time.

Once the new configuration is created, you can modify the initialization parameters by editing the configuration file using any ASCII text editor, such as Notepad. The initialization parameter normally used is the INIT.DBA file on the computer hosting your Oracle7 database. To use the same parameters in Database Manager, simply copy the INIT.DBA and rename the copy with a new configuration file name, i.e. ALIAS1.DBA.

You may wonder why Database Manager does not automatically use the INIT.DBA file. It is because Database Manager is designed as a remote administration tool; your Oracle7 server's file system is not always accessible to Database Manager running remotely.

4. Finding the configuration file of the selected configuration

Database Manager finds a configuration name by looking up the config_name=fil_name pair in the [CONFIGS_EDITED] section of the C:\DBAMT\VB10\VB10.INI file for Windows NT, C:\DBAMIBS\VB10\VB10.INI for Windows BS, or C:\DBAM\VB10\VB10.INI for Windows 3.1x or 16-bit.

Database Manager running on Windows NT or Windows BS.

5. Shutting down a database

Database Manager always issues a SHUTDOWN IMMEDIATE command rather than SHUTDOWN NORMAL to stop a running database. Database Manager also checks for current user sessions before initiating a shutdown. When there are open sessions, Database Manager displays a warning dialog and waits for your approval to proceed with the shutdown.

WINDOWS NT와 ORACLE7 TUNING GUIDE

Bulletin no : 10915

ORACLE7 RUNNING FOR WINDOWS NT의 구조

NT에서 동작하는 ORACLE7은 Microsoft 32 API를 사용하는 완전한 32bit 프로그램으로서 하드웨어와 완전하게 결합되어 동작한다. NT상의 ORACLE7은 NT의 메모리 모델에 맞도록 SINGLE PROCESS/MULTI THREADED 구조를 갖도록 설계되었다.

UNIX에서 동작하는 ORACLE7과는 달리 SINGLE PROCESS상에서 MULTI THREAD 을 갖기 때문에 단일 주소 공간하에서 메모리를 공유하게 된다. ORACLE7은 NT가 제공하는 선점형 스케줄링과 여러 개의 CPU를 최대한 활용하는 LOAD BALANCING 기능을 활용하고 있다.

NT상의 ORACLE INSTANCE는 SGA(Buffer Cache Area)와 여러개의 BACKGROUND THREADS로 구성된다. SGA는 하나의 ORACLE INSTANCE를 위한 다양한 정보를 갖고 있다. 최고의 퍼포먼스를 내기 위해서는 SGA는 주어진 메모리 한도 내에서 충분한 크기를 갖어야 한다. 만약 SGA를 제외한 나머지 메모리가 너무 작게 되면 PAGING과 SWAPPING이 계속 일어나게 되므로 퍼포먼스를 크게 저하시키게 되므로 SGA 영역이 과도하게 커서도 만된다. SGA는 다음과 같은 부분들로 이루어져 있다.

DATABASE BUFFER CACHE

데이터 파일에서 읽어들이는 데이터 블록이 저장되는 곳으로 읽음의 CACHE 역할을 한다. 디스크에 기록되기 이전의 데이터가 저장되는 버퍼의 기능도 하는데 적절한 크기로 세팅함으로써 DBE I/O를 줄여서 퍼포먼스를 증가시킬 수 있다.

LOG BUFFER AREA

데이터 블록의 이전 정보와 현재되고 난 후의 정보가 기록되는 곳이다. LOG_BUFFER 파라미터가 이 영역의 크기를 결정한다(BYTE 단위). 이 영역의 크기가 너무 작으면 LOG WRITE는 디스크에 WRITE 작업들 자주 해야 하기 때문에 퍼포먼스가 저하하게 된다. MULTIPROCESSOR MACHINE에서는 LOG_BUFFER_SIZE_COPIES 파라미터를 CPU 갯수의 2배로 설정함으로써 REDO COPY LATCH에 대한 걸림을 줄일 수 있다.

SHARED POOL AREA

SHARED_POOL_SIZE(BYTE 단위) 파라미터로 그 크기가 결정되는데 SHARED SQL/PLSQL 명령들 저장하고 DICTIONARY CACHE 역할을 한다. SQL문의 PAGING 정보가 기록이 되므로 같은 명령은 REPARSING 없이 바로 실행될 수 있다.

CURSOR

커서는 SQL 명령과 관련된 메모리 영역을 지시하는 HANDLE이다. 대부분의 경우 커서는 내부적으로 생

성직는데 직접 사용자가 커서를 생성하여 명령을 수행할 수도 있다.

BACKGROUND THREADS는 다음과 같이 이루어져 있다.

DBWR(DATABASE WRITER) THREAD : DBWR은 BUFFER CACHE의 변경된 데이터 블록을 디스크로 쓰는 작업을 한다.

LGWR(LOG WRITER) THREAD: LGWR은 REDO LOG ENTRY를 디스크에 기록하는 작업을 한다.

CERT(CHECKPOINT) THREAD: LOG SWITCH가 일어나 DBWR은 버퍼내의 모든 변경된 블록을 디스크에 WRITE하며 CERT는 CONTROL FILE과 DATA FILE을 UPDATE한다. 만약 CERT가 ENABLE되어 있지 않으면 LGWR가 CERT의 기능까지 함께 수행한다. 퍼포먼스 증대를 위해서는 CERT를 ENABLE시켜 주는 것이 좋다.

SMON(SYSTEM MONITOR) THREAD: INSTANCE STATUS에 INSTANCE RECOVERY를 수행한다.

PMON(PROCESS MONITOR) THREAD: USER PROCESS가 비정상적으로 끊어지는 등 FAILURE가 발생했을 경우 PROCESS RECOVERY를 수행하여 점유한 RESOURCE를 RELEASE한다.

RECO(RECOVERY) THREAD: NETWORK/SYSTEM FAILURE 등으로 인해 발생하는 분산 트랜잭션의 PENDING 문제를 해결한다.

ARCH(ARCHIVER) THREAD: ONLINE REDO LOG FILE이 재사용 되기 전에 다른 곳으로 복사하는 역할을 한다.

----- PERFORMANCE BOTTLENECK을 찾는 방법과 SYSTEM MESSAGE

시스템의 다음과 같은 부분에서 퍼포먼스 저하가 많이 발생하게 된다.

SYSTEM MEMORY, CPU, DISK I/O SUBSYSTEM, NETWORK

위의 사항들을 최적상태로 튜닝함으로써 DBACLE의 퍼포먼스를 증대시킬 수 있다. 메모리는 그 크기가 한정되어 있으므로 PROCESS들은 한정된 메모리를 서로 이용하려고 하기 때문에 PAGING과 SWAPPING이 일어나는 경우가 생긴다. 여러명의 사용자가 동시에 디스크를 읽거나 쓰거나 할 경우 DISK CONTENTION이 발생하게 된다. 마찬가지로 여러 개의 프로그램이 CPU를 동시에 사용하고자 하면 CPU CONTENTION이 발생한다. PHYSICAL MEMORY가 부족하게되면 CPU TIME의 많은 부분이 PAGING과 SWAPPING에 할당되게 되므로 퍼포먼스가 저하하게 된다. 한정된 BANDWIDTH에 다다르도록 NETWORK TRAFFIC이 증가하게 되면 NETWORK BOTTLENECK이 발생한다.

PERFORMANCE MONITOR를 통한 CPU BOTTLENECK 검사

PERFORMANCE MONITORING은 응용프로그램이 CPU TIME과 MEMORY와 같은 시스템 리소스를 얼마나 사용하는가를 추적하는 것을 말한다. 따라서 CPU/MEMORY BOTTLENECK을 알아는데 있어서 매우 유용하게 사용되는 방법이다.

CPU는 프로세스들간에 CONTENTION이 일어나는 가장 중요한 시스템 요소의 하나로 만약 시스템안에 CPU가 여러개가 존재한다면 다른 데칼의 CONTENTION이 일어날 것이다. PERCENTAGE PROCESSOR TIME 카운터는 하나의 PROCESS 내의 모든 THREAD가 명령을 수행하기 위해서 소요된 시간 액분율을 알려준다. 만약 이 값이 85%를초과하게 된다면 CPU에서 BOTTLENECK이 발생하고 있음을 알 수 있다.

PERCENTAGE PRIVILEGED TIME와 PERCENTAGE USER TIME의 합은 PROCESSOR TIME 전체를 알려준다. PERCENTAGE PRIVILEGED TIME은 CPU가 응용프로그램을 위해서 사용된 시간이 아닌 OS를 위해서 사용된 시간을 나타낸다. 이 시간의 대부분은 디스크에서 데이터를 가져오고 저장하는데 사용된다.

WINDOWS NT는 각각의 BYTES를 분리하기 위하여 PROCESS BOUNDARY를 사용한다. 모든 응용프로그램 코드는 사용자의 BYTES에 존재하며 $\times$ USER TIME COUNTER를 통하여 알아볼 수 있다. 만약 시스템이 효율적으로 작동한다면 대부분의 시간이 USER PROCESS에 할당되어 있을 것이므로 $\times$ USER TIME이 매우 높은 값을 나타낼 것이다. 하지만 CPU를 많이 사용하는 작업을 수행하고 있고, 이 COUNTER의 값이 높다면 응용프로그램의 용량이 요구된다. 이상의 사항들은 각각의 PROCESS와 THREAD 별로 CPU사용현황을 알려주는 WINDOW PERFORMANCE MONITOR를 통하여 알 수 있다.

DBACL7 BACKUP의 퍼포먼스에 영향을 미치는 사항들

RAW PARTITION

DBACL7은 I/O 작업이 많은 경우 퍼포먼스를 증가시키기 위하여 RAW PARTITION을 사용할 수 있다. RAW PARTITION은 FILE SYSTEM OVERHEAD가 없으므로 I/O 성능이 우수하다. 그러나 RAW PARTITION은 FILE SYSTEM에 비해서 용연성이 떨어지기 때문에 관리하는데 있어 많은 주의와 노력이 요구된다. 적절한 크기로 설정된 충분한 갯수의 RAW PARTITION을 확보하는 것이 쉬운 작업이 아니기 때문에 RAW PARTITION을 이용하여 DATABASE CONFIGURATION을 잡는 것은 까다로운 문제에 속한다. 하나의 데이터 파일은 하나의 RAW PARTITION에 해당된다. 만약 RAW PARTITION을 사용한다면 RAW PARTITION의 크기를 모두 같게 유지함으로써 I/O BALANCING과 관리상의 용연성을 가질 수 있다.

DBACL7 BACKUP BACKGROUND THREAD PRIORITY 높이기

DBACL7 BACKUP과 BACKGROUND THREAD는 NORMAL PRIORITY CLASS상태에서 수행되도록 설정되어 있다. 이 CLASS에서는 BACKGROUND는 PRIORITY를 1에서 15 사이에서 동적으로 전환을 시킨다. 하지만 15~31 사이에 해당하는 REAL-TIME PRIORITY CLASS로는 올라가지 못한다.

DBACL7은 NORMAL PRIORITY CLASS 상태에서 가장 좋은 PERFORMANCE를 보인다는 사실이 알려져 있다. 이 경우는 주로 DLT와 같은 섬적의 프랜잭션일 때에 대한 것으로 WINDOW NT의 REGISTRY에 저장된 DBACL7 PRIORITY 파라미터를 변경함으로써 DBACL7 PROCESS와 관련 THREAD의 PRIORITY를 변경할 수 있다. 한편 각 BACKGROUND THREAD의 PRIORITY를 하나씩 별도로 지정할 수도 있다. 하지만 모든 BACKGROUND THREAD의 PRIORITY를 동일한 데칼에 설정하는 것이 가장 바람직하다. 만약 보다 높은 PRIORITY를 갖는 THREAD가 CPU를 요구할 경우 그보다 낮은 PRIORITY를 갖는 THREAD는 CPU를 할당 받지 못하게 된다. DBACL7 PRIORITY 값을 조정하여 만점성에 문제가 발생하는 경우에는 이 값을 다시 원상태로 돌려놓으면 된다. 오라클의 CONFIGURATION PARAMETER는 REGISTRY 상의 \HKEY_LOCAL_MACHINE\SOFTWARE\ORACLE에 저장되어 있는데 REGISTRY EDITOR인 REGEDIT32.EXE 이용하여 편집이 가능하다. REGISTRY 편집 후에는 DBACL7의 SERVICE와 DATABASE를 다시 기동하여야만 새로운 값이 적용된다.

OPTIMIZATION

WINDOWS NT SERVICE의 최적화는 기본적으로 MAXIMIZE THROUGHPUT FOR FILE SHARING으로 설정되어 있다. 만약 시스템이 데이터베이스 작업을 위해서 주로 사용된다면 CONTROL PANEL/NETWORK/SERVICE에서 MAXIMIZE THROUGHPUT FOR NETWORK APPLICATIONS로 변경하면 된다.

WINDOWS NT SYSTEM CACHE는 디스크 I/O를 위한 BUFFER를 가지고 있다. 32MB RAM을 가진 시스템에서는 NT는 보통 4MB를 SYSTEM CACHE로 할당한다. ORACLE DATABASE는 독자적인 BUFFERING MECHANISM을 갖고 있기 때문에 디스크에 기록 SYSTEM CACHE를 거치지 않게 된다. REGISTRY의 \SYSTEM\LOCAL_MACHINE\Control\Section Manager\Memory Management에서 LargeSystemCache 값을 0으로 변경함으로써 WINDOWS NT DB의 SYSTEM CACHE가 0이 되어 오라클에 맞게 시스템을 최적화 할 수 있다.

SERVICES

ORACLE DATABASE와 관련된 NETWORK LISTENER 들은 WINDOWS NT의 SERVICE로 이루어져 있다. 사용되지 않는 SERVICE 들도 START 되어 있으면 시스템에 부하를 주게 되므로 이들을 STOP 시킴으로써 퍼포먼스를 증가시킬 수 있다.

TUNING DISK I/O

SEQUENTIAL I/O

REDO LOG FILE는 순차적으로 사용된다. 퍼포먼스를 증가시키려면 LOG FILE 들은 I/O가 필요 없어지지 않는 디스크에 주는 것이 좋다. 그리고 ARCHIVING TIME를 가동하면 각각의 LOG FILE이 서로 다른 디스크 상에 있어야지만 LGWR가 LOG FILE을 복사하거나 ARCHIVED TIME를 가 클러스터 REDO LOG FILE을 읽을 때 DISK CONTENTION이 일어나지 않는다.

RANDOM I/O

OLTP 또는 DECISION SUPPORT 환경하에서는 I/O는 디스크와 디스크 컨트롤러 상에 걸려서 골고루 일어나야 최고의 퍼포먼스를 얻을 수 있다. RAID 0 DISK ARRAY를 사용하는 것이 만능도 전하고 I/O LOAD BALANCING도 잘 이루어진다고 알려져 있다. DISK ARRAY CONTROLLER는 I/O의 LOAD BALANCE를 유지함으로써 디스크에 대한 접근접근이 가능하게 한다. ORACLE의 MONITOR FILE I/O는 ORACLE에 의해서 사용되는 모든 파일과 I/O 활동상황에 관한 통계정보를 제공한다. 한 디스크에 대한 전체 I/O는 READ/B와 WRITE/B의 합으로 계산한다. 각 디스크에 대한 이들 값을 계산한 다음 이들 디스크의 I/O CAPACITY와 비교하여 DISK CONTENTION 상태를 알 수 있다.

과도한 DISK FRAGMENTATION의 점검

파일 크기와 디스크 사용계획을 세울 때 앞으로 확장될 상황을 예상하여 점검하여야 한다. 그렇지 않으면 데이터파일이 불연속적인 EXTENT 들로 구성됨에 따라서 I/O 에 많은 시간이 걸리게 된다. DATABASE FRAGMENTATION은 퍼포먼스를 저하시키므로 EXPORT/IMPORT를 수행함으로써 FRAGMENTATION을 제거하여야 한다. 그리고 BLOCK AREA SIZE 파라미터가 적게 설정된 가운데 이 크기에 넘치는 소량작업을 실행하게 되면 I/O가 많이 발생하여 퍼포먼스가 저하된다.

메모리 튜닝

BGA 영역의 튜닝

BIBB에서 데이터를 읽는 것 보다 메모리에서 읽는 것이 수천, 수만 배 빠르기 때문에 메모리에 대한 튜닝은 매우 중요하다.

BGA 영역은 반드시 PHYSICAL MEMORY 내에 존재해야 한다. 그렇지 않으면 PACING에 의해서 퍼포먼스가 크게 저하된다. BQLBDB에서 BQBU BGA 영역을 통하여 BGA에 할당된 메모리를 확인할 수 있다. INIT.DBA 에 FREE_PAGE_BGA-TIME 라고 설정함으로써 BGA 영역을 항상 PHYSICAL MEMORY 상에 고정시킬 수 있다. 이는 DATABASE의 STATUP에 더 많은 시간을 요구하지만 DATABASE 사용중에 발생하는 CACHE MISS를 줄여 준다.

BUFFER CACHE의 튜닝

DB_BLOCK_BUFFER 파라미터는 시스템의 퍼포먼스에 큰 영향을 미친다. 여기에는 TABLE, INDEX, ROWSPACE BROWSE의 블록내용을 보관하고 있으며 HIT RATIO는 몇 퍼센트의 데이터가 이곳에서 사용되었는 지를 알려주는 값이다. 계산방법은 다음과 같다.

$$\text{HIT RATIO} = (\text{LOGICAL READS} - \text{PHYSICAL READS}) / (\text{LOGICAL READS})$$

STAT/STAT 스크립트를 수행함으로써 HIT RATIO를 알아낼 수 있다. 만약 이 값이 0.95 이하라면 DB_BLOCK_BUFFER 값을 늘려주도록 한다.

WINDOWS NT 상의 DBCLER의 DB_BLOCK_SIZE 값은 기본적으로 2K BYTE이다. 이 값은 WINDOWS NT의 PAGE SIZE와 같은 값이다. 가능한 DB의 PAGE SIZE의 정수배로 DB_BLOCK_SIZE를 설정하는 것이 바람직하다. 만약 응용프로그램이 OLTP 와 같은 작업 위주라면 DB_BLOCK_SIZE를 2K BYTE 로 하도록 하고 응용프로그램이 FULL TABLE SCAN과 같은 작업을 많이 한다면 DB_BLOCK_SIZE를 8K BYTE 정도로 키워주는 것이 퍼포먼스를 크게 증대시키는 요인이 될 수 있다.

사용자 메모리

사용자 프로그램에서 요구하는 메모리는 WINDOWS NT의 PERFORMANCE MONITOR나 FREE SIZE(WINDOWS NT RESOURCE KIT에 있음)를 통하여 알아낼 수 있다.

응용프로그램이 실행되기 전과 후의 남아있는 메모리 양을 검사함으로써 응용프로그램에서 사용하는 메모리양을 알아 낼 수 있다. DBCLER의 SHOW THREASB는 DBCLER UNDERING SET에 할당된 메모리를 사용한다. WINDOWS NT의 특정상 UNDERING SET의 크기는 최대 256MB를 넘을 수 없도록 제한되어 있다.

WINDOWS NT PACING

WINDOWS NT상에서 오다들을 운영하기 위해서는 최소 32MB 이상의 메모리를 확보하는 것이 좋다. 만약 BGA 영역에 너무 많은 메모리가 할당되어서 남 메모리가 너무 적게 되면 PACING이 발생하게 되며 과도한 PACING/BUFFING은 시스템의 퍼포먼스를 크게 저하시키게 된다. 이 문제를 해결하려면 보다 많은 PHYSICAL MEMORY를 갖도록 시스템을 확장하거나 아니면 BGA영역을 줄이거나, 동시 사용자수를 줄여야 한다.

PAGE FAULTS/BEC은 PROCESS의 PAGE FAULT 횟수를 보여준다. PAGE FAULT는 한 PROCESS가 자신의 PHYSICAL MEMORY내의 ADDRESS SET에 들어있지 않는 PAGE(VIRTUAL MEMORY PAGE)를 요구하는 경우에 발생한다. PAGE FAULT가 발생하더라도 해당 PAGE가 STANDBY LIST상에 있어서 MAIN MEMORY 상에 남아 있는 경우라면 시스템의 PAGING은 일어나지 않는다. 잘 튜닝된 시스템에서는 PAGING이 거의 일어나지 않아야 한다.

지원되는 BULKYNET PROTOCOLS

WINDOWS NT 상의 OS에서 지원되는 NETWORK PROTOCOL로는 TCP/IP, NAMED PIPES, SPX/IPX 등이 있다. CLIENT/SERVER 환경하에서는 TCP/IP를 NETWORK TRANSPORT로 사용하는 경우 가장 좋은 퍼포먼스를 얻을 수 있다고 알려져 있다.

BENCHMARKING TIPS

LOADING DATA

BULK/DATABASE를 사용할 때에는 DIRECT PATH OPTION을 사용하도록 한다.

PROGNOSTIC INTERFACE

DCI(DIAGNOSTIC CALL INTERFACE)는 PRODC에 비해서 DBACLE에 대한 보다 섬세한 제어가 가능하다. 만약 DCI를 사용한다면 DEFERRED LOGGING과 PARALLEL을 사용하면 좋는데, 이 OPTION은 CLIENT/SERVER 환경에서 NETWORK TRIP을 줄여줌으로써 퍼포먼스를 올려주게 된다.

CLIENT APPLICATION의 튜닝

INDEX 샘플을 빠르게 하려면 BOUT_AREA_SIZE 값을 키워주면 된다.

게다가 BOUT 작업과 BOUT/REDOE 작업 또한 PHYSICAL MEMORY상에서 일어나게 되므로 BISE I/O를 줄여주는 장점도 있다. DISCRETE_TRANSACTION_ENABLE=TRUE로 설정하면 길이가 짧고 분산 트랜잭션의 다른 트랜잭션의 퍼포먼스를 증대시켜 준다. 이 OPTION을 사용하면 트랜잭션이 COMMIT 명령을 만나기 전까지 모든 연결된 데이터가 저장되지 않고 기다리게 된다. 그리고 UNDO INFORMATION이 발생하지 않는다. STORED PROCEDURE, DATABASE TRIGGER, ADHOC PROCESSING 또한 NETWORK TRAFFIC을 줄여주는 등 퍼포먼스 증가에 큰 영향을 줄 수 있다.

DIAGNOSTIC TOOLS

EXPLAIN PLAN과 TYPDEF 튜닝리틀을 사용하여 OPTIMIZER의 EXECUTION PLAN을 점검할 수 있다. COST-BASED OPTIMIZER가 디폴트 이므로 이를 사용하는 것이 좋다. OPTIMIZER는 해당 OBJECT에 관련된 통계정보를 이용하여 최적의 EXECUTION PATH를 선정하여 준다.

ROLLBACK SEGMENTS

OLTP TRANSACTION의 특징은 서미전수는 많으나 TRANSACTION 하나 하나의 크기는 작은 데 있다. 이 때에는 각 TRANSACTION에 10K BYTES 정도의 매우 작은 ROLLBACK SEGMENT를 할당하는 것이 매우 효과적이다. 한편으로는 BENCHMARKING APPLICATION과 같이 큰 데이터를 다루는 경우라면 역시 ROLLBACK SEGMENT도 큰 것을 사용하는 것이 좋다.

새로운 DATABASE 생성

Bulletin no : 10082

1. 환경 설정

새로운 데이터베이스를 생성하기 위해서는 기존의 ORACLE HOME 디렉토리를 이용하는 경우 새로운 ORACLE_SID를 사용하여 데이터베이스를 생성한다.

1) ORACLE_SID 설정

새로운 DB ACCOUNT 를 생성하고 기존 ORACLE HOME 디렉토리에 있는 `profile(,.ora)` 파일을 새로운 ACCOUNT의 HOME 디렉토리에 복사하여 ORACLE_SID 부분을 다음과 같이 변경 한다.

예) 기존 환경 : ORACLE_SID=ORACLE7:export ORACLE_SID

새로운 환경 : ORACLE_SID=ORATEST:export ORACLE_SID

2) Parameter 파일 및 config 파일 생성

- 기존의 `$ORACLE_HOME/dbs/initORACLE7.ora` 파일을 이용하여 새로운 ORACLE_SID를 갖는 Parameter 파일을 만든다.

```
$ cd $ORACLE_HOME/dbs
```

```
$ cp initORACLE7.ora initORATEST.ora
```

```
$ cp configORACLE7.ora configORATEST.ora
```

- `initORATEST.ora` 및 `configORATEST.ora`의 내용중에 ORACLE7 부분을 ORATEST로 수정한다.

2. CREATING DATABASE

1) 기존의 DATABASE를 생성하면서 사용된 SCRIPT를 복사하여 수정하고 새로운 환경의 DATABASE를 생성

```
$ cd $ORACLE_HOME/dbs
```

```
$ vi cp2dORATEST.sql <- ORACLE7으로 되어있는 부분을
```

```
$ vi cp2dORATEST.sql ORATEST로 변경한다.
```

2) 수정된 SCRIPT FILE을 수행을 통해서 새로운 데이터베이스 생성.

```
$ cd $ORACLE_HOME/dbs
```

```
$ sqlplus / as sysdba
```

```
SQL> connect internal
```

```
SQL> startup nomount
```

```
Database is not mounted.
```

```
SQL> @cp2dORATEST
```

```
;
```

```
SQL> @cp2dORATEST
```

```
;
```

3) CATALOG VIEW 생성 및 PROCEDURAL OPTION INSTALL

```
$ cd $ORACLE_HOME/rdbms/admin
```

```
$ sqlplus / as sysdba
```

```
SQL> connect internal;
```

```

SQLDBA> DataLog
SQLDBA> DataProc
SQLDBA> DataExp
SQLDBA> create user scott identified by tiger
        default tablespace tools
        temporary tablespace temp
        quota unlimited on tools
        quota unlimited on temp;
SQLDBA> grant connect to scott;

```

3. 각 TOOL에 관련된 TABLE 생성

1) SQLPlus Install

```

$ cd $ORACLE_HOME/sqlplus/admin/help
$ SYSTEM_PASSWORD=$SYSTEM_MANAGER:export SYSTEM_PASSWORD
$ help1loc
$ cd $ORACLE_HOME/sqlplus/admin
$ sqlplus system/manager@pupbld
$ dbmsid scott/tiger

```

2) SQLForms30 Install

```

$ cd $ORACLE_HOME/forms30/admin
$ sqlplus system/manager
SQL> set compatibility v8
SQL> @loadtable
SQL> @loadindex
SQL> @loadview
SQL> @loadgrant

```

3) SQLMenu50 Install

```

$ cd $ORACLE_HOME/menu5/admin
$ sqlplus system/manager
SQL> set compatibility v8
SQL> @menuutabx
SQL> @menuvwex
SQL> @menuidxx
SQL> @menugrtx

```

4) SQLReporter11e Install

```

$ cd $ORACLE_HOME/sqlreport/admin
$ sqlplus system/manager
SQL> @srw_updproc
SQL> @srw_proc
SQL> @srw_lcom
SQL> @srw_smlid
SQL> @srw_cmls
SQL> @srw_grnt

```

인포믹스 STATIC PARTITIONING KNOCKOFFS

Bulletin no : 10177

인포믹스는 "데이터 파티셔닝(Data Partitioning)" 혹은 "테이블 파티셔닝(Table Partitioning)"을 최강점으로 내세우며 자사의 소위 "Blurred Nothing"이란 마케팅서를 오마칼에 비해 월등한 것으로 주장합니다. 이와 관련하여 오마칼은 인포믹스의 주장대로 "Blurred Black"에 포함되어 있는 것이 아니며 하드웨어의 어떠한 형태의 구성에도 최적화 될 수 있는 "Hybrid Architecture"를 지닌다고 할 수 있습니다. 이는 하드웨어가 "Blurred Nothing"인 경우는 물론 "Blurred Black"을 포함 "Blurred Anything", "Blurred Everything"을

모든 형태 configuration 이나 architecture를 오마칼은 지원할 수 있다는 뜻입니다. 반대로 인포믹스는 Blurred Black 구조의 지원을 하지 못하며(예, 오마칼의 DFB와 같은 기능이 없음), 이를 고모히 위장하고 있습니다.

게다가 인포믹스의 파티셔닝은 "Static Partitioning"으로서 다음과 같은 한계를 지닙니다.

1. Informal partitioning is done manually.

인포믹스의 컴퓨팅기는 전적으로 파티셔닝에 의존합니다. 또한 이같은 파티셔닝은 전적으로 DB 관리자가 책임지도록 되어있습니다. 따라서 성공적인 컴퓨팅의 수행을 위해서는 DB가 반드시 데이터베이스내의 데이터 분산 상태를 알아야 함은 물론 사용자의 데이터 액세스 패턴도 알고 있어야만 합니다. 이를 파악하려는 항상 전하는 것이며, 기존 디자인이 잘못되면 성능은 쇠져가 됩니다.

2. Static partitioning is inflexible.

인포믹스의 컴퓨터기는 파티션내에서는 불가능합니다. 만약 하나의 파티션내에서 데이터를 많이 가져오는 경우는 파티션내에서 컴퓨터기가 되는 경우와 비교하여 상당한 성능의 차이를 가져올 수 있습니다.

3. No partitioning Tools

인포믹스 사용환경에서는 위에서 설명한 대로 DB가 파티셔닝의 디자인을 최적화해야만 하는데도 데이터 distribution 이나 data access path에 관한 정보를 주고 모니터링하는 일을 제공하고 있지 않습니다.

4. Data distribution changes require repartitioning.

OLTP환경에서는 데이터가 항상 변합니다. 즉, data는 dynamic 반면 인포믹스의 파티셔닝은 static입니다. 즉, 파티션내의 데이터를 어떻게 배치하는가에 따라 즉, 파티션을 어떻게 나누는가에 따라 성능이 영향을 받게 되므로 데이터의 변용에 따라 최적의 성능을 내기 위해서는 그때그때 파티션을 새로 해 주어야 합니다.

5. Schema changes require repartitioning.

데이터베이스를 사용하다 보면 데이터의 structure를 바꾸어 주어야 할 때도 생깁니다. 이때도 역시 인포믹스는 파티셔닝을 다시 해주어야 합니다.

6. Hardware changes require repartitioning.

인포믹스의 파티셔닝 서미는 파티션별 프로세서의 할당으로 이루어집니다. 이는 하드웨어의 expansion 중시에도 새로이 파티셔닝을 해주어야만 한다는 것을 의미합니다.

7. Parallelism or Administration ?

인포믹스의 static partitioning은 결국 성능과 관리의 두가지 측면에서 사용자들을 어둠속에 빠지게 됩니다.

8. Repartitioning data means downtime.

파티셔닝을 새로 한다는 것은 결국 데이터베이스를 off line해야 된다는 것을 감안하면 dynamic data partitioning과 static(manual) partitioning과의 차이가 분명히 availability로 드러납니다.

(V3.0) BASE TABLES FOR MENU 5.0 BASE TABLES

Bulletin no : 10283

The BglMenu base tables store information for all menu applications installed in the database. These are needed to make application library files and check user security. If you do not have these tables, you will not be able to create or save an application.

In sqlplus as SYSTEM, run the following sql statement to see if these base table objects already exist:

```
1 select owner||'      '||pad(object_name,20,' ')||'      '||object_type
2 from sys.dba_objects
3 where object_name like 'MENU_%'
4 and (object_type='VIEW' or object_type='TABLE')
5 order by owner, object_type, object_name
```

SYSTEM	MENU_B_APPL	TABLE
SYSTEM	MENU_B_APPL_GRP	TABLE
SYSTEM	MENU_B_CIRCLE	TABLE
SYSTEM	MENU_B_GROUP	TABLE
SYSTEM	MENU_B_GRP_PRIV	TABLE
SYSTEM	MENU_B_INFO	TABLE
SYSTEM	MENU_B_OBJ_TEXT	TABLE
SYSTEM	MENU_B_OPTION	TABLE
SYSTEM	MENU_B_PARAM	TABLE
SYSTEM	MENU_B_PARAM_XREF	TABLE
SYSTEM	MENU_B_PRIV	TABLE
SYSTEM	MENU_B_PROCEDURE	TABLE
SYSTEM	MENU_B_REF	TABLE
SYSTEM	MENU_B_USER	TABLE

SYSTEM	MENU_U_APPL	VIEW
SYSTEM	MENU_U_APPL_GRP	VIEW
SYSTEM	MENU_U_CIRCLE	VIEW
SYSTEM	MENU_U_GROUP	VIEW
SYSTEM	MENU_U_GRP_PRIV	VIEW
SYSTEM	MENU_U_INFO	VIEW
SYSTEM	MENU_U_OBJ_TEXT	VIEW
SYSTEM	MENU_U_OPTION	VIEW
SYSTEM	MENU_U_PARAM	VIEW
SYSTEM	MENU_U_PARAM_XREF	VIEW

SYSTEM	MENU_U_PRTU	VIEW
SYSTEM	MENU_U_PROCEDURE	VIEW
SYSTEM	MENU_U_DEF	VIEW
SYSTEM	MENU_U_USAGE	VIEW

28 rows selected.

You need to run the base table scripts if you get no rows or any of the records missing for above query. In addition, if other base objects like synonyms, sequences, index, and grants were not created properly, it is likely that you will encounter problems and will need to re-install.

In the Oracle 800MB U7, if the tables have data type of CHAR instead of VARCHAR2 for the base table columns, then you need to re-install the base tables with VARCHAR2 data type.

These are the scripts you need to run to re-install the base tables.

You should log into sqlplus as user system, and start these scripts in the order specified. Note that if you have U7 database, then you would need to do

```
SQL> SET COMPATIBILITY 08
```

before you run them to ensure char vs varchar2 difference is resolved.

Also, before running the scripts, you should unload all the applications in the existing tables or have a back up of the database. Note that the scripts will drop all the related existing base objects first if there are any before creating them.

```
mmutake.sql  mmuldxr.sql  mmuuvwr.sql  mmuugtrr.sql
```

After the base table objects have been reinstalled correctly, you can put the mmu applications back from the unloaded sql files. For locations of these files, you should look into your installation guide or maybe you can do some kind of global file search (like find in unix) to look for these files. In unix, for example, these are usually under \$ORACLE_HOME/mmu50/admin directory.

(V3.0) FIELD를 이동할 때 자동으로

한글/영문 MODE를 변경하려면?

Bulletin no : 10288

1. emulator나 터미널에서 사용하는 한글/영문 toggle escape code값을 알아본다.

2. osdata 수집

α. device attribute : 사용하지 않는 attribute를 선택하여 한글, 영문 toggle하는 escape code값을 입력한다.

α. example : underline hold : 한글전환 code
 underline inverse : 영문전환 code

α. mapping attribute : product name, device video name를 확인한다.

α. example : BOLD-UNDERLINE : underline hold
 INVERSE-UNDERLINE : underline inverse

α. save한다.

3. form300에서 trigger생성 :

 : map.name에 field level trigger를 생성한다.

α. POST-FIELD : display_field(*map.name*,*BOLD-UNDERLINE*);

α. PRE-FIELD : display_field(*map.name*,*INVERSE-UNDERLINE*);

display_field에서 주는 attribute name은

osdata의 mapping attribute의 product attribute name을 주어야 한다.

(V3.0) FORMS3.0에서 USER EXIT이란?

Bulletin no : 10248

USER EXIT이란?

User Exit이란 사용자가 직접 만든 C나 Cobol등의 Subroutine을 Oracle Forms를 사용하면서 특정 목적을 위해 실행되어 질 수 있도록 하는 기능이다.

▶ 이용 형태

- 1) oracle database에서의 정보를 처리
- 2) BQLForms로부터의 정보를 파일에 기록
- 3) array data 처리
- 4) .fex file의 크기 축소등

▶ 작성 방법

- 1) 현재 디렉토리를 \$ORACLE_HOME/forms30/lib 로 위치한다.
- 2) gmxth scott/tiger
: scott user에 lpxth 데이터를 생성한다.
- 3) lq30 -to gmxth scott/tiger
: gmxth, lpx 또는 gmxth, fex file를 생성
- 4) runform30 -c vt220 gmxth scott/tiger
: gmxth를 띄워 "User Exit"부분에 새로 만든 userexit 프로그램의 function name을 등록하고 "Type"부분에는 C혹은 COB(대문자)를 등록 한다.
- 5) gmxth scott/tiger lpxth.c
: compiler와 link작업이 될 LPMIT definition을 담고 있는 lpxth.c를 생성
- 6) sqlform30.mk를 editor로 열어서 LPMIT부분에 userexit 프로그램의 object file명을 등록
예) LPMIT=lpxth.o
--> LPMIT=lpxth.o sample5.o
(여기서 등록한 userexit program은 sample5.o이며 실제 DB상에는 sample5.pc가 존재)
- 7) make -f sqlform30.mk sqlform30x
: sqlform30x와 runform30x를 생성

B) 수행시에는 `sqiform30x -c v1220 scott/tiger`로 들어가서

`DBMS_CRYPTO('userexit_mmm');`를 기술한다.

예) KEY-MATERIAL trigger에서

`user_exit('text');` --> text는 4)에서 등록한 function mmm

참고) COBOL user_exit 사용자는

`$ORACLE_HOME/for30/lib/cobol` 디렉토리에서 수행하며 1 - 5 의

단계는 같고

B) `procobxt.mk` 파일내의 `LIBKIT` 부분에 user_exit 프로그램의 object

파일명을 등록

7) `make -f procobxt.mk sqiform30x`

(V3.0) FORMS30에서의 LOCKING

Bulletin no : 10113

Forms내에서 Locking 사용

* FOR UPDATE Option은 Record가 Select되는 순간 Lock을 걸어 다른 User들이 Change를 못하게 한다.

예) 두 User가 같은 Record를 Query한 상태일때 한 User에 의해

```
SELECT *all database fields in the block  
FROM <table which the block is based on>  
WHERE rowid=<current row>  
FOR UPDATE OF <all database fields in the block>
```

가 수행되었다면, Select되는 Record들은 Lock이 걸려 다른 모든 User들은 Update를 할 수 없게 된다.

* 한 User가 Record를 Update 할때, BQWForm은 현재 그 Record가 Lock이 걸려있는지를 확인하기 위해 SELECT...FOR UPDATE문을 발생시키며, 만약 LOCK상태가 아니다면, 그 다음단계로 현재 Database 내의 값과 Form내의 값이 같은지를 비교하여, 같다면 Data를 Change하고, 값이 다르다면 다른 User에 의해 Change된 후 Commit된 것이므로 다시 Query하기를 요구한다.

(FMH-40854:Record changed by another user,Re-query to see change)

* 관련 trigger와 procedure

- ON-LOCK Trigger : Row가 Lock될 때마다 Fire.
- LOCK_RECORD Procedure : 현재 Record에 해당하는 Database의 Row를 Lock.
- ENTER_QUERY(FOR_UPDATE), EXECUTE_QUERY(FOR_UPDATE)
 - : 실제로 Fetch되는 record들만 lock을 건다.
- LAST_RECORD를 만나거나, ENTER_QUERY(ALL_RECORDS, FOR_UPDATE)는 Query에 만족하는 모든 Record를 Lock시킨다.

* PL/SQL을 사 한 Locking 제어

ON-ENTER/REBASE trigger로는 Record가 Lock이 걸려 있는지 확인할 수 없기 때문에, PL/SQL PACKAGE EXCEPTION을 사용한다.

```
DECLARE  
  dummy CHAR(1);  
  could_not_obtain_lock EXCEPTION;  
  PRAGMA EXCEPTION_INIT(could_not_obtain_lock, -54);  
BEGIN  
  SELECT 'X' INTO dummy  
  FROM emp  
  WHERE rowid=:emp.rowid  
  FOR UPDATE OF empno NOWAIT;
```

```

MESSAGE('Locked the Current Row Successfully');
EXCEPTION
WHEN could_not_obtain_lock THEN
    MESSAGE('Record is Already Locked. Please Try Later...');
RAISE form_trigger_failure;
END;

```

Locking으로 인한 예기치 않은 결과

- * 만약 한 Form내에 두개의 Block이 같은 Record를 Query하면서 동시에 Update 하였다면, Block Definition에서 더 높은 Sequence Number를 가진 Block의 Change가 Commit된다.
- * 한 Form에 두개의 Database Block A와 B가 있고 Block A에는 PRE-UPDATE Trigger(A.field='X')가 있고 Block B에는 PRE-INSERT Trigger(A.field='Y')가 있다. Block A는 Update작업을 Block B는 Insert작업을 수행 할때 Commit을 하면 Block A는 PRE-UPDATE가 fire가 되어

```

UPDATE A
SET column=A.field
WHERE rowid=current rowid = 'X' (the actual value)

```

를 수행하며, Block B는 다음 Block B로 옮겨가며 PRE-INSERT Trigger를 Fire한다. 따라서 block A에 A.field를 다시 'Y'로 Update하려고 하면

ORA-40854: Record changed by another user ...

가 나온다.

- * NUMBER(5,2)인 Column을 가진 Table K를 Form에서 그 Field를 Access 해서 10.243을 입력하고 Commit을 한 후 다시 Edit하면 ORA-40854 Error가 생긴다. 왜냐하면 Database에 저장된 10.24라는 값과 Form상의 10.243이 서로 일치하지 않기 때문이다.

- * 두 User A와 B가 동시에 같은 Record를 Query하고 있을때, User A가 Modify를 하고 Commit을 하면, User B가 Modify하려던 ORA-40854 Error가 발생한다. 이때 User A가 다시 그 Record를 Modify하려고 하면 "Attempting To Reserve Record For Update/Delete"가 나타난다. 이는 User B가 Modify를 시도했을 때 이미 BLOCKY...FOR UPDATE문이 실행되었기 때문이다. B는 A가 Commit을 수행했으므로 Lock을 걸 수 있었으며 Database에 저장된 값과 Form의 값이 불일치하여 Error를 발생시켰다.

- * calling form F00H_A와 called form F00H_B가 있을때, user A가 한 record를 수정한 후 CALL F00H_B를 한 상태에서 F00H_A에서 수정된 그 record를 다시 수정이 가능하다. 이때 F00H_B가 commit을 수행하면 F00H_A가 걸어놓은 lock이 clear됨을 주의해야 한다. 따라서 그 이후에 다른 user가 그 record를 update할 수 있게 된다.

(V4.5,3.0) FRM-40735 ERROR 에 관 하 여

Bulletin no : 10277

I. Introduction

FRM-40735 Error는 DBACLE Form가 실행중Unexpected Error를 만나면 발생하는 Error이다. 이 Error는 대부분 다음과 같은 형태로 표시된다.

FRM-40735: Raised An Unhandled Exception

은 Unexpected Event 가 발생하였을 때 수행되고 있던 Trigger명을 나타낸다. Trigger에 기술한 내용내에서 발생한 Unexpected Event를 처리 할 수 없을 때 DBACLE Form은 사 자에게 FRM-40735 Error를 표시해 주는 것이다.

은 Trigger수행중에 발생한 Unexpected Event 의 이름을 나타내준다. 이것은 PL/SQL에 미리 선언되어있는 Exception일 수도 있고, User가 Defined Exception이거나, Server나 Application에서 나오는 것일 수도 있다.

FRM-40735 Error를 만나게되면, [Display Error] Key를 눌러 자세한 Error Message를 확인하여야 한다. 여기에서는 이 Error를 발생하는 몇가지의 조건과 해결방법에 대해 살펴본다.

II. FRM-40735와 함께 나타나는 일반적인 FRM-Xxxx Error

FRM-40831: Truncation Occurred: Value Too Long For Field .

원 인

Item이나 Parameter에 값을 지정할 때, 가능한 길이보다 긴 값을 준 경우에 발생.

조치사항

해당 값을 지정할때 Item이나 Parameter의 Maximum Length를 늘려준다.

Item에 값을 지정하는 경우에는그 Item의 Query Length도 늘려준다.

더 자세한 정보는 PL/SQL에서의 Value_Error Exception부분을 참조한다.

III. FRM-40735와 함께 나타나는 일반적인 PL/SQL Exception들

a. Unhandled Exception이란?

Trigger가 수행중에 PL/SQL에 정의되어있는 Exception 사항을 만나면 Program은 이미 정의되어있는 PL/SQL Exception을 처리하는 Routine으로 넘어지게 된다. 만약 해당 Exception을 처리하는 Routine이 없다면, 사용자는 FRM-40735 Unhandled Exception Error를 만나게 된다.

α. Value_Error

계산시, 변환시, 절단시, 제약사항 등에 관한 Error가 발생한다.

원 인

1. Char Type의 Out Parameter를 가진 Stored Procedure를 호출할 때
2. Subprogram을호 할 때 Parameter간의 Type이 맞지 않을 때
3. Item이나 Parameter에서 값을 수 있는 값보다 큰 값을 지정하려 할 때
4. DBMS Error 4, 5, 8, 9 이전 Version에서 Built-in의 Set_Max_Item_Property나 Set_Max_Item_Property에서 Label Property를 바꾸려 할때

조치사항

1. Out Parameter의 Data Type을 Varchar2로 변경한다.
만약 단지 한개의 값을 Return하는 Procedure라면, Procedure가 아닌 Function으로 바꾸어
생성하라. Function은 Char Type을 Return할 수 있기 때문이다.
사용되는 Item이나 Parameter의 Datatype이 Subprogram의 것과 같거나 호환되는 것이어야 한다.
3. 해당 Item이나 Parameter의 Maximum Length를 증가시켜라.
만약 Item에 값을 지정하는 것이라면, 해당 Item의 Query Length도 증가시켜준다.
PL/SQL Variable에 값을 지정하는 경우는 Variable의 Size를 증가시켜준다.
Variable Character나 Fixed Character를 정의하는 Format은 다음과 같다.

Char(Maximum Length)

Varchar2(Maximum Length)

Maximum Length를 지정하지 않으면 Default값은 10이 지정된다.

Example

A. PL/SQL Program Unit에서 Variable에 값을 지정하는 경우

Declare

My_String Varchar2;

Begin

My_String := 'Abcdxyz';

End;

위의 경우는 Built-in에 Value_Error가 발생한다.

다음과 같이 변경해준다.

Declare

My_String Varchar2(7);

Begin

My_String := 'Abcdxyz';

End;

B. PL/SQL Program Unit에서 Item에 값을 지정하는 경우

Item Name: Empno

Maximum Length: 4

Query Length: 4

:Empno := 12345678;

위의 경우는 Built-in에 Value_Error 발생한다.

다음과 같이 연결해준다.

```
Item Name:      Kypno
Maximum Length: 8
Query Length:   8
:Kypno := 12345678;
```

4. Label Property를 바꾸기 위해 Get_Menu_Item_Property 나 Set_Menu_Item_Property Built-in을 꼭 사용해야 한다면 4.5.B.K도 Upgrade해야 한다.

Cursor_Already_Open

권 련

이미 Open되어있는 Cursor를 Explicit하게 다시 Open하려할 때 발생한다.

일반적으로 대부분 Cursor For Loop문에서 Open 문을 사용한 경우 발생한다.

조치사항

Cursor를 Open하는 문장을 기술하기 전에 Cursor가 이미 Open되어 있는지 확인하기 위하여 %IBOpen을 사용한다.

Example

```
IF C1%IBOpen Then
    ...
Else
    Open C1;
End IF;
```

Invalid_Cursor

권 련

Open되어 있지 않은 Cursor를 Close하려고 한 경우 발생한다.

조치사항

1. Close하기전에 Open되어 있는지 확인한다.
2. Cursor를 Close하기전에 이미 Open되어 있는지 확인하기 위하여 %IBOpen을 사용한다.

Example

```
IF C1%IBOpen Then
    Close C1;
Else
    ...
End IF;
```

No_Data_Found

권 련

SELECT... INTO ... 문장에 대한 결과가 없을 때 발생한다.

조치사항

1. BQLPlus에서 같은 Select문장을 실행하여, 실제 결과가 나오는지 확인한다.
2. 이 Exception을 잡아낼수 있는 Exception 처리문장을 삽입한다.

Not_Logged_On

원 인

DATABASE에 대한 Call을 수행하고자 할때, ORACLE에 현재 Login되어 있지 않은경우에 발생 DATABASE에 Backlog이 쌓여지지 않았거나, Backlog이 Time Out에 걸렸거나, Connection이 끊긴 경우에도 발생할 수 있다.

조치사항

Server에 다시 Login한다.

Too_Many_Rows

원 인

SELECT... INTO ... 문장에서 Return되는 결과가 한건 이상일때 발생한다.

조치사항

1. 한건의 Record가 Return되기를 원하는 경우라면 BULK문장을 수정한다.
2. 한건 이상의 자료가 Return되기를 원하는 경우라면 Explicit한 Cursor를 사용한다.

IV. FRM-40735와 함께 나타나는 일반적인 ORA-Xxxxxx Error

ORA-604: Error Occurred At Recursive SQL Level %S

원 인

DBACLE Form Application에서 Server에 대하여Cursor를 Open하는 BULK문을 실행하고자 하는데, Server에서는 Cursor를 Open할 수 있는 Maximal Number에 도달한 경우에 발생한다.

조치사항

1. ORA-00604 Error는 DATABASE Parameter중 Open_Cursor의 값을 증가시켜주면 해결된다.

* Cursor란 무엇인가?

Cursor는 BULK문장을 실행하기위해 ORACLE이 사용하는 Memory의 영역을 말한다.

DATABASE에서 갖는 Open_Cursor의 Default값은 50이다.

Maximal값은 User가 사용하는 System 에 따라 결정된다.

User의 환경에 따라 Open_Cursor의 적정값을 설정할 필요 있다.

주의 : Open_Cursor는 DBACLE Server DATABASE의 Parameter이지 DBACLE Form에만 해당하는 Parameter가 아니다.

2. Open_Cursor의 수를 증가시키는 것이 용이하지 않을 때에는 DBACLE Form Application에서 사용하는 Cursor의 수를 줄인다.

* DBACLE Form Application에서 사용하는 Cursor수의 절감

가능하다면 Implicit Cursor대신에 Explicit Cursor를 사용한다. 이것은 DBACLE Form가 종료하면서 Cursor가 Close될 때 Memory Backlog가 최소화되므로 개발자가 좀더 많이 관리할 수 있게 해준다.

Example

다음의 문장을 implicit Cursor를 Open한다.

```
SELECT Empno
FROM Emp
WHERE Empno = :Empno;
```

만약 Form에서 Cursor Mode Property가 Dyn으로 되어있다면, 이 Cursor는 이 BQL문을 사용한 Form이 Close될때까지 남아있게 된다.

User는 위의 BQL문을 Explicit한 Cursor를 사용하도록 문장을 수정할 수 있다.

```
Declare
    Cursor My_Cursor Is
        SELECT Empno
        FROM Emp
        WHERE Empno = :Empno;
Begin
    Open My_Cursor;
    Fetch My_Cursor INTO :Empno;
    Close My_Cursor;
End;
```

이 BQL문에서는 Close문을 수행하면 Cursor는 Close된다.

기본적으로 DBACLE Form은 DATABASE에 각각의 BQL문마다 implicit한 Cursor를 지정한다. 이것은 Processing을 향상시키기는 하지만, Buffer의 Backlog시에 DBACLE Form가 더 많은 Cursor를 사용하게 한다. Buffer Parameter 중Optimize를 "No"로 설정하면, DBACLE Form는 INSERT, UPDATE, DELETE, SELECT FOR UPDATE Statements에 사용한 Cursor를 재사용한다. 이렇게 사용하면 Application에서 Cursor의 수를 적게 사용하게 되지만, Performance는 저하된다.

ORA-942: Table Or View Does Not Exist

원 인

해당 Object가 없거나, 잘못 Reference했거나, 해당 Object에 대한 권한이 없을 때 발생

조치사항

1. SQLPlus로 Login하여 해당 Table이나 View의 Description을 확인한다.
만약 DBA-B420이 다시 나온다면, 해당 Object에 대한 권한을 부여 받았는지 확인한다.
2. Object명을 다음과 같이 기술한다.
Schema_Name.Object_Name.
3. Object에 대한 Synonym을 생성하고, 해당 User에 적절한 권한을 준다.

ORA-1000: Maximum Number Of Open Cursors Exceeded

원 인

DBA-B04를 참조

조치사항

DBA-B04의 조치사항 참조

원 인

해당 Sequence가 없거나, 작업할 수 있는 권한이 없는 경우에 발생한다.

조치사항

1. 정확한Sequence명을 기술한다. 다음의 Data Dictionary View에서 가능한 Sequence를 확인한다.

```
User_Sequence
```

```
Ma_Sequence
```

```
All_Sequence
```

2. 해당 Sequence에 대한 Select권한이 있는지 확인한다. 해당 Sequence로 부터 다음 권호를 받아오려면 Select권한이 반드시 있어야 하기 때문이다. 이 사항을 확인하기 위해서 SQLPlus에서 다음 문장을 실행한다.

```
SQL> SELECT Sequence_Name, Sequence_Downer FROM All_Sequence
```

3. 해당 Sequence에 대해 Public Synonym을 생성한다. 만약 해당 Sequence가 다른 User가 생성한 것이라면 반드시 다음과 같이 기술하여 사용하여야 한다

```
SELECT ,Nextval INTO :Block_Field_Name FROM Dual;
```

Sequence의 Owner는 다른 User들이 Schema_Name을 기술하지 않고 해당 Sequence를 사용할 수 있도록 다음과 같이 해당 Sequence에 대해 Synonym을 생성할 수 있다.

```
SQL> Create Public Synonym FOR Schema_Name,Sequence_Name>
```

다른 User들은 위에서 생성한 Synonym을 다음과 같이 기술하여 사용할 수 있다.

```
SELECT ,Nextval INTO :Block_Field_Name FROM Dual;
```

4. 만약 Sequence가 Remote DATABASE에 있는 것이라면, 다음과 같은 방법으로 사용할 수 있다.

- 1> Remote Sequence에 대해 DATABASE Link명을 감출 수 있는 Synonym을 생성한다.

```
SQL> CONNECT System/Manager
```

```
SQL> CREATE Public Synonym Remote_Seq_Syn
```

```
2 FOR Remote_SeqDB_Link;
```

ORACLE Form Application에서는 DATABASE Link명 없이 Synonym을 사용한다.

```
SELECT ,Nextval FROM Dual;
```

- 2>ORACLE Form에서 Sequence와 Link명을 주어 사용한다.

```
SELECT ,Nextval FROM DualDB_Link
```

ORA-106552: DDE Exception Dmlerr_Dataacktimeout

원 인

DDE Data Request가 Timed Out에 걸린 경우에 발생한다.

조치사항

1. DDE, Execute Built-in에서 사용하는 Timeout Parameter의 값을 증가시켜준다.
2. DDE, Dmlerr_Dataacktimeout Exception에 대한 처리문장을 기술해 준다.

ORA-106556: DDE Exception Dmlerr_No_Conv_Established

원 인

DDE Conversion을 하려는 시도가 실패한 경우 발생한다

조치사항

1. DDE, Initiate 에서의 Service와 Topic Name 을 Checking한다.

Excel의 경우

Service Name : *Excel*

Topic Name : Spreadsheet Name 예를 들어 'C:\Yact.Klx'

Word의 경우

Service Name : *Word*

Topic Name : Document Name 예를 들어 'C:\Yact.Doc'

2. DDE, App_BugIn 에서 Open하고자 하는 File명을 Checking한다.

Excel의 경우

```
AppId := DDE.App_BugIn('C:\Microsoft\Excel\Excel.Exe C:\Yact.Klx',  
                        DDE.App_Made_Normal)
```

Word의 경우

```
AppId := DDE.App_BugIn('C:\Microsoft\Word\Word.Exe  
                        C:\Yact.Doc', DDE.App_Made_Normal)
```

ORA-304500: Failed ORA_FFI Call

원 인

지정된 DLL File의 Function Call을 할 수 없을 경우에 발생한다.

조치사항

1. 해당 DLL File이 ORACLE_HOME 이나 DDE Path에 포함되어 있는지 확인한다.
2. 32-Bit Application을 사용할 경우 32-Bit DLL인지 확인한다. 마찬가지로, 16-Bit Application에서는 16-Bit DLL인지 확인한다.

V. General Information

FMH-40735 Error가 발생하면, User는 때로는 해당 Exception을 잡아서 처리 할 수 있는 문장을 기술하여야 한다. 이 Exception Handler는 Error가 언제 발생하든지 같은 처리를 수행할 수 있다. Ora_FFI 와DDE에 관한 Package를 사용할 때는 Package안에 해당 Exception에 관한 Handler를 기술해 주는 것이 좋다.

(V4.5.5.X) SUPPORT FOR VBX AND OCX CONTROLS

Bulletin no : 10404

**** Summary of Current and Planned Support : 더 자세한 사항은 Release Notes를 참조하십시오.**

1. Release 1.3 Win95/NT

DCX - Limited DCX support for 32 bit DCX controls (see below)

VBX - No VBX support

2. Release 1.3 Win 3.1

DCX - Limited DCX support for 16 bit DCX controls (see below)

VBX - Continued VBX support with bug fixes and increased stability

3. Release 2.0 Win95/NT

DCX - Full support including events for 32-bit DCX controls. Automatic generation of PL/BQL "wrapper" packages for methods and events. Ability to set properties at design time.

VBX - Open, edit and save a form containing a VBX. No support for modifying an existing VBX item or creating a new VBX item. NOTE: You will not be able to run a form containing a vbx under a 32 bit runtime. This support is provided only to allow you to edit an existing form to compile and run it under a 16 bit runtime.

4. Release 2.0 Win 3.1

NOTE: current plans for Win 3.1 are for a runtime only (no designer) in 2.0. The inclusion for creating a 16 bit DCX in the 32 bit designer is still under review.

DCX - Support including methods for 16-bit DCX controls. May exclude ability to set properties at design time (see 1.3).

VBX - Run an upgraded form containing a VBX. No support for modifying an existing VBX item or creating a new one.

5. Corrections to the Release Notes

The release notes for 1.3 contain some errors:

4.1B.1. DCX Support

We have limited DCX support in Developer/2000 Release 1.3. This will be expanded to full support in Developer/2000 Release 2.0. In Forms 4.5.7 you will be able to:

o Insert an DCK into an DLE container

o Bring up the DCK's property sheet to set properties

by invoking the Edit action from the right-mouse-menu

>>>This is incorrect: although one attempt to set properties in the designer, they are not retained at runtime! See the White Paper for details on setting properties.

o Calling the DLE-style automation interfaces at runtime

Until Release 2.0, you will not be able to :

o Show and set the DCK properties in the property palette

>>>In 2.0 you will (probably) set the properties through the DCK's own dialog, not the property palette

o Programmatically get and set properties at runtime

>>>This is incorrect: in fact you MUST get and set properties at runtime in 1.9. See the White Paper and the sample.

o Receive events at runtime

>>>See the White Paper and the sample for a workaround for this restriction.

o Call DCK-style methods.

>>>In many cases this is not a problem because the DCK exposes an DLE automation interface

(V4.5.7 LATER) SQL*NET V2 SUPPORT

Bulletin no : 10405

From Release 1.3.2 of Developer/2000, we are only including SQL*Net V2 on the product CD as a corporate policy.

Note that Release 1.3.2 supports BSNB 7.3 which doesn't support SQL*Net V1.

(V4.5) CDE2 AND RDBMS

Bulletin no : 10008

Does anyone out there know that what is the latest release version certified with the cde2 tools? possible hardware will be hp/ux or sun solaris 2.4...

There's a lot of confusion about this issue. Let us try to shed some light on the subject. There are actually two separate issues involved in this question.

They are:

- (a) Can the CDE tools connect to a particular version of the database?
- (b) Can the CDE tools coexist in the same \$ORACLE_HOME as a particular version of the database?

Note that these are *not* the same question.

The answer to question (a) is almost always "yes". The CDE tools can connect to *any* Oracle 7.X database. If the proper tables have been installed in the database, the CDE tools can save forms/reports/documents in any 7.X database without any problems.

The answer to question (b) is "it depends".

On UNIX, the most current version of the CDE tools is the CDE1 Production 2 bundle, with Forms 4.0.12 and Reports 2.0.13. On most ports, this bundle was built using the Toolkit and Core Libraries from the 7.0.13 RDBMS version. This means that the only RDBMS version with which the CDE tools are *guaranteed* to coexist in the same \$ORACLE_HOME is the 7.0.13 RDBMS version.

Now, *some*, *NOT NOT ALL*, of the product lines have certified their particular release of CDE with later versions of the RDBMS. For example, the Sun BBU has said that they support installing CDE1P2 in the same \$ORACLE_HOME as the 7.1.3 RDBMS, but the HP BBU has specifically said that this combination is *not* supported. I know of no BBU who has said that you can install CDE1P2 and the 7.1.4 RDBMS in the same \$ORACLE_HOME.

My personal recommendation is that you *never* install the CDE tools and the RDBMS in the same \$ORACLE_HOME. Instead, install them in separate \$ORACLE_HOME's, and set the \$TWO_TASK environment variable to control which database you are connecting to. This strategy has the following benefits:

- (1) There is no performance penalty, since UNIX always uses a two-task architecture anyway.

- (2) It is possible to upgrade the CDE tools version without being forced to upgrade the BOMB at the same time.
- (3) It is possible to upgrade the BOMB version beyond the latest version supported by the CDE tools. (For example, using this strategy it's no problem to have CDEIP2 and BOMB U7.1.4 on the same machine.)
- (4) It is possible to upgrade the BOMB version without being forced to upgrade the CDE tools at the same time.
- (5) This strategy completely avoids problems in which applying a BOMB patch will cause problems in the CDE tools, and vice versa.

Anyway, I hope this clears things up.

~

(V4.5) FORMS4.5 NEW FEATURE

Bulletin no : 10013

New Features

The following table summarizes new features in Oracle Forms 4.5 in six areas: Designer Interface, object and code reusability, GUI control, PL/SQL Interface, database integration, and MS Windows extensibility.

Designer Interface

Object The Object Navigator presents a hierarchical view of Navigator as entire Oracle Form design session, showing all form, menu, and library modules in an easy-to-manipulate list. For more information, refer to the Oracle Forms Developer's Guide, Ch. 2, "Working with the Design Tools."

Properties The Properties window displays an object's properties. Window is a list format, and replaces the Release 4.0 property sheets. For more information, refer to the Oracle Forms Developer's Guide, Ch. 2, "Working with the Design Tools."

Graphical The graphical layout Editor lets you size, position, layout Editor and edit stacked canvases in the same layout Editor window with the content view. For more information on editing windows and canvases, refer to the Oracle Forms Developer's Guide, Ch. 11, "Working with Windows and Canvases."

Graphical The graphical Menu Editor allows you to create Menu Editor pull-down menus quickly, working from a visual display of your menu. For information, refer to the Oracle Forms Developer's Guide, Ch. 22, "Basic Menu Design."

PL/SQL The PL/SQL Editor provides a common editing Editor environment for both triggers and PL/SQL subprograms. The PL/SQL Editor lets you enter and compile PL/SQL code and view and resolve any errors. A global search and replace function allows you to find all occurrences of any string in trigger or program unit code. For more information, refer to the Oracle Forms Developer's Guide, Ch. 2, "Working with the Design Tools."

Object and Code Reusability

Copy/Paste The Copy Properties command lets you copy a list of Properties properties from one object or a group of selected objects. To apply these properties to another object or group of objects, use the Paste Properties command.

For more information, refer to the Oracle Forms Developer's Guide, Ch. 4, "Setting Object Properties."

Property You can save a collection of current property settings *Classes* as a named property class, then have objects on the class, making local overrides as necessary. For more information, refer to the Oracle Forms Developer's Guide, Ch. 5, "Building Objects and Code."

Object Groups Object groups allow you to share common objects among many modules. You define an object group by naming a group of functionally related objects. For example, a calendar function might include a window, a content canvas, several blocks and their items, as well as related triggers. By creating an object group, you can include this calendar window in every form in your application with a single drag-and-drop operation. For more information, refer to the Oracle Forms Developer's Guide, Ch. 5, "Building Objects and Code."

Copy/Run drag-and-drop Copy/Reference function new Reference allows referencing modules in the file system as well as Modules in the database. For more information, refer to the the File Oracle Forms Developer's Guide, Ch. 5, "Building Objects and Code."

Source You can use Oracle Forms 4.5 with PNCB (Polytron Control Version Control System) for source control. For more information, refer to online Help for the File->Administration command.

GUI Control

Responding You may want to write code that responds to mouse to Mouse events, such as a single or double click. Now you can Events determine where a mouse event occurs, whether it is a single or double click, and whether the mouse has entered or left an item, canvas, or window. For information, refer to Oracle Forms Advanced Techniques, Ch. 8, "Responding to Mouse Events."

Dynamic List Dynamic List items allow you to programmatically item change list item elements, including deriving values and labels from the database at runtime. For more information, refer to the Oracle Forms Developer's Guide, Ch. 8, "Defining Items."

Combo Box Combo box list items, which combine the functionality Style List of a text item and a poplist, are supported in Item Version 4.5. For more information, refer to the Oracle Forms Developer's Guide, Ch. 8, "Defining Items."

Toolbar You can create a horizontal, vertical, or MDI window Support toolbar as a canvas and assign it to a window. For more information on editing stacked canvas-views, refer to the Oracle Forms Developer's Guide, Ch. 11, "Working with Windows and Canvas-views." But Cursor In Oracle Forms 4.5, you can programmatically control Style the cursor shape using the new Cursor_Style property. For more information, refer to the Oracle Forms Developer's Guide, Ch. 8, "Basic Form Design."

Dynamic In Oracle Forms 4.5, you can dynamically change Forest Mask forest masks using the Forest_Mask property Support with the GET_ITEM_PROPERTY and SET_ITEM_PROPERTY built-ins. For more information, refer to the Oracle Forms Reference Manual, Vol.2, Ch. 5, "Properties."

Maximize You can now programmatically maximize or minimize Window a window by calling the `SET_WINDOW_PROPERTY` built-in, using the new `Window_State` property, with "state" settings of `MINIMIZED`, `MAXIMIZED`, or `NORMAL`.

For more information, refer to the *Oracle Forms Reference Manual*, Vol. 2, Ch. 5, "Properties."

Display You can specify for each form which window should Console display the console (message line and status line), using the form-level property `Console Window`.

For more information, refer to the *Oracle Forms Reference Manual*, Vol. 2, Ch. 5, "Properties."

No Root A form is not required to have a root window in Oracle Window Form 4.5.

Required For more information on editing windows, refer to the *Oracle Forms Developer's Guide*, Ch. 11, "Working with Windows and Canvas Views."

Display Single You can now independently control the number of and `Multiple` instances of each item displayed in a multi-record item instances block, using the new `Items Displayed` property. In a Block For more information, refer to the *Oracle Forms Reference Manual*, Vol. 2, Ch. 5, "Properties."

Base Multiple You can now have more than one item corresponding items on the to the same base table column, using the new `Item Base Column` property, `Mirror Item`. For more information, refer to the *Oracle Forms Reference Manual*, Vol. 2, Ch. 5, "Properties."

Alter Default You can alter the default navigation sequence at the Navigation item level using the new `Next Navigation Item` and `Previous Navigation Item` properties. For more information, refer to the *Oracle Forms Reference Manual*, Vol. 2, Ch. 5, "Properties."

Display You can create a named visual attribute and apply it to `Current Record` the current row using the `Current Record Attribute Is Color` property.

For more information, refer to the *Oracle Forms Reference Manual*, Vol. 2, Ch. 5, "Properties."

PL/SQL Interface

Source-Level The source-level debugger allows you to debug Debugger PL/SQL code at runtime, including setting breakpoints, single stepping, examining and depositing values, and examining the call stack. For more information, refer to the *Oracle Forms Developer's Guide*, Ch. 21, "Debugging Applications."

Editing Through Oracle Forms 4.5, you have access to a new Environment editing facility for creating and modifying stored for Stored procedures and database triggers.

Procedures For information, refer to *Oracle Forms Advanced Techniques*, Ch. 2, "Stored Procedures and Database Triggers."

Incremental In Oracle Forms 4.5, the Generate utility will recompile compiling only code which has been changed. You can use a new `COMPILE_ALL` argument to override this default functionality when you make a change you know will affect many of your program units. For more information, refer to the Oracle Forms Reference Manual, Vol. 1, Ch. 1, "Compiling and Options."

Trigger Using a new Execution Style trigger property, you will Execution be able to specify whether you want the lowest-level Style trigger to override any higher-level triggers at higher levels (Oracle Forms 4.0 default behavior), or if you want the lower-level trigger to fire before or after higher-level triggers fire.

For more information, refer to the Oracle Forms Reference Manual, Vol. 2, Ch. 5, "Properties."

Calling In Oracle Forms 4.5, you can call foreign functions `Foreign` (functions written in any 3GL programming language, Functions such as C) from a PL/SQL interface.

For more information, refer to Oracle Forms Advanced Techniques, Ch. 13, "PL/SQL Interface to Foreign Functions."

PECS The Performance Event Collection Service (PECS) is a tool you can use to locate performance problems, and to measure resource usage and test coverage.

For more information, refer to Oracle Forms Advanced Techniques, App. C, "Analyzing Performance with PECS."

Database Integration

Multiple Oracle Forms applications can now open multiple Open Forms forms simultaneously. Complex applications can be developed as a collection of modular forms, which may cooperate at runtime. Any number of these forms may be active at once on the desktop.

For information, refer to Oracle Forms Advanced Techniques, Ch. 5, "Multiple-Form Applications."

Multiple In Oracle Forms 4.5, you can establish a separate Backend per transaction environment for each form in a Connect multiplex-form application, so an independent form can focus commits even if other forms have pending transactions.

For information, refer to Oracle Forms Advanced Techniques, Ch. 5, "Multiple-Form Applications."

Application Oracle Forms 4.5 allows you to implement user Security security using the Oracle? Server application security Using Oracle? roles stored in the Oracle? data dictionary.

Notes For information, refer to the Oracle Forms Developer's Guide, Ch. 23, "Customizing Menus."

Insert/Update You can use the new Update Changed Columns Only property to save on network traffic by specifying that changed only changed columns should be included in UPDATE Columns statements.

For more information, refer to the Oracle Forms Reference Manual, Vol. 2, Ch. 5, "Properties."

MS Windows Extensibility

VBK Controls VBK controls, such as knobs and sliders, are available from several third-party sources and can now be included in Oracle Forms applications as custom items. For more information, refer to Oracle Forms Advanced Techniques, Ch. 11, "VBK Controls."

OLE2 Object Linking and Embedding (OLE) allows MS Container Windows applications to create compound documents, support documents that include objects from other Windows applications. Oracle Forms 4.5 provides OLE container support, allowing you to, for instance, display an Excel spreadsheet in an Oracle Forms application. For more information, refer to Oracle Forms Advanced Techniques, Ch. 10, "Object Linking and Embedding (OLE)."

OLE2 PL/SQL built-in can interrogate and call the methods Automation of any registered OLE2 server that supports automation. For more information, refer to Oracle Forms Advanced Techniques, Ch. 10, "Object Linking and Embedding (OLE)."

BDE Support If you use the MS Windows Software Developer's Kit (SDK), you can use the new Window_Handle property to programmatically obtain the Windows handle for any Oracle Forms object so you can refer to it in user exit code. For more information, refer to the Oracle Forms Reference Manual, Vol. 2, Ch. 5, "Properties."

B4E Limit In Oracle Forms 4.5, the B4E limit on record size has lifted been removed, which particularly affected applications that displayed large files. There is now essentially no limit on record size for MS Windows.

(V4.5) CDE2 의 ON-LOCK 기능

Bulletin no : 10048

Subject: CDE2의 ON-LOCK기능

현재 PowerBuilder와 PDBM간의 OLEDB연결경시 다음 사항을 추가 합니다.

1. 동시에 2명의 사용자가 같은 record를 update할때.

forward5 - 사용자A의 commit여부에 관계없이 수정하였을때,
"이미 라 사용자에게 의해 수정되었다"거나,
"라 사용자에게 의해 수정lock이 걸려 있다"는 message제공.

PR40 - 라 사용자의 작업여부에 관계없이 최종 사용자의 update된 값이
무조건 입력되어 자료의 오류가 발생한다.

PowerBuilder PR사용자들의 이전.

- forward5처럼 lock이 발생하면, lock이 필요없는 업무에서는 오히려 문제가 많다. (화면에서만 수정하고 commit하지 많은 상태에서 아무런 작업을 하지 않고 있으면, 같은 데포트를 사용하는 라 모든 사용자는 lock이 발생하여 업무를 할수 없다).

해결책.

- forward5의 forward-level trigger에 on-lock를 이용하여 내용을 NULL:만 입력하여 작업하면, PowerBuilder과 같은 lock이 발생하지 않는 작업처리를 할 수 있다.

- 그러므로, 기본적으로 동시사용자가 많은 경우 forward5처럼 lock 기능을 제공하며, 필요시 lock를 제거하여 programming할수 있다고 유도.

(V4.5) DEVELOPER/2000 REGENERATE FORM

Bulletin no : 10057

말고 계시겠지만, 이전 `form version`에서 `developer/2000 - form`로 upgrade하여 program을 수행 시킬경우 --old version이므로 re-gen하여 다시 run하시요--라는 message를 종종 받게 됩니다.

또한 수많은(수백copy 이상 program)을 하나하나씩 다시 gen 하기 해서는 보통 많은 시간이 소모 되는것이 아닙니다.

이럴때 방법은 다음과 같습니다.

1. 모든 program을 하나의 directory로 모은다.
2. 모든 프로그램의 이름을 밑의 table을 샘플하여 db내에 저장한다.
`cd\dir *.* > filename.txt`
이 filename.txt 의 내용을 exlloader를 이용하여 밑의 table에 load.
3. form프로그램에서 이 program name이 저장되어 있는 table을 query한다.
4. 이때 query시 post-query 프리거에....

```
declare
form_name varchar2(100);

begin
next_record;

loop

next('f40gen c:\...form_name user .....');
if :system.next_record='TRUE' then
exit;
end if;
next_record;
end loop;
end;
```

(V4.5) FORMS4.5 칸 의 PARAMETER PASSING

Bulletin no : 10122

CALLER FORM에서의 SETTING

1. PARAMETER를 생성한다.

- (1) Navigator에서 Parameter를 선택
- (2) Navigator > Create
- (3) Parameter의 Property
 - * Data Type : CHAR, NUMBER, DATE 만 가능
 - * Default : 해당 Parameter의 Default Value
 - * Length
 - CHAR 만 입력가능 : 842까지 가능
 - NUMBER : Default 23 Byte
 - DATE : Default 7 Byte
 - * Name : Caller Form에서 사용될 이름

2. CALLER 전 상태에서 PARAMETER값 REFERENCE

- (1) 직접적인 Reference
 - * Block Item :- PARAMETER.parameter_name;
 - PARAMETER로 기술된 것은 Reserved Word이다
- (2) 간접적인 Reference
 - * NAME_IN, COPY 등의 Built-in Procedure를 사용한다.
 - Name_in('PARAMETER.parameter_name');
 - Copy('BIRTH', 'PARAMETER.parameter_name');
- (3) 다음의 Object들에 대한 Property 접근시에 해당 Parameter를 사용할 수 있다.
 - * Basic Table Block의 NUMBER나 DATE BY컬
 - Property화면이나 SET_BLOCK_PROPERTY에서 가능
 - * LOV에서의 Return Item Property
 - * Item의 Default Value Property
 - * Text Item의 Range Low Value 혹은 Range High Value Property

3. Initial Value Of Parameter In a Caller Form

- (1) Calling Form 에서 Parameter List 없이 부르면
 - * Caller Form 에서 Design시에 지정된 Default값이 접근된다.
 - 지정된 Default값이 없으면 NULL이 된다.
- (2) Calling Form에서 Default Parameter List를 Passing하면
 - * Calling Form 의 Parameter들이 갖고있는 현재값을 받는다.
- (3) Calling Form에서 Program에서 생성한 Parameter List를 Passing하면
 - * APP_PARAMETER Built-in에서 지정된 값을 받는다.

4. Called Form의 Parameter들은 Design시에 정의가 되어 있어야 한다.

(1) 정의되지 않은 Parameter를 Calling Form에서 지정하면

* FHM-47023 : No such parameter named exists in

Error가 발생한다.

(2) Called Form에 지정된 Parameter들은 Pass되는 Parameter들과 Data Type,

Length가 Match되어야 한다. 그렇지 않으면

* FHM-47024:Parameter type does not match definition in

Error가 발생한다.

* CALLING FHM에서의 BETTING

Called Form으로의 Parameter Passing은 Parameter List의 Passing으로 가능하다. Call_Form,Open_Form,New_Form등의 Built-in Procedures를 이용하는 Called Form으로의 Parameter Passing이 가능하다. 다른 Tool로는 Run_Product Built-in을 사용한다.

a) Parameter 값은 여러개의 Form간에 호환이되지 않는다.

b) 만약 Calling Form과 Called Form에서 같은 이름의 Parameter는 생성하여도 서로에게 영향을 미치지 않는다.

c) 만약 Called Form에서도 영향을 주는 가시적인값들을 원한다면 Global Variable을 사용하여야 한다.

d) Global Variable은 ERASE Built-in을 사용하여 Explicit 하게 삭제하거나, Backspace이 끝나기전에는 Called Form에 Active상태로 남아있게 된다.

1. Creating Parameter List

(1) Default Parameter List

* 각각의 Form은 Default라는 Built-in Parameter List를 포함하고 있다.

- Default Parameter List는 Form Design시에 생성한 모든 Parameter를 포함한다.

- 예를 들어 Form A에서 Design시에 Parameters를 p1, p2, p3 생성하였을 때, 그것들은 Form A의 Default Parameter List에 자동으로 등록된다.

* 다른 어떤 Parameter List와 마찬가지로 Default Parameter List도 OPEN_FHM,CALL_FHM,NEW_FHM Built-in을 통하여 Passing가능하다.

```
예)      Declare
          the_list ParamList;
      Begin
          the_list := Get_Parameter_List('default');
          Open_Form('Form_B',ACTIVE,NO_SESSION,'default');
      End;
```

(2) Programatic Parameter List

* Calling Form에 Parameters를 갖지않은 경우에는 Trigger나 User-command Routine에서 다음사항들을 이용하여 Parameter를 Passing한다.

* Parameter Creation 및 Passing

- Create_Parameter_List

a) 이 Function은 생성된 Parameter List의 ID를 Return한다.

b) ID의 Type은 ParamList로 지정한다.

c) 처음 생성된 Parameter List는 Parameter이름과 Value를 가진 Construct에 불과하다.

d) 이 인 Construct에 Parameter를 지정하기 위해서는 ADD_PARAMETER Built

-in을 사용한다.

- Add_Parameter
 - a) Parameter List에 Parameter를 Add한다.
 - b) 지정하는 Parameter명을 Called Form의 Parameters에 정의된 이름을 사용하여야 한다.
 - c) 지정할 Parameter의 갯수만큼 Call한다.
- * Parameter의 Type
 - Text Parameters
 - a) CHAN Bit-Flag
 - b) CALL_FORM, OPEN_FORM, NEW_FORM Built-in에 의해 Invoke된다.
 - Data Parameters
 - a) Called된 Product로 Pass될 Data Parameter의 값은 현재사용중인 Form에 정의된 Record Group의 이름이어야 한다. (Record Group은 Query에 의해 생성된 Record들이나 Program상에 Assign된 Record들의 Data Structure를 말한다.)
- * Reference Manual참조
 - b) Data Parameter들은 RUN_PRODUCT Built-in을 통하여 다른 Product (Reports나 Graphics)에 자료를 Pass한다. Form으로도 Pass가 가능하다.
- * Parameter List의 Manipulate Built-in Subprogram들
 - (Oracle Form Reference Manual 참조)
 - ADD_PARAMETER
 - CREATE_PARAMETER_LIST
 - DELETE_PARAMETER
 - DESTROY_PARAMETER_LIST
 - GET_PARAMETER_ATTR
 - GET_PARAMETER_LIST
 - SET_PARAMETER_ATTR

(V4.5) FORMS4.5에서 REPORT2.5로의 DATA PARAMETER PASSING

Bulletin no : 10124

RECORD GROUP 이란?

: Record Group은 Database Table의 Row/Column구조와 유사한 Oracle Form의 내부적 구조라 할수 있다. 즉, 프로그래밍적으로 그 기능이 Two-Dimensional Array와 같은 역할을 수행하게끔 해 준다. 또한 이는 Column의 총 갯수가 848을 초과하지 않 한도내에서 Char, Long, Number, Date Type의 Column들을 제공한다.

Record Group의 3가지 Type

1. Query Record Group

: SELECT문과 관련된 Record Group으로 Default로 Database Column의 Type과 길이를 갖는 Column들이 생성.

- Design Time에 Create

- (1) Navigator * Record Group에서 Create
- (2) Based on the Query Table을 선택한 후 SELECT 문 기술
- (3) Populate_Group Built-in Subprogram으로 Query를 실행

- Runtime에 Create

- (1) Trigger에서 Query문을 직접 기술

```
group_id := Create_Group_From_Query('BULKQUERY',  
                                     'select deptno, dname from dept');
```
 - (2) Populate_Group Built-in Subprogram으로 Query 실행
- Populate_Group이 성공적으로 실행된 경우 Return Value는 0.
Populate_Group_With_Query는 Group의 현재Query를 Replace할때 사용

2. Non_query Record Group

- Runtime에 프로그래밍적으로 그 구조와 값을 지정

- (1) Create_Group으로 Group 생성
- (2) Add_Group_Column, Add_Group_Row로 Column과 Row를 추가
- (3) Set_Group_Chars_Cell등으로 Group 의 각 Cell에 값을 Assign

3. Static Record Group

- Design Time에 그 구조와 값을 정의를 하고 Runtime에 값이 고정되어 사용.

- (1) Navigator * Record Group에서 Create
- (2) Static Values를 선택한 후 Column Name, Type, Length, Value를 Assign
- (3) Runtime에 바로 사용(get_group_chars_cell등으로)

Record Group을 이용하여 Report2.5로의 Data Parameter Passing (Non_query Record Group을 이용)

- (1) Record Group을 Create한다. (Create_Group)
- (2) Column들을 Add한다. (Add_Group_Column)
- (3) Row를 Add한다. (Add_Group_Row)
- (4) 각 Cell에 값을 Assign 한다. (Set_Group_Cell_Value)
- (5) Report로 Record Group을 전송한다.

<예제1> Record Group을 Create하고 값을 Assign한다. Cursor를 사용하여 Field에 Display 하고 그 값들을 Record Group에 입력

DECLARE

```

Rg_Name   VARCHAR2(15) := 'My_Group';
Rg_Id     Recordgroup;
Cc_Id     Groupcolumn;

Cursor Temp Is Select Deptno, Empno, Ename, Job From Emp;
Cnt       Number(5) := 0;

```

BEGIN

```

Rg_Id := Find_Group(Rg_Name);
If Not Id_Null(Rg_Id) Then Delete_Group(Rg_Id);
End If;

If Id_Null(Rg_Id) Then
    Rg_Id := Create_Group(Rg_Name);
    Cc_Id := Add_Group_Column(Rg_Id, 'Deptno', Number_Column);
    Cc_Id := Add_Group_Column(Rg_Id, 'Empno', Number_Column);
    Cc_Id := Add_Group_Column(Rg_Id, 'Ename', Char_Column, 10);
    Cc_Id := Add_Group_Column(Rg_Id, 'Job', Char_Column, 8);
End If;

If Not Id_Null(Rg_Id) Then
    Delete_Group_Row(Rg_Id, All_Rows);

    Open Temp;

    Loop
        Cnt := Cnt+1;
        Fetch Temp Into :Emp.Deptno, :Emp.Empno, :Emp.Ename, :Emp.Job;
        Exit When Temp%Notfound;

        Add_Group_Row(Rg_Id, End_of_Group);

        Set_Group_Number_Cell('My_Group', Deptno', Cnt, Name_In('Emp.Deptno'));
        Set_Group_Number_Cell('My_Group', Empno', Cnt, Name_In('Emp.Empno'));
        Set_Group_Char_Cell('My_Group', Ename', Cnt, Name_In('Emp.Ename'));
        Set_Group_Char_Cell('My_Group', Job', Cnt, Name_In('Emp.Job'));

        Rows;
    End Loop;

    Close Temp;
End If;

END;

```

<예제2> Report로 전송

이때 Report에는 'Exp_Query'라는 Query가 존재하고 보내고자 하는 Record Group의 Column명과 Type이 일치하는 Column들이 있어야 한다.

- Form에서

```

DECLARE
    Record_Pa    Parameter_List;
BEGIN
    Record_Pa := Get_Parameter_List('a');
    IF NOT IS_NULL(Record_Pa) THEN
        Destroy_Parameter_List(Record_Pa);
    END IF;
    Record_Pa := Create_Parameter_List('a');
    Add_Parameter(Record_Pa, 'Exp_Query', Data_Parameter, 'My_Group');
    Run_Product(Rptname, 'C:\Oracle\Report25\Bata_Pa', Synchronous, RunTime,
        Filesystem, Report_Pa, Null);
END;
- Report의 Query 'Exp_Query'
Select Depto, Empno, Ename, Job From Emp

```

<예제3> 현재 들어있는 Record Group의 Row들을 Delete

```

DECLARE
    Rg_Id    Recordgroup;
BEGIN
    Rg_Id := Find_Group('My_Group');
    IF NOT IS_NULL(Rg_Id) THEN
        Delete_Group_Row(Rg_Id, All_Rows);
    END IF;
END;

```

<예제4> WHEN-NOISE-CLICK Trigger에서 Mouse로 Click 한 Record만 Record Group으로 담은 Logic

```

DECLARE
    Rg_Name    Variable2(15) := 'My_Group';
    Rg_Id    Recordgroup;
BEGIN
    :Global.Count := to_Number(:Global.Count)+1;
    Rg_Id := Find_Group(Rg_Name);
    Add_Group_Row(Rg_Id, End_of_Group);
    Set_Group_Member_Cell('My_Group, Deptno', to_Number(:Global.Count),
        Name_In('Exp, Deptno'));
    Set_Group_Member_Cell('My_Group, Empno', to_Number(:Global.Count),
        Name_In('Exp, Empno'));
    Set_Group_Char_Cell('My_Group, Ename', to_Number(:Global.Count),
        Name_In('Exp, Ename'));
    Set_Group_Char_Cell('My_Group, Job', to_Number(:Global.Count),
        Name_In('Exp, Job'));
END;

```

(V4.5) DDE(DYNAMIC DATA EXCHANGE)

Bulletin no : 10125

1. About DDE?

- (1) MS Windows와 Communicate하거나 Data교환을 할 수 있는 기법을 말한다.
- (2) Oracle Form 4.5에서는 DDE를 가능케하는 PL/SQL Package를 제공한다.
- (3) Oracle Form 4.5에서의 DDE Function들은 MS Windows의 다른 DDE Server들과 다음과 같은 방법으로 Data를 교환할 수 있다.
 - * Data를 Import하는 형식
 - * Data를 Export하는 형식
 - * DDE Server에 대한 Command를 실행하는 형식

2. Form 4.5에서의 DDE 제한사항

- (1) Data Linking : Data Item이 연결된 경우 자동으로 연결된 내용을 인식하거나, Update가 되지 않는다. 다시 한번 조치하여 처리하여야 한다.
- (2) Server Support : DDE Client로 부서의 Command나 Data요구에 응답 할 수 없다.

3. Function Overview

Form에서 제공하는 DDE Package를 기능에 따라 분류하면 다음과 같다.

- (1) Support Functions : DDE Server Application을 시작, 종료시키는 기능을 한다.

- DDE.App_Begin : DDE Server를 시작한다.

Syntax: DDE.APP_BEGIN(Command, Mode)

Return Value

Application Identifier (PLB_INTEGER Type)

DDE.App_Begin에서 Return된 Application Identifier는

DDE.APP_END, DDE.APP_FINISH Function Call시에 사용된다.

Command

Type : Varchar2

Program을 실행하기 위한 Command

Path가 지정되지 않은 경우 Form은 다음 Directory를 찾는다.

Current Directory

Windows Directory

Windows System Directory

Current Task를 위한 Executable File을 포함한 Directory

PATH환경에 설정된 Directory

Network에 설정된 Directory

Command String 내에 Application에 필요한 Argument가 필요한 경우
Application Program Name과 Space로 분리하여 표기하면 된다.

Mod

Type : PLB_INTEGER

실행할 Application의 Window의 Mode를 말한다.

DBE_APP_MODE_NORMAL : Normal Size로 Start한다.

DBE_APP_MODE_MINIMIZED : Minimized Size로 Start한다.

DBE_APP_MODE_MAXIMIZED : Maximized Size로 Start한다.

[Example]

DECLARE

AppId PLB_INTEGER;

BEGIN

/* MS Excel을 Start한다 */

AppId := DBE.App_Begin('EXCEL.EXE', DBE_APP_MODE_MINIMIZED);

EXCEPTION

WHEN DBE,DBE_APP_FAILURE

Message('Excel 실행에 실패하였습니다');

END;

- DBE.App_End : DBE Server를 종료한다.

Syntax : DBE.App_End(AppId)

Return Value : 없음.

AppId

Type : PLB_INTEGER

DBE_APP_BEGIN에 의해 얻어진 Application Identifier를 지정한다.

Application은 Application 자체의 Close에 의하거나, Application 자체의
Control Menu를 Double Click함으로써 종료될 수도 있다.

DBE.App_End를 실행하기전에 반드시 DBE.App_Begin이 실행되어 있어야 한다.

[Example]

DECLARE

AppId PLB_INTEGER;

BEGIN

/* MS Excel을 Start한다 */

AppId := DBE.App_Begin('EXCEL.EXE', DBE_APP_MODE_MINIMIZED);

...

... /* Excel Program 을 종료한다. */

DBE.App_End(AppId);

EXCEPTION

WHEN DBE,DBE_APP_FAILURE THEN

Message('Excel 실행에 실패하였습니다');

```

        WHEN BDE, BDE_APP_NOTFOUND THEN
            Message('Excel을 먼저 실행하십시오. ');
        END;
    *-----*

    - BDE.App_Focus : Window의 Focus를 BDE Server로 옮긴다.

    Syntax: BDE.APP_FOCUS(AppId)
    Return Value : 없음.
    AppId
        Type : PLB_INTEGER
    BDE.APP_BEGIN에 의해 얻어진 Application Identifier를 지정한다.
    BDE.App_End를 실행하기전에 반드시 BDE.App_Begin이 실행되어 있어야 한다.
    *-----*

    [ Example ]
    BEGIN
        AppId      PLB_INTEGER;
    BEGIN
        /* MS Excel을 Start한다 */
        AppId := BDE.App_Begin('EXCEL.EXE', BDE.APP_MODE_MINIMIZED);
        ...
        ...
        /* Application Window를 Activate시킨다. */
        BDE.App_Focus(AppId);
    EXCEPTION
        WHEN BDE, BDE_APP_FAILURE THEN
            Message('Excel 실행에 실패하였습니다. ');
        WHEN BDE, BDE_APP_NOTFOUND THEN
            Message('Excel을 먼저 실행하십시오. ');
    END;
    *-----*

```

(2) Connect/Disconnect Functions : BDE Server에 접속, 분리하는 기능을 한다.

- BDE.Initiate : 다른 Application과 BDE작업을 시작한다.

```

Syntax: BDE.INITIATE(Service, Topic)
Return Value
    Conversation Identifier( PLB_INTEGER Type)
    BDE.App_Initiate Function에서 Return되는 ConvId는
    Execute, Poke, Request, Terminate Function Call시에 사용된다.
Service
    Type : Unicode2
    Server Program의 BDE Service Code
    Service의 값은 밀란적으로 Application Program명이 지정된다.
Topic

```

```

Type : Varchar2
Conversation의 Topic Name
File-Based로 실행되는 Operate되는 Application의 경우 Topic의 값은
일관적으로 Document의 File명이 지정된다.
Service와 Topic의 값은 DBE Server Application에 의해 달라진다.
DBE.Initiate로 시작된 Conversation은 DBE.Terminate를 만나야 끝난다.

```

-----*

[Example]

```

DECLARE
    ConvId    PLB_INTEGER;
BEGIN
    /* NB Excel과 DBE Conversation을 Dynam한다 */
    ConvId := DBE.Initiate('EXCEL','SHEET1');
EXCEPTION
    WHEN DBE.DBE_INIT_FAILED THEN
        Message('Excel SHEET1 Init실행에 실패하였습니다');
END;

```

-----*

- DBE.Terminate : 지정된 Application과의 DBE작업을 종료한다.

```

Syntax: DBE.Terminate(ConvId)
Return Value
    없음.
Conv
    Type : PLB_INTEGER
    DBE.INITIATE에서 Return된 Conversation Identifier
    DBE.Terminate를 실행하기전에 반드시 DBE.Initiate가 실행되어 있어야 한다.
    DBE.Terminate가 실행된 이후에 모든 Execute, Poke, Request, Terminate등의
    Command는 Error이다.

```

-----*

[Example]

```

DECLARE
    ConvId    PLB_INTEGER;
BEGIN
    /* NB Excel과 DBE Conversation을 Dynam한다 */
    ConvId := DBE.Initiate('EXCEL','SHEET1');
    ....
    ....
    /* NB Excel과의 DBE Conversation을 끝낸다 */
    DBE.Terminate(ConvId);
END;

```

-----*

(3) Transaction Functions : DBE Server와 Data를 교환하는데 사용한다.

- DBE.Execute : Application에 의해 인식된 Command를 실행한다.

Syntax: DBE.Execute(ConnId, Command, Timeout)

Return Value : 없음.

Conn

Type : PLB_INTEGER

DBE.INITIALIZE에서 Return된 Connection Identifier

Command

Type : VARCHAR2

Server에서 실행에 필요한 명령어

Server Application에서 제공하는 Command를 사용한다.

Timeout

Type : VARCHAR2

Millisecond단위

Default : 1000

[Example]

DECLARE

ConnId PLB_INTEGER;

BEGIN

/* MS Excel과 DBE Conversation을 Open한다 */

ConnId := DBE.Initiate('EXCEL', 'SHEET1');

....

....

DBE.Execute(ConnId, '[Calculate, Now()]', 1000);

/* Word에서 File Close */

DBE.Execute(ConnId, '[FileClose1(0)]', 1000);

END;

- DBE.Poke : Application에 자료를 제공한다.

Syntax: DBE.Poke(ConnId, Item, Data, DataFormat, Timeout)

Return Value : 없음.

Conn

Type : PLB_INTEGER

DBE.INITIALIZE에서 Return된 Connection Identifier

Item

Type : VARCHAR2

Application으로 보내지는 Data Item명

Data

Type : VARCHAR2

전송할 Data Buffer

DataFormat

Type : PLB_INTEGER

MS Window에 기 지정된 Data Format명을 다.

(Oracle Form 4.5 Advanced Techniques Manual참조)

Timeout

Type : VARCHAR2

Millisecond단위

Default : 1000

[Example]

DECLARE

ConvId PLB_INTEGER;

BEGIN

/* MS Excel과 BDE Conversation을 Dynam한다 */

ConvId := BDE.Initiate('EXCEL', 'SHEET1');

....

....

/* EXCEL의 Row 2, Column 2 Cell에 Exam Item의 값을 보낸다 */

BDE.PutIn(ConvId, 'B2C2', Exam, BDE.CF_TEXT, 1000);

END;

- BDE.Request : Application으로부터 자료를 요청한다.

Syntax: BDE.request(ConvId, Item, Buffer, Dataformat, Timeout)

Return Value 값.

Conv

Type : PLB_INTEGER

BDE.INITIALIZE에서 Return된 Conversation Identifier

Item

Type : VARCHAR2

Application으로 부터 보내지는 Data Item명

Buffer

Type : VARCHAR2

전송할 Data Buffer

Dataformat

Type : PLB_INTEGER

MS Window에 기 지정된 Data Format명을 쓴다.

(Oracle Form 4.5 advanced techniques manual참조)

Timeout

Type : VARCHAR2

Millisecond단위

Default : 1000

[Example]

```

DECLARE
    Convid      PLS_INTEGER;
BEGIN
    /* MS Excel과 DDE Conversation을 Dynam한다 */
    Convid := DDE.Initiate('EXCEL','SHEET1');

    ....

    /* EXCEL의 Row 2, Column 2 Cell에 해당된 값을 Kmax Item으로 받는다 */
    DDE.Request(Convid,'B2C2',:Kmax,DDE.CF_TEXT,1000);
END;

```

(4) Datatype Translation Functions

- DDE.ConvertFrom : Data Forest String을 Number로 Convert/Register한다.
- DDE.ConvertTo : Data Forest Number를 String으로 Convert한다

4. Sample Program - MS Windows의 EXCEL과의 Data Exchange

(1) Package Spec

```

PACKAGE DDE_EXE IS
    APPID PLS_INTEGER;
    CONVID PLS_INTEGER;
    ISRUN BOOLEAN := FALSE;
    PROCEDURE EXCEL_BEGIN;
    PROCEDURE EXCEL_FOCUS;
    PROCEDURE EXCEL_PEEK;
    PROCEDURE EXCEL_REQ;
    PROCEDURE EXCEL_END;
END;

```

(2) Package Body

```

PACKAGE BODY DDE_EXE IS

PROCEDURE EXCEL_BEGIN IS
BEGIN
    IF DDE_EXE.ISRUN = TRUE THEN
        MESSAGE('ALREADY STARTED');
    ELSE
        DDE_EXE.AppId :=
            DDE.App_Register('C:\Microsoft\Excel\Excel.exe',Dde.App_Mode_Normal);
        DDE_EXE.ISRUN := TRUE;
    END IF;

```

```

EXCEPTION
    WHEN DDE,DDE_APP_FAILURE THEN
        MESSAGE('DDE 실패');
END;

/* tipx : Excel이 시작되는 시간이 필요합니다. User-New-Form-Instance나
User-Button-Process에서 처리하되 DDE,Initiate를 실행하기전에 Excel이
Start되기위한 충분한 시간을 가져야 합니다. 만약 Excel이 정상 Start되기
전에 DDE,Initiate를 실행하면 FWH-10855B Error가 발생합니다 */

PROCEDURE EXCEL_END IS
BEGIN
    IF DDE_EXE.ISRUN = FALSE THEN
        MESSAGE('먼저 START하십시오');
    ELSE
        DDE_APP_END(DDE_EXE,APPID);
        DDE_EXE.ISRUN := FALSE;
    END IF;
EXCEPTION
    WHEN DDE,DDE_APP_NOT_FOUND THEN
        MESSAGE('APPID FOUND 실패');
END;

PROCEDURE EXCEL_FOCUS IS
BEGIN
    IF DDE_EXE.ISRUN = FALSE THEN
        MESSAGE('먼저 BEGIN을 실행하십시오!');
    ELSE
        DDE_APP_FOCUS(DDE_EXE,APPID);
    END IF;
EXCEPTION
    WHEN DDE,DDE_INIT_FAILED THEN
        MESSAGE('INITIATE 실패');
    WHEN DDE,DDE_APP_NOT_FOUND THEN
        MESSAGE('해당 APPID가 없음');
END;

PROCEDURE EXCEL_FONE IS
    Y      Number := 1;
    A%IL   U%CH%N%Z(B);
BEGIN
    IF DDE_EXE.ISRUN = FALSE THEN
        MESSAGE('먼저 BEGIN을 실행하십시오!');
    ELSE
        DDE_EXE.COMMID := DDE,INITIATE('EXCEL',:CNT,BHEET);
    END IF;
    Co_Block('Key');

```

```

next_record:
LOOP
IF :SYSTEM_LAST_RECORD = 'TRUE' THEN
EXIT:
ELSE
AKIL := 'B' || TO_CHAR(Y) || 'C' || TO_CHAR(1);
DBE.REQUEST(DBE_EXE.COMUID, AKIL, TO_CHAR(:ENPNO), DBE.CF_TEXT, 100000);
AKIL := 'B' || TO_CHAR(Y) || 'C' || TO_CHAR(2);
DBE.REQUEST(DBE_EXE.COMUID, AKIL, :ENAME, DBE.CF_TEXT, 100000);
AKIL := 'B' || TO_CHAR(Y) || 'C' || TO_CHAR(3);
DBE.REQUEST(DBE_EXE.COMUID, AKIL, :JDM, DBE.CF_TEXT, 100000);
AKIL := 'B' || TO_CHAR(Y) || 'C' || TO_CHAR(4);
DBE.REQUEST(DBE_EXE.COMUID, AKIL, TO_CHAR(:HIREDATE, 'YY/MM/DD'),
DBE.CF_TEXT, 100000);
END IF;
Sum:
Y := Y+1;
END LOOP;
DBE.TERMINATE(DBE_EXE.COMUID);
FIRST_RECORD:
EXCEPTION
WHEN DBE.DBE_INIT_FAILED THEN
MESSAGE('INITIATE 실패');
END;
/* Type1 : Exp Block에 Query되어 있는 Data를 Excel Sheet에 전송합니다.
Numeric이나 Data Type의 Item은 Char Type으로 전처리하여 전송하여야 합니다. */
DBE_EXE.COMUID := DBE.INITIATE('EXCEL', :CNT, SHEET);
FOR y IN 1..5 LOOP
AKIL := 'B' || TO_CHAR(y) || 'C' || TO_CHAR(1);
DBE.REQUEST(DBE_EXE.COMUID, AKIL, DUMMY, DBE.CF_TEXT, 100000);
dummy := to_number(substr(dummy, 1, length(dummy)-2));
AKIL := 'B' || TO_CHAR(y) || 'C' || TO_CHAR(2);
DBE.REQUEST(DBE_EXE.COMUID, AKIL, :ENAME, DBE.CF_TEXT, 100000);
ename := substr(ename, 1, length(ename)-2);
next_record:
END LOOP;
DBE.TERMINATE(DBE_EXE.COMUID);
FIRST_RECORD:
END;
/* Type1 : Excel에서 가져올 수 있는 Data는 Char Type이어야 합니다.
Numeric이나 Data Item으로 Data를 가져오려면 먼저 Char Type인 Variable에
할아서 Numeric Item으로 Assign하여야 합니다. */
/* Type2 : Excel에서 가져오는 Data뒤에 Space가 2 Byte가 붙어서
들어옵니다. DB Item에 Assign하기전에 뒤에서 2 Byte를 잘라내야 합니다 */
END;

```


(V4.5) FORMS45와 TEXT FILE간의 INTERFACE

Bulletin no : 101B0

Form에서 text file로 write하거나 혹은 text file의 data를 form의 block으로 read하는 기능은 Form가 제공하는 TEKTY_ID라는 built-in package에 의해 가능하다.

TEKTY_ID.FCLOSE(file file_type)

: close할 file을 지정

TEKTY_ID.FILE_TYPE

: file의 datatype을 지정

TEKTY_ID.FOPEN(syntax varchar2, filename varchar2)

syntax : filemode

filename : B 읽기전용의 file을 open

W file에 있는 내용을 지우고 write를 할때

A file에 있는 내용에 덧붙여서 write를 할때

TEKTY_ID.IS_OPEN(file file_type)

지정한 file이 현재 open되어 있는지 check

TEKTY_ID.GET_LINE(file file_type, item OUT varchar2);

한 line에서 null character(즉 carriage return)이나 end-of-

file(EOF)를 만날때까지 character를 읽어서 item에 입력

TEKTY_ID.NEW_LINE

새로운 line을 지정

TEKTY_ID.PUT(file file_type, item varchar2)

open된 file의 현재line에 제공되는 data를 덧붙인다.

TEKTY_ID.PUTF(file file_type,format varchar2,arg varchar2)

open된 file에 format대로 arg 쓴다.

예) TEKTY_ID.PUTF(out_file, 'Today is %s',

TO_CHAR(SYSDATE));

TEKTY_ID.PUT_LINE(file file_type, item varchar2)

open된 file의 현재line에 제공된 data를 덧붙인 후 null

character(즉 carriage return)을 string 마지막에 추가한다.

PROCEDURE text_read IS

```

in_file TEXT_ID.FILE_TYPE;
lseekf  varchar2(80);

BEGIN
    in_file := TEXT_ID.FOPEN('c:\temp\read.txt','r');
    go_block('map');
    first_record;
    loop
        TEXT_ID.GET_LINE(in_file,lseekf);
        map.mapno := substr(lseekf,1,10);
        map.name := substr(lseekf,13,10);
        TEXT_ID.NEW_LINE;
        next_record;
    end loop;
EXCEPTION
    when no_data_found then
        TEXT_ID.FCLOSE(in_file);
        message('읽기가 종료되었습니다. ');
    when others then
        TEXT_ID.FCLOSE(in_file);
        message('읽기에 실패하였습니다. ');
end;

```

PROCEDURE text_write IS

```

out_file TEXT_ID.FILE_TYPE;
lseekf  varchar2(80);

BEGIN
    out_file := TEXT_ID.FOPEN('c:\temp\write.txt','w');
    go_block('map');
    first_record;
    loop
        TEXT_ID.PUTF(out_file, ' %c %c %c %c',
            rpad(Name_In('map.mapno'),10),
            rpad(Name_In('map.name'),20),
            rpad(Name_In('map.job'),20));

        if :mapno,first_record = 'TRUE' then
            message('File Write가 종료되었습니다. ');
            exit;
        end if;
        next_record;
    end loop;
    TEXT_ID.FCLOSE(out_file);
END;

```

(V4.5) DEVELOPER2000으로 작성한 MODULE를 DATABASE에 저장하려면?

Bulletin no : 10218

User가 작성한 Form4.5나 Report2.5 Module을 DB에 save하기 위해서는 그 Module을 수록하는 table
들이 먼저 create되어 있어야 한다. 이 table의 생성 script들은 install당시에 자동적으로 만들어지
며 BQLLIB의 SYSTEM account에서 불러주어야 한다.

1) Oracle Form을 위한 Table Create

```
BQL> @C:\ORACLE\NCR2\BQL\Y00LIB.BQL
BQL> @C:\ORACLE\NCR2\BQL\B08BIL.BQL
BQL> @C:\ORACLE\NCR2\BQL\W00LIB.BQL
BQL> @C:\ORACLE\NCR2\BQL\B00LIB.BQL
BQL> @C:\ORACLE\NCR2\BQL\F00LIB.BQL
```

2) Oracle Report를 위한 Table Create

```
BQL> @C:\ORACLE\NCR2\BQL\Y00LIB.BQL
BQL> @C:\ORACLE\NCR2\BQL\B08BIL.BQL
BQL> @C:\ORACLE\NCR2\BQL\W00LIB.BQL
BQL> @C:\ORACLE\NCR2\BQL\B00LIB.BQL
```

3) Oracle Graphics를 위한 Table Create

```
BQL> @C:\ORACLE\NCR2\BQL\B08BIL.BQL
BQL> @C:\ORACLE\NCR2\BQL\Y00LIB.BQL
BQL> @C:\ORACLE\NCR2\BQL\W00LIB.BQL
BQL> @C:\ORACLE\NCR2\BQL\B00LIB.BQL
BQL> @C:\ORACLE\NCR2\BQL\G00LIB.BQL
BQL> @C:\ORACLE\NCR2\BQL\G00LIB.BQL
```

상호간에 중복된 Script를 다시 실행할 필요는 없다.

(V4.5) FORMS4.5 RUNTIME INSTALL 방법

Bulletin no : 10217

Form Application이 각 client machine에서 작동되기 위해서는 반드시 Form Runtime이 설치되어 있어야만 하는데, 기본적으로 설치되어야 할 것들은

dll Common File
Runtime Support File
Tool Utilities
Form Runtime

이고, 만약 BQLNet V2를 사용한다면

Required Support File 7.1
BQLNet 2.1
Oracle (protocol) Adapter 2.1

를 추가로 install을 해야하고, BQLNet v1을 사용한다면 위 Product 대신에

Required Support File 7.0
Required Support File 7.1
BQLNet (protocol) 1.x

이 필요하다.

Runtime Module을 install한 후에 PC에 연결된 사항들은 다음과 같다.

- 1) C:\NBAWIN directory 및 sub directory 생성
- 2) C:\WINOUB\OUBACLE.INI file 생성
- 3) C:\WINOUB\WIN.INI file에
[Oracle]
ORA_CONFIG=C:\WINOUB\OUBACLE.INI
가 Insert 됨
- 4) AUTOEXEC.BAT가 연결
- 5) program group과 icon의 생성
- 6) C:\WINOUB\SYSTEM directory에 여러 DLL file의 생성

CTL3D.DLL	CTL3D42.DLL	COMPAG.DLL
DLE2.DLL	DLE2COMV.DLL	DLE2DIBP.DLL
DLE2MLB.DLL	DLE2PROM.DLL	TYPELIB.DLL
STORAGE.DLL	USHAME.386	STSOLE.TLB
HFC0250.DLL	HFC0250.DLL	HFC0250.DLL

MFCDLGUI.DLL COMNLS.DLL MFCLENUM.DLL
UNDASO.DLL

EE Install이 끝난 후 BIMPDM 수행시에 FHM-4034 (Cannot attach the library file) Error가 발생한다면?

library를 attach 할 수 없는 이유는 지정한 directory에 attach하려는 DLL file이 존재하지 않기 때문이다.
Form을 작성할때 Attach library를 then path를 remove할 것민지를 묻는 Alert Box가 뜨게 된다. 이때, remove path를 'NO'로 하게 되면
Form은 그 DLL file의 directory를 기억하게 되어 후에 runtime module
을 install하는 PC에도 같은 directory에 반드시 attach한 DLL file이
존재해야 한다. then remove path를 'YES'로 하면 path를 기억하지 않으며
항상 그 Form을 실행할때 DBACLE.INI file의 FDMB45_PATH에 setting
되어 있는 path명으로 가서 그 DLL file을 찾게 점을 유의해야 한다.

(V4.5) FORMS4.5에서 FILE DIALOG 처리 방법

Bulletin no : 10218

File명을 직접 입력하지 않고 Dialog Box를 이용하여 원하는 File를 선택하려면 해당 DLL을 호출하거나 cdialog.vbx를 이용하는 방법이 있는데 여기서는 'FILEMUL.DLL'를 사용하여 작성된 예이다.
(보통 FILEMUL.DLL은 C:\WINDOWS\RESOURCES\COMMON\FFI\FILEMUL directory에 존재하며 size는 3818Bytes)

PACKAGE SPEC 정의내역

PACKAGE BELFILE IS

```
FUNCTION mulFile return varchar2;
END;
```

PACKAGE BODY 정의내역

PACKAGE BODY mulFile IS

```
dll_handle  DBA_FFI.LIBHANDLETYPE;
fct_handle  DBA_FFI.FUNCHANDLETYPE;

FUNCTION mulFilecd(handle IN DBA_FFI.FUNCHANDLETYPE,
                  arg1 IN OUT VARCHAR2)
RETURN PLB_INTEGER;
pragma interface(c, mulFilecd,11285);
FUNCTION mulFile
RETURN VARCHAR2
IS
    tmp VARCHAR2(512) := space(' ',512);
BEGIN
    IF (mulFilecd(fct_handle,tmp)) = 1 THEN
        RETURN NULL;
    ELSE
        RETURN tmp;
    END IF;
END;

BEGIN
    dll_handle := DBA_FFI.REGISTER_LIBRARY(NULL,
                                           'C:\WINDOWS\RESOURCES\COMMON\FFI\FILEMUL\FILEMUL.DLL');
    fct_handle := DBA_FFI.REGISTER_FUNCTION(dll_handle,
                                           'ChooseFile',DBA_FFI.PASCAL_BYTE);
    DBA_FFI.REGISTER_RETURN(fct_handle, DBA_FFI.C_INT);
    DBA_FFI.REGISTER_PARAMETER(fct_handle, DBA_FFI.C_CHAR_PTR);
END;
```

(V45) HOW TO IMPLEMENT MENU SECURITY IN FORMS

4.5?

Bulletin no : 10243

Oracle Forms 4.5에서의 보안은 DATABASE USER의 ROLE의 개념에 기초를 둔다. 모든 user들이 database에 저장된 모든 정보를 사용할 수 없기때문에, role은 user들을 grouping하는데 유용하게 사용한다. (role에 대한 자세한 개념은 manual을 참조하람.)

oracle server에서 role이 정의되면, form개발자는 menu module과 menu item 에 role을 지정하여, menu program에 누가 그것을 사용할 수 있는지를 조정할 수 있게된다.

*MENU APPLICATION을 사용하려면 EXP_FULL_DATABASE PRIVILEGE가 있어야한다.

I. ROLE생성방법

1.DATABASE FORMS 4.5 DESIGNER에서 SYSTEM ACCOUNT로 CONNECT

2.FILE ADMINISTRATION DATABASE ROLE

만약, ADMIN PRIVILEGE가 없는 USER로 CONNECT를 하게되면 이 OPTION이 DATABASE상대로 나타난다.

3.새로운 ROLE을 생성하기 위해 MENU를 선택

새로운 ROLE의 이름을 등록할 수 있는 WINDOW가 나타난다.

새로운 ROLE의 이름을 입력하고, 그 ROLE 갖게될 USER명을 입력한다. MENU를 실행시에 사용할 수 있는 OPTION(BASIC ROLE, DB COMMAND, BACKGROUNDB MENU)을 원하는 대로 선택한다. MENU에서 위의 OPTION을 사용하지 않으려면 BLANK로 남겨두면된다.

4.DE BUTTON을 선택

ROLE이 생성된다.

II. MENU MODULE에서의 SECURITY SETTING

1.MENU APPLICATION OPEN한 후 MENU APPLICATION의 PROPERTY화면을 떠난다.

2.SECURITY MENU SECTION에서

1)MENU MODULE SECURITY HOME BUTTON 선택 혹은 DOUBLE CLICE

선택한 MENU APPLICATION의 사용을 허락할 ROLE명을 등록한다.

DE

이 작업은 각각의 MENU ITEM을 사용할 수 있는 권한을 부여한 것은 아니다.

단지, 각 ITEM마다 선택할 수 있는 ROLE을 지정한 것에 불과하다.

2)USE SECURITY를 TRUE로 SETTING

만약, TRUE로 SETTING하는 것을 잊어버린다면

모든 USER들이 모든 MENU OPTION을 사용할 수 있게된다.

III. MENU ITEM에 ROLE을 지정하는 방법

1. MENU ITEM의 PROPERTY화면을 띄운다.

2. SECURITY SECTION에서

1) MENU ITEM ROLE에서 HOME BUTTON선택 혹은 DOUBLE CLICK

말의 과정에서 SETTING에 놓은 ROLE들이 나타난다.

이 ITEM을 사용할 수 있게 하고자 하는 ROLE들을 선택한다.

만약, 이 LIST에서 아무것도 선택하지 않으면,

이 MENU는 모든 USER들이 사용할 수 있게된다.

2) DISPLAY AND PRIVILEGE는 이 MENU ITEM을 사용할 수 없는 USER에게도 MENU ITEM을 보여 줄 것인가를 의미한다. 만약, TRUE인 경우 이 ITEM 을 사용 수 없는 USER에게는 CHAYED OUT된상태 즉 DISABLE된 상태로 나타난다.

만약, FALSE인 경우 이 ITEM을 사용할 수 없는 USER에게는 이 MENU ITEM이 나타나지 않게된다.

IV. MENU 를 RUN시키는 방법

MENU를 RUN시키려면, 먼저 FORM에 MENU를 ATTCH하는 작업이 필요하다.

1. FORM MODULE를 OPEN한다.

2. FORM MODULE의 PROPERTY화면을 띄운다. (TOOLB PROPERTIES)

3. MENU SECTION에서

MENU MODULE에 생성해 놓은 MENU MODULE명을 등록한다.

MENU STYLE 선택한다.

STARTING MENU를 입력한다.

이것은 생성해 놓은 MENU MODULE에 있는 MENU TREE구조중 어느 POINT에서 부터 사용할 것인가를 지정하는 것이다.

4. FORM을 SAVE한 후 RUN시킨다.

V. 나타날 수 있는 ERROR들

FORM-1025B : "User is Not Authorized To Run Oracle Forms Menu"

이것은 USER가 FORMS TABLE들을 ACCESS할 수 없다는걸 의미한다.

FORMS에서의 SECURITY를 구현하기 위해서는, FORMS45에 완전 TABLE과 VIEW들이 생성되어 있어야 하고, 해당 USER는 이 TABLE과 VIEW를 사용할 수 있는 권한이 있어야 한다. FORMS45에 완전권 TABLE과 VIEW를 생성하기 위해서는 완전권 SQL*BYSTEN ACCOUNT 하에서 실행되어야 한다.

(INSTALLATION GUIDE참조)

FORM-1024B : "No Authorization To Run Application"

이 ERROR는 MENU APPLICATION을 사용할 수 있도록 지정된 ROLE에 해당 USER가 ASSIGNED되어 있지 않다는 것을 의미한다.

(V4.5) DEV/2000에서 DOCUMENTATION 생성 방법

Bulletin no : 10252

Form Doc/Report Doc

1. Running Form Doc.

Developer/2000의 Form4.5이전의 Version에서는 Form Doc를 실행하려면 해당 Form이 Database에 Save가 되어있어야 하고, Doc를 실행하면 내부에서 실제로 F40doc.Biz가 호출되었다. 그러나 Developer/2000의 Form4.5에서 Doc를 실행하면 File System에 저장되어있는 Module에서 직접 정보를 읽어 들이도록 되었다.

Form에서 Doc Utility를 사용하려면 File * Administration * Form Doc.를 선택하면 된다. 이 Utility는 Object Navigator에 나타나있는 순서에 따라 Text-Formatted File를 생성하게 된다.

Document File에 나타나는 Object의 순서를 바꾸려면, Object Navigator의 Option을 Ownership View에서 Visual View로 바꾸어 선택하면 된다.

1) Navigator*Ownership View(Default) : Object Navigator는 Ownership에 의한 Object의 Hierarchy에 따라 나뉘었다.

2) Navigator*Visual View : Object Navigator 는 Display될 수 있는 것을 예들들어 Window, Canvas, Item등만을 나뉘었다. 이러한 Object들은 그것들이 Display되는 Canvas/Window들과 연관된 것들이다.

3) Navigator*Only Object With PL/SQL Option은 On/Off로 Toggle이 된다. 이 Option을 On으로 Setting하면 Trigger, Program Unit등이 있는 Object를 제외한 모든 Object들이 나타나질 않게 된다. 이것은 Document의 양을 줄일 수 있게 해준다. 그러나 이것 Trigger나 Program만이 Form Documentation에 나타난다는 것을 의미하는 것은 아니다.

2. Output To Export

Form4.0에서의 Doc과는 달리 Form4.5에서 Documentation에서는 PL/SQL Code나 Trigger만을 Print할 수 없다. 이점에 관해서는 아래의 Section에서 서미한 .

Documentation을 선택하면 현재 작업중인 Directory내에 .Text File이 생성된다. 만약 Form의 이름들 아직 생성하지 않았으면 .Text가 생성된다.

3. Generating PL/SQL Documentation

Program에서 단지 Trigger나 Program Unit만 Document로 출력하고자 한다면 Form Doc Utility를 사용해서는 할 수가 없다. 대신, Form Debugger Utility를 사용하여야만 한다. Form Debugger에서 원하는 Program을 Open한 다음 Debug Option을 선택한다.

Form을 Run시켜 Debugger가 나타나면 F10QL> Prompt에 다음과 같이 Type한다.

.Export Prog * File C:\,Plc

이 작업을 수행하면 현재의 Form에 포함되어있는 PL/SQL Code들이 포함되게 된다. 이론아니다

Attach되어 있는 Library의 Code도 포함하게 된다.

주의 : 이 작업을 통하여 생성되는 File내에는 Procedure나 Trigger의 이름들은 출력되지 않는 제약이 있으므로 각각의 Trigger나 Procedure에 Comment처리를 하여야 한다.

이 작업은 Graphical나 Reports와 같은 다른 Tool에서도 처리가 가능하다.

4. Notes On Forms Doc

Document는 Release에 따라 다르게 처리되므로, 이 작업을 Source Control에 사용하거나 Application간의 차이점을 Check하는데 사용하지 않는 것이 좋다.

5. Running Reports Doc

Oracle Reports2.5에서의 Report Doc은 Reports 2.0과 흡사하다. Reports에서 Documentation을 알아내려면 해당 Report Database에 저장되어 있어야 하고, 해당 Table을 Access할 수 있도록 Logon이 되어 있어야 한다.

Report Doc을 선택하면 Landscape나 Portrait양식을 선택하게 된다. 이것은 결국 내부에서 C:\Oracle\Reports25P\의 Bwdoc1k와 Bwdocpk Report를 선택하여 출력 게 된다.

(V4.5) EXITING MULTIPLE FORM APPLICATIONS

Bulletin no : 10254

1. General Overview

가끔 User들은 Menu에서 선택하거나, Button을 선택하여 Open된 Form들을 동시에 Close하기를 원하기도 한다. 이렇게 동시에 Open되어 있는 Form들을 Close하기 위해서는 각 Form에 Focus가 갈때 Global값이 Check되어야 한다. 이 Global값은 스스로 Close되어야 하는지 아닌지를 알려주게된다. 이 값들은 Call_Form, Open_Form과 New_Form으로 새로운 Form을 Open할 때 모두 사용된다.

2. Triggers

다음의 3가지 Trigger가 필요하다.

```
+-----+
User-New-Form-Instance
    :Global.Quit_All := 'N';
```

주의 : 처음시작하는 Form에 걸어야 한다.
Global변수를 처음 생성한다.

```
+-----+
User-Window-Activated
    If :Global.Quit_All = 'Y' Then
        Exit_Form;
    End If;
```

주의 : 이 Trigger는 처음 시작하는 Form을 포함한 모든 Form에 필요하다.
이 Trigger는 Form에 다시 Focus가 돌아올때 실행된다.

```
+-----+
User-Button-Pressed
    :Global.Quit_All := 'Y';
    Exit_Form;
```

주의 : 이 Trigger는 Close를 시작하게 한다.
만약 Form들이 같은 Menu에 연결된 것이라면 Menu Item으로 Battling
하고, 각각의 Form에 걸려있는 이 Trigger를 없애도 된다.

```
+-----+
```

단, Open_Form과 Call_Form을 섞어서 사용하지 말아야 한다.

(V3.0,4.5) PRE-QUERY TRIGGER 사용 방법

Bulletin no : 10271

Advanced BQLEDBMS Techniques Version 3.0 manual 중 'Performing Dynamic Sorting' 참조.

Enter query mode에서 field에 값을 입력하거나, 원하는 조건을 입력함으로써 query를 제한적으로 처리할 수 있다. field에 입력한 문자열은 field에 연결되어있는 database column명과 결합되고, BQLEDBMS에서 사용하는 BQL문으로 변환, 추가되어 Database로 부터 자료를 가져오게된다. 예를 들어 NUMBER type인 field에 대해 제한적으로 자료를 조회하려면 [Enter query] key를 누른 후 다음과 같이 처리된다.

Typed into Field	Effect on Query
12	column의 값이 12인 모든 자료
<12	column 값이 12보다 작은 모든자료
Bin (12,13,14)	column의 값이 12,13,14인 모든 자료
Between 12 and 15	column의 값이 12에서 15인 모든 자료
@@	syntax error발생

위의 예에서 보여준 것처럼 @@를 입력하여 syntax error를 발생시키면 BQLEDBMS에서 생성한 BQL문장을 볼수가 있다. query를 실행하면 다음과 같은 error가 발생한다.

FM1-40505: DBACLE error — unable to perform query.

이때 [Display Error] key를 눌러보면 BQLEDBMS가 database로 부터 data를 가져오기위해 생성한 error가 포함된 BQL문장의 list를 볼 수 있다.

Enter query mode를 사용하는 이와같은 제한된 query의 수행은 form designer에서 PRE-QUERY trigger를 사용하여 처리할 수 있다. PRE-QUERY trigger는 EXECUTE_QUERY event가 처리되기전에 수행(field)된다. PRE-QUERY trigger는 [Enter Query] key를 누르지 않고도 같은 결과를 갖게한다. PRE-QUERY trigger에 적당한 BQL문을 입력함으로써 [Enter Query] key를 눌렀을 때와 같은 제한적인 query를 수행할 수 있는 것이다.

NUMBER type의 DB-field인 :BLDCE.FIELD를 가정해보자.

위에서 든 enter query와 같은 결과를 갖는 PRE-QUERY trigger의 예를 보자.

Typed into Field	PRE-QUERY Trigger Statement
12	:BLDCE.FIELD := 12;
<12	COPY ('<12', 'BLDCE.FIELD');
Bin (12,13,14)	COPY ('Bin (12,13,14)', 'BLDCE.FIELD');
Between 12 and 15	COPY ('Between 12 and 15', 'BLDCE.FIELD');

```

-----
SAMPLE
-----

```

Special

Field Name	Type	Attributes	Description
CONTROL.M	CHAR		control field for the name
CONTROL.Z	NUMBER	fixed length = 2	control field for two zip code
CHAR			DB-field for the name
BLOCK.ZIP	NUMBER		DB-field for zip code

PSE-QUERY Trigger Code on BLOCK

```

+-----+
if (:CONTROL.M is null) and (:CONTROL.Z is null) then
    message('You must enter a name and/or a zip to search. ');
    raise form_trigger_failure;
end if;

if :CONTROL.M is null then
    copy('***** ' || to_char(:CONTROL.Z*1000) || ' and ' ||
        to_char((:CONTROL.Z+1)*1000-1), 'BLOCK.ZIP');
elseif :CONTROL.Z is null then
    :BLOCK.NAME := :CONTROL.M || 'x';
else
    copy('***** ' || to_char(:CONTROL.Z*1000) || ' and ' ||
        to_char((:CONTROL.Z+1)*1000-1), 'BLOCK.ZIP');
    :BLOCK.NAME := :CONTROL.M || 'x';
end if;
+-----+

```

```

-----
주의사항
-----

```

1. field query length가 query문장의 길이가 충분하도록 지정하여야 한다.
2. number field의 값을 지정할 때는 assign문(:=)을 사용하여야 한다.
3. alphanumeric string을 지정할 때는 COPY packaged procedure를 사용한다.
4. query string에서 지정된 값이 null이면 query가 실행될 때 syntax error가 발생하기 때문에 이 경우에 대한 처리를 해주어야 한다.

(V45) ALERT을 이용하여 MESSAGE를 처리하는 방법 (ADVANCED)

Bulletin no : 10274

Form에서 Message List에 Message를 표시하기보다는 Alert Box를 사용하여 Message를 표시하는 방법에 대해 살펴본다.

Message Library를 생성하기 위해 다음과 같은 Log ic를 사용한다. 만약 User가 Message의 내용을 바꾸고 싶다면, User는 단지 Library만 편집하면 된다.

1. 필요한 Database Object를 생성한다.

```
-----  
Create Table Message_Library  
  (Message_Num      Number           Not Null,  
   Message_Text      Varchar2(80)     Not Null,  
   Message_Help_Text Varchar2(2000)    Not Null,  
   Message_Type      Varchar2(1)      Not Null,  
   Constraint Pk_Message_Num  
       Primary Key (Message_Num),  
   Constraint Check_Message_Type  
       Check (Message_Type In ('E', 'U', 'I', 'V', 'D'))  
);  
-----
```

2. Message_Library Table이 생성되고나면 필요한 Message들을 등록한다.

```
-----  
Insert Into Message_Library  
  Values(-1,  
         'Unknown Message Type For Specified Message',  
         'Please Contact Your Dba',  
         'E');  
  
Insert Into Message_Library  
  Values(-2,  
         'Requested Message Number Not Found In Message Library',  
         'Please Contact Your Dba',  
         'E');  
-----
```

3. Form에 Message를 표시 할 때 다음과 같은 Alert Box를 생성한다.

'Error_Alert' : 시정적인 Error를 표시한다.
 Ok와 Help Button을 갖는다.
 'Warning_Alert' : 주의사항을 표시한다.
 Ok와 Help Button을 갖는다.
 'Info_Alert' : 안내사항을 표시한다.
 Ok와 Help Button을 갖는다.
 'Yes_No_Alert' : Yes/No의 확인에 사용한다.
 Yes, No, Help Button을 갖는다.
 'Ok_Cancel_Alert' : Ok/Cancel의 확인에 사용한다.
 Ok, Cancel, Help Button을 갖는다.

4. 해당하는 Message를 표시하는 Procedure를 생 한다.

해당Message의 번호와 5개의 Input Parameter를 갖는다. 만약 Message에 대시될 값이 없다면, Procedure는 Null Value인 5개의 Parameter를 가지고 호출되게 된다.

Procedure에서 Message_Library Table로 부터 해당 Message의 Type이 무엇인지 확인하여 Alert Box의 Type을 결정한다. Message의 Type이 *I*라면 Information Type으로 하고, *W*라면 Warning Type으로 하고, *E*라면 Stop Type을 표시한다. 모든 Alert Box에 나타날 Message는 Message_Library Table에서 조회하여 표시한다. Procedure는 입력받은 Button에 대한 Out변수에 담아 Return하게된다. Return된 값에 따라 User가 어떤 Button을 눌렀는지를 알게되는 것이다.

Procedure Display_Error_Prc(

P_Message_Number	In	Number,
P_Place_Holder1	In	Varchar2,
P_Place_Holder2	In	Varchar2,
P_Place_Holder3	In	Varchar2,
P_Place_Holder4	In	Varchar2,
P_Place_Holder5	In	Varchar2,
P_Button_Value	Out	Number,
P_Error_Help_Text	Out	Varchar2) Is

Alert_In Alert;
 Alert_Button Number;
 Err_Message Varchar2(200);
 Err_Help Varchar2(300);
 Err_Type Varchar2(1);

Cursor Errg_Cur Is

Select Message_Text, Message_Help_Text, Message_Type
 From Message_Library
 Where Message_Num = P_Message_Number;

Begin

Open Errg_Cur;
 Fetch Errg_Cur Into Err_Message, Err_Help, Err_Type;
 If Errg_Cur%Notfound Then

```

    When No_Data_Found:
End If:
Close Prog_Cur:

Err_Message := Replace(Err_Message, '<<1>>', P_Place_Holder1);
Err_Message := Replace(Err_Message, '<<2>>', P_Place_Holder2);
Err_Message := Replace(Err_Message, '<<3>>', P_Place_Holder3);
Err_Message := Replace(Err_Message, '<<4>>', P_Place_Holder4);
Err_Message := Replace(Err_Message, '<<5>>', P_Place_Holder5);
Err_Message := To_Char(P_Message_Number) & ' ' & Err_Message;

Err_Help := Replace(Err_Help, '<<1>>', P_Place_Holder1);
Err_Help := Replace(Err_Help, '<<2>>', P_Place_Holder2);
Err_Help := Replace(Err_Help, '<<3>>', P_Place_Holder3);
Err_Help := Replace(Err_Help, '<<4>>', P_Place_Holder4);
Err_Help := Replace(Err_Help, '<<5>>', P_Place_Holder5);
P_Error_Help_Text := Err_Help;

If (Err_Type = 'E') Then
    Alert_In := Find_Alert('Error_Alert');
ElseIf (Err_Type = 'U') Then
    Alert_In := Find_Alert('Warning_Alert');
ElseIf (Err_Type = 'I') Then
    Alert_In := Find_Alert('Info_Alert');
    ElseIf (Err_Type = 'Y') Then
        Alert_In := Find_Alert('Yes_No_Alert');
        ElseIf (Err_Type = 'D')
            Then Alert_In := Find_Alert('Ok_Cancel_Alert');
            Else
                /* Unknown Value For Error Message Type */
                Display_Error_Pec(-1, Null, Null, Null, Null, Null,
                                P_Button_Value, P_Error_Help_Text);

                If P_Button_Value = Alert_Button3 Then
                    Display_Error_Help_Pec(P_Error_Help_Text);
                End If:
            End If:
        End If:

If (Err_Type In ('E', 'U', 'I', 'Y', 'D'))
Then
    Set_Alert_Property(Alert_In, Alert_Message_Text, Err_Message);
    Alert_Button := Show_Alert(Alert_In);
    P_Button_Value := Alert_Button;
End If:
Exception
When No_Data_Found Then
    /* Error Message Number Provided To This Procedure Is Not Found In The Message_Library

```


Table 4/

```

Display_Error_Pec(-2, To_Char(P_Message_Number), Null, Null, Null, Null,
                  P_Button_Value, P_Error_Help_Text);

If P_Button_Value = Alert_Button3 Then
    Display_Error_Help_Pec(P_Error_Help_Text);
End If;

Select Form_Trigger_Failure;
When Others Then
    Begin
        If (Sqlcode = 100) Then
            Select No_Data_Found;
        Elif (Sqlcode = -100501) Then
            Select Form_Trigger_Failure;
        Else
            Message(Sqlwarn);
            Select Form_Trigger_Failure;
        End If;
    End;
End;
-----

```

5. Message포시를 요청하는 Trigger를 생성한다.

Error번호 '20710'에 해당하는 Message를 표시해야 하는 경우를 생각해보자.

```

U_Button_Value      Number;
U_Err_Help          Message_Library.Message_Help_Text%Type;

Begin
    Display_Error_Pec(20710, Null, Null, Null, Null, Null, U_Button_Value, U_Err_Help);
    If U_Button_Value = Alert_Button3 Then
        Display_Error_Help_Pec(U_Err_Help);
    End If;
End;
-----

```

'<<1>> Is Not An Active Contract Number'라는 Message Text에서 <<1>>부분을 실제의 Contract Number로 대체되어 표시되게 하려면 다음과 같이 처리한다.

```

U_Button_Value      Number;
U_Err_Help          Message_Library.Message_Help_Text%Type;

Begin
    Display_Error_Pec(20710, To_Char(:Block.Contract_Number), Null, Null, Null, Null,
                  U_Button_Value, U_Err_Help);
    If U_Button_Value = Alert_Button3 Then

```

```

        Display_Error_Help_Pec(U_Err_Help);
    End If;
End;

```

5. 포함된 Error Message를 표시하는 Procedure를 생성한다.

```

Procedure Display_Error_Help_Pec(Err_Help in Varchar2) Is
Begin
    :Control_Block_Menu.Help_Text := Err_Help;
    Show_Window('Help_Window');
End;

```

(V45) ALERT을 이용하여 MESSAGE를 처리하는 방법 (BASIC)

Bulletin no : 10275

Introduction

기본적으로 User가 생성하거나, Oracle Form 4.5에서 내주는 모든 Message와 Error는 Message List에 Display된다. Oracle Form 4.5에서는 Alert를 이용하여 Error를 처리 할 수 있다. 이것은 필요한 시점에 User에게 좀더 Message를 강조하여 확인하게 하는 효과를 갖는다..

Alert생성방법

Alert은 두 지 Type으로 나누어 볼 수 있다. 하나는 Message를 처리하기 위한 것이고, 하나는 Error가 발생한 경우를 처리하기 위한 것이다.

1. Error용 Alert생성

- Alert Mode를 Click한 후에 Navigator --> Create선택
- Properties Window에서
- Alert의 이름설정 : Error_Alert
- Alert Style Property 설정 : Stop
- Alert에서 나타내고 싶은 Button을 선택하거나 Default를 사용

2. Message용 Alert생성

위와 같은 방법을 이용하여 Message용 Alert을 하나 더 생성한다.

- Alert의 이름 설정 : Message_Alert
- Alert Style Property 설정 : Caution

Alert를 사용하기 위한 Trigger 생성

Form Level의 주개의 Trigger를 생성한다. : On-Message And On-Error.

On-Message Trigger는 Form에 Message가 표시될 때마다 Fire될 것이고, On-Error Trigger는 Error가 발생 할 때마다 Fire될 것이다.

On-Error

Declare

Alert_Button Number;

Begin

Set_Alert_Property('Error_Alert', Alert_Message_Text, Error_Text);

Alert_Button := Show_Alert('Error_Alert');

End;

```

Der-Message
Declare
    Alert_Button Number;
Begin
    Set_Alert_Property('Message_Alert', Alert_Message_Text, Message_Text);
    Alert_Button := Show_Alert('Message_Alert');
End;
-----

```

Alert를 이용하여 User가 생성한 Message를 처리하는 방법

위의 두 Trigger들은 Oracle Form4.5에서 내주는 모든 Error와 Message를 처리해 주지만, 만약 User가 생성한 Message가 다른 그것은 Message List에 나타나게 된다. 이와 같은 경우의 Message를 Alert를 이용하여 표시하는 방법은 다음과 같다.

1. Alert를 생성한다.
 - 위에서와 같은 방법으로 또하나의 Alert를 생성한다.
 - 이름을 설정한다 : User_Alert
 - Alert Style Property를 설정한 : Note
 - 표시하고 싶은 Button을 설정하여 설정한다.

2. Alert를 사용할 Procedure를 생성한다.

```

Procedure Alerts (Msg VARCHAR2) Is
    Alert_Button Number;
Begin
    Set_Alert_Property('User_Alert', Alert_Message_Text, Msg);
    Alert_Button := Show_Alert('User_Alert');
End;

```

3. 필요한 Trigger에서 위에서 생성한 Procedure를 호출한다.

```

Trigger: User-Validation
Item : Popten
Declare
    Msg VARCHAR2(80);
Begin
    If :Popten > 50 Then
        Msg := ('Invalid Number. Please Try Again. ');
        Alerts(Msg);
        Raise Form_Triggers_Failure;
    End If;
End;

```

(334) USER가 아무 ACTION이 없이 일정 시간이 지나면 자동으로 LOGOUT시키는 방법

Bulletin no : 10279

* BQL>PLU>에서

CREATE PROFILE Profile_name LIMIT IDLE_TIME N; 실행

: idle time은 분단위로 setting한다.

BQL Language Reference manual참조.

* 기존에 생성되어 있는 USER에 대하여

ALTER USER PROFILE Profile_name; 실행

: session에서 아무 action이 없이 위해서 지정한 시간이 지나면

error message를 내주고 강제로 rollback이 된 후 session이 끊어지게된다.

BQL Language Reference manual참조.

* \$ORACLE_HOME/dbs/initSID.ora 파일에서

SESSION LIMIT parameter의 값을 TRUE로 SETTING한 후

DBACLE를 RESTART하여야만 위해서 setting한 profile에 대한 처리가 적용된다.

Admin Manual참조.

(V4.5) CUSTOMIZING LOGON BEHAVIOR

Bulletin no : 10278

Introduction

By default, Oracle Forms will attempt to establish a connection to a database when starting up a Runform session. If the connect information (username, password, connect string) is not specified as parameters to the Runform executable, Oracle Forms will pop up a logon window.

This bulletin details methods of commonly-requested customization of this logon behavior, including running a form without logging onto a database, defaulting the connect string, and creating a custom logon window.

Objective

This bulletin will primarily focus on the On-Logon trigger and the LOGON built-in. The On-Logon trigger, if defined, replaces the default logon behavior of Oracle Forms. Within that trigger, the LOGON built-in actually establishes the connection. If called from outside the On-Logon trigger, the LOGON built-in calls the On-Logon trigger.

Running Without a Connection

Forms that do not access any data do not require connections to the database. By telling Forms not to establish a connection, startup time is reduced and the operator will not be prompted for connect information. The following On-Logon trigger eliminates logon entirely:

On-Logon (Form Level)

NULL;

Small Changes to Default Behavior

Here is an On-Logon trigger which mimics the default behavior of Oracle Forms when it performs a connect.

On-Logon (Form Level)

DECLARE

un VARCHAR2(30);

pw VARCHAR2(30);

cc VARCHAR2(30);

```

    upc          VARCHAR2(200);
    connected    BOOLEAN := FALSE;
    times        NUMBER := 3;

    PROCEDURE get_connect_info IS
    BEGIN
        us := GET_APPLICATION_PROPERTY(USERNAME);
        pw := GET_APPLICATION_PROPERTY(PASSWORD);
        cx := GET_APPLICATION_PROPERTY(CONNECT_STRING);

    END;

BEGIN
    get_connect_info;
    IF us IS NOT NULL THEN
        LOGON(us, pull'D'illcx, FALSE);
        IF FORM_SUCCESS THEN           /* Successful login */
            connected := TRUE;
        END IF;
    END IF;

    WHILE connected = FALSE and times > 0 LOOP
        upc := us || pullcx;
        LOGON_SCREEN;
        get_connect_info;
        IF upc = us || pullcx THEN
            EXIT;
        END IF;
        LOGON(us, pull'D'illcx, FALSE);
        IF FORM_SUCCESS THEN           /* Successful login */
            connected := TRUE;
        END IF;
        times := times - 1;
    END LOOP;

    IF NOT connected THEN
        RAISE FORM_TRIGGER_FAILURE;
    END IF;
END;

```

Note that the GET_APPLICATION_PROPERTY(CONNECT_STRING) will not work in versions prior to 4.0.13 of Oracle Forms. In prior versions, the connect string is part of the password.

Small deviations from the default connect behavior can be accomplished by modifying this code:

- * To allow more or fewer login attempts, increase the initial value of the counter variable, times.
- * To default the connect string, change the call to LOGON from:
 LOGON(us, pull'D'illcx, FALSE);
 to use a hard-coded connect string:

```
LOGON(us, pull'D', FALSE);
```

* These lines can also be modified to hardcode username or password.

Hiding the Logon Information

For security reasons, many customers do not want their users to be able to access the database except through the form. One way to do this is to have the form handle connecting to the database, which keeps passwords away from the users. In V7, the best way to do this is to have the form set role to a privileged role. However, a quick and dirty way to do it is to have the form connect to a privileged, secret ORACLE account.

The following Or-Logon trigger establishes a connection using hard-coded values for username and password. Users will be able to connect to the database without knowing the account or password:

Or-Logon (Form Level)

```
LOGON('SCOTT','TIGER@MYHDBT:U71B');
```

Deferring Logon

Connection can be deferred until later in the Forms processing.

Or-Logon (Form Level)

```
DECLARE
```

```
us CHAR(30);
```

```
pw CHAR(30);
```

```
cx CHAR(30);
```

```
BEGIN
```

```
DEFAULT_VALUE('N', 'global_logon_now');
```

```
IF :global_logon_now = 'Y' THEN
```

```
LOGON_SCREEN;
```

```
us := GET_APPLICATION_PROPERTY(USERNAME);
```

```
pw := GET_APPLICATION_PROPERTY(PASSWORD);
```

```
cx := GET_APPLICATION_PROPERTY(CONNECT_STRING);
```

```
LOGON(us, pull'D'||cx);
```

```
END IF;
```

```
END;
```

When the form starts up, it will not try to log on since the flag logon_now will be defaulted to N. When you do wish to log onto the database, use the syntax:

```
:global_logon_now := 'Y';
```

```
LOGON(NULL, NULL);
```


This sets the flag to Y and causes the On-Login trigger to fire, as the form connects to the database.

Customizing the Login Screen

The built-in LOGIN_SCREEN calls up the default Form login screen to capture the username, password, and connect string. However, you can also use a customized login screen. The following tutorial creates a form which can be used as a replacement login screen from any form.

NOTE: Coordinate System is assumed to be pixel.

1. Create a new form module and change only the following properties:

Name: LOGIN
Title: LOGIN (v4,5)

2. Modify only the following properties for WINDOW:

(v4,0, Create this window):

Name: LOGIN_WINDOW
Width: 128
Height: 100
Title: Login
Style: Dialog
Modal: TRUE
Fixed Size: TRUE
Iconifiable: FALSE
Inherit Name: FALSE
Zoomable: FALSE

3. Create a new canvas (v4,0, Modify CANVAS) and modify only the following properties:

Name: LOGIN_CANVAS
Window: LOGIN_WINDOW
Width: 128
Height: 100

4. Modify only the following properties for LOGIN_WINDOW:

View: LOGIN_CANVAS

5. Create a new block and modify only the following properties:

Block Name: LOGIN_BLOCK

6. Create three text items on canvas LOGIN_CANVAS for block LOGIN_BLOCK with properties as follows:

Name: USERNAME

```

X Position: 12
Y Position: 8
Width: 102
Height: 18

```

```

Name: PASSWORD
X Position: 12
Y Position: 31
Width: 102
Height: 18
Secure: TRUE

```

```

Name: CONNECT
X Position: 12
Y Position: 54
Width: 102
Height: 18

```

7. Create two push buttons on canvas LOGON_CANVAS for block LOGON_BLOCK with properties as follows:

```

Name: LOGON
X Position: 12
Y Position: 84
Width: 52
Height: 18
Label: Logon
Mouse Navigates: FALSE
Default Button: TRUE

```

```

Name: CANCEL
X Position: 72
Y Position: 84
Width: 52
Height: 18
Label: Cancel
Mouse Navigates: FALSE

```

8. Code the following program units:

```

PROCEDURE login(status BOOLEAN DEFAULT TRUE) IS
BEGIN
    IF status THEN
        :global.logged_on := 'TRUE';
    ELSE
        :global.logged_on := 'FALSE';
    END IF;

```

```

EXIT_FORM;

END;

PROCEDURE Initialize_Form IS
    un VARCHAR2(30) := GET_APPLICATION_PROPERTY(USERNAME);
    pw VARCHAR2(30) := GET_APPLICATION_PROPERTY(PASSWORD);
    cx VARCHAR2(30) := GET_APPLICATION_PROPERTY(CONNECT_STRING);
BEGIN
    DEFAULT_VALUE('3', 'global.login_timeout');
    DEFAULT_VALUE(un, 'global.username');
    DEFAULT_VALUE(pw, 'global.password');
    DEFAULT_VALUE(cx, 'global.connect_string');
    :login_block.username := :global.username;
    :login_block.password := :global.password;
    :login_block.connect := :global.connect_string;
END;

FUNCTION connect_to RETURN BOOLEAN IS
BEGIN
    IF :login_block.username IS NULL THEN
        NULL;
    END IF;
    LOGIN(:login_block.username,
          :login_block.password||'@'||:login_block.connect, FALSE);
    IF FORM_SUCCESS THEN
        RETURN TRUE;
    ELSE
        RETURN FALSE;
    END IF;
END;

```

B. Code the following triggers:

On-Login (Form Level)

```

LOGIN(:login_block.username,
      :login_block.password||'@'||:login_block.connect,
      FALSE);

```

When-New-Form-Instance (Form Level)

```

Initialize_Form;

```

When-Button-Pressed (Item Level, on LOGIN_BLOCK.CANCEL)

```

Leave(FALSE);

```

When-Button-Pressed (Item Level, on LOGIN_BLOCK.LOGIN)

```

BEGIN

```

```

    IF connect_to THEN
        Inven(TRUE);
    END IF;

    :global.logon_tries := TO_NUMBER(:global.logon_tries) - 1;
    IF TO_NUMBER(:global.logon_tries) = '0' THEN
        Inven(FALSE);
    END IF;
END;

```

When-Window-Activated (Form Level)

```

DECLARE
    screen_height NUMBER;
    screen_width  NUMBER;
    window_height NUMBER;
    window_width  NUMBER;
BEGIN
    IF :SYSTEM.EVENT_WINDOW = 'LOGIN_WINDOW' THEN
        screen_height := GET_APPLICATION_PROPERTY(DISPLAY_HEIGHT);
        screen_width  := GET_APPLICATION_PROPERTY(DISPLAY_WIDTH);
        window_height := GET_WINDOW_PROPERTY(:SYSTEM.EVENT_WINDOW, HEIGHT);
        window_width  := GET_WINDOW_PROPERTY(:SYSTEM.EVENT_WINDOW, WIDTH);
        GET_WINDOW_PROPERTY('LOGIN_WINDOW', X_POS,
                             (screen_width - window_width)/2);
        GET_WINDOW_PROPERTY('LOGIN_WINDOW', Y_POS,
                             screen_height/2 - window_height);
    END IF;
END;

```

10. Generate and save this form.
11. Create a new form. This form is an example of how to use the logon form that has just been created.
12. Create a default block, giving this form some items.
13. Code the On-Logon trigger as follows:

On-Logon (Form Level)

```

CALL_FORM('logon', NO_HIDE);
IF :global.logged_on != 'TRUE' THEN
    RAISE FORM_TRIGGER_FAILURE;
END IF;

```

This will complete the coding for the main form. When the form now requires a connection to the database, it will use this customized logon screen instead of the Forms default one.

(V4.5) BASE TABLES FOR DEVELOPER 2000 TOOLS

Bulletin no : 10282

This document explains how to install base tables for Developer 2000 Tools (i.e. Oracle Forms V4.5, Oracle Reports V2.5 and Oracle Graphics V2.5). Developer 2000 Tools allow saving modules to file system or database. To be able to save modules to the database, all base tables must exist. If any of the base tables are missing or corrupt, usually ORA-042 : "Table or view does not exist" or other errors will occur.

Types of Base Tables

There are a number of tables, views and synonyms which are shared by Oracle Forms, Oracle Reports and Oracle Graphics. They have prefixes TOOLS, ORS, OG, GG or GE. Base tables for Oracle Forms 4.5, Reports 2.5, and Graphics 2.5 have prefixes FORMS, BMR2, and GG respectively.

Check Common Base Tables, Views and Synonyms

Before you install the base tables, you need to find out if they exist already and thus skip the installation of existing objects. This process can be started from checking objects shared by all ORE products. Put the following code into a SQL*Plus script called checkobjects.sql.

```
set space 0
set escape off
set pagesize 100
column object_name format a30
column owner format a30
column object_type format a20
select object_name, owner, object_type from sys.dba_objects
where object_name like 'TOOLS_%' escape '\' or
      object_name like 'ORS%' or
      object_name like 'OG_%' escape '\' or
      object_name like 'GG_%' escape '\' or
      object_name like 'GE_%' escape '\'
order by object_name, object_type;
```

Now log into SQL*Plus as SYSTEM (default password is MANAGER) and type:

```
SQL>start checkobjects.sql
```

You should see the following list of objects returned :

OBJECT_NAME	OWNER	OBJECT_TYPE
DE_ATTACHED_LIBS	PUBLIC	SYNONYM
DE_ATTACHED_LIBS	SYSTEM	VIEW
DE_ATTACHED_LIBS_IDX	SYSTEM	INDEX
DE_ATTACHED_LIBS	SYSTEM	TABLE
CC_CHARTING	PUBLIC	SYNONYM
CC_CHARTING	SYSTEM	VIEW
CC_FIELD	PUBLIC	SYNONYM
CC_FIELD	SYSTEM	VIEW
CC_REFERENCE	PUBLIC	SYNONYM
CC_REFERENCE	SYSTEM	VIEW
CC_CHARTING	SYSTEM	TABLE
CC_FIELD	SYSTEM	TABLE
CC_REFERENCE	SYSTEM	TABLE
DBATTNIBS	PUBLIC	SYNONYM
DBATTNIBS	SYSTEM	VIEW
DBLFWBSC	PUBLIC	SYNONYM
DBLFWBSC	SYSTEM	TABLE
DBLFWBSC_ID	SYSTEM	SEQUENCE
DBLFWBSC_ID	PUBLIC	SYNONYM
DBDDBJMAP	PUBLIC	SYNONYM
DBDDBJMAP	SYSTEM	TABLE
DBBSEQUENCES	PUBLIC	SYNONYM
DBBSEQUENCES	SYSTEM	TABLE
DBBSTRINGS	PUBLIC	SYNONYM
DBBSTRINGS	SYSTEM	TABLE
DBBSTRUCTS	PUBLIC	SYNONYM
DBBSTRUCTS	SYSTEM	VIEW
DBTFWBSC	PUBLIC	SYNONYM
DBTFWBSC	SYSTEM	TABLE
DBTFWBSC_ID	SYSTEM	SEQUENCE
DBTFWBSC_ID	PUBLIC	SYNONYM
TOOL_ACCESS	PUBLIC	SYNONYM
TOOL_ACCESS	SYSTEM	VIEW
TOOL_ACCESSIBLE	PUBLIC	SYNONYM
TOOL_ACCESSIBLE	SYSTEM	VIEW
TOOL_ACCESSIBLE_IDX	SYSTEM	INDEX
TOOL_ACCESS_IDX	SYSTEM	INDEX
TOOL_COMMENT	PUBLIC	SYNONYM
TOOL_COMMENT	SYSTEM	VIEW
TOOL_COMMENT_IDX	SYSTEM	INDEX
TOOL_DEPEND	PUBLIC	SYNONYM
TOOL_DEPEND	SYSTEM	VIEW

TOOL_DEPENDENT	PUBLIC	SYNONYM
TOOL_DEPENDENT	SYSTEM	VIEW
TOOL_LIBRARY	PUBLIC	SYNONYM
TOOL_LIBRARY	SYSTEM	VIEW
TOOL_LIBRARY_IDX	SYSTEM	INDEX
TOOL_LBAW	PUBLIC	SYNONYM
TOOL_LBAW	SYSTEM	VIEW
TOOL_LBAW_ID	SYSTEM	SEQUENCE
TOOL_LBAW_ID	PUBLIC	SYNONYM
TOOL_LBAW_IDX1	SYSTEM	INDEX
TOOL_LBAW_IDX2	SYSTEM	INDEX
TOOL_LTEXT	PUBLIC	SYNONYM
TOOL_LTEXT	SYSTEM	VIEW
TOOL_LTEXT_ID	SYSTEM	SEQUENCE
TOOL_LTEXT_ID	PUBLIC	SYNONYM
TOOL_LTEXT_IDX1	SYSTEM	INDEX
TOOL_LTEXT_IDX2	SYSTEM	INDEX
TOOL_MODID	SYSTEM	SEQUENCE
TOOL_MODID	PUBLIC	SYNONYM
TOOL_MODULE_IDX	SYSTEM	INDEX
TOOL_MODULE	PUBLIC	SYNONYM
TOOL_MODULE	SYSTEM	VIEW
TOOL_MODULE_IDX	SYSTEM	INDEX
TOOL_PLSQL	PUBLIC	SYNONYM
TOOL_PLSQL	SYSTEM	VIEW
TOOL_PLSQL_IDX	SYSTEM	INDEX
TOOL__ACCESS	SYSTEM	TABLE
TOOL__COMMENT	SYSTEM	TABLE
TOOL__DEPEND	SYSTEM	TABLE
TOOL__LIBRARY	SYSTEM	TABLE
TOOL__LBAW	SYSTEM	TABLE
TOOL__LTEXT	SYSTEM	TABLE
TOOL__MODULE	SYSTEM	TABLE
TOOL__PLSQL	SYSTEM	TABLE
UC_COLOR	PUBLIC	SYNONYM
UC_COLOR	SYSTEM	VIEW
UC_DRAWING	PUBLIC	SYNONYM
UC_DRAWING	SYSTEM	VIEW
UC_FONT	PUBLIC	SYNONYM
UC_FONT	SYSTEM	VIEW
UC_IMAGE	PUBLIC	SYNONYM
UC_IMAGE	SYSTEM	VIEW
UC__COLOR	SYSTEM	TABLE
UC__DRAWING	SYSTEM	TABLE
UC__FONT	SYSTEM	TABLE

UC_INAME	SYSTEM	TABLE
----------	--------	-------

BB row selected.

Make sure all the objects listed above exist. The owner and type of the objects also have to be the same as above. If there is any discrepancy, you need to re-install those objects. Use SQL*Plus command DESCRIBE to check the structure of some of the tables. All the string data type must be VARCHAR2, otherwise re-installation of base objects is also needed.

Install / Re-install Common Base Tables

BEFORE YOU RUN ANY SCRIPTS, MAKE SURE YOU HAVE A BACKUP OF THE DATABASE AND SOURCE FILES FOR ALL APPLICATIONS (FHM AND HMB FILES FOR ORACLE FORMS; PLL FILES FOR PL/SQL LIBRARY; RPT AND RQL FILES FOR ORACLE REPORTS; GGP FILES FOR ORACLE GRAPHICS). Since the common base tables are shared by different OME Tools, dropping them may cause loss of data from other tools.

The following scripts need to be run in order to build the base tables. Log into SQL*Plus as SYSTEM to execute them and make sure the order in which they are specified is followed. Note that when you run the "grant" scripts (the xxgrant.sql files), you will be prompted for a username. Specify a username if you want to grant access to one user at a time, or you can specify PUBLIC and every user will have access to the base tables.

On Microsoft Windows

The installation process creates a number of program groups which appear on your desktop. The install and upgrade scripts can be run by double clicking the icons in the Developer 2000 Administration program group. The actual scripts to install the common base tables for the Developer 2000 tools are in C:\ORAWIN\Developer 2000\SQL. The product specific script are located in C:\ORAWIN\PRODUCT\SQL, where PRODUCT refers to product name eg. for forms45 the scripts are in C:\ORAWIN\FORMS45\SQL directory.

The following are scripts needed to be run to rebuild the base tables.

1. TOOLBOP.SQL	TOOLBILB.SQL	TOOLCNTY.SQL
2. ROBBOP.SQL	ROBBILB.SQL	ROBCNTY.SQL
3. UCBBOP.SQL	UCBBILB.SQL	UCBCNTY.SQL
4. DEBBOP.SQL	DEBBILB.SQL	DEBCNTY.SQL

Additionally, you need (only for Oracle Graphics 2.5)

5. GGBOP.SQL	GGBILB.SQL	GGBCNTY.SQL
--------------	------------	-------------

Check Product Specific Base Tables

You are not required to install product-specific base tables for all OME Tools. Any one of them can be installed independently. Log into SQL*Plus as SYSTEM and use the following SELECT statements to check the existence of the base tables first.

Oracle Forms

```
select object_name, owner, object_type from sys.dba_objects
where object_name like 'FM45%'
order by object_name;
```

you should get the following list:

FM45_BINDVAR	PUBLIC	SYNONYM
FM45_BUFFER	PUBLIC	SYNONYM
FM45_ENABLED_ROLES	PUBLIC	SYNONYM
FM45_CBP	PUBLIC	SYNONYM
FM45_OBJECT	PUBLIC	SYNONYM
FM45__BINDVAR	SYSTEM	TABLE
FM45__BUFFER	SYSTEM	TABLE
FM45__CBP	SYSTEM	TABLE
FM45__OBJECT	SYSTEM	TABLE
FM45_ENABLED_ROLES	SYSTEM	VIEW

10 rows selected.

Important: Please note that unlike Oracle Forms 4.0 which utilizes 82 base table objects, Oracle Forms 4.5 utilizes only the above 10 base table objects. As a result, each of the above base table objects tend to be very large in size.

Oracle Reports

```
select object_name, owner, object_type from sys.dba_objects
where object_name like 'RM2%'
order by object_type;
```

you should get the following list:

RM2_ANCHOR_IDX	SYSTEM	INDEX
RM2_BODY_LOCATION_IDX	SYSTEM	INDEX
RM2_BITMAPINDEX_IDX	SYSTEM	INDEX
RM2_COLUMN_IDX	SYSTEM	INDEX
RM2_ELEMENT_IDX	SYSTEM	INDEX
RM2_FRAME_IDX	SYSTEM	INDEX
RM2_LAYOUT_GROUP_IDX	SYSTEM	INDEX
RM2_LINE_IDX	SYSTEM	INDEX
RM2_MATH_IDX	SYSTEM	INDEX
RM2_LIST_IDX	SYSTEM	INDEX
RM2_TEXT_SEQUENCE_IDX	SYSTEM	INDEX
RM2_QUERY_IDX	SYSTEM	INDEX
RM2_PARAM_FORM_IDX	SYSTEM	INDEX
RM2_DOC_DOCUMENT_IDX	SYSTEM	INDEX
RM2_DOC_COLUMN_MAP_IDX	SYSTEM	INDEX

BMJ2_LAYOUT_IDX	SYSTEM	INDEX
BMJ2_GROUP_IDX	SYSTEM	INDEX
BMJ2_DISPLAY_TAG_IDX	SYSTEM	INDEX
BMJ2_DATA_MODEL_IDX	SYSTEM	INDEX
BMJ2_BUTTON_IDX	SYSTEM	INDEX
BMJ2_FIELD_IDX	SYSTEM	INDEX
BMJ2_ANCHOR	PUBLIC	SYNONYM
BMJ2_BODY_LOCATION	PUBLIC	SYNONYM
BMJ2_NO_DISPLAY	PUBLIC	SYNONYM
BMJ2_BUTTON	PUBLIC	SYNONYM
BMJ2_COLUMN	PUBLIC	SYNONYM
BMJ2_DATA_MODEL	PUBLIC	SYNONYM
BMJ2_DISPLAY_TAG	PUBLIC	SYNONYM
BMJ2_ELEMENT	PUBLIC	SYNONYM
BMJ2_FIELD	PUBLIC	SYNONYM
BMJ2_FRAME	PUBLIC	SYNONYM
BMJ2_GROUP	PUBLIC	SYNONYM
BMJ2_LAYOUT	PUBLIC	SYNONYM
BMJ2_LAYOUT_GROUP	PUBLIC	SYNONYM
BMJ2_LINE	PUBLIC	SYNONYM
BMJ2_LISTS	PUBLIC	SYNONYM
BMJ2_MATHK	PUBLIC	SYNONYM
BMJ2_MODULE	PUBLIC	SYNONYM
BMJ2_OCP_COLUMN_MAP	PUBLIC	SYNONYM
BMJ2_OC_DOCUMENT	PUBLIC	SYNONYM
BMJ2_PARAM_FORM	PUBLIC	SYNONYM
BMJ2_QUERY	PUBLIC	SYNONYM
BMJ2_TEXT_SEGMENT	PUBLIC	SYNONYM
BMJ2_ANCHOR	SYSTEM	TABLE
BMJ2_DISPLAY_TAG	SYSTEM	TABLE
BMJ2_DATA_MODEL	SYSTEM	TABLE
BMJ2_TEXT_SEGMENT	SYSTEM	TABLE
BMJ2_QUERY	SYSTEM	TABLE
BMJ2_PARAM_FORM	SYSTEM	TABLE
BMJ2_OC_DOCUMENT	SYSTEM	TABLE
BMJ2_OCP_COLUMN_MAP	SYSTEM	TABLE
BMJ2_MATHK	SYSTEM	TABLE
BMJ2_LISTS	SYSTEM	TABLE
BMJ2_LINE	SYSTEM	TABLE
BMJ2_LAYOUT_GROUP	SYSTEM	TABLE
BMJ2_LAYOUT	SYSTEM	TABLE
BMJ2_GROUP	SYSTEM	TABLE
BMJ2_FRAME	SYSTEM	TABLE
BMJ2_FIELD	SYSTEM	TABLE
BMJ2_ELEMENT	SYSTEM	TABLE
BMJ2_BODY_LOCATION	SYSTEM	TABLE

BMJ2_BDILEMPLATE	SYSTEM	TABLE
BMJ2_BUTTON	SYSTEM	TABLE
BMJ2_COLUMN	SYSTEM	TABLE
BMJ2_ANCHOR	SYSTEM	VIEW
BMJ2_FIELD	SYSTEM	VIEW
BMJ2_ELEMENT	SYSTEM	VIEW
BMJ2_DCB_COLUMN_MAP	SYSTEM	VIEW
BMJ2_MODULE	SYSTEM	VIEW
BMJ2_MATRIX	SYSTEM	VIEW
BMJ2_LISTB	SYSTEM	VIEW
BMJ2_LINE	SYSTEM	VIEW
BMJ2_LAYOUT_GROUP	SYSTEM	VIEW
BMJ2_LAYOUT	SYSTEM	VIEW
BMJ2_GROUP	SYSTEM	VIEW
BMJ2_TEXT_SEGMENT	SYSTEM	VIEW
BMJ2_QUERY	SYSTEM	VIEW
BMJ2_PARAM_FORM	SYSTEM	VIEW
BMJ2_DC_DOCUMENT	SYSTEM	VIEW
BMJ2_FRAME	SYSTEM	VIEW
BMJ2_BDILEMPLATE	SYSTEM	VIEW
BMJ2_COLUMN	SYSTEM	VIEW
BMJ2_DATA_MODEL	SYSTEM	VIEW
BMJ2_DISPLAY_TAG	SYSTEM	VIEW
BMJ2_BUTTON	SYSTEM	VIEW
BMJ2_BODY_LOCATION	SYSTEM	VIEW

88 rows selected.

Important: Please note that base table objects for Oracle Reports 2.5 and
Oracle Reports 2.0 are one and the same.

Oracle Graphics

```
select object_name, owner, object_type from sys.dba_objects
where object_name like 'OD%_%' escape '\'
```

order by object_type;

you should get the following list:

OD_COLUMN	PUBLIC	SYNONYM
OD_DOCUMENT	PUBLIC	SYNONYM
OD_PARAMETER	PUBLIC	SYNONYM
OD_QUERY	PUBLIC	SYNONYM
OD_SOUND	PUBLIC	SYNONYM
OD_TIMER	PUBLIC	SYNONYM
OD_VISAYEN	PUBLIC	SYNONYM
OD_COLUMN	SYSTEM	TABLE

CD_PARAMETER	SYSTEM	TABLE
CD_BOUND	SYSTEM	TABLE
CD_TINER	SYSTEM	TABLE
CD_VIBLAYEN	SYSTEM	TABLE
CD_QUERY	SYSTEM	TABLE
CD_DOCUMENT	SYSTEM	TABLE
CD_COLUMN	SYSTEM	VIEW
CD_DOCUMENT	SYSTEM	VIEW
CD_PARAMETER	SYSTEM	VIEW
CD_QUERY	SYSTEM	VIEW
CD_BOUND	SYSTEM	VIEW
CD_TINER	SYSTEM	VIEW
CD_VIBLAYEN	SYSTEM	VIEW

21 rows selected.

Important: Please note that the base table objects for Oracle Graphics 2.5 and Oracle Graphics 2.0 are one and the same.

Re-install Product Specific Base Tables

BEFORE YOU RUN ANY SCRIPTS, MAKE SURE YOU HAVE A BACKUP OF THE DATABASE AND SOURCE FILES FOR ALL APPLICATIONS (FMB AND HMB FILES FOR ORACLE FORMS; PLL FILES FOR PL/SQL LIBRARY; REP AND SQL FILES FOR ORACLE REPORTS; OGP FILES FOR ORACLE GRAPHICS).

If any object of a ODE Tools application is showing as the string data type of any table is CHAR instead of VARCHAR2, you need to re-install the base objects. Again, log into SQL*Plus as SYSTEM and execute the scripts. You can grant access to individual user or PUBLIC when you run the "grant" scripts.

Oracle Forms 4.5:

On Windows: In C:\ORANT\N\FORMS45\SQL\

FM45REP.SQL	FM45MLB.SQL	FM45CNT.SQL
-------------	-------------	-------------

Oracle Reports 2.5:

On Windows: In C:\ORANT\N\REPORTS\SQL\

RM25REP.SQL	RM25ILB.SQL	RM25CNT.SQL
-------------	-------------	-------------

Oracle Graphics 2.5:

On Windows: In C:\ORANT\N\GRAPH25\SQL\

CG25REP.SQL	CG25ILB.SQL	CG25CNT.SQL
-------------	-------------	-------------

Upgrading from Earlier Versions:

If you are upgrading from an earlier version of Developer 2000 products, you should run one or more upgrade scripts instead of creating the tables from scratch. Doing so will update the information in your tables (preserve your existing information) rather than rebuild the tables.

Note: During installation, program items are created in the Developer 2000 Administration program group for the SQL scripts needed to upgrade the Developer 2000 tables and the individual product tables. Double-click on the program items to upgrade the tables.

Oracle Forms:

From previous versions of 4.0 to 4.5:

```
Run      C:\ORADMIN\FORMS45\FB45B1L0.SQL
          FB45B1L0.SQL uxxxxxx
          FM45B1L0.SQL
          FM45CBT.SQL uxxxxxx
```

From previous versions of 4.5:

```
Run      C:\ORADMIN\Developer 2000\BQ1\BQ1ALYB.SQL
          FM45UP1.SQL
```

Oracle Reports:

Using the following table, run the upgrade SQL scripts that correspond to the version of Oracle Reports you want to upgrade. The SQL scripts listed are located in:

C:\ORADMIN\REPORTS\BQ1

Version to Version	Script
2.0.13.0 to 2.0.14.0	BRU20140.SQL
2.0.14.0 to 2.5	BRU2500.SQL
2.5 to 2.5.3.1.1B	BRU2530.SQL

Oracle Graphics:

To upgrade Oracle Graphics tables, you must run the following scripts:

```
C:\ORADMIN\Developer 2000\GG2SUP.SQL
C:\ORADMIN\GRAPH25\BQ1\GG2SUP.SQL
```

(V4.5) APPLICATION을 DB에 저장하는 방법

Bulletin no : 10909

- User가 작성한 **Form**, **SL** **Report**, **2.5 Module**를 DB에 Save하기 위해서는 그 Module을 수록하는 Table들이 먼저 Create되어 있어야 한다. 이 Table의 생성 Script들은 **install**당시에 자동적으로 만들어지며 **BQPLIB**의 **SYSTEM Account**에서 불러주어야 한다.

1. Oracle Form을 위한 Table Create

```
BQL> @C:\ORACLE\NLS\BQPLIB\T00LIB.BQL
BQL> @C:\ORACLE\NLS\BQPLIB\B08LIB.BQL
BQL> @C:\ORACLE\NLS\BQPLIB\WLIB.BQL
BQL> @C:\ORACLE\NLS\BQPLIB\BIB.BQL
BQL> @C:\ORACLE\NLS\BQPLIB\B45\BQPLIB\B45LIB.BQL
```

2. Oracle Report를 위한 Table Create

```
BQL> @C:\ORACLE\NLS\BQPLIB\T00LIB.BQL
BQL> @C:\ORACLE\NLS\BQPLIB\B08LIB.BQL
BQL> @C:\ORACLE\NLS\BQPLIB\WLIB.BQL
BQL> @C:\ORACLE\NLS\REPORT25\BQPLIB\B12LIB.BQL
```

3. Oracle Graphics를 위한 Table Create

```
BQL> @C:\ORACLE\NLS\BQPLIB\B08LIB.BQL
BQL> @C:\ORACLE\NLS\BQPLIB\T00LIB.BQL
BQL> @C:\ORACLE\NLS\BQPLIB\WLIB.BQL
BQL> @C:\ORACLE\NLS\BQPLIB\BIB.BQL
BQL> @C:\ORACLE\NLS\BQPLIB\B45\BQPLIB\B45LIB.BQL
BQL> @C:\ORACLE\NLS\GRAPH25\BQPLIB\B1LIB.BQL
```

* 상호간에 종속된 Script는 다시 실행할 필요가 없다.

(v4) F45에서 특정 FIELD에서 한/영키 자동 변환방법

Bulletin no : 10304

1. 한글 Blob을 등록하여 한/영키에서 발생된 Event를 Program에서 발생케 하여 특정 Field에서 자음으로 원하는 Mode로 전환케 한다.

2. Program은 DLL로 만들어졌으며, Ora_FFI Function으로 Call해야한다.

3. 다음은 Form4.5의 Package 내용이다.

PACKAGE ham IS

PROCEDURE ham(flag IN NUMBER);

PROCEDURE hamoff(flag IN NUMBER);

END;

PACKAGE BODY ham IS

all_handle DBA_FFI.LIBHANDLETYPE;

ham_handle DBA_FFI.FUNCHANDLETYPE;

hamoff_handle DBA_FFI.FUNCHANDLETYPE;

PROCEDURE hamload(handle IN DBA_FFI.FUNCHANDLETYPE);

pragma interface (C, hamload, 11285);

PROCEDURE ham(flag IN NUMBER) IS

BEGIN

hamload(ham_handle);

END;

PROCEDURE hamoffload(handle IN DBA_FFI.FUNCHANDLETYPE);

pragma interface (C, hamoffload, 11285);

PROCEDURE hamoff(flag IN NUMBER) IS

BEGIN

hamoffload(hamoff_handle);

END;

BEGIN

all_handle := DBA_FFI.RESISTER_LIBRARY(NULL, 'C:ORA_HAM.DLL');

ham_handle := DBA_FFI.RESISTER_FUNCTION(all_handle,
'cvttoHAM', DBA_FFI.PASCAL_STY);

```

korean2_ksuid := DBA_FFI_REGISTER_FUNCTION(dll_ksuid, 'convertKSC',
                                             DBA_FFI_PASCAL_BYTE);

DBA_FFI_REGISTER_PARAMETER(korean_ksuid,  DBA_FFI_C_INT);
DBA_FFI_REGISTER_PARAMETER(korean2_ksuid, DBA_FFI_C_INT);
END;

krm, krmcom (1);
krm, krmcom2(1);

```

4. 특정 Field의 proc_text_item의 내용

```

krm, krmcom (1);    --> Hangul   Input Mode
krm, krmcom2(1);    --> English Input Mode

```

5. "ora_krm.dll" 파일은 선미만에서 go oraclem하여, 정보 보출 상고에 가면 Download할 수 있다.

8. 수정할 부분은 ora_krm.dll의 Path만 변경한다.

(V45) FORMS4.5에서 GRAPHIC2.5로의 DATA PARAMETER PASSING

Bulletin no : 10307

o forms4.5에서 graphic을 data parameter를 이용하여 사용하는 방법.

1) forms에서의 작업

. attached library : DC

. form level에서 사용하는 trigger

```
- when-new-form-instance의 내용
  og.open('c:\oracul\graph25\text_22g.ogd',
          'display_bar.chart_item',TRUE,true);
  go_block('manager');
  execute_query;
```

- post-form의 내용

```
og.close('c:\oracul\graph25\text_22g.ogd',
         'display_bar.chart_item');
```

. 사용하는 block

```
- manager ( where clause : where job is ('manager', 'president')
            order by clause : name )
  emp table를 사용하여 생성
```

when-new-record-instance trigger를 사용하여
data parameter를 graphic에 넘겨준다.

when-new-record-instance의 내용은 아래와 같습니다.

```
declare
  val char(20) := 'chart_data';
  pl paramlist;
  rg recordgroup;
  qry varchar(2000);
  stat number;
begin
  /* prepare a query in a string */
  qry := 'select initcap(name) name, sal from emp
         where mgr = '||to_char(manager,empno);
```

```

/* try to get the ID of the 'chart_data' record group */
rg := find_group('chart_data');

/* If it doesn't exist, create the group based on the
query in the 'qry' string */
if id_null(rg) then
    rg := create_group_from_query('chart_data', qry);
end if;

/* make sure the 'chart_data' record group is empty */
delete_group_rows(rg, all_rows);

/* populate the 'chart_data' record group with the query
in the 'qry' string */
stat := populate_group_with_query(rg, qry);

pl := create_parameter_list('foo');
add_parameter(pl, 'query0', data_parameter, val);

/*~~~~~*/
/* MIN_PRODUCT를 사용하여 parameter packing */
/*~~~~~*/

/* run_product(graphicx, 'c:\oracle\graph25\data_f2g.ogd',
synchronous, batch, fill_option, pl, 'BIDDER_BAB, CHART_ITEM'); */

/*~~~~~*/
/* DC.INTERPRET를 사용하여 parameter packing */
/*~~~~~*/

/* og.interpret('c:\oracle\graph25\data_f2g.ogd',
'display_bar, chart_item', 'DCDCTYPE='; true, pl); */

/*~~~~~*/
/* DC.DEPUSH를 사용하여 parameter packing (DC, CLOSE와 함께
사용) */
/*~~~~~*/
    og.refresh('c:\oracle\graph25\data_f2g.ogd', 'display_bar,
chart_item', pl);
    og.close('c:\oracle\graph25\data_f2g.ogd',
'BIDDER_BAB, CHART_ITEM');

    destroy_parameter_list(pl);
end;

- control block : display_bar ( chart_item을 포함 )

```

```

2) graphic에서 2) 작업
. attached library : DG

. query0
  select dname name, deptno sal
  from dept

. program unit : execute_me <= run_product에서 필요하지 않습니다.
  PROCEDURE execute_me IS
  BEGIN
    dg_execute_query(dg_get_query('query0'));
  END;

. chart
  name : flag_bar
  type : bar chart

. categories : name

. value      : sal(bar)

```

(V4.5) FORMS4.5 에서 SERVER 쪽 APPLICATION을 실행하는 방법

Bulletin no : 10320

방법 :

Server쪽에서 우선 proc로 만든 daemon을 띄워놓고 Client의 Forms4.5에서는 DBMS_PIPE package로 만든 Stored Procedure를 call함으로서 message를 Server로 보낸다. daemon은 message를 계속해서 기다리다가 message가 receive되는 즉시 실행한다. 이때 'STOP'이라는 message를 만나면 daemon은 실행을 마친다.

주의사항 :

dhnc_pipe에 관한 실행권한이 해당 user에 주어져야 한다. dhnc_pipe package는 oracle install시에 생성되어 있으나 일반 user들에게는 사용권한이 없도록 되어있기 때문이다. sqlldr에서 다음의 문장들을 이용하여 해당 user에게 권한을 준다.

```
grant execute on dhnc_pipe to :
```

1) daemon.sql

```
create or replace procedure execute_sysops
  (command varchar2, timeout number default 10)
is
  status NUMBER;
begin
  DBMS_PIPE.PACK_MESSAGE(command);
  status := DBMS_PIPE.RECV_MESSAGE('daemon', timeout);
  IF status <> 0 THEN
    RAISE_APPLICATION_ERROR(-20010, 'Error while reading .
    status = ' || status);
  END IF;
end;
```

2) daemon.pc

```
!include
!include
!include

EXEC SQL BEGIN DECLARE SECTION:
  VARCHAR username(20);
  VARCHAR password(20);
```

```

    let status;
    VARCHAR command(100);
    VARCHAR value(2000);
EXEC SQL END DECLARE SECTION;

main()
{
    EXEC SQL WHENEVER SQLERROR DO sqlerror();
    connect_sql();
    execute_message();
}

connect_sql()
{
    strcpy(username, user, "scott");
    strcpy(password, user, "tiger");
    username[0] = '\0';
    password[0] = '\0';
    EXEC SQL CONNECT :username IDENTIFIED BY :password;
    printf("Connect to oracle %s / %s\n",
           username, password);
}

execute_message()
{
    while (1) {
        printf("Daemon exiting ... \n\n");
        EXEC SQL EXECUTE
        BEGIN
            status := DBMS_PIPE.RECEIVE_MESSAGE('daemon');
            IF status = 0 THEN
                DBMS_PIPE.UNPACK_MESSAGE(:command);
            END IF;
        END;
        END-EXEC;
        If (status == 0)
        {
            command[0] = '\0';
            If (strcmp(char *) command, "STOP")
            {
                printf("Daemon exiting.\n");
                break;
            }
        }
        else
        {

```

```

        printf("Will execute system command '%s'\n", command,app);
        status = system(command,app);
        if (status)
            printf("System error while responding to system\n\n");
        else
            printf("\n\nCompleted Successfully!\n\n");
    }
}
}
}

sqlserver()
{
    EXEC BQL WHENEVER BQLERR OR CONTINUE;
    printf("\n% .70s\n",sqlca,sqlwarn,sqlserver);

    EXEC BQL ROLLBACK WORK RELEASE;
}

```

3) Forcmd,5,5] when-button-pressed trigger@h]

```

declare
    temp varchar2(100);
begin
    temp := 'runway map, map scott/tiger parameters=desttype=fil';
    execute_system(temp,5);
end;

```

(V3.0, MENU V5.0) FORM과 MENU의 연결

Bulletin no : 10387

FORM3.0과 MENU5.0을 연결하여 사용하는 방법은 크게 2가지로 나누어 볼 수 있다.

첫째는 MENU가 먼저 실행되고, MENU에서 FORM을 호출하는 형식이다.

둘째는 FORM에 MENU가 ATTACH되어 FORM이 먼저 실행되는 형식이다.

다음에서 2가지의 방법에 대해 좀더 자세히 살펴본다.

방법1) MENU가 먼저 실행되고, MENU에서 FORM을 호출

1. `qx12000`에서 사용할 FORM들을 생성
2. `qx10000`에서 MENU생성
3. `qx10000`으로 MENU실행 -> FORM 호출

방법2) FORM에 MENU ATTACH

준비) 항) `$OBACLE_HOME/bin/qx12000`에 `menu`에 관한 library가 link되어 있어야 한다.

```
$ cd $OBACLE_HOME/fox30/lib
$ make -f qx12000.mk qx12000menu
```

1. `qx10000`에서 MENU생성
2. `qx12000`에서 FORM생성
3. FORM PROPERTY화면에서 MENU부분을 `DEFAULT`가 아닌 `USER`가 생성한 MENU명으로 지정
4. KEY-STARTUP TRIGGER에 `BETTING`사항
BDM_MENU:
BET_INPUT_FOCUS(MENU):
5. FORM을 실행하면 MENU가 나타남

(V3.0) BUFFER AND ARRAY SIZE WITHIN SQL*FORMS 3.0

Bulletin no : 10250

BQl*Form 3.0 Block Definition에서의 Buffer and Array Size parameters는 tuning parameters이다. BQl*Form Designer's Reference에서 보면:

ARRAY SIZE : database에서 한전에 fetch할 수 있는 최대record수를 말한다. Default는 block의 한전에 display하는 숫자로 되어있다.

RECORDS BUFFERED : block에서 query를 할 때 memory내에

BQl*Form가 가져가는 record의 최대수를 말한다.

BQl*Form는 어떤 추가적인 record들에 대해서는 disk위에 temporary file을 이용하여 buffering을 한다.

이 값은 display되는 record의 수보다 커야 한다.

<예>

record 5개씩을 multi로 display하는 block에서 1000건의 record를 query해오는 경우, Array Size를 10으로 setting하면, execute_query를 수행하면 10건의 record가 database로부터 BQl*Form으로 들어온다. 만약 사 자가 11번째의 record로 down키를 누르면, BQl*Form는 database로부터 11에서 20까지의 record들을 가져오게 된다.

적절한 Array Size를 setting함으로써, 한전에 모든 record를 query해 올 때 시간이 오래 걸리는 것을 줄여 미지 않도록 적절하게 시간 분배를 할 수 있 한다.

위의 예에서, 10개의 record를 query해오는 것은 1000건의 record를 query하는 것보다 두 배 많이 빠르기 때문에 11번째, 21번째, 31번째의 record를 access할 때 fetch하는 것을 느끼지 못한다.

Unfortunately, due to a bug, versions 3.0.1B.7 to 3.0.1B.10 of BQl*Form

only perform the Array processing on the first batch of records. Please see bugs B7173 and 78733. (Abstract: "Arraysize is only used for first arraysize number of records"). Version 3.0.1B.10 and above have this bug fixed.

Records buffered 는 disk의 temporary file에 write하기 전에 memory내에 buffering되는 최대 record수를 나타낸다. disk에서 read/write하는 것보다, memory내에서 read/write하는 이 훨씬 빠르다. 이 값은 Array Size와는 전혀 관계가 없는 value이다.

Records Buffered parameter과 같은 수의 record가 BQl*Form에 load되고나면, 다음에 query되는 record들은 temporary disk file에 기록된다.

이 parameter들은 application을 tuning하기 위해 query하는 동안에 setting될 수

있다. 만약 그 값들이 지정되어 있지 않다면, BQl*Form는 block당 query되는 수와 관계없이 모든 block에 대해 가능한 record -buffered의 수만큼 allocate하 고 시도하게 된다.

(V3.0) FORMS사용시 ORA-942 ERROR 처리

Bulletin no : 10272

----- BQL>Form30 BASE TABLE생성 -----

ACTIDN에서 MENU를 선택하였을때 ORA-942를 만나는 경우가 있다.

이것은 BQL>Form3.0을 사용하기 위해 필요한 BASE TABLE들이 없기 때문에 발생하는 것이다. 다음은 BQL>Form 3.0의 BASE TABLE을 생성하는 방법이다.

```
$ cd $ORACLE_HOME/Form30/admin
```

```
$ sqlplus system/manager
```

```
BQL> create compatibility v3
```

```
BQL> @indtab1a
```

```
BQL> @indind1a
```

```
BQL> @indvw1a
```

```
BQL> @indgrant
```

FORM에서의 2차원 TABLE 집계 처리

Bulletin no : 10285

다음과 같은 화면구성을 갖고자 하는 경우

먼 부서별 매출 현황														

부서명 :1월12월13월14월15월16월17월18월19월110월111월112월1총계														

BASE TABLE를 생성하여서는 위와 같은 표를 처리할 수 없다. 부서명 FIELD만 BASE TABLE FIELD로 생성하여 QUERY한 후 POST-QUERY TRIGGER에서 'SELECT SUM(... FROM 부서_TABLE WHERE DEPT = :DEPT' 의 방식으로 처리하면 PERFORMANCE가 나쁘다.

해결방법 예.

1> 우선 위의 표와 같은 형식을 갖는 인 DUMMY TABLE를 생성한다.

```
CREATE TABLE BUI_DUMMY ( DEPT CHAR(5),
                           M1 NUMBER(5),
                           M2 NUMBER(5),
                           M3 NUMBER(5),
                           .
                           .
                           M11 NUMBER(5),
                           M12 NUMBER(5),
                           M13 NUMBER(5));
```

2> FORM위에서 생성한 DUMMY TABLE를 BASE로 하는 BLICE를 생성한다.

3> BLICE DEFINITION 화면의 DEFAULT WHERE/ORDER BY절에 다음과 같이 기술한다.

```
WHERE 1 = 2
UNION
SELECT DEPT, BUI(RECODE(BUBSTR(MDATE,3,2),%01%, AMT)),
        BUI(RECODE(BUBSTR(MDATE,3,2),%02%, AMT)),
        .
        .
```

```

        SUN(DECODER(SUBSTR(MDATE,3,2),*12*, AMT)),
        SUN( AMT),
        CHAR(ORDMID(1))
FROM      매출
WHERE     YMD LIKE *8810*
GROUP BY  DEPT

```

4> POST-QUERY TRIGGER를 생성한다.

```

SELECT DEPTNAME INTO :DEPTNAME
FROM   부서
WHERE  DEPT = :DEPT

```

(V45) HOW TO IMPLEMENT MENU SECURITY IN FORMS 4.5?

Bulletin no : 10255

DBACLE FORMS 4.5에서의 보안은 DBACLE SECURITY의 ROLE의 개념에 기초를 둔다.

모든 USER들이 DATABASE에 저장된 모든 정보를 사용할 수 있기 때문에, ROLE은 USER들을 GROUPING하는데 유용하게 사용한다. (ROLE에 대한 자세한 개념은 MANUAL을 참조 하십시오.)

DBACLE SECURITY에서 ROLE이 정의되면, FORMS개발자는 MENU MODULE과 MENU ITEM에 ROLE을 지정하여, MENU PRODUCTION에 누가 그것을 사용할 수 있는지를 조정할 수 있게 된다.

* MENU APPLICATION을 사용하려면 EMP_FULL_DATABASE PRIVILEGE가 있어야 한다.

I. ROLE생성 방법

1. DBACLE FORMS 4.5 DESIGNER에서 SYSTEM ACCOUNT로 CONNECT

2. FILE * ADMINISTRATION * DATABASE ROLES

만약, ADMIN PRIVILEGE가 없는 USER로 CONNECT를 하게 되면
이 OPTION이 DISABLE상태로 나타난다.

3. 새로운 ROLE을 생성하기 위해 MENU를 선택

새로운 ROLE의 이름을 등록할 수 있는 WINDOW가 나타난다.

새로운 ROLE의 이름을 입력하고, 그 ROLE을 갖게 될 USER명을 입력 한다.

MENU를 실행 시에 사용할 수 있는 OPTION (MENU MODULE, DB COMMAND, BACKGROUNDED MENU)을 원하는 대로 선택한다. MENU에서 위의 OPTION을 사용하지 않으려면 BLANK로 남겨두면 된다.

4. DE BUTTON을 선택

ROLE이 생성된다.

II. MENU MODULE에서의 SECURITY SETTING

1. MENU APPLICATION OPEN한 후 MENU APPLICATION의 PROPERTY화면을 띄운다.

2. SECURITY MENU SECTION에서

1) MENU MODULE SECURITY * MODULE BUTTON 선택 혹은 DOUBLE CLICE

* 선택한 MENU APPLICATION의 사용을 허락할 ROLE명을 등록한다.

* DE

이 작업은 각각의 MENU ITEM을 사용할 수 있는 권한을 부여한 것은 아니다.

단지, 각 ITEM마다 선택할 수 있는 ROLE을 지정한 것에 불과하다.

2) USER SECURITY를 TRUE로 SETTING

만약, TRUE로 SETTING하는 것을 잊어버린다면

모든 USER들이 모든 MENU OPTION을 사용할 수 있게 된다.

III. MENU ITEM에 ROLE을 지정하는 방법

1. MENU ITEM의 PROPERTY화면을 띄운다.

2. SECURITY SECTION에서

1) MENU ITEM ROLLS에서 HOME BUTTON선택 혹은 DOUBLE CLICE

* 앞의 과정에서 SETTING에 놓은 ROLLS들이 나타난다.

* 이 ITEM을 사용할 수 있게 하고자 하는 ROLLS들을 선택한다.

만약, 이 LIST에서 아무것도 선택하지 않으면,

이 MENU는 모든 USERS들이 사용할 수 있게 된다.

2) DISPLAY W/D PRIVILEGE는 이 MENU ITEM을 사용할 수 없는 USER에게도

MENU ITEM을 보여 줄 것인가를 의미한다.

만약, TRUE인 경우 이 ITEM을 사용할 수 없는 USER에게는 CHAYED OUT권 상태
로 DISABLE된 상태로 나타난다.

만약, FALSE인 경우 이 ITEM을 사용할 수 없는 USER에게는 이 MENU ITEM이
나타나지 않게 된다.

IV. MENU 를 RUN시키는 방법

MENU를 RUN시키려면, 먼저 FORM에 MENU를 ATTACH하는 작업이 필요하다.

1. FORM MODULE을 OPEN한다.

2. FORM MODULE의 PROPERTY화면을 띄운다. (TABLE PROPERTIES)

3. MENU SECTION에서

MENU MODULE에 삽입해 놓은 MENU MODULE명을 등록한다.

MENU STYLE를 선택한다.

STARTING MENU를 입력한다.

이것은 삽입해 놓은 MENU MODULE에 있는 MENU TREE구조 중
어느 POINT에서 부터 사용할 것인가를 지정하는 것이다.

4. FORM을 SAVE한 후 RUN시킨다.

V. 나타날 수 있는 ERROR들

FORM-1025B : "User is Not Authorized To Run Oracle Forms Menu"

이 큰 USER가 FORMB TABLE들을 ACCESS할 수 없다는 걸 의미한다.

FORMB에서의 SECURITY를 구현하기 위해서는, FORMB45에 관련된 TABLE과 VIEW들이
생성 되어 있어야 하고, 해당 USER는 이 TABLE과 VIEW를 사용할 수 있는 권한이 있어야 한다.

FORMB45에 관련된 TABLE과 VIEW를 생성하기 위해서는 관련된 ORACLE SYSTEM ACCOUNT

하에서 실행되어야 한다. (INSTALLATION GUIDE참조)

FORM-1024B : "No Authorization To Run Application"

이 ERROR는 MENU APPLICATION을 사용할 수 있도록 지정된 ROLLS에 해당 USER가
ASSIGN되어 있지 않다는 것을 의미한다.

(Ⅲ) FORMS30 에서 LEFT,RIGHT KEY 를 사용하여 MENU DESIGN

Bulletin no : 10079

1. ?/forms30/adain/resources DIRECTORY 에서 RESOURCE FILE 들 COPY 한다.

```
$ cp vt220.r vt220a.r
```

2. DBATERN 들 이통 vt220a.r FILE 들 OPEN 하여 F0 및 F1 KEY 들 등록 한다.

a. DBATERN 이 Product(BUMFORM) 에서 다음과 같이 등록한다.

Product Key Name	Function Code
Key 0	10022
Key 1	10023

b. Mapping 에서 Left,Right Key 이 Product Key Name 들 수정한다.

Product key name	Device Key Name
Key 0	Left
Key 1	Right

c. 수정된 내용들 Save 한다.

3. FORM30 에서 Key-F0, Key-F1 TRIGGER 들 U2 TYPE 으로 정의한다.

Trigger Level : FORM

Trigger Style : U2 (U3 Trigger 화면에서 Trigger Style 들 U2 로 선택 하고 ZOOM IN 들 화면된다.)

Key-F0

```
EXECUTED CASE input, trigger_field is
  when 'MENU_FL01' then gofid MENU_LAST_FL0;
  when 'MENU_FL02' then gofid;
  .....
  when 'MENU_LAST_FL0' then gofid;
  when others then movelst
end case;
```

Exp-71

Example case input, trigger_field is

when 'MENU_FL01' then extfid;

.....

when 'MENU_LAST_FL0' then gofid MENU_FL01;

when others then noflight

end case;

4. F000B 기동한다.

\$ runform30 menu - vt220:vt220 scott/tiger

(V4.5) MENU UPGRADE

Bulletin no : 10280

BQLMenu 5.0에서 Oracle Form 4.0호 혹은 Form U4.0에서 U4.5로의 upgrade는 *form upgrade*의 과정과는 다르게 구성되어 있다.

Upgrading From BQLMenu 5.0

Menu Upgrade를 위해서는 *form upgrade*에서와 마찬가지로 Oracle Form Generator를 사용한다. Form을 upgrade할 때는 File-To-File Conversion으로서 하나의 file (.INF)에서 form 4.5 module인 .FMB를 생성한다. 그러나 BQLMenu 5.0 Module BYTEFMB에 저장되어 있기 때문에 generator에 source file을 명시하지 않는다. (BIN File은 menu module의 compiled version으로 읽을 수 없는 file이다). 결국, menu를 upgrade하기 위해서 generator는 Base Table To Base Table Conversion이 이루어진다.

준비확인사항

Menu upgrade를 위해서는 우선적으로 BQLMenu 5.0에 완전히 base table과 Oracle form 4.5에 완전히 base table이 생성되어 있어야 한다.

1. BQLPlus에서 BYTEFMB로 BYTEFMB에 log on. 한다.

2. BEBC HMENU_B_APPL

만약 description이 나타나지 않 나, 어떤 column이 CHAR datatype으로 나타나면 BQLMenu 5.0 base table을 install하여야 한다.

3. BEBC FORM40_APP

만약 ORA-B42 error가 나타나면서 description이 나타나지 않으면 Oracle form base table을 다시 install하여야 한다.

4. form U4.5 사용자는 위의 3번 사항대신에 다음을 확인하여야 한다.

BEBC FORM45_OBJECT

만약 ORA-B42 error가 나타나면 Oracle form U4.5 base table을 다시 install하여야 한다.

BQLMenu 5.0 Application을 직접 U4.5로 upgrade할 때

\$ORACLE_HOME/Form45/Doc/Readme.Doc The Developer 2000 Release Notes참조.

BQLMenu 5.0 Application을 upgrade하려면 Form 4.0 Base Table이 install되어져 있어야 한다.

만약 Form 4.0의 base table이 install되어 있지 않으면 다음의 작업들이 필요하다.

Form 4.0 base table을 생성하는 BQL script들은 ORADIM\FORMB45\BQL directory 밑에 있으며 BYTEFMB user상에서 수행되어야 한다. BQLMenu 5.0 Application을 migration하기 위해서는 BQLPlus 에서 BYTEFMB user로 다음의 script를 수행하여야 한다.

BQL> @Form45\Bql

만약 BQLMenu 5.0 Application의 base table을 생성하였었다면, 다음의 script를 이용하여 Form 4.0 Base Table을 drop시킬 수 있다.

BQL> DFWadddrop.Bql

위 script를 수행하기 전에, 사용자는 반드시 다른 사용자가 Form 4.0 Base Tables를 생성해 놓았거나, 사용하고 있지는 않는지 확인하여야 한다.

반대로, mmu upgrade를 수행하여 같은 이름의 application을 생성하면 다른 application을 일괄 수도 있으므로 UEBBIDM=50 (command line으로 수행할 때) 혹은 *50* (generator 화면에서)를 반드시 지정하여야 한다.

Menu Module이 반드시 DATABASE에 저장되어 있어야 한다

1. BQLPlus에서 SYSTEM으로 DATABASE에 log on. 한다.

2. upgrade하고자 하는 mmu module이 존재하는지 확인한다.

SELECT * FROM MENU_MODULE WHERE APPLICATION_NAME='MYAPPNAME';

How To Upgrade Menu From BQLMenu 5.0

mmu의 upgrade작업은 Oracle Form Generator를 통해서 이루어 진다. Generator를 호출하기 위해서 Icon을 double click하거나(windows에서), F40GEN (Or F45GEN For U4.5 User)(UNIX에서)를 사용한다. 그러면 Generator Dialog Box가 나타난다.

기본적인 mmu upgrade를 하려면 다음의 property를 지정한다.

File:	DATABASE에 저장되어 있는 mmu module . 대소문자 구별없음
Username, Etc.:	Username, Password, Connect String
Logon To The Database:	ON
Module Type:	Menu
Module Access:	Database
Upgrade 3.0 Form...:	ON
Upgrade 5.0 Base...:	OFF

나머 는 기본적으로 남겨둔다. OK button을 눌러 수행하면 주기적으로 수행중에 안내정보가 나타난다.

generator를 command line상에서 수행할 수도 있다.

```
F40GEN MODULE= UEBBIDM= MODULE_TYPE=MENU
MODULE_ACCESS=DATABASE UPGRADE=YES
```

Upgrading From Oracle Form 4.0 To Oracle Form 4.5

Oracle Form 4.0에서 Oracle Form 4.5 으로 upgrade하는 것은 상대적으로 쉽다.

각각의 U4.0 .HHH File를 단순히 Oracle Form 4.5에서 open하여 다시 Save하면 된다. 그러면 해당 application은 U4.5 file이 된다. 하지만 현재의 작업은 불가능하다. 즉, U4.5 file은 U4.0 file로 되돌릴 수 없다.

또 다른 방법은 다음과 같이 Oracle Form 4.5 Generator를 사용하는 것이다.

File: .HHH File명. Wildcard 사용할 수 없다.

```

Username, Etc.:      Username, Password, Connect String
Module Type:         Menu
Module Access:        File
Upgrade 3.0 Form...:  OFF

```

Batch Mode에서는

```
F45gen Module=Menu Userid=Scott/Tiger Module_Type=Menu Batch=YES
```

Batch Upgrading From BQL Menu 5.0

Menu 5.0 Bql을 Oracle Form으로 upgrade하는 방법은 다음과 같다.

```

File:                Blank
Username, Etc.:       Username, Password, Connect String
Module Type:         Menu
Module Access:        Database
Upgrade 3.0 Form...:  OFF
Upgrade 5.0 Bql...:   ON

```

Batch Mode에서는

```
F40gen Userid=Scott/Tiger Module_Type=Menu Upgrade_Bql=YES
```

NOTE: Earlier Versions Of Oracle Form Required A Menu Module To Be Upgraded. When Upgrading Bql, Be In Versions Earlier Than U4.0.13, You Need To Supply A Module Menu To Upgrade And The Upgrade 3.0 Form Flag To ON.

5.0 Bql을 Form U4.5에서 사용하는 ORACLE Bql을 upgrade하려면 다음과 같이 지정한다.

```

File:                Blank
Username, Etc.:       Your Username, Password, Connect String
Module Type:         Menu
Module Access:        Database
Upgrade 3.0 Form...:  OFF
Upgrade 5.0 Bql...:   ON
Version:              50

```

Batch Mode에서는 다음과 같이 사용한다.

```
F45gen Userid=Scott/Tiger Module_Type=Menu Upgrade_Bql=YES Version=50
```

Batch Upgrading From Oracle Form 4.0

Menu security를 위한 Version7의 BQL사용부분은 form U4.0과U4.5의 기능면에서의 큰 차이점중의 하나이다. 과거에는 HEML에서의 role은 customer가 지정한 role에 의존하여 security를 관리하였고, 이것들은 base table내에 저장되어 관리되어 왔다. 그러나 U4.5에서는 ORACLE 7의 Bql을 사용할

수 있다. U4.0에 지정된 Role를 DBACLE 7 server의 Role로 upgrade하려면 BQLPlus에서 SYSTEM user로 FBN45DBL.BQL script를 수행하여야 한다.

이 script는 windows에서는 %DBHOME%\FBN45\BQL에 위치하고,

UNIX에서는 \$DBACLE_HOME/Form45/Adm/BQL에 위치하며 Dracm:Script:Cmd에 위치한다.

이 script는 PL/SQL script이며 Dracm Form 4.0 Base Tables에서 자료를 읽어 DBACLE7 data dictionary table에 정보를 insert하는 script를 생성한다. 이 script가 또다른 script를 생성하고 실행하기 때문에 해당 user는 DB에 대해 writer permission을 가지고 있어야만 한다.

Batch Upgrading

한개이상의 U5.0 Menu를 한전에 upgrade하기 위해서는 Generator가 module명으로 wildcard를 받을 수 있어야 한다. 예를 들어 module명에 AX로 지정하면 owner에 관계없이 %*로 시작하는 모든 menu module이 upgrade되어야 한다. 또한 menu module전부를 upgrade하려면 module명에 %만 입력하면 되도록 되어야 한다. 그러나 불행히도 File to file conversion은 wildcard처리가 되지 않기 때문에 하나 이상의 U4.0 menu를 U4.5로 upgrade하려면 generator를 여러번 수행하거나 shell script를 이용하여 batch로 수행하여야 한다.

Troubleshooting

Q. F40gen 을 수행하면 FBN-B1500: Unable To Start/Complete The Generation 발생

A. F40gen을 사용한다면 DISPLAY Variable을 checking해보고, K Windows System이 정상적으로 run 되는지 확인한다.

F40gen을 사용한다면 TERM과 DBACLE_TERM Environment 을 확인한다.

Q. BQLMenu 5.0 Menu를 Dracm Form Module로 upgrade하려면

NMI-10837: Error Performing Upgrade(발생(FBN-10837, FBN-31837도 함께 발생)

A.

1. 이것은 BQLMenu U5.0 application을 DBACLE form U4.0으로 upgrade할 때 나타나는 일반적인 error이다. 이 error가 발생하였다면 이것은 대부분 BQLMenu U5.0 base table과 DBACLE Form U4.0 base table이 같은 database에 있지않거나 사용할 수 없을 때 발생한다.

2. 그외의 check할 점

1) conversion하고자하는 application이 BQLMenu base table에 저장되어 있는가?

다음 sql을 이용하여 확인한다.

SELECT * FROM NMI_B_APPL WHERE APPLICATION_NAME LIKE 'MYAPP';

2) BQLMenu U5.0 base table에 item이 CHAR type을 갖지 않는가?

SELECT NMI_B_APPL

3) 충분한 tablespacememory와 rollback segment가 확보되어 있는가?

4) 같은 이름의 form application 이 이미 기존에 존재하지 않는가?

그렇다면 generator는 같은 이름의 menu application으로 convert할 수 없다.

Q. upgrade된 Menu Application을 수행하면서 다른 form program을 호출하는 menu를 선택하면 FWH-10242: Cannot Call Linked-In Oracle Forms From Oracle Forms 발생

A. 이것은 BQLMenu U5.0에서 생성한 application을 upgrade한 경우에 발생한다. BQLMenu U5.0 application은 대부분 사용자가 menu를 run하고, 그 안에서 form을 호출(type 4 command)하도록 되어 있다. 그러나 ORACLE form에서는 form이 중심이 되고 있다. 즉 사용자는 menu가 attach되어 있는 form을 사용하게 되어 있다.

위의 문제는 type 4 command를 conversion한데서 발생한다. 이것은 link되어 있는 form을 실행하려 한다. 그러나 U4.5에서는 이미 form이기 때문에 error가 발생하는 것이다.

이 문제를 해결하려면 type 4인 command를 PL/SQL command로 바꾸고 command link에 지정되어 있는 runform30의 문장을 CALL_FORM(*FORM_NAME*)으로 변경해야 한다.

Q. (MS Windows Only) Menu Item에서 BQLplus를 실행하려하면 화면이 전력하고나서 아무 변화가 없다.

A. BQLMenu U5.0 Application에서 upgrade한 경우에 많이 발생하는 현상이다.

BQLplus를 호출하는 command를 이전의 "Bqlplus Scott/Tiger" 에서 "Plus31 Scott/Tiger"로 변경한다. ORACLE Form menu에서는 BQLplus를 ok command로 호출하게 된다.

PL/BQL menu Item에서 BQLplus를 HSBT문을 이용하여 호출하거나, BQLplus menu item type을 이용하여도 마찬가지이다.

이것은 window에서의 window command로 인식하도록 되는 것이다. 그것이 실패하면 이것은NOB로 넘겨진다.

결국 ORACLE form에서는 처음에 window program에서 BQLPLUS를 실행하려 한다. 그러나 window program중에는 BQLPLUS라는 program이 없기 때문에(window에서는 executable menu이 PLUS31로 되어 있기 때문에) 실패하고 NOB로 넘겨져 그곳에서 실행하려 한다. 그러나 NOB에서도 실패하게 되는 것이다. 이것이 밀란적으로 화면이 전력하게 되는 원인이다.

Q. BQLMenu 5.0 Menu를 Oracle Forms 4.5 Menu 또는 F45COM을 사용하여 upgrade하려 할 때 FWH-10043 "Cannot Open File" 가 발생한다.

A. Upgrade option에 YEB인 것과 Upgrade = 50으로 upgrade하려는 version을 정확히 수어있는지 확인한다.

(V4.5) MULTI FORM GENERATION (UPGRADE)

Bulletin no : 10258

I. Introduction

Developer/2000의 'Form Generate' Icon에서 실행하는 Oracle Form Generator(F45gen.exe)는 한 Module별로 처리하도록 되어있어서 User가 계속 지켜보다가 실행이 끝나면 다음 Module을 다시 입력하여야만 한다. 이 자료는 이러한 불편함을 해소할 수 있게 해 주며, Form 2.0, Form 2.3, Form 3.0, Form 4.0, And Form 4.5 어느 Version에 대해서도 처리할 수 있게한다.

Form 2.3까지는 File System에만 저장할 수 있었다. Form 3.0부터는 Source를 File System과 Database중 하나를 선택할 수 있게 되었다. 이 자료는 2가지의 큰 Category 즉 'Generating From The Filesystem' 과 'Generating From The Database'로 나누어 처리한다.

II. Generating From The Filesystem

1. Unix/UMB Files

Form의 모든 Version의 Source를 Filesystem에 저장할 수 있기때문에 Source File에 대한 Generate/Upgrade작업을 Batch로 처리 할 수 있다.

다음은 Form 2.3에서 Upgrade하는 예이다. 이것을 하기위해서 Version=23 Upgrade=Y을

```
F45gen Module=Form Userid=USERMAN/Password Version=23 Upgrade=Y
```

```
.
```

```
.
```

```
.
```

```
F45gen Module=Form Userid=USERMAN/Password Version=23 Upgrade=Y
```

SQL*Form 43.0에서도 Version=30을 Batting할 수 있으나, 꼭 해야 할 필요는 없다. Oracle Form 4.0에 대해서는 Version0이나 Upgrade Parameter를 특별히 지정 할 필요가 없다.

2. Windows Files

Microsoft Windows를 사용하는 System에서 Generation/Upgrade는 DOS Batch File로 Window Executable을 실행하는 것이 쉽지 않다. Form을 Generate/Upgrade하기 위해서는 Batch Processor의 역할을 하는 Form을 생성하여 처리하는 것이 좋다.

2.1 모든 Field를 PC에 가져다 놓는다.

2.2 위에서 언급한 바와 같이 F45gen은 Window Executable이어서 PC에서 Batch File을 생성하는 것이 어렵다. 대신 Convert/Convert할 File들을 Temporary File에 모아서, 이 File로 부러 이름들 밑어들여 Host Command를 이용하여 Convert하는 방법을 사용한다.

2.3 다음의 Item들을 갖는 Non-Box Block을 생성한다.

Item1: Force Version

이 Field는 꼭 필요하다. 이것은 Force 2.0, 2.3과 3.0에 대한 Version(20, 23, 30, 40, 45)을 넣기위한 것이다.
40 And 45에 대해서는 다음에서 언급한다.
2.0, 2.3, 3.0을 입력하지 않도록 한다.

Item2: Directory Name

이 Item은 File들이 위치하는 Directory를 지정하거나 Null로 비워져 있어야 한다. 만약 값이 주어지지 않으면 Icon과 연결된 현재의 Directory나 F00B645_PATH에 지정된 Directory를 의미한다.

Item3: Username

이 Force를 가지는 User의 이름을 지정한다.

Item4: Password

위에 지정한 User에 대한 암호를 입력한다.

Item5: Connect String

Server Database에 Connect하기위한 String을 쓴다.

Button1: 작업을 시작할 Button을 생성한다. (Label=File)

다음의 Source Code를 갖는 User-Button-Pressed Trigger를 갖는 Button을 생성한다.

```

File_Pointer TEXT_ID, FILE_TYPE;
Record_Buf   VARCHAR2(200);
Host_Buf     VARCHAR2(200);
User_Name    VARCHAR2(40);
User_Pwd     VARCHAR2(40);
User_Conct   VARCHAR2(40);
File_Ext     VARCHAR2(4);

BEGIN
  /* Checking The File Extension
  */
  IF :Item1 IN ('20', '23', '30') THEN
    File_Ext := 'IMP';
  ELSE
    File_Ext := 'FIM';
  END IF;
  /*Get The Username And Password
  */
  IF :Item3 IS NULL THEN
    User_Name := GET_APPLICATION_PROPERTY(USERNAME);
    User_Pwd  := GET_APPLICATION_PROPERTY(PASSWORD);
    User_Conct := GET_APPLICATION_PROPERTY(CONNECT_STRING);
  ELSE

```

DECLARE

```

User_Name := :Item3;
User_Pwd := :Item4;
User_Cact := :Item5;
END IF;

IF User_Cact IS NOT NULL THEN
    User_Pwd := User_Pwd || '0' || User_Cact;
END IF;

/* Build The Host String To Collect The .log/.Pak Files
*/
IF :Item2 IS NOT NULL THEN
    Host_Buf := 'Dir ' || :Item2 || '\*. ' || File_Ext || ' > Junk.Txt';
ELSE
    Host_Buf := 'Dir *. ' || File_Ext || ' > Junk.Txt';
END IF;

/* The Following Message Will Display The String To Be
   Executed In The HDBT Command. If Junk.Txt Does Not
   Contain The Correct File Name, Modify The Above
   Code To Fix The Problem.
*/
MESSAGE(Host_Buf);
SYNCHRONIZE;
HDBT(Host_Buf, NO_SCREEN);
/* Opening Junk.Txt
*/
File_Pointer := TEXT_IO.OPEN('Junk.Txt', 'B');
/*
   Looping To Read And Convert The File
*/
File_Ext := ' ' || File_Ext;
LOOP
    TEXT_IO.GET_LINE(File_Pointer, Record_Buf);
    IF INSTR(UPPER(Record_Buf), File_Ext) > 0 THEN
        Host_Buf := REPLACE(SUBSTR(Record_Buf, 1, 8), '*', Null);
        Host_Buf := 'C:\DBAH\IN\BIN\F45gmw.Exe Module=' ||
            Host_Buf || ' User Id=' || User_Name || '/' ||
            User_Pwd || ' Batch=Ymc Upgrade=Ymc Version=' || :Item1;
        HDBT(Host_Buf);
    END IF;
END LOOP;

/* Close The File When It Is Done
*/
EXCEPTION

```

```

        WHEN NO_DATA_FOUND THEN
            TEXT_ID.FCLOSE(FILE_POINTER);
END;

```

III. Generating From The Database

Form이 Database에 저장되어 있는 경우를 살펴보자.

이 경우에 Generator가 Server에서 이루어지느냐, PC에서 이루어 지느냐에 따라 다르다. 어느 경우이든지 Form 4.5에 관한 Base Table들은 반드시 Install이 되어져 있어야 한다.

Installation Guide참조하람.

1. Server

Form 2.0과 Form 2.3에서는 Source가 File System에만 저장되기 때문에 이 과정을 사용할 수 없다.

```

F45gen Module=Form UserId=Userman/Password Module_Access=Database Version=30 Upgrades=Yes

```

```

F45gen Module=Form UserId=Userman/Password Module_Access=Database Version=30 Upgrades=Yes

```

Form 4.0인 경우는 Version이나 Upgrade Parameter를 기술할 필요가 없다.

만약 Database에 저장된 Form의 이름들 모른다면, 다음에 설명하는 PC에서의 방법과 같이 사용하 여도 된다.

2. PC

Database에 저장된 Form에 대해 Generate/Upgrade하는 경우는 PC의 Filesystem에서의 작업에 몇 가지의 과정이 더 필요하다.

Button2: Button To Start The Process (Label=Database)

When-Button-Pressed Trigger에 다음의 Source Code를 갖는 Button을 생성한다.

```

DECLARE

```

```

    Host_Name    VARCHAR2(100);
    User_Name     VARCHAR2(40);
    User_Pwd      VARCHAR2(40);
    User_Conct    VARCHAR2(40);

```

```

/* Cursor Declaration For Form 3.0

```

```

*/

```

```

CURSOR C30 IS

```

```

    SELECT Appname
    FROM Form_App

```



```

WHERE Appowner = User_Name;
/* Cursor Declaration For Form 4.0
*/
CURSOR C40 IS
SELECT Appname
FROM Frm40_App
WHERE Owner = User_Name;
/* Cursor Declaration For Form 4.5
*/
CURSOR C45 IS
SELECT Name
FROM Frm45_Objct
WHERE Owner = User_Name
AND Objcttype = 1;

BEGIN
/* Checking The File Extension
*/
IF :Item1 NOT IN ('30', '40', '45') THEN
MESSAGE('Only Valid Values Are 30, 40 And 45. ');
RAISE FORM_TRIGGER_FAILURE;
END IF;

/* Get The Username And Password
*/
IF :Item3 IS NULL THEN
User_Name := GET_APPLICATION_PROPERTY(USERNAME);
User_Pwd := GET_APPLICATION_PROPERTY(PASSWORD);
User_Conct := GET_APPLICATION_PROPERTY(CONNECT_STRING);
ELSE
User_Name := :Item3;
User_Pwd := :Item4;
User_Conct := :Item5;
END IF;

IF User_Conct IS NOT NULL THEN
User_Pwd := User_Pwd || ' ' || User_Conct;
END IF;

/*Opening The Cursor
*/
IF :Item1 = '30' THEN
OPEN C30;
ELSEIF :Item1 = '40' THEN
OPEN C40;
ELSEIF :Item1 = '45' THEN

```

```

OPEN C45:
END IF:

/* Looping To Read And Convert The File
*/
LOOP
  IF :Item1 = '30' THEN
    FETCH C30 INTO Host_Buf;
    EXIT WHEN C30$NOTFOUND;
    Host_Buf := 'C:\DB\BIN\BIN\F45gen.exe Module=' ||
      Host_Buf || ' UserId=' || User_Name || '/' || User_Pwd
      || ' Batch=Ymc Module_Access=Database Version=30';
  ELSEIF :Item1 = '40' THEN
    FETCH C40 INTO Host_Buf;
    EXIT WHEN C40$NOTFOUND;
    Host_Buf := 'C:\DB\BIN\BIN\F45gen.exe Module=' ||
      Host_Buf || ' UserId=' || User_Name || '/' ||
      User_Pwd || ' Batch=Ymc Module_Access=Database';
  ELSEIF :Item1 = '45' THEN
    FETCH C45 INTO Host_Buf;
    EXIT WHEN C45$NOTFOUND;
    Host_Buf := 'C:\DB\BIN\BIN\F45gen.exe Module=' ||
      Host_Buf || ' UserId=' || User_Name || '/' ||
      User_Pwd || ' Batch=Ymc Module_Access=Database';
  END IF:

  HDST(Host_Buf);
END LOOP:

END:
-----

```

(V4.5) NEW FEATURES IN 4.5.7

(DEV/2000 R1.3)

Bulletin no : 10280

1. Native 32-Bit Windows95 And Windows NT Support

2. Improved Runtime Startup Performance

Runtime Module의 Startup 시간이 줄어들게 되었다.

그것으로 인해 2개의 Runtime Executable를 가지게 되었다. 하나는 P/BOL Debugger를 사용하기 위한 것이고 다른 하나는 Non-Debug Runtime을 위한 것이다. Force Debugger 상태에서 Debug Option을 On 시킨 후 Force를 On시키면 Debug Executable이 자동으로 호출되게 된다.

만약 Debug-Yes로 Building한 뒤 Non-Debug Version을 실행하면 다음과 같은 Error Message를 받게 된다.

For-41838 : Use The Debugger-Enabled Executable If Specifying Debug-Yes

새로운 Executable의 이름은 다음과 같다.

Win18 Win32

Non-Debug	F45run F45run32
Debug	F45rdbg F45rdbg32

3. List of Recently Used Files

MS Word나 MS PowerPoint에서와 같이 File Menu에 가장 최근에 사용했던 File들에 대한 List가 나타난다. 이 기능은 모든 Platform에서 가능 하다.

4.1 Get_Application_Property

Get_Application_Property Built-in에 Windows95에 해당하는 값이 추가되었다.

Windows95에서 다음과 같은 문장을 실행하면

```
ul := Get_Application_Property(User_Interface)
oc := Get_Application_Property(Operating_System)
```

ul에는 "MSWINDOWS"라는 값이 oc에는 "WINDOWS95"의 값이 들어온다.

5.1 Developer/2000 Fast Loader

MS Windows의 startup group에 B2INIT.EXE Icon을 설치해 주면, Developer/2000 DLL 들이 Memory 내에 미리 상주하여, 다음에 있을 Developer/2000제품이 빠르게 Start Up하게 해준다.

단, 이 기능은 MS Windows에서만 가능하고 Windows95에서는 사용할 수 없다.

B. New Get_Group_Record_Number Built-in

Get_Group_Record_Number Built-in이 새로 추가되었다.

이 Built-in은 Record Group에서 보다 쉽게 Match되는 Record를 찾게 해 다.

정의는 다음과 같다.

```
Get_Group_Record_Number(groupcolumn_id, cell_value) returns Number  
Get_Group_Record_Number(groupcolumn_name, cell_value) returns Number
```

Group내에서 지정한 cell_value와 Match되는 첫번째 Record의 번호를 Return한다.
만약 Match되는 Record가 없으면 0(Zero)를 Return한다.
Cell_Value가 Column의 Datatype과 Match되지 않는다면, Error가 발생한다.
같은 정확히 Match될때만 Return된다. 대소문자를 구별하여 처리하여야 한다.

7. Japanese BioBlot, DLL For Mac Windows

Iconic Button에 Bubble Help를 표시하는데 사용하는 BMDHINT.DLL의 일본어 version을 포함하고 있다. 만약 이것을 사용하고 있다면, 단순히 BMDHINT.DLL을 BMDHINT.DLL로 같은 directory내에서 Re-Name시켜주면 된다.

Windows5에서는 DLL들이 BMDHINT32.DLL과 BMDHINT32.DLL로 되어있다.
또한 Windows5에서는 32-bit BMDHINT32.DLL 을 사용하기위해 새로운 HINT.FLL이 제공된다.

B. New Generator Check Relation

새로운 Generator는 불완전한 Relation에 대해서는 다음과 같은 주의Message를 준다. 주의 Message가 생겨도,FMK File은 생성된다. 그러나 Developer/2000 B2.0에서는 Warning이 아닌 Error로 대체되어, FMK File이 생성되지 않게 될 것이므로 B 2.0으로 Upgrade하기 전에 사용자는 문제를 Fix해야만 할 것이다.

```
FMK-30370: Warning! Relation's detail block is control block.  
FMK-30371: Warning! Relation's master block is control block.  
FMK-30372: Warning! Relation's detail block is the same as its master block.
```

다음의 새로운 Message는 Error이며, 이 Error가 발생하면, FMK File이 생성되지 않게 된다.

```
FMK-3038B: Warning! Relation has no detail block.  
FMK-3038B: Warning! Relation's detail block does not exist.
```

B. New User-Form-Navigate Event For Closing Form

Open_Form Procedure를 사용하여 Open되었던 Form이 Close되면서 호출한 Form으로 되돌아올때 Fire되는 User-Form-Navigate Trigger가 추가되었다.

예를들어, Form1과 Form2가 있고 Form1에 User-Form-Navigate Trigger가 있다고 가정하자. 이전 version에서는 사용자는 Form2에 위치하고 있으면서 Exit_Form를 수행하였다면 Form1의 User-Form-Navigate Trigger는 Fire되지 않았었다.

현재의 Version에서는 이 Trigger가 Fire되어,Open_Form을 사용하여 Multi-Form을 사용할 때 추가적인 Event를 잡아낼 수 있게 되었다.

10. New TE23_Icon Variable For Icon Search Path

Developer/2000 B1.3 (Form4.5.7)에 는 이전 Version에서 ICON에 관한 PATH를 지정하던 TE21_ICON Variable대신에 TE23_ICON을 사용한다.

DBACLE Installer에서는 기존에 DBACLE.INI에 지정되어 있던 TE21_ICON에 대한 정보를 TE23_ICON으로 UPDATE 한다. 만약 사용자가 자신의 Application에서 TE21_ICON 수를 사용하였다면 수정해 주어야 할 것이다.

11. More Meaningful Errors If Unable To Write To Disk

Disk에 Buffer Data를 Write할 때 발생한 Error에 대해 좀더 자세한 Error를 제공한다.

FBM-41835 : Unable to open temporary record buffer file

FBM-41838 : Disk I/O error on temporary record buffer file

은 File명이거나 File의 위치를 나타낸다.

만약 이 Message를 만나게 되면, 사용자는 반드시 해당 Drive나 File System이 Write할 수 있는지 확인해 보아야 할 것이다.

12. Changed Behaviour Of Bat_Record_Property

Non-Basic Table Block에 대한 Bat_Record_Property Built-in은 더 이상 작동하지 않는다. 이 Built-in은 Record Status만 하기 때문에 Non-Basic Table Block에 대해서는 명함을 주지 않게 된다.

만약 Non-Basic Table에 대해서 이 Built-in을 사용한다면 Consistency가 불일치하는 현상이 발생할 수도 있으며, 다음과 같은 Message를 만나게 될 것이다.

FBM-40750 : Record . : Can't set status to QUERY at CHANGED in a control block

13. New Error Message For Character Set

만약 다음과 같 Message를 만난다면

FBM-B1123: Internal Error : Unable to load MIB character set.

Force가 Character Set에 관련된 *.B File를 찾지 못하여 발생하는 것이다. 이 문제를 해결하려면 환경변수중 DBA_MIB가 있는지 확인하고, *.B File이 위치한 곳을 지정하고 있는지, 실제 그 위치에 해당 File들이 있는지를 확인한다.

14. Improved Consistency Of Standard And Non-Standard Fields

이전 version에서는 Standard, Non-Standard Filed들이 Field가 Disabled되었을 때 Color의 전화가 일정치 않았었으나 이제는 Consistency를 유지하게 되었다.

15. Re-Linking User Exits

이전 Version에서 제출되었던 문제를 해결하기 위해 F45KTB.DLL File이 Update되었다. Update된 이 File을 사용하기 위해서는 가지고 있는 User Exit들을 Re-link해 주어야 한다.

16. New Windows95 Features

32-Bit Windows 환경에 사용할 때 Performance를 위해, Developer/2000에서는 Windows95의 다음 Feature들을 Support한다.

- Long Filenames
- 32-Bit Executables
- MAPI Support
- OLE2 Drag-and-Drop
- User Registry (DBACLE.INI 대신)
- Explorer View
- UNC (Universal Naming Convention)
- Multiple Source Files Loaded Into Database
- Recent List of Files Used

1) Doc Support

Developer/2000 BI.3에서는 Doc Support의 제약사항이 있다. 이러한 점들은 Developer/2000에서 Full Support될 것이다.

Form 4, 5, 7에서의 기능은 다음과 같다.

- . DLE container내에 Doc를 Insert한다.
- . 오른쪽 Mouse를 눌러 Property화면을 불러 Doc의 Property를 Setting한다.
- . 실행중에 DLE-style Automation Interface를 호출한다.

BI.0전까지는 다음 기능은 제공하지 않는다.

- . Property palette에서 Doc Property를 확인하고 지정한다.
- . 실행중에 Property를 Programmatically라거나 지정한다.
- . 실행중에 Event를 잡는다.
- . Doc-style method를 호출한다.

2) Adoption Of Windows5 Look-And-Feel Standards

Windows5의 Look-And-Feel 형식을 따른다.

그중 특히 것중 하나는 Disabled된 Field는 Text와 Background가 전부 Grayed된다.

3) New Balloon Help For 32-Bit Applications

Developer/2000 32-Bit version에서는 Balloon Help를 사용하기 위해 새롭게 32-Bit용 BHMHT32.DLL을 제공한다.

이것을 사용하기 위해서는 단순히 DBA\WIN5\COMMON\BELLIB Directory내의 HINT.DLL을 사용하면 된다. 이것은 자동으로 새로운 32-Bit DLL을 찾아온다.

1 New Get_Application_Property Routine For Win5

Get_Application_Property Built-in에 Windows5에 해당하는 값이 추가되었다.

Windows4에서 다음과 같은 문장을 실행하면

```
ul := Get_Application_Property(User_Interface)
oc := Get_Application_Property(Operating_System)
```

ul에는 "MSWINDM32"라는 값이 oc에는 "WIN320MHDM"의 값이 들어온다.

4) Host() - Synchronous Vs Asynchronous

HOST() command는 일반적으로 synchronous하게 실행된다. 그러나 Windows5에서는 원칙적으로 16Bit Version의 Form에서 32-Bit Command를 Asynchronous하게 Hostling한다.

Win5에서의 HOST() 의 작동에 대한 Matrix는 다음과 같다.

Form	16Bit Command	32Bit Command
16Form	sync	asynch
16Form32	asynch	asynch

5) Regenerating Files For 32bit (And Vice Versa)

16 Bit Version에서 생성한 File을 32Bit Tool에서 실행하려면 다시 Generate하면 된다.

같은 방식으로 32Bit에서 16Bit로 옮기려면 역시 다시 Generate하면 된다

6) Changes In Office 95 DLE2 Automation

MS Windows9.1하에서는 AppActive Method만을 사용하여야 했으나 Windows5에서는 AppShow Method도 사용할 수 있게 되었다.

7) Win5 And Winnt Installers

Developer/2000 CD에는 2개의 Installer가 들어 있다. 하나는 Windows5용이고 하나는 WinNT용이다. 환경에 맞게 정확한 Installer를 실행하여야 한다.

만약 두가지 방식의 Host를 하는 기계라면, 각각의 DB에 맞게 Install하여야 한다.

8) Double Click Events On Poplist

Windows5에서는 Poplist에서 Double Click이 실행되지 않는다. 그것은 서클 Click은 List를 Open하

는 것으로 인식하고 두번째 Click은 Close하는 것으로 인식하기 때문이다.

만약 Application의 PopUp에서 Double Click이 제공되게 하려면 Windows에서 PopUp가 다시 Coding되어야 한다.

B) Windows Memory Requirements

Windows 에서 Developer/2000 Form을 사용할때 Design시에는 16M, Runtime시에는 12M를 Recommend한다. 만약 Report와 Graphics 역시 사용해야 한다면 각각 2M씩 더 필요하다고 본다 .

(V3.0) ORATERM 을 이 용 한 FORMS30 FUCTION KEY 수정

Bulletin no : 10087

먼저 EMULATON 상에서 FORMS30 을기용하였을경우 GRAPHIC(BOX 및 LINE)을 확인하여 정상적으로 DISPLAY 되는 RESOURCE FILE을 선택한다. EMULATON 을 이용하는경우에는 VT220 TERMINAL FILE 을 사 용하면 FUNCTION KEY 를 쉽게 수정 할수 있다.

1. EMULATON 을 사 하여 HOST 에 login 상태에서 각 FUNCTION KEY 의 ESCAPE SEQUENCE 값을 확인 한다. 예를들어 F1 KEY 를 입력했을 경우 \u00P, F2 는 \u00Q 등이 나오는지 확인 하고 값이 나오지않 으면 그 FUNCTION KEY 를 사용 할수 없으므로 EMULATON 에서 값을 정리한다.

2. RESOURCE FILE 이 있는 DIRECTORY 로 전경하여 FILE 을 COPY 한다.

```
$ cd $ORACLE_HOME/forms30/admin/resource
$ cp vt220.r vt220a.r
```

3. oraterm 을 기용한다.

```
$ oraterm -c vt220
```

4. RESOURCE FILE 을 OPEN 한다. (Accept key : ^Q)

```
File                                     Help
----- OracleTermial -----
Name
Open <          Open Resource File
Save      +-----+
...       |   Filename : vt220a   |
Exit      +-----+
```

5. 수정하고자 하는 Device 를 선택하여 1. 에서 확인한 FUNCTION KEY 값을 등록 한다.

```
File      Product      Mapping      Device      Help
----- OracleTermial -----
Modify    <- a
-----
Key       <- b
Attribute
Display

a
b
```


File	Product	Mapping	Device	Help
----- Device -----				
	Device Name		Description	

	UT220		DEC VT220 terminal	

File	Product	Mapping	Device	Help
----- Device*Terminal -----				
	Device Key Name		Physical Terminal Sequence	

	Tab		\011	
	Return		^M	
	...		...	
c	PF1 --> F1		\u0010	
	PF2 --> F2		\u0011	
	PF3 --> F3		\u0012	
	PF4 --> F4		\u0013	
	...		...	
d	End		\u0014	

사용하고자 하는 PC 의 모든 KEY NAME 과 Terminal Sequence 값을등록하고(d), KEY NAME 이 틀린경우(c) 수정한다. FORM30 에서는 위에서 등록된 KEY 만사용 할 수 있고 각 KEY NAME 에 해당하는 TERMINAL SEQUENCE 값이 일치 되어야만 동작이 된다.

B. 수정 하고자 하는 PRODUCT 를 선택 한다. (BUNFORM30 의경우)

File	Product	Mapping	Device	Help
----- Device*Terminal -----				
	Modify <-			

	Key			
	Attribute			

File	Product	Mapping	Device	Help
----- Device*Terminal -----				
	Product name		Description	

m	BUNFORM-BUNMENU <-		BQL*Form(Bus Form) and BQL*Menu	

```

2 BQLFORMB          I BQLFormB( Design ) key mapping
   BQLMENU           I BQLMenu( Design ) key mapping
5 ORATEM             I OracleTerminal mapping

```

FORM30 에서는 UNIFORM30 (u 선택) 과 BQLFORM30 (2 선택) 께도로 KEY 를 정의한다. 특히 OracleTerminal(oracle) 을 수정할때 KEY 가 틀린경우에는 5 를 선택하여 oracle 의 KEY 를 정확 하게 Battling 한다를 작업한다. 선택하는 방법은 CUIB30 을 해당 PRODUCT 에 놓고 ACCEPT 한면된다.

7.등록된 DEVICE KEY NAME 를 사용 할 수 있도록 해당 PRODUCT KEYMAPPING 시킨다. (UNIFORM)

```

File      Product      Mapping      Device      Help
-----
                                OracleTerminal
                                Key      <-
                                Attribute

                                II
File      Product      Mapping      Device      Help
-----
-----
Product Key Name      I Device Key Name
-----
Accept/Commit      I Do      --> F3
                   I ^D
Exit                I PF4      --> F4
Block Menu         I PF1 F14   --> F1 F4
Copy               I PF1 F14   --> F1 End
....              I ....

```

8. BQLFORM30 의 KEY 도 B,7. 을 만족하여 수정한다.

9. BQLFORM30 FILE 를 저장하고 Exit 한다.

4. 에서 Save 와 Exit 를 순서대로 선택한다.

10. 수정한 Terminal File 요서 FORM30을 기동한다.

```

$ xglform30 -c vt220a:vt220

```

```

vt220a : Resource File name (생략하면 oracle.r 이 지정됨)
vt220  : Device Name

```

11. Attribute 를 수정하는 경우 Device 와 Mapping 에서 Attribute를 지정하여 수정한다.

(V3.0) OS의 환경 변수를 받아들이는 방법

Bulletin no : 10112

UNIX BYBTE를 사용하면서 DBACLE.BIB와 같은 환경전수를 GLOBAL VARIABLE 이나 FORMB2의 FIELD로 받아들이고자 할 때가 있다. 그러나, 이러한 기능을 제공하는 FUNCTION이 없다.

- 해결방법

(1) USER EXIT를 사용한다.

예를들어 PROCEDURE에서 DB 환경전수를 읽어 FORMB로 넘겨주도록 한다.

(2) 간단한 BQPLIB BCRIPT를 만들어 사용한다.

< EXAMPLE >

. BQPLIB에서

```
BQL> DROP TABLE ENV;
```

```
BQL> CREATE TABLE ENV ( VAR VARCHAR2(20));
```

. UNIX에서

```
$?l mvvar.sql
```

```
BET ECHO OFF;
```

```
BET VERIFY OFF;
```

```
DELETE FROM ENV;
```

```
INSERT INTO ENV VALUES ( '21');
```

```
EXIT;
```

. FORMB 3.0에서

필요에 따라 :GLOBAL.BIB 혹은 ENVVAR FIELD를 샘플한다.

PROCEDURE 샘플

```
PROCEDURE GET_ENVVAR IS
```

```
BEGIN
```

```
HOST('sqlplus -s user/pass mvvar'||'$?l:mvvar');
```

```
SELECT VAR INTO :GLOBAL.BIB FROM ENV;
```

```
END;
```

(v30) SQL*FORMS V3에서 SQL*REPORTWRITER로 PARAMETER 전송하는 방법

Bulletin no : 10937

1. ReportWriter에서

```
SELECT * FROM EMP  
WHERE l=100 and c=100 and d=100
```

2. SQL*Forms에서

```
HOST('runway long scott/tiger parameteres sh='  
    || to_char(10) || ' sc=' || to_char(10) || ' rd=' || to_char(10));
```

(30) FORMS30 및 STORED PROCEDURE 에서 ROLLBACK SEGMENT 사용

Bulletin no : 10083

1. FORMS30 예제 'SET TRANSACTION USE ROLLBACK SEGMENT' 사용

```
NAME = log-mathik
TRIGGER_TYPE = U3
TEXT = <<<
    BEGIN
        DECLARE
            rk varchar2(10);
        BEGIN
            rk := 'R02';
            COMMIT;
            dbms_transaction.use_rollback_segment(rk);
            INSERT INTO emp SELECT * FROM emp;
        END;
    END;
```

2. STORED PROCEDURE 예제 'SET TRANSACTION USE ROLLBACK SEGMENT' 사용

```
SQL> CREATE PROCEDURE rlc_proc IS
2  BEGIN
3  DECLARE
4      rk varchar2(10);
5  BEGIN
6      rk := 'R02';
7      COMMIT;
8      dbms_transaction.use_rollback_segment(rk);
9      INSERT INTO emp SELECT * FROM emp;
10 END;
11 END;
12 /
```

RUNNING SQL_TRACE FROM ORACLE TOOLS

Bulletin no : 10245

1)SQL*Form Version 3 예제

Oracle Korea Technical Bulletin (BND 101) 참조

방법 1 : Designer에서 -> Option 서 statistics 선택 -> Run Form

방법 2 : RunForm300.c command line에 -x option 사용

2)Oracle Form Version 4.5 예제

the Oracle Form Reference Manual, Volume 1참조

방법 1 : Designer에서 -> Edit/Preferences 에서 statistics선택
-> Run Form

방법 2 : F45BIM command line에 "statistics=YES" 사용

3)SQL*Reporter Version 1.1 예제

Designer에서 새로운 field생성

Name sql_trace

Source SQL alter session set sql_trace true

Group Report

Width 1

Skip N

4)ORACLE Reports Version 2.5 예제

Oracle Reports Reference Manual 참조

방법 1 : Report Layout의 Formula Column생성

Formula: BM,DD_BQL('ALTER SESSION SET SQL_TRACE TRUE');

방법 2 : Before Form Trigger 들 생성

BM,DD_BQL('ALTER SESSION SET SQL_TRACE TRUE');

5)SQL*Plus 예제

Oracle Korea Technical Bulletin (BND 101) 참조

STORED PROCEDURE & DATABASE TRIGGER

Bulletin no : 10123

* STORED PROCEDURE

1. Oracle Form 4.5에서는 PL/SQL Code를 Server-Side에 저장하거나 Form에 Trigger, Procedure를 갖출 수도 있다.

다음과 같은 경우 Procedure를 Database Server에 저장하는 방법을 선택하는 것이 좋다.

- Procedure가 Standard한 기능을 제공하여 다른 Tool이 공유 할 수 있을 때.
- Procedure가 중요한 SQL기능을 가질 경우.
- Oracle Form 4.5에서는 PL/SQL의 Version을 1.1까지 지원하지므로, PL/SQL Version 2 의 기능이 필요한 경우.

2. Stored Procedure의 제한사항

- (1) Server가 7.0 이상이어야 한다
- (2) Parameter의 Variable은 2000 Byte이하이어야 한다.
 - 2000이 넘으면 Truncata가 되고, PL/SQL에서 Value_Error Exception이 발생한다.
- (3) Main PL/SQL 에서는 Stored Procedure와 Function을 부를 수 없다.
- (4) Stored Subprogram의 정의가 마친 경우, Form을 다시 Server와 Connect 하기전에는 명함을 주지 않는다.

3. Oracle 7 Server 에서 제공하는 Standard Package들

- dbms_alert
- dbms_ddl
- dbms_descrlhs
- dbms_lock
- dbms_mail
- dbms_output
- dbms_pipe
- dbms_session
- dbms_utility
- dbms_utility

< Oracle7 Developer Guide 참조>

4. Calling Stored Procedure

- (1) Database Procedure는 Server-Side PL/SQL Engine에 의해 실행되도록 Design PL/SQL Block이다.
- (2) Stored Procedure를 Call하기 위해서는 EXECUTE Privilege가 필요 하다.
- (3) Server-Side Procedure와 Form에서의 PL/SQL의 중요한 차이점은 Server-Side 에서는 Form의 Bind Variable (예 : :Block_Name, Item_Name등)을 인식하지 못한다는 것이다.
- (4) Parameter/Return의 가능한 Data Type

- VARCHAR2 : Maximum of Varchar2(2000)
- NUMBER
- DATE
- BOOLEAN

제한사항

- TABLE/DUOTYPE, TABLE, COLUMN/DUOTYPE으로 정의된 것은 Reference가 만된다.
- Unsupported Parameter나 Return Value를 사용하면 다음의 Error가 발생한다.
 - * PL/SQL error 313 at line xxx, column yyy 'PROCNAME' not declared in this scope
 - * PL/SQL error 201 at line xxx, column yyy identifier 'FUNCTION' must be declared
- (5) Formal Parameter의 Default Value
 - Default Value는 제공하지 않는다.
 - User가 임의로 줄 수 있다.
- Procedure Private_Test(a in VARCHAR2 := 'Hello' & in
 VARCHAR2 := 'There') IS
 BEGIN
 Mmc_Output.Put_Line(a);
 Mmc_Output.Put_Line(h);
 END;

(8) 다른 User의 Stored Subprogram이나 Remote Database를 Call하려면

- User Name이나 Database Link명을 줄이기 위하여 Synonym를 생성하여야 한다.
- CREATE SYNONYM lth_hr_syn FOR lthowner.lth_hr;
- Program에서 Call할 때는 xx_syn := lth_hr_syn.get_syn(:EMP, EMPNO);

5. PL/SQL Compiler가 해당 Procedure를 찾는 순서

- (1) Current PL/SQL Block내에 정의되어 있는가?
- (2) Standard PL/SQL Command인가?
- (3) Oracle Form내의 Built-in Procedure혹은 Function인가?
- (4) User Named Procedure혹은 Function인가?
- (5) Server-Side에 정의된 ORMB-STANDARD Package인가?
- (6) Current user가 Server-Side의 어떤 Procedure나 Function을 Access하는가?

* 이상의 질문에 대하여 전부 'NO'라는 대답이 나오면 Compiler는 다음과 같은 Error Message를 준다.

- PL/SQL error 313 at line xxx, column yyy 'PROCNAME' not declared in this scope
- PL/SQL error 201 at line xxx, column yyy identifier 'FUNCTION' must be declared

* DATABASE TRIGGER

1. Database Trigger는 Form에서의 Trigger와 개념적으로 거의 동일하다.

사이점은 Trigger가 Fire되는 원인을 제공하는 사건과, 후속코드가 실행되는 장소가 둘인 것이다.

2. Database Trigger는 Table과 연관된 PL/SQL Block들이다. Table에 대해 UPDATE, INSERT, DELETE등의 행위에 의해 Fire된다.

3. Error발생시에는 BAISE_APPLICATION_ERROR Built-in Procedure를 사용하여 Error를 표시한다.
BAISE_APPLICATION_ERROR Procedure에서는 Error Number가 20000 - 20999의 Range를 가진다.

4. Creating & Editing Database Triggers

아래와같은 적절한 권한이 주어진다면 Forms Designer Mode에서 Database Trigger를 직접 Create, Edit, Compile, Remove할 수 있다.

- Execute Privilege, Create Privilege, Compile Privilege, Drop Privilege

* Create하려면

- Navigator에서 Database Objects와 Tables Mode를 Expand한다.
- 원하는 Table을 선택하고 Expand한다.
- Triggers Mode를 선택하고 Menu Bar의 Navigator>Create를 선택한다.
- Database Trigger Editor에서 작성, compile한다.

* Edit 하려면

- Navigator에서 Database Objects와 Tables Mode를 Expand한다.
- 원하는 Table을 선택하고 Expand한다.
- 원하는 Trigger에서 Double-Click하면 Editor가 나타난다.
- Database Trigger Editor에서 편집, compile한다.

(V45) USING DATABASE ROLES WITH ORACLE FORMS 4.

5

Bulletin no : 10253

Database Role에 관하여

Oracle Database의 Role은 권한들의 집합이라고 할 수 있다. 먼저 Role은 Create Role권한을 가진 User에 의하여 생성된다. Role이 생성되고나면 특정 권한이 해당 Role에 Grant된다. 일반적으로 Role에 주어지는 권한은 Table에 대한 Select, Insert, Update등 이다. Role은 다시 특정User나 Role에 Grant되고, 해당Role이 가지고 있는 권한이 계승된다.

Example : 다음의 예는 사원데이터를 입력, 수정, 조회하는 Role을 생성하는것이다.

--Create A Role Called Hr

Create Role Hr;

--Grant Specific Privileges To The Hr Role

Grant Select, Insert, Update On Emp To Hr;

--Create A Password Protected Role Called Payroll

Create Role Payroll Identified By Paycheck;

--Grant Specific Privileges To The Payroll Role

Grant Select, Insert, Update On Compensation To Payroll;

--Create A Role Called Auditor

Create Role Auditor;

--Grant Specific Privileges To The Auditor Role

Grant Hr, Payroll To Auditor;

이상은 Role이 다른 Role에 Grant되는 것을 보여준다.

다음은 위에서 생성한 Hr, Payroll, Auditor를 User에게 Grant하는 것이다.

Grant Hr To Johnson;

Grant Hr To Jansmith;

Grant Payroll To Jansmith;

Grant Auditor To Bushrow;

만약, User의 Role이 Enable되어 있지 않으면 Set Role Command를 사용하여 특정한 User에게 Session중에 Role을 지정할 수 있다.

General Overview Of The DBMS_SESSION.SET_ROLE

Oracle에서 제공하는 DBMS_SESSION Package는 Database Creation Category, SQL을 실행하여 생성된다. 이 Package는 Oracle Form에서 직접적으로 실행할 수 없는 SQL Command를 처리 할 수 있게 해준다.

DBMS_SESSION Package의 SET_ROLE Procedure를 이용하여 SQL Set Role Command와 같은 결과를 처리 할 수 있다. 이것은 어느 Form의 Procedure나, PL/SQL Block, Form Trigger에서뿐만 아니라 Database Trigger에서도 사용 할 수 있다.

현재 Session을 맺어 실행중인 Form Program에서 DBMS_SESSION.SET_ROLE Procedure를 이용하여 현재의 Session에서 필요한 특정한 Role을 Setting할 수 있다.

DBMS_SESSION.SET_ROLE의 Syntax는 다음과 같다.

DBMS_SESSION.SET_ROLE(Role_Col VARCHAR2)

Example:

--Setting The Hr Role From Form

DBMS_SESSION.SET_ROLE('Hr');

--Setting The Payroll Role From Form

DBMS_SESSION.SET_ROLE('Payroll Identified By Paycheck');

Oracle? Server Application Developer's Guide참조.

Building A Generic Procedure To Set A Role Before Calling A Form

개발자들은 종종 Form에 Database Role을 연관하여 처리하고 싶어한다. 다음은 그 예이다.

Procedure Check_For_Qualified_User(Role_Col IN VARCHAR2,

Form_Name IN VARCHAR2) IS

--The Procedure Has Two In Arguments: The Role To Be Set(Role_Col)

--And The Form To Be Called(Form_Name)

--The Following Declared Exception And Pragma Are Used To "Trap" A User Who

--Does Not Have The Required Role.

Unqualified_User EXCEPTION;

```

Program Exception_Init(Unauthenticated_User, -1824);
Begin
--Get The Role To The Role Identified By The Role_Cod In Argument
  DBMS_SESSION.SET_ROLE(Role_Cod);
--Call The Form Identified By The Form_Name In Argument
  Call_Form(Form_Name);
Exception
  When Unauthenticated_User Then
    Message('You Have Not Been Granted Privileges To Use The '
           || Upper(Form_Name) || ' Form', Acknowledge);
    Message('Check With The DBA If You Think You Should Have Access. ');
End;

```

General Restrictions

Call_Form하여 Called된 Form에서 DBMS_SESSION.SET_ROLE을 실행 한 경우
 Calling Form에서도 Called Form에서 적용된 것이 유효하다.
 하지만 Open_Form으로 새로운 Form이 Run된 경우는 Calling Form에는 Role이 적용되지 않는다. 이
 제한사항은 When-Window-Activated Trigger에서 Activated된 Role을 Setting하여 주어야 한다.

Common Errors

Err-1818 : DBMS_SESSION.SET_ROLE에서 지정한 Role이 존재하지 않는 경우
 Err-1878 : Role_Command에 Password가 필요한 경우

(V3.0) VT100 에서 KEY-F0 ~ KEY-F9 사용법

Bulletin no : 10281

1. customer를 invoke한다.

(customer사용강령은 Oracle Korea Customer Support Technical Bulletin 101참조)

2. resource : vt100 open

3. [product]의 [key] option을 선택

4. product key name에 KEY B function code에 10031 을 입력

5. [mapping]의 [key] option을 선택

6. product key name에 KEY B device key name에 FB 을 입력

7. [device]의 [key] option을 선택

8. device key name에 FB 을 입력

9. [file]의 [save]를 선택

10. exifore30에서 KEY-FB 프리거에서 execute_query를 지정

11. exifore30시에 [FB]키를 누르면 query가 수행된다.

(V4.5) WINDOWS PERFORMANCE TUNING FOR FORM S4

Bulletin no : 10187

Subject: Window Tuning For F4

PERFORMANCE TIPS FOR WINDOWS

The following suggestions should assist you in maximizing the performance of your Windows 386 Enhanced mode installation (Windows is capable of using Virtual memory in 386 Enhanced mode). Many of these suggestions apply to standard and real modes as well.

Of course, running Windows in Standard mode is much faster than 386 Enhanced mode, but a lot of applications require Windows in 386 Enhanced mode, including our very own BQLWNET for Windows!

1. USE SMARTDRIVE. The Microsoft SMARTDrive disk caching driver can produce the largest single Windows 3.1 performance improvement.

Use SMARTDrive whenever possible. You can enable SMARTDrive by loading the SMARTDRV.BYP driver in your CONFIG.BYP or by loading SMARTDRV.EXE in your AUTOEXEC.BAT. There are a variety of options available on these drivers.

I recommend turning OFF the Write caching if you have a LOCAL Dracis RAMMS on your PC. For basic information on SMARTDrive installation and operation, refer to the "Microsoft Windows Users Guide" version 3.1 manual or even SMARTDRV /? at the DOS prompt.

Caution: If you run into a lot of GPFs in your Windows applications you will most likely have cross linked files on your hard disk due to SMARTDrive not being able to flush its write buffers before the crash. Run CHKDSK or SCANDISK at least once a week to guard against losing these files. Even better, run something like the Norton Disk Doctor.

2. KEEP YOUR HARD DISK OPTIMIZED. A fragmented hard disk greatly impacts Windows's performance, especially when a temporary swap file and/or SMARTDrive is installed. Use a hard disk optimizer program on a weekly basis to keep your disk contiguous.

3. CREATE A PERMANENT SWAP FILE. Using a permanent swap file improves performance over using a temporary one. Under Windows 3.1, if supported by your hardware, also select the Use 32-Bit Disk Access check box in Control Panel by choosing the 386 Enhanced icon, choosing the

Virtual Memory button, then choosing the Change button.

4. **TURN OFF GRAPHICS PORT TRAPPING.** The speed of MS-DOS applications running under 386 enhanced mode can be noticeably improved by not selecting any of the Monitor Ports options in the Advanced section of the PIF Editor. The High Graphics option provides the widest range of MS-DOS application compatibility but is not required for most applications.
5. **TURN OFF THE FILESYNCHANCE-SETTING.** Windows 3.1 386 enhanced mode can monitor disk access by MS-DOS applications and send directory update messages to File Manager. This allows File Manager to be automatically updated by changes MS-DOS applications have made to files or directories. However, this option is not a necessity, and leaving it off (the default) speeds file access by MS-DOS applications. To disable this feature, set FileSyncChange=0 in the [386enh] section of the SYSTEM.INI file.
6. **TURN OFF THE RESERVEPAGEFRAME-SETTING.** Turn this setting off if you do not require expanded memory support for MS-DOS applications. Turning this option off ensures that you're getting the most possible memory in virtual MS-DOS machines. To disable this feature, set ReservePageFrame=0 in the [386enh] section of the SYSTEM.INI file.
7. **USE THE RIGHT NUMBER OF MS-DOS BUFFERS.** If you are using SMARTdrive, set the number of MS-DOS disk access buffers in your CONFIG.SYS file to 15 (that is, BUFFERS=15). Using a greater number of buffers with SMARTdrive will actually decrease efficiency. If you are not using SMARTdrive, use BUFFERS=30.
8. **USE THE LOWEST COMMON DISPLAY DRIVER.** Using a display driver with a high resolution or large number of colors results in slower display performance. If you do not require the extra features of the display driver, use a driver with less capability. Usually this suggestion applies to display systems that are VGA compatible but offer an extended mode driver, such as the Video Beam or BS14. Using the standard VGA driver instead offers faster display performance but less resolution and/or color support.

(V45) ORACLE FORMS 4.5 에서 WINDOWS APPLICATION을 호출하는 방법

Bulletin no : 10278

General Overview

=====

Oracle Form에서 Ora_FFI를 이용하여 Windows Application을 호출하면 Form Program과 Window Application이 Asynchronous하게 실행이 가능하다. 호출된 Window Application은 사용자가 임의로 Close하기 전까지 남아있게 된다. 현재 사용하고 있는 Form을 Close하거나, Form Runtime에서 빠져 나가도 호출한 Window Application을 Close시키지 않는다.

출론 사용자는 사용중인 Form과 호출한 Application간을 자유롭게로 움직이면서 사용할 수 있게 된다. 이것은 이전 Version에서 사용하던 Host Command와는 다른 형태로 실행된다. Host Command를 이용하여 Window Application을 호출하면 이것은 Synchronous하게 실행된다. 즉, Host로 호출한 Window Application이 끝나기 전에는 사용중이던 Form로 Focus가 돌아 오지 않게 된다.

Form에서 Window Application을 호출하기 위해서, Ora_FFI Command에서 Kwal388.Exc라는 Windows Library를 사용한다.

구체적인 예들 처론다.

Creating Winexec Package Spec & Package Body

=====

Form에서 다음과 같은 Winexec Package Spec를 생성한다.

```
Package Winexec Is
    Function Winexec(Cmd In Out Varchar2) Return Boolean;
End;
```

Form에서 다음과 같은 Winexec Package Body를 생성한다.

```
Package Body Winexec Is
    Dll_Handle      Ora_FFI.Libhandletype;
    Winexec_Handle  Ora_FFI.Funchandletype;

    Function Winexec1(Handle In Ora_FFI.Funchandletype,
        Cmd In Out Varchar2,
        Disp_Mode In Plo_Integer)
        Return Plo_Integer;
    Pragma Interface(C, Winexec1, 11285);

    Function Winexec(Cmd In Out Varchar2)
        Return Boolean Is
        Dll_Handle := Ora_FFI.Libhandletype;
```



```

    Top Pic_Integer;
Begin
  Top := Unpackcd(Winexec_Handle, Cmd, 1);
  If Top = 2 Then /* Cannot Find Program Or One Of Its Components */
    Message('Cannot Find File ' || Cmd || ' (Or One Of Its Components)');
    Return False;
  Else Return True;
  End If;
End Winexec;

Begin
  Dll_Handle := Dev_F21.Register_Library(NULL, 'Kernel32.Exe');
  Winexec_Handle :=
    Dev_F21.Register_Function(Dll_Handle, 'Winexec', Dev_F21.Pascal_Bid);
  Dev_F21.Register_Return(Winexec_Handle, Dev_F21.C_Int);
  Dev_F21.Register_Parameter(Winexec_Handle, Dev_F21.C_Char_Ptr);
  Dev_F21.Register_Parameter(Winexec_Handle, Dev_F21.C_Int);
End Winexec;

```

Unpackcd의 Display_Mode Argument 의 설정된 값은 다음과 같다.

- 1 = Normal
- 2 = Minimized
- 3 = Maximized
- 4 = Starts Normal Size But Without Focus
- 7 = Starts Minimized Without Focus

Creating A Trigger To Call Windows Application

=====

```

User-Button-Pressed Trigger
Declare
  Top Boolean;
  Command Variable2(30) := 'Calc.Exe';
Begin
  Top := Winexec(Winexec(Command));
End;

```

위의 Example은 Windows의 Calculator를 호출하는 예이다.

다른 Windows Application을 호출하려면 Winexec, Winexec의 Parameter인 Command에 원하는 Executable를 지정하면 된다.

단, 만약 호출하고자 하는 Executable이 Box Path 내에 없는 것이라면, Command Parameter 내에 Full Path를 지정해 주어야 한다.

예 : Command Variable2(30) := 'C:\Windows\Winword.Exe'

PRO*C 2.0의 새로운 특징

Bulletin no : 10288

1. C Preprocessor 지원

C Preprocessor를 포함한다. 즉, Define 명령을 사용하여 constant와 macro를 정의하고 Variable과 같은 Pro*C Data Types를 매개변수를 이용하여 선언할 수 있다. 또한 `sqlca.h`, `osaca.h`, `sqlids.h`같은 Pro*C header 파일들을 `%include` 명령을 사용하여 `%include` 가능하다.

```
%include  
%define NAME_LEN 10  
.....  
VARCHAR      name(NAME_LEN)
```

2. Host변수에 Structure 사용가능

Embedded SQL 문장에 Host Variables를 포함하는 일반적인 C Structure를 사용할 수 있다. 예를 들어, `SELECT empno, name, sal FROM emp`를 아래와 같이 정의된 C Host Structure에 넣을 수 있다.

```
struct  
{  
    int emp_no;:  
    char name[10];  
    float salary;  
} emp_rec;
```

그리고 아래와 같은 Embedded SQL 문장에서 이 Structure를 사용한다.

```
EXEC SQL SELECT empno, name, sal  
      INTO :emp_rec  
      FROM emp  
      WHERE empno = 7499;
```

Host Structure는 또한 Host Array를 포함할 수 있다.

3. Declare Sections가 필요치 않다

NOBEC-00000E로 Precompile하, Declare Section에서 Host 변수, Structure를 선언할 필요가 없다. 이것은 ANSI C-Style Function Prototypes를 사용하는 프로그래머에게는 커다란 장점이다.

말의 버전에서 Function으로 Host 변수를 보내려면, Old-Style C Function Definition를 사용해야 했다. Name, Salary가 Host 변수인 Function은 아래와 같이 사용해야 했다.

```

let my_func(name, salary)
EXEC SQL BEGIN DECLARE SECTION:
    char *name;
    float salary;
EXEC SQL END DECLARE SECTION:
{
/* function body using name and salary as
   host variables */
}

```

Oracle Release 2.0이전 이 function을 다음과 같이 정의할 수 있다

```

let my_func(char *name, float salary)
{
/* function body using name and salary as
   host variables */
}

```

4. EXEC SQL WHENEVER

Whenever . . . No사용시 Parameters를 받아 값을 돌려주는 Function Call을 지원한다. 예를 들어 아래와 같이 하면

```
EXEC SQL WHENEVER SQLERRORS DO rv = error_func('Message');
```

message를 프린트하고 값을 return하는 에러처리 function을 call할 수 있다.

5. Optimizer Hints

SELECT, DELETE, UPDATE command에 Oracle7 optimizer hint syntax를 지원한다.

6. DBMS = V6_CHAR Option

이 옵션으로 모든 Standard Oracle7 기능을 가질 수 있다(The Semantics Of Character Strings를 제외함). 이 Option을 사용하면 Character Strings는 Fixed Length가 아닌 Oracle Version 8과 같이 Variable Length로서 사용된다.

Nullintim : 102B4

This bulletin addresses the topic of automatic startup and shutdown of BQLNet II servers when a UNIX system is brought up or down respectively.

Please refer to Bulletin 105870.888 (Automatic Startup and Shutdown of Oracle Database Instances) for background information.

For more detailed technical explanations and alternatives, readers are advised to refer to the System Administrator's Guide for the appropriate UNIX port.

BQL-Net v1.0:	\$ORACLE_HOME/bin/tnslsnr - BQL-Net Listener for protocol tnsn.
	For example, tnslsnr(TCP/IP), tnslsnr (SPK/IPK) etc.
	\$ORACLE_HOME/bin/lsnrctl - Control program to start, stop or debug the corresponding listener process. For example, lsnrctl(TCP/IP), lsnrctl (SPK/IPK) etc.
BQL-Net v2.0:	\$ORACLE_HOME/bin/tclsnr - BQL-Net Listener for various incoming connection requests on different protocols.
	\$ORACLE_HOME/bin/lsnrctl - Control program to start, stop or debug the working of tclsnr.
	/etc/tclsnr.ora or \$TNB_ADMIN/tclsnr.ora - tclsnr aliases for connect strings for clients to connect to server
	/etc/ltnsr.ora or \$TNB_ADMIN/ltnsr.ora - ltnsr details for listener specifying machine, port, protocol etc. that listener should be listening for.

File names: /etc/oratab or - specifies Oracle Instance Information
to v1.0 & v2.0 /var/opt/oracle/oratab

/etc/services - specifies Listener ports on the
UNIX system

For more details on individual BQLNet files above, please refer to the
appropriate BQLNet documentation.

Multi Threaded Server (MTS)

MTS is a facility by which an incoming client connects with a shared dispatcher and server
instead of a dedicated server, unless specifically requested. In BQLNet v1.0, an Oracle
shadow process is spawned to serve each individual incoming connection. This leads to more
memory and process usage.

MTS, on the other hand pre-spawns dispatcher and server processes that serve the client
connection. The dispatcher process looks at the incoming connection, and depending on the
protocol of the incoming request and the server load distribution, "introduces" the client to
a server process, that might already be serving other clients. Thus, a new dedicated process is
not spawned for each incoming connection.

Thus the dispatcher and server processes are shared among various incoming connections.
Processes named as d000 and s000 are shared dispatcher and server processes respectively.

Ownership and Permissions

The Listener binary, RLServ and tclsr, should be owned by the superuser(root) and the
dba group. The permissions on the binary should be 4751. The initial '4' in the 4751 sets the
SETUID bit for the binary.

To change ownership and group ownership to root/dba, do:

```
chown root
```

```
chgrp dba
```

as superuser.

To change permissions on the Listener binary, do:

```
chmod 4751
```

as superuser.

Sample startup and shutdown of SQL*Net Listeners

To start/stop/check status for a SQL*Net v1 Listener:

lsnrctl start	\$ starts lsnrsv
lsnrctl stop	\$ stops lsnrsv
lsnrctl stat	\$ gives status for lsnrsv

To start/stop/check status for the SQL*Net v2 Listener:

lsnrctl start	\$ starts tnsclsp
lsnrctl stop	\$ stops tnsclsp
lsnrctl stat	\$ gives status for tnsclsp

Platform Specific Details

To automate the Listener startup and shutdown process, the appropriate command should be listed in the correct directory/files so that the UNIX system startup process includes this command in its list of things to do.

Following are some details to implement the automatic startup and shutdown of SQL*Net Listeners on six major flavors of UNIX.

NOTE: If you are using NTBS, please make sure you start the SQL*Net Listener(s) **BEFORE** the Oracle instance is started.

NOTE: In the following sections, 'oracle' refers to the owner of the Oracle installation, usually 'oracle'. <\${ORACLE_HOME}> refers to the expanded form of \${ORACLE_HOME} since it might not be set for the superuser.

NOTE: In the following sections, 'lsnrctl' refers to the appropriate control program for the corresponding Listener. For example 'tspctl' for 'oracle', which listens for incoming TCP/IP connections.

(1) SunOS 4.1.3

Important Files

/etc/rc.local - The startup script for the system

Details

For startup, make the following entry in /etc/rc.local:

```
For BQL:Net v1: su -c '<${ORACLE_HOME}/bin/hsxct1 start'
For BQL:Net v2: su -c '<${ORACLE_HOME}/bin/lsxct1 start'
```

Since only /etc/shutdown is invoked at system shutdown time, a script such as the following can be used to bring down Oracle before the system facilities are shutdown:

```
#!/bin/csh

/bin/su -c '<${ORACLE_HOME}/bin/hsxct1 stop' & For BQL:Net V1
/bin/su -c '<${ORACLE_HOME}/bin/lsxct1 stop' & For BQL:Net V2
/bin/su -c <${ORACLE_HOME}/bin/diskut

/etc/shutdown &&
```

The script should be owned and executable only by the supervisor.

(2) OSF/1

```
/etc/initdb -- Controls the initialization process
/sbin/rc3 -- Script to start/stop processes at run level 3
/sbin/rc0 -- Script to start/stop processes at run level 0

/sbin/rc3.d/888orac1e -- link to the script /sbin/init.d/oracle
/sbin/rc0.d/801orac1e -- same as above
```

Details

In this case, the system executive scripts in the /sbin/rc3.d directory at system startup time. It executes scripts in /sbin/rc0.d at system shutdown time.

If the name of the script starts with a 'B', a parameter of 'start' is provided to the script. On the other hand, if the name starts with a 'E', 'stop' is sent as the parameter to the script. That's why, names such as '888orac1e' and '801orac1e' have been suggested here.

CAUTION: If there are separate scripts to shutdown the Listener and the Oracle Instance, the order of invocation should be given some thought. Thus, you might want to call the startup script for SQL*Net Listener as 'BBBOracle' and that for the Instance startup as 'BBOracle'.

At startup, the Listener should be started after the Instance has been brought (unless you are using NTB, in which case, the order MUST be reversed).

At shutdown, the Listener should be shutdown before the Instance is.

The following commands should be included in the appropriate script:

For startup:

/bin/su -c '\$ORACLE_HOME/bin/tnslsnr start' # For SQL*Net v1

/bin/su -c '\$ORACLE_HOME/bin/lsnrctl start' # For SQL*Net v2

For shutdown:

/bin/su -c '\$ORACLE_HOME/bin/tnslsnr stop' # For SQL*Net v1

/bin/su -c '\$ORACLE_HOME/bin/lsnrctl stop' # For SQL*Net v2

NOTE: The links to /sbin/init.d/oracle might not be correctly established. There may be more links than those 2, forcing the startup or shutdown to happen more than once.

(3) Solaris 2.3

Important Files

```
/etc/init.d      -- Controls the initialization process
/etc/rc2        -- Script to start/stop processes at run level 2
/etc/rc0        -- Script to start/stop processes at run level 0

/etc/rc2.d/BBOracle -- link to the script /etc/init.d/lsnrstart
/etc/rc0.d/DDOracle -- link to the script /etc/init.d/lsnrshut

/var/opt/oracle/oratab -- Oracle oratab file
```

Details

For startup, put the following commands in the script /etc/init.d/lsnrstart:


```

#ls/mu -c '$ORACLE_HOME/bin/kmctl start' # For BQLNet v1
#ls/mu -c '$ORACLE_HOME/bin/lscctl start' # For BQLNet v2
For shutdown, put the following commands in the script /etc/init.d/lscshut:

```

```

#ls/mu -c '$ORACLE_HOME/bin/kmctl stop' # For BQLNet v1
#ls/mu -c '$ORACLE_HOME/bin/lscctl stop' # For BQLNet v2
Again, the order of startup/shutdown should be decided keeping NYS in mind.

```

(4) AIX

Important Files

```

/etc/initab -- System initialization file
/etc/rc      -- System initialization script
/etc/mktab  -- Utility to make an entry in /etc/initab
/etc/shutdown -- System shutdown script

```

Details

The user can choose one of the following methods for startup:

1. Use /etc/mktab to make an entry in /etc/initab :

The command(s):

```

/etc/mktab "2:mult:/ls/mu -c
'$ORACLE_HOME/bin/kmctl start' > /dev/console 2>&1"

```

```

/etc/mktab "2:mult:/ls/mu -c
'$ORACLE_HOME/bin/lscctl start' > /dev/console 2>&1"

```

can be used to introduce kmctl or lscctl respectively in the /etc/initab file. Thus, on system startup, at run level 2, the appropriate listener control program is invoked by the system as the Oracle owner.

2. This method is essentially the same as (1) above, but with one level of indirection. Use /etc/mktab as follows:

```

/etc/mktab "rcconsole:2:mult:/etc/rc.oracle > /dev/console 2>&1"

```

Also create a script `/etc/rc.oracle`, owned and executable only by root.
The contents of this script should be:

```
#
# Startup the oracle database
#
/bin/su -c '<${ORACLE_HOME}/bin/lsnrctl start' & For BQLNet U1
<${ORACLE_HOME}/bin/lsnrctl start & For BQLNet U2
EOF
```

3. Make the following entry at the end of the startup script `/etc/rc`:

```
/bin/su -c '<${ORACLE_HOME}/bin/lsnrctl start' & For BQLNet U1
/bin/su -c '<${ORACLE_HOME}/bin/lsnrctl start' & For BQLNet U2
```

For shutdown, write a script similar to the following. The script should be owned and executable by the supervisor only:

```
#!/bin/sh

/bin/su -c '<${ORACLE_HOME}/bin/lsnrctl stop' & For BQLNet U1
/bin/su -c '<${ORACLE_HOME}/bin/lsnrctl stop' & For BQLNet U2
/etc/shutdown &*
```

(5) HP/UX

Important Files

```
/etc/rc          -- System startup file
/etc/shutdown    -- System shutdown executable
/etc/shutdown.d  -- System shutdown directory
```

Details

At system startup, the script `/etc/rc` is executed. Thus, to provide automatic startup for the BQLNet listeners, add the following to the logical end of the script:

```
/bin/su -c '<${ORACLE_HOME}/bin/lsnrctl start' & For BQLNet U1
/bin/su -c '<${ORACLE_HOME}/bin/lsnrctl start' & For BQLNet U2
```

At system shutdown, all scripts and executables in `/etc/shutdown.d` are

executed. To bring the listeners down, write a simple script as follows and place it in /etc/shutdown.d:

```
#!/bin/csh

/bin/cu -c '<${ORACLE_HOME}/bin/hsnctl stop' & For SQL*Net U1
/bin/cu -c '<${ORACLE_HOME}/bin/lsnctl stop' & For SQL*Net U2
/bin/cu -c <${ORACLE_HOME}/bin/dhsnctl

/etc/shutdown &#
```

(6) SCO UNIX

Important Files

```
/etc/init.d      -- Controls the initialization process
/etc/rc2         -- Script to start/stop processes at run level 2
/etc/rc0         -- Script to start/stop processes at run level 0

/etc/rc2.d/S000svcs -- link to the script /etc/init.d/lsnstart
/etc/rc0.d/S000svcs -- link to the script /etc/init.d/lsnshut

/etc/psvcs       -- Oracle svcs file
```

Details

Please refer to the section for Solaris 2.3. The implementation details are very similar to those.

CLINET/SERVER 환경에서의 PERFORMANCE TUNING

Bulletin no : 10188

PERFORMANCE TUNING에는 3가지면에서의 APPROACH가 가능하다.

- . SYSTEM LEVEL : HARDWARE AND SOFTWARE
- . DATABASE CONFIGURATION LEVEL : INIT, DBA AND STRUCTURE
- . APPLICATION LEVEL : APPLICATION SPECIFIC DEVELOPMENT

다음에서는 위의 3가지 접근방법을 혼합하여 PERFORMANCE TUNING에 관하여 살펴본다.

INIT, DBA

INTERNAL PARAMETERS중에서 DBACLE PERFORMANCE에 중요한 영향을 미치는 3가지의 PARAMETERS를 살펴본다.

- . DB_BLOCK_BUFFERS
- . SHARED_POOL_SIZE
- . DBWR_AREA_SIZE

< DB_BLOCK_BUFFERS >

DB_BLOCK_BUFFERS는 DATA CACHING을 위해 BUFFER 할당되는 MEMORY를 지정한다.

이 숫자가 증가 될수록 PERFORMANCE는 향상된다. SQL 문장이 실행될때, 만약 CACHE 내에 RECORD가 없다면 실행결과를 읽어들이고 CACHE에 올려놓는다.

CACHE내에 올려진 DATA들은 LRU 알고리즘에 따라 삭제될때까지 BUFFER에 위치하게 된다. 필요한 자료를 BUFFER로부터 읽어오지 않고 MEMORY내에서 찾아오는 비율이 높을수록 PERFORMANCE가 높다. DB_BLOCK_BUFFERS는 실행되는 MEMORY를 지정하는 것이 아니다, BUFFER수를 지정한다. 실제로 지정되는 MEMORY의 양은 DB_BLOCK_BUFFERS * DB_BLOCK_SIZE의 값이된다. DB_BLOCK_BUFFERS의 DEFAULT 값은 HARDWARE에 따라 다르게 지정된다.

일반적으로 실행 MAIN MEMORY의 25%이하로 잡도록 한다.

< SHARED_POOL_SIZE > (Data Dictionary Cache, Shared Sql Area)

SHARED_POOL_SIZE는 BUFFER에 DATA DICTIONARY CACHING과 SHARED SQL문장을 위한 MEMORY의 할당량을 지정한다. DATA DICTIONARY CACHE는 DATA DICTIONARY 요소의 BUFFERING과 관련되어 두 중요하다.

하나의 SQL문장이 실행될때 DBACLE이 DATA DICTIONARY자료를 여러번 참조하기 때문에 DATABASE의 APPLICATION BUFFER와 STRUCTURE가 MEMORY에 많이 올라와 있을수록, BUFFER로부터 읽어들이어오는 자료

가 적어지게된다.

DATA DICTIONARY도 DB_BLOCK_BUFFERS의 경우와 마찬가지로 CACHING 된다. 가장 이상적인 것은 모든 DATA DICTIONARY정보가 MEMORY에 올라와 있는 것이다. SHARED SQL AREA는 DATABASE INTERNAL 이 실행하는 것이나 USER의 독립한 실행 문장을 갖고있다. SQL문장이 실행될때, SHARED SQL STATEMENT AREA에 같은 문장이 있는지를 확인하여 보아 있으면 그 문장이 실행된다. 이것은 PARALLEL TIME를 줄여 전체적인 PERFORMANCE를 향상시킨다. 이때 문장이 같다는 것은 문자열과 SPACE까지도 같아야 하는것을 의미한다. 만약, 문장이 다르다면 새로운 문장이 PARALLEL되어 실행되고 SHARED SQL AREA에 다시 저장된다.

<예>

```
SQL> select name,customer
2   from customer_information;

SQL> select name,customer
2   FROM customer_information;
```

위의 두 문장은 결과를 RETURN하지만, 'FROM'절의 대소문자 때문에 DATABASE에서는 다른 문장으로 인식하여 SHARED SQL AREA의 있는 CURSOR를 재사용하지 않게된다. 이와같이 SHARED SQL AREA의 자료에 대한 재 이용률을 높이려면, BYTE와 POCESIZES를 가능한 많이 사용하는 것이 좋다. SHARED_POOL_SIZE PARAMETER의 값은 BYTE 단위로 지정된다. DEFAULT값은 BYTES에 따라 다양하다.

< SHARED_POOL_SIZE >

SHARED_POOL_SIZE는 USER당 SHARED POOL에 필요한 MEMORY의 양을 지정한다. 이것은 SGA 외부에 존재하며, 필요시에만 사용된다. SHARED_POOL_SIZE PARAMETER의 값은 BYTE단위로 지정하며, USER당의 POCESIZE에 해당하는 값이다.

< DATABASE STRUCTURE ANALYSIS >

DATABASE REORGAN의 기능과 저장 능력이 증가됨에 따라 각기 다른 TABLESPACE로 DATABASE STRUCTURE를 나누고, 다중 BIBE와 CONTROLLER를 통하여 성능을 향상시키고, TABLESPACE의 FRAGMENTATION을 감소하는것이 중요하게 대두되고 있다.

DATABASE를 INSTALL하면 DEFAULT로 DATABASE REORGANIZATION TABLESPACE(DATABASE_PDATA), TEMPORARY TABLESPACE(TEMPORARY_PDATA),USER TABLESPACE(USER_PDATA)가 생성된다. 가능하면 각기 다른 BIBE와 CONTROLLER로 분산 시켜놓는 것이 좋다. 여기에 더하여 USER가 생성하는 APPLICATION이나 OBJECT들도 분산하여 TABLESPACE간의 BIBE I/O를 분산시키는 것이 좋다.

< CLIENT-SERVER ANALYSIS >

. Database Communication Protocol

현재 DATABASE와 접속하기 위해서는 DBCC를 통하거나 DATABASE의 SQL*NET PRODUCT를 통하여 가능하다. PERFORMANCE 분야이다. REDO TRAPPING, REDO LOG HANDLING을 위해서 DATABASE의 SQL*NET을 쓰는것을 권장한다.

. Client Versus Server Processing

CLIENT-SERVER환경을 생각할때 PC나 MACINTOSH UDISTRIBUTION을 SERVER에 CONNECT 하는 것 뿐만아니라 UNIX SYSTEM을 BULKNET을 통하여 다른 UNIX와 CONNECT 하는 것까지도 포함하여 생각하여야 한다. EXECUTION에 대하여 CLIENT 대 SERVER PERFORMANCE의 차이점을 TEST하더라면, QUERY문을 CLIENT BULKPLUS에서 (BEST TIMING ON 상태에서) 실행하여 보고, 그 문장을 SERVER에 전송하여 SERVER BULKPLUS에서 (BEST TIMING ON 상태에서) 실행하여 본다. SERVER SIZE에서 수행하는 것의 주요한 및점은 실행시간이 짧아지는 것이고, 결국 그것은 SYSTEM의 PERFORMANCE를 향상시키는 결과를 갖게된다.

. Client System Configuration

CLIENT 에서 DBL 속도는 매우 중요하다. 더 빠른 PROCESSOR가 추가적인 차이를 가져오게된다. 그러나, 큰 차이점은 RAM의 증가에서 온다.

. Location Of Products And Source Code

대부분의 CLIENT-SERVER 구성은 NOVELL과 같은 NETWORK D/B를 FILE SERVER로 이용한다. 이 구성은 PRODUCT와 SOURCE CODE를 NETWORK DRIVE위에 놓고, CLIENT 에서는 참조만 하도록 한다. 이러한 구성은 제품의 PERFORMANCE간의 접박이나 VERSION 관리에 용이하다. 그러나 SOURCE CODE를 포함하여 제품들 PC CLIENT에 INSTALL하는 것이 속 는 더 빠르다. NETWORK I/O가 감소되기 때문이다.

. Windows Swap File

UNIQUE SWAP FILE은 다른 것들에해 참조되는 FILE들이므로, CLIENT UDISTRIBUTION 에서 가장 빠른 DRIVE에 위치하게 하는 것을 권장한다. 다른 방법은 별도의 MEMORY를 추가하여 (1BM을 권장한다) 1BM VIRTUAL DRIVE를 생성하여 UNIQUE SWAP FILE을 이 DRIVE에 생성하는 것도 좋다. 이 방법은 DBE I/O를 줄이고 MEMORY를 ACCESS하게 한다. 이러한 방법은 과거에 많이 사용하였으나, 몇몇 문제를 야기하기도 한다.

. Stored Procedures, Functions And Database Triggers vs. PL/SQL In Source

STORED PROCEDURE나 DATABASE TRIGERS는 이미 PARBING되어 있어 SQL문장이나 PL/SQL BLOCK을 직접 호출하는 것보다 실행에 필요한 DUEHED를 줄여준다.

. Eliminating Network I/O And Repetitiveness

CLIENT-SERVER 구성에서 PERFORMANCE를 향상시키는 또 다른 방법은 DATABASE를 호출하여 NETWORK DUEHED를 초대하는 반복적인 PROGRAM의 LOGIC을 재구성하는 방법이 있다. 이 방법은 각 RECORD를 조작하는 것이 아니라 한 문장에 의하여 DATA를 조작하는 방법을 간구하는 것이 . 이것은 DATABASE CALL의 숫자도 줄이는 결과를 가져온다.

다음의 방법에서 보면

Method A

1) Start;

2) Fetch Invoice Record;

```

3) Print Invoice Record;
4) Update Invoice Record To Printed;
5) Loop To Start;

```

Method B

```

1) Update all Invoice not Printed to In Process;
2) Start;
3) Fetch Invoice record;
4) Print Invoice Record;
5) Loop To Start;
6) Update Invoice Record To Printed;

```

1000건의 해당 DATA가 있다면, METHOD A에 의해서는 DATA가 1건씩 FETCH되어 1000건의 UPDATE가 발생하게 다. 반면 METHOD B의 경우, 한번의 UPDATE에 의해 1000건의 DATA가 UPDATE되게된다.

. RULE BASED vs. COST BASED OPTIMIZATION

ORACLE 7에 들어오면서, COST BASED OPTIMIZER라는 새로운 ORACLE QUERY OPTIMIZER 개념이 도입되었다. ORACLE 7이전에는 실행 PLAN을 결정하는데 14가지 정도의 RULE에 따르는 RULE BASED OPTIMIZER가 사용되어져 왔다. 이 방법은 SQL OPTIMIZATION에 중요한 요소가 되는 DATA의 분산정도에 대해서는 전혀 인식하지 못하는 방법이었다. DATA의 분산정도는 QUERY 실행의 방법을 결정하는데 중요한 요소이 . 결국, ORACLE7 이전에 있어서의 PERFORMANCE TUNING은 SQL문장을 쓰는 개발자의 방법 선택에 의해 결정 되어져 왔다. COST BASED OPTIMIZER를 사용하는 ORACLE7 에서도 OPTIMIZER HINT를 추가함으로써 RULE BASED OPTIMIZER에 의한 방법의 사용이 가능하다.

ORACLE RULE BASED OPTIMIZER는 INIT.ORA FILE의 OPTIMIZER_MODE PARAMETER를 'RULE'로 SETTING 함으로써도 사용가능하다. 이 PARAMETER의 DEFAULT 값은 'CHOSE'이다. COST BASED OPTIMIZER는 DATA DICTIONARY에 저장되어 있는 DATA 분산 현황을 사용한다. 현황은 다음의 TABLE에서 확인할 수 있다.

USER_TABLES	ALL_TABLES	DBA_TABLES
USER_TAB_COLUMNS	ALL_TAB_COLUMNS	DBA_TAB_COLUMNS
USER_INDEXES	ALL_INDEXES	DBA_INDEXES
USER_CLUSTERS	DBA_CLUSTERS	

현황 정보는 'ANALYZE' COMMAND에 의하여 생성된다. 이 COMMAND는 각각의 TABLE 에 대하여 실행 되어야 한다. ANALYZE COMMAND에는 DATA DICTIONARY저장되는 현황의 종류를 결정 할 수 있는 몇개의 OPTION이 있다.

< ANALYZE COMMAND의 예 : OPTION포함 >

SQL>LIB에서 실행한다.

1. TABLE 대한 ESTIMATED STATISTICS의 생성
: 1024 개의 RECORD를 RANDOM하게 SAMPLING한다.

```
SQL> ANALYZE TABLE TABLE_NAME ESTIMATE STATISTICS;
```

: DATA의 SAMPLE를 위한 2가지의 OPTION이 있다.

```
SQL> ANALYZE TABLE TABLE_NAME ESTIMATE STATISTICS SAMPLE 5000 ROWS;
```

```
SQL> ANALYZE TABLE TABLE_NAME ESTIMATE STATISTICS SAMPLE 25 PERCENT;
```

2. TABLE에 대한 COMPLETE STATISTICS의 생성

: TABLE의 모든 ROW에 대한 TABLE과 INDEX의 자료현황을 생성한다.

```
SQL> ANALYZE TABLE TABLE_NAME COMPUTE STATISTICS;
```

3. TABLE에 대한 STATISTICS의 DELETE

: TABLE과 INDEX의 자료현황을 삭제한다.

```
SQL> ANALYZE TABLE TABLE_NAME DELETE STATISTICS;
```

4. CHAINED_ROWS TABLE에 CHAINED ROW를 등록

: UTLCHAIN.SQL SCRIPT(CHAINED_ROWS TABLE을 생성한다)를 아래의 COMMAND를 실행 하기전에 수행하여야 한다. CHAINED_ROWS TABLE에는 CHAINED ROW의 ROWID를 포함한 각각의 ROW들이 INSERT 된다. 이 ROWID를 이용하여 원래의 TABLE로 부터 CHAINED된 ROW들을 DELETE한 후, CHAINED_ROWS TABLE의 ROW를 이용하여 새로운 ROW를 생성하여 CHAIN을 연결 수 있다.

```
SQL> ANALYZE TABLE TABLE_NAME LIST CHAINED ROWS;
```

5. TABLE과 INDEX에 CORRUPTION이 있는지를 VALIDATE

: 만약 CORRUPTION이 존재한다면, 다음 COMMAND를 실행 하였을때 'BAD AND RECREATE' MESSAGE가 표시된다.

```
SQL> ANALYZE TABLE TABLE_NAME VALID STRUCTURE CASCADE;
```

ANALYZE COMMAND가 실행되면, BADDED SQL AREA에 존재하면서 참조된 OBJECT 들은 다음에 새로운 현황으로 참조되기 위해 BADDED SQL AREA에서 삭제된다.

ANALYZE COMMAND는 위에서와 같이 한 TABLE씩에 대하여 실행될 수도 있으나, DBACL에서 공하는 PACKAGE PROCEDURE인 DBMS_UTILITY.ANALYZE_SCHEMA를 호출하여 한 USER의 전체 OBJECT에 대해서 전체적으로 실행 할 수도 있다.

```
SQL> BEGIN
```

```
2   DBMS_UTILITY.ANALYZE_SCHEMA('USER_NAME', 'COMPUTE');
```

```
3   END;
```

```
4   /
```

ANALYZE되지 않은 DATA DICTIONARY TABLE에 대해서는 RULE BASED OPTIMIZER가 사용된다. COST BASED OPTIMIZER를 사용하게 하기 위해서는, 위의 PACKAGE PROCEDURE를 SYS의 SCHEMA에 대해 실행 되어져야 한다.

THE DBACLE MONITOR AND DBACLE UTILITIES

DBACLE MONITOR는 SQLDBA UTILITY상 서 가능하다. MONITOR UTILITY는 DBACLE의 VIEW이나 여러

개의 VIEW를 JOIN하여 정보를 가져온다.

INDEXING STRATEGIES AND REVIEW

TABLE에 대한 INDEXING은 SYSTEM PERFORMANCE에 중요한 KEY가 되며, INDEXING의 설계에는 주의를 요한다. 한 TABLE에 대하여 INDEX의 갯수를 3개까지로 제한하는 것이 좋으며, 가능한한 5개 이상은 만들지 않는 것이 좋다.

3가지 TYPE의 TABLE에 대하여 각기 일관적인 INDEX기법에 대하여 다음에서 살펴본다.

1. 만약 TABLE이 QUERY만을 위한 것이라면, 위에서와 경우보다 많은 수의 INDEX를 생성하여도 좋다.
2. 만약 TABLE에 대한 QUERY는 적고, INSERT나 UPDATE가 주로 이루어 진다면 INDEX를 적게 가져가도록 권장한다. (DATA의 분포는 TABLE의 DATA분포만 아니라 TABLE에 대한 INDEX까지 UPDATE가 이루어져야 하기 때문이다)
3. 만약 TABLE이 INSERT, UPDATE, QUERY가 모두 필요하다면, INDEXING에 매우 주의를 요하나, INDEX는 꼭 필요하다.

EXPLAIN PLAN

EXPLAIN PLAN COMMAND는 ORACLE OPTIMIZER가 SQL문의 실행에 사용할 EXECUTION PLAN을 개발자에게 제공해준다. 이 COMMAND는 SQL문장이 실제로 실행 되지는 않고, 실행 PLAN이나 OUTLINE만이 ORACLE TABLE에 저장되므로 SQL문장의 PERFORMANCE를 높일는데 매우 도움이된다. EXPLAIN PLAN COMMAND를 실행하기 전에 COMMAND를 수행할 ORACLE USER에 대해 UTLPAN.SQL이 먼저 수행되어져야 한다. 이 SCRIPT는 PLAN_TABLE 생성한다.

```
BQL> EXPLAIN PLAN
2 SET STATEMENT_ID = 'QUERY1' FOR
3 SELECT CUST_ID, NAME FROM CUST_INFO WHERE CUST_ID = '100';
```

이 TABLE의 자료를 조회하기 위해서는 SQL문을 사용하여야 한다.

```
BQL> SELECT Operation, Options, Object_Name, Id, Parent_Id, Position
2 FROM PLAN_TABLE
3 WHERE STATEMENT_ID = 'QUERY1'
4 ORDER BY Id;
```

< OUTPUT >

OPERATION	OPT	OBJECT_NAME	ID	P_ID	POS
SELECT STATEMENT					0
TABLE ACCESS	FULL	CUST_INFO	1	0	

```
BQL> SELECT Lpad(' ', 2*(Level-1))||
2 Operation||' '||
3 Options||' '||
4 Object_Name||' '||
5 Decoln(Id,0,'Cost = '||Position) 'Query Plan'
```

```

6 FROM PLAN_TABLE
7 START WITH ID = 0
8 AND STATEMENT_ID = 'QUERY1'
9 CONNECT BY PRIOR_ID = PARENT_ID
10 AND STATEMENT_ID = 'QUERY1';

```

< OUTPUT >

QUERY PLAN

```

-----
SELECT STATEMENT      Cost =
TABLE ACCESS          FULL  CUST_INFO

```

SAMPLE: DB에 IMAGE FILE을 저장

Bulletin no : 10111

- * 이미지 파일의 Type에는 여러가지 형태가 있다.
예를 들면 BMP, PCX, TIF, ...
이 중에서도 압축 효율이 높은 File의 형태를 선택하는것이 좋다. 가령, BMP Type보다는 PCX Format이 거의 1/4정도로 압축을 해주므로 보다 효율적이다.
- * 일단 PC의 Window의 Paintbrush 또는 Image 처리하는 Tool로써 이미지 파일을 적절한 형태로 처리하여 File로 만들어서 Unix Server로 옮긴다.
- * Long Data를 처리하기 위해서는 File Size를 알아야하며, 추후 File의 Size로써 처리해야 하는 일이 있다면, Table에 File의 Size도 저장하는 것이 효율적이다.
- * 아래의 프로그램은 Image File을 읽어서 Table에 저장하는 내용이다.

Include

```
typedef char bitmap[58282];          /* file size */
EXEC SQL INCLUDE INCLUDE SECTION:
    VARCHAR username[20];
    VARCHAR password[20];
EXEC SQL TYPE bitmap IS LONG RAW(58282) DEFERRABLE;
    bitmap buffer;
EXEC SQL END INCLUDE SECTION;
EXEC SQL INCLUDE sqlca;

void sqlerror();                      /* handling unrecoverable errors */
FILE *fp;
main()
{
    strcpy(username, "SCOTT");
    username, len = strlen(username, app);
    strcpy(password, "TIGER");
    password, len = strlen(password, app);

EXEC SQL WHENEVER SQLERROR GO sqlerror();
EXEC SQL CONNECT :username IDENTIFIED BY :password;
EXEC SQL WHENEVER SQLERROR GO sqlerror();

    fp=fopen("xxx.bmp", "r");
    fread(&buffer, 1, 58282, fp);

EXEC SQL INSERT INTO IMAGE VALUES (20, &buffer);
EXEC SQL COMMIT WORK RELEASE;
```

```

}
void sqlserver()
{
    EXEC SQL WHENEVER SQLERROR CONTINUE;
    printf("NODATABASE error detected:\n");
    printf("%s\n", sqlca.sqlwarn.sqlwarn);
    EXEC SQL ROLLBACK WORK RELEASE;
    exit(1);
}

```

DIGITAL UNIX 4.0/4.0A 사 용 시 PRO*C PRECOMPI LE 주 의 사 항

Bulletin no : 10388

최근 Digital unix 4.0/4.0a 를 사용하는 곳이 증가하고 있으며,이러한 장미에서 Pro*C로 precompile 하는 경우에는 몇가지 주의사항이 있습니다.

- 몇가지 precompile option을 정의하여야만 합니다.

```
code=unix_c
define=bit84
define=KMPDIR_BDIBCE
define=cd1
|
make
```

- * 위의 사항들을 '\$ORACLE_HOME/precomp/adal/precfg.h'에 정의해 놓으면 모든 사용자에게 자동 적으로 적용됩니다.

(precfg.h 는 normal text file로 이음으로 편집하시면 됩니다.)

- 한가지 더 주의 하실 점은 C++ 를 사용하는 경우에는 precomp (or partial)로 하여야만 합니다.

상기 사항들에 대한 자세한 설명은 'Programmer's Guide to the Oracle Pro*C/C++ Precompiler Release 2.2' 의 7장을 참조하시기 바랍니다.

DOS, WINDOWS에서의 PRO*C와 C COMPILER 와의 관계

Bulletin no : 10110

1. Oracle 7의 Application(Pro*C)은 Oracle 8에서 사용할 수 없다.
2. Pro*C를 Compile하면 오로지 C code로 생성되며, C++ Code로는 생성할 수 없다.
3. Pro*C 1.5.8 이후의 Version은 MS C7.0을 지원하지 않는다.
(MS C7.0은 Pro*C 1.4.8.0.0에서 지원한다.)
4. Forms 4.0, Report 2.0, Graphics 2.0 또는 상위 Version의 User Exit Program은 Pro*C 1.5에서
만 지원한다. 이러한 USER EXIT는 Visual C++ 1.0 또는 1.0 Version과 Link되거나 Compile되며,
Borland Compiler는 지원하지 않는다.
5. Pro*C 1.5.8의 RDB Application은 BQLPHE를 사용하여 RDB Protected Mode에서 만든다. Pro*C
1.5.10의 Application은 MS Windows 환경에서 만든다.
6. Pro*C 1.5.10은 RDB와 Windows 2개의 Product로 구분된다. Windows Version은 MS-Windows하에
서 DLL을 Interface하며, RDB Version은 이전 Version처럼 RDB Command Line을 Interface 한다.

DYNAMIC SQL METHOD1(CREATE TABLE)

Bulletin no : 10341

```
/*=====*/
Sample Program : Dynamic SQL Method 1( Create Table)
/*=====*/
```

#include

typedef char ucclx[80];

EXEC SQL BEGIN DECLARE SECTION;

EXEC SQL TYPE ucclx IS VARCHAR(80) REFERENCE;

VARCHAR username[20];

VARCHAR password[20];

ucclx sqlstat;

ucclx tablename;

EXEC SQL END DECLARE SECTION;

EXEC SQL INCLUDE sqlca;

void sqlerror(); /* handle unrecoverable errors */

void connect();

main() {

strcpy(username, "SCOTT"); /* copy the username */

username, len = strlen(username, app);

strcpy(password, "TIGER"); /* copy the password */

password, len = strlen(password, app);

EXEC SQL WHENEVER SQLERROR DO sqlerror();

EXEC SQL CONNECT :username IDENTIFIED BY :password;

strcpy(tabname, "TEST_TAB");

strcpy(sqlstat, "create table ");

strcat(sqlstat, tablename);

strcat(sqlstat, " (as number)");

EXEC SQL EXECUTE IMMEDIATE :sqlstat;

EXEC SQL COMMIT;

EXEC SQL COMMIT WORK RELEASE; /* log off DBACLE */

exit(0);

}

```

void sqlserver()
{
    EXEC SQL WHENEVER SQLERROR CONTINUE;
    printf("NODATABASE error detected:\n");
    printf("Nod .70s\n", sqlca, sqlwarn, sqlserver);
    EXEC SQL ROLLBACK RELEASE;
    exit(1);
}

```


SAMPLE: DYNAMIC SQL METHOD2 SAMPLE(INSERT)

Bulletin no : 10182

```
/******
```

```
Dynamic SQL Method 2 : INSERT
```

```
This program uses dynamic SQL Method 2 to insert a row into  
the EMP table.
```

```
*****
```

```
Define USERNAME "SCOTT"
```

```
Define PASSWORD "TIGER"
```

```
Include
```

```
Include
```

```
EXEC SQL INCLUDE sqlca;
```

```
EXEC SQL BEGIN DECLARE SECTION:
```

```
char *username = USERNAME;
```

```
char *password = PASSWORD;
```

```
char sqlstat[2][80];
```

```
int sqlca = 1234;
```

```
int deptno = 87;
```

```
char temp[180];
```

```
EXEC SQL END DECLARE SECTION;
```

```
main()
```

```
{
```

```
EXEC SQL WHENEVER SQLERROR GOTO sqlerror;
```

```
EXEC SQL CONNECT :username IDENTIFIED BY :password;
```

```
puts((char *) "%Connected to DBACLE, %");
```

```
sqlstat[sqlstat[1],
```

```
    "INSERT INTO EMP (:col, :col)", "empno", "deptno");
```

```
sqlstat[sqlstat[2], "VALUES (:val, :val)", "1", "2");
```

```
strcpy(temp, sqlstat[1]);
```

```
strcat(temp, sqlstat[2]);
```

```
/* Display the SQL statement and its current input host  
variables. */
```

```

printf(" v1 = %d, v2 = %d\n", mparam, idparam);

EXEC SQL PREPARE B FROM :stmt;

EXEC SQL EXECUTE B USING :mparam, :idparam;

EXEC SQL COMMIT RELEASE;

puts((char *) "%Have a good day!\n");

exit(0);

sqlerror:
printf("%d,%d\n", sqlca, sqlwarn, sqlwarn1,
        sqlca, sqlwarn, sqlwarn2);
}

```

SAMPLE: DYNAMIC SQL METHOD2 SAMPLE(UPDATE)

Bulletin no : 10183

/*****

Dynamic SQL Method 2 : UPDATE

This program uses dynamic SQL Method 2 to update a record on EMP table

*****/

Define USERNAME "SCOTT"

Define PASSWORD "TIGER"

Include

Include

EXEC SQL INCLUDE sqlca;

EXEC SQL BEGIN DECLARE SECTION;

char *username = USERNAME;

char *password = PASSWORD;

varchar sqlstat(80);

int sqlno = 1234;

int sqltad = 30;

EXEC SQL END DECLARE SECTION;

main()

{

EXEC SQL WHENEVER SQLERROR GOTO sqlerror;

EXEC SQL CONNECT username IDENTIFIED BY :password;

putc((char *) "%Connected to ORACLE.\n");

sqlstat(sqlstat,sqlno,
"UPDATE emp SET sal = :sal WHERE empno = :empno",
"empno", "1", "deptno", "2");

sqlstat, sqltad(sqlstat,sqlno);

EXEC SQL PREPARE S FROM :sqlstat;

EXEC SQL EXECUTE S USING :empno, :deptno;

EXEC SQL COMMIT RELEASE;

```

        print(color * "\nHave a good day!\n");
    exit(0);

    cg1screen;

    printf("\n\n", cg1cs, cg1screen, cg1screen1,
           cg1cs, cg1screen, cg1screen);
}

```

Publication no. : 10205

- 499 -

```

EXEC SQL WHENEVER NOT FOUND GOTO notfound;
/* Loop until the NOT FOUND condition is detected. */
while (TRUE)
{
EXEC SQL FETCH C INTO :aaaaa;
/* Null-terminate the array before output. */
aaaaa,aaa[aaaaa,100] = '\0';
puts((char *) aaaaa,aaa);
}
notfound:
printf("Query returned id %s\n", sqlca.sqlword[2],
      (sqlca.sqlword[2] == 1) ? '\0' : 'x');
EXEC SQL CLOSE C;
/* Commit any pending changes and disconnect from
   DBACLE. */
EXEC SQL COMMIT RELEASE;
puts((char *) "Have a good day\n");
exit(0);
sqlerror:
printf("%s\n", sqlca.sqlword,sqlword1,
      sqlca.sqlword,sqlword2);
EXEC SQL WHENEVER SQLERROR CONTINUE;
/* Release resources associated with the cursor. */
EXEC SQL CLOSE C;
/* Roll back any pending changes and disconnect from
   DBACLE. */
EXEC SQL ROLLBACK RELEASE;
exit(1);
}

```

FUTURE TCP 와 SQL*NET V2 사용 시 PERFORMANCE

Bulletin no : 10055

Subject: Re: Know How.. [2. Future/tcp 사용시]

주변에 항목의 경우 future/tcp 에서 Named Server를 잘못설정하면 Host의 Named Server를 다 찾고서, 일으면 Client의 host file를 뒤져서 자기 host alias로 처리하는 경우가 있었습니다. (1-2초 걸리던 connection time이 12초까지 걸렸음)

정남은행에도 BLIP으로 데스크를 하는데 BQNet을 통하면 엄청 느다고 계속 불평 해서 가서 보니 BQNet의 문제가 아니라 future/tcp의 Client host file과 Named Server의 Host file의 불일치 때문으로 판명난 적이 있었습니다.

이런경우,

1. Client에서 future/tcp의 install program으로 Named Server를 일깨는지
2. Host와 Client의 Host file를 일치 시키면 되었습니다.

sqlnet v1 & v2 test 지원 중 발견된 사항입니다.

1.

아직도 많은 사용자가 sqlnet v2 가 잘 효과가 없다고 생각합니다. 이런 user는 주로 NTB 환경으로 setting을 하였으며 shared server와 dispatcher를 한개씩 띄우고 사용하고 있었습니다.

근본적으로 oracle의 nls는 FIFO 방식으로 BQL을 처리하기 때문에 server가 하나만 떠 가지고는 당연히 sqlnet v1의 dedicated server 보다 느리게 처리됩니다.

따라서 nls 환경으로 setup 할 때에는 최소한 shared server와 dispatcher를 2개 이상씩 (3개이상 권장) 띄워서 사용할 수 있도록 하 바랍니다. 일부에서는 이런이유로 oracle nls 기능이 절름발이 없다 라고 오해하며 열사람에게도 전달하고 있습니다.

2.

future tcp/ip 사용시에 tnsnames.ora에 host name 을 사용하면 sqlnet v2 가 v1보다 3배 정도 더 느리게 되는데 이때에는 ip address number를 직접사용하면 됩니다.

참석전환 pdx test의 connection time : sqlnet v1 --> 8-8초

sqlnet v2 host name --> 25초

sqlnet v2 ip number --> 2-3초

3.

sqlplus 의 사용 : 생각보다 참 편한 tool 입니다.

sql parsing time, sql execution time, connection test & 결과시간 확인, long raw field의 long 실행 저장 size, db 내의 image data를 읽어내서 doc의 file로 write 하는 등의 기능을 제공하고 floppy 1장에 속 들어가는 size이므로 고객지원이나 sqlnet v1 & v2 test시에 적극 활용하시기 바랍니다.

PRO*C 에서 HOLD__CURSOR,RELEASE CURSOR 의 의미

Bulletin no : 10184

=====

The HOLD_CURSOR and RELEASE_CURSOR Options

=====

What are the Oracle Precompiler options **HOLD_CURSOR** and **RELEASE_CURSOR** and how do they affect the performance of a PCC program?

First of all, these options apply to implicit and explicit cursors in all precompiled languages except Pro*Ada. They apply **ONLY** to implicit cursors associated with **INSERT**, **UPDATE**, **DELETE**, or a single row **SELECT** in Pro*Ada. An explicit cursor in a program is one explicitly created with **EXEC SQL DECLARE c1 CURSOR FOR...**

Following will be a discussion of what these options do internally and how changing them affects program performance.

What exactly do we mean by **CURSOR**? Unfortunately, we mean two different things:

1. The program cursor - A data structure associated with a SQL statement.

A program cursor is declared for each SQL statement that the precompiler finds in your program. For the statement:

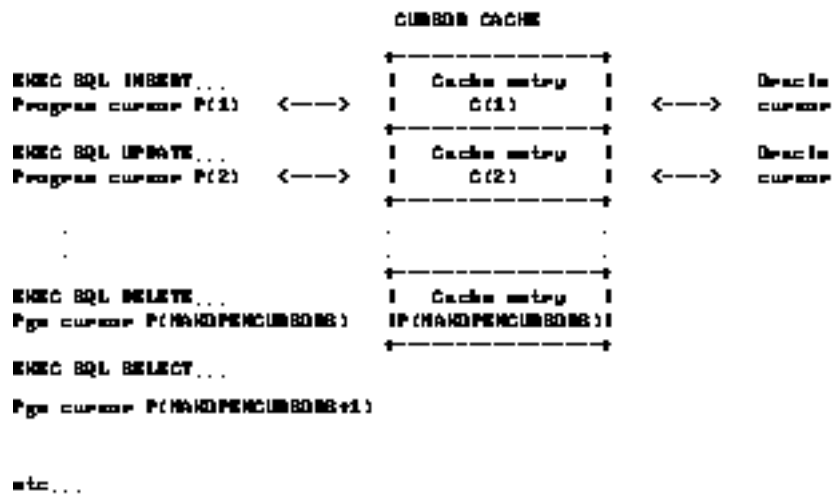
```
EXEC SQL DECLARE BEL_EMP_CURS CURSOR FOR...
EXEC SQL INSERT...
```

PCC will declare two program cursors, say c1 and c2.

2. The Oracle cursor (also called the context area) - The work area created dynamically at run time; this area contains the parsed statement, the addresses of the host variables, and other information necessary to execute the SQL statement.

These two cursors are linked together via the cursor cache. The initial size of the cursor cache is determined by the **MAXOPENCURSORS** option.

The following diagram illustrates the relationship described above after an insert and an update have been executed in your program:



How are the **HOLD_CURSOR** and **RELEASE_CURSOR** options related to this view?

The **HOLD_CURSOR** option deals with the link between the program cursor and its cache entry.

The **RELEASE_CURSOR** option deals with the link between the Oracle cursor and the cache entry.

For BQL statements that are **FREQUENTLY RE-EXECUTED**, the bottom line is this: If you want to maximize performance, make sure these BQL statements stay "glued" to their respective Oracle cursor.

What does it mean when a BQL statement is "glued" to its Oracle cursor? It means that both links between the BQL statement and its Oracle cursor are made permanent.

Why would you want to keep a statement "glued" to its context area?

Because the context area contains the parsed statement and other information necessary to execute the statement, such as the addresses of the host variables. Maintaining access to this information makes subsequent execution of the statement much faster.

How do you "glue" a statement to a cache entry? By correct use of the **HOLD_CURSOR** and **RELEASE_CURSOR** options via the PGC command line or tables with **EXEC ORACLE OPTION(...)**.

For instance, with **HOLD_CURSOR=YES** as the Oracle option, a cache entry cannot be flagged for reuse. This has important implications. If all cache entries have been used up and a new cache entry is needed for a new BQL statement such that the number of cache entries would now exceed the number specified in **NANDPENCURBOS**, Oracle will use the first cache entry marked reusable.

For example, in the above diagram, if the cache entry C(1) is marked reusable, and the program is about to execute the **EXEC SQL SELECT...** (program cursor P(NANDPENCURBOS+1), and the number of cache entries in use already equals

MAXOPENCURSORS, cache entry C(1) and its Oracle cursor will now be linked to the select statement. A subsequent execution of the insert statement would require precompiling a cache entry and its Oracle cursor from another SQL statement and performing a reparse.

Correspondingly, with the default RELEASE_CURSOR=NO as the Oracle option, the link between the cache entry and the Oracle cursor (the context area) is maintained after the statement is executed so that the parsed statement and, more importantly, the allocated memory stay available.

The freeing up of this memory by RELEASE_CURSOR=YES means that the next statement that gets linked to this cache entry will require an expensive reallocation of memory in addition to a reparse. Ugh! Why would anybody want RELEASE_CURSOR=YES? We will see later on.

Program cursor - - - - -	[Cursor cache entry]	- - - - - Oracle
for SQL statement		cursor
	^	^
	HOLD_CURSOR=YES	RELEASE_CURSOR=NO
program cursor is permanently		cache entry maintains the
linked to its cache entry.		address of its context area.

So the HOLD_CURSOR option is intimately tied to the MAXOPENCURSORS option. What exactly is the MAXOPENCURSORS option? First of all, MAX OPENCURSORS is a misnomer. It should more appropriately be called INITIAL_CURSOR_CACHE_SIZE. (Okay, so it's a mouthful.) Anyway, if all cursor cache entries are currently marked "not reusable" either because of the HOLD_CURSOR option or because the associated statement is currently being executed (an explicitly opened cursor is still being fetched on and hasn't been closed yet), then a request for a new cursor will actually result in the extension of the cursor cache at runtime (i.e. if MAXOPENCURSORS=10, and all 10 entries are active, then an 11th will be created). Just letting the precompiler reuse the oldest cache entry won't always work, as the following example illustrates: Imagine the case where the user has ten explicitly declared cursors opened, and wants to execute an eleventh. If the program actually reuses the oldest program cursor, the user would lose his current position in the first cursor and would not be able to fetch from it anymore.

By the way, if an eleventh cache entry is created, when that cursor is closed the eleventh entry is not removed. Getting MAXOPENCURSORS low saves memory, but causes potentially expensive dynamic allocations of new cache entries if they're needed. Getting it high assures quick execution, but may use more memory than necessary.

What if a statement is not executed repeatedly in a program? Then you could go with the other options: HOLD_CURSOR=NO and RELEASE_CURSOR=YES.

With the HOLD_CURSOR=NO option, the link between a program cursor and its cache entry is not permanent. The cache entry is automatically marked reusable in case it is needed. With the RELEASE_CURSOR=YES option, the Oracle cursor (the context area) is automatically freed and the parsed statement lost. A reason you might use this option is if you are limited by the number of Oracle cursors (MAXOPENCURSORS) at your site due to memory issues. You may want to incur

the cost of reallocating memory and reparsing in order to manage memory more effectively.

An advantage of setting `RELEASE_CURSORS=YES` is that until the link between the cache entry and the Oracle cursor (context area) is removed, Oracle keeps parse locks on any tables referenced in the SQL statement. These parse locks prevent other users and you from ALTERing or DROPping the tables (does DBA-0057 sound familiar?). Also, in Version5, it will free up the read-consistent image of the referenced tables stored in Oracle's Before Image file.

What do we mean when we say that `RELEASE_CURSORS=YES` takes precedence over `HOLD_CURSORS=YES`? With `RELEASE_CURSORS=YES`, the link between the Oracle cursor and the cache entry is cut and the Oracle cursor is freed (closed), so even if your program cursor is permanently linked to the cache entry because `HOLD_CURSORS=YES`, you will still have to reallocate memory and reparse the statement. So subsequent executions of a statement don't benefit from the `HOLD_CURSORS=YES` option because `RELEASE_CURSORS=YES`.

For programmers experienced with OCI, here's the OCI equivalent of what's happening:

Define `MAXOPENCURSORS 5`

```
char    *sql_text[10];
cur_def cursor(MAXOPENCURSORS);

open(cursor[0],...);
ocg13(cursor[0],...,sql_text[0],...);
```

An example of a "cache entry" being linked to another SQL statement later on in the program is as follows:

```
ocg13(cursor[0],...,sql_text[5],...);
```

I am forced to reuse one of my "cache entries" to execute the sixth SQL statement.

An example of a context area being freed is:

```
oclose(cursor[0]);
```

Reusing `cursor[0]` would require another `open()` and another `ocg13()`—another dynamic allocation of memory and another reparse.

Conclusion

As a programmer, you will get the most from these options by using them selectively inline rather than specifying them as options at precompile time.

SQL-2101(ORA-2101): INCONSISTENT CURSOR C ACHE (UCE/CUC MISMATCH)

Bulletin no : 10174

Causes of SQL-2101

'SQL-2101: inconsistent cursor cache (UCE/CUC mismatch)' can be a very confusing and frustrating error to receive while running a Pro*C program. To start with, since the error is generated at runtime, the most logical place to begin looking for an explanation is in the Oracle Error Messages and Codes Manual. However, there is no listing for an ORA-2101. It looks like it's time to call Oracle Support:

there's another undocumented error.

But there is a reason that this error is not listed among the ORA errors in the Error Messages and Codes Manual. It is because Oracle does not generate this error. Rather, it originates from the BQLLIB runtime libraries used by precompiler programs. This error is documented as BQL-2101 in the Programmer's Guide to the Oracle Precompiler V1.3 and V1.4 in Appendix A, in the Programmer's Guide to the Oracle Precompiler V1.5 in Appendix B, and in the Oracle7 Server Messages and Codes Manual on page 4-28. In the Programmer's Guide to the Oracle Pro*C Precompiler Release 2.0, it is documented in Appendix C as BQL-2101.

However, the three word message 'inconsistent cursor cache' is more useful than the full paragraph cause and action given in the manuals. The true meaning of this error is that the cursor cache entry for the statement being executed has been corrupted. To understand why this corruption can occur, it is necessary to review the explanation of the cursor cache in the Precompiler Manual. The important fact here is that the cursor cache data structure used to manipulate every SQL statement is located in memory allocated by BQLLIB in the program's data space. Given this, the most likely culprit for corruption of a program's data space is the program itself.

The following is a list of program errors which can possibly corrupt the cursor cache:

- o Using a pointer returned by malloc, calloc, or realloc for more space than was actually allocated. For example:

```
char *p;

if (( p = malloc(5) ) != NULL)
    strcpy (p, '12345678');
```

Here, p points to a block of 5 bytes but a string of 8 characters was copied into it.

The extra 3 bytes can cause corruption in the cursor cache. Note that SELECTing or FETCHing into space allocated by malloc, calloc, realloc can also cause corruption if the selected data is larger than the space allocated.

- a Be sure to check the return value from calls to malloc, calloc, realloc, and free. If a NULL value is returned, then it means that the call did not complete successfully and the pointer should not be used.
- a The pointer passed to free must be the same as one that was returned from a call to malloc, calloc, or realloc. If it has been modified, then the free call may corrupt program space.
- a Check the usage of array indices. Make sure that they do not exceed the value used when declaring the array.

A temporary solution to the symptom (getting the BQL-2101) is to use the precompiler option RELEASE_CURSOR=YES. Many times this will get rid of the error, but the fact remains that the corruption is still happening. The full effect of this option is described in document#:

B2B10.087. In this case, it frees the corrupted memory in the cursor as soon as the cursor (implicit or explicit) is closed, before BQLLIB will detect the corruption. Tracking down stray pointers or checking for overwritten memory can be the most difficult C programming errors to debug. However, the BQL-2101 has repeatedly been proven to be caused by memory corruption due the above types of errors.

PRO*C ON MS-DOS AND MS-WINDOWS PLATFORM

S

Bulletin no : 10004

1. Support of Oracle 8.0 indicates that your applications will work against an Oracle 8.0 RDBMS (if Oracle 7 Server functionality is used, i.e., only VB type calls are made). However, the reverse is not true. Oracle 7 applications cannot be run against Oracle 8.0.
2. Some of the C (or C++) compilers listed above are capable of generating C++ code as straight C.
You must put all C++ code in one file and all PRO*C/C code in another, then link these two modules.
PRO*C can only generate straight C code and not C++ code.
Include of C++ code in PRO*C modules is not supported.
3. Pro*C 1.5.8 and later do not support MS C7.0
4. User Exit support for Oracle Forms 4.0, Oracle Reports 2.0 and Oracle Graphics 2.0 are provided through Pro*C 1.5 only.
These User Exits may be compiled and linked with Visual C++ 1.0 or 1.5 only (you cannot use the Borland compiler)
5. Pro*C 1.5.8 applications are built entirely in DOS protected mode and hence require the use of BQLPME.
Pro*C 1.5.10 applications are built in the MS-Windows environment.
6. Pro*C 1.5.10 onwards is now two separate products, one for Windows and one for DOS.
The Windows version runs completely under MS-Windows and has a GUI (no command line) interface.
The DOS version will use the same DOS command line interface as previous versions.
Pro*C 1.5.10 is not yet available on DOS.
7. If you have any questions or concerns about future releases of Pro*C please contact your Oracle representative.

Oracle Customer Support does not have information on future releases.

정판용 User	Appx For	Create In	DBMS User	Supported Compliance	User Exit Support	OCI supp	PL/SQL supp
1,3	DBS	DBS	B	MBC B,0	F30 for DBS	Yes	1,x
1,5,B	DBS	DBS	7	MBC B,0 UC++ 1.0/1.5	None	Yes	2,x
	MIN	DBS	7	UC++ 1.0/1.5	DP4,0,12 DB2,0,13 DC2,0,8		
1,5,10	MIN	MIN	7	UC++1.0/1.5	DP4,0,13 DB2,0,14 DC2,0,10	Yes	2,x
	MIN	MIN	7	Revised C4,0	None	Yes	2,x

기타 자세한 내용은 Support News Volume 3 Issue 3 1994를 참조 하십시오.

ORA-1002 FETCH OUT OF SEQUENCE IN PROC

Bulletin no : 10287

*** ORA-1002 : FETCH OUT OF SEQUENCE ***

ORA-1002 error가 의미하는 것은 cursor가 더이상 필요하지 않은 cursor로부터 fetch를 하려고 하기 때문이다.

이러한 경우의 가장 흔한 경우를 다음 두가지로 설명하겠다.

1) ORA-1002 등과 같이 NO DATA FOUND를 return하는 fetch작업을 수행할때

: 즉 fetch는 항상 현재의 row를 return하며, 마지막 row를 return한 후에 더 retrieve할 data가 있는지 check하기 위해 한번 더 fetch를 수행하는데 이때 특정 exit구문으로 loop를 종료시키지 않으면 ORA-1002 error를 return 한다.

```
DECLARE
EMP_NAME EMP.EMPNAME%TYPE;
CURSOR C1 IS SELECT EMPNAME FROM EMP;
BEGIN
OPEN C1;
LOOP
    FETCH C1 INTO EMP_NAME;
END LOOP;
CLOSE C1;
END;
```

2) SELECT FOR UPDATE를 가진 cursor 의 fetch작업내에 commit이 있는 경우

```
:
```

```
DECLARE
EMP_NAME EMP.EMPNAME%TYPE;
CURSOR C1 IS SELECT EMPNAME FROM EMP
FOR UPDATE OF SAL;
BEGIN
OPEN C1;
LOOP
    FETCH C1 INTO EMP_NAME;
    UPDATE EMP SET SAL = 50;
    -- cannot do this commit in version 8
    COMMIT;
END LOOP;
```



```
CREATE CL:
END;
```

database consistency를 유지하기 위해 user는 자신이 update/delete 할 row를 select시점에서 lock를 걸어주는 경우가 있는데 commit이나 rollback을 만나기 이전까지는 lock이 걸려있다. 위 예에서 select for update 로 cursor가 open되는 시점에 lock를 걸어 놓지만 fetch를 수행중 commit을 만나 lock이 release된다. 이 때 cursor는 더이상 종료하지 않으며 fetch out of sequence를 return한다.

해결책은 commit을 fetch loop의 하향측으로 빼거나 select for update문을 사용하지 말아야 한다.

ORA-1405 처리 방법

Bulletin no : 10079

ORA-1405 는 ERROR 가 아니고 WARNING MESSAGE 이다. 따라서 프로그램내에서 ERROR CHECK 하는 작업에따라 PRECOMPILER OPTION 을 상이하게 설정 ERROR 를 처리 할 수있다.

1. SQLCODE 로 ERROR CHECK 하는경우

```
/* connection 이 null 인경우 */
EXEC SQL SELECT EMPNO, SAL, COMM
        INTO emp_name, salary, commission
        FROM EMP
        WHERE EMPNO = :emp_number;
If (sqlca.sqlcode != 0 ) exit(0);
```

위 경우 sqlca.sqlcode 가 -1405 이므로 exit 하게된다. 따라서 값을 정상적으로 FETCH 를 하였으나 ERROR 로 처리되게 된다. 위의 프로그램을 변경하지않고 처리 할 수 있는방법은 dbms-v8 를 PRECOMPILER OPTION 에 추가해준다. dbms-v8 로 SETTING 할경우는 HOST 변수에 NULL 이 RETURN 되더라도 sqlca.sqlcode 는 0 이 된다.

2. NOT FOUND 로 CHECK 하는경우

```
EXEC SQL WHENEVER NOT FOUND GO break;
EXEC SQL SELECT EMPNO, SAL, COMM
        INTO emp_name, salary, commission
        FROM EMP
        WHERE EMPNO = :emp_number;
```

FOUND 인경우 1403 이 RETURN 되므로 위의 SQL 문은 정상적으로 처리가 된다.

ORA-3113,3114 의 발생원인 및 진단 방법

Bulletin no : 10120

ORA-3113은 가장 포괄적인 오류입니다. 원초적인 의미로는 예기치 않은 이종 때문에 통신이 부정확 상태를 의미합니다. 이 오류에는 대개 ORA-3114 'not connected to ORACLE'이 동반됩니다.

원인을 살펴보면

1. 가장 많은 원인은 서버의 Oracle 속도 프로세스가 예기치 않게 종료된 경우입니다. 따라서 수행 중에 갑자기 ORA-3113과 3114가 발생했다면, 우선 서버의 alert.log를 점검하여 다른 Oracle 오류가 발생했는지 알아보십시오.

<< alert.log >>

서버가 UNIX 민정주 \$ORACLE_HOME/rdbms/log/alert_.log 파일에 ORA-3113 에러가 발생했던 시점에서 다른 에러가 발생했는지 점검 합니다. 특히 ORA-800[1,1]이 발생했으면 에러 내용을 Oracle Technical Support Center로 연락 하십시오.

2. ORA-3113의 원인 중 그 다음으로 많은 것은 BQL*NET 드라이버가 Unix의 ORACLE 실행 파일과 연결 되지 않아 발생한 경우입니다. 연결을 공식적으로 수신하고 그것을 ORACLE 속도 프로세스에 전달한다 해도, 속도 프로세스는 서버랑질을 모르기 때문에 어떤 랑립으로도 응답하지 못할 수 있습니다. 그러므로 클라이언트는 연결순간에 ORA-3113을 보게 됩니다.

3. 세련제로 많은 원인은 서버쪽의 기계 손상이나 네트워크 고장입니다.

4. 자주 있는 것은 아니지만 같은 네트워크에서 두 서버가 같은 노드 이름을 가질 때에도 이 오류가 발생합니다.

5. ORA-3113은 포켓팅 카드의 공통 BMM 크기가 18BB가 아니라 8BB로 설정 되었음을 나타내기도 합니다. 포켓팅을 사용중이라면 공통 버크 크기를 점검하고 키워 보십시오.

6. ORA-3113은 INIT.ORA 매개변수 CONTEXT_AREA와 CONTEXT_INCR이 40BB이라는 값으로 설정된 경우에도 발생합니다. 그럴때는 값을 8182로 키우면 ORA-3113이 해소됩니다.

이상 말한 모든 원인은 결국 클라이언트가 서버로부터 어떤 정보를 믿으며 갔다가 거기서 더 이상 연결이 없음을 발견했다는 뜻입니다. ORA-3113은 좀 더 진단해야 추적 가능한 더 큰 문제가 있음을 알리는 신호탄에 불과합니다. 다행히도 앞서 말한 여섯가지 정보를 참고하면 해결책을 찾는 방향은 잡힐 것입니다. 우선 ORA-3113을 디버깅하려면, 루프락을 수행중에 같은 CONNECTING 을 여러번 시도 해 보는 것이 좋습니다. 즉, 서버의 어떤 줄이든 데스크톱 클라이언트에서 지점하는 것과 같은 연결 스트림을 사용하여 연결할 수 있습니다. 루프락을 수행중에도 똑같은 문제가 발생하면 데스크톱 클라이언트 쪽이 아니라 서버쪽에 문제가 있다고 보아야 합니다. 루프락을 수행하더라면 서버에서 BQL*WB 또는 BQL*BA를 호출하고, 서버의 BQL*WB 또는 BQL*BA 프롬프트에서 다음과 같이 입력하십시오.

CONNECT USENAME/PASSWD@:/:

예를 들어, BQL*NET TCP/IP를 통해 Unix 서버에 연결해 있고 BQL*PIus를 호출하고, 같은 "t::" 연결 스트림을 사용하여, 같은 SELECT 문을 내서 루프락을 해 보십시오.

ORA-6401 ERROR 발생시 처리 방법

Bulletin no : 10058

sql>mt v1,sql>mt v2 TEST 시에 아래와 같은 error message가 발생할 경우에 다음과 같이 해결
요.

ORA-6401:NETCNM-invalid driver designation

1) \$ORACLE_HOME/bin

\$adapter oracli

sql> mt v2 protocol adapter linked with oracli:

v2 pipe

ipc (unix domain socket)

tcp/ip -----> 주의하여 볼것.

2) \$ORACLE_HOME/bin

\$adapter sqlplus

sql>mt v2 protocol adapter linked with sqlplus:

v2 pipe

ipc (unix domain socket)

위와 같으면 error 발생 ..

sqlplus 실행 시에 tcp/ip protocol link가 빠져서,

=> 1. cd \$ORACLE_HOME/sqlplus/lib

2. make -f sqlplus.mk install

기타 다른 실행 시에 도 확인하여 make 작업을 다시 하세요.

ORACLE의 TWO-TASK DRIVERS

Bulletin no : 10233

Unix 시스템에서 User Process와 ORACLE Shadow Process 사이에 Communication 하기 위해 다양한 Driver 들이 사용된다.

1. Pipe(Default)과 Fast Driver

Single - System 환경에서 사용됨.
1개의 Machine에 있는 Multiple DB에 연결하기 위해 사용
Oracle 의 Standard Part로서 지원
Fast Driver는 OS에 만 지원

2. Networking Drivers(BQ14Net TCP/IP 등)

Networking 환경하에서 사용
분리된 Product로 판

3. BQ14NET V2

(1) BQ14

BQ14Net V2의 pipe adapter
Unix pipe에 만 사용, NTB가 아님

(2) IPC(Inter-Process Communication)

BQ14NET V2의 Default Adapter(NTB)(7.0.13 이후)
UNIX Domain Sockets 의 Top에서 수행됨
Local Host에 Running 중인 Process 간에 Socket-Based Communication
NTB에 Locally Connection 하기 위해 사용.
Oracle IPC Adapter 는 IPC Communication Protocol 과 Integrate 하기
위하여 NTB와 그것의 Application 을 이해해야 한다.
NTB Connect Descriptor으로 IPC-Specific Parameter를 사용할 수 있다.
IPC Adapter 는 BQ14NET V2를 설치 할 경우 자동적으로 설치된다.

```
IPC ADDRESS
( ADDRESS =
  ( ( COMMUNITY = community_name ) )
  ( PRODUCT=IPC )
  ( KEY=alphnumeric )
)
```

```
ntx_dinputsize="1pc,1"  
ntx_max_dinputsize=10  
ntx_convsize=0007  
ntx_convsize=1  
ntx_max_convsize=10  
ntx_filter_size_address="(address=(protocol=1pc)(length=400))"
```

PC CLIENT TOOL 과 ORACLE DB 와 의 INTERFAC E 방 법

Bulletin no : 10012

PC의 다양한 환경에서 일부 잘 알려진 제품들 중심으로 Oracle과의 Interface 방법을 기술 하였습니다. 업무에 참조 하시기 바랍니다.

분 류	제 품 명	회 사 명	Oracle 과의 Interface
데이터 베이스	Access	MS	Glue, DBC
	FoxPro	MS	Connectivity
	Clipper		Now
	dBase IV		Now
	Paradox		BQULink
Spread Sheet	Excel	MS	Glue, Q+E
	Lotus1-2-3	Lotus	Glue, Q+E(?)
	Quantica Pro		Glue
Word Processor	Word	MS	Glue
	Word Perfect	WordPerfect	Glue
	Ami Pro	Lotus	Glue
Window Program	Visual Basic	MS	Glue, Q+E
	ToolBook	Amymatrix	Glue, Q+E
Tool			

(Remarks):

- MS: Microsoft
- Q+E: Planner Software
- Glue: Oracle
- Connectivity: Microsoft
- BQULink: Paradox
- 기타 기술된 내용 중에 잘못된 부분이나 추가시킬 내용이 있으면 메일로 보내 주시면 감사하겠습니다..

PRECOMPILE에서 MODE OPTION으로 저장하는 방법

Bulletin no : 10901

1. Pro*C Version 1.5 에서 대입되는 Mode option은 DISABLE (default)와 ANSI로 구분되어진다. 두 Mode의 큰 차이는 Char문자의 입력(insert)과 비교(where)에서 나타난다.

2. Oracle Version 8에서 Pro*C Version 1.3를 이용하여 Application Program작성시 문자전수에 대하여 [char]를 선언할 때와 [varchar]를 선언할 때 입력과 비교에 걸다른 차이를 발견할 수 없다.

Oracle Version 7에서 Pro*C Version 1.5사용 시는 Mode Option에 따라 커다란 차이를 볼 수 있다.

3. Mode=Oracle, 또는 Mode를 선언하지 않은 경우

(1) 문자 전수에 대하여 작업 (insert, update, where)을 하는 경우 집히[varchar]의 TYPE를 선언해야 한다. 만일 문자전수에 대하여 [char]로 선언된 경우 insert시 정상적으로 작업된 것과 같이 수행이 되나 insert된 자료를 BQL*Plus에서 [where]를 이용하여 Select시 검색할 수 없다. 그러나 Pro*C로 프로그램을 작성하여 검색시는 정상적으로 발견할 수가 있다. [char]전수로 선언하여 insert시 보이지는 않으나 문자의 뒷부분에 \0값이 모두 들어가 있다.

```
예)      exec sql begin declare section;
          char name(10);
          exec sql end declare section;
          strcpy(name, 'abc');
          exec sql insert into emp (name) value (:name);
```

위의 경우, 실질적으로 [name]전수내에는 [abc\0\0\0\0\0\0\0]의 형태로 값이 저장되어 insert작업을 수행한다. 이를 확인하는 방법은 BQL*Plus에서 다음의 쿼리를 수행한다:

```
BQL> select value(name), name from emp;
```

이때 보이는 문자는 [abc]이나 value값은 [10]으로 나타나게 된다.

그러므로 BQL> select name from emp where name='abc';를 수행하면 검색된 자료가 없다는 message가 출력된다.

(2) 문자전수를 [char]로 선언하고 사용 가능한 방법

[char]로 전수를 선언한 경우 문자값 뒤에 \0값이 저장되므로 그 값을 [] (space)로 대체하면 된다. 위의 예제에서

```
strcpy(name, 'abc');
strcat(name, ' ');
```

를 이용하면 BQL*Plus에서 정상적으로 검색되고 value의 값도 정상적으로 출력된다.

4. Mode = ANSI를 선언한 경우

(1) ANSI Mode를 이용하기 위해서는 우선

```
Long BQLCODE;
```

를 선언해 주어야 한다. 이 BQLCODE에는 BQL문장수행시 Error값을 저장한다.

(2) ANSI Mode의 장점은 Oracle Version 8 사용시와 동일하게 문자 변수에 대하여 [char]과 [varchar]를 함께 작업을 한다.

```
예)      exec sql begin declare section;
          char      name(10);
          varchar  name2(10);
          exec sql end declare section;
          strcpy(name, "abc");
          strcpy(name2, 'def');
          name2, len = strlen(name2, 'def');
          exec sql insert into emp(name) values (:name);
          exec sql insert into emp(name) values (:name2);
```

(위와 같이 작업을 수행한 경우 BQL#1uc에서 동일한 작업이 가능하다.)

Note: Oracle Version 8에서는 ANSI 드가 없으므로 사용할 수 없다.

PROC 2.X 에서 INCLUDE FILE 사용 방법

Bulletin no : 10200

PROC 2.0 에서는 include file 에대한 디렉토리 PATH를 INCLUDE option 으로 지정하여 사용 할수 있다. 이경우 Precompiler 는 다음 순서로 디렉토리를 BCAN 하면서 include file 을 찾는다.

1. 현재 디렉토리
2. standard header file 에대한 built-in 디렉토리
3. INCLUDE option 에 지정된 디렉토리

* 2 의 경우 ex1a.h, ex1da.h 와 같은 standard header file 에 대한 디렉토리는 지정할 필요가 없고, UNIX에서는 precompiler 가 DBACLE_HOME /sqllib/public 디렉토리에서 이 파일들을 찾는다.

* Nonstandard header file은 현재 디렉토리에 없으면 다음과 같이 INCLUDE Precompile option 으로 path를 지정한다.

```
... INCLUDE=patk1 INCLUDE=patk2 ..
```

<사용예>

<< 프로그램 >>

```
/* example2.pc */
```

```
#include
```

```
#include
```

```
#include
```

```
#include
```

```
typedef char ucclix[PUB_LEN];
```

```
EXEC BQL TYPE ucclix IS BSTRING(40) REFERENCE;
```

<< INCLUDE FILE >>

```
/usr2/oracle7/lib/ex2.h
```

```
-----  
#define NAME_LEN 20
```

```
#define PUB_LEN 40
```

<< MAKE FILE >>

```
/usr2/oracle7/lib/proc INCLUDE=/usr2/oracle7/lib Iname=$1.pc
```

```
*****
```

```
cc -I. -D -Am -D_HPLIB_BOUNDC +EBzfc +EBlit -I/usr2/oracle7/sqllib/public
```

```
-I/usr2/oracle7/lib -c $1.c
```

```
*****
```

```
cc -U1 -archlib -L/usr2/oracle7/lib -o $1 $1.o -lsql -lqlint -lora
```

```
-lqlint -lora -lscrt1 -lcvs -lcorn -lscrt1 -lcvs -lcorn 'cat
```

```
/usr2/oracle7/rdlib/lib/sqlb11c'
```

SAMPLE: PROC 에서 SESSION 을 KILL 하는 방법

Bulletin no : 101B4

```

#include
EXEC SQL BEGIN DECLARE SECTION:
    VARCHAR    username(20);
    VARCHAR    password(20);
    varchar    user(30);
    varchar    site(130);
EXEC SQL END DECLARE SECTION;

EXEC SQL INCLUDE sqlca;
void sqlerror();    /* handles unrecoverable errors */

main() {

    strcpy(username, user, "BYB");    /* copy the username */
    username, len = strlen(username, user);
    strcpy(password, user, "MANAGER");    /* copy the password */
    password, len = strlen(password, user);

    EXEC SQL WHENEVER SQLERROR DO sqlerror();
    EXEC SQL CONNECT :username IDENTIFIED BY :password;
        EXEC SQL SELECT '''||to_char(BID)||', '''
                        to_char(BENIAB)||'''
                INTO :user
                FROM v$session
                WHERE sid=:B;
    strcpy(site, user, "After system kill session ");
    attach(site, user, user, user);
    site, len= strlen(site, user);
    printf("Xx %d", site, user, site, len);
    EXEC SQL EXECUTE IMMEDIATE :site;
}

void sqlerror()
{
    EXEC SQL WHENEVER SQLERROR CONTINUE;
    printf("\nDBACLE error detected:\n");
    printf("%s .70s %s", sqlca, sqlwarn, sqlwarn);
    exit(1);
}

```

PROC 에서 외부 환경변수를 받아 들이는 방법

Bulletin no : 10213

```

#include
typedef char ucclx[20];
EXEC SQL BEGIN DECLARE SECTION:
    /* Define type for null-terminated strings */
    EXEC SQL TYPE ucclx IS VARCHAR(20) REFERENCE:
        ucclx      username;
        ucclx      password;
        ucclx      emp_name;
        short      ind;
        float      salary;
        float      commission;
EXEC SQL END DECLARE SECTION;
EXEC SQL INCLUDE sqlca;

void sqlerror(); /* handles unrecoverable errors */

main()
{
    /* Log on to ORACLE */
    strcpy(username, "SCOTT");
    strcpy(password, "TIGER");

    /*
    ** ORA_NLS 는 ORACLE 7.2 에서만 필요 함.
    **/
    putenv("ORACLE_HOME=/oracle/oracle");
    putenv("ORACLE_SID=OR722");
    putenv("NLS_LANG=American_American, latin5B01");
    putenv("ORA_NLS=/oracle/oracle/ocommon/nls/ade/ade");

    EXEC SQL WHENEVER SQLERROR GO sqlerror();
    EXEC SQL CONNECT :username IDENTIFIED BY :password;
    printf("Connected to ORACLE as user: %s\n", username);
    EXEC SQL DECLARE empname CURSOR FOR
    SELECT empname, sal, comm
        FROM emp
        WHERE job LIKE 'SALES%';
    EXEC SQL OPEN empname;
    for ( ; ; )

```

```

{
    EXEC SQL FETCH xslsqmptm
        INTO :mp_mmm, :xslary, :commxslmtd;

    IF (xslca, xslcda == 1409) break;
    IF (xslca, xslcda == 0) {
        printf("x-11x19.2fx13.2fx\n", mp_mmm, xslary, commxslmtd);
    }
}

EXEC SQL CLOSE xslsqmptm;
printf("\nHave a good day.\n");
EXEC SQL COMMIT WORK RELEASE;
exit(0);
}

void xglerror()
{
    EXEC SQL WHENEVER SQLERROR CONTINUE;
    printf("\nORACLE error detected:\n");
    printf("\n% .79s\n", xslca, xglerrn, xglerrm);
    EXEC SQL ROLLBACK WORK RELEASE;
    exit(1);
}

```

PROC의 MAKE FILE을 이용하여 SHELL 프로그래밍을 만드는 방법

Bulletin no : 10154

A. DBACLE Server 버전이 7.0.K 미만일 경우

[DBACLE7 버전확인 방법]

```
$ sqlldr loader
SQL*Loader: Release 7.1.3.2.0 - Production on Thu Nov 18 10:08:42 1995
Copyright (c) Oracle Corporation 1978, 1994. All rights reserved.
Oracle7 Server Release 7.1.3.2.0 - Production Release
*****
```

[단계1]

1. \$ cd \$ORACLE_HOME/proc/demo
2. \$ make -af proc.mk example1 > mk
-- mk 내용이 "example1 is up to date" 이면 example1.pc 만 남겨놓고
-- example1.c example1.o example1 을 삭제후 다시 실행한다.
3. \$ vi mk (example1 부분을 \$1 으로 수정 한다)

```
/usr3/ora7018/bin/proc lsclean=132 orclean=132 select_orproc=0
lsclean=$.pc
/usr3/ora7018/bin/echo2 cc -I. -D -L/usr3/ora7018/lib -o $1 $1.c
/usr3/ora7018/lib/libq1.a /usr3/ora7018/lib/orclib.a -lq1lib -lora
/usr3/ora7018/lib/libp1.a -lq1lib -lclib1 -lc8 -lc9 -lclib1
-lc8 -lc9 -lib -lclib1 -lq1 'cat /usr3/ora7018/rdbms/lib
/orclib1lib1'
```

4. \$ chmod 777 mk

[프로그램 수행]

```
$ ./mk example2
```

B. DBACLE Server 버전이 7.1.K, 7.2.K 미만일 경우

< PROC Version 1.K 을 사용하는 경우 >

[단계2]

1. `cd $ORACLE_HOME/proc18/dmso`
2. `make -af proc18.mk xcomp1d > mk`
3. [단계1] 의 3, 4. 를 수행

< PDBC Version 2.4 를 사용하는 경우 >

[단계3]

1. `cd $ORACLE_HOME/proc/dmso`
2. `make -af proc.mk xcomp1d > mk`
3. [단계1] 의 3, 4. 를 수행

SQL*NET SESSION의 DEAD CONNECTION 처리 방법

Bulletin no : 10223

■ Client Server 환경에서 PC를 Client로 사용시 미정상적인 망접으로 Server
과 Disconnect 하면 Server 쪽에 있는 Dedicated Server 가 남아 있다.

이는 Sql*net V1 & V2 에서 발생하던 문제로 Sql*net V2.1 이후에는 아래와
같은 망접을 이용하면 Dead connection 이 정상적으로 Disconnect 된다.

* 환경 : Server - Sql*net tcp/ip V2.1.3

Client - Sql*net V2.0

* Setup 망접 : Server 와 Client 에 있는

\$ORACLE_HOME/network/admin/sqlnet.ora 에

sqlnet.expire_timeout

으로 setting 한다.

여기서 * 은 분단위이며 실제 disconnect 는 * 분보다 더걸린다.

SQL*NET과 관련된 기본적인 점검 사항

Bulletin no : 10231

SQL*NET을 사용하면서 문제가 발생한다면 기본적으로 다음과 같은 사항들을 점검해 보아야 합니다.

Client PC에서 점검할 사항(Windows 3.1의 경우)

- 1) TCP/IP 레이어에서 제공하는 ping test가 제대로 되는지를 확인합니다.
- 2) tnscntz 파일에 사용하는 alias가 제대로 등록이 되어 있는지 확인합니다.
- 3) tnsnames 파일에 다음과 같은 항목이 등록이 되어 있는지 확인합니다.

listener	1521/tcp
oracle	1525/tcp

port 번호는 작은 것부터 큰 순서로 점검하도록 합니다. 경우에 따라서는 listener와 oracle을 tnsnames 파일의 맨 위에 등록을 한 다음 테스트를 해 볼 필요도 있습니다.

- 4) 다른 프로그램보다 우선 nettime과 sqlplus 이 하여 connect test를 해봅니다.

SQL*DB driver를 사용하는 경우라면 모뎀에서 제공하는 dial test 프로그램을 이용하여 test를 해 봅니다. 그리고 autoexec.bat 파일에 다음과 같은 부분을 추가합니다(ORA-3121 에러의 경우)

```
SET COMPIL=0:WINNBD32ORACLE.INI
```

WINNBD32를 사용하는 경우라면 oracim.ini 파일에

```
TCP_USENBD3=WINNBD32
```

으로 등록이 되어 있는지 확인합니다.

tnsconnect를 지점할 때 alias를 쓰지 말고 직접 ip address를 사용해서 접속해 봅니다. U1을 쓸 경우에는 connect string에서 alias를 ip address로 대체해서 테스트 하면 되고 U2의 경우는 tnsnames.ora 파일에서 alias로 정의된 부분을 ip address로 고쳐서 테스트 하면 됩니다.

Binwinsock.dll 파일이 여러 디렉토리에 걸쳐 있어서 충돌하는 경우 문제가 발생합니다. 사용하는 TCP/IP에서 제공하는 winsock.dll 을 제외한 나머지 winsock.dll 파일들을 다른 이름으로 rename 하도록 합니다.

BEHAVE 서 점검할 사항(UNIX의 경우)

1) oracle(U1)과 listener(U2)가 START 되어 있는지 확인합니다.

```
$lspsctl status (U1)
$lspsctl status (U2)
```

만약 start가 만되어 있다면 start를 시킵니다.

```
$lspsctl start (U1)
$lspsctl start (U2)
```

2) /etc/services 파일에 다음과 같은 항목이 등록이 되어 있는지 확인합니다.

```
listener      1521/tcp
oracle        1525/tcp
```

port 번호는 작은 것부터 큰 순서로 정렬하도록 합니다. 경우에 따라서는 listener와 oracle을 services 파일의 맨 뒤에 등록을 한 다음 테스트를 해 볼 필요도 있습니다.

3) oracle, listener 파일의 owner가 root, permissions이 4751인지를 확인합니다. (listener 파일은 owner가 root가 아니고 oracle user이어도 상관없지만 oracle은 반드시 root 소유이어야 함)

4) root권자로 들어가서 \$ORACLE_HOME/orainst 디렉토리에 있는 root.sh 스크립트를 실행합니다. 이것을 실행하면 현재 환경상태를 확인하여 물고 그문가를 물어보는데 물린 부분이 없으면 YEB를 선택합니다. 그 다음에 local bin directory를 물어보면 /etc 를 입력합니다. 이 스크립트를 통하여 오차를 완전 환경을 정확하게 세팅하게 됩니다.

5) /etc/oratab 또는 /var/opt/oracle/oratab 파일이 있는지를 확인하고 다음과 같은 내용이 들어 있는지 점검합니다.

```
ORA7:/home/oracle:N
```

여기서 ORA7은 ORACLE SID, /home/oracle 은 ORACLE HOME DIRECTORY, N은 system booting시에 ORACLE를 자동으로 STARTUP시키지 않는다는 것을 의미합니다. 이와같은 약명이 꼭 들어 있어야 합니다.

B) BEHAVE 문제인 CLIENT 문 인지를 알기 위해서 BEHAVE 상에서 LOADPAGE TEST를 해봅니다.

```
$sqlplus scott/tiger@tl:orcl2:ORA7 (U1)
$sqlplus scott/tiger@TOLN (U2)
```

여기서 TOLN은 \$ORACLE_HOME/network/admin/tcnames.ora 파일에 등록이 되어 있 service name입니다.

7)UNIX BYSTER를 rebooting 해줍니다.

8)executable file를 compile 해 줍니다. (U2)

```
cd $ORACLE_HOME/network/lib
make -f network.mk tnsname
cd $ORACLE_HOME/bin
mv tnsname tnsname.org
mv $ORACLE_HOME/network/lib/tnsname tnsname
chmod 4755 tnsname
```

이렇게 하고 listener를 죽였다가 다시 띄워 줍니다.

9)oracle kernel에 BDLWRK U2가 제대로 LINK되었는지 확인합니다. (U2)

```
cd $ORACLE_HOME/bin
adaptora oracle
```

여기서 아무것도 출력되지 않으면 link가 안된 것입니다. 이럴 경우에는 ORACLE를 완전히 SHUTDOWN 하고 다음과 같이 합니다.

```
cd $ORACLE_HOME/rdbms/lib
make -f oracln.mk oracln
cd $ORACLE_HOME/bin
mv oracln oracln.org
mv $ORACLE_HOME/rdbms/lib/oracln oracln
chmod 4755 oracln
```

SQL*NET V1 FOR APPC/LU6.2 TEST

Bulletin no : 10084

DIIvettI system에서 SQL*Net V1 for APPC/LU6.2 test 결과입니다.

customer : 한국컴퓨터

Test System : DIIvettI UNIX System 2대

D / B version : BUN4 2.4

1. Server Install

1.1 Shared Memory Parameter

BHMHAK388808

BHMBE68

BHMHMI

BHMHMI00

1.2 Kernel Parameter

BUNHLIN0x7FFFFFFF

HUNHLIN0x7FFFFFFF

BFBZLIN0x7FFFFFFF

HFBZLIN0x7FFFFFFF

파라메터가 이보다 적으면 oracle kernel을 만들면서 에러가 발생함.

1.3 Oracle Server Install 시 rlink option 에서 Yes 선택.

LU6.2를 사용하기 위해서 반드시 해주어야 함.

1.4 Version@ 2.30이하인 경우 kernel 생성시 에러 발생.

2. SQL*Net Install

2.1 Transaction Program 을 Generate.

```
$ $ORACLE_HOME/bin/genops ORACLE_SID Max_MU_size Min_MU_size
```

MU_size는 256 에서 1024 범위내에서 set 해야함.

```
ex) $ genops ORA7 512 512
```

generate 후 TPORA7 이라는 Transaction Program 이 generate 되었다는 message가 찍어짐

TP name은 ORACLE_SID 앞에 TP가 붙어서 생성.

generate후 \$ORACLE_HOME/lu62/server directory 에 TPORA7이라는 binary file이 생성.

2.2 TPORA7을 APPC Services에 등록.

expadm tool을 이용하여 등록.

TP_Name : TPORA7

TP_Fileappc : /home/oracle7/lu62/server/TPORA7

3. Connection.

3.1 Connect String

- DB:remote_LU_name:local_LU_name:node_name:TP_name

remote_LU_name, local_LU_name, node_name 의 value는 APPC
service에 등록이 되어있음.

ex) \$ sqlplus scott/tiger@DB:00000000:AAAAAAAA:CCCCCCCC:TP0000?

4. 특이사항.

- gantpa@ Max_BI_size와 Min_BI_size 256으로 set:

sqlplus에서 접속시 BQL> prompt가 떨어지지 않고 waiting 상태.
한편 상대방 remote system에 connection 은 연결됨.

- gantpa@ Max_BI_size와 Min_BI_size 512으로 set:

sqlplus에서 접속은 됨.

그러나 512를 넘어서는 데이터밀집주어는 DBA-B412 에러가 떨어지면서
disconnect 가 되어버림. (DBA-B412:bad read length)
remote에서 select 된 데이터는 local system까지는 가져옴.

data size 는 54B.

sqlplus 에서 array size를 '1'로 하면 정상적으로 작동.

ex) 1). select empno, ename from emp;	----> 정상
2). select * from emp;	----> disconnect
3). select empno, ename, hiredate, sal, age, comm, deptno from emp;	----> disconnect
4). set array 1 select * from emp;	----> 정상

- gantpa@ Max_BI_size와 Min_BI_size 1024으로 set:

데이터 사이즈가 1024를 넘어설경우 동일한 현상발생.

SQL*NET V1, V2의 STATUS를 확인하는 방법

Bulletin no : 10117

* SQL*NET V1의 적절한 사용 방법

ORACLE의 Permission, Owner 및 Group은 다음과 같이 설정이 되어 있어야 하며 파일의 Size는 Version에 따라 다를 수 있다.

```
-rwxr-xr-x 1 oracle dba      7544832 Jul 21 20:35 oracle
-rwxr-xr-x 1 root      dba      303104 Jul  5 18:24 oracle
```

만일 위와 같지 않다면 Unix 명령 프롬프트에서 다음을 수행하여 모드를 바꾸어 준다.

```
Echoed 4755 oracle
```

```
Echoed root oracle <- oracle의 owner는 반드시 root로 되어야 합니다.
```

```
Echoed 4755 oracle
```

1. SQL*NET V1을 종료하면서 가장 밀란적으로 oracle을 시작 및 중지 하는 경우이다.

```
Stopctl start ( stop )
```

2. oracle의 현상태를 확인하는 방법

```
Stopctl status
```

```
tcputl: Status summary follows
```

```
Server is running      :
  Started               : 4-OCT-85 15:58:48
  Total connections     : 0
  Total rejections      : 0
  Active subprocesses   : 0
  ORACLE SID            : BC, ORA722
  Default SID           : (null)
```

```
Logging mode is ENABLED.
```

```
ORA logins are DISABLED.
```

```
OPB$ logins are ENABLED.
```

```
OPB$BDOT logins are DISABLED.
```

```
Oracle is detached from the terminal.
```

```
Break mode = OUT OF BAND.
```

```
Debug level = 1
```

```
Timeout (on oracle handshake) = 5 seconds.
```

```
Length of listen queue = 10
```

```
Oracle logfile = /u0002/oracle7/tcp/log/oracle.log
```

```
Oracle sapfile = /etc/oratab
```

* SQL*NET V2의 적절한 사용 방법

SQL*NET V2를 사용하는데 있어서 갖추어야 할 파라미터 파일이 있다.

기존으로 listener.ora, tnsnames.ora, sqlnet.ora가 \$ORACLE_HOME/network/admin directory 내에 있어야 합니다. (상기 파일에 대한 configuration은 SQL*NET Administrator's Guide v 2.0을 참고하시기 바랍니다.)

1. BQLWNET V2를 사용하면서 가장 밀란적으로 Listener를 시작 및 정 하는 경우이다.

```
lsnrctl start (stop)
```

2. Oracle과 Listener의 Process를 확인하는 방법

```
$ ps -ef |grep ORA?
ora_? 1343 1 ? Aug 22 ? 30:53 ora_dbwr_ORA?
ora_? 1342 1 ? Aug 22 ? 0:32 ora_pmon_ORA?
ora_? 1348 1 ? Aug 22 ? 0:00 ora_pcoo_ORA?
ora_? 1347 1 ? Aug 22 ? 0:00 ora_x000_ORA?
ora_? 1345 1 ? Aug 22 ? 12:25 ora_xmon_ORA?
ora_? 1344 1 ? Aug 22 ? 58:57 ora_lgwr_ORA?
ora_? 1349 1 ? Aug 22 ? 0:00 ora_x001_ORA?
ora_? 8038 1 ? Aug 22 ? 0:00 /ora/ora7/bin/ttnlsnr LISTENER -
lsnrctl
ora_? 24278 1342 ? 08:40:18 ? 0:05 ora_d001_ORA?
ora_? 888 1342 ? 14:54:38 ? 0:21 ora_d000_ORA?
```

3. Listener의 현상태를 확인하는 방법

```
$ lsnrctl service
LSNRCTL for Oracle UNIX: Version 2.0.14.0.0 - Production on 24-AUG-04
08:18:37
Copyright (c) Oracle Corporation 1993. All rights reserved.
Connecting to (ADDRESS=(PROTOCOL=IPC)(KEY=ORA7))
Services Summary...
ORA7 has 3 service handlers
DEDICATED SERSERV established:0 refused:0
DISPATCHER established:0 refused:0 current:0 max:80 state:ready
POO0 (machine: A1110M, pid: 888)
(ADDRESS=(PROTOCOL=tcp)(DEV=8)(HOST=152.88.1.11)(PORT=2753))
DISPATCHER established:0 refused:0 current:0 max:80 state:ready
POO2 (machine: A1110M, pid: 1357)
(ADDRESS=(PROTOCOL=ipc)(DEV=5)(KEY=81357,1))
The command completed successfully
```

4. BQLWNET V2를 NTS와 DEDICATE방식으로 사용포자 할때.

```
$cd $ORACLE_HOME/network/admin/ttnsnmr.ora
```

NTS =

```
(DESCRIPTION =
  (ADDRESS_LIST =
    (ADDRESS =
      (PROTOCOL = TCP)
      (HOST = Hostname)
      (PORT = 1521)
    )
  )
  (CONNECT_DATA =
    (SID = ORA7)
  )
)
```

```

    }
}

NTS_DEF =
(DESCRIPTION =
  (ADDRESS_LIST =
    (ADDRESS =
      (PROTOCOL = TCP)
      (HOST = Hostname)
      (PORT = 1521)
    )
  )
(CONNECT_DATA =
  (SID = DBA7)
  (SERVICE_NAME = MEDICATED)
)
)

```

5. BQLENET V2 Fire Adapter를 사용중이라면 `ps command`를 통해서 `oraclib (LOCAL=YES)`라는 Window Process로써 확인 할 수 있다.

SQL*NET V1, V2의 차이점

Bulletin no : 10118

* 다음의 내용은 BQL*NET에 대해 사용자들이 가장 진련히 문의한 내용을 간추린 것입니다.

1. BQL*NET V2과 BQL*NET V1은 서로 호환성이 있습니까 ?

- 아니요. BQL*NET V2과 BQL*NET V1은 Client 또는 Server사이에 주고 받는 Data Packets에 있어 서로 호환성이 없습니다. 서로 다른 BQL*NET Version으로 네트워크를 통하여 BQL*NET V1 을 사용하는 Client 또는 Server는 BQL*NET V2를 이용하는 Client 또는 Server와 통신할 수 없습니다.

2. BQL*NET V2과 BQL*NET V1을 같이 사용할 수 있습니까 ?

- 사용자는 Client 또는 Server상에서 BQL*NET V2과 BQL*NET V1을 같이 사용할 수 있는 데 이는 이 둘이 동시에 공존할 수 있기 때문이다. 즉, Server에는 BQL*NET V2과 BQL*NET V1을 동시에 지원하며 두 가지 Version으로 접속을 할 수 있습니다. 다시 말하자면 Client에서 BQL*NET V1을 이용할 경우 scott/tiger@tbs.tbs.tbs:1521로 BQL*NET V2일 경우에는 scott/tiger@payroll로 접속하며 여기서 payroll은 V2 접속기출자 입니다.

3. 통신 가능한 BQL*NET Version과 Oracle Server Version에 대한 소개

- | | |
|--------------------------|-------------------|
| . BQL*NET V2.0 | - BQL*NET V1 only |
| . Oracle 7 Server 7.0.15 | - BQL*NET V1 |
| | - BQL*NET V2.0.14 |
| . Oracle 7 Server 7.0.18 | - BQL*NET V1 |
| | - BQL*NET V2.0.15 |
| . Oracle 7 Server 7.1.3 | - BQL*NET V1 |
| | - BQL*NET V2.1.3 |
| . Oracle 7 Server 7.1.4 | - BQL*NET V1 |
| | - BQL*NET V2.1.4 |
| . Oracle 7 Server 7.2 | - BQL*NET V1 |
| | - BQL*NET V2.2 |

4. BQL*NET V1에 없는 BQL*NET V2.0 만의 새로운 기능은 ?

- . Comprehensive tracing and logging facilities of clients, servers and listeners
- . Better error reporting and more granular debugging capabilities.
- . Support for Oracle MultiProtocol Interchange
- . Alternate Path Routing and network fault tolerance
- . Improved generic architecture and improved performance (various per protocol)
- . Full support of the Oracle7 Multi-Threaded Server
- . Consistent addressing schemes
- . Oracle Form-based network configuration tool and many more features

5. BQL4NET V2.1의 새로운 기능

- . Oracle Network Manager, the graphical enterprise configuration tool
- . Oracle Names, the global naming and directory service for Oracle
- . Dead Connection Detection, for detection of defunct connections
- . Prestarted Server Process, for improved connection startup time.
- . Support for Oracle Secure Network Services, the database encryption and data integrity product (a separate product, licensable separately)
- . Encrypted passwords for BQL4NET utility
- . More protocol support in BQL4NET and the MultiProtocol Interchange

SQL*NET V1 사용 시 ORA-6107 조치 방법

Bulletin no : 1011B

* Windows용 BQWNET TCP/IP User 1.4는 BQWTCP.BLL과 BQWTCPI.BLL 두 개의
BLL이 하나로 구성되어 있다. 이들 BLL은 ORACLE를 연결 스트림이 TCP/IP
프로토콜 스트림으로 확인되면 작업진행중에 OCI BLL에 의해 물려진다.
ORACLE Interface BLL은 BQWTCP.BLL을 먼저 물리려고 한다. 이것이 실패하면
ORACLE 버전의 드라이버를 찾는다. 두 가지 모두 실패하면 ORA-3121
메세지가 나온다.

1. BQWNET TCP/IP BLL이 물려지고 초기화가 되는 즉시 Logon 시도를 시작한다.

2. 이때 모든 것이 정상적이라면 Logon시 제공된 연결 스트림내의 I.P
Address대신 Hostname을 사용하여 해결해 본다. 이러한 과정이 정상적이라면
클라이언트측의 통신관련 디렉토리에 있는 Service 파일에 Oracle Service's
Partnumber를 제대로 부여한 것이며 BQWNET TCP/IP BLL이 ORACLE.INI (또는
WIN.INI의 [Oracle] Section의 ORACLE_CONFIG=ORACLE.INI에 기술된 내용)
내에 기술된 TCP_SERVICE_FILE = %service% 를
사용한다는 것이다.

ORACLE Service를 갖는 Part Number SERVICE File내에 "oracsv" 라고
기술된다. SERVICE File의 예를 보면 하단과 같다.

E This list has been derived from BFC10B0

E

E TCP Ports

E Service Name Port/Protocol Alias

echo	7/tcp		
discard	8/tcp	clink null	
exec	11/tcp	ucore	
listener	1521/tcp	orac1s	BQWNET U2 인 경우
oracsv	1525/tcp	orac1s	BQWNET U1 인 경우

3. 만일 TCP_SERVICE_FILE에 SERVICE파일의 구성되어있지 않았다면
"oracsv"의 접속이 실패된다. 만일 SERVICE File Name이 제대로 구성되어
있다면 File내에 표현되어 있는 "oracsv" 항목을 점검한다. SERVICE File
내의 각 항목 끝에 이 없을 경우나 "oracsv" Item 상에
이 있을 경우에도 이러한 메세지를 수반한다.

* ORA-6107을 진단하는 방법

1. ORA-6107의 경우 ORACLE.INI내의 TCP_SERVICE_FILE이 무엇인지 (또는
WIN.INI내의 [ORACLE] Section에 ORACLE_CONFIG가 무엇인지)를 점검한다.

- DBACLE.INI의 경우

TCP_SERVICES_FILE=\SERVICES

- WIN.INI의 경우

DBACLE=C:\winnt\winnt\acl

- 만약 설정이 되어 있지 않았다면 Client상에서 사용하고 있는 특정 TCP/IP Winnt에서 사용하고 있는 SERVICES 파일을 찾고 그 파일내에 Parameter를 설정한다.

2. 만약 TCP_SERVICES_FILE에 설정되어 있는 SERVICES 파일의 위치가 맞다면 파일내에 "oracle" 항목을 점검하여 본다. 파일내에 을 확인해 본다.

파일내의 각 항목 끝에 이 있는지 확인한다.

3. 만약 이 모든 것이 정상적이라면 새로운 SERVICES 파일내에 "oracle" 항목을 추가한다.

4. 만약 SERVICES 파일의 도움없이 개발 응용프로그램에서 직접 접속을 시도할 경우에는 명시적으로 다음과 같이 Connect String에서 Port Number를 사용하여 접속할 수 있다.

"t://<port number>:<instance name>"

[주의] instance name = DBACLE BID

5. tcp_services_file=<drive>\path\services

[주의 1] DBACLE TCP/IP Listener의 기본 시작포트 번호는 십 수 1525 이다. SERVICES 파일은 대부분의 장비에 /etc 디렉토리만에 있으며 또한, "tcpctl" Utility를 사용하여 UNIX 장비상의 DBACLE oracle 을 Start 시키고 Stop할 수 있다. Oracle 을 중지하려면 "tcpctl stop"을 사용하고 다시 시작하려면 "tcpctl start" 사용한다.

FIG 1 : 다음과 같은 경우에도 예외가 발생할 수가 있다.

oracle 1525/tcp<CR>

위와 같이 기술되었을 경우 다음과 같이 하게되면 ora-B107 error 메시 접속이 가능하다. 즉, oracle항목에서 <CR>전에 공란이 있을 경우에 예외없이 접속할 수 있다.

oracle 1525/tcp <CR>

* 이 버그는 v1.1.7.10.1 이후 버전에서는 수정되었다.

FIG 2 : 동일한 응용 프로그램에서 동일한 Connect String으로 접속을 시도하였을 때 예외가 발생한다면 다음과 같이 Connect String에 Port Number를 사용한다.

t://<host name>/<port number>:<oracle_BID>

* 이 버그는 v1.1.7.8 이후 버전에서는 수정되었다.

SQL*NET V2 SETUP 방법

Bulletin no : 10074

UNIX에서 SQL*NET V2를 사용하려면 ORACLE 인스턴스 후에 다음과 같이 SETUP을 한 다음 사용하면 된다.
여기서

ORACLE HOME DIRECTORY는 /uap/oracle7

ORACLE SID는 ORA7

이라고 가정한다.

1. Multi-threaded server 프로세스들을 먼저 기능을 하여야 한다.

\$ORACLE_HOME/dbs/initSID.ora 파일에 다음과 같은 내용을 추가하고 DB 를 다시 startup 을 한다

```
#####
* File_name = /uap/oracle7/dbs/initSID.ora
* Protocol = tcp
#####
```

```
mtx_dispatchers="tcp,3"
mtx_max_dispatchers=10
mtx_servers=2
mtx_max_servers=10
mtx_services=ORA7
mtx_listener_address="(ADDRESS=(PROTOCOL=tcp)(port=1521)(host=hostname))"
```

2. /etc/services file 에 다음내용을 추가한다. services 파일은 Machine에 따 디렉토리가 다를수 있다.

```
...
listener      1521/tcp
```

3. tnsnames.ora 및 listener.ora 를 작성 한다. 이 파일들은 /uap/oracle7/network/admin 디렉토리에 만들어야 하며 유닉스 환경변수를

TNS_ADMIN=\$ORACLE_HOME/network/admin

와같이 지정하고 사용하는 것이 좋다.

```

#####
# File name : /uwer/oracle7/network/sdnlm/listerm.ora
# This configuration supports tcp
#####

```

```

LISTENER =
  (ADDRESS_LIST =
    (ADDRESS =
      (PROTOCOL = tcp)
      (HOST = hostnam)
      (PORT = 1521)
    )
  )

STARTUP_WAIT_TIME_LISTENER = 0
CONNECT_TIMEOUT_LISTENER = 0

SID_LIST_LISTENER =
  (SID_LIST =
    (SID_DESC =
      (SID_NAME = DBA7)
      (ORACLE_HOME=/uwer/oracle7)
    )
  )

```

```

#####
# File name : /uwer/oracle7/network/sdnlm/tacmnamc.ora
#####

```

```

TNSA =
  (DESCRIPTION =
    (ADDRESS_LIST =
      (ADDRESS =
        (PROTOCOL = tcp)
        (HOST = hostnam)
        (PORT=1521)
      )
    )
    (CONNECT_DATA =
      (SID = DBA7)
    )
  )

```

4. Listener 프로세스를 start 한다.

```
$lsnrctl start
```

5. BQLWNET v2 를 사용하여 NTB 에 접속한다. (LOADBACK TEST)

```
$ xglflux scott/tiger@TDBA
```

위의 5가지 과정이 성공적으로 수행되면 UNIX SERVICES 쪽에서의 BQLWNET V2 SETUP은 완료된 상태로 볼 수 있다. 만약 CLIENT에서 접속을 하는 경우라면 SERVICES의 /usr/oracle7/network/adm 디렉토리에 있는 파일 전체를 CLIENT의 \$ORACLE_HOME/network/adm 디렉토리로 COPY하면 CLIENT 쪽에서 BQLWNET V2를 사용하여 접속할 수 있다. 접속방법은 위의 LOBACK TEST 하는 경우와 같이 CONNECT STRING 부분에 tnsname.ora 에 등록된 SERVICE NAME(위에서는 TDBA)를 사용하여 접속하면 된다.

*SERVICES가 MINIMAL NT인 경우에는 다른 부분은 UNIX와 동일하게 셋업하면 되고 단지 listener를 띄우는 방법만 다르다. 제어판 -> SERVICES 에서 ORACLE 관련 SERVICES 들을 모두 START 시켜 주면 된다.

SQL*NET V2 사용시 ORA-12154 조치 방법

Bulletin no : 10129

* ORA-12154의 원인은 tnsnames.ora 파일의 Alias처럼 정의된 Connect String 으로 사용하는 db_alias를 찾지 못할 경우 발생한다.

다음의 사항을 점검한다.

* WINDOWS, OS/2 CLIENT PART

1. Login Box를 통해 접속을 할 경우에는 Connect String의 Field에서 @ 로 시작하지 않는다. @ 기호는 패스워드 뒤에 공란없이 바로 Connect String(예:username/password@db_alias)을 지정하는 경우에 사용한다. Connect String을 입력하는 Field에서 아래와 같이 기술한다.

tns://db_alias

또는

db_alias

2. TNSNAMES.ORA 파일은 반드시 Client PC의 ORACLE_HOME\etc\tnsnames 디렉토리에 있어야 하며, 사용자가 지정한 Connect String은 TNSNAMES.ORA 파일에 정의된 db_alias명이 기술되어 있어야 한다.

3. TNSNAMES.ORA 파일의 괄호(괄호쌍)를 제대로 열고 닫았는지 확인한다.

4. 만일 사용자가 TCP/IP를 사용하고 있다면 TNSNAMES.ORA 파일에 기술되어 있는 HOST=HOSTNAME을 HOST=IP ADDRESS로 변경한다.

5. BQLENT.ORA 파일들 점검하여 '#####.default_####-' 파다매를 확인한다. 만일 BQLENT.ORA 파일에 이 파다매가 존재하면 'E'으로 막는다.

#####.default_####=

6. 하나의 단일 Community에서 이동한다면 TNSNAMES.ORA에 기술되어 있는 'COMMUNITY' 파다매를 'E' 으로 막는다.

```
-
(DESCRIPTION =
  (ADDRESS_LIST =
    (ADDRESS =
      E      (COMMUNITY = SAMPLE_COMMUNITY)
            (PROTOCOL = TCP)
            (HOST = )
            (PORT = 1521)
          )
    )
  )
(CONNECT_DATA =
```



```

        (BID = <BID>)
    }
}

```

[주의] 상기의 <...>에 기술된 내용의 `dk_aliax`는 사용자 정의 필드이고, `Server`는 현재 사용자가 사용하는 `server`의 `host_aliax name` 또는 IP Address를, `BID`는 설치된 `DBACL instance` 명을 기술하여야 하며, `dk_aliax` 명은 `BID`와 동일해서는 안된다. 접속 기술자로 사용되는 `dk_aliax`의 경우 속시 대.소문자 구분을 한다.

7. `BQLNET.DBA` 파일을 점검하여 본다.

만일 파일내에 `name.default_domain=<ctrlng>` 파라메타가 있다면 `THENAMEB.DBA`의 `dk_aliax` 이름의 확장자로서 `.<ctrlng>`를 가지고 있어야 한다. 그렇지 않으면 `BQLNET U2` 접속을 할때 `ora-12154` 에러가 발생한다. 만일 `THENAMEB.DBA` 파일에 `dk_aliax`명에 `.<ctrlng>`를 포함하고 있고 사용자가 확장자 없이 `dk_aliax`로 접속을 시도시에 `ora-12154` 에러가 발생하였다면 `BQLNET.DBA` 파일에 `name.default_domain=<ctrlng>` 를 추가해야만 한다.

<예제 : `BQLNET.DBA` >

```

TRACE_LEVEL_CLIENT = OFF
sqlnet.expire_time = 1
name.default_domain = world
name.default_xname = world

```

<예제 : `THENAMEB.DBA` >

```

NT_BBU.world =
(DESCRIPTION =
  (ADDRESS_LIST =
    (ADDRESS =
      (COMMUNITY = NTCommus.world)
      (PROTOCOL = TCP)
      (Host = 152.88.31.100)
      (Port = 1521)
    )
  )
  (CONNECT_DATA = (BID = DBCL)
)
)

```

<예제 : `LISTENER.DBA` >

```

LISTENER =
(ADDRESS_LIST =
  (ADDRESS =
    (PROTOCOL = IPC)
    (KEY = orainc.world)
  )
)

```

```

    }
    (ADDRESS=
      (PROTOCOL=TCP)
      (KEY= DBCL)
    )
  }
  (ADDRESS =
    (COMMUNITY = MYCommunityName)
    (PROTOCOL = TCP)
    (Host = 152.88.31.100)
    (Port = 1521)
  )
}

```

```

STARTUP_WAIT_TIME_LISTENER = 0
CONNECT_TIMEOUT_LISTENER = 10
TRACE_LEVEL_LISTENER = ADMIN
BID_LIST_LISTENER =
  (BID_LIST =
    (BID_NAME = DBCL)
  )
}

```

* UNIX CLIENT PART

1. 상기의 UNIQUE, DB/2 Client의 1, 2 번 참조

2. 환경 전 요 THEADMIN, DBA의 LISTENER, DBA 파일이 위치하고 있는 디렉토리가 THEADMIN에서 제대로 지정하고 있나 확인한다.

Shell을 사용한다면 : `% mv l group THEADMIN`

정확한 디렉토리를 지정하고 있지 않으면 다음과 같이 THEADMIN 환경변수에 정확한 디렉토리 설정한다.

`% setenv THEADMIN <directory path>`

3. 사용자의 Configuration 파일들에 혹시 CTBL-H의 문자가 따민리 글에 삽입되어 있나 확인한다.

4. 사용자의 Connect String이 정확하다면 THEADMIN, DBA의 db_alias 이름들 BID명과 동일하게 수있는지 확인한다. BID와 db_alias는 다르게 기술해야 하며, 혹 Host명과 동일하게 수는 경우도 있는데, 이렇게 기술하면 Error를 발생시킬 수도 있다.

Sample V2 Connect String :

`sqlplus <user>@<password>@<tnsnames.ora에 있는 db_alias>`

Example : `sqlplus scott/tiger@0715tcp`

THENAMEB.Dba 파일의 예제 :

```
U715tcp =
(DESCRIPTION =
  (ADDRESS_LIST =
    (ADDRESS =
      (protocol = TCP)
      (host = korea)
      (port = 2000)
    )
  )
)
(CONNECT_DATA =
  (SID = U715)
)
)
```

5. THENAMEB.Dba File의 여러곳에 있는지 확인한다.

THENAMEB.Dba 파일은 3곳중 한곳에 있을 수 있다.

* BQLWEY U2의 Search Path Priority.

여기서는 BQLWEY U2 구성파일(LISTENB.Dba와 THENAMEB.Dba)을 찾는 순위에 대해 기술한다.
DBACL에서는 구성파일들 다음과 같은 순서로 찾는다.

1. 파일 THENAMEB.Dba 환경변수가 설정되어 있고 또한 디렉토리를 지정하고 있으면 확인한다.
2. /etc 디렉토리를 확인한다.
3. \$ORACLE_HOME/network/adfile를 확인한다.

SQL*NET V2 에서 BUFFER SIZE 지정 방법

Bulletin no : 10178

SQL*NET V2과 DBMS Driver를 사용하는 경우 Power Builder, Visual Basic, BQL Window, Object View, Excel, Access 등과 같은 프로그램을 사용하면서 대량의 데이터를 처리하는 경우 세션이 끊어지거나 하는 문제가 발생을 하던 SQL*NET V2의 Buffer Size를 줄여 줌으로써 문제를 해결할 수 있습니다.

PC의 %system% 하입(보통 c:\windows\system32\drivers 디렉토리에 있음) 에 BBU 를 적용하여 다음과 같이 수정하면 됩니다. 만약 BBU를 적용하여도 문제가 해결이 만된다면 REDIRECTED 방식으로 접속을 하여서 사용하기 하합니다.

BBU (Backlog Data Unit)

Syntax : BBU=m (Byte)
Range for 'm': 512 <= m <= 2048
Default value = 2048 Bytes.

바꾸기 전

```
TODAY =
(DESCRIPTION=
  (ADDRESS=
    (PROTOCOL=TCP)
    (PORT=1521)
    (HOST=krnp2)
  )
  (CONNECT_DATA=(SERVICE=))
)
```

바꾸기 후

```
TODAY =
(DESCRIPTION=
  (ADDRESS=
    (PROTOCOL=TCP)
    (PORT=1521)
    (HOST=krnp2)
  )
  (CONNECT_DATA=(SERVICE=))
  (BBU=1024)
)
```

SQL*NET V2에서 DEDICATED로 접속하는 방법

Bulletin no : 10230

SQL*NET V2를 사용하면서 MTS(Multi Threaded Server) 방식이 아닌 DEDICATED 방식으로 서버에 접속하려면 Client PC의 tnsnames.ora 파일을 다음과 같이 수정하면 됩니다 (tnsnames.ora 파일은 보통 c:\oracle\network\admin 디렉토리에 있음). 여기서 DRA_M을 사용하여 접속하면 MTS로 접속되고 DRA_D로 접속하면 DEDICATED로 접속됩니다.

Client가 PC가 아닌 경우에도 동일한 방법으로 수정하면 됩니다.

=====
MTS로 접속하는 경우
=====

```
DRA_M =
  (DESCRIPTION =
    (ADDRESS =
      (PROTOCOL = tcp)
      (HOST = krhp2)
      (PORT=1521))
    (CONNECT_DATA =
      (SID = DRA7)))
```

=====
DEDICATED로 접속하는 경우
=====

```
DRA_D =
  (DESCRIPTION =
    (ADDRESS =
      (PROTOCOL = tcp)
      (HOST = krhp2)
      (PORT=1521))
    (CONNECT_DATA =
      (SID = DRA7)(SERVICE=DEDICATED)))
```

WINDOWS PERFORMANCE TUNING

Bulletin no : 10118

WINDOWS 386 확장모드에서 설치후 PERFORMANCE 향상을 시대화하기 위한 방안들 다음과 같이 제시한다. (WINDOWS는 386확장모드에서 가상메모리를사용할 수 있다.)

이러한 제시 사항들은 표준 모드와 실행모드에 적용할 수 있으며 잘 작동한다.

물론 표준모드에서 WINDOWS의 수행은 386 확장모드 보다는 빠르나 대부분의 많은 APPLICATION은 386 확장모드를 요구하고 있으며 BQLMNET 또한 이에 포함되어 있다.

1. USE BHABTBIIVE

* MICROSOFT BHABTBIIVE 디스크 캐쉬 드라이버는 WINDOWS 3.1 PERFORMANCE 향상을 얻을 수 있는 가장 큰 요소이다. BHABTBIIVE는 언제든지 사용이 가능하며 사용자는 CONFIG.BYB에 BHABTBIIV.BYB를 로드하거나 AUTOEXEC.BAT에 BHABTBIIV.BYB를 로드하여 BHABTBIIVE를 사용할 수 있다. 이 driver를 사용시 적용할 수 있는 Option에는 여러가지가 있다. 사용자의 PC에 LOCAL Dracile BBMB를 사용중이 라면 Write caching을 OFF로 한다. (BHABTBIIVE의 설치및 운영에 대한 기본적인 정보는 "MICROSOFT WINDOWS USER'S GUIDE" VERSION 3.1 매뉴얼이나 DOS prompt에서 BHABTBIIV /?를 이용하여 참조.)

2. KEEP YOUR HARD DISK OPTIMIZED.

* WINDOWS 수행에 있어 가장 크게 영향을 주는 것은 하드디스크의 단편화로서 특히 임시 BMAP 파일과 또는 BHABTBIIVE가 설치 전후 파일이 여러부분으로 나뉘고 각기 다른 위치에 기록될 때 발생한다. 하드 디스크 최적화 프로그램을 수단위로 수행하여 디스크의 단편화를 줄이는 것이 좋다.

3. CREATE A PERMANENT BMAP FILE

* WINDOWS 3.1에서 사용자의 하드웨어가 사용중인 영구 BMAP 파일을 임시 BMAP 파일로 사용하여 PERFORMANCE를 향상시키려면 기본프로그램 그룹의 제어판 내에 있는 386확장 아이콘을 선택하고, 가상메모리 버전을 누르고 전경 버전을 누른 후 32 비트 디스크 액세스 체크박스를 선택한다.

4. TURN OFF GRAPHICS PORT TRAPPING

* 386 확장모드에서 수행하는 MS-DOS Application의 속도는 PIF 에디터의 고급옵션 버전을 누르고 디스플레이 옵션의 모니터 포트에서 어떠한 것도 선택하지 않는다면 눈에 띄게 PERFORMANCE가 향상될 수 있다. 고해상도 그래픽 옵션은 왕복위하게 MS-DOS 응용프로그램에 대해 호환성을 제공하지만 대부분의 응용프로그램에서는 이를 요구하지 않는다

5. TURN OFF THE FILESCACHE-SETTING

* WINDOWS 3.1 386 확장모드에서는 수정된 디렉토리의 매세지를 파일 관리자에게 보낼수 있고 MS-DOS Application에 의해 접근하는 디스크를 감시할 수 있다. 이는 MS-DOS Application에서 생성한 파일이나 디렉토리의 변경이 자동적으로 수정될 수 있도록 허용한다는 것을 의미 다. 이 기

능을 사용하지 않으려면 BYSTEN.INI 파일의 [386mb]부분에 FileSysChange =no로 설정한다.

B. TURN OFF THE BACKGROUND-PAGING

* 사용자가 MS-DOS 응용프로그램에서 확장메모리 지원을 요구하지 않는다면 이것을 Off로 설정한다. 이 설정을 Off로 설정하면 가장 MS-DOS 기계상에서 사용할 수 있는 대부분의 메모리를 활용할 수 있다. 이 기능을 사용하지 않으려면 BYSTEN.INI 파일의 [386mb]부분에 BackgroundPage Frame=no로 설정한다.

7. USE THE RIGHT NUMBER OF MS-DOS BUFFERS

* BHABTBIv를 사용중이라면 CONFIG.BYS 에 있는 MS-DOS disk에 접근할 수 있는 BUFFERS의 수를 15로 설정한다. (즉, BUFFERS=15). BHABTBIv와 함께 BUFFERS를 너무 크게 설정하여 사용하면 실질적으로 효율이 감소할 수도 있다. 만약 BHABTBIv를 사용하지 않으면 BUFFERS=30을 사용한다.

[주의] Device Client Tool들을 사용할 때 권고하는 config.sys 환경전수는 아래와 같다.

```
Buffers = 15  
Files = 30
```

WINDOWS용 SQL*NET V1 사용할 때 ORA-6136 조치 방법

Bulletin no : 10127

Client and Server 환경에서 간혹 BQL*NET으로 Server에 접속하려고 할때, ORA-6136 Error가 발생하면서, Unix Server에서 \$tcpctl stop 으로 orasrv의 Process를 중지시키려고 해도 아무런 반응없이 Holding되는 경우가 발생하면서 다음과 같은 메시지가 나옵니다.

```
ORA-6136, 00000, "NETTCP: error during connection handshake"
// *Cause: Network I/O failure occurred while communicating with the
//         host's BQL*Net TCP/IP server.
//         See the BQL*Net TCP/IP server log file for more details.
// *Action: Contact your customer support representative.
```

상기의 메시가 나옵니다 경우에는 아래와 같은 방법으로 orasrv를 기동시키십시오.

1. TCPCCTL Utilities를 이용하여 다음의 Option을 부여하여 Start하는 방법.

```
$tcpctl start listen=30 timeout=30 forkon
listen=이며, 성취자 열의 크기를 지정.
timeout=이며, 지정된 시간에 orasrv와의 응답 확인 시간을 나타냄.
forkon BQL*Net의 orasrv process에 접근하는 방법을 나타냄.
Listen에 따라서 forkon option이 적용 받지는 경우도 있음.
```

2. Client에서 접속을 하여 서버 다가 비정상 종료(Backlog이 맺어진 상태에서 Client의 Power Off)를 하여 Server에 Process가 남아 있고, orasrv를 통해 접속할 수 있는 Backlog의 수가 점점 줄어들 경우를 있는 데 이러한 경우에는 orasrv를(\$tcpctl stop or UNIX kill command를 이용)강제로 종료 시키고 다음과 같이 Start 시켜 주십시오.

Backup \$tcpctl start (Client의 비정상 종료가 Orasrv에는 영향을 미치지 않음)

또는

\$oracore (ORACLE_HOME\bin directory에서 직접 orasrv process를 리운다)

WINDOWS용 SQL*NET V1,V2 사용시 ORA-9241, ORA-9301 조치 방법

Bulletin no : 10129

ORA-9241, 9301 에러는 개발물이 해당 물 또는 BQL*NET에 필요한 매세지 파일을 찾을 수 없을 때 발생한다.

* BQL*NET V 1

1. BOB 환경에서 SET을 수행하여 CLIENT상에 CONFIG라는 환경변수가 설정되어 있는지 확인한다.

2. 설정되어 있지 않으면 BOOT directory에서 CONFIG.DBA 또는 DBACLE.INI라는 파일을 찾는다.
(Directory Search)

(상기의 파일은 보통 \DBACLE_HOME 또는 \WINDOWS directory 내에 있다.)

상기의 파일이 존재하면 BOB환경에서 다음의 CONFIG 환경변수를 추가한다.

```
C:\> SET CONFIG = \DBACLE.INI
```

이로써 문제가 해결될 수 있으며, BOB 환경에서 SET과 PATH 명령을 수행하여 DBACLE_HOME 디렉토리가 PATH에 설정되어 있는지, 그리고 CONFIG가 BOB환경에 설정되어 있는지 확인하고 WINDOWS를 실행해야 한다.

3. 환경변수 CONFIG가 이미 설정되어 있고 DBACLE.INI 또는 CONFIG.DBA를 지시하고 있는데도 ORA-9241 또는 ORA-9301 에러가 계속 발생한다면 WINDOWS를 완전히 빠져나와서 DBACLE.INI를 CONFIG.DBA 라는 또 다른 파일로 복사를 하고 BOB 환경에서 환경변수 CONFIG를 추가한다.

```
C:\> SET CONFIG = \CONFIG.INI
```

4. 만약 DBACLE.INI를 autotrac.bat 파일에 SET CONFIG = \DBACLE.INI 와 같이 추가하여도 계속 에러가 발생한다면 DBACLE.INI 파일에 LOCAL = t:\autotrac\id를 추가한다.

5. 단계2 또는 단계3 에서 문제가 해결된다면 이를 AUTOTRAC.BAT 파일에 SET CONFIG문을 추가한다.

* BQL*NET V 2

1. DBACLE.INI 파일에 LOCAL = 를 추가한다. 만약 DBACLE.INI 파일에 LOCAL 파라미터를 추가한 후에도 계속 ORA-9301 에러가 계속 발생한다면 autotrac.bat 파일에 SET CONFIG = \DBACLE.INI를 추가한다.

[주의 1] CONFIG가 DBACLE.INI를 지시하도록 설정하면 나중에 다시 설치할 문제가 발생할 수 있다. 그럴 때는 AUTOTRAC.BAT 에서 SET CONFIG 행을 삭제하고 다시 Booting 한후 설치를 시작한다.

[주의 2] HB ACCESS를 이용하여 DBACLE의 데이터를 읽어들일 경우는 환경변수를 다음과 같이 설정한다.

```
SET CONFIG_FILES = \DBACLE.INI
```

[주의 3] SET 다음의 CONFIG 나 CONFIG_FILES 는 반드시 대문자 이어야 한다.

WINDOWS용 SQL*NET 사용시 ORA-6106

Bulletin no : 10133

다음은 먼저 WINBOUND를 BQL*NET TCP/IP와 DBA-B10B를 간단하게 소개한 다음 DBA-B10B를 진단하는 방법을 설명하겠습니다.

* WINBOUND를 BQL*NET TCP/IP

WIN UI.K를 BQL*NET TCP/IP는 현재로서는 BQLTCP.BLL과 BQLTCP1.BLL 등 두개의 BLL이 한 세트가 되어 있습니다. 이들 BLL은 DBACLE를 먼저 스트림이 TCP/IP 프로토콜 스트림으로 변환되면 OCI BLL에 의해 작업 진행중에 돌려집니다. DBACLE INTERFACE BLL은 BQLTCP.BLL을 먼저 돌리려고 합니다. 이것이 실패하면 DBB TBB 버전의 드라이버를 찾습니다. 두 가지 모두 실패하면 DBA-3121 메시지가 나옵니다.

1. BQL*NET BLL은 돌려지자마자 HBDCCLIB.BLL(DBACLE에서 제공됨)이 돌려집니다.

이 BLL은 DBACLE.INI로부터 TCP_UENBDB 스트림으로 전달받습니다. 그 후, 이 HBDCCLIB.BLL은 TCP_UENBDB 스트림을 사용하여 항상 WINBOUND 디렉토리에 있는 UBL.INI라는 파일을 엽니다. UBL 파일은 JBH 호출을 각 업체 고유의 API에 대응시킵니다. UBL.INI가 없으면 HBDCCLIB.BLL이 초기화에 실패하여 DBA-B120 "NETTCP : network driver not loaded" 메시지가 나오게 됩니다. 이것은 고각이 다른 기계에서 BQL*NET BLL을 복사하거나 KCOPY 명령으로 DBAWIN 디렉토리 전체를 복사하기만 하고 이 UBL.INI 파일을 복사하지 않을 때 자주 발생합니다.

2. UBL.INI가 있다 하더라도 TCP_UENBDB 매개 변수에 DBA-B120 또는 DBA-B10B "NETTCP : socket creation failure"를 통탈할 수 있는 오류가 적지 않습니다.

3. 이 문제의 다른 원인은 HBDCCLIB.BLL이 대응을 제대로 설정할 수 있지만 HBDCCLIB.BLL API를 해당 업체의 TBAHBPDRT 인터페이스 또는 API에 결합하도록 글루(glu)에 지시할 수 없는 것입니다. 이 글루는 BQL*NET TCP/IP를 설치할 때 선택하는 TCP/IP 업체에 따라 BLL 또는 TBB일 수 있습니다. 예를 들면, WINBOUND로 들어가기 전에 글루 BLL이 실패할 수도 있고 (이때면 HMDULUP.BLL) 랜더 고유의 TBB(예를 들면 HFTP.EXE)이 돌려지지 않았을 수도 있습니다. 이 글루를 구성하는 BLL과 TBB의 목록은 다음과 같습니다.

```
. m3open.dll : 3Com 3+Open v2.0
. m3openx.dll : HP AMPA v2.0 & v2.1 그리고 Microsoft TCP UI.0(LAN
Manager v2.2의 전용)
. novniup.dll : Novell LAN WorkPlace v4.0, v4.01, & v4.1
. nycnf4.dll : Sun PC-NFB v4.0a
. nycnfup.dll : Hellingong Business TCP v1.1 & v1.2
. uclmsock.dll : UBL 일관형 windows sockets BLL 버전 1.1
. Frontier Technology Super - TCP v3.0
. NetManager Channeler TCP v3.1과 v3.22
. FTP PCTCP v2.2 - Winsock interface
```

```

(1989년 5월 4일자 WINBDCB.DLL)
. BSM PC/NFS 5.0 - Winsock Interface
(1989년 3월 2일자 WINBDCB.DLL)
. BOB용 IBM TCP v2.1.0.2
(1989년 3월 30일자 WINBDCB.DLL)
. n3sysm.exe : 3Com 3+Dsym v1.1 & v1.2 및 BOB용 Digital
Pathworks(TCP/IP) v4.1
. n42.exe : Bsmx 및 Utilmacidm TCP/IP v3.0
. nftp.exe : FTP PC/TCP v2.1 부러 v2.2까지
. nycsfk.exe : Bsm PC-NFS v3.0
. nycsfk2.exe : Bsm PC-NFS v3.5

-----
I Decide Win App I
-----

I
I (연결)
I

-----
I DCI 데이터 : I
I DBA6WIN.DLL 또는I
I DBA7WIN.DLL I
-----

I
I (진행중에 물려지며 진입점을 갖게 됨)
I (여기서 DBA-3121이 발생할 수 있음)
I

----- (DLL의 초기화 -- Winsock가 민헨스드 모드에
I BQLTCP1.DLL I 있지 않으면 실패함.
-- DBA-3121이 발생)
-----

I I
I (연결) I-----I (연결)
I I

-----
IDBA7WIN.DLL -- I INBDCB.DLL -- I
I BQLWEET에서 일부 I 모든 TCP/IP API의 I
I 찾아보기 루틴들 I 진입점 I
I 위해 사용됨 I
-----
I I
I (연결) I I
I I
-----
I COMBWIN.DLL I UBL.INI와 TCP_USERBOS를 사용하여 업제 특종의
-----
민터페이스(예를 들면, MODULUP.DLL)를 물리거나
민터페이스 TSB(예를 들면, HTTP.EXE)에 접속함.

```

여기서 Dna-B120 또는 Dna-B100이 발생할 수 있음.

4. 지원되는 TCP/IP 패키지 또는 버전을 설치하지 않은 것도 Dna-B100의 원인이 될 수 있습니다. 예를 들어서 TCP_VENDOR=PCNFB4를 가지고 있는 데 Sun PC-NFB v4.0을 실행하면, TCP_VENDOR=PCNFB4를 실행할 때 반드시 사용해야 하는 버전인 Sun PC-NFB v4.0a를 불러오기 때문에 dnm-B100이 발생할 수 있습니다. 우리는 Sun PC-NFB v4.0을 지원하지 않습니다. 지원되는 버전과 지원되는 TCP/IP 제품을 사용중인지 확인하십시오. (3단계 참조)

5. 지원되는 TCP/IP패키지와 버전을 설치 했더라도 TCP/IP 패키지에 필요한 실행 파일을 모두 설치하지 않았으면 Dna-B100이 발생할 수 있습니다. 예를 들어, BTH와 BINFILE(또는 NIB를 사용할 경우 BINNIB)를 불러오지 않았다면 PC-NFB를 사용할 때 Dna-B100이 발생할 수 있습니다. NICconnect TCP를 사용할 경우에는 반드시 BDCRETE, BKE를 불러야 합니다.

6. 또한 CONFIG.BYB의 FILEB와 BUFFERS를 충분히 높게 설정하지 않으면 Dna-B100이 발생할 수 있습니다. 다음은 이들 매개변수의 최소 요구사항입니다.

```
BUFFERS = 16
FILEB = 80
```

7. Dna-B100의 또 다른 원인은 실제 메모리의 부족입니다.

* Dna-B100을 진단하는 단계

1. TCP/IP 업체의 PING과 TELNET이 잘 작동하는 지 점검하십시오. 이들 유틸리티의 CUI 버전이 도움이 될 수 있습니다. 이들 유틸리티가 작동하지 않으면 TCP/IP 설치 액자를 참조하거나 TCP/IP 업체에 연락하십시오.

2. 이들 유틸리티가 작동하지 않으면 WINDOWS 디렉토리에 UBL.INI 파일이 있는 지 점검 하십시오. 만약, 없으면 WINDOWS를 BQLWNET TCP/IP를 다시 설치하거나 다른 곳에서 복사해 오십시오.

3. WINDOWS 디렉토리에 UBL.INI가 있으면 TCP/IP 업체를 마르게 선택 하면서 TCP/IP를 설치 했는지 점검 하십시오.

4. 제대로 설치했으면 DnaCLE.INI의 TCP_VENDOR 매개 변수를 읽어 보십시오. TCP_VENDOR의 값에 정확한 TCP/IP 패키지과 버전을 사용중인지 확인하십시오. 그런다음 UBL.INI로 가서 그 TCP_VENDOR의 프로파일을 살펴 보십시오. 프로파일의 "netmodule"은 무엇입니까? "netinterface"가 DLL이라면, 그 DLL을

찾을 수 있습니까? 필요할 때 WINDOWS가 BDB PATH를 통해 그 DLL을 불러올 수 있습니까? "netinterface"가 VB60이라면 WINDOWS로 들어가기 전에 반드시 그 VB6을 실제로 불러오십시오.

5. CONFIG.BYB의 FILEB와 BUFFERS를 충분히 높게 설정했는지 확인 하십시오.

다음은 이들 두 매개변수의 최소 요구사항입니다.

```
BUFFERS = 16
FILEB = 80
```

B. 앞서 말한 단계들 다 수행했는데도 여전히 DMA-BIOS이 감당하던 기존 메모리의 일부를 해제해 (불필요한 VRAM을 내리거나 일부 VRAM을 상위 메모리로 돌려서) 보십시오.

7. Microdot TCP/IP를 사용 한다면 TCPUTILB.INI의 BACKVB값을 점검하십시오. 값이 너무 낮으면 올리십시오. 또한 BACKVB, BKE를 호출했는 지도 확인하십시오.

B. WINDDMB를 통해 BIN PC-NFB US를 사용할 때 문제점이 생길 때가 있습니다. 그러한 경우에는 BACKSET TCP/IP를 다시 설치하고, BIN PC-NFB U4.Om(US가 아니다)를 업로드 선택해 보십시오. 이 때 BTH와 BMMFILE(또는 NIB를 사용할 경우 BMMNIB)를 반드시 돌려야 합니다.

WINDOWS용 SQL*NET 사용시 ORA-6108

Bulletin no : 10135

다음은 먼저 Windows용 SQL*Net TCP/IP과 ORA-6108을 간단하게 소개하고 글부분에서 ORA-6108을 진단하는 과정을 요약합니다.

* WINDOW용 SQL*NET TCP/IP 개요

1. 연결을 시도하는 중에 클라이언트 응용프로그램이 소켓을 열고 `Connect()`를 사용하여 원하는 서버를 찾습니다. `Destination`은 `oserv` Listener가 `Listening`하는 `machine`의 `port`와 `destination address`를 지정합니다. 이것의 디폴트는 십진 1525입니다.

```
-----
! 클라이언트          ! 서버의 IP 주소와 목적지의 포트번호, 즉 서비스
! 응용 프로그램이      ! 파일의 oserv를 가지고 Connect()를 합니다.
! 소켓 tx를 엽니다.   !
-----
```

2. 서버 응용 프로그램은 대개 호스트(여기서는 BB 소켓이라고 가정)에서 소켓을 연 다음, `Bind()` 호출을 통해 호스트 `machine`의 `listener` 주소와 수신 포트 번호를 소켓에 결합합니다. 이어 `Accept()`를 내서 수신 `Connect()` 요구를 받았다는 의사를 표시합니다. 그 후, `Listen()` Operation을 수행하는 다이브 루프(`tight loop`)로 들어갑니다. 이 `Listen()`은 TCP 세션 진행을 위한 클라이언트의 `Connect()`를 믿습니다.

```
-----
! 서버 응용 프로그램,  ! (a) 서버의 IP 주소 (또는 기계에서 이동할 수
! 즉 oserv Listener가 ! 있는 모든 IP주소) 와 수신포트번호, 즉
! 소켓 tx를 엽니다.   ! 기계의 /etc/passwd에서 가져온 oserv를
!                       ! 가지고 Bind()를 합니다.
-----
!                       ! (b) MAX_PENDING_CONNECTION_QUEUE_SIZE를 매개
!                       ! 변수로 하여 Accept()를 합니다. 이 매개
!                       ! 변수는 tcpd 프로세스에서 구성할 수 있는
!                       ! 경우도 있습니다. 서버에 대한 Connect()
!                       ! 요구가 채도할 때는 일부 Connect()요구를
!                       ! 큐에 넣어서 TCP/IP 데이터를 멈춥니다.
!                       ! (c) 루프 들어감 : Listen()을 합니다.
!                       ! TCP/IP는 미결 Connect() 요구가 없으면
!                       ! oserv 프로세스를 휴면 상태로 만듭니다.
!                       ! oserv는 Connect()가 그 Listen()과 쌍을
!                       ! 이룰 때 작업을 재개합니다.
-----
```

3. 서버는 `Listen()` 호출에서 작업을 재개할 때 특히 코드로서 하나의 추가 소켓을 더 가져옵니다. 이어 주어진 소켓을 제공하는 `DBALIS Bindw process`를 만들며, 이때 `Listener` 자체는 원래 소켓상에

서 `ListenIng` 루트로 되돌아갑니다.

```

----- (a) oserve가 소켓 Ex에서 Listen()을 내서 ListenIng
I DBABU I 합니다. 클라이언트 응용 프로그램의 Connect()
I Listen가 소켓 I (b) oserve는 Listen()으로부터 작업을 재개합니다.
I Ex를 엽니다. I 새 소켓 번호 Rx를 가져오고 fork() 만듭니다.
-----
페어언트는 소켓 Rx를 닫고 루트를 계속합니다.
차밀드는 이제 소켓 Ex를 닫고 나서 소켓 Rx로부터
recv()과 send()을 실행하며, 그 후로는 페어언트
프로세스를 계속하게 됩니다. BIB 참조가 일어나고
만약 성공하면 이 차밀드 oserve는 BIB에 대한
DBACLE 이미지와 겹쳐집니다. 이렇게 해서
클라이언트의 연결을 위한 DBACLE 복도 프로세스가
이루어집니다.

```

```

----- DBABU 차밀드 프로세스 -- 클라이언트에 대한
I DBACLE 복도 I DBACLE 복도 프로세스가 됩니다.
I 프로세스가 I <- BIB에 대한 DBACLE 이미지와 겹치기 전에
I 만들어집니다. I 차밀드가 이 소켓을 닫습니다.
-----
<- 복도 프로세스는 이 소켓을 계속합니다.
recv()을 사용하여 클라이언트로 부터 받음.
send()을 사용하여 클라이언트로 보냄.

```

4. `DBA-BIB`은 부적절한 `machin`, 또는 `machin`은 맞지만 틀린 포트를 지정할 때 발생합니다. 포트와 주소가 맞더라도 `TCP Listen`은 그냥 종료될 수 있습니다. 사용량이 많은 시스템에서는 `Listen`가 정상적으로 실행할 때에도 `DBA-BIB` 에러가 발생할 수 있습니다. 이것은 `TCP Listen`가 `Accept()` 호출 중에 해당 소켓에 대해 `MARK_PENDING_CONNECT_REQUEST`를 설정했을 수 있기 때문입니다. 결국 `TCP/IP` 에 이어는 모든 연결 요구를 `Listen`의 소켓 류에 넣을 수 없으므로 `DBA-BIB`이 발생할 수 있습니다.

5. `BQLWNET U1`은 `BQLWNET U1`으로만 송화한다는 점도 말 주어야 합니다. `BQLWNET U2`은 송화할 수 없습니다.

8. 네트워크가 아주 혼잡하고 호스트에 도달하려는 중에 시간이 종료할 경우에도 `DBA-BIB`이 나올 수 있습니다. `BQLWNET`가 기하기까지 연결을 재시도할 횟수를 알려주는 `conn_retrytime`라는 `BQLWNET TCP/IP` 연결 스트림에서 `conn_retrytime`를 지정하는 구문은 다음과 같습니다.

```

tsockt(/dev/ice):BIB[, buffer_size][:conn_retrytime]

```

* `DBA-BIB`을 진단하는 단계

1. 클라이언트에서 호스트 `machin`에 대해 `ping`을 실행하십시오. 대부분의 PC `TCP/IP`업체는 "`ping`" 유틸리티를 제공합니다. 클라이언트 `machin`에서 다음을 입력하십시오

```

ping

```

이 방법으로 잘 되지 않으면 아마도 호스트 `machin`이 다운된 것입니다. IP 주소를 사용하여 호스트

에 대해 ping을 성공적으로 실행할 수 없으면, 서버의 호스트 이름을 사용하여 ping을 실행해 보십시오.

ping

호스트 이름을 사용하여 ping을 실행할 수 없으면 TCP/IP 구성을 점검하십시오. 호스트 이름을 가지고 ping을 실행할 수 없으면, 연결 스트림에 호스트 이름을 사용하여 BQLWNET과 연결할 수 없습니다.

2. BQLWNET TCP/IP Listener가 해당 서버에서 실행중인지 점검하십시오. 서버의 UNIX프롬프트에서 다음을 입력하면 됩니다.

```
ps -all grep "oracle"
```

이 때 최소한 한 행이 표시되어야 합니다. 그렇지 않으면 UNIX 프롬프트에서 "oracle" 또는 "tcpctl start"를 입력하여 수화자를 띄우십시오. BYBAMIN 특권을 가지고 해당 기계에 로그인해야 합니다.

3. 서버측에서 루트락을 할 수 있는 지, 다시 말해서 PC 클라이언트에서 지정한 것과 같은 연결 스트림을 사용하여 서버의 출을 연결할 수 있는 지 점검하십시오. 예를 들면, 서버의 BQLPUB 또는 BQLBBA를 호출하고 서버의 BQLPUB 또는 BQLBBA 프롬프트에서 다음을 입력하십시오.

```
CONNECT BQLBBA/PA8BQDBB4t:/;
```

4. 루트락 검색되면 호스트 서버의 BQLBBA 포트 번호를 확인하십시오.

(대부분의 기계에서 BQLBBA 파일은 /etc 디렉토리에 있습니다.) 또한 "tcpctl" 용집리리를 사용하면 대부분의 UNIX 기계에서 BQLBBA Listener를 시작하거나 멈출 수 있습니다. "tcpctl stop"로 Listener를 종료하십시오. "tcpctl start" 으로 다시 시작하십시오. 이때 시작 포트에 관한 정보가 표시됩니다.

5. 이것이 성공하면 포트를 지정하지 않고 포트를 연결해 보십시오.

```
t::
```

연결되지 않으면 클라이언트에서 BQLBBA 파일을 정확하게 설정하 않았기 때문입니다.

a) WINDBBAMIN.INI를 점검하여 [Oracle] 부분의 ORA_CONFIG 매개 변수가 어떤 구성 파일을 지시하고 있는지 알아보십시오. 이дол르는 다음과 같습니다.

```
[Oracle]
ORA_CONFIG=C:\WINDBBAMIN\ORACLE.INI
```

b) ORACLE.INI 파일을 보고 TCP_BQLBBA_FILE 매개변수가 설정되었고 BQLBBA 파일을 지시하고 있는지 확인하십시오.

c) BQLBBA 파일을 보고 다음 항목이 있는지 확인하십시오.

```
oraclev 1525/tcp oracle
```


7. 또한 서버가 BQLWNET U2가 아니라 BQLWNET U1을 실행중인지 확인하십시오.

B. 걸 스프링의 제시도 매개변수를 증가시켜 보십시오. 제시도 횟수를 지정하는 구문은 다음과 같습니다.

```
tihost[/comvc]:BIB[,buffer-size][:com-rate/sec]
```

com-rate/sec의 디폴트는 1입니다.

B. UAK에 연결할 경우에는 UAK config.ora 파일에 다음행이 있는지 확인하십시오.

```
BQLWNET (BQLWNET)ORAProc
```

이것은 UAK account가 없는 PC가 디폴트 사용자 account를 사용하여 연결 할 수 있게 해 줍니다.

WINDOWS용 SQL*NET 사용시 ORA-6110

Bulletin no : 10138

ORA-6110과 버퍼 크기의 개요

ORA-6110 "NETTCP: message send failure"는 대개 Windows 클라이언트의 TCP/IP상에서 버퍼 조정문제가 있을 때 발생합니다.

버퍼 크기를 이해하기 위해 BQL*NET을 통해 응용 프로그램과 원격 데이터베이스 사이의 데이터 흐름을 살펴보려고 합니다. 클라이언트측에서 응용 프로그램은 BQL*NET에 데이터를 전달하고, BQL*NET은 그 데이터를 기판 TCP/IP 계층에 전달하며, TCP/IP 소프트웨어는 그 데이터를 네트워크에 전달합니다. 서버측에서 TCP/IP 소프트웨어는 네트워크에서 데이터를 선점하여 BQL*NET에 전달하고, BQL*NET은 그 데이터를 데이터베이스에 전달합니다. 데이터베이스가 데이터를 다시 응용 프로그램에 보낼 때의 과정 이와 반대입니다.

망목에서 BQL*NET TCP/IP는 데이터를 다음 계층에 실제로 전달하기 전에 메모리에 저장해 놓습니다. 이와 같이 BQL*NET 버퍼 크기까지 데이터를 저장했다가 전달합니다.

클라이언트가 서버에 연결할 때 한 쪽의 BQL*NET 계층은 먼저 버퍼 크기를 조정해야 합니다. 망목의 기존 BQL*NET 버퍼 크기는 서로 다를 수 있으며, 어느 한 쪽이 다른 쪽에서 다를 수 있는 BQL*NET 보다 많은 데이터를 보내지 않도록 하기 위해 둘 중의 더 작은 크기를 사용합니다.

BQL*NET TCP/IP 가 버퍼를 사용하듯이 기판 TCP/IP 소프트웨어도 버퍼를 사용합니다. 이 버퍼 조정이 어떤 이유에서든 잘못 이루어지면 ORA-6110이 발생할 수 있습니다.

[해결책]

ORA-6110의 해결책은 버퍼 크기를 연결 스트림에 포함시켜 밀접한 크기로 고정하는 것입니다.

예:

연결 스트림에 버퍼 크기를 포함시킨 후에도 여전히 ORA-6110이 발생하면 더 작은 값을 사용해 보십시오. WINBQUE용 BQL*NET TCP/IP의 기본 버퍼 크기는 4096입니다. 이것을 1024로 사용하면 대개 ORA-6110에러가 없어집니다.

[서버측]

1. 서버에서 루트락을 수행할 수 있습니까? 다시 말해서 PC 클라이언트에서 지정한 것과 같은 연결 스트림을 사용하여 서버의 쿼리를 연결할 수 있습니까? 예를 들어 서버에서 BQLPLUS 또는 BQLDBA를 호출하고 에러가 없어집니다.

CONNECT USERNAME/PASSWORD::

부트락을 실행하면 실제 문제를 더 잘 보여주는 다른 예제가 나타나는 수도 있습니다.

2. DBA-B110은 Oracle실행 파일에 사용 권한이 정확하게 설정 되어 있지 않으면 Unix 서버에 연결할 때도 발생할 수 있습니다. Oracle과 oracle의 사용권한은 다음과 같이 설정되어야 합니다.

Oracle v8의 경우:

Unix 프롬프트에서 다음 두 행을 입력하십시오.

```
chmod 4755 oracle
chmod 4755 oraclev
```

이 때, Permissions는 다음과 같이 설정됩니다.

```
-rwxr-xr-x 1 oracle dba ...size ...date oracle
-rwxr-xr-x 1 root dba ...size ...date oracle
```

Oracle v7의 경우

Unix 프롬프트에서 다음 두 행을 입력하십시오.

```
chmod 8755 oracle
chmod 8755 oraclev
```

이 때, Permissions는 다음과 같이 설정됩니다.

```
-rwxr-xr-x 1 oracle dba ...size ...date oracle
-rwxr-xr-x 1 root dba ...size ...date oracle
```

DBA-B110과 관련된 네트워크 주소

TCP/IP 네트워크에서 올바른 IP주소가 살아 있으면 DBA-B110이 발생할 수 있습니다. 네트워크의 모든 IP주소가 고유의 것인지 확인하십시오.

WINDOWS용 SQL*NET 사용시 ORA-6120

Bulletin no : 10137

Windows용 BQL*NET TCP/IP과 DBA-6120에 대해서 간단하게 소개하고 DBA-6120을 진단하는 방법들 요약합니다.

* WINDOWS용 BQL*NET TCP/IP

WIN U1.K용 BQL*NET TCP/IP는 현재로서는 BQLTCP.BLL과 BQLTCP1.BLL을 두 개의 BLL이 한 세트에 포함되어 있습니다. 이들 BLL은 DBACLE용 연결 스트림이 TCP/IP 프로토콜 스트림으로 확인되면 작업 진행중에 DCI BLL에 의해 불러집니다. DBACLE INTERFACE BLL은 BQLTCP.BLL을 먼저 불러들여 줍니다. 이것이 실패하면 DBB TSB 버전의 드라이버를 찾습니다. 두 가지 모두 실패하면 DBA-6121 메시지가 나옵니다.

1. BQL*NET BLL이 불러지지 않자마자 HBDCCLIB.BLL(DBACLE에서 제공됨)이 불러집니다. 이 BLL은 DBACLE.INI로부터 TCP_VENDOR 스트림을 전달받습니다. 그 후, 이 HBDCCLIB.BLL은 TCP_VENDOR 스트림을 사용하여 WINDOWS 디렉토리에 있는 UBL.INI라는 파일을 읽습니다. UBL 파일은 JB8 호출을 각 업체고유의 API에 대응시킵니다. UBL.INI가 없으면 HBDCCLIB.BLL을 복사하거나 KCOPY 명령으로 DBAWIN 디렉토리 전체를 복사하기만 하고 이 UBL.INI 파일을 복사하지 않을 때 자주 발생합니다.

2. UBL.INI가 있더라도 TCP_VENDOR 매개변수에 오류가 생길 수 있습니다. 예를 들어 WINDOWS용 BQL*NET TCP/IP U1.1.7.5에서 VENDOR 스트림의 오른쪽에 쓸데 없는 스페이스가 있으면 UBL.INI에서 Mapping을 분석할 때 HBDCCLIB.BLL이 실패하게 됩니다.

3. 이 문제의 다른 원인은 HBDCCLIB.BLL이 Mapping을 제대로 접할 수 있지만 HBDCCLIB.BLL API를 해당 업체의 TBAIMPORT INTERFACE 또는 API에 결합하도록 글루(CLUK)에 지시할 수 없을 때 발생합니다. 이 글루는 BQL*NET TCP/IP를 설치할 때 선택하는 TCP/IP 업체에 따라 BLL(이들때면 HBDCCLIB.BLL)이 실패할 수도 있고 랜더고유의 TSB(예를 들면 HTTP, BKE)이 불러지지 않았을 수도 있습니다. 이 글루를 구성하는 BLL과 TSB의 목록은 다음과 같습니다.

```
. n3cpm.dll : 3Com 3+Dpm v2.0
. n3cpm1.dll : HP AMP v2.0 & v2.1 그리고 Microsoft TCP U1.0(LAN
Manager v2.2의 전용)
. novellup.dll : Novell LAN WorkPlace v4.0, v4.01, & v4.1
. npsmf4.dll : Sun PC-NFB v4.0a
. npsmf4p.dll : Mullongong Bulletin TCP v1.1 & v1.2
. npsmf4p1.dll : UBL 일관형 windows sockets BLL 버전 1.1
. Frontier Technology Super - TCP v3.0
. NetManager Channeler TCP v3.1과 v3.22
. FTP PC/TCP v2.2 - Winsock Interface
(1989년 5월 4일자 WINBDC.BLL)
. SUN PC/NFB 5.0 - Winsock Interface
(1989년 3월 2일자 WINBDC.BLL)
```


4. DMA-B120 에러를 없애려면, 먼저 TELNET, PING 등 TCP/IP 유틸리티가 잘 작동하는 지 점검하십시오. 이들 유틸리티가 작동하지 않으면 TCP/IP 업체에 연락하거나 TCP/IP의 설치 액자를 읽어보아야 합니다.

5. 그렇지 않으면 WINDOWS 디렉토리에 UBL.INI가 있는지 점검하고, DBACLR.INI의 TCP_VENDOR가 무엇인지 알아 보십시오. 그런다음 UBL.INI에서 VENDOR의 프로파일을 찾아보십시오. 다음은 UBL.INI의 일부분입니다.

```
[ vcl-transport ]
3opm="3Com 3+Opus TCP"
3opm2="3Com 3+Opus TCP version 2.0"
pcmfcd="Bus PC-MFB v4.0"
.....

[ 3opm-transport ]
protocol=logio,telnet,ssh,rsync
dllmodule=MSDCCLIB.DLL
netmodule=MSOPM
netinterface=VB0
ipprivate=

[ pcmfcd-transport ]
protocol=logio,telnet,ssh,rsync
dllmodule=MSDCCLIB.DLL
netmodule=PCMFCD.DLL
netinterface=0LL
ipprivate=
```

글루 모듈이 DLL이라면, WINDOWB에서 그것을 불러올 수 있는지 확인하시고 글루 모듈이 VB0이라면 그것은 WINDOWB를 시작하기 전에 불러져 있는지 확인하십시오.

B. CONFIG.BYB의 FILES와 BUFFERS를 충분히 높게 설정하지 않아도 DMA-B120이 발생할 수 있습니다. 다음은 이들 두 환경변수에 대한 최소 요구사항입니다.

```
BUFFERS=18
FILES=80
```

7. 일관형 WINDOWS 소켓 인터페이스(HMINBDCR.DLL)를 사용한다면, 업체 특유의 MINBDCR.DLL 파일이 손상되거나 버전이 틀릴 경우 DMA-B120이 발생할 수 있습니다.

DMA-B120을 진단하는 단계

1. TCP/IP 업체의 PING이나 TELNET이 잘 작동하는 지 점검하십시오. 이들 유틸리티의 GUI 버전이 도움이 될 수 있습니다. 이들 유틸리티가 작동하지 않으면 TCP/IP 설치 액자를 조사하십시오.

2. 이들 유틸리티가 잘 작동하면 WINDOWS 디렉토리에 UBL.INI 파일이 있는지 점검하십시오. 만약 없으면 WINDOWB를 BQUNET TCP/IP를 다시 설치하거나 다른 곳으로 복사해 오십시오.

3. WINDBG 디렉토리에 URL.INI가 있으면 TCP/IP 업체에 대한 빠른 선택 하면서 BQLNET TCP/IP를 설치했는지 점검하십시오.

4. 제대로 설치 되었으면, ODBC.INI의 TCP_VENDOR 매개변수를 읽어 보십시오. 그다음 URL.INI로 가서 그 TCP_VENDOR의 프로파일을 살펴보십시오. 프로파일의 "module"은 무엇입니까? "module"이 DLL이라면, 그 DLL을 찾을 수 있습니까? 필요할 때 WINDBG는 ODBC PATH를 통해 그 DLL을 올릴 수 있어야 합니다. "module"이 YBB이라면 WINDBG로 들어가기 전에 반드시 그 YBB를 실제로 올리십시오.

5. CONFIG.BYB의 FILES와 BUFFERS를 충분히 높게 설정했는지 확인하십시오.

다음은 이들 두 매개변수의 최소 요구사항입니다.

BUFFERS = 18

FILES = 80

6. TCP/IP 업체가 winsock(Winsock 소켓)을 사용한다면 정확한 또는 최신버전의 winsock.dll을 가지고 있지 않을 경우 CFP 또는 ODB-BI20이 발생할 수 있습니다. 다음 표는 밑 winsock TCP/IP 업체가 공급하는 몇 가지 WINBDC.BLL입니다. 여기에 나열하지 않았지만 우리가 지원하는 winsock TCP/IP 업체라면, 그 업체로부터 최신의 winsock.dll을 구해다가 쓰는 것이 최선입니다. 그러나 TCP/IP 제품들의 winsock.dll 복사본을 구하려면 해당 TCP/IP 업체에 직접 연락하십시오.

제품이름	WINBDC.BLL 버전 또는 일자
FTP PC/TCP V2.2	1993년 5월 4일자 V1.0B
IBM TCP/IP V2.1	1993년 3월 30일자
Bul PC-WF8 V5.0	1993년 3월 2일자
SCDM TCP/IP V2.1	1993년 5월 13일자

예:FTP의 PC/TCP V2.2를 사용하고 1993년 3월 10일자 winsock.dll을 사용하면, 실행중인 프로그램이 PCCTCPAPI.BLL에서 CFP를 야기한다는 예러가 발생할 수 있습니다. winsock.dll을 1993년 5월 4일자로 교체하면 문제가 해결됩니다.

주: 또한 다른 디렉토리에 winsock.dll의 이전 복사본이 없는 것을 확인하십시오. 그 드라이브의 루트 디렉토리에 "DIR WINBDC.BLL/*" 명령을 내면 특정 드라이브의 모든 디렉토리를 점검할 수 있습니다. 검토에 있는 모든 드라이브에 해 이것을 수행해야 합니다.

WINBDC.BLL의 다른 모든 복사본은 WINDBG가 용요한 WINBDC.BLL 대신 그것을 올릴 루터가 있으므로 이름을 바꾸어 주어야 합니다.

7. FTP PC/TCP V2.2이상을 사용할 경우에는 반드시 system.ini에 vpcitcp386 드라이버가 있어야 합니다.

```
...
[386Emul]
device = (path)\vpcitcp.386
...
```

8. URL 입란형 Winsock 소켓 인터페이스를 사용할 경우에는 WINBDC.BLL이 손상 되지 않은 것을 확인해야 합니다.

WINDOWS용 SQL*NET 사용 시 ORA-6122

Bulletin no : 10139

ORA-6122 "NETCP: setup failure"는 SQL*NET 구성이 적절하게 설정되지 않은 상태에서 WINDBUS용 SQL*NET TCP/IP를 가지고 연결하려 할 때 발생합니다. 구성을 점검하여 다음 사항을 제대로 설정했는지 확인하십시오.

1. WINDBUS\BIN.INI를 조사해 보십시오. ORA_CONFIG 매개 변수를 정의하는 DBACLE 부분이 있어야 합니다

```
[DBACLE]
ORA_CONFIG=C:\WINDBUS\DBACLE.INI
```

2. DBACLE.INI(또는 ORA_CONFIG 매개변수가 지시하는 파일)를 보십시오. DBACLE_HOME이 WINDBUS용 SQL*NET TCP/IP와 다른 Oracle Windows 응용 프로그램이 실행된 디렉토리를 지시하는 지 확인하십시오.

디폴트는 다음과 같습니다.

```
DBACLE_HOME=%DBASIM
```

3. 또한 DBACLE.INI에 TCP_VENDOR를 정확하게 설정했는지도 확인하십시오.

종료한 열쇠 키 목록은 WINDBUS용 SQL*NET TCP/IP설정의 표 3-4 '열쇠키'를 참조하십시오.

4. 경로에 C:\DBASIM\BIN(또한 DBACLE_HOME를 설정한 BIN 디렉토리)이 있는지 확인하십시오. BOB 프롬프트에서 SET을 입력하고 줄 누르면 됩니다. 이 명령은 경로를 보여줍니다.

5. DBASIM\BIN 디렉토리에 BQYTCP.BLL과 BQYTCP2.BLL이 모두 있는지 확인하십시오.

6. 경로의 다른 어떤 디렉토리에든 BQYTCP.BLL이 없는 것을 확인하십시오.

7. DBASIM\BIN 디렉토리와 경로의 다른 디렉토리에 HBCCLIB.BLL이 있는지 확인하십시오. 또한 파일의 주 특성을 가지고 있지 않도록 하십시오.

특성이 줄일 경우, 이전 특성의 이름을 하 연 문자가 줄어들 수 있습니다.

8. WINDBUS 디렉토리에 UBL.INI 파일이 있는지 확인하십시오. 만약 없으면 DBACLE INSTALL을 통해 SQL*NET을 다시 입력하십시오.

WINDOWS용 SQL*NET 사용시 TNS-12203

Bulletin no : 10139

TNS-12203 "TNS:unable to connect to destination"은 여러가지 원인 때문에 발생할 수 있습니다. 여기서는 가장 일반적인 몇 가지 원인을 설명합니다.

1. WINDOWS용 TCP/IP 어댑터를 설치하지 않은 상태에서 연결하려 하면 TNS-12203이 발생할 수 있습니다. BQL*NET TCP/IP U2는 BQL*NET U2와 U2 TCP/IP 어댑터 등 두가지 제품으로 구성됩니다. 이들은 별도의 두 키트로 되어 있는데, 반드시 두 키트를 모두 설치해야 합니다.

2. TNS-12203 에러는 WINDOWS용 ORACLE BQL*NET소프트웨어가 ORACLE 홈 디렉토리를 찾을 수 없다는 의미일 수 있습니다. 이 문제를 해결하려면, 먼저 WIN.INI파일의 ORACLE 부분에 다음 항목이 있는지 확인하십시오.

```
[ORACLE]
ORA_CONFIG=C:\WINDOWS\ORACLE.INI
```

WINDOWS디렉터리가 C:\WINDOWS가 아니면, 위 행의 C:\WINDOWS 부분을 그 이름으로 바꾸십시오. 그런 다음, ORACLE 소프트웨어를 다시 설치하십시오. ORACLE.INI 파일이 있으면 그 ORACLE.INI파일에 다음 행이 있는지 확인하십시오.

```
ORACLE_HOME=C:\ORACLE
ORACLE 홈 디렉터리가 C:\ORACLE이 아니면 위 행의 C:\ORACLE 부분을 그 이름으로 바꾸십시오.
```

3. TNSNAMES.ORA가 ORACLE 홈 디렉터리 아래의 NETUSER\ADMIN에 있는지 확인하십시오. ORACLE 홈 디렉터리는 ORACLE.INI(또는 WIN.INI의 ORA_CONFIG매개변수가 지시하는 파일)의 ORACLE_HOME 에 의해 정의됩니다.

4. 서버측에서 실행중인 BQL*NET U2 TCP/IP Listener가 없어도 TNS-12203이 발생할 수 있습니다. 서버측에서 실행중인 BQL*NET U2가 있는지 확인하십시오. 서버측에서 Listener Control 유틸리티 (LSNRCTL)를 사용하면 서버의 U2 Listener가 실행중인지 확인할 수 있습니다. 서버에서 "LSNRCTL STATUS"명령을 실행하십시오. Listener Control이 유틸리티 대해서는 BQL*NET Administrator's Guide를 참조하십시오.

5. 연결할 SERVICES 이름에 대해 CONNECT DESCRIPTOR에 정확한 ADDRESS 매개변수를 지정했는지 확인하십시오. 정확한 ADDRESS 매개변수를 사용중인지 확인하는 방법은, WINDOWS 클라이언트에서 사용할 것과 같은 ADDRESS 매개 변수를 가진 TNSNAMES.ORA를 사용하여 서버에서 쿼리를 해 보는 것입니다. 쿼리를 수행한다는 것은 데이터베이스와 같은 기계에서 BQL*NET 등을 호출하고 연결 스트림을 지정함으로써 BQL*NET U2를 통해 연결한다는 뜻입니다.

6. JDB URL 소켓이 초기화되지 않으면 TNS-12203이 발생할 수 있습니다. 이 문제를 해결하려면 다음 사항을 점검하여 JDB URL 데이터가 정확하게 초기화되었는지 확인하십시오.

1) DBACLE.INI 파일(또는 WIN.INI의 DBACLE_CONFIG 매개변수가 지시하는 파일)의 업로드 매개변수 TCP_VENDOR가 정확하게 설정되었는지 확인하십시오.

2) DBACLE_HOME\BIN 디렉토리에 HSDCELIB.BLL이 있는지 확인하십시오.

3) DBACLE_HOME\BIN 디렉토리에 선택된 JDB 업로드 종류의 BLL이 있는지, 또는 그 JDB 업로드 종류의 TBL 파일이 실행되는 지 확인하십시오.

4) WINODBS 홈 디렉토리에 UBL.INI 파일이 있는지 확인하십시오.

7. TMB-12203에 이어 실제 문제가 무엇인지 더 자세하게 나타내 주는 또 다른 예머가 발생할 수 있습니다. 화면에서 다른 예머가 보이지 않으면

DBACLE_HOME\NET\ODBS\LOC 디렉토리의 BQLNET.LOC 파일을 점검하십시오.

WINDOWS용 SQL*NET 사용시 TNS-12533

Bulletin no : 10140

TNS-12533에러는 대개 TNSNAMES.ORA에서 기술자의 ADDRESS 부분에 프로토콜 CONNECT DESCRIPTION의 매개 변수가 틀릴 때 발생합니다. TCP/IP 어댑터의 경우, 프로토콜 목록의 매개 변수는 다음과 같습니다.

```
(ADDRESS =
  (COMMUNITY=community_name)
  (PROTOCOL=TCP)
  (HOST=host_name 또는 ip_address)
  (PORT=port_number)
)
```

TCP/IP 어댑터를 사용할 때 TNSNAMES.ORA 의 완전한 구문은 다음과 같습니다.

구문	해설
<pre> service_name (DESCRIPTION= (ADDRESS_LIST= (ADDRESS= (COMMUNITY=community_name) (PROTOCOL=TCP) (HOST=host_name 또는 ip_address) (PORT=port_number))) (CONNECT_DATA= (SID=sid))) </pre>	ADDRESS_LIST는 생략할 수 있다. COMMUNITY는 생략 수 있다.

TNSNAMES.ORA에 정의된 서버에 연결하려면, 연결중인 응용 프로그램의 Log On dialog box에서 호스트 스크림 필드에 연결 스크림 "TNS:service_name"을 지정하십시오. 또한 WINBOUNWIN.INI의 DBA_CONFIG가 지시하는 파일에 LOCAL 매개 변수를 지정하여 기존 데이터베이스를 정의하는 것이 좋습니다. DBA_CONFIG는 대개 WINBOUNDBACLE.INI를 지시합니다. DBACLE.INI에서 "LOCAL=TNS:service_name"행을 추가할 수 있습니다. 그 후, 연결을 할 때는 호스트 스크림 필드를 비워 둘 수 있습니다.

WINDOWS용 SQL*NET 사용 시 TNS-12545

Bulletin no : 10141

예제 TNS-12545 "TNS: name lookup failure"는 TCP/IP 프로토콜 어댑터를 가진 BQLNET V2를 통해 클에서 연결하려는 데 TCP/IP 프로토콜 어댑터가 TNSNAMEB.DMA 파일의 ADDRESS 부분에 제공된 호스트 이름을 분석할 수 없을 때 발생합니다.

TNS-12545는 BQLNET TCP/IP V1에서 발생하는 DBA-BIOS "NETTCP: remote host is unknown"과 비슷합니다.

* TNS-12545를 진단하는 단계

1. 호스트 이름을 이용하여 호스트에 대해 TELNET과 PING을 실행해 보십시오. 즉 PING 또는 TELNET <server>를 실행 해 보십시오. 호스트 이름을 통하여 PING이나 TELNET을 성공적으로 실행할 수 없으면, 먼저 TCP/IP 구성을 점검해야 합니다. PING나 TELNET의 WINDOWS버전을 사용하여 해 보십시오. 몇몇 BOB 버전이 잘 작동하더라도 WINDOWS 구성에 문제가 있으면 WINDOWS버전에서 예제가 발생할 수 있습니다.

2. 버에서 루트락을 수행하여 서버측이 정확하게 설정되었는 지 확인하십시오. 루트락을 수행하려면 데이터베이스가 있는 MACHINE에서 아주 줄이나 호출하고, TNSNAMEB.DMA에 WINDOWS 클라이언트에 사용한 것과 같은 ADDRESS 매개 변수를 사용하여 BQLNET V2를 통해 데이터베이스에 연결해 보십시오.

3. 다음에는 사용자의 TCP/IP 소프트웨어가 IP 주소의 이름 분석을 위해 PC에서 DOMAIN NAME SERVICE(DNS)를 사용하는 지 아니면 파일 찾아보기를 사용하는 지 점검하십시오.

4. DNS를 사용한다면, DNS에 해당하는 모든 드라이버를 올리도록 하십시오. 예를 들어, SUN PC/NFB는 BIND.B, EXE를 올리야만 Naming Information Service를 사용할 수 있게 됩니다.

5. DNS를 사용하지 않는다면, HOST 파일에 연결해야 할 호스트의 항목이 있는 지 확인하고, 만약 없으면 추가하십시오. WINDOWS용 TCP/IP 어댑터는 원 TCP/IP 업체가 예상한 디렉토리에 상주하는 HOST 파일을 사용합니다. HOST 파일의 각 항목 뒤에는 반드시 커리지 미션을 넣어야 합니다. 또한 그러한 서비스에 필요한 드라이버를 모두 점검해야 합니다. 예를 들어, SUN PC/NFB는 그러한 서비스를 올리려면 BIND.B, EXE 필요합니다.

6. WINDOWS 호출을 사용하지 않고 net() 호출을 사용했기 때문에 파일 핸들이 초과할 수 있습니다. BQLNET V1은 여기에 해당하지 않습니다. 이 버그는 버전 2.1.5에 와서 해결되었습니다.

7. 연결을 시도하여 여전히 TNS-12545가 나오면, TNSNAMEB.DMA 파일의 ADDRESS 부분 아래에 있는 HOST 매개 변수에 호스트 이름을 지정하지 말고 IP 주소를 지정하는 것도 한 가지 해결책이 될 수 있습니다. 다음 예를 참조하십시오.

HOST=13B,141,B1,B4

WINDOW용 SQL*NET 를 사용 시 ORA-3121

Bulletin no : 10121

다음은 ORA-3121을 해결하기 위해 단계별 조치방법을 설명하겠습니다.

* BQLENET에서 ORA-3121

BQLENET U2를 통해 연결하려 할 때 연결 스크립에 't:=net접두어를 사용하지 않으면 ORA-3121이 나오게 됩니다. 구버전인 BQLENET U1의 net 접두어 (BQLEnet TCP/IP에 대한 "t: "을 사용하지 않도록 주의하십시오.

* BQLENET U10에서 ORA-3121

1. Windows에서 나와서 DOS 프롬프트로 가십시오.

2. DOS 프롬프트에서 PATH를 입력하고 엔터를 누르십시오. 현재 경로에 있는 모든 드라이브를 적어 주십시오. (C:, D:)

3. 2에서 적은 DOS 경로의 모든 드라이브에서 DBASUIM.BLL 이라는 파일을 찾으십시오. (Windows에서 FILEFIND 유틸리티를 사용하거나 DOS 5.0 이상의 경우 루트 디렉토리에서 간단하게 DIR DBASUIM.BLL/8를 실행하는 것이 좋은 방법입니다. Windows 파일 관리자의 파일 찾기 유틸리티를 사용할 수도 있습니다.)

DBASUIM.BLL이 여러개이면 한 가지만 제외하고 모든 복사본의 이름을 바꾸십시오. (DBASUIM.BLL같은 것으로). 한 가지 예외는 4/13/82 (285573 bytes) 또는 10/15/82 (380838 bytes)라고 적힌 DBASUIM.BLL은 복사본입니다. 이들 두 버전의 DBASUIM.BLL은 Oracle Corporation이 직접 배포한 것입니다. 10/15/82이후 날짜로 되어있고 380838 바이트보다 큰 파일을 가지고 있다면 그것을 사용해도 좋습니다.

(이들 날짜의 DBASUIM.BLL 복사본이 없으면 제3자 응용 프로그램 업체에 연락해서 이 파일의 복사본을 구하십시오. 디스크 데이터가 "Windows Required Support Files Version B.0xx.x.x"인 Oracle제품이라면, 유틸리티 DBASUIM.BLL의 최신 복사본이 있을 것입니다.)

4. DBASUIM.BLL이 있는 디렉토리를 적어 주십시오.

5. 현재 경로에 앞서 말한 디렉토리가 있는 지 확인하십시오. DOS프롬프트에서 PATH를 입력하고 엔터를 누르면 됩니다.

6. 현재 경로에 DBASUIM\IN 디렉토리가 있는 지 확인하십시오. (또는 Windows를 BQLENET에 설치된 곳, 잘 모르겠으면 DBACLE_HOME\IN 디렉토리를 보십시오. DBACLE.INI 파일에 DBACLE_HOME이 적혀 있습니다.) 또한 DBACLE_HOME\IN 디렉토리에 BQLENET BLL이 있는지 확인하십시오. (예를 들어, Windows를

BQLENET.BIN의 경우는 BQLEPC.BLL과 BQLEPCI.BLL, WINBOUB용 BQLENET.TCP/IP의 경우는 BQLETCP.BLL과 BQLETCPI.BLL), DBACLE_HOME\BIN 디렉토리에 DBA7WIN.BLL이 있는 지도 확인해야 합니다.

7. WINBOUB 디렉토리의 WIN.INI 파일을 편집하십시오. 다음과 같은 DBACLE이라는 구를 반드시 넣어 야 합니다.

```
[Database]
DBA_CONFIG=C:\WINBOUB\DBACLE.INI
```

B. A) DBB 프롬프트에서 BBT를 다시 입력하고 엔터를 누르십시오.

CONFIG과 DBB환경 변수가 CONFIG.DBA라는 파일에 존재하도록 설정되어 있는지 점검하십시오. CONFIG 환경 변수는 다음과 같아야 합니다.

```
CONFIG=C:\DBACLE\CONFIG.DBA
```

B) 환경 변수에 CONFIG 변수가 설정되어 있지 않으면 AUTOEXEC.BAT 파일에 다음 명령문을 추가하 고 나서 다시 부팅하십시오.

```
<server : unix, tool : windows>
```

```
BBT CONFIG=C:\WINBOUB\DBACLE.INI
```

(Windows 디렉토리에 해당 경로에 넣으십시오)

CONFIG가 설정되지 않아서 BB) 단계를 수행했으면 B단계는 무시하십시오.

B. CONFIG.DBA와 DBACLE.INI FILE의 다음 매개변수를 일치 시키십시오. (이들 파일의 위치에 대해서 는 B단계를 참조하십시오.) 이들 매개변수 중에는 두 파일에 없는 것도 있습니다. 이들 매개 변수가 CONFIG.DBA나 DBACLE.INI에 있으면 정확히 일치하게 하십시오. 이들 매개변수 중에 두 파일 중 어디 에도 없는 것이 있다면, 그것을 추가하지 마십시오. 또한 이들 매개변수 중에 불필요한 것이 있을 수 도 있지만, 그것을 CONFIG.DBA와 DBACLE.INI에 모두 추가해도 전혀 문제는 없습니다.

매개변수 목록 :

```
WIN_LOCAL_SESSIONS
WIN_REMOTE_SESSIONS
LOCAL
REMOTE
INTERLUPT
KNITB
TCP_HOSTS_FILE
TCP_SERVICES_FILE
BQLENET_BNAME
```

10. 지금 WINBOUB를 시작한 다음, 프로그램 관리자의 도움말(HELP)을 클릭하고 ABOUT PROCEMAN NAME를 클릭하여 WINBOUB가 민헨스드 모드인지 확인해 보십시오. WINBOUB용 BQLENET은 표준 모드 로 작동하지 않습니다.

11. 로그인 화면에 연결 스크린을 명시적으로 제공한 경우에는 연결 스크린 앞에 'B'를 지정하지 마 십시오. 다음 예와 같이 그냥 연결 스크린만을 입력하십시오.

`tiomacsviornaid` 혹은 `xiomacsv`

12. `DBACLE.INI`과 `CONFIC.DBAC`에 `LOCAL` 매개 변수가 있으면, 로그인 화면에서 연결 스트림을 지정하지 않고 연결 스트림 필드를 비워 둘 수 있습니다.

`LOCAL=tiomacsviornaid`

주: Oracle Corporation은 사용자의 `%ORACLE_HOME%` 디렉토리에 설치되는 `NETEASY.DXE`라는 WINDOWS 시험 프로그램을 배포합니다. 이 프로그램을 사용하면 원격 `DBACLE` 서버에 대한 연결을 시험할 수 있습니다. 이 프로그램은 연결되지만 제3자 데이터 연결되지 않으면 해당 줄 밑줄에 표시하십시오.

SAMPLE: XA PROGRAM

Bulletin no : 10383

```
-----
myfunc,sql
-----
create or replace function myfunc ( xxx number )
    return number
is
    kkk number;
begin
    for c1 in (
        select sql(conc,0) as kkk
        from emp
        where empno=7489
    ) loop
        return c1.kkk;
    end loop;
    return 0;
end;
/

-----
xx_prog.pl
-----
/*
*/

EXEC DBACLE OPTION (RELEASE_CURSOR=NO);
EXEC DBACLE OPTION (HOLD_CURSOR=YES);
/* EXEC DBACLE OPTION (SQLCHECK=SEMANTICS);
*/

/*#include */
#include

EXEC SQL INCLUDE BQLCA.H;
EXEC SQL INCLUDE DBACA.H;

#include

EXEC SQL BEGIN DECLARE SECTION;
```



```

        int exp_mutex;
        char exp_name[15];
        char *username = "BCDYY";
        char *password = "TIGER";
        VARCHAR sqlstmt[80];

        int outval;
EXEC SQL END DECLARE SECTION;

int xresult = 0;          /* xx result code */
int uxxxx = 1;           /* 1 -> use xx, otherwise normal sql */
int debug = 1;           /* print ALL result codes */
int valid = 1;
KID xldx = { 0x1a0a0a1a, 12, 8, "ExecGTID01Branch0001" };
KID *xld = &xldx;
char xcoldefn[] =
    "oracle_xstACC=P/BCDYY/TIGER+SEBTH=800";
int xcoldefn = 1;        /* next transaction this pid */
#define LDDPMAN 1

void
xldgm(xld)
KID* xld;                /* output parameter */
{
    char msg [11];

    sprintf(msg, "%d", xcoldefn);
    strcpy((xld->data[8]), msg, 8);
    xcoldefn++;
}

int dldgm()
{
    int xerror=0;
/* open database */
    if ( uxxxx )
    {
        xresult = xcopy(xcoldefn, valid, TMMOFLAGB);
        if ( xresult || debug )
            printf("xcopy = %d\n", xresult);
        if ( xresult )
            xerror++;
    }
    else
    {
        /* make space */

```

```

        opack, opackbf=1;
        opack, opackbf=1;
        opack, opackbf=3;

EXEC SQL CONNECT :username IDENTIFIED BY :password;
if (sqlca.sqlcode != debug )
    printf("connect  sqlcode = %d\n", sqlca.sqlcode);
if ( sqlca.sqlcode )
    sqlerror++;
}

return(sqlerror);
}

int starttr()
{
    int sqlerror=0;
    int xrcflt;

/* start transaction */
if ( uconn )
{
    xldgss(xld);
    xrcflt = xstart(xld, rld, TMOPLAGB);
    if ( xrcflt != debug )
        printf("xstart = %d\n", xrcflt);
    if ( xrcflt )
        sqlerror++;
}

else
{
    /* normal sql done by default */
}

return(sqlerror);
}

int jointr()
{
    int sqlerror=0;
    int xrcflt;

/* start transaction */
if ( uconn )
{
    xldgss(xld);
    xrcflt = xstart(xld, rld, TMOJOIN);
    if ( xrcflt != debug )

```

```

        printf("xstart = %d\n", xstart);
    if ( xstart )
        xstart++;
}
else
{
    /* can't join a session w/o xa */
    printf("can't join if not xa\n");
    xerror++;
}
return(xerror);
}

int debug()
{
    int xerror=0;
    int xstart;

    /* do database operation */
    EXEC SQL EXECUTE
    begin :outval := myfunc(1); end;
    END-EXEC;
    if (xqlc.sqlcode == debug )
        printf("execute sqlcode = %d, result=%d\n", xqlc.sqlcode, outval);
    if ( xqlc.sqlcode )
        xerror++;

    return(xerror);
}

int commit()
{
    int xerror=0;
    int xstart;

    /* commit transaction */
    if ( xerror )
    {
        xstart = xocommit(xid, rvid, THORPHASE);
        if ( xstart == debug )
            printf("xocommit = %d\n", xstart);
        if ( xstart )
            xerror++;
    }
    else

```

```

{
    EXEC SQL commit work;
    if (sqlca.sqlcode != debug )
        printf("commit  sqlcode = %d\n", sqlca.sqlcode);
    if ( sqlca.sqlcode )
        sqlerror++;;
}
return(sqlerror);
}

```

```

int rollback()
{
    int sqlerror=0;
    int xsqlit;

    if ( uuser )
    {
        xsqlit = xrollback(xld, rld, THROFLAG);
        if ( xsqlit != debug )
            printf("xrollback = %d\n", xsqlit);
        if ( xsqlit )
            sqlerror++;;
    }
    else
    {
        EXEC SQL rollback work;
        if (sqlca.sqlcode != debug )
            printf("commit  sqlcode = %d\n", sqlca.sqlcode);
        if ( sqlca.sqlcode )
            sqlerror++;;
    }
    return(sqlerror);
}

```

```

int dclose()
{
    int sqlerror=0;
    int xsqlit;

    /* close database */
    if ( uuser )
    {
        xsqlit = xcclose(xcinfo, rld, THROFLAG);
        if ( xsqlit != debug )
            printf("xcclose = %d\n", xsqlit);
        if ( xsqlit )

```

```

        myerror++;
    }
    else
    {
        EXEC SQL rollback work release;
        if (sqlca.sqlcode != debug )
            printf("rollback  sqlcode = %d\n", sqlca.sqlcode);
    }
    return(myerror);
}

```

```

int detach()
{
    int myerror=0;
    int xresult;

    if ( success )
    {
        xresult = xcommit(xid, rvid, TMSUCCESS);
        if ( xresult != debug )
            printf("xcommit = %d\n", xresult);
        if ( xresult )
            myerror++;
    }
    else
    {
    }
    return(myerror);
}

```

```

void main()
{
    int i;

    char temp[11];

    /* printf(temp,"%d", getpid()); */
    printf(temp,"%d", 100000);
    strcpy(xid->data, temp, 8);
    if (debug()) exit(1);
    for ( i=0; i < LDOPMAX; i++ )
    {
        if (activate()) exit(1);
        if (debug()) exit(1);
        if (detach()) exit(1);
        if (commit()) exit(1);
    }
}

```

```

>
if (dhc_lname()) exit(1);

exit(0);
>

=====
proc.mk
=====

include $(DHCLE_HOME)/precomp/new_precomp.mk

.SUFFIXES: .c .o

LDSO_LIBS=
PRODUCT_LIBHOME=
MAKEFILE=proc.mk
PROCPFLAGS= xgldcheck=full uuserld=$(USERID) dhasv8_cbase
PROCPPLFLAGS= code=cpp include=/usr/include include=/opt/BINMkpro/BC3.0.1/include/cc
USERID=ccott/tiger
INCLUDE=$(I_BYN). $(PRECOMPUBLIC)

SAMPLES=samples samples4 samples8 samples7 samples \
        samples samples10 xx_prog samples12 area xgldcv cv_data
CPPSAMPLES=cppsamples2 cppsamples3

# Rule to compile any program (specify EXE= and OBJ= on command line)

build: $(OBJ)
        $(CC) -o $(EXE) $(OBJ) -L$(LIBHOME) $(PRODLIBS)

cppbuild:
        $(PDOC) $(PROCPPLFLAGS) -x$(EXE)
        cc -o $(INCLUDE) $(EXE).c
        cc -o $(EXE) $(OBJ) -L$(LIBHOME) $(PRODLIBS)

samples: $(SAMPLES)
cppsamples: $(CPPSAMPLES)

$(SAMPLES):
        $(MAKE) -f $(MAKEFILE) build OBJ=$(O).o EXE=$(O)

$(CPPSAMPLES):
        $(MAKE) -f $(MAKEFILE) cppbuild OBJ=$(O).o EXE=$(O)

samples:

```

Becky 'example5 is a user-exit demo: use a form makefile to build it.'

```
.pc.o:
$(PDOC) $(PROCLASB) issue-$(*.pc)

.pc.o:
$(PDOC) $(PROCLASB) issue-$(*.pc)
$(CC) $(CLASB) -c $(*.c)

.c.o:
$(CC) $(CLASB) -c $(*.c)

exampleB.o: exampleB.pc
$(PDOC) dhas-vB_chas issue-$(*.pc)
$(CC) $(CLASB) $(PNECOMPUBLIC) -c $(*.c)

exampleB.o: exampleB.pc calldemo-egl
$(PDOC) $(PROCLASB) issue-$(*.pc)
$(CC) $(CLASB) $(PNECOMPUBLIC) -c $(*.c)

cv_demo.o: cv_demo.pc cv_demo-egl
$(PDOC) $(PROCLASB) issue-$(*.pc)
$(CC) $(CLASB) $(PNECOMPUBLIC) -c $(*.c)

xx_prog.o: xx_prog.pc
$(PDOC) $(PROCLASB) issue-$(*.pc) dhas-vB
$(CC) $(CLASB) $(PNECOMPUBLIC) -c $(*.c)

calldemo-egl:
eglpus scott/tiger 0.1 /egl/calldemo
```

NUMBER DATA TYPE

Bulletin no : 10042

This article attempts to explain, with examples, the ORACLE Number datatype (type 2) as stored internally. Oracle Corporation reserves the right to change this internal representation at any time without reason and notice. Numeric data is stored internally in variable length format, and consists of two parts:

- SIGN/EXPONENT Byte (1st byte)
- DATA Byte(s) (2nd byte onwards)

SIGN/EXPONENT BYTE : The sign/exponent byte is divided into two parts again:

- Sign bit (high-order bit)
- Exponent bits (lower order 7 bits)

The key to the calculation of a number is the sign bit which can take values 0 for negative and 1 for positive numbers.

The 7 lower order bits are the exponent, stored in excess B4 notation i.e. whatever the number you calculate from the 7 bits, you will have to deduct B4 from it to determine exponent value. The Sign/Exponent byte is evaluated as follows -

If the sign bit is set then the number is positive and the exponential is calculated in the following manner. Determine the value of the exponent using the lower order 7 bits. As this value is in excess B4 notation, deduct B4 from this number that was calculated to give the exponent value.

E.g. Assume the SIGN/EXPONENT byte was 223 which in binary is 11011111.

From the binary representation of the example we determine that bit 8 is set, hence this is a positive number. To determine the exponent, clear bit 8 and then calculate the resultant which in this case is 85. As 85 is in excess B4 notation deduct B4 from it to give 31, which is the exponent.

If the sign bit is cleared i.e. 0 (number is negative), a little more work goes into calculating the number.

Perform a 1's complement of the sign/exponent byte (all 8 bits). Determine the value of the exponent using the lower order 7 bits. As this value is in excess B4 notation, deduct B4 from the value that was calculated to give the exponent value.

E.g. Assume the SIGN/EXPONENT byte was 100 which in binary is 01100100.

From the binary representation we can determine that the number is negative as bit 8 is clear. Now take the 1's complement of 01100100 to give 10011011. Mask off bit 8 (or just set it to 0) so that only bits 1 through 7 are used to calculate the exponent which in our example

will be 27. As 27 is in excess B4 notation, deduct B4 from it to give -37. Hence, exponent is -37.

DATA BYTE(x) : Data is stored in a series of bytes with the representation being in base 100. To avoid a binary zero a 1 is added to the number. Hence, each data byte represents digits in base 100 with values ranging from 1 to 100 representing values 0 to 99. Negative numbers have a terminator byte with a value of 102.

Let us take some examples and then try to understand the methodology of the calculation. These examples work off a table with the following description.

BQL> describe num_tab

Name	Null?	Type
c1		NUMBER

CASE 1 : Insert number 123458. Dump of column c1 gives the following

DUMP(c1)

 Typ=2 Len=4: 185,13,35,57

1st byte = 185 decimal → 11000011 binary Number is positive as bit 8 is set. The exponent value is 01000011 binary which is 87 decimal. As this number is in excess B4 notation deduct B4 from 87 giving 3. Exponent is 3 and positive.

2nd byte = 13 decimal

As the sign is positive deduct 1 from 13 (as it is stored with one added) to get 12

3rd byte = 35 decimal

As the sign is positive deduct 1 from 35 to get 34

4th byte = 57 decimal

As the sign is positive, deduct 1 from 57 to get 58

The exponent is 3, the data bytes are in base 100, so the number is

$$12 \times (100 + 2) + 34 \times (100 + 1) + 58 \times (100 + 0) = 123458$$

CASE 2 : Insert number -123458. Dump c1 gives the following

DUMP(c1)

 Typ=2 Len=5: 80,88,B7,45,102

1st byte = 80 decimal → 00111100 binary

Number is negative as bit 8 is not set. Take the 1's complement of 00111100 which is 11000011. From this the exponent is 87 decimal (take the lower order 7 bits to calculate the

exponent). As this number is in excess B4 notation deduct B4 from B7 giving 3. Exponent is positive and is 3.

2nd byte = BB decimal

Deduct 1 from this which gives BB. As the sign is negative, you need to take the 100's complement of this i.e. $100 - BB$ to get 12.

3rd byte = B7 decimal

Deduct 1 from this which gives B6 and take the 100's complement to get 34.

4th byte = 45 decimal

Deduct 1 from this which gives 44 and take the 100's complement to get 5B.

5th byte = 102 decimal

This is the last byte.

The calculation is

$$12 \times (100 \times 2) + 34 \times (100 \times 1) + 5B \times (100 \times 0) = 12345B$$

and as the sign is negative, the answer is -12345B.

CASE 3 : Insert number 12345B, 78B. Dump (c1) gives the following

DUMP(C1)

Typ=2 Len=8: 1B5,13,35,57,7B,B1

1st byte = 1B5 --> 11000011 binary

Number is positive as bit 8 is set. The exponent value is 01000011 binary which is B7 decimal. As this number is in excess B4 notation deduct B4 from B7 giving 3 which is a positive number.

2nd byte = 13 decimal

As the sign is positive, deduct 1 from 13 to get 12.

3rd byte = 35 decimal

As the sign is positive, deduct 1 from 35 to get 34.

4th byte = 57 decimal

As the sign is positive, deduct 1 from 57 to get 5B.

5th byte = 7B decimal

As the sign is positive, deduct 1 from 7B to get 7B.

6th byte = B1 decimal

As the sign is positive, deduct 1 from B1 to get B0. The exponent is 3, the data bytes are

is base 100, so the number is

$$12 \times (100 = 2) + 34 \times (100 = 1) + 58 \times (100 = 0) + 78 \times (100 = -1) + 80 \times (100 = -2) = 123458.788$$

CASE 4 : Insert number -123458.788 Dump of c1 gives the following

DUMP(c1)

Typ=2 Len=7: B0,BB,B7,45,23,11,102

1st byte = B0 decimal $\rightarrow$ 00111100 binary

Number is negative as bit 8 is not set. Take the 1's complement of 00111100 which is 11000011. From this the exponent is 87 decimal (take the lower order 7 bits to calculate the exponent). As this number is in excess B4 notation deduct B4 from 87 giving 3. Exponent is positive and is 3.

2nd byte = BB decimal

Deduct 1 from this which gives BB. As the sign is negative, you need to take the 100's complement of this i.e. $100 - BB$ to get 12.

3rd byte = B7 decimal

Deduct 1 from this which gives BB. As the sign is negative, you need to take the 100's complement of this i.e. $100 - BB$ to get 34.

4th byte = 45 decimal

Deduct 1 from this which gives 44. As the sign is negative, you need to take the 100's complement of this i.e. $100 - 44$ to get 58.

5th byte = 23 decimal

Deduct 1 from this which gives 22. As the sign is negative, you need to take the 100's complement of this i.e. $100 - 22$ to get 78.

6th byte = 11 decimal

Deduct 1 from this which gives 11. As the sign is negative, you need to take the 100's complement of this i.e. $100 - 11$ to get 89.

7th byte = 102 decimal

This is the last byte.

The calculation is

$$12 \times (100 = 2) + 34 \times (100 = 1) + 58 \times (100 = 0) + 78 \times (100 = -1) + 80 \times (100 = -2) = 123458.788$$

and as the sign is negative, the answer is -123458.788

The above explanation is intended to further the knowledge of ORACLE's internal datatype representation only.

Note : Given below are examples of calculating the 1's and 100's complement. 1's complement of 10001100 = 01110011 (Just 'not' the bits) 100's complement of 28 = $100 - 28 = 74$

DATE TYPE 데이터를 효과적으로 조회하는 방법

Bulletin no : 10073

DATE TYPE 데이터라는 오차를 TIME를 사용하여 시간, 분, 초를 지정하지 않고 년월일만 입력하면 시간, 분, 초 값이 '0'으로 입력된다. 이 데이터는 다음과 같이 조회하면 확인할 수 있다.

```
BQL> SELECT TO_CHAR(HIMDATE, 'YYYYMM HH24:MI') HIMDATE FROM EMP;
```

```
HIMDATE
-----
B40508 00:00
B40407 00:00
B40408 10:11 (*)
```

```
BQL> SELECT TO_CHAR(HIMDATE, 'YYYYMM HH:MI') HIMDATE FROM EMP;
```

```
HIMDATE
-----
B40508 12:00
B40407 12:00
B40408 10:11 (*)
```

만일 시간, 분이 입력되어 있는 데이터를 (*) 년월일로 조회하는 경우는 다음 BQL은 데이터를 조회할 수 없다.

```
BQL> SELECT * FROM EMP WHERE HIMDATE = '08-APR-84';
```

```
no rows selected
```

데이터를 조회 하기 위해서는 다음과 같은 방법으로 BQL을 작성 한다.

```
A. BQL> SELECT * FROM EMP WHERE TO_CHAR(HIMDATE, 'YYYYMM')
      = 'B40408'
```

```
B. BQL> SELECT * FROM EMP WHERE HIMDATE BETWEEN '08-APR-84' AND
      TO_DATE('08-APR-84') + .99999;
```

위 BQL들은 같은 결과의 데이터가 조회되지만 B. 방법을 사용할 수 있으므로 보다 더 효율적인 BQL이다.

ORACLE7 OPTIMIZER 에 대하여

Bulletin no : 10077

OPTIMIZER의 기능은 QUERY를 실행하기 위해 가장 좋은 방법을 선택하는 것이다.
ORACLE 7과 비교하여 ORACLE7은 QUERY OPTIMIZER에 관한 2가지 주요기능 즉, COST-BASED 접근방법과 HINT-MECHANISM을 이용할 수 있는 것이다.
COST-BASED 접근방법은 OPTIMIZER가 이용 가능한 접근 경로를 결정하고 그 접근 경로에 근간을 둔 여러개의 EXECUTION PLAN을 만든다. 이 EXECUTION PLAN들에 대해서 COST가 추정되고 OPTIMIZER는 가장 COST가 적은 EXECUTION PLAN을 선택한다.
OPTIMIZER는 EXECUTION PLAN의 COST를 추정하기 위하여 통계치를 사용한다. 통계치들은 ANALYZE 명령에 의해 생성되고 다음과 같은 DATA DICTIONARY VIEW를 통해서 볼 수 있다.

```
DBA/ALL/USER TABLES, DBA/ALL/USER TAB COLUMNS,  
DBA/USER CLUSTERS, DBA/ALL/USER INDEXES
```

syntax ANALYZE명령어 :

ANALYZE	INDEX	index	COMPUTE	STATISTICS
	TABLE	tablename	ESTIMATE	
	CLUSTER	cluster	DELETE	

[COMPUTE] : 정확한 정보를 얻을수있지만 데이터베이스에 상당한 LOAD가 있으므로 RECORD 수가 작은 TABLE에 적용한다. (약 20000 RECORD 이하)

[ESTIMATE] : RECORD 수가 20000 전이상 TABLE에 적용한다.

ORACLE7 OPTIMIZER는 통계치가 적어도 한개 TABLE에 존재하면 COST-BASED OPTIMIZER를 사용하고 어느 TABLE에도 통계치가 없으면 RULE-BASED OPTIMIZER를 사용한다.

또한 ORACLE7은 HINT MECHANISM을 이용하여 USER가 원하는 EXECUTION PLAN을 optimizer에게 강제적으로 사용하도록 할 수 있다.

HINT에는 COST (cost-based approach), NOCOST (rule-based approach), FULL (full table scan), INDEX (use a certain index), ORDERED (from절에 나오는 순서대로 TABLE을 JOIN) 등이 있다.

다음 예들을 통하여 OPTIMIZER의 기능을 이해하기로 하자.

CASE1) ORACLE User에 DB OPTIMIZER

```
Table TAB1 : 18,384 rows  
Table TAB2 : 1 rows
```

2. TAB2를 DRIVING TABLE로 하는 경우

```
select count(*) from TAB1, TAB2 0.88 seconds elapsed
```

	Count	Phys	cr	ca2	rows
Parse	1	0	0	0	
Execute	1	85	100	4	0
Fetch	1	0	0	0	1

b. TABLE를 DRIVING TABLE로 하는 경우

```
select count(*) from TAB2, TAB1      28.08 seconds elapsed
```

	Count	Phys	cr	ca2	rows
Parse	1	0	0	0	
Execute	1	85	48247	32770	0
Fetch	1	0	0	0	1

위의 경우 TAB2 DRIVING인 경우가 더 효율적이므로 APPLICATION은 a.과 같이 사용하여야 한다.

CASE2) ORACLE Version7 Optimizer

각 TABLE에 ANALYZE COMMAND를 시키지 않으면 결과는 VERSION 6의 경우와 유사하고 만일 두개의 TABLE에 ANALYZE를 실행시킬 경우는 다음과 같은 결과를 얻을수 있다.

```
a. select count(*) from TAB1, TAB2    1.20 seconds elapsed
```

	Count	Phys	cr	ca2	rows
Parse	1	0	0	0	
Execute	1	0	0	0	0
Fetch	1	123	124	5	1

```
b. select count(*) from TAB2, TAB1    1.32 seconds elapsed
```

	Count	Phys	cr	ca2	rows
Parse	1	0	0	0	
Execute	1	0	0	0	0
Fetch	1	123	124	5	1

위의 결과로서 ORACLE7에는 ANALYZE COMMAND를 실행할 경우 TABLE의 순서를 바꾸더라도 성능에 큰차이가 없이 향상된 결과를 얻을 수 있고 APPLICATION TUNING에 부담을 줄일수 있다.

한글이 ??? 로 DISPLAY 되는 경우

Bulletin no : 10087

ORACLE TOOLS(SQLPLUS,FORMS30,FORMS40,REPORT20 등)을 이용하여 한글DATA를 조회할 때 ???로 출력는데 해결 방법

DATABASE는 SQL COMMAND 'CREATE DATABASE'를 포함하는 STATEMENT를 수행할 때 만들어지는데 루티가 그STATEMENT를 수행하기 앞서 고려해야 사항중의 하나가 DB CHARACTERSET 이다.

DB를 CREATE할때 DATABASE CHARACTERSET을 명시해야만 하는데 한편 선택되고 난후에는 CHARACTERSET을 변경하는 것은 쉽지가 않다.

DATA DICTIONARY에 있는 DATA를 포함해서 모든 DATA는 선택된 CHARACTERSET 에 의해 입출력 되기 때문에 USER가 다른 CHARACTERSET으로 ACCESS한다면 한글 데이터가 ???로 출력된다.

또한 분산 DB 환경에서나 UPGRADING할 경우에는 DATABASE CHARACTERSET이 같아야 함으로 사용자들은 DATABASE의 CHARACTERSET을 알아야 주어야 한다.

< 현재 DATABASE 의 CHARACTERSET 확인 및 변경 >

1. 데이터베이스 CHARACTERSET 확인

```
$ sqlplus lmsadm
SQL> connect internal
SQL> select * from v$sql_parameters;
PARAMETER                                VALUE
-----
...
NLN_CHARACTERSET                          ED16E8C8D1 (or US7ASCII)
                                           (A)
```

2. 환경 변수의 NLN_LANG 확인

```
$ env
...
NLN_LANG=American_American.US7ASCII
                                           (B)
```

위의 (A)와 (B) 가 동일 해야만 한글 데이터 처리가 잘되고 서로 다른 상태에서 한글 데이터를 조회 할경우 ??? 로 출력 된다.

3. CHARACTERSET 을 일치 시키는 방법

* NLN_LANG 환경 변수를 변경하여 일치시키는 방법

```
Source shell, k-shell 을 사용 하는경우 . profile 을 수정
NLN_LANG = American_American, ED16E8C8D1; export NLN_LANG
```

c:\bll을 사용하는 경우 .cblrc 혹은 .logln 수정
setenv NLB_LANC America_America, E01B8C5B01
수정후 다시 @ .env 를 실행하여 변경 되었는지 확인한다.

C:\WINDOWS\OEM\OEM.INI 수정
NLB_LANC=America_America, E01B8C5B01
WINDOWS 재기동

* 그리고 예를 들어서 BEBUEB의 character set이 US7A8C11 이고 PC의 NLB_LANC가
America_America, E01B8C5B01 과 같이 서로 다르게 설정되어 있는 경우 다음을 하나 더
oemcln.ini에 추가하면 한글문제가 해결됩니다.

DATA_NLB_CHARACTERSET_CONVERSION=NO_CHARACTER_SET_CONVERSION

WINDOWS BS에서는 NLB_LANC가 OEMCLN.INI에 들어 있지 않고 BECIBTRY에 기록되므로 BECIBTRY
EDITOR를 이용하여 수정해야 한다.

MSDOS 상으로 나가서 BECIBTRY.EXE 실행
MENU_LOCAL_MACHINE -> BOTTOMLINE -> OEMCLN 선택
오른쪽 마우스 버튼을 이용하여 NLB_LANC를 수정

BECIBTRY 편집 후에 PC를 REBOOTING 할 필요는 없습니다.

WINDOWS NT에서도 WINDOWS BS의 경우와 마찬가지로 BECIBTRY 에 기록된 정보를 편집해 주면
됩니다. 다음과 같이 합니다.

DOS 상에서 BECIBTRY2.EXE 실행
MENU_LOCAL_MACHINE -> BOTTOMLINE -> OEMCLN 선택
메뉴를 선택하여 NLB_LANC를 수정

SQLPLUS 기동시 WARNING MESSAGE 처리

Bulletin no : 10108

SQLPLUS를 기동할때 다음과 같은 경고 MESSAGE가 나타나는 경우가 있다.

```
$ sqlplus scott/tiger
SQLPLUS: Release 3.1.2.3.1 - Production on Tue Aug 30 17:07:38 1994
Copyright (c) Oracle Corporation 1978, 1992. All rights reserved.
```

```
Warning: Product user profile information not loaded! <--
Error in disabling roles in product user profile. <--
```

```
Connected to:
Oracle7 Server Release 7.0.15.4.0 - Production
With the procedural and distributed options
PL/SQL Release 2.0.17.1.0 - Production
```

이 경고 MESSAGE는 사용자나 DBA가 SQLPLUS를 사용하는데 방해가 될 수 있기 때문에 다음과 같은 방법으로 제거 한 후 사용한다.

Unix 의 경우

```
$ cd $ORACLE_HOME/sqlplus/admin
$ sqlplus system/manager
SQL > start psqlid
```

OpenVMS 의 경우

ORA_SQLPLUS_BMOD:U7PUP, SQL을 SQLPLUS SYSTEM 사용자로 LOGIN하여 실행.

Windows 의 경우

c:\oracle\sqlplus31\psqlid.sql을 SQLPLUS SYSTEM 사용자로 LOGIN 하여 실행.

SQL*PLUS 를 사용하여 SQL SCRIPT를 동적으로 생성, 수행

Bulletin no : 10107

이 Document 는 SQL 과 SQL*PLUS를 동적으로 생성하고 수행하는 방법을 소개하고 있다. 이를 위해 자주 사용하는 3가지 사용 예를 소개한다.

이 Document는 다음과 같이 구성 되어져 있다.

```
CREATING A DYNAMIC DROP TABLE SCRIPT
CREATING A DYNAMIC GRANT PRIVILEGES SCRIPT
CREATING A DYNAMIC SCRIPT TO SET SQL PROMPT TO DATABASE NAME
```

1. CREATING A DYNAMIC DROP TABLE SCRIPT

개발 과정 중 우리는 Table을 Drop하고 Original Data를 Import하는 작업을 빈번하게 수행하는데, 이때 기존의 Drop 문을 사용하지 않고 SET_UP_DROP.SQL 이라 불리는 다음의 Script를 사용한다.

```
set heading off
set feedback off
spool drop_all.sql
select 'drop table '||table_name||';' from user_tables;
spool off
set feedback on
start drop_all.sql
```

DROP_ALL.SQL script 는 다음과 같다.

```
drop table DEPT;
drop table BONUS;
drop table SALGRADE;
drop table DUMMY;
drop table CUSTOMERS;
drop table DEMO;
drop table ITEM;
drop table PRODUCT;
drop table PRICE;
```

2. CREATING A DYNAMIC GRANT PRIVILEGES SCRIPT

자신의 모든 Table 을 다른 사람에게 SELECT 권한을 부여하고자 할때 다음의 SET_UP_GRANT.SQL 을 이용하면 빠르고 손쉽게 해결할 수 있다.

```

set heading off
set feedback off
spool grant_all.sql
select 'grant select on ||table_name|| to SCOTT;' from user_tables;
spool off
set feedback on
start grant_all.sql

```

GRANT_ALL.BQL 의 script 는 다음과 같다.

```

grant select on DEPT to SCOTT;
grant select on EMP to SCOTT;
grant select on SALGRADE to SCOTT;
grant select on DUMMY to SCOTT;
grant select on CUSTOMER to SCOTT;
grant select on ORD to SCOTT;
grant select on ITEM to SCOTT;
grant select on PRODUCT to SCOTT;
grant select on PRICE to SCOTT;

```

3. CREATING A DYNAMIC SCRIPT TO SET SQL PROMPT TO DATABASE NAME

위의 개념들을 보다 더 잘 사용하기 위해서는 Bq Prompt를 자신의 Database로 전환해주는 SET_UP_PROMPT.BQL Script를 login.sql File 에 추가시켜 자동적으로 BQL*LIB 플러그인 들수있게 한다. 이는 Multiple Database에 유용하다.

```

set timeout off
set heading off
set feedback off
spool set_prompt.sql
select 'set sqlprompt ||value||' from v$parameter
      where name='db_name';
spool off
start set_prompt.sql

```

SET_UP_PROMPT.BQL 은 다음과 같다.

```

set sqlprompt U715>

```

USPARAMETER 는 Dynamic Performance Table 의 View 이다. Install후 SYS만이 Dynamic Performance Table을 수행할 수 있으므로 위의 Script를 사용하기 위해 View를 Create 하거나 User에게 Access 권한 부여하여야 한다.

SQL TIPS(1): SORT

Bulletin no : 10108

[Q] BQLEPLIB 에서 Salary가 많은 순서로 1위부터 5위까지 Fetch하여 Name에 대해 Ordering하는 방법은?

[A] ROWNUM을 이용하면 만된다. Subquery를 이용한다.

잘못된 문장 :

```
select name, salary
from emp
where rownum < 5
order by sal desc;
```

바른 문장 :

```
select name, salary
from emp a
where 5 > (select count(*)
           from emp b
           where b.sal > a.sal)
order by sal desc;
```

SQL TPSC 2) 특정조건의 순서대로 번호를 부여하는 방법

Bulletin no : 10108

* 특정조건의 순서대로 번호를 부여하는 방법

[Q1] 아래의 Record를 Deptno 순으로 Sequence를 부여하려면?

ENAME	JOB	DEPTNO
-----	-----	-----
ALLEN	SALESMAN	20
WARD	SALESMAN	30
JONES	MANAGER	20
MARTIN	SALESMAN	30
BLAKE	MANAGER	30
CLARK	MANAGER	10
SCOTT	ANALYST	20
KING	PRESIDENT	10
TURNER	SALESMAN	30
ADAMS	CLERK	20
JAMES	CLERK	30
FORD	ANALYST	20
MILLER	CLERK	10

[A1]

1. Emp Table의 Deptno Column에 Index를 생성한다.

```
create index i_emp on emp(deptno);
```

2. Table에 Sequence Number Column을 추가한다.

```
alter table emp add (seq1 number(5));
```

3. update emp

```
set seq1 = rownum
where deptno > '0';
```

ENAME	JOB	DEPTNO	SEQ1
-----	-----	-----	-----
CLARK	MANAGER	10	1
KING	PRESIDENT	10	2
MILLER	CLERK	10	3
ALLEN	SALESMAN	20	4
JONES	MANAGER	20	5
SCOTT	ANALYST	20	6
ADAMS	CLERK	20	7

FORD	ANALYST	20	8
WARD	SALESMAN	30	8
MARTIN	SALESMAN	30	10
BLAKE	MANAGER	30	11
TURNER	SALESMAN	30	12
JAMES	CLERK	30	13

[Q2] 각 부서별로 Sequence를 부여하려면?

[A2]

```
update emp A
set A.empid = (select A.empid - min(B.empid)+1
              from emp B
              where B.deptno = A.deptno);
```

ENAME	JOB	DEPTNO	SEQ1
CLARK	MANAGER	10	1
KING	PRESIDENT	10	2
MILLER	CLERK	10	3
ALLEN	SALESMAN	20	1
JONES	MANAGER	20	2
SCOTT	ANALYST	20	3
ADAMS	CLERK	20	4
FORD	ANALYST	20	5
WARD	SALESMAN	30	1
MARTIN	SALESMAN	30	2
BLAKE	MANAGER	30	3
TURNER	SALESMAN	30	4
JAMES	CLERK	30	5

SQL TRACE FACILITY

(TKPROF UTILITY 사용)

Bulletin no : 10134

BQL TRACE FACILITY 는 BQL 문 사용에대한 성능을 분석하기위해서 사용된다.
이러한 BQL TRACE FACILITY 를 이용하면 각 BQL 문에 대해서 다음과 같은 정보를 얻을수 있다.

- . parse,execute,fetch count
- . CPU 와 elapsed 시간
- . physical reads 와 logical reads
- . 처리된 row 의 수

BQL TRACE FACILITY 는 SESSION 혹은 INSTANCE 단위로 할 수 있고 TRACE 결과 파일은 tkprof UTILITY 에 의해 사용자가 읽을수 있는 형태로 전환시킨다.

BQL TRACE SET UP 하여 사용하는방법

1. BQL TRACE enable 및 TRACE 파일 디렉토리 지정

< INSTANCE 단위 >

?/dba/initdb.ora 파일에 다음 주개의 PARAMETER 를 추가하고 DATABASE 를 다시 STARTUP 시킨다.

```
sql_trace = true
```

```
timed_statistics = true
```

timed_statistics 는 시스템에 많은 LOAD 가 걸리므로 사용하지 않는 것이 좋다.

< SESSION 단위 >

BQL>PLIB

```
$ sqlplus scott/tiger
```

```
BQL> ALTER SESSION SET BQL_TRACE = TRUE;
```

BQLFDBMB30, BUNFDBMB30 : --x OPTION 사용

```
$ runfom30 -x fmfllm scott/tiger -c vt220
```

PROD>C

```
EXEC BQL CONNECT success;
```

```
EXEC BQL ALTER SESSION SET BQL_TRACE = TRUE;
```

TRACE FILE 디렉토리는 initdb.ora 파일에 다음과 같이 지정하고 지정하지 않으면 ?/dba/log 디렉토리에 생긴다.

```
user_dump_dest = /user/dump
```

2. BQL 문 실행

TRACE 파일은 SESSION 단위로 1 개씩 생성으로 한 SESSION 에서 사용된 모든 BQL 문의 TRACE 결과 는 1개의 파일에 존재하게된다.

예. BQL> ALTER SESSION SET BQL_TRACE = TRUE;

```
BQL> SELECT COUNT(*) FROM EMP E,DEPT D
```

```
WHERE E.DEPTNO = D.DEPTNO;
SQL> EXIT;
```

3. TRACE 파일 전환

2. 단계에서 SQL 문을 실행하면 `user_dump_dest` 에 지정된 디렉토리에 TRACE 파일이 생기고 `tkprof` 클리용하여 파일을 전환시킨다. TRACE 파일은 쉽게 찾을 수 있는 형태가 아니므로 SQL문을 실행하기 전에 `dump` 디렉토리에 있는 `ora_XXXX.trc` 파일을 모두 삭제하거나 가장 최근에 생긴 파일중에서 찾아야 한다.

```
예. $ cd $ORACLE_HOME/rdbms/log
$ tkprof ora_1111 out scott=scott,tchou explain=scott/tiger print=20
```

```
ora_1111 : TRACE 파일
out      : OUTPUT 파일. 디렉토리에 out.prf 로 생긴다.
scott    : 지정된 DPID(scott,tchou) 에 ASCENDING 순으로 SQL 문을
          SORTING 한다.
explain  : SQL 문의 EXECUTION PLAN 을 말함시킨다.
print    : 지정된 갯수의 SQL문에대해서만 TRACE 결과를 PRINT 한다.
```

4. SQL TRACE 결과 분석

```
*****
count   = number of times OCI procedure was executed
cpu     = cpu time in seconds executing
elapsed = elapsed time in seconds executing
disk    = number of physical reads of buffers from disk
query   = number of buffers gotten for consistent read
current = number of buffers gotten in current mode (usually for update)
row     = number of rows processed by the fetch or execute call
*****
```

```
SELECT COUNT(*)
FROM EMP E,DEPT D WHERE E.DEPTNO=D.DEPTNO
```

call	count	cpu	elapsed	disk	query	current	row
Parse	2	0.00	0.00	0	0	0	0
Execute	2	0.00	0.00	0	0	0	0
Fetch	2	0.00	0.00	0	33	2	1

Misses in library cache during parse: 1

Parsing user id: B (SCOTT)

Row Execution Plan

```
0 SELECT STATEMENT
0   SORT (AGGREGATE)
15  NESTED LOOPS
15    TABLE ACCESS (FULL) OF 'EMP'
```


LONG COLUMN을 가진 TABLE의 COPY

Bulletin no : 10147

문제설명

LONG COLUMN 을 가진 테이블을 COPY 를 해야 하는경우 LONG COLUMN은 SQL-QUERY 를 사용 할수 없기 때문에 다음과 같은 현상이 발생한다.

1. "CREATE TABLE .. AS SELECT .." 명령이나
"INSERT INTO .. SELECT .." 를 사용하는 경우 ORA-00807 에러가 발생한다.
2. SQL*Plus 의 COPY 명령을 사용 하는 경우 COPY005 에러가 발생한다.

해결방법

가장 쉬운 방법은 PL/SQL에서 VARCHAR2 혹은 LONG DATA TYPE을 이용하여 COPY한다. PL/SQL에서 VARCHAR2 및 LONG 변수는 32K 까지 사용할 수 있기 때문에 32K 이하 데이터라면 가능하고 그 이상의 데이터는 3GL을 이용한다.

< TABLE>

SQL> CREATE TABLE LONGTAB(NC NUMBER(2), TEXT LONG);

< PL/SQL Script >

DECLARE

 string varchar2(32000);

 no number(2);

BEGIN

 SELECT no,longcolumn INTO no, string FROM longtab;

 INSERT INTO longtab VALUES(no, string);

END;

/

SQL TIPS(3): DATE의 기간을 계산하는 방법

Bulletin no : 10158

```
$ sqlplus scott/tiger
```

```
SQL> SELECT
    trunc(((DATE2-DATE1)*80*80)/3600) || ' DAYS ' ||
    floor(mod(((DATE2-DATE1)*24*80*80)/3600, 24)) || ' HOURS ' ||
    floor(((date1-date2)*24*80*80)
    - floor(((date1-date2)*24*80*80)/3600)*3600)/80) || ' MINUTES ' ||
    round((((date1-date2)*24*80*80)
    - floor(((date1-date2)*24*80*80)/3600)*3600
    - (floor(((date1-date2)*24*80*80)
    - floor(((date1-date2)*24*80*80)/3600)*3600)/80)*80)) || ' SECS '
    TIMEDIFF
FROM dual
```

/

SQL TIPS(4): 중복된 RECORD 삭제 방법

Bulletin no : 10185

중복된 RECORD 들 삭제 하는 방법은 2가지가 있다.

방법1.

```
BQL> CREATE TABLE emp2 AS SELECT distinct * FROM emp;  
BQL> DROP TABLE emp;  
BQL> RENAME emp2 TO emp;
```

방법2.

```
BQL> DELETE FROM emp A  
      WHERE rowid > (SELECT min(rowid) FROM emp B  
                    WHERE B.empno = A.empno);
```

외부에서 값을 받아서 SQLPLUS의 SCRIPT 실행하는 방법

Bulletin no : 10211

1. PARAMETER를 받을수 있도록 SQL SCRIPT를 작성한다.

```
< BQL_BCM1.sql >
select count(*) from emp where deptno = &1;
```

```
< BQL_BCM2.sql >
select count(*) from emp where deptno = &1;
```

2. SHELL 프로그램을 작성 한다.

```
< BQL_BCM >
read dev
sqlplus -u scott/tiger @BQL_BCM1 &dev;
sqlplus -u scott/tiger @BQL_BCM2 &dev;
```

3. SHELL 프로그램 기동

```
$ ./BQL_BCM
```

SQL TIPS(5): DECODE이용 사례 (1) : 월별로 집계하는 방법

Bulletin no : 10218

```
sql> select sum(decode(to_char(hiredate, 'mm'), '01', sal), 0) Jan,
       sum(decode(to_char(hiredate, 'mm'), '02', sal), 0) Feb,
       sum(decode(to_char(hiredate, 'mm'), '03', sal), 0) Mar,
       sum(decode(to_char(hiredate, 'mm'), '04', sal), 0) Apr
  from emp
 where hiredate between to_date('810101', 'yyymmdd') and
                        to_date('811231', 'yyymmdd')+0.99999
```

sql>/

JAN	FEB	MAR	APR
0	2850	0	2875

SQL TIPS(6): 해당 COLUMN에 걸려 있는 CONSTRAINT 확인

Bulletin no : 10931

BQL TIP#(6):

```
/*
** Name      : Show_Constraints_Bql
**
** Usage     : @Show_Constraints Table_Name Column_Name
** Description : 해당 Table의 Column에 걸려 있는 Constraint를 보여준다.
**
**/
```

```
SET VERIFY OFF
CLEAR BREAK
BREAK ON TABLES ON COL_NAME
SELECT SUBSTR(TABLE_NAME,1,15)   TABLES,
       SUBSTR(COLUMN_NAME,1,15) COL_NAME,
       SUBSTR(CONSTRAINT_NAME,1,25) CONSTRAINT_NAME
FROM   USER_CONS_COLUMNS
WHERE  TABLE_NAME = UPPER('61')
AND    COLUMN_NAME = UPPER('62');
```

< 실행 예 >

BQL> @SHOW_CONSTRAINTS UIDGETS LENGTH

TABLES	COL_NAME	CONSTRAINT_NAME
-----	-----	-----
UIDGETS	LENGTH	CE_UIDGETS_LENGTH
		CE_UIDGETS_PRODUCTION

SQL TIPS(7): 한 CONSTRAINT에서 REFERENC E하는 COLUMN확인

Bulletin no : 10932

SQL TIPS(7):

```
/*
** Name      : Show_Columns_Bgl
**
** Usage     : DBShow_Columns_Constraint_Name
** Description : Show The Columns Bound By A Constraint
**
**/
```

```
SET VERIFY OFF
CLEAR BREAK
BREAK ON CONSTRAINT_NAME ON TABLES
SELECT SUBSTR(CONSTRAINT_NAME,1,25) CONSTRAINT_NAME,
        SUBSTR(TABLE_NAME,1,15)    TABLES,
        SUBSTR(COLUMN_NAME,1,15)    COL_NAME,

FROM    ALL_CONS_COLUMNS
WHERE   CONSTRAINT_NAME = UPPER('&1');
```

< 실행 예 >

SQL> DBSHOW_COLUMNS CE_WIDGETS_PRODUCTION

CONSTRAINT_NAME	TABLES	COL_NAME
CE_WIDGETS_PRODUCTION	WIDGETS	LENGTH
		WIDTH
		HEIGHT

SQL TIPS(8): PARENT-CHILD관계를 갖는 TABLE의 P RIMARY,FOREIGN KEY의 COLUMN명과 POSITION확인

Bulletin no : 10339

SQL TIPS(8):

```
/*
** Name      : Show_Position_Bgl
**
** Usage     : BShow_Positions Parent_Table Child_Table
** Description : Show Primary And Foreign Key Positions
**
** WARNING   : 이 문장은 해당 Table의 Constraint생성시 Naming
**              Convention을 따른 경우에 적용되도록 되어 있다.
**
**
**/
```

SET VERIFY OFF

CLEAR BREAK

BREAK ON CONSTRAINT_NAME ON TABLES

```
SELECT SUBSTR(CONSTRAINT_NAME,1,27) CONSTRAINT_NAME,
       SUBSTR(TABLE_NAME,1,15)      TABLES,
       SUBSTR(COLUMN_NAME,1,15)     COL_NAME,
       SUBSTR(POSITION,1,3)         POSITION,
       SUBSTR(OWNER,1,7)            OWNER
FROM   USER_CONS_COLUMNS
WHERE  TABLE_NAME = UPPER('61')
AND    CONSTRAINT_NAME LIKE 'FK%'
UNION
SELECT SUBSTR(CONSTRAINT_NAME,1,27) CONSTRAINT_NAME,
       SUBSTR(TABLE_NAME,1,15)      TABLES,
       SUBSTR(COLUMN_NAME,1,25)     COL_NAME,
       SUBSTR(POSITION,1,3)         POSITION,
       SUBSTR(OWNER,1,7)            OWNER
FROM   USER_CONS_COLUMNS
WHERE  TABLE_NAME = UPPER('62')
AND    CONSTRAINT_NAME LIKE 'FK%'
ORDER BY 1 DESC , 4 ASC;
```

< 실험 예 >

SQL> DBMS_FDSITIONS EMP_SERVICE EMP_SERVICE_LOG

CONSTRAINT_NAME	TABLES	COL_NAME	POS
FK_EMP_SERVICE	EMP_SERVICE	EMP_ID	1
		CUST_ID	2
FK_EMP_SERVICE_LOG_EC	EMP_SERVICE_LOG	EMP_ID	1
		CUST_ID	2

SQL TIPS(9): 해당 TABLE에 걸려있는 CONSTRAIN T의 확인

Bulletin no : 10334

SQL TIPS(9):

```

/*
** Name      : Show_Table_Constraints.Sql
**
** Usage     : BShow_Table_Constraints Table_Name
** Description : Returns The Constraint Name And Type
**
**
*/

SET VERIFY OFF
CLEAR BREAK
BREAK ON COLUMN_NAME ON CONSTRAINT_TYPE

SELECT SUBSTR(A.COLUMN_NAME,1,15)  COLUMN_NAME,
       DECODE(B.CONSTRAINT_TYPE, 'P','PRIMARY KEY',
                                'U','UNIQUE KEY',
                                'C','CHECK OR NOT NULL',
                                'R','FOREIGN KEY') CONSTRAINT_TYPE,
       A.CONSTRAINT_NAME  CONSTRAINT_NAME
FROM   USER_CONS_COLUMNS A, USER_CONSTRAINTS B
WHERE  A.TABLE_NAME = UPPER('E1')
AND    A.TABLE_NAME = B.TABLE_NAME
AND    CONSTRAINT_NAME = B.CONSTRAINT_NAME
ORDER BY 1,2:

```

< 실행 예 >

SQL> BSHOW_TABLE_CONSTRAINTS EMP_SERVICE

COLUMN_NAME	CONSTRAINT_TYPE	CONSTRAINT_NAME
CUST_ID	PRIMARY KEY	PK_EMP_SERVICE
	FOREIGN KEY	FK_EMP_SERVICE_CUST_ID
EMP_ID	PRIMARY KEY	PK_EMP_SERVICE
	FOREIGN KEY	FK_EMP_SERVICE_EMP_ID
LENGTH	CHECK OR NOT NULL	BYB_C001408B

주의 : BYB로 시작하는 CONSTRAINT명은 USER가 CONSTRAINT NAME를 지정하지 않아 SYSTEM에서 DEFAULT로 생성한 경우를 보여준다.

SQL TIPS(10): CHECK CONSTRAINT의 조건 확인

Bulletin no : 10935

SQL TIPS(10):

```
/*
** Name      : Show_Search_Condition_Bgl
**
** Usage     : @Show_Search_Condition_Constraint_Name
** Description : Show The Rule For A CHECK Constraint
**
**/
```

SET VERIFY OFF

SET ARRAYSIZE 1

SET LONG 75

```
SELECT SEARCH_CONDITION
FROM   USER_CONSTRAINTS
WHERE  CONSTRAINT_NAME = UPPER('&1');
```

< 실행 예 >

BQL> @SHOW_SEARCH_CONDITION CE_WAREHOUSE_PRODUCTION

SEARCH_CONDITION

```
(HEIGHT >= 0)
AND (LENGTH <= HEIGHT + 10)
AND (WIDTH <= LENGTH + 10)
```

SQL TPS(11) :TABLE에 대하여 EXTENT 발생량 산정방법

Bulletin no : 10351

/* 모든 TABLE에 대하여 각각의 EXTENT 발생량을 산정한다 */

```
set linesize 80
set pages 88
set echo off
set timeout off
spool report/extent.txt

column owner no print new_value owner
break on owner page
title on 'Extent Definition Owner : ' owner skip 1 -
       on '-----' skip 1;

col f1 format a40 heading 'OBJECT NAME'
col f2 format a10 heading 'TYPE'
col f3 format 88,888,888 heading 'TOTAL EXTS'
select a.owner owner, a.extent_name f1,
       extent_type f2,
       extents f3
from dba_extents a
where a.owner not in ('SYS','SYSTEM','SCOTT')
and extents > 1
/
spool off
exit
```

SQL TIPS(12): PRINTING A HIERARCHICAL LIST OF REFERRING AND REFERENCED

Bulletin no : 10384

This SQL script generates a hierarchical list of tables with a level number (tables that are linked with foreign key constraints), from any Oracle schema. The list is useful in identifying the 'order of tables', while loading data, dropping tables or deleting rows from tables.

The script collects referential integrity constraints data from the data dictionary tables, creates and populates temporary tables and prints the hierarchical table list.

PURPOSE: Print an Hierarchical list of referring and referenced tables in any Oracle Schema.

```
-----
FILE   : table_level.sql
OUTPUT : level_tree.txt
USAGE  : sqlplus userId/password @table_level.sql

/
--
-- Create temporary tables, collecting data from Dictionary tables
-- -----

set timeout off
drop table parent_child_tab
/
drop table parent_child_temp_tab
/
drop table level_tab
/
drop table parent_child_level
/
create table parent_child_level (p_level number, c_level number)
/
insert into parent_child_level values (1,1)
/
commit
/
```

```

create table parent_child_tab as
  (select x.p_level,
         p.table_name parent_table_name,
         p.owner p_owner,
         x.c_level,
         c.table_name child_table_name,
         c.constraint_name c_constraint_name,
         c.owner c_owner

    from user_constraints p, user_constraints c,
         parent_child_level x
   where p.constraint_type IN ('P','U') AND
         c.constraint_type = 'R' AND
         p.constraint_name = c.p_constraint_name )
/

-----
-- Collect data if the referenced tables are in another schema.
-----

create table parent_child_temp_tab as
  (select x.p_level,
         a.table_name parent_table_name,
         c.p_owner p_owner,
         x.c_level,
         c.table_name child_table_name,
         c.constraint_name c_constraint_name,
         c.owner c_owner

    from user_constraints c, all_constraints a,
         parent_child_level x
   where c.owner <> c.p_owner and
         c.p_owner = a.owner and
         c.p_constraint_name = a.constraint_name )
/

insert into parent_child_tab
  ( p_level, c_constraint_name, parent_table_name, p_owner,
    c_level, child_table_name , c_owner)
  (select p_level, c_constraint_name, parent_table_name, p_owner,
         c_level, child_table_name , c_owner
    from parent_child_temp_tab )
/

create table level_tab
  ( table_name    varchar2(30),
    level_number  number )
/

-----
-- << Create performance indexes on the tables - val >>
-----

```



```

create index child_level_name_ix
    on PARENT_CHILD_TAB(child_tab_name,c_level,c_name)
/

create index parent_name_ix
    on PARENT_CHILD_TAB(parent_tab_name,c_name)
/

create index level_tab_name_ix
    on LEVEL_TAB(tab_name)
/

create index level_level_tab_ix
    on LEVEL_TAB(level_number,tab_name)
/

-----
-- The following pl/sql block will populate the 'level_tab' table
-- with 'tab_name' and 'level number'.
-- At the end it updates 'parent_child_tab' with level numbers.
-----

declare
    dummy_val      char(1);
    level_num      number;
    total_level    number;
    tab_name       varchar2(30);
    child_tab_name varchar2(30);
--
    cursor get_child_tab_name(pk1 number) is
        select distinct child_tab_name
        from parent_child_tab
        where parent_tab_name in (select tab_name
                                from level_tab
                                where level_number = pk1 ) ;
--
    cursor delete_duplicates(pk2 number) is
        select tab_name
        from level_tab
        where level_number = pk2 ;
--
begin
-----
-- Insert all Master table names with out any foreign key
-- definition, into 'level_tab' table.
-----

insert into level_tab
(select distinct parent_tab_name, 1
 from parent_child_tab
 where parent_tab_name not in
   (select child_tab_name from parent_child_tab) ) ;

commit;

```

```

-----
-- << Update self-referencing FK's by appending an '0' onto the end of
--      the parent_table_name - val >>
-----

update parent_child_tab
  set parent_table_name = parent_table_name || '0'
 where parent_table_name = child_table_name
    and p_owner = c_owner;
commit;

-----
-- << Update bi-directional FK's by appending an '1' onto the end of
--      the parent_table_name - val >>
-----

update parent_child_tab
  set parent_table_name = parent_table_name || '1'
 where p_owner||', '||parent_table_name||';'||
       c_owner||', '||child_table_name||' is
       (select c_owner||', '||child_table_name||';'||
          p_owner||', '||parent_table_name
          from parent_child_tab);
commit;

-----
-- Identify all tables with foreign key definition, calculate
-- level number and insert 'table name', 'level number'
-- into 'level_tab' table.
-----

level_num := 1;
<>
loop
  open get_child_tab_name(level_num);
  fetch get_child_tab_name into child_tab_name; /* first record */
  if get_child_tab_name%notfound then
    total_level := level_num;
    -- 'total_level' variable gives highest depth value in the Schema
    close get_child_tab_name;
    exit outer_loop;
  else
    level_num := level_num + 1;
    insert into level_tab values(child_tab_name, level_num);
  end if;
  <>
  loop
    fetch get_child_tab_name into child_tab_name; /* other records */
    exit inner_loop when get_child_tab_name%notfound;
    insert into level_tab values(child_tab_name, level_num);
  end loop inner_loop;
  close get_child_tab_name;
end loop outer_loop;

```

```

consult:
-----
-- Keep only the highest level number row for a table name
-- and delete rest of the duplicate rows from 'level_tab'.
-----
<>
for loop_var in reverse 1..total_level: loop
  open delete_duplicates(loop_var);
  <>
  loop
    fetch delete_duplicates into tab_name;
    exit for_inner_loop when delete_duplicates%notfound;
    delete from level_tab
      where level_number < loop_var and table_name = tab_name;
  end loop for_inner_loop;
  close delete_duplicates;
end loop for_loop;
consult:
-----
-- update level numbers into parent_child_tab table
-- by fetching the value from 'level_tab' table.
-----
update parent_child_tab
  set p_level = (select level_number
                  from level_tab
                  where table_name = parent_table_name );
consult:
--
update parent_child_tab
  set c_level = (select level_number
                  from level_tab
                  where table_name = child_table_name );
consult:
end ;
/
-----
-- prepare spoolfile 'level_tree.llc' to capture the output
-- generated by dbms_output utility package
-----
--
set serveroutput on size 1000000
-- set linesize 132
spool level_tree.llc
--
-- -----
-- Format and print the output
-- -----

```

```

declare
    level_num          number;
    text_line          varchar2(132);
--
    cursor get_parent_child_tab is
        select distinct child_table_name, c_level, c_owner
        from parent_child_tab
        order by 2 desc, 1 ;
--
    cursor get_level_num_tab is
        select distinct parent_table_name, p_owner from parent_child_tab
        where p_level = 1
        order by 1 ;
--
    cursor print_tree(key1 varchar2) is
        select lpad(' ',2*(level)) lpad_value,
               parent_table_name t_name, p_owner, c_owner, c_constraint_name
        from parent_child_tab
        start with child_table_name = key1
        connect by prior parent_table_name = child_table_name;
--
    cursor get_level_numbers(key2 varchar2) is
        select level_number from level_tab
        where table_name = key2;
--
begin
    for xrec in get_parent_child_tab loop
        dbms_output.put_line(
            xrec.c_owner||', '||xrec.child_table_name||'('||xrec.c_level||')');
        for yrec in print_tree(xrec.child_table_name) loop
            open get_level_numbers(yrec.t_name);
            fetch get_level_numbers into level_num;
            close get_level_numbers;
            text_line := null;
            if level_num = 1 then
                text_line := '- '||yrec.lpad_value||yrec.p_owner||', '
                    ||yrec.t_name||'('||level_num||')';
            else
                text_line := '- '||yrec.lpad_value||yrec.c_owner||', '
                    ||yrec.t_name||'('||level_num||')';
            end if;
        end loop;
    end loop;

```

```

-----
-- << If you don't want the constraint name listed, comment-out
-- the following line of code.
-- SET LINESIZE is located at the SPPOOL statement some 40+ lines above.
-- For padding out to fill an 80 column page, spad to 47.
-- For padding out to fill a 132 column page, spad to 88 - end >>
-----

text_line := spad(text_line,47,' ');
            ||'('||xrec.c_constraint_name||')';
-----

            dbms_output.put_line(text_line);
        end loop;
        dbms_output.put_line('-----');
    end loop;
    for xrec in get_level_one_tab loop
        dbms_output.put_line(
            xrec.p_name||', '||xrec.parent_table_name||'(1)');
    end loop;
end ;
/
-----
spool off

exit
-----

/* *****
The following scripts are included for operating against the
PARENT_CHILD_TAB. The first two provide an 'immediate neighbour' picture
one level above or below this table:

run

run  FINDREF.BQL - tables whose key is referenced by a field in this table
run
            (immediate 'neighbour' - 1 level outward) - end Beg/BB
run

column ref heading 'Has as FK named ...' format a35
column tab heading 'Pointing to This Table's PK' format a35
title center 'This Table: &&1.' BREP 2

set feedback off
set verify off
set pagesize 80

select c_constraint_name ref, parent_table_name tab
-- or if you want 'pure' names translate(parent_table_name,'91',' ') tab
from parent_child_tab
where child_table_name = upper('&&1.') -- 'normal' name
      or child_table_name = upper('&&1.9') -- self-referencing name
      or child_table_name = upper('&&1.1') -- bi-directional name

```

```

order by 2,1
/

run

run  FEDEF,BQL - tables which reference this table's key
run                (immediate 'neighbours' - 1 level only) - see Rep/BB
run
run  NOTE: If you run this and specify the table name with an 0 or 1,
run        you will get a listing of all self-referencing relationships (0)
run        or all tables which refer to each other (1 bi-directional)
run

column tab heading "... In This Table" format a35
column ref heading "Is Referenced by This FEKey" format a35
title center 'The FE of This Table: &&1.' BEIP 2

set feedback off
set verify off
set pagesize 80

select c_constraint_name ref, child_table_name tab
-- or if you want 'pure' names translate( child_table_name,'01',' ') tab
from parent_child_tab
where parent_table_name = upper('&&1.0') -- 'normal' name
    or parent_table_name = upper('&&1.0') -- self-referencing name
    or parent_table_name = upper('&&1.1') -- bi-directional name
order by 2,1
/

A third script provides a subset of the entire tree starting with
the 'seed' table. It's like a recursive FENAME,BQL listing:

run

run  FENAME,BQL - a subset of the entire application's FE tree
run
run  NOTE: It may be necessary to run this for each of
run        &&1, &&1.IID , and &&1.III to get the entire subset picture
run        depending on the complexity of your application's relationships
run
run                                     - see Rep/BB

column ref heading "Is Referenced BY These Tables" format a40
column fk heading "... via This Foreign Key" format a30
column parent_table_name NEW_VALUE a newline format a1 trunc
break on parent_table_name skip 0 on report
title center 'This Table's Primary Key: &&1.' BEIP 2

```

```

set pagesize 80
set linesize 78
set newpage 0
set verify off

select parent_table_name,
       lpad('>',2*(LEVEL-1))||child_table_name ref,
       c_constraint_name PK
  from parent_child_tab
start with parent_table_name = UPPER('d11,')
connect by parent_table_name = PRIOR child_table_name
/

```

SQL TIPS(13): DROPPING DATABASE OBJECTS FOR A USER

Bulletin no : 10985

```
--
--
--MODULE NAME:
--    drop_obj.sql
--
--SYNOPSIS:
--    sqlplus / @drop_obj.sql
--
--DESCRIPTION:
--    This script will create another SQL script which will
--    drop all of the objects owned by a user.
--
--
--
set pages 5000
set echo off
set heading off
set feedback off

spool drop.sql

select 'spool drop.11c' from dual
/

SELECT 'DROP ' || OBJECT_TYPE || ' '
       || OBJECT_NAME ||
       DECODE(OBJECT_TYPE, 'TABLE', ' CASCADE CONSTRAINTS', NULL)
       || ';'
FROM USER_OBJECTS
ORDER BY DECODE(OBJECT_TYPE, 'INDEX', 'A_INDEX',
                'TRIGGER', 'A_TRIGGER',
                OBJECT_TYPE),
        OBJECT_NAME
/

select 'spool off' from dual
/
```


spool off

set echo on

set heading on

set feedback on

run

run end of script

run

SQL TIPS(14): PROVIDING USERS WITH SESSION INFORMATION

Bulletin no : 10388

```

run
run oraclesql
run
-----
--
run
run This script is used to provide users with information regarding their oracle
run sessions. The USER_SESSION view provides information related only to the
run current session for a user; while the ALL_SESSIONS relates to all sessions
run within the database for a user. The DBA_SESSION can only be viewed by the
run Oracle sys id (and anyone granted select on this view). The DBA_SESSION
run lists all oracle sessions excluding the Oracle generic processes (dbs,
run pmon, smon, etc.). The SID and SERIAL# can be used with killing sessions.
run
run This script must be run from the Oracle sys id.
run
run
-----
--
run

set echo on;
spool oraclesql;

run
-----
--

drop public synonym ALL_SESSIONS;

create or replace view ALL_SESSIONS
    (USERNAME, OS_USER, SID, SERIAL#, STATUS, SERVICE, DATABASE,
    CURSOR, MACHINE, TERMINAL, PROGRAM)
as
select substr(v1.username,1,20), substr(v1.osuser,1,12),
    to_char(v1.sid,'BBB'), to_char(v1.serial#, 'BBBBBB'),
    v1.status, v1.osuser, substr(v3.value,1,10), v2.value,
```

```

        substr(v1.machine,1,15), v1.terminal, v1.program
    from v$session v1, v$sessionstat v2, v$parameter v3
    where v1.username = user
        and v2.sid      = v1.sid
        and v2.statistic# = 3
        and v3.name      = 'db_name' ;

```

```

grant select on      ALL_SESSIONS to PUBLIC;
create public synonym ALL_SESSIONS for ALL_SESSIONS;

```

run

```

drop public synonym USER_SESSIONS;

```

```

create or replace view USER_SESSIONS
    (USERNAME, OS_USER, SID, SERIAL#, STATUS, SERVICE, DATABASE,
     CURSOR#, MACHINE, TERMINAL, PROGRAM)

```

as

```

select substr(v1.username,1,20), substr(v1.osuser,1,12),
        to_char(v1.sid,'BBB'), to_char(v1.serial#, 'BBBBBB'),
        v1.status, v1.osuser, substr(v3.value,1,10), v2.value,
        substr(v1.machine,1,15), v1.terminal, v1.program
    from v$session v1, v$sessionstat v2, v$parameter v3
    where v1.sid      = userenv('SESSIONID')
        and v2.sid      = v1.sid
        and v2.statistic# = 3
        and v3.name      = 'db_name' ;

```

```

grant select on      USER_SESSIONS to PUBLIC;
create public synonym USER_SESSIONS for USER_SESSIONS;

```

run

```

create or replace view DBA_SESSIONS
    (USERNAME, OS_USER, SID, SERIAL#, STATUS, SERVICE, DATABASE,
     CURSOR#, MACHINE, TERMINAL, PROGRAM)

```

as

```

select substr(v1.username,1,20), substr(v1.osuser,1,12),
        to_char(v1.sid,'BBB'), to_char(v1.serial#, 'BBBBBB'),
        v1.status, v1.osuser, substr(v3.value,1,10), v2.value,
        substr(v1.machine,1,15), v1.terminal, v1.program
    from v$session v1, v$sessionstat v2, v$parameter v3
    where v1.username is not null
        and v2.sid      = v1.sid

```

```
and v2.statistic# = 3  
and v3.name      = 'dk_name' ;
```

```
from -----
```

```
pool off;
```

SQL TIPS(15): FORECASTING ON NEXT EXTENTS

Bulletin no : 10387

A DBA needs to know when a table or index is reaching its maximum number of extents, and you can use the SQL*PLUS script `KEXTENTS.BQL` to report this information. The output lists tables/indexes that are within "K" extents of reaching their maximum. `KEXTENTS.BQL` is operating-system independent.

```
/*
Constructs:
(1) Find out the Oracle data block size by querying V$PARAMETER:
(2) Translate it into a maximum number of extents. Because the size of the Oracle data block
    is operating-system dependent, the following approach:
    select decode(value, 2048, 121, 4096, 240) from v$parameter is not appropriate.
(3) Without the loss of generality, a weighted average method is used for constructing
    a stored function named expected_extents. This function is DB independent and it returns
    max_ext when called.
*/

/*
    KEXTENTS.BQL
*/

REM
REM Creating stored function expected_extents.
REM

DROP FUNCTION expected_extents;

CREATE OR REPLACE FUNCTION expected_extents(vkey IN varchar2)
RETURN varchar2 IS
    n_value varchar2(20);
BEGIN
    n_value:= ROUND(((1/2)*((121/2048)+(240/4096))*vkey, 0);
    RETURN n_value;
END expected_extents;
/

REM
```

```

REM Generating a forecasting report.
REM

```

```

set linesize 79;

```

```

title
  center 'Report of Next Extent Within ' &N ' of maximum extent' skip -
  center 'Date: ' datevar skip -
  right '(T/I - Table/Index)' skip 2

```

```

column today      max_value datevar      format A10 right;
column owner      max_value owner        format A8;
column object     max_value object       format A15;
column tx         max_value tablespace   format A25;

```

```

select  sysdate              "today",
        t.owner              "owner",
        t.table_name        "object",
        t.tablespace_name    "tx",
        'T/I'               "T/I",
        t.max_extent        "max_extent",
        s.extent             "extent"
from     sys.dba_tables      t,
        sys.dba_segments    s,
        v$parameter          p

```

```

where    table_name = segment_name
and      (t.max_extent < s.extent + &N or
expected_extent(p.value) < s.extent + &N)
and      p.name = 'db_block_size'

```

```

UNION

```

```

select  sysdate,
        l.owner,
        l.index_name,
        l.tablespace_name,
        'I'              Indicator,
        l.max_extent max_extent,
        s.extent current_extent

```

```

from     sys.dba_indexes      l,
        sys.dba_segments     s,
        v$parameter          p

```

```

where    index_name = segment_name
and      (l.max_extent < s.extent + &N or
expected_extent(p.value) < s.extent + &N)
and      p.name = 'db_block_size';

```

```

EXIT;

```

SQL TIPS(16): CREATING SCRIPTS TO RECREATE A TABLE STRUCTURE

Bulletin no : 10988

The script creates scripts that can be used to recreate a table structure. For example, this script can be used when a table has become fragmented or to get a definition that can be run on another database.

/4

CREATE SCRIPT TO RECREATE A TABLE-STRUCTURE
INCL. STORAGE, CONSTRAINTS, TRIGGERS ETC.

This script creates scripts to recreate a table structure.
Use the script to reorganize a table that has become fragmented,
to get a definition that can be run on another database/schema or as
a basis for altering the table structure (eg. drop a column).

IMPORTANT: Running the script is safe as it only creates two new scripts and does not do anything to your database! To get anything done you have to run the scripts created.

The created scripts does the following:

1. save the content of the table
2. drop any foreign key constraints referencing the table
3. drop the table
4. create the table with an initial storage parameter that will accommodate the entire content of the table. The Next parameter is 25% of the initial.
The storage parameters are picked from the following list:
84K, 128K, 256K, 512K, multiples of 1K.
5. create table and column comments
6. fill the table with the original content
7. create all the indexes incl storage parameters as above.
8. add primary, unique key and check constraints.
9. add foreign key constraints for the table and for referencing tables.
10. Create the table's triggers.
11. Compile any depending objects (cascading).
12. Grant table and column privileges.
13. Create synonyms.

This script must be run as the owner of the table.
 If your table contains a LONG-column, use the COPY command in SQL*Plus to store/restore
 the data.

USAGE

from SQL*Plus:

start scriptk

This will create the scripts REORGES1.BQL and REORGES2.BQL

REORGES1.BQL contains code to save the current content of the table.

REORGES2.BQL contains code to rebuild the table structure.

```

-----
-----
*/

undef tsk;
set echo off
column s1 new_val stop
column k1 new_val set

select
decode(ceil(1024-sum(bytes)/1024),-1,to_char((round(sum(bytes)/(1024*1
024))+1)||'M', /* > 1M now rounded up to nearest Megabyte */
decode(ceil(512-sum(bytes)/1024), -1, '1M',
decode(ceil(256-sum(bytes)/1024), -1, '512K',
decode(ceil(128-sum(bytes)/1024), -1, '256K',
decode(ceil(64-sum(bytes)/1024), -1, '128K',
          'B4K'
))))
s1,
decode(ceil(1024-sum(bytes)/4096),-1,to_char((round(sum(bytes)/(4096*1
024))+1)||'M', /* > 1M now rounded up to nearest Megabyte */
decode(ceil(512-sum(bytes)/4096), -1, '1M',
decode(ceil(256-sum(bytes)/4096), -1, '512K',
decode(ceil(128-sum(bytes)/4096), -1, '256K',
decode(ceil(64-sum(bytes)/4096), -1, '128K',
          'B4K'
))))
k1
from user_extents
where segment_name=upper('t1');
```



```
set pages 0 feed off verify off lines 150
col c1 format a80
```

```
spool merge1.sql
```

```
PROMPT drop table tk_#1
```

```
prompt /
```

```
PROMPT create table tk_#1 storage (initial 8000) as select * from #1
```

```
prompt /
```

```
spool off
```

```
spool merge2.sql
```

```
PROMPT spool merge2
```

```
select 'alter table '||table_name||' drop constraint
```

```
'||constraint_name||';'
```

```
from user_constraints where r_constraint_name
```

```
is (select constraint_name from user_constraints where
```

```
table_name=upper('#1')
```

```
and constraint_type in ('P','U'));
```

```
PROMPT drop table #1
```

```
prompt /
```

```
prompt create table #1
```

```
select decode(column_id,1, '('','')'
```

```
||pad(column_name,40)
```

```
||decode(data_type,'DATE','DATE','
```

```
','LONG','LONG','
```

```
','LONG RAW','LONG RAW','
```

```
','RAW','RAW','
```

```
','CHAR','CHAR','
```

```
','VARCHAR','VARCHAR','
```

```
','VARCHAR2','VARCHAR2','
```

```
','NUMBER','NUMBER','
```

```
','unknown')
```

```
||pad(
```

```
decode(data_type,'DATE',null
```

```
','LONG',null
```

```
','LONG RAW',null
```

```
','RAW',decode(data_length,null,null
```

```
','('||data_length||')')
```

```
','CHAR',decode(data_length,null,null
```

```
','('||data_length||')')
```

```

        , 'VARCHAR' , decode(data_length,null,null
                                , '||data_length||')
        , 'VARCHAR2' , decode(data_length,null,null
                                , '||data_length||')
        , 'NUMBER'   , decode(data_precision,null,' '
                                , '||data_precision||

```

```

decode(data_scale,null,null

```

```

        , '||data_scale||')
        , 'unknown' , B,' '
        ||decode(nullable,'Y','NULL','NOT NULL') c1
from user_tab_columns
where table_name = upper('&1')
order by column_id
/

```

prompt)

```

select 'pctfree      '||t.pct_free   c1
      , 'pctused     '||t.pct_used   c1
      , 'initrans    '||t.ini_trans  c1
      , 'maxtrans    '||t.max_trans  c1
      , 'tablespace   '||t.tablespace_name c1
      , 'storage (initial '||'factor'   c1
      , '          next    '||'factor'   c1
      , '          increments '||t.inc_extents c1
      , '          maxextents '||t.max_extents c1
      , '          pctincrease '||t.pct_incrinc||')' c1
from user_segments s, user_tables t
where s.segment_name = upper('&1') and
      t.table_name = upper('&1')
and s.segment_type = 'TABLE'
/

```

prompt /

```

select 'comment on table &1 is '''||comment||''';' c1 from
user_tab_comments
where table_name=upper('&1');

```

```

select 'comment on column &1. '||column_name||
' is '''||comment||''';' c1 from user_col_comments
where table_name=upper('&1');

```

prompt insert into &1 select * from tk_&1

```

prompt /

set serveroutput on

declare
cursor c1 is select index_name,decode(uniqname,'UNIQUE','UNIQUE')
        asq
from user_indexes where
table_name = upper('&1');
index_name varchar2(50);
cursor c2 is select
        decode(column_position,1,','','')||pad(column_name,40) c1
from user_ind_columns where table_name = upper('&1') and
        index_name = index_name
order by column_position;
i1 varchar2(100);
i2 varchar2(100);
i3 varchar2(100);
i4 varchar2(100);
i5 varchar2(100);
i6 varchar2(100);
i7 varchar2(100);
i8 varchar2(100);
i9 varchar2(100);
begin
dhex_output.put_line(100000);
for c in c1 loop
        dhex_output.put_line('create '||c.asq||' index '||c.index_name||' on
&1');
        index_name := c.index_name;
        for q in c2 loop
                dhex_output.put_line(q.c1);
        end loop;
        dhex_output.put_line('');
select 'partname          '||i1||',part_name          ',
        'in1name          '||i1||',in1_name          ',
        'maxname          '||i1||',max_name          ',
        'tablespace          '||i1||',tablespace_name          ',
        'storage (initial          '||i1
decode(c1gn(1024-sum(n,bytes)/1024),-1,
to_char(floor(sum(n,bytes)/(1024*1024))+1))||'M',
decode(c1gn(512-sum(n,bytes)/1024),-1,'1M',
decode(c1gn(256-sum(n,bytes)/1024),-1,'512K',
decode(c1gn(128-sum(n,bytes)/1024),-1,'256K',
decode(c1gn(64-sum(n,bytes)/1024),-1,'128K',
        'B4K'))))',

```

```

        next      '11
decode(c_lg((1024-sum(m_bytes)/4096),-1,
to_char(round(sum(m_bytes)/(4096*1024)+1))||'H',
decode(c_lg((512-sum(m_bytes)/4096), -1, '1H',
decode(c_lg((256-sum(m_bytes)/4096), -1, '512E',
decode(c_lg((128-sum(m_bytes)/4096), -1, '256E',
decode(c_lg((64-sum(m_bytes)/4096), -1, '128E',
        'B4E')))) ,
        minextents '11e,min_extents
        maxextents '11e,max_extents
        pct_increase '11e,pct_increase11')
into 11,12,13,14,15,16,17,18,19
from user_extents u,user_segments s, user_indexes i
where s.segment_name = c.index_name
and s.segment_type = 'INDEX'
and i.index_name = c.index_name
and s.segment_name=s.segment_name
group by s,min_extents,s,max_extents,s,pct_increase,
1,pct_free,1,1st_trunc,1,max_trunc,1,takingspace_name ;
      dbms_output.put_line(11);
      dbms_output.put_line(12);
      dbms_output.put_line(13);
      dbms_output.put_line(14);
      dbms_output.put_line(15);
      dbms_output.put_line(16);
      dbms_output.put_line(17);
      dbms_output.put_line(18);
      dbms_output.put_line(18);
      dbms_output.put_line('/');
end loop;
end;
/

declare
cursor c1 is
select constraint_name, decode(constraint_type,'U',' UNIQUE',' PRIMARY
KEY') typ,
decode(status,'DISABLED','DISABLE',' ') status from user_constraints
where table_name = upper('E1')
and constraint_type in ('U','P');
column varchar2(100);
cursor c2 is
select decode(position,1,'(',')')||pad(column_name,40) coln
from user_cons_columns
where table_name = upper('E1')
and constraint_name = cname

```

```

order by position;
begin
for q1 in c1 loop
    cname := q1.constraint_name;
    dbms_output.put_line('alter table #1');
    dbms_output.put_line('add constraint '||cname||q1.type);
    for q2 in c2 loop
        dbms_output.put_line(q2.col);
    end loop;
    dbms_output.put_line('') || q1.status;
    dbms_output.put_line('/');
end loop;
end;
/

declare
cursor c1 is
select c.constraint_name,c.r_constraint_name cname2,
       c.table_name table1, r.table_name table2,
       decode(c.status,'DISABLED','DISABLE','') status,
       decode(c.delete_rule,'CASCADE',' on delete cascade ','')
delete_rule
from   user_constraints c,
       user_constraints r
where  c.constraint_type='R' and
       c.r_constraint_name = r.constraint_name and
       c.table_name = upper('#1')
union
select c.constraint_name,c.r_constraint_name cname2,
       c.table_name table1, r.table_name table2,
       decode(c.status,'DISABLED','DISABLE','') status,
       decode(c.delete_rule,'CASCADE',' on delete cascade ','')
delete_rule
from   user_constraints c,
       user_constraints r
where  c.constraint_type='R' and
       c.r_constraint_name = r.constraint_name and
       r.table_name = upper('#1');
cname varchar2(50);
cname2 varchar2(50);
cursor c2 is
select decode(position,1,('','')||pad(column_name,40) column
from user_constraints
where  constraint_name = cname
order by position;
cursor c3 is

```

```

select decode(position,1,'(',')')||pad(column_name,40) refcol
from user_constraints
where constraint_name = cname2
order by position;
begin
dhex_output.put_line(100000);
for q1 in c1 loop
    cname := q1.constraint_name;
    cname2 := q1.constraint;
    dhex_output.put_line('alter table '||q1.table||' add constraint ');
    dhex_output.put_line(cname||' foreign key');
    for q2 in c2 loop
        dhex_output.put_line(q2.column);
    end loop;
    dhex_output.put_line(') reference '||q1.table2);
    for q3 in c3 loop
        dhex_output.put_line(q3.refcol);
    end loop;
    dhex_output.put_line(') '||q1.delete_rule||q1.status);
    dhex_output.put_line('/');
end loop;
end;
/

```

```

col c1 format a78 word_wrap
set long 32000
set arraysize 1

```

```

select 'create or replace trigger ' c1,
       description c1,
       'WHEN ('||when_clause||')' c1,
       trigger_body ,
       '/' c1
from user_triggers
where table_name = upper('t1') and when_clause is not null
/
select 'create or replace trigger ' c1,
       description c1,
       trigger_body ,
       '/' c1
from user_triggers
where table_name = upper('t1') and when_clause is null
/

```

```

select 'alter trigger '||trigger_name||decode(status,'DISABLED','

```

```

DISABLE',' ENABLE')
from user_triggers where table_name='E1';

set serveroutput on

declare
cursor c1 is
select 'alter table
'||'E1'||decode(substr(control_name,1,4),'BYB_',' ',
      ' add constraint ') a1,
      decode(substr(control_name,1,4),'BYB_',' ',
' ,constraint_name)||' check (' a2,
      search_condition a3,
      ') '||decode(status,'DISABLED','DISABLE','') a4,
      '/' a5
from user_constraints
where table_name = upper('E1') and
constraint_type='C';
k1 varchar2(100);
k2 varchar2(100);
k3 varchar2(32000);
k4 varchar2(100);
k5 varchar2(100);
f1 number;
begin
open c1;
loop
  fetch c1 into k1,k2,k3,k4,k5;
  exit when c1%NOTFOUND;
  select count(*) into f1 from user_tab_columns where table_name =
upper('E1') and
      upper(column_name)||' IS NOT NULL' = upper(k3);
  if f1 = 0 then
    dbms_output.put_line(k1);
    dbms_output.put_line(k2);
    dbms_output.put_line(k3);
    dbms_output.put_line(k4);
    dbms_output.put_line(k5);
  end if;
end loop;
end;
/

create or replace procedure dmccvcsorg_deploy varchar2,typ
varchar2) as

```

```

cursor cur is
select type, decode(type, 'PACKAGE BODY', 'PACKAGE', type) type1,
name from user_dependencies
where referenced_name=upper(name) and referenced_type=upper(type);
begin
dhex_output, nvarchar(500000);
for c in cur loop
    dhex_output.put_line('alter '||c.type1|| ' '||c.name1||' compile;');
    dxxxxxvncorg_dep(c.name, c.type);
end loop;
end;
/
exec dxxxxxvncorg_dep('E1', 'TABLE');

drop procedure dxxxxxvncorg_Dep;

select 'grant '||priv||ingall' on '||table_name1||' to '||grantname1
decode(granttable, 'YES', ' with grant option', '') from
user_tab_privs where table_name = upper('E1');

select 'grant '||priv||ingall' ('||column_name1||') on E1 to
' ||grantname1
decode(granttable, 'YES', ' with grant option', '')
from user_col_privs where grantor=user and
table_name=upper('E1')
order by grantor, privileges;

select 'create synonym '||ksynonym_name1||' for
' ||table_owner1||', ' ||table_name1||';'
from user_synonyms where table_name=upper('E1');

PROMPT BEN
PROMPT BEN YOU MAY HAVE TO LOG ON AS SYSTEM TO BE
PROMPT BEN ABLE TO CREATE ANY OF THE PUBLIC SYNONYMS!
PROMPT BEN

select 'create public synonym '||ksynonym_name1||' for
' ||table_owner1||', ' ||table_name1||';'
from all_synonyms where owner='PUBLIC' and table_name=upper('E1') and
table_owner=user;

prompt sql> off

sql> off

set echo on feed on verify on

```


/*

The scripts REORGCS1.SQL and REORGCS2.SQL have been created. Alter these script as necessary.
To recreate the table-structure, first run REORGCS1.SQL.
This script saves the content of your table in a table called tk_.
If this script runs successfully run REORGCS2.SQL.
The result is spooled to REORCSYB.LST.
Check this file before dropping the tk_ table.

*/

SQL TIPS(17): AVAILABLE SPACE, FRAGMENTATION, AND EXTENTS

Bulletin no : 10988

The script below lists available space, fragmentation, and potential extent problems.

```
/*
This script displays information about space in tablespaces, fragmentation and
potential extent-problems. It is divided in 3:

1. Space available per tablespace.
   Shows total size in bytes, total free space in bytes, percentage free space
   to total space, the size of the largest contiguous free segment and the number of
   free segments.

2. Lists tables and indexes with more than 20 extents.

3. Lists tables/indexes whose next extent will not fit into their tablespace.
*/

set pagination 300

column chunk format 999,999,999
column extents format 9999
column bytes format 999,999,999
column largest format 999,999,999
column Tot_Size format 999,999,999
column Tot_Free format 999,999,999
column Pct_Free format 999,999,999
column Chunks_Free format 999,999,999
column Max_Free format 999,999,999
set echo off
spool space

PROMPT SPACE AVAILABLE IN TABLESPACES

select s.tablespace_name, sum(s.totals) Tot_Size,
       sum(s.chunk) Tot_Free,
       sum(s.chunk)*100/sum(s.totals) Pct_Free,
       sum(s.largest) Max_Free, sum(s.chunks) Chunks_Free
from
```

```

(
select tablespace_name,0 tota,sum(bytes) sumb,
max(bytes) largest,count(*) chunks
from dba_free_space a
group by tablespace_name
union
select tablespace_name,sum(bytes) tota,0,0,0 from
  dba_data_files
group by tablespace_name) a
group by a.tablespace_name;

```

```

column owner format a15
column segment_name format a30

```

PROMPT SEGMENTS WITH MORE THAN 20 EXTENTS

```

select owner,segment_name,extents,bytes ,
max_extents,next_extent
from dba_segments
where segment_type in ('TABLE','INDEX') and extents>20
order by owner,segment_name;

```

PROMPT SEGMENTS WHERE THERE'S NOT ENOUGH ROOM FOR THE NEXT EXTENT

```

select a.owner, a.segment_name, b.tablespace_name,
       decode(ext,extents,1,b.next_extent,
       a.bytes*(1+b.pct_increase/100)) nextext,
       framp.largest
from   dba_extents a,
       dba_segments b,
       (select owner, segment_name, max(extent_id) extent_id,
              count(*) extents
        from dba_extents
        group by owner, segment_name
       ) ext,
       (select tablespace_name, max(bytes) largest
        from dba_free_space
        group by tablespace_name
       ) framp
where  a.owner=b.owner and
       a.segment_name=b.segment_name and
       a.owner=ext.owner and
       a.segment_name=ext.segment_name and
       a.extent_id=ext.extent_id and

```

```

h.tblspace_name = femp.tblspace_name and
decode(ext, extsize, 1, h.next_extsize,
        n, htabc*(1+h.pct_increase/100)) > femp.largest
/

spool off

```

SQL TIPS(18): DBMS_OUTPUT을 이용하여 SQL*PLUS 에서 REPORT 작성하기

Bulletin no : 10370

SQL*PLUS에서 원하는 값을 화면에 DISPLAY하여 확인하거나 REPORT로 출력해 보려면 어떻게 해야 할
까?

I. HINT

1. DBMS_OUTPUT PACKAGE를 사용한다.

I SQL*PLUS상에서 SET SERVEROUTPUT ON을 하거나

I DBMS_OUTPUT.ENABLE를 사용한다.

2. 필요하다면 PL/SQL BUILT-IN FUNCTION을 사용한다.

3. CHAR datatype을 출력할 때는 RPAD를 사용한다.

4. NUMBER나 INTEGER datatype을 출력할 때는 LPAD를 사용한다.

II. EXAMPLE : TABLESPACE의 사용 정도에 대한 REPORT작성

SET SERVEROUTPUT ON;

DECLARE

```
VAR_BYTES NUMBER(8);
CURSOR FIND_INFO IS
SELECT SUM(B.(D.TABLESPACE_NAME, 1, 22))    TSPACE,
       D.FILE_ID                               FILE_ID,
       ROUND(D.BYTES/1024/1024, 4)            TOT_MB,
       D.BYTES                                  DBA_BYTES,
       SUM(E.BLOCKS)                           TOT_BLOCKS,
       ROUND(SUM(E.BLOCKS)/D.BYTES, 4)*100    PCT_USED
FROM   SYS.DBA_DATA_FILES D,
       SYS.DBA_EXTENTS E
WHERE  D.FILE_ID = E.FILE_ID (+)
GROUP BY D.TABLESPACE_NAME,
         D.FILE_ID,
         D.BYTES/1024/1024,
         D.BYTES;
```

BEGIN

```
DBMS_OUTPUT.PUT_LINE(
  RPAD('TSPACE', 20, ' ') ||
  LPAD('FILE_ID', 10, ' ') ||
  LPAD('TOT_MB', 10, ' ') ||
  LPAD('DBA_BYTES', 10, ' ') ||
  LPAD('TOT_BLOCKS', 10, ' ') ||
  LPAD('PCT_USED', 10, ' ') ||
```

```

        LPAD('PCT_USED',10,' ');
DBMS_OUTPUT.PUT_LINE(
    RPAD('-----',20,' ')||
    LPAD('-----',10,' ')||
    LPAD('-----',10,' ')||
    LPAD('-----',10,' ')||
    LPAD('-----',10,' ')||
    LPAD('-----',10,' ');

FOR SEARCH IN FIND_INFO
LOOP
    SELECT DISTINCT BYTES/BLOCKS
    INTO VAR_BYTES
    FROM SYS.DBA_SEGMENTS;

    DBMS_OUTPUT.PUT_LINE(
        RPAD(SEARCH.TSPACE,20,' ')||
        LPAD(SEARCH.FILE_ID,10,' ')||
        LPAD(SEARCH.TOT_MB,10,' ')||
        LPAD(SEARCH.DBA_BYTES/VAR_BYTES,10,' ')||
        LPAD(SEARCH.TOT_USED,10,' ')||
        LPAD(SEARCH.PCT_USED*VAR_BYTES,10,' ');
END LOOP;
END;
/

```

Mullins no. : 10372

It's like a map of your Oracle files. Moreover, it also makes available other information (size and physical I/Os, if applicable) related to the files.

THE SCRIPT

- 645 -

UNION

```
select substr(lgt.number,1, instr(lgt.number, '/', -1)-1) path,
       substr(lgt.number, instr(lgt.number, '/', -1)+1 ) fname,
       lg.hytes/1048576 fsize,
       0 pr,
       0 pu
  from v$logfiles lgt, v$log lg
 where lgt.group# = lg.group#
 order by 1,2
/
spool off
exit
```

-----Sample Output-----

Path	File Name	N Hytes Phys.	Blocks Phys.	Blocks
/u01/oradata/P0007	control3.ctl			
	redo11.dbf	50	884	2781
	redo11.log	10		
	redo21.log	10		
	redo31.log	10		
	temp11.dbf	30		
/u02/oradata/P0007	control1.ctl			
	redo11.dbf	40	8	2813
	redo11.log	10		
	redo21.log	10		
	redo31.log	10		

Path	File Name	N	Bytes	Phys.	Bytes	Phys.	Wrt	Time
/u03/condata/P0007	control2.cti							
	geomf1.dbf	50		1551		388		
	geomf1.dbf	100		4403		2335		
	geomf1.dbf	50		5805		1118		
	toolsf1.dbf	15		2878				

18 rows selected.

SQL TIPS(20): HIERARCHICAL DATA의 SORT와 ORDERING

Bulletin no : 10973

DBCLER에서의 HIERARCHICAL QUERY에서는 SORT를 하기 위한 ORDER BY 절을 사용할 수 없었다. 그러나 DBCLER 7.10이상 UEBBIDM에서는 USER-DEFINED STORED PROCEDURE를 이용하여 HIERARCHY순서로 출력되면서 ORDERING할 수 있게 되었다.

세계의 지역에 관한 자료를 예로 보자.

```
CREATE TABLE UNIVERSE(
  PARENT VARCHAR2(30) REFERENCES UNIVERSE,
  NAME  VARCHAR2(30) PRIMARY KEY
);

-- SOME TEST DATA
INSERT INTO UNIVERSE VALUES ( NULL, 'WORLD' );
INSERT INTO UNIVERSE VALUES ( 'WORLD', 'EUROPE' );
INSERT INTO UNIVERSE VALUES ( 'EUROPE', 'ENGLAND' );
INSERT INTO UNIVERSE VALUES ( 'EUROPE', 'THE NETHERLANDS' );
INSERT INTO UNIVERSE VALUES ( 'EUROPE', 'GERMANY' );
INSERT INTO UNIVERSE VALUES ( 'WORLD', 'ASIA' );
INSERT INTO UNIVERSE VALUES ( 'ASIA', 'JAPAN' );
INSERT INTO UNIVERSE VALUES ( 'ASIA', 'CHINA' );
INSERT INTO UNIVERSE VALUES ( 'WORLD', 'AMERICA' );
INSERT INTO UNIVERSE VALUES ( 'AMERICA', 'UNITED STATES' );
INSERT INTO UNIVERSE VALUES ( 'AMERICA', 'MEXICO' );
INSERT INTO UNIVERSE VALUES ( 'WORLD', 'AFRICA' );
INSERT INTO UNIVERSE VALUES ( 'AFRICA', 'EGYPT' );
INSERT INTO UNIVERSE VALUES ( 'AFRICA', 'MOROCCO' );
```

위의 자료를 다음과 같이 보고자 하는 경우

```
WORLD
  AFRICA
    EGYPT
    MOROCCO
  AMERICA
    MEXICO
    UNITED STATES
  ASIA
    CHINA
    JAPAN
```

```

EUROPE
    ENGLAND
    GERMANY
    THE NETHERLANDS

```

만약, ORDER BY절이 없이 QUERY하면

```

SELECT SPAD( ' ', LEVEL * 5 ) || NAME FROM UNIVERSE
CONNECT BY PRIOR NAME = PARENT START WITH PARENT IS NULL;

```

다음과 같은 결과를 얻게 된다.

```

WORLD
  EUROPE
    ENGLAND
    GERMANY
    THE NETHERLANDS
  ASIA
    JAPAN
    CHINA
  AMERICA
    UNITED STATES
    MEXICO
  AFRICA
    EGYPT
    MOROCCO

```

만약, 위 문장에 ORDER BY절을 사용하면

```

SELECT SPAD( ' ', LEVEL * 5 ) || NAME FROM UNIVERSE
CONNECT BY PRIOR NAME = PARENT START WITH PARENT IS NULL
ORDER BY NAME;

```

다음과 같은 형식 많은 결과를 얻게 된다.

```

AFRICA
AMERICA
ASIA
    CHINA
    EGYPT
    ENGLAND
EUROPE
    GERMANY
    JAPAN
    MEXICO
    MOROCCO
    THE NETHERLANDS
    UNITED STATES
WORLD

```

7. 10]상 VERSION에서는 다음과 같이 USER DEFINED FUNCTION을 이용하여 원하는 자료를 얻을 수 있다.

```

CREATE OR REPLACE FUNCTION UNIVERSEBORTOMMER( PKEY UNIVERSE, NAME&TYPE )
RETURN VARCHAR2 IS
    PATH VARCHAR2(2000);
BEGIN
    PATH := PKEY;
    -- INSERT ALL PREVIOUS PARENT RECORDS LIKE A DIRECTORY STRUCTURE
    -- E.G. WORLD/EUROPE/...
    FOR CIRC IN ( SELECT PARENT FROM UNIVERSE
                  CONNECT BY PRIOR PARENT = NAME
                  START WITH NAME = PKEY ) LOOP
        PATH := CIRC.PARENT || '/' || PATH;
    END LOOP;
    RETURN PATH;
END;
/

SELECT SUBSTR( MPAD( ' ', LEVEL * 5) || NAME, 1, 40) "THE UNIVERSE"
FROM UNIVERSE
CONNECT BY PRIOR NAME = PARENT
START WITH PARENT IS NULL
ORDER BY UNIVERSEBORTOMMER( NAME )
;

```

SQL TIPS(21): DBA가 가지고 있어야 할 10개의 SQL

Bulletin no : 10374

=====

NO 1: Display the Current Archivelog Status

=====

The command below, when entered via SQL*DBA, will display the current archivelog status of the database. The script will display the active log_archive_dest location, as well as the current and oldest online redo log sequence numbers.

ARCHIVE LOG LIST:

=====

NO 2: Creating a Control File Trace File

=====

The command below, when entered via SQL*DBA, will create a trace file in your trace file destination directory. The trace file will contain the complete CREATE CONTROLFILE command for the current database, saving you vast quantities of time if you ever need to regenerate that command.

ALTER DATABASE BACKUP CONTROLFILE TO TRACE;

=====

NO 3: Tablespace Free Extents and Free Space

=====

The query below will ONLY work in Oracle7.2 and higher. In previous versions, it was difficult to query

DBA_DATA_FILES and DBA_FREE_SPACE in one query, since both could have multiple rows for each tablespace.

Using 7.2's FROM clause subquery feature, you can group the two queries separately, then join them on the

Tablespace_Name value. The result shows the largest free extent in the tablespace, the number of free extents in the

tablespace, the total free space in the tablespace, and the percentage of the tablespace's available space that is free.

column Tablespace_Name format A20

column Pct_Free format BBB.BB

```

select Tablespace_Name,
Max_Blocks,
Count_Blocks,
Sum_Free_Blocks,
100*Sum_Free_Blocks/Sum_Alloc_Blocks AS Pct_Free
from
(select Tablespace_Name, SUM(Blocks) Sum_Alloc_Blocks
from DBA_DATA_FILES
group by Tablespace_Name),
(select Tablespace_Name TB_NAME,
MAX(Blocks) AS Max_Blocks,
COUNT(Blocks) AS Count_Blocks,
SUM(Blocks) AS Sum_Free_Blocks
from DBA_FREE_SPACE
group by Tablespace_Name)
where Tablespace_Name = TB_NAME;

```

=====

No. 4: Display Allocated Space & Quota by User

=====

DBA_TB_QUOTAS displays the allocated space, by user, in each tablespace, along with the quota for each of those users. If a user has unlimited quota in a tablespace, the quota will be displayed as 0 or negative. Running this script is a great quick way to see which users own objects in which tablespaces, and how close they are to their quotas. You'll need to SET LINESIZE 132 before running this script.

```

select * from DBA_TB_QUOTAS
order by Tablespace_Name, Username;

```

=====

No. 5: Show Allocated Storage for All Objects

=====

DBA_SEGMENTS shows the allocated storage for all objects in the database. Valid values for the segment_type variable include:

```

TEMPORARY--for currently used temporary segments. Watch these grow!
ROLLBACK--for rollback segments
INDEX--for indexes
TABLE--for tables

```

There is one record in DBA_SEGMENTS for each segment. There is a one-to-many relationship between

DBA_SEGMENTS and DBA_EXTENTS: DBA_SEGMENTS shows you the total space allocated to a segment;

DBA_EXTENTS shows the space allocated to each extent within the segment.

column Segment_Name format A40

```
select Segment_Name, Extents, Blocks
from DBA_EXTENTS
where Segment_Type = 'Segment_type'
order by Segment_Name;
```

=====

No. 6: Map a Tablespace's Used and Free Space

=====

Since **DBA_EXTENTS** shows the starting block of each extent (the combination of **File_ID** and **Block_ID**), plus the length of each extent (the **Blocks** column), you can map out the allocated space within a given tablespace as a series of consecutive extents. **DBA_FREE_SPACE** shows the free extents, so the combination of **DBA_EXTENTS** and **DBA_FREE_SPACE** produces a map of the used and unused free space within a tablespace. You can use this map of the tablespace's space to determine whether an object has been properly sized or if the tablespace needs to be defragmented.

```
select Segment_Name, File_Id, Block_Id, Blocks
from DBA_EXTENTS
where Tablespace_Name = '<<tablespace>>'
UNION
select 'Free Space', File_Id, Block_Id, Blocks
from DBA_FREE_SPACE
where Tablespace_Name = '<<tablespace>>'
order by 2,3;
```

=====

No. 7: Blocks Used by Data in a Table

=====

When you replace "**<table>**" with the name of a table in your database, the query below will return the number of blocks used by the data in that table. The query excludes the **Block_ID** (columns 1-8) and the **File_ID** (columns 15-18) of the **ROWID**. The query then returns the number of distinct blocks used by the records within the table. The **ANALYZE** command can also provide this information, but this query is faster.

```
select COUNT(DISTINCT(SUBSTR(ROWID,1,8)||SUBSTR(ROWID,15,4)))
       Blocks_Used
```

from scott@hr:

=====

No. 8: Reset a User's Password

=====

The script below generates an ALTER USER command that will reset a user's password (the user whose username is

passed to the script as &1). If you spool the output of this command, you will be able to reset the user's password

without ever having to know what the password was. You can change the user's password to anything you like, use the

account for testing purposes, and then reset the password when your testing is complete. For a detailed discussion, see

Chapter 8 of the Oracle DBA Handbook.

```
select 'ALTER USER &1 IDENTIFIED BY VALUEB '
||''''||Password||''''||';'
  from DBA_USERS where Username = UPPER('&1');
```

=====

No. 9: Query U\$PARAMETER

=====

Don't try to guess what parameters are in effect. You can never tell for sure which INIT.ORA file was used to start an

instance, or if the INIT.ORA file has been modified since the database was started. The only sure way to tell the setting

of an initialization parameter is to query U\$PARAMETER.

```
column Name format A50
column Value format A25
```

```
select Name, Value from U$PARAMETER;
```

=====

No. 10: Show Statement Execution Path

=====

This query selects the execution path from the PLAN_TABLE, ordered according to the hierarchy of the steps

involved. The query shows in the listing specifically queries for the execution path of a statement using the

Statement_ID of 'TEST'. Prior to running this query, you should use the EXPLAIN PLAN command to populate

PLAN_TABLE.

```
select
  LPAD(' ',2*LEVEL)||Operation||' '||Options
```



```

            II' 'IID&ject_Name    Q_PLAN
from PLAN_TABLE
where Statement_ID = 'TEST'
connect by prior ID = Parent_ID and Statement_ID = 'TEST'
start with ID=1;

```

SQL TIPS(22): DETAILED LOOK AT DATABASE INDEX

Bulletin no : 10978

This script prompts the user for an index name and owner, and provides a detailed look at any index in the database by using the little known index_stats view.

```
-----set verify off
```

```
set pagesize 23
```

```
set linesize 80
```

```
set pause on
```

```
set pause 'Hit enter to continue'
```

```
set feedback off
```

```
set showmode off
```

```
set echo off
```

```
set space 0
```

```
set heading off
```

```
set timeout off pause off
```

```
column blk size value BLOCK_SIZE_F
```

```
select value / 1024 blk
```

```
from v$parameter
```

```
where name = 'db_block_size';
```

```
set timeout on pause on
```

```
PROMPT
```

```
ACCEPT SUBS_INPUT1 CHAR PROMPT 'Please enter an index to analyze: >'
```

```
ACCEPT SUBS_INPUT2 CHAR PROMPT 'Please enter the owner of the index: >'
```

```
PROMPT
```

```
PROMPT Working, Please wait.....
```

```
PROMPT
```

```
analyze index SUBS_INPUT2, SUBS_INPUT1 validate structure;
```

```
col BMB_PER_KEY          format BBB.BB    heading '          '
```

```
col BMB_BYTES_PER_ACCESS format BB,BBB.BB heading '          '
```

```

select
'%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%',
'
'           Name of the index = ' || NAME,
'
'           Height of the b-tree = ' || HEIGHT,
'
'           Blocks allocated to the index = ' || BLOCKS,
'
'           Number of leaf rows (values in the index) = ' || LF_ROWS,
'
'           Number of leaf blocks in the b-tree = ' || LF_BLK,
'
'           Sum of the lengths of all the leaf rows = ' || LF_ROWS_LEN,
'
'           Usable space in a leaf block = ' || LF_BLK_LEN,
'
'           Number of branch rows = ' || BR_ROWS,
'
'           Number of branch blocks in the b-tree = ' || BR_BLK,
'
'           Sum of lengths of all the branch blocks in the b-tree = ' || BR_ROWS_LEN,
'
'           Usable space in a branch block = ' || BR_BLK_LEN,
'
'           Number of deleted leaf rows in the index = ' || DEL_LF_ROWS,
'
'           Total length of all deleted rows in the index = ' || DEL_LF_ROWS_LEN,
'
'           Number of distinct keys in the index = ' || DISTINCT_KEYS,
'
'           How many times the most repeated key is repeated = ' ||
MOST_REPEATED_KEY,
'
'           Total space currently allocated in the b-tree = ' || BYTES_SPACE,
'
'           Total space that is currently being used in the b-tree = ' || USED_SPACE,
'
'           % of space allocated in the b-tree that is being used = ' || PCT_USED,
'
'           Average number of rows per distinct key = ', ROWS_PER_KEY,
'
'           Expected number of consistent reads block gets per row = ',
BLK_GETS_PER_ACCESS, ' '
from index_stats;

```

```

col NAME                format a30      fold_after
col dummy_col_0          fold_after
col dummy_col_1          fold_after
col dummy_col_2          fold_after

col LF_ROWS              format B,BB,BB0 heading '
col DEL_LF_ROWS          format B,BB,BB0 heading '
col BR_ROWS              format B,BB,BB0 heading '
col DISTINCT_KEYS        format B,BB,BB0 heading '
col MOST_REPEATED_KEY    format B,BB,BB0 heading '
col ROWS_PER_KEY          format B,BB,BB0 heading '
col HEIGHT               format 0         fold_after heading '
col BLK_GETS_PER_ACCESS  format BB,BB,BB fold_after heading '

col LF_BLK_LEN           format B,BB0     fold_after heading '
col lf_row_size           format BB0       fold_after heading '
col lf_row_per_blk        format BB0       fold_after heading '
col BR_BLK_LEN           format B,BB0     fold_after heading '
col br_row_size           format BB0       fold_after heading '
col br_row_per_blk        format BB0       fold_after heading '

col avg                  format BB,BB0    fold_after heading '

```

```

col lf_avg          format BBB.B0          heading ' '
col lf_avg_pct      format BB.B0          fold_after heading ' '
col hr_avg          format BBB.B0          heading ' '
col hr_avg_pct      format BB.B0          fold_after heading ' '
col uu_avg          format BBB.B0          heading ' '
col uu_avg_pct      format BB.B0          fold_after heading ' '
col kt_avg          format BBB.B0          heading ' '
col kt_avg_pct      format BB.B0          fold_after heading ' '
col ktu_avg         format BBB.B0          heading ' '
col ktu_avg_pct     format BB.B0          fold_after heading ' '
col ktuu_avg        format BBB.B0          heading ' '
col ktuu_avg_pct    format BB.B0          fold_after heading ' '
col ktuuk_avg       format BBB.B0          heading ' '
col ktuuk_avg_pct   format BB.B0          fold_after heading ' '

select
  '*****'
dummy_col_0,
  'Index Name.....', NAME, ' ' dummy_col_1,
  'Leaf Block Size.....', LF_BLOCK, ' ',
  'Leaf Block Size.....', LF_BLK_LEN,
  'Painted Leaf Block...', DBL_LF_BLOCK, ' ',
  'Leaf Row Size.....', LF_BLOCK_LEN / LF_BLOCK lf_row_size,
  'Branch Block Size.....', BR_BLOCK, ' ',
  'Leaf Row Per Block...', LF_BLK_LEN / (LF_BLOCK_LEN / LF_BLOCK) lf_row_per_blk,
  'Distinct Keys.....', DISTINCT_KEYS, ' ',
  'Branch Block Size.....', BR_BLK_LEN,
  'Max Common Key.....', MOST_REPEATED_KEY, ' ',
  'Branch Row Size.....', BR_BLOCK_LEN / (BR_BLOCK + 1) br_row_size,
  'Avg Common Key.....', BLOCK_PER_KEY, ' ',
  'Branch Row Per Block...', BR_BLK_LEN / ((BR_BLOCK_LEN / (BR_BLOCK + 1)) + 1)
br_row_per_blk,
  'Height Of B-Tree...', HEIGHT,
  'Nodes Per Access...', BLK_GETS_PER_ACCESS,
  ' ' dummy_col_2,
  'Index Avg.....', (BLOCKS * BLOCK_SIZE_K) / 1024 avg,
  'Leaf Avg/Pct.....', (LF_BLK * BLOCK_SIZE_K) / 1024 lf_avg, ' / ',
                                (LF_BLK / BLOCKS) * 100 lf_avg_pct,
  'Branch Avg/Pct.....', (BR_BLK * BLOCK_SIZE_K) / 1024 br_avg, ' / ',
                                (BR_BLK / BLOCKS) * 100 br_avg_pct,
  'Unused Avg/Pct.....', ((BLOCKS - (LF_BLK + BR_BLK)) * BLOCK_SIZE_K) / 1024 uu_avg,
  ' / ',
                                ((BLOCKS - (LF_BLK + BR_BLK)) / BLOCKS) * 100 uu_avg_pct,
  'B-Tree Avg/Pct.....', ((LF_BLK + BR_BLK) * BLOCK_SIZE_K) / 1024 kt_avg, ' / ',
                                (LF_BLK + BR_BLK) / BLOCKS) * 100 kt_avg_pct,
  'B-Tree Used Avg/Pct.....', (USED_SPACE / 1024) / 1024 ktu_avg, ' / ',

```

```

        PCT_USED ktu_avg_pct,
'B-Town Unknd Mag/Pct...', ((INTAKE_SPACE - USED_SPACE) / 1024) / 1024 ktu_avg, ' / ',
        100 - PCT_USED ktu_avg_pct,
'B-Town Unknd In Mag/Pct..', ((DEL_LF_ROWS_LEN / 1024) / 1024 ktu_k_avg, ' / ',
        (((DEL_LF_ROWS_LEN / 1024) / 1024) /
        (((LF_ROWS + RR_ROWS) * BRIDGE_SIZE_K) / 1024))
        * 100 ktu_k_avg_pct from Index_stats;

exit;
-----

```

SQL TIPS(23): EXPORTING TABLES BASED ON TABLE SPACE NAME

Bulletin no : 10377

This script allows the user to enter the tablespace name and creates a list of tables in any format. Additional choices allow the export of over-extended tables or tables over a certain size.

```
*****
run list.sql
run Give list of tables depending on parameters

set timeout on linesize 75 pagesize 0 verify off feedback off
clear buffer
clear column
prompt
prompt This script will create list.out, which can be used
prompt to give export tables parameter file
prompt

accept Segment_Name Prompt 'Enter Seg Name          : '
accept owner       Prompt 'Enter Seg Owner          : '
accept tbsname      prompt 'Enter TableSpace Name    : '
accept min_extents  Prompt 'Enter Minimum size of exts : '
accept min_size     Prompt 'Enter Minimum seg size(M)  : '
prompt
accept exportchoice Prompt 'Is this list for exports ? : '

col ext                exportlist
BPOOL list.out

-- start list
select 0 ext, decode(upper('!exportchoice'),'Y','tablename=','')
from    dual
union
-- data list without and segment
select 1 ext, segment_name||decode(upper('!exportchoice'),'Y',' ','')
from    dba_segments
where   segment_type = 'TABLE'
and     segment_name like upper('!%Segment_Name%')
```

```

and owner like upper('BENAMER')
and tablespace_name like upper('BENAMER')
and extent >= (delta_extent + 0)
and bytes >= (delta_size + 0) * (1024*1024)
union
-- remove last one (with ',' after the name)
select 1 ext, max(segment_name) || decode(upper('exception'), 'Y', ', ', '')
from dba_segments
where segment_type = 'TABLE'
and segment_name like upper('BENAMER')
and owner like upper('BENAMER')
and tablespace_name like upper('BENAMER')
and extent >= (delta_extent + 0)
and bytes >= (delta_size + 0) * (1024*1024)
union
-- data bit with end segment
select 3 ext, max(segment_name) || decode(upper('exception'), 'Y', ', ', '')
from dba_segments
where segment_type = 'TABLE'
and segment_name like upper('BENAMER')
and owner like upper('BENAMER')
and tablespace_name like upper('BENAMER')
and extent >= (delta_extent + 0)
and bytes >= (delta_size + 0) * (1024*1024)
order by 1, 2
/

spool off

set heading on
*****

```

SQL TIPS(24): CALCULATING HIT RATIO

Bulletin no : 10378

Calculating hit ratios is sometimes a drudgery. This little script helps and requires no parameters. The script uses

PL/SQL.

```
-----  
rem  
rem  is hit ratio  
rem  
col global_name for A20  
variable d number  
variable c number  
variable r number  
begin  
  select value into ic from v$sgstat  
  where name = 'consistent gets';  
  
  select value into id from v$sgstat  
  where name = 'db block gets';  
  select (1- value/( ic + id)) into ir from v$sgstat where name in  
    ( 'physical reads');  
  
end;  
/  
select global_name, ir "Hit Ratio by CHN" from global_name ;  
-----
```


SQL TIPS(25): ANALYZING TABLESPACE USAGE WITH AN OS-INDEPENDENT SQL SCRIPT

Bulletin no : 10379

When tables, indexes, and rollback segments are created, they are pre-allocated a storage allocation (extent) that is reserved and cannot be used by any other object. Although these objects may not use all of the space allocated at creation, the amount of available free space diminishes over time as more information is placed into the database. The following query helps you instantly find application problems resulting from space shortage. Run this query at regular intervals to obtain the latest information. Note that this script assumes that you have Oracle data blocks of any size `n*E`, where 'n' is an positive integer and 'E' equals 1,024 bytes.

The query uses a PL/SQL variable called `VAR_BYTES` to eliminate the operating system-dependent features of the size of the Oracle data blocks. You will not have to modify the query; this SQL script is operating system-independent.

REM

REM Drop TABLESPACETEMP before creation.

REM

DROP TABLE TABLESPACETEMP;

REM

REM Create TABLESPACETEMP.

REM

CREATE TABLE TABLESPACETEMP

TSPACE	CHAR(22),
FILE_ID	NUMBER(5),
TOT_MB	NUMBER(10,2),
DATA_BYTES	NUMBER(10,2),
TOT_USED	NUMBER(10,2),
PCT_USED	NUMBER(10,2);

COMMIT;

DECLARE

UAB_BYTES NUMBER(8);

CURSOR FIND_INFO IS

```
SELECT SUBSTR(D.TABLESPACE_NAME,1,22)      TSPACE,
       LTTRIM(TO_CHAR(D.FILE_ID,'9999'))      FILE_ID,

       LTTRIM(TO_CHAR(D.BYTES/1024/1024,'9999999.99'))

       TOT_MB,
       LTTRIM(TO_CHAR(D.BYTES,'9999999.99'))  DBA_MB,
       LTTRIM(TO_CHAR(ROUND(SUM(E.BLOCKS),'999999.99')) TOT_USED,
       LTTRIM(TO_CHAR(ROUND(SUM(E.BLOCKS)
       /((D.BYTES)/4)*100,'999999.99'))      PCT_USED
FROM      SYS.DBA_DATA_FILES D,
          SYS.DBA_EXTENTS E
WHERE     D.FILE_ID = E.FILE_ID (+)
GROUP BY  D.TABLESPACE_NAME,
          D.FILE_ID,
          D.BYTES/1024/1024,
          D.BYTES;
```

BEGIN

FOR SEARCH IN FIND_INFO

LOOP

```
SELECT DISTINCT BYTES/BLOCKS
      INTO      UAB_BYTES
FROM      SYS.DBA_SEGMENTS;
```

INSERT INTO TABLESPACE_TEMP VALUES

```
(SEARCH.TSPACE,
      SEARCH.FILE_ID,
      SEARCH.TOT_MB,
      SEARCH.DBA_MB/UAB_BYTES,
      SEARCH.TOT_USED,
      SEARCH.PCT_USED*(UAB_BYTES));
```

END LOOP;

END;

/

SQL

SQL Now, you can select information from TABLESPACE_TEMP without
SQL modifying anything.

SQL

```
SELECT * FROM TABLESPACETEMP;
```

```
REM
```

```
REM You may drop TABLESPACETEMP!
```

```
REM
```

```
DROP TABLE TABLESPACETEMP;
```

SQL TIPS(26): CREATING A DATABASE LAYOUT

Bulletin no : 10388

The script below is one I used quite frequently when I was with Oracle Consulting. It is handy for walking into a company and getting a quick layout of their database. I call it dump.sql.

```
spool dump.txt;
prompt
prompt =====
prompt   Tablespace/Datafile Listing
prompt =====
prompt
prompt
column "Location"          format A30;
column "Tablespace Name"    format A15;
column "Size(M)"           format BBB,BB0;
```

```
break on "Tablespace Name" skip 1 nodup;
compute sum of "Size(M)" on "Tablespace Name";
```

```
SELECT tablespace_name "Tablespace Name",
       file_name "Location",
       bytes/1048576 "Size(M)"
FROM sys.dba_data_files
order by tablespace_name;
```

```
prompt
prompt =====
prompt   Redo Log Listing
prompt =====
prompt
prompt
column "Group"             format BBB;
column "File Location"      format A40;
column "Bytes (K)"         format BB,BB0;
```

```
break on "Group" skip 1 nodup;
```

```
select a.group# "Group",
       b.member "File Location",
```

```

        a.bytes/1024 "Bytes (K)"
from   v$log a,
        v$logfil# b
where a.group# = b.group#
order by 1,2;

prompt
prompt =====
prompt Rollback Listing
prompt =====
prompt
prompt
column "Segment Name" format A15;
column "Tablespace" format A15;
column "Initial (K)" format BB,BB0;
column "Next (K)" format BB,BB0;
column "Min Ext.," format BB0;
column "Max Ext.," format BB0;
column "Status" format A7;

select segment_name "Segment Name",
       tablespace_name "Tablespace",
       initial_extent/1024 "Initial (K)",
       next_extent/1024 "Next (K)",
       min_extents "Min Ext.,"
       max_extents "Max Ext.,"
       status "Status"
from   sys.dba_rollback_seg
order by tablespace_name,
       segment_name;

spool off;

```

SQL TIPS(27): DECODE 이 용 사 례 (2)

Bulletin no : 10388

다음과 같은 자료를 갖는 TABLE을 가정해보자.

Employee Name	Employee Address	City	State	ZIP
Donald Buck	101 Cool Pond	Bedford	DB	12345
Bugs Bunny	540 Carrot Way	Atlanta	GA	28282
Elmer Fudd				
Tax Devil	888 Wildwood	Nashville	TN	88888

REPORT를 생성할 때는 종종 CITY와 STATE를 CONCAT()로 연결하여 출력하곤한다.
이때 다음과 같이 COLUMN간에 CONCATENATING을 이용하면 가능하다.

```
select employee_name,
       employee_address,
       city || ', ' || state || ' ' || zip city_state_zip
from decode_example;
```

위의 문장은 다음과 같은 결과를 준다.

EMPLOYEE_NAME	EMPLOYEE_ADDRESS	CITY_STATE_ZIP
Donald Buck	101 Cool Pond	Bedford, DB 12345
Bugs Bunny	540 Carrot Way	Atlanta, GA 28282
Elmer Fudd		,
Tax Devil	888 Wildwood	Nashville, TN 88888

위와 같은 경우에서 주소의 자료가 없는 경우 필요없는 COMMA만 출력되게 된다.
이런경우에 DECODE 를 이용하여 다음과 같이 처리하면 깔끔한 OUTPUT을 얻을 수 있게 된다.

```
select employee_name,
       employee_address,
       city || decode(' ' || state, ' ', ',', ' ' || state) || ' '
       || zip city_state_zip
from decode_example;
```

EMPLOYEE_NAME	EMPLOYEE_ADDRESS	CITY_STATE_ZIP
Donald Buck	101 Cool Pond	Bedford, DB 12345
Bugs Bunny	540 Carrot Way	Atlanta, GA 28282
Elmer Fudd		
Tax Devil	888 Wildwood	Nashville, TN 88888

SQL TIPS(28): DECODE 이 용 사 례 (3)

Bulletin no : 10387

예 : salary가 1100보다 큰 경우면 (+)를
 같은 경우면 (=)를
 작은 경우면 (-)를 표시하려면?

```
select DECODE(sign(sal-1100),0,'(=)',1,'(+)', '(-)'),BAL from emp;
```

DEC	BAL
(-)	800
(+)	1800
(+)	1250
(+)	2875
(+)	1250
(+)	2850
(+)	2450
(+)	3000
(+)	5000
(+)	1500
(=)	1100
(-)	850
(+)	3000
(+)	1300

(V1.1) REPORTWRITER 에서 LOCAL PRINTER 로 출력하는 방법

Bulletin no : 10001

ReportWriter에서 단말기에 연결된 printer로 출력하는 방법

form30에서 host문을 이용하여 ReportWriter를 호출할때 printer bypass on/off에 해당하는 message sequence를 parameter로 전송한다.

ReportWriter에서 file로 출력한 후 cat를 이용하여 printer로 전송한다.

```
on : ^[[5] : host단말기 기준
off : ^[[4]
```

주의: simulator를 사용할 경우 해당 simulator에서 나타내는 bypass on/off 의 값을 확인하여야 한다.

```
ex> ReportWriter : . mmu bar -> parameter 에서 P_ON, P_OFF parameter를 add한다. (type isbar)
. mmu bar -> parameter 에서 destination을 file로 지정한다.
screen으로 지정할 경우, 출력시 holding되는 현상이 생기는 경우
발생하기 때문
. mmu bar -> text에서 page header, trailer에 각각 BP_ON, BP_OFF를 등록한다.
form30 : trigger>catting
```

```
begin
select chr(27)||chr(81)||chr(53)||chr(105),
chr(27)||chr(81)||chr(52)||chr(105)
into global.P_ON, global.P_OFF from dual;
host('runwp k1 userId=ccitt/tlgw timestamp=100
paramform=on P_ON='||global.P_ON||'
P_OFF='||global.P_OFF||' ',no_screen);
host('cat k1,lik',no_screen);
endisplay;
end;
```

(V1.1) 출력물을 현 단말기에 연결된 PRINTER로 출력하는 방법

Bulletin no : 10114

* Form30에서 Host를 이용하여 ReportPrinter를 호출할 때 Printer Escape on/off에 해당하는 Escape Sequence를 Parameter로 전송한다.

* ReportPrinter에서 File로 출력한 후 Cnt를 이용하여 Printer로 전송한다.

on : [[51 : fact단말기 기준
off : [[41

[주의] Emulator를 사용할 경우 해당 Emulator에서 나타내는 Escape on/off값을 확인하여야 한다.

예) * ReportPrinter

- Menu Bar : Parameter에서 P_ON, P_OFF parameter를 Add한다.(type :char)
- Menu Bar : Parameter에서 Destination을 File로 지정한다. Screen으로 지정할 경우, 출력시 Holding되는 현상이 생기는 경우가 많았기 때문.
- Menu Bar Text에서 Page Header, Tailor에 각각 P_ON, P_OFF를 등록.

* Form30 - Trigger 에 Setting

begin

```
select chr(27)||chr(81)||chr(53)||chr(105),  
chr(27)||chr(81)||chr(52)||chr(105)  
into global, P_ON, global.P_OFF from dual;  
host ('runsp k1 userid=ccott/tiger term=vi100 parameter=on P_ON=  
'||global.P_ON||' P_OFF='||global.P_OFF||':cctk1,lix',no_echo);  
endisplay;
```

end;

④ PAGE HEIGHT가 66보다 작은 SPECIAL FORM지에 출력하는 방법

Bulletin no : 10115

* Height를 33으로 가정할 때 : 예를 들어 연속지로용지 출력시

1. 우선 Report Source에서 Page Height를 33으로 Battling한다.

Menu Bar * Report * Height조정

2. \$BOL\$HOME/\$qlreport/\$adslm/printer directory에서 Page Height = 33인 Printer File를 생성한다. (Printerist Utility 이용)

* BQL\$Report에서 Report의 Height를 조정하고 Printer File의 Page Height를 조정하여도 실제로 출력해 보면 BBLines를 기준으로 Form Feed가 밀어나게 된다.

* 이것은 BQL\$Report에서는 33Line에 한련씩 Form Feed를 보내주지만 실제 Printer(H/W)에서는 한 Page의 Length를 88으로 인식하고 있기 때문이다.

3. 위의 현상을 방지하기 위해서는 Printer에서의 Page Length Battling을 33 Line으로 Battling하여야 한다. 만약 Printer에서 Page Battling하는 기능이 없다면 다음의 방법을 참조한다.

* 실제로, BQL\$Report에서 출력을 File로 받아보면 해당 Page의 끝에(Control L :form Feed)이 삽입되어 있다. 이것은 BQL\$Report Tool에서 붙여주는 Character 코더 User가 밀리대로 처리할 수가 없다.

* User가 위에서의 질문처럼 밀리대로 Page Length를 조정하려면 3가지 정도의 방법이 있다.

[방법1] 생성된 File에서 Form Feed Character(^\)의 기능을 없애는 방법.

- 출력하고자 하는PrinterDescription(\$BOL\$HOME/\$qlreport/\$adslm/printer 에 위치)에서 ^^값을 다른 것으로 교체합니다. 이것은 BQL\$Report에서 받아오는 ^^값이 기능을 상실하므로 빈칸이 넘어가는 결과를 갖게 됩니다.

[방법2] Unix Command에서 Form Feed Character의 기능을 없애는 방법.

- BQL\$Report에서는 정상적으로 출력하고, 내부에서 실행되는 Bpscmd (BQL\$Report 에서 Printer 출력시 Unix System에서 실제로 출력하는 부분임.) 내부에서 !p나 !pr command를 사용하기 전에 Unix의 Sed Command나 C Program을 이용하여 ^^(\)값이 들어간 Line를 삭제하거나 ^^값을 삭제합니다.

[방법3] Report에서 매 Page에서 Printer의 Page Length를 33으로 Battling.

- Page마다 Page Length를 33으로 Battling on/off esc key 값을 Battling.

Report의 맨 밑에 맨 위에 Page Length Battling on/off 값을 Battling.

(BQL\$Report에서 Esc Key 값 Battling하는 참고자료 참조 할 것.)

(V2.5) ASCII DRIVER로 WIDE FORMAT 출력

Bulletin no : 10128

1. Oracle Reports ASCII driver를 install한다.

- (1) `orautil\reports25\platform\ascii\drv.txt`를 참조.
- (2) `orautil\reports25\platform` 하에 사용하실 수 있는 `printer destination format file` 들이 존재한다.
 - default : `df1t.prt` 사용, `B0column`용
 - `B0 column` 이상을 출력할 경우 : `wide.prt`, `widw180.prt`를 사용 (general)
 - 일반적인 PC Editor를 사용하여 필요에 따라 새로운 `.prt file`을 생성하여도 된다.

2. Reports2.5 Runtime Module을 사용 할 경우

- (1) Mode = Character
- (2) Defformat = Wide or Widw180

3. Reports2.5 Designer Module를 사용 할 경우

- (1) Tools * Properties * Character Mode : Page Width 를 조정
- (2) Tools * Tools Option * Runtime Parameters : Mode = Character로 변경
- (3) File * Print Setup * Oracle Reports ASCII Driver 선택: 출력방향이냐 용지선택은 할 필요가 없다.
- (4) Default Parameter Form * Defformat 을 설정

(V1.1) SQL*REPORTWRITER 1.1 에서 프로그램의 BACK-UP 및 RESTORE 방법

Bulletin no : 10142

SQL*ReportWriter에서는 사용자가 DBSQL MODULE인 BQLBREP를 통하여 프로그램을 작성한 후 컴파일 시 또는, CDBREP를 수행할 경우에도 실행 모드인 *.REP 파일(예, TEST1.REP)만을 생성하고 BQBCRE 파일을 만들지 않는다.

따라서, 작성한 SQL*ReportWriter 프로그램의 BQBCRE는 BUNPREP를 이용하여 생성하여야 한다. BUNPREP는 데이터베이스로부터 SQL*ReportWriter의 REPORT DEFINITION을 ASCII 파일(*.REP 확장자를 가짐)인 작업 파일을 만든다.

이러한 ASCII 파일인 BACK-UP 파일을 DBACLE 데이터베이스로 로드하기 위해서는 LOABREP를 이용한다.

1) BUNPREP 사용 방법

```
$ dumprep report-report_name userid=scott/tiger file=backup_file
report : SQL*ReportWriter 프로그램의 이름(*.rep 파일의 이름)
userid : DBACLE userid/password
file   : 모드 파일의 이름
```

2) LOABREP 사용 방법

SQL*ReportWriter 프로그램의 작업 파일을 DBACLE 데이터베이스에 로드시키는 명령이다.

```
$ loadrep file=backup_file userid=scott/tiger
file      : 로드할 작업 파일(*.REP)의 이름
userid    : DBACLE userid/password
```

Bulletin no. : 10143

(표-2 TERMINDEF.BAT 파일의 예)

다음은 실제 FUNCTION KEY 수정방법을 살펴보기로 한다.

- BBU_100.EOF 파일에서 [ACCEPT] FUNCTION KEY를 [F3] KEY로 수정하는 방법을 살펴보자.

(1) \$DBACLE_HOME/oxlrapport/adm/terminal 디렉토리 아래 BBU_100.EOF 파일을 에디터로 오픈한다.
- 예를 들어 VT220 파일을 수정시는 BBU_220.EOF 파일을 이용한다.

(2) 표-1의 BBU_100.EOF 파일에서 [ACCEPT] FUNCTION에 대한 ASCII CODE와 KEY ASSIGNMENT를 변경한다.

- [ACCEPT]의 KEY ASSIGNMENT를 "F3"으로 변경
- [ACCEPT]의 ASCII CODE를 실제로 [F3] KEY에서 발생하는 CHARACTER SEQUENCE인 "^OD"로 변경
- 수정할 FUNCTION KEY를 위의 방법과 동일하게 KEY ASSIGNMENT와 ASCII CODE를 변경

(3) BBU_100.EOF의 KEY DEFINITION 파일을 수정한 다음 TERMINDEF 통일타리를 사용하여 새로운 TERM 타일을 생성한다.

```
$ terminalf terminal=vt100 outfile=bbu_100 infil=bbu_100.kdf
userld=syctim/manager tdfil=terminalf.dat
```

- terminal : TERMINDEF.BAT 파일에서 지정된 TERMINAL NAME으로 두 번째 인수가 해당된다.
- outfile : 생성될 새로운 TERM 타일
- infil : 수정한 KEY DEFINITION 파일 (예, BBU_100.EOF 파일)
- userld : DBACLE USER ID
- tdfil : TERMINAL DEFINITION 파일 (예, TERMINDEF.BAT 파일)

(4) TERMINDEF를 사용하여 생성된 TERM 타일을 BQLDEF, BUNDEF 시에 지정한다

```
$ oxlrap term=bbu_100 userld=ccott/tlgm
```

(M.1) SQL*REPORTWRITER 1.1 에서 PRINTER DRIVER DEFINITION 수정 방법

Bulletin no : 10144

SQL*reportwriter의 BDLDEF, BDLDEF에서 프로그램 출력시 제어되는 PRINTER DRIVER DEFINITION을 수정하는 방법을 살펴보자. 새로운 파일의 DRIVER를 생성하기 위해서는 PRINTER DEFINITION FILE과 이 파일을 참조하는 PRINTER 명령을 수정하여야 한다.

PRINTER DEFINITION 파일에는 BDL에서 프로그램들 프린터로 출력시 사용되는 한 LINE의 컬럼 수, CHARACTER SETTING PRINTER, PRINTER 초기화 값 등의 제어 코드를 포함하고 있다. 예를 들어, PRINTER DEFINITION인 printdef.dat(\$ORACLE_HOME/sqlreport/adm/prnter 디렉토리에 존재)은 다음과 같이 구성되어 있다.

```
-----
B DEC -- DEC printer in portrait mode
BDECDEC printerc:\                <----- (A)

      :ff=^L:co=^M:nl=^J:lc=^L:co=^E(l:co=^E(0m:1188:co880:\ <-- (B)
      :sp:co=^E(d:co=^E(0m:

B DEC180 -- DEC printer, landscape wide mode (/FORM-LANDSCAPE)
B1188CO180DEC printerc:\
      :ff=^L:co=^M:nl=^J:co=^E(l:co=^E(0m:1188:co8180:\
      :sp:co=^E(d:co=^E(0m:
      :lc=^E(lp^E(721 J^E(15m:\
      :fc=^E(lp:

B DPLY -- Printer with no special codes -- also used for files
B1188CO180DEC logical printerc:\
      :sp:1188:co880:

B
-----
```

(표 - 1 PRINTERDEF.BAT 파일의 예)

표-1에서 (A)는 PRINTER DEFINITION 파일의 LABEL 규칙이며, (B)는 FORMAT CODE의 규칙을 나타낸다.

(A) LABEL SYNTAX

PRINTER ID | PRINTER NAME | COMMENT :\

- **PRINTED ID** : 내부적으로 사용되는 임의적인 이름,
2 문자로 구성, **PRINTED DEFINITION** 파일내에서는 유일
- PRINTED NAME** : **PRINTED** 내의 **DRIVER** 속성을 지정
PRINTED DEFINITION 파일내의 유일한 **PRINTED** 이름
- COMMENT** : **DRIVER** 기능 설명

(표 - 2 **PRINTED DEFINITION** 파일의 LABEL)

(B) **FORMAT CODE SYNTAX**

FORMAT CODE [-STRING | NUMBER]

- **FORMAT CODE** : **PRINTED FORMAT CODE** 또는 **DEFAULT** 형태로 제공되거나
사용자가 정의할 수 있음.
- STRING** : **PRINTED** 제어 코드에 해당하는 **CONTROL CHARACTER** 또는
ESCAPE SEQUENCE
- NUMBER** : 정의된 **PRINTED FORMAT CODE**의 **NUMBER** 라인의 값
PRINTED 기호를 지정

(표 - 3 **PRINTED DEFINITION** 파일의 **FORMAT CODE**)

표-1의 **PRINTED DEFINITION** 파일은 **REL-4reportutils-1.1**에서 제공하는 **DEFAULT DRIVER(DFTL, DEC, DECIBO)**이며, 각 **DRIVER DEFINITION**은 아래 형태의 **PRINTED FORMAT CODE**들로 구성되어 있다.

NAME	TYPE	DESCRIPTION
an	NUMBER	여백을 조정
ap	NUMBER	LINE FEED
cn	NUMBER	한 라인의 컬럼 수
cp	STRING	CARRIAGE RETURN(DEFAULT='M')
fp	STRING	PRINTED PAGE EJECT (DEFAULT='L')
fc	STRING	PRINTED TERMINATION STRING
ic	STRING	PRINTED INITIALIZATION STRING
il	NUMBER	한 페이지의 라인 수
ml	STRING	NEWLINE STRING
mm	STRING	사용자가 정의하는 PRINTED 제어 코드. 'mm'은 숫자로 명시.
oc	STRING	OVER STRIKE
oh	STRING	HIGHLIGHT MODE의 종료
ohs	STRING	HIGHLIGHT MODE의 시작
uh	STRING	UNDERSCORE MODE의 종료
uhs	STRING	UNDERSCORE MODE의 시작

(표 - 4 **PRINTED FORMAT CODE**의 예)

PRINTER DEFINITION 및 PRINTER FORMAT CODE를 수정하거나 생성할 경우에는 다음의 규칙에 따라야 한다.

- COLUMN(:)은 CODE LIST의 시작.
- COLUMN은 CODE LIST의 종료.
- COLUMN은 각 CODE DEFINITION의 분리지.
- FORMAT CODE의 라인 끝에 BLANK(/) 기호를 사용하여 다음 라인에 연속적으로 FORMAT CODE를 지정.

다음은 실제 PRINTER DRIVER DEFINITION을 수정하는 방법을 살펴보기로 한다.

- "BFLY" PRINTER DRIVER를 이용하여 한 라인의 컬럼을 132 컬럼으로 출력할 수 있도록 변경하고자 한다.
- 즉, `iccol80`(한 라인의 컬럼 수 = 80컬럼)를 132 컬럼으로 변경하는 방법

- (1) `$ORACLE_HOME/sqlmpart/adm/prnter` 디렉토리 아래의 `PRINTERF.DAT` 파일을 에디터로 오픈한다.
- (2) PRINTER FORMAT CODE 작성 규칙에 따라 PRINTER DEFINITION 파일 (예, 1의 `PRINTERF.DAT`)의 "BFLY" PRINTER DRIVER에서 "한 라인 컬럼 수"를 나타내는 `iccol80`를 `iccol132`로 변경한다.
- (3) 수정된 `PRINTERF.DAT` 파일을 저장한다.
- (4) `BQLmpartutil`의 `PRINTER` 유틸리티를 사용하여 새로운 PRINTER DRIVER DEFINITION을 생성한다.
`PRINTER` 유틸리티의 사용 방법은 다음과 같다.
 - `$ printdef printer=BFLY outfile=NEW_BFLY pfile=printdef.dat`
 - `printer` : PRINTER의 이름.
 PRINTER DEFINITION 파일의 LABEL에 명시된 주전체
 변수를 참조
 - `outfile` : 새로운 PRINTER DRIVER DEFINITION 이름
 - `pfile` : PRINTER DESCRIPTION FILE (예, `PRINTERF.DAT`)
- (5) `BQLmpartutil` 프로그램 출력시 [PARAMETER VALUE FORM]의 [PRINTER DESCRIPTION] 옵션에서 수정된 `NEW_BFLY`를 지정한다.

(V1.1) 고정된 위치에 합계를 PRINT 하는 방법

Bulletin no : 10179

합계를 프린트하는 행의 첫부분에 인쇄되는 데이터의 행이 가변적일 때 고정된 (ex. 세금액산서양식) 위치에 합계를 프린트하기 위해서는

1> 주계의 `summary` 설정

- 개별 data 인쇄 `summary`
- 합계 data 인쇄 `summary`

2> `text options`에서 합계데이터를 인쇄하는 `group`의 `header`부분에서

- `relative position : margin`
- `lines before : 20`
- `space before : 10`

3> 항상 합계는 (20,10)에서 인쇄된다.

(V1.1) PRINTER로 찍을때 확대 축소하려면?

Bulletin no : 10181

1. PRINTER MANUAL에서 확대값을 찾는다.

KK) 행정전산망용 : ^[y1 ^[y0

2. cd \$ODDACLE_HOME/ox/transport/xdm/printer

3. vi printdef.dat : printer device에 값을 입력한다.

HP1HPLJ11P LaserJet11

:zf=^L:ce=^H:nl=^J:cs880:1883:\

:005=^[y1:008=^[y0:

4. printdef hp11k.mshp

5. oxlsp scott/tiger

Text Screen : 85 확대할 문자 88

Runform parameters:

Printer Description File : mshp

B. 결과를 확인한다.

제대로 만 나오면 먼저 output을 zll로 보낸 후 그 zll내에

^[y1,^[y0 으로 되어 있는지 확인한다.

(V2.5) ORACLE REPORT SERVER에 관하여

Bulletin no : 10251

Report Server에 관하여

정의와 목적

Oracle Reports Server는 Oracle Forms나 Oracle Graphics로 부터 호출되어 클라이언트에 발려있는 Report 들을 쉽고도 강력하게 관리하여 준다. 이것은 발려있는 실행될 Report들을 나열해주고, Report들의 상태를 볼수 있게하며, 클라이언트에서 Report를 뒤,아래로 움직일 수도 있게하며,클라이언트에서 삭제 할 수도 있다.

Oracle Report Server가 초기화될때, DLL들은 먼저 Load되어 있다. 이것은 Oracle Report,Oracle Form,Oracle Graphics의 Start시간을 줄여주는 효과를 준다. Executable이 이미 Memory에 Load되어 있고 Report Server는 이미 Database에 Connect되어 있기때문에 Report의 실행시간 역시 빠르게 해 준다. 처음 Report Server가 호출되면 이것은 특별한 Userid로 Database에 Connect하게된다. 다른 Client에서 다음 Report Server가 호출되면 자동적으로 다른 Login String으로 Connect하게 된다.

Server는 다음과 같은 경우에 자동적으로 기동된다.

- . Oracle Forms나 Oracle Graphics에서 RUN_PRODUCT를 이용하여 Report를 호출할 때
- . 다른 Report에서 RUN_PRODUCT를 이용하여 Report를 호출 할 때

RUN_PRODUCT를 사용하면, Report Server Icon이 Window화면상에 위치하게된다. Reports Server는 User가 임의로 Close하기전까지 Open된 상태로 남아있게 된다.

Reports Server Action Menu

1. Cancelling/Removing Reports

Reports Server 클라이언트에 발려있는 Print Job를 삭제하려면

방법1 : Highlight/Select된 Report가 Cancel된다.

방법2 : ACTION Menu에서 CANCEL JOB나 REMOVE JOB를 선택한다.

CANCEL JOB는 Report List에서 현재 실행중인 Report를 종료시킨다. 이 Menu는 현재 실행중인 Report가 선택되었을 때만 Displayed/Enabled된다.

REMOVE JOB는 Report List에서 현재 선택된 Report를 삭제한다. 이 Menu는 현재 실행중인 Report가 아닌것을 선택하였을때만 Displayed/Enabled된다.

2. SHOW/HIDE LOG

이 Menu는 각 Report의 Logging을 별도의 다른 Page으로 나타내주거나 열에 주거나 한다.

Report Server를 종료시키면 이 Log의 내용들은 현재작업 Directory내에 Reportx.Log로 저장된다.

3. Exiting The Reports Server

ACTION Menu에서 QUIT를 선택하면 현재작업 중인 Report를 포함한 모든 Report들이 종료되고, Server Log들은 Reportx.Log에 저장된다.

4. Report List

Report Server내의 Report List는 현재 실행중인 Report와 큐에 쌓인 Report들에 대한 우선순위와 상태를 나타내준다.

NAME는 큐내에서의 현재의 우선순위를 의미한다. NAME 1인 것은 현재 실행 중인 Report이고, NAME 2는 다음에 실행될 것이다. Report를 선택하여 뭐야대로 담겨 새로운 위치에 가져다 놓음으로써 Report의 우선순위를 바꿀 수도 있다. 단 이것은 큐에 쌓여있는 Report에 한해서만 가능하며 현재 실행중인 Report에 대해서는 할 수가 없다.

STATUS는 현재 실행중인 Report의 상태를 나타내준다.

- . Formatting Page K : Report의 Page K가 작업중이다.
- . In Parameter Form : Runtime Parameter Form에 화면에 나타나있다.
User가 Parameter Form에서 "Run Reports"를 선택하 기전 까지 Report
는 실행되지 않는다.
- . In Reviewer : Report는 현재 Oracle Reports Previewer상태로 나타나있다.
다음 Report는 Previewer가 종료되기 전까지 실행되지 않는다.
- . Initializing Report : Report 실행이 시작중이다.
- . Waiting : Report가 큐에 쌓여 실행되기 기다리고 있다.

(V2.5) ORACLE REPORTS PERFORMANCE TIPS

Bulletin no : 10257

Oracle Reports Performance Tips

1. 일반적으로

- 1) Formula나 Summary Field를 이용하는 것보다는 Query생성시에 계산을 직접하는 것이 빠르다.
- 2) Group Filter나 Format Trigger에서 Record를 선택해 내는 것보다 WHERE절에서 처리하는 것이 빠르다.
- 3) Oracle Reports에서의 Truncate기능을 사용하는 것보다 BULKTRUNC Function에 사용하는 것이 빠르다.

2. SRW.DO_SQL

SRW.DO_SQL은 Report상에서 SQL이나 SQL서리를 할 수 있게한다.

이 기능을 매우 유용하지만, 잘못사용하면 많은 시간을 낭비하게 되므로 꼭 필요한 경우에만 사용하도록 한다. SRW.DO_SQL문장이 Parsing되면, Database에 Cursor가 Open되고 문장이 실행된다. Query에 과 달리, SRW.DO_SQL은 해당 Group에서 Data가 Fetch 될 때마다 매번 그렇게 처리된다. 예를 들어, 10건의 Record를 Fetch하게될 Group에 SRW.DO_SQL문이 포함되어 있다면 이 문장은 10번 Parsing되고, 10번 Cursor가 Open되고, 10번 실행되게 된다. SRW.DO_SQL을 사용하는 것보다는 Query나 Group의 PL/SQL에서 계산을 수행해 주는 것이 좋다.

3. CDE_MM.GET_REF

CDE_MM.GET_REF Package역시 꼭 필요한 경우에만 사용하는 것이 좋다.

이것은 Oracle Reports에서 사용할 Temporary Space의 양을 감소시키는 결과를 가져오기 때문이다. Temporary Space에 대한 요구가 줄어드는 동안, Column의 값을 Database로 부터 가져와야 하기 때문에 Performance가 낮아지게 된다.

4. Multi-Query Data Model을 사용할 때

가능한대로 Report내에 Query의 수를 적게 가져가도록 한다. Query의 수가 적을 수록 실행속도는 빨라지게된다. Multi-Query Data Model이 이해하기는 쉬우나, Single Query가 실행속도는 빠르다.

다음과 같은 경우라면 Multi-Query를 사용한다.

- 1) Parent로 부터 많은 자료를 Fetch하여 적은 양의 Child값을 가져올 때
- 2) SELECT문으로부터 직접적으로 자료를 가져올 수 없을 경우
- 3) 복잡한 View를 가진 경우 (분할된 Query들이나, Group By Query사용시)
- 4) View가 필요하지만 View를 가지고 있지 않거나, View를 사용하고 싶지 않을 때

5. Index에 관하여

WHERE절의 Key Column이나, Detail Query에 걸린 Table에 대해 Index를 생성한다.

Master Query에 걸린 Index는 대부분 한전 Database를 찾기 때문에 큰 영향을 주지 않으나, Master/Detail형식의 Report의 Performance향상에는 중요한 역할을 하게된다. View-Query의 Master에 대한 Detail Record의 비율이 적어질 수록, Detail Query에 걸린 Index는 더욱 중요하게 된다.

Oracle Report에서는 Detail Query를 생성하면 내부적으로 Parent/Child관계를 갖는 WHERE절을 생성하기때문에 Detail Query에 연결된 Table에 대한 Index 생성을 권장한다.

6. Query문 수정

Oracle Report는 User의 Query들을 다음과 같은 경우에 편집한다.

- 1) Link를 생성하면 Oracle Report는 Child Query에 해당 Link에 대한 다음과 같은 절을 추가한다.

예를 들어:

```
SELECT Deptno, Empno, Sal
FROM Emp
WHERE Sal > 1000
```

만약, 위의 Query에 대해 Child Column으로 DEPTNO를 사용하는 Query에 Link를 생성하면 User의 Query는 다음과 같이 수정된다.

```
SELECT Deptno, Empno, Sal
FROM Emp
WHERE (Sal>1000) And (Deptno = :Deptno)
```

단, Multi-Query Matrix에서는 위의 경우와 다르다.

- 2) Oracle Report는 Break Order Bat를 갖는 Database Column에 대해서 Query에 ORDER BY절을 추가한다.

예를 들어:

```
SELECT Deptno, Empno, Sal
FROM Emp
ORDER BY Sal
```

만약, 위의 Query에 대해 DEPTNO를 Break Column을 하는 Break Group을 생성하면 Query는 다음과 같이 편집된다.

```
SELECT Deptno, Empno, Sal
FROM Emp
ORDER BY 1, Sal
```

이러한 SQL문장은 Database로 보내져서, SQL Optimizer가 Database로 부러 Data를 가져오는 데 적당한 방법을 결정하여 왔든 Data를 Oracle Report로 보내오게 된다.

Optimizer는 Index를 사용할 것인지 아닌지, 어느 것을 "Beliving" Table로 사용할 것인지 등을 결정하게 된다.

7. Break Column

Break Group을 생성한다면, 가능한대로 Group내의 Column을 적게 가져가도록 한다. Break Group과 Break Column간에 1:1비율이 되는 것이 좋다. 또한 Break Column이 가능하면 작도록 한다. Break Column이 긴정보다는 짧은 것이 일반적으로 좋은 Performance를 가져온다. 만약 긴 Column에 대해 Break Column을 가져가야 한다면, DBMS Function을 이용하여 Length를 줄이는 것도 Performance에 도움이 된다.

각각의 Break Group에 대해, Oracle Reports는 각각의 Break Column에 대해 DBMS BY접을 가져간다. (한가지의 예외가 있다면 Break Column이 Formula Column일때) Break Group의 Break Column의 수를 적게가져갈 수록, 추가되는 DBMS BY접이 적어지게 된다. DBMS BY접에 추가되는 Column이 적을 수록 Query가 수행될 때수행되는 Processing이 적어지게 된다. 이것은 결국 Oracle Reports가 내부적으로 생성하는 Index Key의 Size가 작아지게 되고, 결국 좋은 Performance를 가져오게 되는 것이다.

8. Maximum Rows And Group Filters

Query Property화면에서 Maximum Row Property를 지정하는 것은 Report의 Query 에서 Retrieve하는 Record의 수를 줄여준다. 많은 양의 Data를 Access하나, Retrieve 하는 Data의 양에 제한 주면 Test하는 동안 Report는 좀더 빠르게 수행될 것이다.

Query Property화면에서의 Maximum Row를 지정하면 Query에 의해 Fetch 되는 Record의 수를 제한 할 수 있다. Group Filter는 어떤 Record를 포함할 것인가, 제외 할 것인가를 결정해준다. Maximum Row를 지정하는 것이 실제로 Retrieve되는 Data의 양에 제한을 주었기때문에,대부분의 경우에 있어서 Group Filter보다 빠르다.

만약 Group Filter를 마지막이나 조건적으로 지정하였다면, Oracle Reports는 Group내에서 Filtering을 하 전에 모든 Record를 Retrieve해야한다. Maximum Row나 First Filter Batting이 더 빠르게 된다. 일반적으로 Maximum Row의 Batting이 First Filter보다 빠르는데,그 이유는 Record의 수가 결정하게 된다.

Performance의 차이는 User가 지정하는 NUMBER에 따라 달라 질수 있다.

9. Unused Data Model Objects

Report에서 실제로 사용하지 않는 Data Model Object가 있는지 확인하고 있다. 먼 Remove하도록 한다. 특정한 경우에만 사용하는 경우가 있다면 BML, BSW_MAKINGDI Packaged Procedure를 이용하여 조건적으로 Suppress하도록 한다. BML,BSW_MAKINGDI(Query_Name,0)로 Batting하면 Query는 한건의 Data도 Fetch하지 않게 된다.

10. Unused Frames

Layout에서 필요없는 Frame들은 지우도록 한다. Oracle Reports는 Default Layout를 생성 할 때 모든것에 대해 가상의 Frame을 생성한다. 이것은 Output에서 다른 Object에 의해 Frame내의 Object들이 Overwritten 되는 것을 막기위한 것이다. 만약 User가 Frame내의 Object가 Over Written될 가능성이 없다고 판단된다면, Report의 Output에 영향을 주지 않는 Frame은 제거해도 된다.

Layout상에 Object수가 적을 수록, Oracle Reports가 Runtime시에 Format해야 하는 Object수가 적어지게 되어 결국 Performance가 향상되게 된다.

11. Total Number Of Pages

Field Source의 Total Number Of Pages의 값을 가능한 쓰지 않는 것이 좋다.

Field Source에서 Total Number Of Pages를 사용하면, Oracle Reports는 Total Pages의 수를 결정하기 위해 모든 Page를 Temporary Space에 저장해야만 하기 때문이다.

이것은 결정적으로 Report가 사용하는 Temporary Disk Space의 양을 증가시키게 되고, File에 Write하기 때문에 속도가 느려지게 된다.

12. Format Triggers

Object의 렌더수가 적은 Object에 Format Trigger를 걸도록 한다. 일반적으로 Field보다 Frame에 지정된 Format Trigger의 PL/SQL이 빠르다.

Format Trigger내의 PL/SQL은 그 Object가 나타날 때 마다 실행된다. Object의 출현렌더수가 적을 수록, PL/SQL이 실행되는 렌더수가 적어지므로 속도가 빨라지게 되는 것이다.

13. Oracle Graphics Integration

Oracle Reports의 자료를 일부 혹은 전부를 이용하는 Oracle Graphics Display를 Report에서 사용하면, Report에서 Display로 Data를 전송하는 것이 좋다.

Report와 Display가 같은 Data를 사용하면, Data를 Parsing하는 것이 Fetch되는 Data의 양을 줄여주기 때문이다. 만약 Report에서 Display로 Data를 전송하지 않는다면, Data는 실제로 한번은 Report에서 한번은 Display에서 Fetch되어 두번 Fetch된다.

(V1.1) REPORT SOURCE LISTING GENERATION

Bulletin no : 10253

Decal에서는 BQLReportWriter Base Table에 저장되어있는 Report들의 List를 생성하는 Program을 제공하고있다.

이것은 BBU00.exe, BBU132.exe, BBU175.exe의 3가지 Report호퍼 B0, 132, 175은 출력되는 Report의 Column Width를 의미한다. 즉, BBU00 Report를 실행하면 결과가 B0 Column용지에 맞게 출력된다.

* Location

1. UMB - DBA_ROOT:[BQLREPORTLISTEN,BBU_MENB]
2. UNIX - \$DBACLE_HOME/sqlreport/dmco

* 생성방법

1. .exe File를 Load한다.

```
loadrep file=crub0 userid=your_user_id
```

2. Generate한다.

```
genrep file=crub0 userid=your_user_id
```

3. crub0를 Execute한다.

Listing을 하고싶은 Report의 이름과 출력방법(Screen/File/...)을 선택하는 Dialog Box가 나타난다.

(M) SQL*REPORTWRITER에서 SOURCE를 자동으로 BACKUP 받는 방법

Bulletin no : 10280

Report가 Report되거나, Source가 들어있는 BDL Table에 손상이 생겨 예전의 File로부터 복구가 필요할 때가 있다.

이때 Source를 이전에 Backup한 .rep File이 있어야만 복구가 가능하다.

Version 2.x에서는 .rep File이 Report를 생성할 때 자동으로 생성될 수 있지만, 1.1에서는 기본적으로 제공하지 않기 때문에 User는 주기적으로 Report Source를 Backup하여 .rep File로 보관하여야만 한다.

이 자료에서는 매일매일 이 작업을 손쉽게 처리하도록 해준다.

내용 : System Date를 받아 BDL Table로 부터 해당 일자에 변경된 Report들을 Backup한다.
Background Job으로 처리 할 수도 있다.

```
# Automatic BDL generation of modified reports
```

```
# Set some local variables
```

```
#
```

```
cat BDL_DIR=/usr/ucb/home/rep/rep # dir to hold .rep files
```

```
cat BDL_DIR=/usr/ucb/home/plus # dir to hold .sql files
```

```
cat USERNAME=scott/tiger
```

```
cat BEFORE_DATE='date -t $(date +%Y%m%d%H%M%S)'
```

```
#
```

```
# Startup BDL
```

```
#
```

```
$ORACLE_HOME/bin/sqlimp $USERNAME
```

```
#
```

```
# Create list on all reports that changed
```

```
#
```

```
cat Reports='sqlplus -x scott/tiger @${BDL_DIR}/get_mod_rep "${BEFORE_DATE}"'
```

```

$
$ Loop for all the reports found to be modified
$

foreach f ($Reports)
    echo Dumping $OBJ_ID/$f
    dumprep reports-$f file=$OBJ_ID/$f userid=scott/tiger >| /dev/null
end

+-----+

아래의 get_mod_new SQL script는 위에서 호출한 SQL문이다.
+-----+

set heading off
set feed off
set pagesize 999
set var off

select report_name
      from new_report
      where to_char(modified_date,'YYYYMMHH24MISS') > 21;

exit;
+-----+

```

(V2.5) REPORT에서 자주 사용되는 SRW PACKAGED PROCEDURE

Bulletin no : 102B4

Introduction

Oracle Reports는 User가 Library나 Report에서 Reference하여 이용할 수 있는 많은 Functions나 Procedure, Exception들을 제공하는 PL/SQL Program을 제공한다.

이 Oracle Reports의 Package의 이름이 BML이다. User는 언제나 Reference하여 사용할 수 있으며 사용할 때는 예를 들어 BML.BD_BQL처럼 반드시 BML을 앞에 붙여서 사용하여 한다.

Oracle Reports에서 제공하는 BML Package내에는 다음과 같은 Function 혹은 Procedure들이 있다.

- | | |
|--------------------------|--------------------------|
| - BML.BMLERR | - BML.BML_REPORT_BATCHNO |
| - BML.COMMENT_FAILURE | - BML.SET_FIELD_CHAR |
| - BML.BD_BQL | - BML.SET_FIELD_DATE |
| - BML.BD_BQL_FAILURE | - BML.SET_FIELD_NUM |
| - BML.SET_ERR_NUM | - BML.SET_MAXNUM |
| - BML.SET_PAGE_NUM | - BML.TRACE_APP_OPTION |
| - BML.INSERTION_ERROR | - BML.TRACE_ERR |
| - BML.MAXNUM_INSERT | - BML.TRACE_ERR_OPTION |
| - BML.MAXNUM_INSERT | - BML.TRACE_STAT |
| - BML.NEEDPAGE | - BML.TERMINATED_VALUE |
| - BML.NULL_ASSIGNMENT | - BML.UNMEDIUM_QUERY |
| - BML.PROGRAM_ABORT | - BML.UNMEDIUM_USER_KEY |
| - BML.REPORTERR | - BML.USER_EXIT |
| - BML.BML_REPORT | - BML.USER_EXIT20 |
| - BML.SET_ATTR | - BML.USER_EXIT_FAILURE |
| - BML.BML_REPORT_FAILURE | |

자주 사용하는 SRW Package와 Sample들

1. BML.BD_BQL은 특별히 지정하는 BQL문장을 실행한다. 이 Procedure는 어떤 BQL이나 BML문장을 실행하지만 일반적으로 BML.BD_BQL을 통해 실행한 것보다 PL/SQL에서 실행한 BML문장이 빠르다. PL/SQL에서는 BQL문장의 사용을 할 수 없기 때문에 이 Packaged Procedure를 User Exit를 사용하는 것보다는 유용하다고 본다.

Report에서 사용하는 BQL 문장에서 Table을 Modify하는 것은 가능한 피하는 것이 좋다. Report를 실행하기전에 Snapshot이 생성되어 Report가 실행되는 동안 계속 남아있기 때문이다.

Syntax/Explanation

```
SQL, DO_SQL(Sql_Statement CHAR): -- 지정된 SQL문을 실행한다.
SQL, DO_SQL_FAILURE:             -- SQL, DO_SQL에서 실패한 경우 Report의 실행을 중지한다.
SQL, MESSAGE(Msg_Number NUMBER,Msg_Text CHAR):
    --지정된Message와Message번호를 화면에 뿌린다.
SQL, PROGRAM_ABORT:              -- Report의 실행을 중지한다.
```

Example

RunTime Parameter Form에 쓰기전에 TEST라는 Temporary Table을 생성하는 예를 본다.

```
BEGIN
    SQL, DO_SQL(CREATE TABLE Temp
                ( Report NUMBER NOT NULL PRIMARY KEY,
                  Bal      NUMBER(10,2))
                PCTFREE 5 PCTUSED 75);

    RETURN TRUE;
EXCEPTION
    WHEN SQL, DO_SQL_FAILURE THEN
        SQL,MESSAGE(100,'ERROR WHILE CREATING CHECK TABLE');
        SQL,MESSAGE(50,'REPORT HAS STOPPED BEFORE THE PARAM FORM');
        RAISE SQL, PROGRAM_ABORT;
END;
```

2.

Syntax/Explanation

```
SQL, RUN_REPORT(Command_Line CHAR): -- RUNTIME Command를 실행.
SQL, RUN_REPORT_FAILURE:            -- SQL, RUN_REPORT에서 오류가 발생하면
    -- Report의 실행을 중지한다.
```

Example

Report Button으로부터 화면으로 출력하는 Drill Down Report를 호출한다.

```
FUNCTION Foo RETURN BOOLEAN IS
BEGIN
    SQL, RUN_REPORT('Report=REP_A P_Param1=20');
EXCEPTION
    WHEN SQL, RUN_REPORT_FAILURE THEN
        SQL,MESSAGE(100,'ERROR CALLING REPORT, ');
        RAISE SQL, PROGRAM_ABORT;

    RETURN TRUE;
END;
```

3.

Syntax/Explanation

BMU.SET_ATTR는 Layout Object에 대한 Font Size혹은 Color와 같은 Attribute의 Setting을 처리한다.

이 Procedure는 Current Frame, Repeating Frame, Field, Ballerplate Object Attribute를 처리할 수 있게한다.

```
BMU.SET_ATTR(Object_Id NUMBER, Attr BMU.GENERIC_ATTR);
```

Object_Id : 항상 ZERO 이다.

Object는 항상 자신의 Attribute를 지정해야만 한다.

Attr : BMU.ATTR이다.

예를 들어 바꾸고자하는 Attribute와 그 값.

Example

Balump가 2000이면 다음의 Attribute를 Setting한다.

Border Width=1

Foreground Border Color = Red

Background Border Color = Blue

Border Pattern = Checker

Foreground Fill Color = Yellow

Background Fill Color = Green

Fill Pattern = Diagoncross

```
IF :Bal = 2000 THEN
```

```
    BMU.ATTR.NBSE := BMU.BORDERWIDTH_ATTR +
```

```
                                BMU.FBCOLOR_ATTR      +
```

```
                                BMU.BBCOLOR_ATTR      +
```

```
                                BMU.BDPATT_ATTR        +
```

```
                                BMU.FFCOLOR_ATTR      +
```

```
                                BMU.BFCOLOR_ATTR      +
```

```
                                BMU.FILLPATT_ATTR;
```

```
    BMU.ATTR.BORDERWIDTH      := 1;
```

```
    BMU.ATTR.FBCOLOR          := 'Red';
```

```
    BMU.ATTR.BBCOLOR          := 'Blue';
```

```
    BMU.ATTR.BDPATT           := 'Checker';
```

```
    BMU.ATTR.FFCOLOR          := 'Yellow';
```

```
    BMU.ATTR.BFCOLOR          := 'Green';
```

```
    BMU.ATTR.FILLPATT         := 'Diagoncross';
```

```
    BMU.SET_ATTR(0, BMU.ATTR);
```

```
END IF;
```

```
RETURN TRUE;
```

4.

Syntax/Explanation

```
SQL.SET_FIELD_CHAR(Object_Id, Text CHAR);  
SQL.SET_FIELD_BYTE(Object_Id, Data BYTE);  
SQL.SET_FIELD_NUM(Object_Id, Number NUMBER);
```

이 SQL.SET_FIELD Procedure는 Format Trigger에서 매우 유용하게 쓰인다.
이것은 Report에서는 Field의 Value를 직접 Assign할 수 없기 때문에 특별한 조건인 경우에만 나타나게 하려는 경우에 자주 사용된다.

Example

Salary가 2000보다 적은 직원들의 경우에 'LOW'라는 문자를 출력하려면 다음과 같은 Format Trigger를 Salary_Range_Field에 걸어야 한다.

```
FUNCTION ChgField RETURN BOOLEAN IS  
BEGIN  
  IF :Sal < 2000 THEN  
    SQL.SET_FIELD_CHAR(0, 'LOW');  
  ELSE  
    END IF;  
  RETURN TRUE;  
END;
```


(V2.5.X) 조건에 따라 HIGHLIGHTING, COLORING 처리하는 방법

Bulletin no : 10288

DATA MODEL에서

다음의 간단한 Query를 생성한다.

```
Q_Highlight      :   SELECT Ename, Ename, Sal
                     FROM Emp
                     ORDER BY Ename
```

LAYOUT에서

1> F_ENAME Field의 Property화면 * General Layout * Format Trigger

```
+-----+
FUNCTION F_Enameformattrigger
RETURN BOOLEAN IS
BEGIN
    IF :Sal > 2000 THEN
        BM.ATTN.HAZE := BM.HEIGHT_ATT + BM.FACE_ATT;
        BM.ATTN.HEIGHT := BM.BOLD_HEIGHT;
        BM.ATTN.FACE := *Helvetica*;
        BM.SET_ATT(0, BM.ATTN);
    END IF;
    RETURN TRUE;
END;
```

2> F_SAL Field의 Property화면 * General Layout * Format Trigger

```
+-----+
FUNCTION F_Salformattrigger
RETURN BOOLEAN IS
BEGIN
    IF :Sal > 2000 THEN
        BM.ATTN.HAZE := BM.HEIGHT_ATT +
                        BM.FACE_ATT +
                        BM.BOLDHEIGHT_ATT +
                        BM.FCOLOR_ATT +
```

```

                                BOLD/NOBOLD_PATT_ATT:
    BOLD_ATT,WEIGHT := BOLD_BOLD_WEIGHT;
    BOLD_ATT,FACE   := *helvetica*;
    BOLD_ATT,FFCOLOR := *BLACK*;
    BOLD_ATT,NOBOLD_PATT := *BOLD*;
    BOLD.SET_ATT(0,BOLD_ATT);
END IF;
RETURN TRUE;
END;
+-----+

```

위의 Trigger들은 Bal이 2000이상이면 BOLD_ATTRIBUTES의 값을 새롭게 적용하게된다.

- 3> F_BAL의 Format Mask를 -MM,00로 Battling한다.
- 4> Report를 Save하고 Run시킨다.

Highlighting대신에 Color를 Battling하는 조건Trigger
 =====

F_FORMAT에 걸려있는 Trigger를 다음의 Color를 Battling하는 Format Trigger로 대체한다.

```

+-----+
FUNCTION F_Formatcolortrigger
RETURN BOOLEAN IS
BEGIN
  IF :Bal > 2000 THEN
    BOLD_ATT,MASK := BOLD_WEIGHT_ATT + BOLD_FACE_ATT +
                      BOLD_FILL_PATT_ATT + BOLD_FF_COLOR_ATT ;
    BOLD_ATT,WEIGHT := BOLD_BOLD_WEIGHT;
    BOLD_ATT,COLOR := *RED*;
    BOLD_ATT,FILL_PATT := *BAY10*;
    BOLD_ATT,FF_COLOR := *BLUE*;
    BOLD.SET_ATT(0,BOLD_ATT);
  END IF;
  RETURN TRUE;
END;
+-----+

```

F_BAL에 걸려있는 Trigger를 다음의 Color를 Battling하는 Format Trigger로 대체한다.

```

+-----+
FUNCTION F_Balformattrigger
RETURN BOOLEAN IS
BEGIN
  IF :Bal > 2000 THEN
    BOLD_ATT,MASK := BOLD_WEIGHT_ATT + BOLD_FACE_ATT +
                      BOLD_FILL_PATT_ATT + BOLD_FF_COLOR_ATT +
                      BOLD_NOBOLD_PATT_ATT + BOLD_FF_COLOR_ATT +

```

```

        BM, BORDPATT_ATT:
        BM, ATT, WEIGHT := BM, BOLD_WEIGHT;
        BM, ATT, CCOLOR := %RED%;
        BM, ATT, FILLPATT := %SHAD10%;
        BM, ATT, FFCOLOR := %BLUE%;
        BM, ATT, BORDWIDTH := 0;
        BM, ATT, FBCOLOR := %BLUE%;
        BM, ATT, BORDPATT := %BLID%;
        BM, SET_ATT(0, BM, ATT);
    END IF;
    RETURN TRUE;
END;
+-----+

```

위에서 사용한 SRW Attributes의 예

ATTRIBUTE	DEFINITION

BM, ATT, NAME	User가 Create하는 Name
BM, WEIGHT_ATT TEXT WEIGHT	
BM, FACE_ATT	TEXT FONT
BM, CCOLOR_ATT TEXT COLOR	
BM, FILLPATT_ATT	FIELD를 채우는 PATTERN
BM, FFCOLOR_ATT	FOREGROUND FILL COLOR
BM, ATT, FBCOLOR	BACKGROUND FILL COLOR
BM, ATT, BORDPATT	BORDER PATTERN

(V25) ORACLE REPORTS에서의 FORMAT TRIGGER사 용에 관하여

Bulletin no : 10270

Format Trigger는 Report를 Format하는데 아주 Powerful하게 사용된다. 이것은 어떤 Object에 대한 Highlighting이나suppressing 혹은 Column의 값이나 Heading을 변경할 수도 있게한다. Format Trigger는 PL/SQL Function 으로서, True와 False의 Boolean값을 Return한다. Trigger내에서 Oracle Reports의 Packaged Function이나 Procedure, Exception을 사용할 수도 있다. Format Trigger는 Oracle Reports가 해당 Layout Object를 Format할 때마다 매년 실행되게 된다.

조건에 따라 해당 Column의 값을 바꾸어 나타내거나, 값을 나타내지 않도록 막으려한다면 해당 Layout Object에 Format Trigger를 생성한다. 여기에서 만약 Report내의 다른 Object의 들 사용할야 한다면, Frequency가 맞는지에 주의하여야 한다.

=====

Format Trigger에 대한 기본 Rule

=====

1. 동일한 Frequency를 갖는 Column이나 Parameter의 값을 믿어들이는 것은 가능하지만, 값을 직접 주는 것은 만된다. (예: :Count = 10)
2. Format Trigger가 생성된 Object과 다른 Frequency를 갖는 Column이나 값은 Reference할 수 없다. (예: Header/Detail Report에서, Header의 Format Trigger에서는 Detail Item의 값을 Reference할 수 없다.)
3. Format Trigger에서 False를 Return하여 Object가 Format되지 않는다면 이것 때문에 다른 Object들이 Print되지 않을 수도 있다. (예: Group Frame에 Format Trigger에서 False가 Return되었다면, 그속에 속한 모든 Object들 역시 Format되지 않게 된다.)
4. Reporting Frame에 Format Trigger가 생성되어있다면, Return되는 Record의 수만큼 Format Trigger가 실행된다.
5. 같은 Object에 대해 Format Trigger는 일관된 결과를 Return한다. (예: 한 Object가 두Page에 걸쳐있다면 Trigger는 한 Page에 한번씩 실행된다.)
6. Matrix형 Report의 Reporting Frame에 생성된 Format Trigger에서 False를 Return하게된다면, Matrix내의 모든 Column이나 Row의 값도 나타나지 않게 된다.
7. Matrix의 Cell의 조건에 따라 서리하고 싶다면, Cell내부의 Field를 감싸는 Frame을 생성하여 그 Frame에 Format Trigger를 생성하면 된다 (여기에서 정확한 Level위에 Frame을 위치시켜야만 한다.)

Sample

Case 1. 값이 Null인 Column에 대해서는 Zero를 표시해 주려면?

```
Function My_Formatting Return Boolean Is
Begin
  If :Cname Is Null Then
    Bwv.Bvt_Field_Num(0,'0');
  End If;
  Return True;
End;
```

Case 2. 값을 큰 값으로 대체하여 표시하려면?

```
Function My_Formatting Return Boolean Is
  Top Number;
Begin
  If :Cname Is Not Null Then
    Top := :Cname * 1.5;
    Bwv.Bvt_Field_Num(0,Top);
  End If;
  Return True;
End;
```

Type : 위의 예에서 Field의 Type이 Date 혹은 Char라면 각각 Bwv.Bvt_Field_Date, Bwv.Bvt_Field_Char를 사용하면 된다.

Case 3. Master/Detail Report에서 Detail자료가 없는 Master Record가 있다고 가정하자. Master 는 그대로 출력되지만 Detail에 관한 Header부분은 생략되는 것이 좋을 것이다.

Step 1 : Detail Group의 Record수를 Count하는 Summary Column을 생성한다.

Master Group에서 Repeat하도록 Betting한다.

Step 2 : Detail Group의 M_Group Object에 Format Trigger를 생성한다.

```
Function My_Formatting Return Boolean Is
Begin
  If :Count_Col = 0 Then
    Return(False);
  Else
    Return(True);
  End If;
End;
```

Case 4. Report를 생성하였지만 조건에 맞는 자료만 출력하고 싶은 경우를 가상한다. 일반적으로는 Report를 생성할 때 Query에 조건을 주어 원하는 Report는 아래 Retrieve를 하지 않는다.. 그러나 Record들은 모두 Retrieve하지만 출력만 하지 않으려면 (예를 들어 Retrieve된 Record의 Summary는 구하고, 단지 출력만 하지 않으려는 경우) Reporting Frame에 다음과 같은 Format Trigger를 생성한다.

Condition이 3000보다 큰 사원 만 출력하려는 경우

Function My_Formatting Return Boolean Is

Begin

If :Csum > 3000 Then

Return(False);

Else

Return(True);

End If;

End;

Case 5. User가 Input한 값에 따라서 Record/Column을 선택하기도 하고, Column의 값이나 Column Heading, Report Title/Header를 바꾸기도 하는 경우를 생각해 보자. 우선 User의 Input을 받은 Parameter를 생성한 뒤 해당 Object에 다음과 같은 Format Trigger를 생성한다.

Function My_Formatting Return Boolean Is

Begin

If :P1 = <Condition1> Then

Bew.Ret_Field_Char(0,'This is My New Value.');

ElseIf :P1 = <Condition2> Then

Bew.Ret_Field_Char(0,'This is My New Value 2.');

End If;

Return(True);

End;

④ TABLE에 IMAGE FILE에 대한 PATH만 가지고 있는 경우의 처리

Bulletin no : 10921

Image가 Table에 직접 들어있는 경우는 Form에서나 Report에서 Default Format으로 간단히 처리할 수 있다. 그러나 Table의 Column에 실제의 Image가 아닌 Image File에 대한 Path와 경로를 가지고 있는 경우에는 다음과 같이 처리하여야 한다.

1. Form4.5에서는

작업하고자 하는 Block에 Image Item을 생성한다.

적당한 Trigger에서 Read_Image_File Procedure를 이용하여 Image를 화면으로 읽어들인다.

→ Form Reference Manual참조

2. Report2.5에서는

1) Image경로를 가진 Column을 Select하는 Query를 생성한다.

2) Data Model에서 해당 Column의 Property

→ Check Box확인 : Read From File Checking

→ List Value확인 : 열의 List에서 Image선택

3) Default Layout생성

→ Image Column의 Size조정

4) Run

주의사항: 해당 Image File들은 Working Directory나 Current Directory, 혹은 Oracle INI File의

Report25_PATH에 지정된 Directory내에 있어야 한다. → Building Reports Manual B-B1참조

(V2.5) REPORT2.5에서 ORACLE GRAPHICS를 띄우려면?

Bulletin no : 10929

1) Oracle Graphics에서 `mpcal.ogd`라는 Oracle Graphics display를 만든다.

```
query name      : Q_DC_Emp
select statement : SELECT ENAME, DE_NAME, BAL, DC_BALANCE
                  FROM EMP
```

2) Report2.5의 data modeling

query name	group name	select statement
Q_dept	G_dept	select deptno, dname, loc from dept
Q_emp	G_emp	select ename, sal, deptno from emp

Q_dept와 Q_emp는 master/detail로 연결

3) Layout Editor의 Oracle Graphics tool을 사용하여 master repeating frame(Q_dept)안에 Oracle Graphics object를 그려 넣는다.

4) Oracle Graphics object의 property에서 Display Name을 `mpcal.ogd`를 입력한다. (path를 지정하지 않으려면 `mpcal.ogd`를 `oracle`의 `REPORTS25_PATH`의 directory에 위치시켜야 한다.)

5) Oracle Graphics object의 Property sheet의 Query tab

Source	Display Query	Report/Display Column
G_emp	Q_DC_Emp	ENAME / DE_NAME BAL / DC_BALANCE

Display Query에 Q_DC_Emp로 지정한 것은 Graphics가 database로부터 query를 하지 않고 Oracle Reports에서 ENAME 과 BAL을 retrieve하겠다는 의미이다.

* 만약 위의 예제처럼 Report의 data를 retrieve하지 않고 단순히 parameter만 packing하여 Oracle Graphics에서 database로부터 query를 하려면 -

Graphics의 query에서 `where deptno=parameter1` 이라면 Report의 Oracle Graphics object의 property에서 Display tab의 Report Column / Display Parameter에 deptno / parameter1 으로 writing하고 query tab부분은 지정하지 않는다.

* 자세한 사항은 Oracle Reports2.5 Reference Manual B-74 참조.

SOLARIS 2.X SETUP PROBLEM

Bulletin no : 10008

1) SPARC system에 SCSI driver를 붙이고자 할때

	SCSI number	device file
CTB driver	4	/dev/rct4
BAT driver	5	/dev/rct5
CB driver	6	/dev/rct6

위와 같이 SCSI number를 맞추고 rebooting을 합니다.

Shutdown이후 Auto booting이 되게 하지 말고 "Stop"과 "Up"을 동시에 눌러 Boot ROM상태로 빠지게 한 후에 SCSI driver가 제대로 연결 되어 있는지를 다음의 command로 확인.

```
ok probe-scsi
```

--> ok는 Boot ROM상태의 Prompt

SCSI driver가 제대로 연결 되었다면 각 SCSI driver의 Vendor Name과 SCSI number가 Display되고 제대로 연결 되지 않았으면 그냥 멈춰있거나 새로 연결된 driver의 정보가 나타나지 않을때 이 때는 SCSI Cable또는 SCSI number를 다시 조사하기 하함.

probe-scsi command의 이후에는 다음 command로 booting

```
ok boot -p
```

2) Solaris 2.2에서 Dracm을 install할 때는 반드시 Console에서 작업하기 하함.

Console이 아닌 OpenWindows의 "Command tool" 이나 "Shell tool"에서 install 할 경우 Dracm Server를 돌리지 못함.

이것은 solaris2.2의 Shared Memory Bug인 듯하며 Solaris2.3에서는 Fix됨. 즉 solaris2.2에서는 OpenWindow가 한전 실행되면 Dracm Server를 실행하지 못하기 때문에 rebooting하고, OpenWindow가 수행되기 전에 "Control-C"로 Kill시킨 후에 Dracm을 install하거나, install이된 system은 xglbks로 가서 startup시켜야함.

3) BusPro에서 본주고 사야하는 Compiler를 구입하지 않아도 Dracm install은 가능함.

사용자가 ProC를 쓰지 않는다면 Dracm install과 사용하는데는 별 문제 없음.

Dracm install하기 전에 반드시 "/usr/ccs/bin" PATH를 설정하여 install하다

"make" command를 못찾아 중간에 끝나는 경우가 생기지 않게 하함.

4) `Proc0`에서 `compile`에 다음과 같은 `error message`가 나오면

```
Undefined                      first referenced
symbol                          in file
__cg82_11k                     (command line)
```

`LD_LIBRARY_PATH`에 실종할거 `"/usr/lib"나 "/lib"가 지정되어 있을 경우이므로 이 PATH를 삭제하기` 바람. 이것은 `BusPro compiler`의 `lib.a`를 `link`해야 하는데 `"/usr/lib/lib.a"를 link하기 때문에` 생기는 문제임.

위의 `error`는 `cc option`에 `"-cg82"`가 빠져서 나오는 다음의 `error message`하고는 다른 것임.

```
Undefined                      first referenced
symbol                          in file
__cg82_used                     @BNDCLRE_HDNE/lib/libcgl.a(cglsxp.o)
```

5) 한글 `Demulimac`에 한글서미들 `kit`라는 `Input Server`를 통하여 한글입력을 받는데 `delimiter`(Data의 시작이나 끝을 나타내는 문자로 대표적인 `key`는 `space`, `enter key`)가 오면 그들만의 문자를 `K=limit`에 넘겨주는데 한전에 넘겨주는 문자의 길이를 사용자가 `Parameter`로 정할 수가 있다.

√ `enter PREEDIT_LEN 1`

위에서 보듯이 `PREEDIT_LEN`의 크기를 1로 정리하면 한글 한글자가 완성되면 `delimiter`가 입력이 되지않아도 바로 `K=limit`로 전달되며 `form`에서 한글 data 입력시 다음 타일까지 넘어가는 현상은 잘 생하게 되지 않음.

`PREEDIT_LEN`의 `default`값은 20이다.

UNIX 에서 RAW DEVICE 사용하는 방법

Bulletin no : 10078

RAW DEVICE 는 CHARACTER DEVICE DRIVEN 을 사용하는 HW DEVICE 이다.
이러한 RAW DEVICE 를 사용했을때 장점으로는

- . RAW DEVICE 는 DB RECOVER 에 의해 BUFFERING 이 되지않고 LOG BUFFER 와 DEVICE 간에 직접 DATA 가 전송되므로 DBACLE BISE I/O 성능 향상되고 CPU OVERHEAD 가 감소된다.
- . DB FILE SYSTEM 의 OVERHEAD 를 피할수있다.
- . DB BUFFER SIZE 를 줄일수 있다.

잠재적인 문제점으로서

- . SETUP 하기 어렵고 BACKUP 절차가 FILE SYSTEM 보다 복잡하다.
- . RAW DEVICE 와 DB FILE 을 혼란하여 사용 할경우 DB FILE 은 ulimit PARAMETER 의 SIZE 보다 작아야 한다. 따라서 ulimit 을 초과하는 TABLE 들은 RAW DEVICE 를 사용하여야 한다.
- . DB 는 CYLINDER 0 을보오하지 못하기 때문에 DBACLE 의해 사용된 RAW PARTITION 은 CYLINDER 0 에서 시작하면 안된다.

RAW DEVICE SETTING 방법

1. FREE BISE PARTITION 의 이름을 설정한다.
FREE PARTITION 은 DB FILE SYSTEM 에 의해 사용 된것이 아니어야한다.

2. DBACLE 의 RAW DEVICE SETTING

```
$ chown oracle /dev/raw/c0x0d1
$ chmod 600 /dev/raw/c0x0d1
```

3. 새로운 PARTITION 을 추가한다.

```
create tablespace B10
logfile 'f/dbs/log1.f' size 100K, 'f/dbs/log2.f' size 100K
datafile 'f/dbs/raw/c0x0d1' size 10000K reuse;

alter tablespace tablespace_name add datafile
'f/dbs/raw/c0x0d2' size 10000K reuse;
```

RAW DEVICE 의 GUIDLINE

1. LOG LOG FILE 와 DATABASE FILE 을 서로 다른 DRIVE 에 놓는다.
2. 다른 DRIVE 에 자주 사용되는 TABLE 을 분산시킨다.

3. TABLE 과 그 TABLE 의 INDEX 를 분리한다.
4. DATABASE 과 DB I/O 를 분리한다.

RAW DEVICE 의 BACKUP 방법

RAW DEVICE 를 DB FILE 로 COPY 하거나 TAPE 로 직접 COPY 하거나 모두 dd 를 사용하여야 한다. TAPE 로 COPY 하는데 하나의 RAW DEVICE 를 여러개 TAPE 에 COPY 걸우는 다음과 같다.

첫번째 TAPE

```
$ dd if=/dev/rdsk/c0c0d1 of=/dev/rct0 bs=20k count=10000
```

두번째 TAPE

```
$ dd if=/dev/rdsk/c0c0d1 skip=10000 of=/dev/rct0 bs=20k count=10000
```

ORACLE 과 UNIX SEMAPHORE 와 의 관계

Bulletin no : 10083

* 유닉스는 내부 프로세스간의 커뮤니케이션을 위하여 세마포어를 사용한다.

세마포어는 유닉스에 의해서 증감되는 정수(integer)값 이다. 하나의 세마포어에 대해서 하나의 프로세스만이 작업을 할 수 있도록 하여 프로세스 간의 동기화를 유지하는데 이용된다. 다른 프로세스가 사용중인 세마포어를 이용하고자 하는 프로세스는 세마포어의 상태가 증가되거나 0 이 될 때까지 기다리게 된다. (물론에 따라 다를 수 있다).

* 유닉스에서의 세마포어는 한개의 이용되는 경우는 거의 없기 때문에 보통 세마포어 집합으로 관리된다. 키스 커널이 설정될 때 시스템에서 사용 가능한 세마포어의 최대 갯수와 세마포어 집합 당 최대 세마포어 갯수 및 할당 가능한 세마포어 집합의 최대 갯수가 설정된다. 이 값들은 유닉스 커널을 다시 만들고 시스템을 리부팅 하여야만 변경이 가능하다.

<오라클과 세마포어>

오라클은 락그라운드 프로세스들 간의 Concurrency 를 조절하기 위하여 세마포어를 사용한다. 그리고 Fast(Shared Memory) Buffer 가 사용되는 경우 여러 프로세스와 세마포어 프로세스 간의 Two-Task Communication 에도 사용된다.

<사용중인 세마포어>

현재 시스템에 할당된 세마포어를 확인하는 명령은 다음과 같다.

```
# ipcs -sk
```

이 명령은 할당된 모든 세마포어 집합과 ID Number, Owner, 각 집합의 세마포어 수 등을 보여준다. 때때로 비정상적으로 종료된 오라클 프로세스에 의해서 점유된 리소스가 릴리스 되지 않고 남아 있는 경우가 있다. 만약, 오라클이 Shutdown 상태인데도 불구하고 ipcs -sk 를 통해서 보았을 때 오라클이 점유한 세마포어가 사용중으로 나온다면 그것들을 IPCSH명령으로 Force 시켜야 한다. 즉, 오라클이 할당한 세마포어 집합에 대하여

```
#ipcrm -s ID (ID는 ipcs 에서 출력됨)
```

명령으로 릴리스 시킨다. 아니면 시스템을 리부팅 하여도 된다.

<데이터베이스의 기동>

오라클은 락그라운드 프로세스에 필요한 모든 세마포어를 데이터베이스 기동시에 할당한다. Init.ora 파일의 Processes 파라미터는 오라클에서 할당할 최대 세마포어 갯수를 설정한다.

가장 흔하게 발생하는 예는 데이타베이스 기동시에 나타나는 다음과 같은 예이다.

DBA-7278 : xpcrmisget error, unable to get first semaphore set

시스템은 한 집합내에 가장 많은 세마포어를 가진 것이나 Process에 의해 설정되는 세마포어 갯수 중 적은 값을 가진 첫번째 세마포어 집합을 할당하려고 한다. 만약, 시스템에 설정된 세마포어가 부족하거나, 세마포어가 이미 너무 많이 사용중이거나, 사용중이지 않은 세마포어 집합이 세마포어를 너무 많이 갖고 있는가를 체크해 보고 그렇지 않다면 시스템에 충분한 세마포어를 할당하도록 한다.

설정된 세마포어가 없거나 모든 세마포어가 이미 할당된 상태라면 다음과 같은 예가 발생한다.

DBA-7251 : xpcrmisget error, could not allocate any semaphores

첫번째 세마포어 집합이 할당되었지만 두번째 세마포어를 할당할지 못하면 다음과 같은 예가 발생한다.

DBA-7252 : xpcrmisget error, could not allocate semaphores

죽어있는 오라클 프로세스에 의해서 세마포어가 점유되어 있는가를 확인하고 만약 그런 문제가 아니라면 더 많은 세마포어를 할당함으로써 문제를 해결할 수 있다.

Shutdown abort 명령이 내려지면 사용자 Process가 끝나기를 기다리지 않고 오라클 백그라운드 프로세스는 죽게되고 그것에 의해서 점유된 세마포어는 Release 된다. 사용자는 세마포어를 늘이거나 줄여서 데이타베이스에 작업들요구할 때와 리소스 데이타베이스가 Shutdown 되었음을 알게 된다. 이 때 사용자에게 의한 세마포어 변경 요구는 실패하게 되며 용제 프로세스와 오라클 Shutdown Process도 함께 죽게 된다. 그러면서 다음과 같은 예가 발생한다.

DBA-7284 : xpcrmisget error, unable to decrement semaphore

DBA-7285 : xpcrmisget error, unable to increment semaphore

DEC BIBC Utiliz, NIPB machine 같은 NIPB BIBC 기한의 용닉스 시스템의 경우에는 오라클은 Startup시에 Latching을 위하여 별도의 세마포어를 할당한다. 오라클은 이 때 할당된 세마포어를 사용자가 데이타베이스에 접속할 때 Latch 로 사용한다. 프로세스가 죽게되면 프로세스에 의해서 변경된 세마포어는 원래의 상태로 되돌아 간다. 따라서 접속하는 모든 프로세스는 Undo Structure를 할당할 수 있어야 하는데 Undo Structure가 충분하지 못하면 다음과 같은 예가 발생한다.

DBA-8702 : xsm_acquire: cannot acquire latch semaphore

해결방법은 Undo Structure가 사용가능할 때까지 기다리든지 시스템의 Undo Structure 의 최대값 (BATCHSIZE)를 늘리면 된다.

Fast Buffer에 의해서 데이타베이스에 접속되는 경우에는 Shared Memory Buffer에 대 사용허가를 관리하기 위하여 세마포어가 사용된다. 이 때 하나의 접속되는 세션마다 3개의 세마포어로 구성된 세마포어 집합이 할당된다.

시스템의 세마포어가 모두 할당되었다면 다음과 같은 예외가 발생한다.

```
DBA-2721 : semaphore: cannot create semaphore set
```

이 경우에는 이용가능한 세마포어가 생길 때 까지 기다려야 한다. 그리고 비정상적으로 프로세스를 종료하는 경우에는 할당된 세마포어가 `time-out`되지 못하며 어떤 프로세스가 어떤 세마포어를 사용하는가를 구분하기 힘들므로 사용상의 주의가 필요하다.

<세마포어 사용 계획>

세마포어 커널 파라미터의 이름, 조정 방법 및 커널 재설정 방법은 시스템에 따라서 다르다. 필요한 세마포어의 갯수는 다음의 공식으로 구할 수 있다.

```
proc(limit.os)에서의 procmaxxx 파라미터)
+ tot(MIPS BIBC 프로세서를 쓰는 경우는 1, 그외에는 0)
+ 3 * 2(fast deliver connection 갯수)
+ nxx(모달을 띄운 프로그램에서 필요한 세마포어 수)
-----
= tot(필요한 최소한의 세마포어 수)
```

세마포어의 최대 갯수를 지하는 파라미터(보통 `BSHMBL`)는 최소한 위에서 계산한 값보다 커야 한다.

일반적으로 하나의 집합마다 가능한 세마포어의 최대 갯수(보통 `BSHMBL`)는 10^{25} 정도이다. `fast deliver`를 많이 쓰는 경우라면 세마포어 집합의 갯수가 더 많이 필요하다.

SUN SOLARIS C COMPILE 시 고려 사항

Bulletin no : 10088

1. Oracle v7에서 SUN C-Compiler version 2.0 or 2.0.1 사용시

C-Compiler 위치 : /opt/BINLisp

DB Version : Solaris 2.4

(1) BYTEMe] BPOCstation 10, classic/LK, 20, 5

BYTEMe] BPOCstation 1000, 2000일 때

이 때의 color graphic library는 cg2

(2) system] BPOCstation 1, 2, 20, 40, BIN4 kernel일 때

이 때의 color graphic library는 cg8

(3) 만약, cg2를 지원하는 시스템이라면, .cshrc에 다음과 같이 정의.

```
setenv LD_LIBRARY_PATH /opt/BINLisp/B2.0.1/cg2:$DHOME/lib:/usr/lib
```

(만약 /opt/BINLisp directory하에 lib directory가 있다면,

/opt/BINLisp/lib:\$DHOME/lib:/usr/lib도 setting함.)

set path를 정의한 부분에 /usr/cc/bin이 나오도록 할 것.

(4) 만약, cg8를 지원하는 시스템이라면, .cshrc에 다음과 같이 정의.

```
setenv LD_LIBRARY_PATH /opt/BINLisp/cc2.0.1:$DHOME/lib:/usr/lib
```

set path를 정의한 부분에 /usr/cc/bin이 맨 앞에 나오도록 할 것.

(왜냐하면, sun은 libc version의 cc도 사용하기 때문)

2. Oracle V7상에서 C-Compiler V3이상을 사용할 때

(1) cg2를 지원하는 시스템의 경우에는 다음과 같이 함.

```
setenv LD_LIBRARY_PATH /opt/BINLisp/lib:$DHOME/lib:/usr/lib
```

(2) cg8를 지원하는 시스템의 경우에는 다음과 같이 함.

```
setenv LD_LIBRARY_PATH /opt/BINLisp/B2.0/cg8:$DHOME/lib:/usr/lib
```

(3) 두 경우 모두 set path에 /usr/cc/bin이 먼저 오도록 할 것.

3. 모든 경우에 /usr/cc/bin directory에 make와 cc가 있는지를 반드시 확인할 것. 만약, cc가 없다면 /opt/BINLisp/B2.0.1 혹은 /opt/BINLisp/B3.0하에 있는 cc와 symbolic link를 시켜줄 것.

```
cc) cd /usr/cc/bin ln -s /opt/BINLisp/B2.0.1/cc cc
```

4. gcc의 문제가 생기면, 먼저 밀란 c program을 먼저 test하는 것이 좋을 것임.

5. SUN C-Compiler version 3.0은 Oracle 7.0.1B, 4에서도 지원됨.

ORACLE DATABASE 2| AUTOMATIC STARTUP/ SHUTDOWN

Bulletin no : 10145

Introduction

This bulletin addresses the topic of automatic startup and shutdown of Oracle instances while a UNIX system is being brought up or down respectively.

The bulletin addresses specific schemes on the more popular UNIX platforms and lists the important scripts and directories that come into play in the process.

For more detailed explanations and alternatives, the reader should consult the System Administrator's Guide for the appropriate UNIX port.

Files of interest in your Oracle Installation

```
$ORACLE_HOME/bin/dbsstart -- startup database instance(s)
$ORACLE_HOME/bin/dbsout  -- shutdown database instance(s)
/etc/oratab      or      -- specifies instance information
/var/opt/oracle/oratab
```

Details:

This section briefly describes the Oracle startup and shutdown scripts invoked at system startup and shutdown.

The dbsstart script ensures a clean startup of database instance(s) even after a system failure. dbsout ensures a clean shutdown for database instance(s).

The /etc/oratab file contains a field that specifies whether a particular database instance should be brought up/down at system startup/shutdown time. By specifying 'Y' in this field, the dbsstart and dbsout scripts bring this database instance up/down.

System V v/s BSD UNIX

This section describes the generic approach taken by System V and BSD based UNIX operating systems at system startup and shutdown time.

For more details, the individual section for a particular operating system should be referred to.

System V based systems have their initialization scripts in the directory named as `/etc/rc.d` where `n` is the run-level of the script. Usually, run-level 0 signifies power shutdown mode while run-level of 2 signifies multi-user mode.

The initialization scripts in these directories are named using the following method:

```
[KIB][two-digit number][descriptive filename]
```

"B75cron" and "K30top" are examples of scripts that are named using the above convention.

The scripts whose name starts with an 'B' are called at startup, while the scripts whose name starts with a 'K' are called at shutdown time.

Also, note that the digits in the name have some importance associated with them. A script with a higher number in its name is executed later in the series of scripts being executed.

Thus, usually, oracle scripts are named something like 'BBBoracle', 'K01oracle' etc. This makes a lot of sense since we want the Oracle startup to happen *after* all the system stuff has been started. On the other hand, we want to shutdown Oracle *before* everything else on the system goes down.

BSD based systems, on the other hand, use the files `/etc/rc*`, such as `/etc/rc`, `/etc/rc.local` etc. at system startup time.

System shutdown in BSD based systems is very simple as well. Usually, no system or system defined scripts are invoked before the command `/etc/shutdown` is invoked.

Some implementations invoke `/etc/rc.shutdown` at shutdown time.

Platform Specific Details

Following are some details for implementing automatic startup and shutdown of Oracle instances on six major flavors of UNIX.

Again, for more details, please refer to your System Administrator's Guide or the on-line man pages.

(1) SunOS 4.1.3

Important Files

/etc/rc.local - The startup script for the system

Details

On a SunOS system, the startup and shutdown procedures are very simple. At startup time, the script /etc/rc.local is executed. Thus, an entry such as:

```
su -c <${ORACLE_HOME}/bin/dhstart
```

would be needed to startup the Oracle instance(s).

There is no script that is executed at shutdown time in SunOS. Thus, the only way to invoke 'dhshut' at system shutdown time is to either do it manually or to write a shell script such as:

```
#!/bin/csh
su -c <${ORACLE_HOME}/bin/dhshut
/etc/shutdown $*
```

which invokes Oracle's dhshut script before invoking the /etc/shutdown executable. Of course, the above script MUST be owned by root and be executable only by root as well.

NOTE: If the system is shutdown without shutting down the database, restarting the instance again after powering back will fail. In this case you must delete the BOA files in \${ORACLE_HOME}/dbs and restart the instance.

NOTE: The two 'su' statements in the above section have \${ORACLE_HOME} inside angled brackets <>. This is to point out that the expanded form of \${ORACLE_HOME} should be written in those lines since the value of \${ORACLE_HOME} might not be set for the supervisor at that point.

(2) DBF/1

Important Files

/etc/initdb - Controls the initialization process
/sbin/rc3 - Script to start/stop processes at run level 3
/sbin/rc0 - Script to start/stop processes at run level 0

```
/sbin/rc3.d/S88orac1n -- link to the script /sbin/init.d/orac1n
/sbin/rc0.d/E01orac1n -- same as above
```

Details

The `/etc/init.d` contains instructions for the init process. It tells it when to create and run initialization processes. One of the entries in the `init.d` file is:

```
m3:3:multi:/sbin/rc3 < /dev/console > /dev/console 2>&1
```

which involves the script `/sbin/rc3` and waits for it to finish. The script is invoked when the initialization is at run level 3 (multiuser state).

Now, `/sbin/rc3` involves scripts that are in the `/sbin/rc3.d` directory.

If the name of the script starts with a 'E', it starts the script with a 'stop' parameter, thereby stopping a process identified by the script.

On the other hand, if the script name begins with a 'S', `/sbin/rc3` provides it with a runline parameter of 'start', thereby starting processes identified by that particular script.

Thus, the `/sbin/rc3.d/S88orac1n` is an oracle script that will be kicked off by `/sbin/rc3` and get a runline parameter of 'start'. It is this script that in turn involves the SQL*Net V1 and V2 listeners, if installed and starts up the script `$ORACLE_HOME/bin/lsnstart`.

The shutdown process works in a very similar fashion. The entry in `/etc/init.d`, for this purpose is:

```
m0:0:multi:/sbin/rc0 off < /dev/console > /dev/console 2>&1
```

This kicks off the script `/sbin/rc0`. `/sbin/rc0`, in turn, calls all scripts in `/sbin/rc0.d` directory providing with a runline parameter of 'start' if their name begins with a 'S', or with a runline parameter of 'stop', if their name begins with a 'E'.

Thus, the script `/sbin/rc0.d/E01orac1n` gets invoked with a parameter of 'stop'. This script stops the the SQL*Net V1 and V2 listeners and involves the script `$ORACLE_HOME/bin/lsnshut`.

Note that both `/sbin/rc3.d/S88orac1n` and `/sbin/rc0.d/E01orac1n` are both symbolic links to the script `/sbin/init.d/orac1n`. They get invoked with a 'start' and 'stop' parameter respectively which returns controls the behaviour of the script.

NOTE: The links to `/sbin/init.d/orac1n` might not be correctly established.
There may be more links than these 2, forcing the startup or shutdown to happen more than once.

Important Files

```
/etc/initdb          -- Controls the initialization process
/etc/rc2             -- Script to start/stop processes at run level 2
/etc/rc0             -- Script to start/stop processes at run level 0

/etc/rc2.d/888dbstart -- Link to the script /etc/init.d/dbstart
/etc/rc0.d/E01dbshut  -- Link to the script /etc/init.d/dbshut

/var/opt/oracle/oratab -- Oracle oratab file
```

Details

The process of startup and shutdown is very similar to that in DBF/1. The only difference being that all the appropriate scripts and directories can be accessed from /etc instead of /sbin. Actually, the directories, such as rc0.d, rc2.d etc, are physically located in /etc. The scripts, such as rc0, rc2 etc, are symbolic links to corresponding scripts in /sbin. Also, note that the scripts /etc/init.d/dbstart and /etc/init.d/dbshut are very simple one scripts containing the following commands respectively:

```
su -c <${ORACLE_HOME}/bin/dbstart
and
su -c <${ORACLE_HOME}/bin/dbshut
```

In the commands above, <\${ORACLE_HOME}> should be replaced by the actual physical path that \${ORACLE_HOME} points to, and should be replaced by the actual user id who owns that Oracle installation.

In Solaris, unlike DBF/1, the "start" and "stop" parameters that the system passes to the scripts are ignored by the /etc/init.d/dbstart and /etc/init.d/dbshut.

NOTE: Please make sure that both dbstart and dbshut in /etc/init.d are executable by the oracle user id.

For more details of the initialization process, please refer to /etc/init.d/README.

(4) AIX

Important Files

/etc/initab — System initialization file
/etc/rc — System initialization script
/etc/mktab — Utility to make an entry in /etc/initab
/etc/shutdown — System shutdown script

Details

There are many ways to incorporate startup of Oracle instances in the general scheme of system startup. On the other hand, system shutdown in AIX is a very straight forward process.

The user can choose one of the following methods for startup:

1. Use /etc/mktab to make an entry in /etc/initab :

The command:

```
/etc/mktab "2:sm:lt:/bin/mu -c  
<${ORACLE_HOME}/bin/lsstart > /dev/console 2>&1"
```

can be used to introduce lsstart in the /etc/initab file. Thus, on system startup, at run level 2, \${ORACLE_HOME}/bin/lsstart will be invoked by the system as the Oracle owner.

2. This method is essentially the same as (1) above, but with one level of indirection.

Use /mktab as follows:

```
/etc/mktab "rc:oracle:2:sm:lt:/etc/rc.oracle > /dev/console 2>&1"
```

Also create a script /etc/rc.oracle, owned and executable only by root.

The contents of this script should be:

```
#!/bin/sh  
#  
# Startup the oracle database  
#  
mu </bin/lsstart  
EOF
```

3. Make the following entry at the end of the startup script /etc/rc:

```
/bin/mu -c <${ORACLE_HOME}/bin/lsstart
```

The system shutdown process in AIX just involves the script /etc/shutdown. Thus, besides the option of manual shutdown, there is only option left to the user to automate shutdown of Oracle instances prior to system shutdown.

The user could write a shutdown script of the one that first shuts down the Oracle instances and then invokes the system shutdown script. The sample script could be as follows:

```
#!/bin/sh

/bin/su -c /bin/dbsdshut
/etc/shutdown &*
```

Please note that the above script should be owned by root and be executable only by root as well. Changing the system shutdown script is NOT recommended by Oracle.

NOTE: In all the above cases, should be replaced by the wordid of the owner of the Oracle installation.
<ORACLE_HOME> should be replaced by the actual path pointed to by ORACLE_HOME.

(S) HP/UX

Important Files

```
/etc/rc          -- System startup file
/etc/shutdown    -- System shutdown executable
/etc/shutdown.d  -- System shutdown directory
```

Details

Unlike Solaris and OSF/1, the HP/UX system startup and shutdown are relatively straight forward procedures.

At system startup, the script /etc/rc is executed. Therefore, to provide automatic startup for Oracle, simply put the following lines just before the logical end of the script:

```
su -c <ORACLE_HOME>/bin/dbsstart
```

System shutdown is brought about by running the binary /etc/shutdown. This binary first executes all scripts and executables in the directory /etc/shutdown.d and then goes on its merry way to bring the system down normally.

Thus, to bring the Oracle instances down before the system is halted, you can do one of the following:

1. Copy the script `$ORACLE_HOME/bin/dhshut` to `/etc/shutdown.d`.
2. Make `/etc/shutdown.d/dhshut` a symbolic link to `$ORACLE_HOME/bin/dhshut`.
3. Write a small script such as:

```
#!/bin/sh
```

```
su -c <$ORACLE_HOME>/bin/dhshut
```

In this case, make sure that the script is owned and executable only by the superuser.

If you are running Oracle instances from both versions 8 and 9, you should use the `dhshut` script that comes with Oracle version 9.

NOTE: In the above section, `<$ORACLE_HOME>` should be replaced by the actual path pointed to by `$ORACLE_HOME`, and should be replaced by the userid who owns the Oracle installation.

BSD UNIX

Important Files

```

/etc/initdb          -- Controls the initialization process
/etc/rc2             -- Script to start/stop processes at run level 2
/etc/rc0             -- Script to start/stop processes at run level 0

/etc/rc2.d/S99dhstart -- Link to the script /etc/init.d/dhstart
/etc/rc0.d/E01dhshut  -- Link to the script /etc/init.d/dhshut

/etc/oratab          -- Oracle oratab file

```

Details

Please refer to the section for Solaris 2.3. The implementation details are very similar to those.

ORACLE 에서 CRON 사용

Bulletin no : 10148

The objective of this bulletin is to acquaint the reader with the use of cron and to provide some examples on how it is used with ORACLE. We will use the Unix att universe for the purpose of discussion. Differences between att and bsd are documented at the end of this report.

For simplicity, let us assume that the database is owned by the account 'oracle' and that the demo account is called 'scott'. Let us also assume that \$ORACLE_HOME is located at /usr/oracle and the SID is B.

What is cron?. Cron is a clock daemon utility provided by unix. This utility, which is located in /etc/cron executes commands specified in the file /usr/lib/crontab at times specified in crontab by the user. cron checks this file EVERY MINUTE.

Not all users are allowed to use cron. In order to use it, your userid must be listed in a file called /usr/lib/cron/cron.allow which is owned by root. Contact your system administrator to get your userid in this file. From now on, let us assume that the file cron.allow only contains userid root and oracle.

Every user listed in cron.allow will have his/her personal crontab file. Each user can only have one crontab file. This crontab will list all the commands that the user wants to execute and what times thereof.

How to set up your own crontab file

All crontab file contains lines in the following format:

* * * * *

where the *'s (often called the five time fields) reference digits that collectively make up the time, and reference the command to be executed. The five time fields in successive order are:

Field	Range
minute	0-59
hour	0-23
day of the month	1-31
month of the year	1-12
day of the week	0-6 (0 is Sunday)

You may directly use * for any one of these values which means that all possible values are accepted. For example, to execute an echo command everyday at 7:28 in the evening you can set up the following file:

```
28 18 * * * echo its dinner time.
```

You can enter more requests on consecutive lines. Once you have entered all these into a file (say, mycron) using your favorite editor, you can now submit this file as YOUR CRONTAB file to the system. To do this type:

```
crontab mycron
```

You can always add more requests in your file mycron and then resubmit it using the above command. The system will only retain your most recent submission.

To view your current submitted file type:

```
crontab -l
```

To delete your current submitted file type:

```
crontab -r
```

Your Results

Now that you have submitted your crontab file, the echo command will execute everyday at 7:30 in the evening. The results are sent to you via unix mail.

Uses of Cron

As you can see, cron is very useful. System administrators use cron often to perform administrative chores automatically. Typically such activities are performed at night when the system load is low. From a database standpoint, we can use cron to perform backups of the database every night or we can run sql reports that will spool output files every night to keep track of system usage.

Scenario

Let us set up a very simple example - let us say that we wish to know the contents of the emp and dept tables in the scott account everyday at 11:30 at night and save the results in a file called scottout. In order to do this, we will need 3 files:

```
sql script containing the queries      - call this scottcron.sql
a shell script to execute scottcron.sql - call this scottshell
a crontab file to run the shell script - call this mycron
(third last file has to be submitted to cron)
```

Let us also say that these files and the results will be located in the directory /user/oracle/personal.

Step 1 - Writing the sql script

```
sqlplus /user/oracle/personal/scottout
select * from emp;
select * from dept;
exit;
```

Now we need to set up a shell script that will enter sqlplus as scott/tiger and execute scottorom.sql. We also need to write the cronsh file. Let us approach this from two angles:

Approach A - Executing as ROOT

In this approach we will let the ROOT execute the commands and save it in /user/oracle/personal. We can set up the shell file scottshell as follows:

```
sqlplus scott/tiger @/user/oracle/personal/scottorom
```

and the cronsh file as

```
30 23 * * * su - oracle -c "/user/oracle/personal/scottshell" >
"/user/oracle/personal/scotterror"
```

Essentially, root is the super user and can thus impersonate oracle without a password. su oracle will enable root to become oracle. The -c option 'execute the following command as oracle'. The extra '-' between su and oracle is crucial - because of this, the script is being executed as if the user oracle had logged in himself. In other words all environment variables like \$ORACLE_HOME,\$ORACLE_SID and \$PATH are set correctly.

This is the approach we use at Oracle Support when maintaining our own online support system. Our cronsh file looks like the following

```
< time fields> su - oracle -c "shell1"
< time fields> su - oracle -c "shell2"
< time fields> su - oracle -c "shell3"
< time fields> su - oracle -c "shell4"
< time fields> su - oracle -c "shell5"
```

Approach B - Executing as oracle

In this approach, oracle will submit the cronsh file instead of root. The shell file scottshell will now look like:

```
ORACLE_HOME=/user/oracle; export ORACLE_HOME;
```

```
ORACLE_SID=S: export ORACLE_SID:
/user/oracle/bin/sqlplus scott/tiger @/user/oracle/personal/scottcron and the cronsh file
will look like:
```

```
30 23 * * * "/user/oracle/personal/scottshell" >
"/user/oracle/personal/scottcron"
```

A common mistake is to forget to set ORACLE_HOME and ORACLE_SID - this usually yields to a DBA 2700 error. This is not a bug - when cron runs the script it uses the environment similar to the one used by root when it does an 'su' to oracle WITHOUT executing the login scripts of oracle. Thus the ORACLE variables are not set. As a matter of fact, you can also set the PATH explicitly to avoid using full path names.

We can do an export or run a report in the same way.

Differences with BSD Environment.

In the ATT environment, we could never edit the cronsh file directly - we had to keep resubmitting our personal cronfiles. In BSD versions, there is only one configuration file called /usr/lib/cronsh. Version 4.3 also has a cronsh.local. You may either use one or the other, not both.

Also on some systems, the cronsh files are set up in a seven field format. The first 5 fields are the time fields, the sixth is the user id and seventh is the command to be executed.

If you are not using cron, be sure to remove your cronsh file - otherwise the system will keep executing it forever!!!

UNIX 에서 ORACLE PERMISSION 문제 - ORA-1034 , ORA-7320,7429

Bulletin no : 10150

문제 설명

ORACLE RDB 사용자만 데이터베이스를 ACCESS할수 있고 다른 사용자는 BDL*PLUE 등을 통하여 CONNECT
를 하려고 할때 다음 에러가 발생 할경우가 있다.

ORA-01034, "ORACLE not available"

ORA-07320, "ksgeget: short error when trying to attach sga."

// *Cause: Unable to attach ksgeget.

// *Action: Check ssga, ssgaconn[0] returns ksgeget id. Verify ksgeget

// exists and that permissions are correct.

UNIX System V Error 13 /* Permission denied */

or:

ORA-01034, "ORACLE not available"

ORA-07429, "ksgeget: ksgeget() failed to get ksgeget."

// *Cause: A shared memory segment used for all part of the RDB could
// not be retrieved.

// *Action: Use the system error number in the error message to
// determine

// why the segment could not be retrieved. If it does not
// exist,

// shutdown the database using the "abort" option, and then

// restart it. If the get failed because the permissions are

// incorrect, make sure that the ownership of the oracle

// executable is the same as that on the shared memory
// segment.

해결 방법

=====

Check for these permission and ownership problems:

- o Check the "oracle" and "oracsv" executables. (You may not have an "oracsv" executable; you may have "tuxclsv" instead.

"oracle" and "oracsv" should have the following permissions (be careful):

```
-----x 1 ucupport dha 7330847 Oct 20 11:01 oracle*
-----x 1 root dha 82008 Dec 31 1989 oracsv*
```

If they are not:

```
% chmod 8751 oracle
% chmod 4751 oracsv
```

- o Verify that the filesystem is not mounted as not ufs. Check /etc/fstab for "nfsuid". For example:

```
/dev/dsk/c3d0e3 /dev/dsk/c3d0e3 /usr4 ufs - yes no,nfsuid
```

- o Check the "umask" of the oracle user. If Oracle is installed with any "umask" other than "022", such as "077", the directory and file permissions will be set incorrectly. To see the current umask, type this at the oracle user's UNIX prompt:

```
% umask
```

You should see:

```
022 or 22
```

- o Check the ownership of the executables in \$ORACLE_HOME/bin. All files in the oracle installation must be owned by the oracle user except the following, which when they are present must be owned by root:

```
-----x 1 root dha 7332 Jul 12 08:47 gateway
-----x 1 root dha 3348384 Jun 8 10:44 oagpr*
-----x 1 root dha 248 Jun 8 10:44 oagpr*
-----x 1 root dha 82008 Dec 31 1989 oracsv*
-----x 1 root dha 8478 Jun 28 08:58 osh*
```

Use the UNIX "chown" command to correct minor ownership problems or reinstall. For example:

```
% chown root osh
```

changes the owner of the executable "ock" to the "root" user:

- o Verify that all oracle files have as their group the oracle dba group.
Use "chgrp" to correct group ownership problems:

```
% chgrp dba *
```

changes the group of all files in the current directory to that of the oracle dba. However, if permissions are not correct in one directory it is likely to be an installation wide problem.

- o Check that all directories in the path of ORACLE_HOME have at least a 755 mask.
For example, if ORACLE_HOME = /usr/oracle/714, then the directories /usr /usr/oracle and /usr/oracle/714 should all have a mask of 755.
This means their permissions should be like:

```
drwxr-xr-x  35 root   sys      1024 Oct  8 18:27 usr
```

If they are not:

```
% chmod 755
```

UNIX 에서 SHARED MEMORY REQUIREMENT

Bulletin no : 10151

Problem Explanation:

=====

Oracle uses shared memory and semaphores to communicate between processes and the SGA (System Global Area). There are certain requirements for shared memory and the semaphores. When the Oracle instance comes up, it allocates a certain portion of the main memory to create the SGA. If the shared memory or the semaphores are not set properly, then it gives an error related to shared memory or semaphores.

The following are the recommended values for shared memory and semaphores for running a SMALL size Oracle database. These values are set at the Unix kernel level.

Solution Description:

=====

The shared memory feature of the UNIX operating system is required by Oracle.

The System Global Area (SGA) resides in shared memory; therefore, shared memory must be available to each Oracle process to address the entire SGA.

Definitions of Shared Memory and Semaphore Parameters

SHMMAX - The maximum size (in bytes) of a single shared memory segment.

SHMMIN - The minimum size (in bytes) of a single shared memory segment.

SHMMNI - The number of shared memory identifiers.

SHMSEG - The maximum number of shared memory segments that can be attached by a process.

SEMMNS - The number of semaphores in the system.

SEMMNI - The number of semaphore set identifiers in the system; determines the number of semaphore sets that can be created at any one time. SEMMNL - The maximum number of semaphores that can be in one semaphore set.

It should be same size as maximum number of Oracle processes (The PROCESSES parameter in the init.ora file).

Recommended Semaphore and Shared Memory Parameters

Operating System	Shared Memory Parameters	Semaphore
Bus DB	BHMBLZE = 32768 BHMMNI = 50	BEMHMB = 200 BEMHNI = 50
Bolaxix	BHMMAK = 8388608 BHMBEC = 20 BHMMNI=100	BEMHMB = 200 BEMHBL = 50 BEMHNI = 70
HP/UX	BHMMAK = 0x4000000 (64 MB) BHMBEC = 12	BEMHMB = 128 BEMHNI = 10
Digital Unix (DEC Alpha OSF/1)	BHMMAK = 4194304 BHMBEC = 32	BEMHMB = 80 BEMHBL = 25
Ultronix	Use system default	BEMHMB = 80 BEMHBL = 5
AT&T Unix	BHMMAK = RAM-Dependent	BEMHMB = 200
8 or 16 MB RAM	BHMMAK = 5 MB	for all RAM
32 MB RAM	BHMMAK = 8 MB	values
64 MB RAM	BHMMAK = 16 MB	
128 MB RAM	BHMMAK = 32 MB	
256 MB RAM	BHMMAK = 64 MB	
512 MB RAM	BHMMAK = 128 MB	
1024 MB RAM	BHMMAK = 256 MB	
2048 MB RAM	BHMMAK = 512 MB	
	BHMBEC = 8 for all RAM values	
	BHMMNI = 1 for all RAM values	
Syrix/PTX	BHMMAK = 11010048 BHMBEC = 20	BEMHMB = 200 BEMHBL = 85
	Other parameter: MDPFILES = 128	
OS/UX	BHMMAK = 4194304 BHMBEC = 15	BEMHMB = 200

Setting the Shared Memory and Semaphore Parameters

The values of the shared memory and semaphore parameters must be set in the kernel configuration file of your operating system (OS). The location of that file is port-specific.

To check your current shared memory and semaphore configuration you can use the command:

% sysctl | more

Example on HP-UX (relevant sections only):

Semaphore Related Parameters

maximum value for semaphore(count)= 18384
Semaphore max(count)= 4096
number of semaphore identifiers(count1) = 4096
total number of semaphore in the system(count) = 8192
number of semaphore undo structures(countu) = 1536
semaphore undo entries per process(countu) = 512
semaphore maximum value(countu) = 32767

Shared Memory Related Parameters

maximum shared memory segment size in bytes(shared) = 536870912
minimum shared memory segment size in bytes(shared) = 1
maximum shared memory segments in system (shared) = 512
maximum shared memory segments per process(shared) = 512

NOTE: The SHMMAX is quite large on this system as there are 8 instances running on this system.

To make changes in shared memory or semaphore parameters:

1. Shut down any running Oracle instances
2. Locate the kernel configuration file for your OS
3. Make the necessary changes using the system utilities or your favorite editor. System utilities include:

OS	Utility
HP-UX	BAMB
BCD	BYBAMBH
AIX	BMIT
Solaris	ADMINTOOL

4. Reconfigure the kernel.
5. Reboot your machine.
6. Restart your Oracle instances

Example, using Solaris 2.3/2.4 parameters and commands:

1. Log into BQLDBA and type:

```
BQLDBA> shutdown  
BQLDBA> exit
```

2. Log in as the supervisor (root) and:

```
# cd /etc/system
```

3. Add the following lines to the /etc/system file:

```
set dbconfig:dbinitfs_channel=8388608  
set dbconfig:dbinitfs_channel=1  
set dbconfig:dbinitfs_channel=100  
set dbconfig:dbinitfs_channel=20  
set dbconfig:dbinitfs_channel=200  
set dbconfig:dbinitfs_channel=70
```

4. Reconfigure the kernel:

```
# touch /reconfigure
```

5. Reboot the machine:

```
#halt &
```

6. Log into BQLDBA and type:

```
BQLDBA> startup  
BQLDBA> exit
```

Oracle, Shared Memory, and SGA Size

There are Oracle `init.ora` parameters which can be modified to influence the size of the SGA. These settings of these parameters, in conjunction with OS shared memory and swapfile parameters, can influence both system and Oracle performance.

UNIX 에서 SHARED MEMORY/SEMAPHORE 의 이해

Bulletin no : 10152

Understanding Oracle and Shared Memory/Semaphores

Shared memory and semaphores are two important resources for an Oracle instance on Unix. An instance cannot start if it is unable to allocate what it needs. This paper primarily discusses the process Oracle goes through to allocate shared memory and semaphores at instance startup. Other important points unrelated to startup as well as some troubleshooting information will be touched.

General

Shared memory is exactly that - a memory region that can shared between different processes. Oracle uses shared memory for implementing the SGA, which needs to be visible to all database processes. Shared memory is also used in the implementation of the SQL*Net V1 Fast driver as a means of communicating between the application and shadow process. On the MS/8000, each shadow process stores its PGA in a shared memory segment (however, only the shadow attaches this segment). In the latter two cases, Oracle allocates the shared memory dynamically as opposed to the allocation of the SGA, which occurs at instance startup. This allocation will not be discussed in this paper.

Semaphores can be thought of as flags (hence their name, semaphores).

They are either on or off. A process can turn on the flag or turn it off.

If the flag is already on, processes who try to turn on the flag will sleep until the flag is off. Upon awakening, the process will reattempt to turn the flag on, possibly succeeding or possibly sleeping again. Such behaviour allows semaphores to be used in implementing a post-wait driver - a system where processes can wait for events (i.e. wait on turning on a semaphore) and post events (i.e. turning of a semaphore). This mechanism is used by Oracle to maintain concurrency control over the SGA, since it is writable by all processes attached.

Also, for the same reason, user of the Fast Driver requires additional semaphores. However, these semaphores will be allocated dynamically instead of at instance startup. This allocation will not be discussed in this paper.

Instance startup

On instance startup, the first things that are done are: read the `init.ora`, start the background processes, and allocate the shared memory and SGA required. The size of the SGA will be calculated from various `init.ora` parameters; this will be the amount of shared memory required. The SGA is broken into 4 sections - the fixed portion, which is constant in size, the variable portion, which varies in size depending on `init.ora` parameters, the redo block buffer, which has its size controlled by `log_buffers`, and the db block buffer, which has its size controlled by `db_block_buffers`.

The size of the SGA is the sum of the sizes of the 4 portions.

There is unfortunately no simple formula for determining the size of the variable portion. Generally, the shared pool dominates all other parts of the variable portion, so as a rule of thumb, one can estimate the size as the value of `shared_pool_size` (in v8, one can ignore the size of the variable portion). The number of SGA segments required is much simpler to determine. Oracle will need exactly as many SGA segments as the value of the processes `init.ora` parameter.

Note that the recommended kernel parameter values in the ICG are enough to support the default database (4M SGA, 50 processes), but may be insufficient to run a larger instance. With the above estimations and the information which follows, a DBA should be able to build a kernel with appropriate settings to support the instance.

Shared memory allocation

Oracle has 3 different possible models for the SGA - oversegment, contiguous multi-segment, and non-contiguous multi-segment.

When attempting to allocate and attach shared memory for the SGA, it will attempt each one, in the above order, until one succeeds or raises an ORA error. On other, non-fatal, errors, Oracle simply cleans up and tries again using the next memory model. The entire SGA must fit into shared memory, so the total amount of shared memory allocated under any model will be equal to the size of the SGA. This calculated value will be referred to below as `SGASIZE`.

The oversegment model is the simplest and first model tried. In this model, the SGA resides in only one shared memory segment. Oracle attempts to allocate and attach one shared memory segment of size equal to total size of the SGA. However, if the `SGASIZE` is larger than the configured `SMMASK`, this will obviously fail (with `EINVAL`). In this case, the SGA will need to be placed in multiple shared memory segments, and Oracle proceeds to the next memory model for the SGA. If an error other than `EINVAL` occurs when allocating the shared memory with `sbrget()`, Oracle will raise an ORA-7308.

If the segment was gotten (i.e. if `SMMASK > SGASIZE`), Oracle attempts to attach it at the start address defined in `base.a`. An error on the attach will raise an ORA-7307. Note: `base.a`

`a` is only used in BSD-based Operating Systems such as Solaris and is not used in BSD-based Operating Systems such as Sun4 and Ultrix.

With multiple segments, there are two possibilities. The segments can be attached contiguously, so that it appears to be one large shared memory segment, or non-contiguously,

with gaps between the segments. The former wastes some space that could be used for the stack or heap, but depending on alignment requirements for shared memory (defined by SHMLBA in the kernel), it may not be possible.

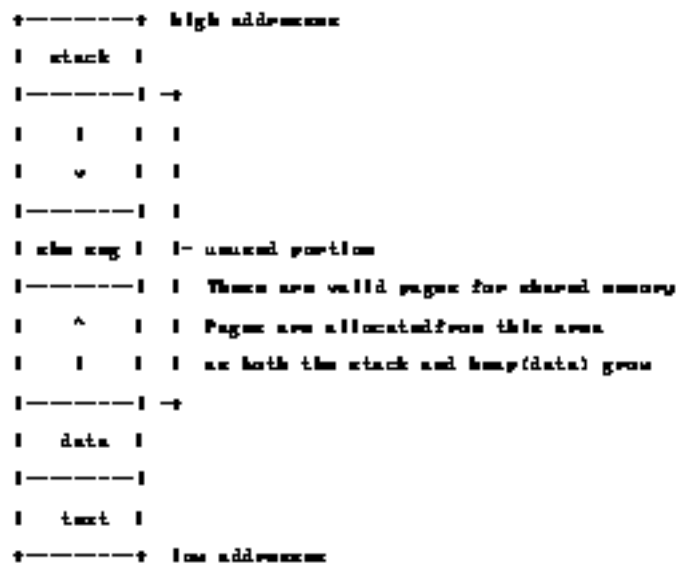
At this point, Oracle needs to determine SHMMAN. It can determine how many segments will be required. This is done via a binary search algorithm over the range [1..SCASIZE] (since Oracle is trying this model and not the one segment model it must be that SHMMAN.dkt file is used to get the necessary information. In version 7, the SCA itself contains the information about the shared memory and segments (how the bootstrap works will be explained later). In either case, the information stored is the key, id, size, and attach address of each shared memory segment and the key, id, and size of each segment set. Note that we need not do anything special to initialize the segments. We can use them with the data structure we read in on connecting.

The version 8 approach is rather simple. It first tries to open the sgadev.dkt file. If it cannot, an ORA-7318 is raised. Once opened, the data written earlier on startup is read. If an error occurs for some reason on the read, an ORA-7318 occurs. Once all the data is read in, Oracle attaches each segment in turn. First, it generates what it believes the key for the segment should be. It then gets that segment, returning ORA-7428 if it fails. The key used and the key stored are then compared. They should be equal, but if not, an ORA-7430 occurs. Once the key is verified, the segment is attached. A failure to attach the segment raises an ORA-7320. If the segment is attached, but not at the address we requested, an ORA-7321 occurs. This process is repeated for all segments until the entire SCA is attached.

Version 7 differs only in the first part, when the shared memory and segment data is read. Once that data is read in, Oracle proceeds in the same manner. To fetch this data, Oracle generates what it thinks should be the key for the first segment of the SCA and attaches it, as if it were the only segment. Once it is attached, the data is copied from the SCA. With this data, Oracle attaches any remaining segments for the SCA. There is one possible problem. If somehow two instances have a key collision (i.e. they both generate the same key for their first segment), it is possible to only have one of the two instances up at a time. Connection attempts to either one will connect a user to whichever instance is up. This is rare, but can happen. Development is currently working on a better key generation algorithm.

Attaching shared memory

As seen in previous sections, shared memory must be gotten (this may mean allocating the shared memory, but not necessarily) and then attached, to be used. Attaching shared memory brings the shared memory into the process' memory space. There are some important things about attach addresses. For one thing, they may need to be aligned on some boundary (generally defined by SHMLBA). More importantly, shared memory must map to pages in the process' memory space which are unaccounted for. Every process already has a text, a data, and a stack segment laid out as follows (in general):



So valid attach addresses lie in the unused region between the stack and the data segments (a shared memory segment is drawn in the diagram to aid in visualization - not every process has shared memory attached). Of course, the validity also depends on the size of the segment, since it cannot overlap another segment. Note that both the stack and data segments can grow during the life of a process. Because segments must be contiguous and overlapping is not allowed, this is of some importance. Attaching shared memory creates a limit on how much the stack or data segment can grow. Limiting the stack is typically not a problem, except when running deeply recursive code. Neither is limiting the data segment, but this does restrict the amount memory that can be dynamically allocated by a program. It is possible (but seldom) that some applications running against the database may hit this limit in the shadow (since the shadow has the BGA attached). This is the cause of ORA-7324 and ORA-7325 errors. How to deal with these is discussed in the troubleshooting section.

The BGA is attached, depending on the allocation model used, more or less contiguously (there may be gaps, but these can be treated as if they were part of the shared memory). So where the beginning of the BGA can be attached depends on the BGA's size. The default address which is chosen by Oracle is generally sufficient for most BGAs. However, it may be necessary to relocate the BGA for very large sizes. It may also need to be changed if ORA-7324 or ORA-7325 errors are occurring. The beginning attach address is defined in the file `base.c`. Changing the attach address requires recompilation of the Oracle kernel and should not be done without first consulting Oracle personnel. Unfortunately, there is no good way to determine what a good attach address will be. When changing the address to allow a larger BGA, a good rule of thumb is taking the default attach address in `base.c` and subtracting the size of the BGA. The validity of an attach address can be tested with the Oracle provided `tetsh` executable. Using `tetsh -t -k` will determine if the address is usable or not.

Trouble shooting

Errors which might have multiple causes are discussed in this section. Errors not mentioned here generally have only one cause which has a typically obvious solution.

ORA-7308, ORA-7338, ORA-7328

Oracle received a system error on a `shgset()` call. The system error should be reported. There are a few possibilities:

1) There is insufficient shared memory available. This is indicated by the operating system error `ENOMEM`. Most likely, `SHMMNI` is too small. Alternatively, there may shared memory already allocated; if it is not attached, perhaps it can be freed. Maybe shared memory isn't configured in the kernel.

2) There is insufficient memory available. Remember, shared memory needs pages of virtual memory. The system error `ENOMEM` indicates there is insufficient virtual memory. Swap needs to be increased, either by adding more or by freeing currently used swap (i.e. free other shared memory, kill other processes)

3) The size of the shared memory segment requested is invalid. In this case, `EINVAL` is returned by the system. This should be very rare -however, it is possible. This can occur if `SHMMAX` is not a multiple of page size and Oracle is trying a multi-segment model. Remember that Oracle rounds its calculation of `SHMMAX` to a page boundary, so it may have rounded it up past the real `SHMMAX` (Whether this is a bug is debatable).

4) The shared memory segment does not exist. This would be indicated by the system error `ENOENT`. This would never happen on startup; it only would happen on connect. The shared memory most likely has been removed unexpectedly by someone or the instance is down.

ORA-7307, ORA-7337, ORA-7320

Oracle received a system error on a `shmat()` call. The system should be reported. There are a few possibilities:

1) The attach address is bad. If this is the cause, `EINVAL` is returned by the system. Refer to the section on the attach address to see why the attach address might be bad. This may happen after enlarging the SGA.

2) The permissions on the segment do not allow the process to attach it. The operating system error will be `EACCESS`. Generally the cause of this is either the setuid bit is not turned on for the oracle executable, or root started the database (and happens to own the shared memory). Normally, this would be seen only on connect.

3) The process cannot attach any more shared memory segments. This would be accompanied by the system error `ENFILE`. `SHMSEG` is too small. Note that as long as `SHMSEG` is greater than `BS_SEG_MAX`, you should never see this happen.

ORA-7328, ORA-7334

Oracle has determined the SGA needs too many shared memory segments. Since you can't change the limit on the number of segments, you should instead increase SHMMAX so that fewer segments are required.

ORA-7338

Oracle has determined it needs too many semaphore sets. Since you can't change the limit on the number of semaphore sets, you should increase SEMBL so fewer sets are required.

ORA-7250, ORA-7278, ORA-7252

Oracle received a system error on a semget() call. The system error should be reported. There should be only one system error ever returned with this, ENOSPC. This can come one of two things. Either the system limit on semaphore sets has been reached or the system limit on the total number of semaphores has been reached. Raise SEMNMI or SEMNMB, as is appropriate, or perhaps there are some semaphore sets which can be released. In the case of ORA-7250, ORANSMBN may be set too high (>SEMBL). If it is, raise SEMBL or decrease ORANSMBN.

ORA-7251

Oracle failed to allocate even a semaphore set of only one semaphore. It is likely that semaphores are not configured in the kernel.

ORA-7318

Oracle could not open the sgadev file. The system error number will be returned. There are a few possible causes:

1) The file doesn't exist. In this case, the system error ENOENT is returned. Maybe ORACLE_SID or ORACLE_HOME is set wrong so that Oracle is looking in the wrong place. Possibly the file does not exist (in this case, a restart is necessary to allow connections again).

2) The file can't be accessed for reading. The operating system error returned with this is EACCESS. The permissions on the file (or maybe directories) don't allow us open for reading of the sgadev file. It might not be owned by the oracle owner. The setuid bit might not be turned on for the oracle executable.

ORA-7319

Oracle did not find all the data it expected when reading the sgadev.dbf file. Most likely the file has been truncated. The only recovery is to restart the instance.

ORA-7430

Oracle expected a key to be used for the segment which does not match the key stored in the shared memory and semaphore data structures. This probably indicates a corruption of the sgadev file (in version 8) or the data in the first segment of the SGA (in version 7). A restart of the instance is probably necessary to recover in that case. It may also be a key collision problem and Oracle is attached to the wrong instance.

ORA-7321

Oracle was able to attach the segment, but not at the address it requested. In most cases, this would be caused by corrupted data in the xgdaffile (in version 8) or the first segment of the BGA (in version 7). A restart of the database may be necessary to recover.

ORA-7324, ORA-7325

Oracle was unable to allocate memory. Most likely, the heap (data segment) has grown into the bottom of the BGA. Relocating the BGA to a higher attach address may help, but there may be other causes. Memory leaks can cause this error. The init.ora parameter sort_area_size may be too large, decreasing it may resolve the error. The init.ora parameter context_size may also be too large, decreasing it may resolve this.

ORA-7284, ORA-7285

Oracle was unable to decrement/increment a semaphore. This generally is accompanied by the system error EINVAL and a number which is the identifier of the semaphore set. This is almost always because the semaphore set was removed, but the shadow process was not aware of it (generally due to a shadow abort or instance crash). This error is usually ignorable.

SystemParameters

BHMMAX - kernel parameter controlling maximum size of one shared memory segment

BHMMHI - kernel parameter controlling maximum number of shared memory segments in the system

BHMBEC - kernel parameter controlling maximum number of shared memory segments a process can attach

BHMMBS - kernel parameter controlling maximum number of semaphores in the system

BHMMHI - kernel parameter controlling maximum number of semaphore sets. Semaphores in Unix are allocated in sets of 1 to BHMMHI.

BHMMBL - kernel parameter controlling maximum number of semaphores in a semaphore set.

BHMLBA - kernel parameter controlling alignment of shared memory segments; all segments must be attached at multiples of this value.

Typically, non-tunable.

System errors

ENOENT - No such file or directory, system error 2

ENOMEM - Not enough core, system error 12

EACCES - Permission denied, system error number 13

EINVAL - Invalid argument, system error number 22

ENFILE - Too many open files, system error number 24

ENOSPC - No space left on device, system error number 28

Oracle parameters

- BB_SEG_MAX** - Oracle parameter specified at compile time (therefore, unmodifiable without an Oracle patch) which defines maximum number of segments the BSA can reside in. Normally set to 20.
- BB_SEG_MAX** - Oracle parameter specified at compile time (therefore, unmodifiable without an Oracle patch) which defines maximum number of segments into oracle will allocate. Normally set to 10.

RAID 와 ORACLE의 관계 소개

Bulletin no : 10157

1. RAID – General Description

RAID (Redundant Array of Inexpensive Disks) is frequently used to provide data redundancy in case of disk failure and involves, in part, striping of data across several physical drives. RAID can be implemented in any one of several levels, ranging from 0 to 8. These levels are characterized by the following:

RAID 0: This is a 'normal' file system, in which there will be data loss with any disk failure.

This level provides good performance and speed of access but no data redundancy. If something on the disk becomes corrupted, there is no way to restore or recalculate that data.

RAID 1: Full data redundancy is provided by complete mirroring of all files.

This method requires twice as much disk space as there is data.

No error correction codes (ECC's) are used, and there is hence no parity information involved.

RAID 2: Redundancy is provided through ECC's (also sometimes called parity), which are calculated for each 'chunk' of data. These ECC's can be used to recreate the data in question and so this method requires less disk space than RAID level 1. However, the need to calculate ECC's will generally make writes slower.

RAID 3: ECC's are again calculated to provide data redundancy, but all the parity bytes are stored on one physical disk with bit level parity.

RAID 4: The same as RAID level 3 but with block level parity.

RAID 5: Redundancy is provided by ECC's as in RAID 3 and 4, but in this case, parity information is stored along with the data and is hence striped across several physical disks. RAID 5 should not be used for write-intensive files, since the continuous calculation of ECC's makes writing data to disk slower. RAID 3 should be used instead. RAID 5 works best for read-only data files.

RAID 6: ECC's are calculated in a more complex way, and redundancy is provided with an advanced 2-dimensional parity.

Raid levels 1 and 5 are most commonly used. However, the choice of which specific RAID level to choose and how to implement that choice can be a complex question which must balance

performance, cost, and data availability issues.

2. Pro's and Cons of Implementing RAID technology

There are benefits and disadvantages to using RAID, and these depend on the RAID level under consideration and the specific system in question.

In general, RAID level 1 is most useful for systems where complete redundancy of data is a must and disk space is not an issue. For large datafiles or systems with less disk space, this RAID level may not be feasible. Writes under this level of RAID are as fast as and as slower than 'usual'. For all other levels of RAID, writes will tend to be slower and reads will be faster than under 'normal' file systems. Writes will be slower the more frequently ECC's are calculated and the more complex those ECC's are.

Depending on the ratio of reads to writes in your system, I/O speed may have a net increase or a net decrease. RAID can improve performance by distributing I/O, however, since the RAID controller spreads data over several physical drives and therefore no single drive is overburdened.

The striping of data across physical drives has several consequences besides balancing I/O. One additional advantage is that logical files may be created which are larger than the maximum size usually supported by an operating system. There are disadvantages, as well, however. Striping means that it is no longer possible to locate a single datafile on a specific physical drive.

This may cause the loss of some application tuning capabilities. Also, in Oracle's case, it can cause database recovery to be more time-consuming. If a single physical disk in a RAID array needs recovery, all the disks which are part of that logical RAID device must be involved in the recovery.

One additional note is that the storage of ECC's may require up to 20% more disk space than would storage of data alone, so there is some disk overhead involved with usage of RAID.

3. RAID and Oracle

The usage of RAID is transparent to Oracle. All the features specific to RAID configuration are handled by the operating system and go on behind-the-scenes as far as Oracle is concerned. Different Oracle file-types are suited differently for RAID devices. Datafiles and archive logs can be placed on RAID devices, since they are accessed randomly. Redo logs should be not be put on RAID devices, since they are accessed sequentially and performance is enhanced in their case by having the disk drive head near the last write location. However, mirroring of redo log files is strongly recommended by Oracle.

References:

1. Oracle for Unix Performance Tuning Tips pp. 28-27 (A22535-1)
2. RAID: High-Performance, Reliable Secondary Storage (Chen, Peter et al, 1984)
3. Personal Communications with Oracle Analysts COMET and GABPOB.

UNIX 에서 사용되는 ORACLE 환경 변수

Bulletin no : 10181

The document describes each Oracle environment variable with its name, its use, general example values, and specific examples in the following manner:

ENVIRONMENT VARIABLE NAME

Use: What is this environment variable?

General: What are some example values?

Example: Specific example

In addition, UNIX environment variables that affect Oracle are briefly described.

General Notes

=====

1. \$D_H == \$ORACLE_HOME
2. \$D_B == \$ORACLE_BID
3. An environment variable followed by a "*" applies to Oracle?
4. An environment variable followed by a "B" is used during database creation for Oracle Version B only.
5. For more information, see your IUG, ICG, or the "ORACLE? Server for UNIX Administrator's Reference Guide"
6. Note that not all UNIX platforms use all of these environment variables.

Oracle Environment Variables

=====

APIPATH*

Use: version 7.0.12 only: directory containing Tool Kit II *.exe files

General:

Example: \$D_H/oracle:\$D_H/tk2/adain

OROE_LOCALREFERENCE*

Use: Oracle's local env var

General: \$ORACLE_HOME/look/adain

Example: \$ORACLE_HOME/look/adain

BOOK_RESOURCE*

Use: directory for DocIn*Book resource files
General: $\$D_H/book/adsl/resource/US$, $\$D_H/book/adsl/resource/JA$
Example: $\$DHOME/book/adsl/resource/US$

BOOK_GLOBALPREFERENCE*

Use: DocIn*Book new var
General: $\$DHOME/book/adsl$
Example: $\$DHOME/book/adsl$

BOOK_HELP*

Use: directory for DocIn*Book help files
General: $\$DHOME/book/adsl/help/US$
Example: $\$DHOME/book/adsl/help/US$

CABE_HP_CMD

Use: Command to plot a file from CABE*Designer
General: Actual plot command or name of a script
Example: `lpr -Pplot`

CABE_PS_CMD

Use: Command to print postscript file from CABE*Designer
General: Actual print command or name of a script
Example: `lpr -P`

CABE_RESOURCE

Use: Tool Kit I terminal type for CABE
General: filename (file from $\$D_H/dict50/adsl/etc$) and
(device from Devtree)
Example: `cabe_hpxterm`

CABE_BPPMINT

Use: to print CABE*Designer screen prints to file or PostScript
printer
General: printer name
Example: `lu`

CABE_XTERM

Use: Tool Kit I GUI window type for CABE
General: `xterm`, `lxterm`, `slxterm`, `dxterm`, etc.
Example: `xterm`

CGEN_HOME

Use: home directory of CABE Generator
General:
Example: $\$D_H/cgen20$

DEBUG_SLFIND:

Use: Debug for Y22 tools

General: 0,1,?

Example: 1

DBG_FILE:

Use: Initial database file name

General: \$D_H/dhc/dhc\$D_B.dbf will hard code db file (you will not be able to move \$D_H); setting to dhc\$D_B.dbf will allow you to move \$D_H

Example: dhcsrcin.dbf

DBG_SIZE:

Use: Initial database file size

General: numberK, numberM

Example: 5000K or 5M minimum, up to partition size

FORMS_DEVICE:

Use: devices for Y22 tools (CDE tools)

General: vt220, vt100, lp, null (see \$ORACLE_HOME/tk2/admin/terminal)

Example: lp

FORMS30PATH:

Use: directory containing Forms 3.0 (Y22) resource files

General:

Example: \$ORACLE_HOME/forms30/admin/resource

LOG:

Use: file to record the install procedure

General: \$D_H/install/install.log

Example: \$D_H/install/install.log

LOG_FILE1:

Use: first redo log file name

General: \$D_H/dhc/log1\$D_B.dbf will hard code log file (you will not be able to move \$D_H); setting to log1\$D_B.dbf will allow you to move \$D_H

Example: log1srcin.dbf

LOG_FILE2:

Use: first redo log file name

General: \$D_H/dhc/log2\$D_B.dbf will hard code log file (you will not be able to move \$D_H); setting to log2\$D_B.dbf will allow you to move \$D_H

Example: log2srcin.dbf

LOG_SIZE

Use: Initial log file size
General: numberK, numberM
Example: no minimum, 500K default, no maximum (tune to your database needs)

MMUSPATH

Use: directory containing Memu 5.0 (TE 1) resource files
General:
Example: \$ORACLE_HOME/mmu5/adml/resource

NN_MERBOURCE

Use: ?
General: \$ORACLE_HOME/nn/adml/resource/US
Example: \$ORACLE_HOME/nn/adml/resource/US

NLS_LANG

Use: National Language Support language, territory, and character set
General: language_territory.characterset
Example: american_american.us7ascii

NO_MAKE

Use: set to true if your system doesn't have C dev tools (cc, make, ld, etc.)
General:
Example: false

ORACLE_BASE

Use: new directory structure for Oracle 7 to follow Oracle Flexible Architecture
General: see OGG "Preparing to Install Oracle Products"
Example: can be any directory

ORACLE_HELP

Use: directory containing help files?
General: \$ORACLE_HOME/help/adml/resource
Example: \$ORACLE_HOME/help/adml/resource

ORACLE_HOME

Use: dir containing top level Oracle directories
General: any directory with enough space
Example: /usr/oracle

DBACLE_ICONS

Use: dir containing icons for GUI tools
 General: \$DBACLE_HOME/gui/common/tk2/adwin/icons
 Example: \$DBACLE_HOME/gui/common/tk2/adwin/icons

DBACLE_LPAGES

Use: which lp arguments to use for Easy*SQL, SQL*Calc, SQL*Form, SQL*Menu, SQL*Report, or SQL*ReportEditor
 General:
 Example: -c -x

DBACLE_LPPRDS

Use: which lp command to use for Easy*SQL, SQL*Calc, SQL*Form, SQL*Menu, SQL*Report, or SQL*ReportEditor
 General: lp, lpr, print
 Example: lp

DBACLE_LPSTAT

Use: which lp status command to use
 General: lpstat, lpr
 Example: lpstat

DBACLE_OWNER

Use: UNIX Operating System username who owns the Oracle files
 General: can be any user, default is oracle
 Example: oracle

DBACLE_PAGES

Use: which UNIX pager to use (more, less, pg) in Oracle Products like SQL*Menu
 General:
 Example: more

DBACLE_PATH

Use: directory for reading and writing to and from SQL*Form, SQL*Menu 5.0, SQL*Plus
 General: can be any directory (if not set, read and write from current directory)
 Example: \$HOME/oracle

DBACLE_REBUILD

Use: For client only install
 General: Y if client only, F if there will be a local database
 Example: Y

ORACLE_BID

Uses: Oracle System Identifier
General: must begin with a letter, followed by a number or character; limited by some OS's to 4 chars
Example: v712

ORACLE_TERM

Uses: Tool Kit II use var pointing to the tk2:\$(ORACLE_TERM).exe file under \$ORACLE_HOME/rdbms/admin/terminal (tk2: stands for character toolkit2)
General: any file like tk2:\$(ORACLE_TERM).exe
Example: vt100

ORACLE_TERMINAL

Uses: directory where the Tool Kit II .exe files reside
General: \$O_H/tk2/admin/terminal
Example: \$O_H/tk2/admin/terminal

ORACLE_TRACE

Uses: allow verification/trace to be turned on and off during install (echoes everything done by sh)
General: Y does not -x; anything else is off
Example: Y

ORACLE_VERIFY

Uses: If true, run the .verify scripts during install
General: Y; anything else is off
Example: Y

ORACLE_DOC

Uses: points to directory containing on-line doc starting v7.0.18
General: to any directory containing on-line doc (file, cd)
Example: \$ORACLE_HOME/doc

ORACLEU_ARE

Uses: If set to anything, prompt for \$O_B or \$O_H when (c)oracle is invoked
General: NO or anything
Example: NO

ORACLEPATH

Uses: set to directory containing the v1 oraklit resource file
General: \$ORACLE_HOME//admin/resource
Example: \$ORACLE_HOME/force30/admin/resource

DBAHAIL_EDITOR

Uses: which editor to use for OracleMail

General: vi, emacs, ed

Example: vi

DBAHEMBR

Uses: number of emulators that make up a set

General: 1-UNIX maximum

Example: 80

DBAPIES4

Uses: sets the default pipe driver to v1 or v2

General: v1, v2

Example: v2

DBATEMPATH

Uses: directory containing Oracle (TE 1) resource files

General:

Example: \$ORACLE_HOME/oracle/adm/resource

PATCH_HOME

Uses: directory containing patches and patch files

General: any directory, \$O_H/patch, \$O_H/install/patch

Example: \$ORACLE_HOME/install/patch

BT_STATUS

Uses: directory for BQL4TextRetrieval V2.0 to find forms

General: can be any directory (if not set, read and write from current directory)

Example: \$HOME/oracle

BDG_HOME

Uses: directory containing top level CAGE Dictionary directories

General: can be any directory containing top level CAGE directories

Example: \$O_H/dict50/adm

BDG_PRINT

Uses: for CAGE: set to UNIX print command to use

General: lp, lpr

Example: lp

BDG_UPRINT

Uses: for CAGE: set to UNIX print command to use for wide

General: lp, lpr

Example: lp

BQLPATH

Use: now superseded by \$ORACLE_PATH
General: can be any directory (if not set, read and write from
 current directory)
Example: \$HOME/oracle

BQLTB_STATUS

Use: directory for BQL*Text*Retrieve V1.1 to find forms
General: can be any directory (if not set, read and write from
 current directory)
Example: \$HOME/oracle

BWJ_TMP

Use: directory to use for BQL*Report*Writer temporary files
General: any directory with >10M of disk space
Example: /usr/tmp

TE2DEU4

Use: version 7.0.12 only: Tool Kit II terminal resource file
General: vt100 portion from tk2_vt100.rmc
Example: vt100

TNB_ADMIN4

Use: directory containing BQL*Net v2 files
General: any dir (can usual for order followed--look at /etc, \$HOME, and \$TNB_ADMIN
Example: \$D_H/network/adale

TND_TABE

Use: hoststring to use for BQL*Net v1
General: p:, f:, x:host:\$D_B, d:host:\$D_B, xtr:host:\$D_B,
 t:host:\$D_B, tt:host:\$D_B, x:host:\$D_B
Example: t:host:oracle

UNIX Environment Variables Affecting Oracle

=====

ADA_PATH set to directory containing Ada compiler

ALSYCOMP_BIN set to directory containing Ada compiler

DISPLAY

Use: Tell K-based tools which Display to use
General: 'hostname':0.0
Example: fubar:0.0

HOME \$HOME

LANG en_US, de_DE

LANGUAGE american_english

LDOPTS ld options for compiling

LDPATH directory containing shared object libraries

LD_LIBRARY_PATH
 directory containing shared object libraries

LOG when set to a filename, log information from install
 session

LOGNAME 'logname' or 'whoami'

MALLOCTYPE 3.1, 3.2 for telling AIX which malloc type to use

MIBPATH DB directory containing language message files

NONAMESERVER
 Use: For tcp/ip networks without a named server
 General: 1 for no named server, 0 for named server
 Example: 1

PATH \$PATH

PRINTER

SHELL /bin/csh or /bin/sh

TERM any valid term: xterm, left, hp, mac2, xms, xterm, vtxxx

THPDIR some UNIX boxes allow /tmp to be renamed to \$THPDIR
 (some directory)

TZ sets local time zone

RESOURCEMENT set to file DeskIt, or other K11 resource file

RAPPLIBDIR set to dir containing resource specifications file
 DeskIt, or other K11 resource files

CORE DUMP TRACING 방법

Bulletin no : 10189

ORACLE product 사용중 발생하는 core dump를 tracing하는 방법은 아래와 같다.

1. 준비사항

- (1) Product의 Full version number, ORDB, PL/SQL(if used), 그리고 관련된 Products 예를들면, Forms과 Menu가 link되어 있다면, menu의 version도 필요
- (2) core dump를 야기시킨 정확한 command
예를들면, sqlform30...

2. Tracing 순서

- (1) core dump file의 위치확인 (일반적으로, current working directory)
- (2) log on as oracle 후 cd \$ORACLE_HOME/bin
- (3) \$ file program (where, program = core를 야 시킨 command, 예를들면,
sqlform30, sqlplus, runsp...)

이 결과를 있는그대로 적어준다.

여기에서, 결과가 'dynamic' or 'dynamically linked'등이 나오면, 반드시 당 executable을 relink 해야 함. 왜냐하면, Oracle은 현재 dynamic linking을 support하지 않으므로. 또한, relinking후 core를 다시 생성하여야 한다. version도 다시 확인할것.

- (4) \$ chmod to program
- (5) log out 후 core dump를 야기했던 user로 log on하고 해당 unix version 에 따라서, 아래에서 선택.

Command	Note	Exit command
dhx		quit
xdk	(HP-UX only)	quit
cdk		q
adk		Ctrl-d

core dump가 발생했던 directory 로 이동후

```
dhx $ORACLE_HOME/bin/program core 2>&1 ltr /tmp/mystack  
(in Bourne or Korn shell)  
dhx $ORACLE_HOME/bin/program core ltr /tmp/mystack  
(in C shell)
```

- (8) 상기의 결과가 `unix prompt`를 바로 `return`하면, `parallelism`이나 `file type`에 문제가 있는 것임. 간혹, `unix`에 따라서 `core file`의 `limit`을 주는 경우가 있는데 만일 이것이 너무 작으면, `core file`이 출력이 없을 수 있음.

`core information`을 얻은 후 `prompt`가 나오면, 아래의 `command`실행.

Command	Enter
dhx	where
adb	t
adb	t
adb	bc

- (9) 상기의 처리과정에서, `/tmp/objstack0`와 `stack trace file`이 생성되, (5)항에서 적당한 `exit command`를 참고하여 `exit`한다.

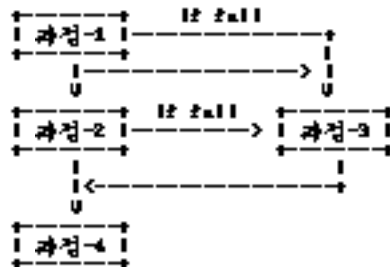
- (10) `stack trace file`을 `print`하여 해당개발부서(문서 or RMB dept.)에 `reporting`한다.

UNIX SVR4 에서 O/S의 REBUILD

Bulletin no : 101B1

요즘 표준화 방향으로 자리가 잡힌 System V Release 4(BUR4) Unix System은 AT&T 3.2, BSD4, BSD 3.4, KERNIX 등을 근간으로 하여 system이 구성되었고, 이 BUR4를 근간으로 여러 H/W vendor가 자신의 H/W에 맞게 각각의 Unix System을 만들었다. 이 과정에서 각 H/W vendor는 자신들이 공급하던 이전 System에 호환성이 있게 system을 만들기도하고, 또는 BSD4를 거의 수정없이 자신의 H/W에 porting하기도 하였다.

다음의 내용은 Unix Lab.에서 개발한 Original BUR4에서 System Parameter를 수정 하고 이를 system에 적용시는 과정을 설명한다.



----- (과정-1) -----

sysdef ----> 실패, 또는 정보가 부족할 경우 (과정-3)를 참조.

```

.....
*
* IPC Shared Memory
*
524288 max shared memory segment size (BHMMAE)
    1 min shared memory segment size (BHMMIN)
    10 shared memory identifiers (BHMNI)
    10 max attached shared segments per process (BHMBSG)
.....
  
```

----- (과정-2) -----

(과정-1)에서 나온 데이터를 기반으로 system parameter를 고치면

```

# cd /etc/conf/kim
# ix
idkuid idconfg idnxtor idkinit idkuid idspace idval
idcheck idinstall idknow idkmod idkroot idtune
# ./idtune BHMNI 20 ----> 실패, 또는 정보가 부족할 경우 (과정-3)를 참조
  
```

-----<과정-3>-----

만약 `sysctl` 명령으로 `system parameter`를 알수 없는 경우, 또는 `idtool` 명령 대신 수작업으로 `system parameter`를 편집시키려 한다면, `/etc/conf/cf.d/stunm`과 `/etc/conf/cf.d/stum`의 `file` 내용을 참조하면 된다.

1) `/etc/conf/cf.d/stum` → `sysctm0` 가진원래의 값

parameter	current_value	min_value	max_value
.....			
BHMMHI	10	10	1000
.....			

2) `/etc/conf/cf.d/stum` → 수정된 내용만 포함.

```
.....
BHMMHI 20
.....
```

3) `/etc/conf/cf.d/stum`으로 정보를 참조하고, `/etc/conf/cf.d/stum`에 편집된 내용을 2)와 같이 만듦편집.

-----<과정-4>-----

D/B를 재구성하고 이를 적용시킨다.

```
# ./idbuild
# cd /
# Bshutdown -p -g0 -IB
```

위의 과정을 응용하면 DBACL에서 필요한 `system parameter`를 `System(BU64제일)`에 적용할수 있다. 위의 `shutdown` 과정에서 `/stand/unix`는 `/stand/unix.old`로 `save`되고 `/stand/unix`는 새로운 D/B로 대체된다. 그러므로 `shutdown`전에 `/stand/unix`를 다른 이름으로 복사하면 현재 상태의 D/B를 보존할수 있다.

예를들어 `/stand/unix`를 `/stand/unix.bak`로 복사하여 현재 상태의 D/B를 보존하고 `shutdown`를 진행하면 다음과 같은 결과를 얻을수 있다.

```
/stand/unix          --> 새로운 환경의 D/B
/stand/unix.old, /stand/unix.bak --> 이전 환경의 D/B
```

-----<첨부>-----

`System vendor`는 `sysctm0`를 몰라몰래(`hooking`), H/W 또는 D/B마다 다르지만, `hooking`되는 D/B를 사용자 지정할수 있는 능력을 제공한다. 예를 들면 `hooktrap`이란 `program`에 이런 능력을 제공하기도 하고, 또는 `firmware mode`를 제공함으로써 이런 능력을 제공한다(`firmware`에서 제공되는 명령을 사용하여 `hooking` D/B를 지정함). 이중 `firmware`로서 제공되는 것은 각 H/W 특성에 적합하게 전부 다른 형식, 다른 명령어를 갖는다.

아래에는 bootsetup program에 의해 booting D/B를 지정하는 BUB4 1388 version (H/U:1388 or 1488 PC, D/B:Unix BUB4)를 예로 간단하게 설명하겠다.

system booting시 다음과 같은 메시지가 출력....

Booting Unix System.....

위의 메시지가 출력시 Enter key를 친다. 다음과 같은 메시지가 출력....

Booting DB:

위의 메시지를 출력하고 System은 중지한다.

이때 다음과 같은 입력을 할수 있다.

Booting DB:(Enter key) ----> 현재의 진행을 계속

또는

Booting DB:/ctamd/unix(Enter key) ----> /ctamd/unix를 D/B로 사용함.

또는

Booting DB:/ctamd/unix.old(Enter key) ----> /ctamd/unix.old를 D/B로 사용함.

또는

Booting DB:/ctamd/unix.1st(Enter key) ----> /ctamd/unix.1st를 D/B로 사용함.

SYSTEM RESOURCE FOR TUXEDO

Bulletin no : 10188

Resource for Tuxedo System/T

: process는 resource를 얻기 위해 Tuxedo System Bulletin Board에 접근한다.

* **maxc** (maximum number of resource in use in the system)

- 대부분의 Unix system V 구조에서 default = 80
- Tuxedo configuration parameter (MAXACCESSSES) 가 maxc로 간주되어 사용되며 일반적으로 250는 되어야 한다.

* **maxa** (maximum number of active resource sets)

- default = 10

* **maxs** (maximum number of resources per resource set)

- default = 25
- equals maxc

* **maxp** (size of control map used to manage resource sets)

- default = 10
- equals maxc

* **maxu** (number of undo structures in the system)

- 동시에 Bulletin board 에 접근 시도하는 Unix system process의 수를 통제
- Bulletin Board 에 접근시도하는 수가 이 값보다 크면 Unix System Error (EIOBPC)발생
- default = 10
- equals maxc

* **maxus** (maximum number of undo entries per undo structure)

- default = 10
- equals maxc

Message Queue for Tuxedo system

: Tuxedo system은 Client/Server communication을 위해 message & message queue를 사용

: 각각 독립적인 server는 message를 받기 위해 message queue(MBQ set)과 reply queue(REPLYQ parameter)가 있다.

: 각각의 Client process는 자신의 queue를 가진다.

: message parameter에 의해 정의된 한계값이 초과되면 "Blocking"발생

: 모든 process가 send message를 시도하면서 block되었다면 application은 deadlock에 빠진 것임.

* **maxq** (number of unique message queue identifiers)

- default = 50 , 그러나 너무 적은 값이다
- process수 (queue수) + one for DBL + one for DBL + two for the DBLDC process
- 만약 distributed transaction processing을 한다면 각 server group을 위해 여분의 queue를 더한다.

```

* msgsz ( size of control msg to manage message segments )
  - msgsz의 두배 이상이어야 한다.
* msgmx ( maximum message size )
  - Byte단위
  - Tuxedo system application을 위한 일관적인 값은 18KB ~ 32KB범위
* msgql ( maximum message queue length )
  - same value (or higher) as msgmx
  - msgql의 75%보다 큰 messages는 file transfer로 보내어진다.
* msgsz ( size of message segment )
  - txctl을 위해 충분히 큰 연속적인 segment, Tuxedo system application = 32
* msgql ( number of outstanding messages )
  - default = 40 , 그러나 busy system을 위해서는 200보다 커야한다.
* msgsz ( number of message segments in the system )
  - msgmx의 multiple
  - 대부분의 platform에서는 <= 128KB(그러나 Sun/solaris2.4에서는 <= 32KB )

```

실행 테스트 (from 대한보증보험 BNT)

=====

문 제)

```

* Transaction 발생 system에서 Oracle( including with )을 강제한 Server B로
Tuxedo application (oracle DB에 접속하는 100개의 process를 발생)이 수행 도중 대부분의
process가 kill되는 현상
- compuser에 관련된 국한 message
- Error : ora-00050
(Could not obtain the operating system resources necessary for oracle msgsz)
- Error : ora-00080 (deadlock detected while waiting for resource )

```

해결방안)

```

* D/B parameter 수정
- sun/solaris2.4 에서 /etc/system
-->>
cat <sys:chaninfo_chansz = 314572800  (-300M, > oracle BGA size )
"      chansz = 1
"      chansl = 100
"      chancg = 1000
cat <sys:chaninfo_chansz = 1024  ( Request에서 maximum = 4096 )
"      chansl = 1024
"      chansl = 1024
"      chansz = 1024
"      chansz = 1024
"      chansz = 1024
cat <sys:msginfo_msgsz = 700
"      msgsz = 1400  ( >= msgsz * 2 )

```


SCRIPT FOR HOT BACKUP

Bulletin no : 10188

```
B -----*
B Hot Backup for oracle? dbase tablespaces          *
B DTPY_YML                                           *
B For each tablespace                               *
B      1. begin backup for data file                *
B      2. dd backup                                  *
B      3. end backup                                 *
B -----*
```

sqlplus >